

Командная строка Linux

Седьмое интернет-издание

Уильям Шоттс

Книга LinuxCommand.org

Copyright ©2026, William Shotts

Данная работа лицензирована на условиях Creative Commons Attribution-Noncommercial-No Derivative Works 3.0 United States License. Копию лицензии можно найти по ссылке выше или запросив по адресу: Creative Commons, PO Box 1866, Mountain View, CA 94042.

Печатная версия этой книги выпущена издательством No Starch Press. Ее можно приобрести в книжных магазинах. No Starch Press также предлагает электронные форматы для популярных ридеров. Связаться с ними можно по адресу:

<https://www.nostarch.com>.

Linux® является зарегистрированным товарным знаком Linus Torvalds. Все остальные товарные знаки принадлежат их владельцам.

Эта книга — часть проекта LinuxCommand.org, сайта по обучению и продвижению Linux, цель которого — помочь пользователям устаревших операционных систем перейти на современные решения. Вы можете связаться с проектом LinuxCommand.org по адресу <https://linuxcommand.org>.

История выпусков

Версия	Date	Описание
25.12	5 января 2026 г.	Седьмое интернет-издание
24.11	1 ноября, 2024	Шестое интернет-издание
19.01A	28 января 2019 г.	Пятое интернет-издание (исправленное оглавление)
19.01	17 января 2019 г.	Пятое интернет-издание.
17.10	19 октября, 2017	Четвертое интернет-издание.
16.07	28 июля 2016 г.	Третье интернет-издание.
13.07	6 июля 2013 г.	Второе интернет-издание.
09.12	14 декабря, 2009	Первое интернет-издание.

Содержание

Введение	6
Часть 1 — Изучение оболочки	9
1 — Что такое оболочка?	15
2 — Навигация	19
3 — Исследование системы	25
4 — Манипулирование файлами и каталогами	39
5 — Работа с командами	56
6 — Перенаправление	66
7 — Мир глазами оболочки	80
8 — Продвинутые приемы работы с клавиатурой	91
9 — Права доступа	100
10 — Процессы	121
Часть 2 — Конфигурация и окружение	140
11 — Окружение	141
12 — Мягкое введение в vi(m)	155
13 — Настройка приглашения	174
Часть 3 — Распространенные задачи и основные инструменты	184
14 — Управление пакетами	185
15 — Носители информации	196
16 — Сети	217
17 — Поиск файлов	233
18 — Архивация и резервное копирование	250
19 — Регулярные выражения	267
20 — Обработка текста	287
21 — Форматирование вывода	327
22 — Печать	347
23 — Компиляция программ	361
Часть 4 — Написание сценариев оболочки	372
24 — Написание вашего первого скрипта	373
25 — Начало проекта	380
26 — Сверху вниз	390
27 — Управление потоком: ветвление с помощью if	399
28 — Чтение ввода с клавиатуры	414
29 — Управление потоком: циклы	424
30 — Поиск и устранение неисправностей	432
31 — Управление потоком: ветвление с помощью case	446

32 — Позиционные параметры	451
33 — Управление потоком: циклы с for	465
34 — Строки и числа	471
35 — Массивы	491
36 — Экзотика	503
Колофон	521

Для Карен

Введение

Я хочу рассказать вам одну историю.

Нет, не историю о том, как в 1991 году Линус Торвальдс написал первую версию ядра Linux. Об этом можно прочесть во многих книгах по Linux. И я не собираюсь рассказывать, как за несколько лет до этого Ричард Столлман запустил проект GNU по созданию свободной Unix-подобной операционной системы. Это тоже важная история, но она также встречается в большинстве других книг по Linux.

Нет, я хочу рассказать вам историю о том, как вернуть контроль над своим компьютером.

Когда в конце 1970-х годов я, будучи студентом, начал работать с компьютерами, в мире происходила революция. Изобретение микропроцессора позволило обычным людям, вроде вас и меня, действительно владеть компьютером. Многим сегодня трудно представить, каким был мир, когда всеми компьютерами управляли только крупные корпорации и государство. Скажем так: сделать что-то существенное было крайне сложно.

Сегодня мир стал совсем иным. Компьютеры повсюду: от крошечных наручных часов до гигантских дата-центров и всего, что между ними. Помимо повсеместного распространения компьютеров, мы получили и вездесущую сеть, объединяющую их. Это открыло чудесную эру личных возможностей и творческой свободы, но последние пару десятилетий происходило и кое-что другое. Несколько гигантских корпораций установили контроль над большинством компьютеров в мире, решая, что вы можете, а чего не можете с ними делать. К счастью, люди по всему миру не стоят на месте. Они борются за контроль над своими компьютерами, создавая собственное программное обеспечение. Они создают Linux.

Многие говорят о «свободе» применительно к Linux, но я не думаю, что большинство понимает истинный смысл этого понятия. Свобода — это возможность решать, что делает ваш компьютер, а единственный способ обрести эту свободу — знать, что именно он делает. Свобода — это компьютер без секретов, где всё можно узнать, если приложить достаточно усилий.

Зачем использовать командную строку?

Замечали ли вы в кино, что «суперхакер» — тот самый парень, способный взломать сверхзащищенный военный компьютер менее чем за 30 секунд — когда садится за компьютер, он

никогда не касается мыши? Это потому, что кинематографисты понимают: мы, люди, инстинктивно знаем, что единственный способ по-настоящему что-то сделать на компьютере — это Большинство современных пользователей знакомы лишь с графическим интерфейсом (GUI); вендоры и эксперты приучили их к мысли, что интерфейс командной строки (CLI) — это пугающий пережиток прошлого. Это прискорбно, ведь хороший интерфейс командной строки — удивительно выразительный способ общения с компьютером, подобный письменной речи для людей. Говорят, что «графические интерфейсы упрощают легкие задачи, в то время как интерфейсы командной строки делают возможными сложные задачи», и это утверждение остается верным и сегодня.

Поскольку Linux построен на базе семейства операционных систем Unix, он унаследовал то же богатое наследие инструментов командной строки. Unix получил широкое признание в начале 1980-х (хотя был разработан на десятилетие раньше) еще до повсеместного внедрения графических интерфейсов, и в результате развил обширный интерфейс командной строки. На самом деле, одной из главных причин, по которым первые пользователи Linux предпочли его, скажем, Windows NT, был мощный интерфейс командной строки, позволяющий «выполнять сложные задачи».

О чем эта книга

Эта книга представляет собой широкий обзор «жизни» в командной строке Linux. В отличие от изданий, сосредоточенных на одной программе, например на оболочке `bash`, эта книга стремится показать, как работать с интерфейсом командной строки в более широком смысле. Как всё это устроено? Что она умеет? Как лучше всего её использовать?

Это не книга по системному администрированию Linux. Хотя любое серьезное обсуждение командной строки неизбежно затрагивает темы администрирования, здесь рассматриваются лишь некоторые из них. Тем не менее, книга подготовит читателя к дальнейшему обучению, заложив прочный фундамент в использовании командной строки — важнейшего инструмента для любой серьезной задачи системного администрирования.

Эта книга ориентирована именно на Linux. Многие другие издания пытаются расширить охват, включая другие платформы, такие как стандартный Unix или macOS. Тем самым они «размывают» содержание, ограничиваясь лишь общими темами.

Эта же книга посвящена только современным дистрибутивам Linux. Девяносто пять процентов контента будет полезно пользователям других Unix-подобных систем, но книга максимально нацелена на современного пользователя командной строки Linux.

Кому стоит читать эту книгу

Эта книга предназначена для новых пользователей Linux, перешедших с других платформ. Скорее всего, вы являетесь «продвинутым пользователем» какой-либо версии Microsoft Windows. Возможно, начальник поручил вам администрировать сервер Linux, или вы открываете для себя захватывающий мир одноплатных компьютеров (SBC), таких как Raspberry Pi. А может, вы просто пользователь настольного ПК, которому надоели проблемы с безопасностью и хочется попробовать Linux. Это нормально. Мы рады всем.

строки — непростая задача, требующая реальных усилий. Дело не в сложности, а в масштабах. В типичной системе Linux буквально тысячи программ, доступных в командной строке. Считайте это предупреждением: изучение командной строки — занятие не для праздного любопытства.

С другой стороны, изучение командной строки Linux приносит огромную пользу. Если вы считаете себя «продвинутым пользователем» сейчас, подождите. Вы еще не знаете, что такое настоящая мощь. И, в отличие от многих других компьютерных навыков, знания командной строки долговечны. Навыки, полученные сегодня, будут полезны и через 10 лет. Командная строка прошла проверку временем.

Предполагается также, что у вас нет опыта программирования, но не волнуйтесь — мы тоже направим вас на этот путь.

Что в этой книге

Материал представлен в тщательно подобранной последовательности, словно рядом с вами сидит наставник и ведет вас за собой. Многие авторы излагают материал «систематически», исчерпывающе разбирая каждую тему по порядку. С точки зрения писателя это логично, но может сильно запутать новичков.

Еще одна цель — познакомить вас с мышлением в стиле Unix, которое отличается от подхода Windows. По пути мы сделаем несколько «отступлений», чтобы помочь вам понять, почему определенные вещи работают именно так и как они к этому

пришли. Linux — это не просто программное обеспечение; это часть большой культуры Unix со своим языком и историей. Возможно, я также позволю себе пару эмоциональных высказываний.

Эта книга разделена на четыре части, каждая из которых охватывает определенный аспект работы с командной строкой:

- Часть 1 — Изучение оболочки открывает наше знакомство с базовым языком командной строки, включая такие темы, как структура команд, навигация по файловой системе, редактирование командной строки, а также поиск помощи и документации к командам.
- Часть 2 — Конфигурация и окружение посвящена редактированию конфигурационных файлов, управляющих работой компьютера через командную строку.
- Часть 3 — Распространенные задачи и основные инструменты рассматривает множество повседневных задач, выполняемых через командную строку. Unix-подобные операционные системы, такие как Linux, содержат множество «классических» программ командной строки для выполнения мощных операций с данными.
- Часть 4 — Написание сценариев оболочки знакомит с программированием на оболочке — признаем, рудиментарным, но простым в освоении методом автоматизации многих типичных задач. Изучив программирование на оболочке, вы познакомитесь с концепциями, применимыми и во многих других языках программирования.

Начинайте с начала книги и читайте до конца. Это не справочник, а скорее история с завязкой, развитием и развязкой.

Предварительные требования

Для работы с этой книгой вам понадобится установленная система Linux. Это можно сделать двумя способами:

1. Установите Linux на (не самый новый) компьютер. Неважно, какой дистрибутив вы выберете, хотя сегодня большинство начинает с Ubuntu, Fedora или OpenSUSE. Если сомневаетесь, попробуйте сначала Ubuntu. Установка современного дистрибутива Linux может быть как невероятно простой, так и невероятно сложной в зависимости от вашего оборудования. Я рекомендую настольный компьютер возрастом в пару лет, имеющий как минимум 2 ГБ оперативной памяти и 6 ГБ свободного места на жестком диске. По возможности

избегайте ноутбуков и беспроводных сетей, так как их настройка часто бывает сложнее.

2. Используйте «live CD» или USB-накопитель. Одна из прелестей многих дистрибутивов Linux заключается в том, что их можно запускать прямо с CD-ROM или USB-накопителя без установки. Просто зайдите в настройки BIOS, установите загрузку с CD-ROM или USB-устройства и перезагрузите компьютер. Этот метод — отличный способ проверить компьютер на совместимость с Linux перед установкой. Недостаток в том, что работа может быть очень медленной по сравнению с установленной системой. Ubuntu и Fedora (помимо прочих) имеют live-версии.

Независимо от способа установки Linux, для выполнения упражнений из этой книги вам периодически потребуются привилегии суперпользователя (т. е. администратора).

Когда система будет готова к работе, начинайте читать и повторяйте всё на своём компьютере. Большая часть материала в этой книге — практическая, так что садитесь и приступайте к вводу команд!

Почему я не называю это «GNU/Linux»

В некоторых кругах считается политически корректным называть операционную систему Linux «операционной системой GNU/Linux». Проблема с названием «Linux» в том, что не существует единственно верного варианта, поскольку система создавалась множеством разных людей в рамках масштабной распределенной разработки. С технической точки зрения Linux — это лишь название ядра операционной системы, не более того. Ядро, безусловно, очень важно, так как оно обеспечивает работу системы, но его недостаточно для создания полноценной операционной системы.

Встречайте Ричарда Столлмана — гениального философа, основателя движения свободного ПО, Free Software Foundation и проекта GNU, автора первой версии GNU C Compiler (gcc), создателя GNU General Public License (GPL) и многого другого. Он настаивает на названии «GNU/Linux», чтобы должным образом отразить вклад проекта GNU. Хотя проект GNU появился раньше ядра Linux и его заслуги заслуживают признания, включение его в название несправедливо по отношению ко всем остальным участникам. К тому же, я считаю, что «Linux/GNU»

технически точнее, так как ядро загружается первым, а всё остальное работает поверх него.

В обиходе под «Linux» понимают ядро и всё остальное свободное ПО с открытым исходным кодом в типичном дистрибутиве, то есть всю экосистему Linux, а не только компоненты GNU. На рынке операционных систем предпочтение отдается односложным названиям, таким как DOS, Windows, macOS, Solaris, Irix и AIX. Я решил использовать популярный формат. Если же вы предпочитаете термин «GNU/Linux», пожалуйста, мысленно заменяйте его при чтении этой книги. Я не возражаю.

Что нового в седьмом интернет-издании

Хотя сама оболочка получает крупные обновления лишь раз в 10 лет, оборудование и инструменты постоянно развиваются. Это издание The Linux Command Line снова было модернизировано с учётом изменений в среде командной строки. Внесено множество мелких правок и исправлений. Данное издание также соответствует печатной версии — The Linux Command Line: A Complete Introduction, Third Edition, выпущенной No Starch Press. Подробный список изменений см. в примечаниях к выпуску на LinuxCommand.org. Кроме того, в это издание включена коллекция примеров скриптов из книги. Их также можно скачать на LinuxCommand.org.

Благодарности

Я хочу поблагодарить следующих людей, благодаря которым эта книга стала возможной:

Дженни Уотсон, редактора по приобретению прав в Wiley Publishing, которая изначально предложила мне написать книгу по написанию сценариев для оболочки.

Джона К. Дворака, известного колумниста и эксперта. В одном из выпусков своего видеоподкаста «Cranky Geeks» господин Дворак описал процесс письма так: «Это ад. Пишите по 200 слов в день, и через год у вас будет роман». Этот совет побудил меня писать по странице в день, пока не получилась книга.

Дмитрий Попов написал статью в Free Software Magazine под названием «Создание шаблона книги в Writer», которая вдохновила меня использовать OpenOffice.org Writer (а позже — LibreOffice Writer) для подготовки текста. Как выяснилось, это сработало великолепно.

Марка Полески, который провел исключительную проверку и тестирование текста первого издания.

Джесси Бекер, Tomasz Chrzczonowicz, Michael Levin и Spence Miner также тестировали и Карен М. Шоттс потратила немало времени, редактируя рукопись и оттачивая мой так называемый английский.

Седьмое интернет-издание

Особая благодарность людям, чьи ценные отзывы были учтены в седьмом интернет-издании: Vitor Centeio, Elmar Deininger, Francesco Di Viesto, Jaroslaw Kolosowski, Kayck Matias и Wang Zheng.

Предыдущие издания

Особая благодарность людям, чьи ценные отзывы были учтены в предыдущих изданиях: Ala'a Ali, Adrian Arpidez, Mikey Barboza, Jesse Becker, Emanuele Bernardi, Andreas Bjørnstad, Hu Bo, Steve Bragg, John Burns, Heriberto Cantú, Enzo Cardinal, Paolo Casati, Tomasz Chrzczonowicz, Richard Cooke, Ethan Dowlatshah, Lixin Duan, Joshua Escamilla, Marc Evans, Ryan Flynn, Bruce Fowler, Devin Harper, Jørgen Heitmann, Janrodion, Jonathan Jones, Sunil Joshi, Ma Jun, Eric Kammerer, Robert Kennington, Seth King, Chris Knight, Jaroslaw Kolosowski, Klaus M. Körmendi, Jim Kovacs, Michael Levin, Bartłomiej Majka, Bashar Maree, Frank McTipps, Vladimir Milovanović, Sea Monkey, Tim Nelson, Mike O'Donnell, Oktay-Akin Okutan, Nick Owens, Justin Page, Michael Parrish, Esra Purba, Parviz Rasoulipour, Amir Razqandi, Patrick, Waldo Ribeiro, Pat Roche, Nick Rose, Satej Kumar Sahu, Avid Seeker, Mikhail Sizov, Ben Slater, Pickles Spill, Gabriel Stutzman, Pooya Taherkhani, Francesco Turco, Wolfram Volpi, Boyang Wang, Carl Westman, John Wiersba, Valter Wierzba и Christian Wuethrich.

И напоследок — огромная благодарность читателям LinuxCommand.org, приславшим мне столько добрых писем. Их поддержка дала мне понять, что я действительно на верном пути!

Мне важен ваш отзыв!

Эта книга — развивающийся проект, как и многие проекты с открытым исходным кодом. Если вы обнаружите техническую ошибку, напишите мне по адресу:

bshotts@users.sourceforge.net

Обязательно укажите точное издание книги, которую вы читаете.
Ваши правки и предложения могут войти в будущие выпуски.

Что почитать

- Вот статьи в Wikipedia об упомянутых выше известных личностях: https://en.wikipedia.org/wiki/Linus_Torvalds
https://en.wikipedia.org/wiki/Richard_Stallman
<https://www.gnu.org>
- Ричард Столлман подробно писал о вопросе именования «GNU/Linux»: <https://www.gnu.org/gnu/why-gnu-linux.html>
<https://www.gnu.org/gnu/gnu-linux-faq.html#tools>

Часть 1 — Изучение оболочки

1 — Что такое оболочка?

Говоря о командной строке, мы на самом деле имеем в виду оболочку. Оболочка — это программа, которая принимает команды с клавиатуры и передает их операционной системе для выполнения. Почти все дистрибутивы Linux поставляются с программой оболочки от проекта GNU под названием `bash`. Название «`bash`» — это акроним от «Bourne Again SHell», что указывает на то, что `bash` является улучшенной заменой `sh`, оригинальной оболочки Unix, написанной Стивом Борном.

Эмуляторы терминала

При использовании графического интерфейса пользователя (GUI) нам нужна еще одна программа — эмулятор терминала — для взаимодействия с оболочкой. Если заглянуть в меню рабочего стола, мы наверняка ее найдем. KDE использует `konsole`, а GNOME — `gnome-terminal`, хотя в меню она может называться просто «terminal». Для Linux доступно множество других эмуляторов терминала, но все они делают по сути одно и то же — предоставляют доступ к оболочке. Вероятно, вы выберете тот или иной эмулятор терминала, исходя из количества доступных в нем функций.

Первые нажатия клавиш

Итак, приступим. Запустите эмулятор терминала! Когда он откроется, вы увидите нечто подобное:

```
[me@linuxbox ~]$
```

Это называется приглашение оболочки; оно появляется всякий раз, когда оболочка готова принимать ввод. Внешний вид может немного отличаться в зависимости от дистрибутива, но обычно он включает `username@machinename`, за которым следует текущий рабочий каталог (подробнее об этом чуть позже) и знак доллара.

Примечание: Если последним символом приглашения является знак решетки («#»), а не знак доллара, сессия терминала имеет привилегии суперпользователя. Это значит, что мы либо вошли как пользователь `root`, либо выбрали эмулятор терминала, предоставляющий привилегии суперпользователя (администратора).

Если до сих пор всё в порядке, давайте попробуем что-нибудь напечатать. Введите в приглашение какой-нибудь бред, например так:

```
[me@linuxbox ~]$ kaekfjaeifj
```

Поскольку эта команда бессмысленна, оболочка сообщит об этом и даст нам еще один шанс.

```
bash: kaekfjaeifj: command not found
[me@linuxbox ~]$
```

История команд

Если мы нажмем стрелка вверх, то увидим, что предыдущая команда `kaekfjaeifj` снова появится после приглашения. Это называется история команд. Большинство дистрибутивов Linux по умолчанию помнят последние 1 000 команд. Нажмите стрелка вниз, и предыдущая команда исчезнет.

Перемещение курсора

Снова нажмите стрелку вверх, чтобы вызвать предыдущую команду. Если использовать клавиши влево и вправо, можно перемещать курсор в любое место командной строки. Это упрощает редактирование команд.

Несколько слов о мыши и фокусе

Хотя работа в оболочке завязана на клавиатуре, в эмуляторе терминала можно использовать и мышь. Механизм, встроенный в оконную систему (движок, обеспечивающий работу GUI), поддерживает быстрый способ копирования и вставки. Если выделить текст, зажав левую кнопку мыши и перетаскив её по тексту (или дважды щелкнув по слову), он попадет в буфер оконной системы. Нажатие на среднюю кнопку мыши вставит текст в месте нахождения курсора. Попробуйте это. Это работает в большинстве графических сред.

Примечание: Не пытайтесь использовать `ctrl-c` и `ctrl-v` для копирования и вставки внутри окна терминала. Они не работают. Эти управляющие коды имеют другое значение для оболочки и были назначены за много лет до выхода Microsoft Windows. Вместо них при работе в окне терминала используйте `shift-ctrl-c` и `shift-ctrl-v`.

Ваша графическая среда рабочего стола (скорее всего, KDE или GNOME), стремясь подражать Windows, вероятно,

использует политику «фокус по клику». Это значит, что для получения фокуса (активации) окна на него нужно нажать. Это противоречит традиционному поведению оконных систем — «фокус за мышью», когда окно получает фокус, когда вы просто наводите на него курсор. Окно не выйдет на передний план, пока вы не кликните по нему, но сможет принимать ввод. Установка политики «фокус за мышью» сделает технику копирования и вставки еще удобнее. Попробуйте, если получится (хотя некоторые среды больше это не поддерживают). Думаю, если вы дадите этому шанс, то предпочтете такой вариант. Найти эту настройку можно в программе конфигурации вашего оконного менеджера.

Простые команды

Теперь, когда мы научились вводить текст в нашем эмуляторе терминала, давайте попробуем несколько простых команд. Начнем с команды `date`, которая отображает текущее время и дату.

```
[me@linuxbox ~]$ date
Thu Mar  8 15:09:41 EST 2025
```

Еще одна полезная команда — `uptime`, которая показывает время работы системы и среднее количество запущенных процессов за разные периоды времени.

```
[me@linuxbox ~]$ uptime
15:12:22 up 3 days, 23:40,  7 users,  load average: 0.37, 0.37, 0.64
```

Чтобы увидеть текущий объем свободного места на дисках, введите `df`.

```
[me@linuxbox ~]$ df
Filesystem      1K-blocks    Used Available Use% Mounted on
/dev/sda2        15115452   5012392   9949716  34% /
/dev/sda5        59631908  26545424  30008432  47% /home
/dev/sda1        147764    17370    122765  13% /boot
tmpfs            256856      0      256856   0% /dev/shm
```

Аналогично, чтобы отобразить объем свободной памяти, введите команду `free`.

```
[me@linuxbox ~]$ free
              total        used         free       shared    buffers     cached
Mem:      513712      503976         9736           0         5312      122916
-/+ buffers/cache: 375748      137964
Swap:    1052248      104712      947536
```

Завершение сеанса терминала

Сеанс терминала можно завершить, закрыв окно эмулятора терминала, введя команду `exit` в приглашении оболочки или нажав `Ctrl-d`.

```
[me@linuxbox ~]$ exit
```

Консоль за кулисами

Даже если эмулятор терминала не запущен, за графическим рабочим столом продолжают работать несколько сеансов терминала. В большинстве дистрибутивов Linux к этим сеансам, называемым виртуальными терминалами или виртуальными консолями, можно получить доступ, нажав `ctrl-Alt-F1 – ctrl-Alt-F6`. При переходе к сеансу появляется приглашение для входа, в которое можно ввести имя пользователя и пароль. Чтобы переключиться с одной виртуальной консоли на другую, нажмите `Alt-F1 – Alt-F6`. В большинстве систем вернуться к графическому рабочему столу можно нажатием `Alt-F7`.

Итоги

Эта глава стала началом нашего пути в мир командной строки Linux: мы познакомились с оболочкой, кратко изучили командную строку и узнали, как начинать и завершать сеанс в эмуляторе терминала. Мы также научились вводить простые команды и выполнять базовое редактирование в командной строке. Не так уж это и страшно, верно?

В следующей главе мы изучим еще несколько команд и совершим прогулку по файловой системе Linux.

Что почитать

- Чтобы узнать больше о Стиве Борне, создателе Bourne Shell, прочитайте статью в Wikipedia:
http://en.wikipedia.org/wiki/Steve_Bourne
- Эта статья в Wikipedia посвящена Брайану Фоксу, первоначальному автору bash:
[https://en.wikipedia.org/wiki/Brian_Fox_\(computer_programmer\)](https://en.wikipedia.org/wiki/Brian_Fox_(computer_programmer))
- Вот статья о концепции оболочек в вычислительной технике:
[http://en.wikipedia.org/wiki/Shell_\(computing\)](http://en.wikipedia.org/wiki/Shell_(computing))

2 — Навигация

Первое, что нам нужно освоить (помимо навыков печати), — это навигация по файловой системе в Linux. В этой главе мы рассмотрим следующие команды:

- `pwd` — Выводит имя текущего рабочего каталога.
- `cd` — Меняет каталог.
- `ls` — Выводит содержимое каталога.

Понимание дерева файловой системы

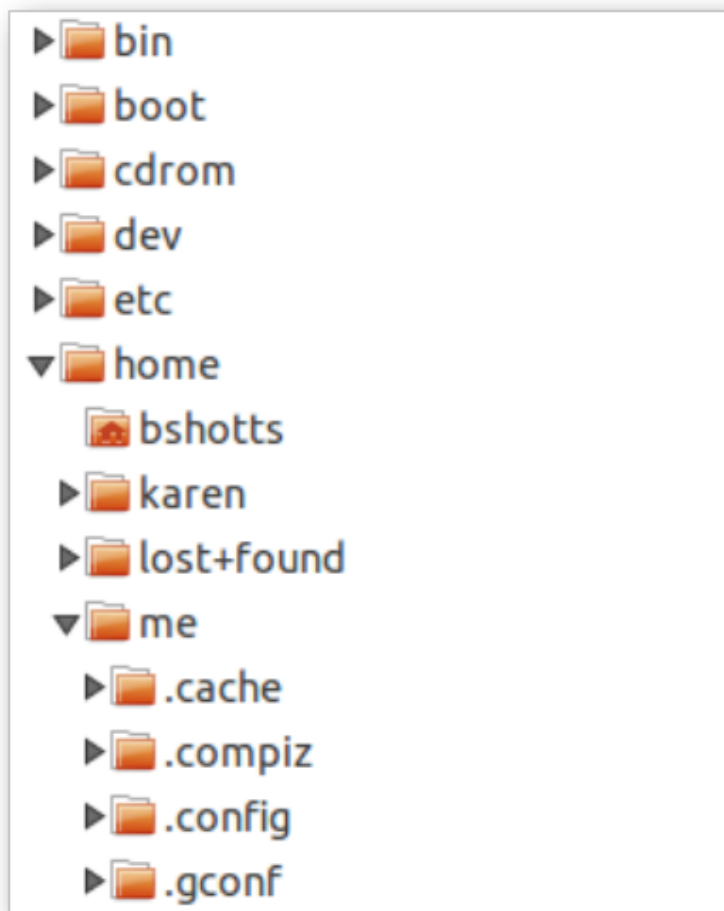
Как и в Windows, в Unix-подобных операционных системах, таких как Linux, файлы организованы в иерархическую структуру каталогов. Это означает, что они представляют собой древовидную систему каталогов (в других системах их иногда называют папками), которые могут содержать файлы и другие каталоги. Самый первый каталог в файловой системе называется корневым каталогом. Корневой каталог содержит файлы и подкаталоги, которые, в свою очередь, содержат еще больше файлов и подкаталогов.

Примечание: в отличие от Windows, где для каждого устройства хранения существует отдельное дерево файловой системы, Unix-подобные системы (например, Linux) всегда имеют единое дерево файловой системы, независимо от количества подключенных к компьютеру дисков или устройств хранения. Устройства хранения подключаются (или, точнее, монтируются) в различных точках дерева по усмотрению системного администратора — лица (или лиц), ответственного за обслуживание системы.

Текущий рабочий каталог

Большинство из нас знакомо с графическими файловыми менеджерами, представляющими дерево файловой системы, как на рисунке 1. Заметьте, что дерево обычно отображается перевернутым: корневой каталог находится сверху, а ветви уходят вниз.

Однако в командной строке нет картинок, поэтому для навигации по дереву файловой системы нам нужно



*Рисунок 1: дерево файловой системы в
графическом файловом менеджере*

Попробуйте взглянуть на это иначе. Представьте, что файловая система — это лабиринт в форме перевернутого дерева, и мы находимся в его центре. В любой момент времени мы находимся внутри одного каталога и видим файлы в нем, путь к каталогу выше нас (называемому родительским каталогом) и любые подкаталоги ниже нас. Каталог, в котором мы находимся, называется текущим рабочим каталогом. Чтобы отобразить текущий рабочий каталог, мы используем команду `pwd` (print working directory).

```
[me@linuxbox ~]$ pwd  
/home/me
```

Когда мы впервые входим в систему (или запускаем сессию эмулятора терминала), наш текущий рабочий каталог устанавливается как наш домашний каталог. Каждой учетной записи пользователя выделяется свой собственный домашний

каталог, и это единственное место, где обычному пользователю разрешено записывать файлы.

Просмотр содержимого каталога

Чтобы вывести список файлов и каталогов в текущем рабочем каталоге, используйте команду `ls`.

```
[me@linuxbox ~]$ ls
Desktop Documents Music Pictures Public Templates Videos
```

На самом деле, с помощью команды `ls` можно просматривать содержимое любого каталога, а не только текущего рабочего каталога, и она умеет много других полезных вещей. Подробнее о `ls` мы поговорим в следующей главе.

Смена текущего рабочего каталога

Чтобы сменить рабочий каталог (наше местоположение в древовидном лабиринте), мы используем команду `cd`. Для этого введите `cd`, затем пробел и путь к нужному рабочему каталогу. Путь — это маршрут по ветвям дерева к желаемому каталогу. Пути можно указывать двумя способами: как абсолютные пути или как относительные пути. Сначала рассмотрим абсолютные пути.

Абсолютные пути

Абсолютный путь начинается с корневого каталога и проходит по ветвям дерева до нужного каталога или файла. Например, в нашей системе есть каталог, в котором установлена большинство программ. Путь к этому каталогу — `/usr/bin`. Это значит, что из корневого каталога (обозначаемого начальным слешем в пути) следует каталог под названием `usr`, который содержит каталог под названием `bin`.

```
[me@linuxbox ~]$ cd /usr/bin
[me@linuxbox bin]$ pwd
/usr/bin
[me@linuxbox bin]$ ls
...Listing of many, many files ...
```

Теперь мы видим, что изменили текущий рабочий каталог на `/usr/bin`, и он полон файлов. Заметили, как изменилось приглашение оболочки? Для удобства оно обычно настраивается так, чтобы автоматически отображать имя рабочего каталога.

Относительные пути

Если абсолютный путь начинается с корневого каталога и ведет к цели, то относительный путь начинается с текущего рабочего каталога. Для этого используются специальные обозначения относительного положения в дереве файловой системы: «.» (точка) и «..» (две точки).

Обозначение. относится к текущему рабочему каталогу, а.. — к родительскому каталогу текущего рабочего каталога. Например, давайте снова сменим текущий рабочий каталог на /usr/bin.

```
[me@linuxbox ~]$ cd /usr/bin
[me@linuxbox bin]$ pwd
/usr/bin
```

Допустим, мы хотим сменить текущий рабочий каталог на родительский для /usr/bin /usr, то есть на. Это можно сделать двумя способами: используя абсолютный путь

```
[me@linuxbox bin]$ cd /usr
[me@linuxbox usr]$ pwd
/usr
```

или используя относительный путь:

```
[me@linuxbox bin]$ cd ..
[me@linuxbox usr]$ pwd
/usr
```

Два разных метода дают одинаковый результат. Какой выбрать? Тот, где нужно меньше печатать!

Аналогично, мы можем сменить текущий рабочий каталог с /usr на /usr/bin двумя способами: либо через абсолютный путь

```
[me@linuxbox usr]$ cd /usr/bin
[me@linuxbox bin]$ pwd
/usr/bin
```

либо через относительный путь:

```
[me@linuxbox usr]$ cd ./bin
[me@linuxbox bin]$ pwd
/usr/bin
```

Здесь важно отметить следующее. Почти во всех случаях ./ можно опустить. Это подразумевается. Ввод

```
[me@linuxbox usr]$ cd bin
```

делает то же самое. В целом, если не указывать путь к объекту, будет использоваться текущий рабочий каталог.

Полезные сокращения

В таблице 2-1 показаны удобные способы быстрого изменения текущего рабочего каталога.

Таблица 2-1: *cd* Сокращения

Сочетание клавиш	Результат
<code>cd</code>	Меняет текущий рабочий каталог на ваш домашний каталог.
<code>cd -</code>	Меняет текущий рабочий каталог на предыдущий рабочий каталог.
<code>cd ~user_name</code>	Меняет текущий рабочий каталог на домашний каталог пользователя <code>_name</code> . Например, <code>cd ~bob</code> изменит каталог на домашний каталог пользователя <code>bob</code> .

Важные факты об именах файлов

В системах Linux имена файлов создаются схожим с Windows образом, но есть важные различия.

1. Имена файлов, начинающиеся с точки, являются скрытыми. Это значит, что `ls` не выведет их, если вы не укажете `ls -a`. При создании вашей учетной записи в ваш домашний каталог было помещено несколько скрытых файлов для настройки параметров. В главе 11 мы подробнее рассмотрим некоторые из них, чтобы понять, как можно настроить окружение. Кроме того, некоторые приложения хранят свои конфигурационные файлы и настройки в вашем домашнем каталоге в виде скрытых файлов.
2. Имена файлов и команды в Linux, как и в Unix, чувствительны к регистру. Файлы «File1» и «file1» — это разные файлы.
3. В Linux нет понятия «расширение файла», как в некоторых других операционных системах. Вы можете называть файлы как угодно. Содержимое или назначение файла определяется другими способами. Хотя Unix-подобные системы не используют расширения для определения содержимого или назначения файлов, многие прикладные программы всё же их используют.
4. Хотя Linux поддерживает длинные имена файлов, которые могут содержать пробелы и знаки препинания, старайтесь ограничивать знаки препинания в именах

создаваемых вами файлов точками, дефисами и подчеркиваниями. Самое главное — не используйте пробелы в именах файлов. Если вы хотите разделить слова в имени файла, используйте символ подчеркивания. Вы еще скажете себе за это спасибо.

Итоги

В этой главе объяснялось, как оболочка обрабатывает структуру каталогов системы. Мы изучили абсолютные и относительные пути, а также основные команды для навигации по этой структуре. В следующей главе мы используем эти знания для ознакомления с современной системой Linux.

3 — Исследование системы

Теперь, когда мы умеем перемещаться по файловой системе, пора отправиться в ознакомительный тур по нашей системе Linux. Однако прежде чем начать, мы изучим еще несколько команд, которые пригодятся нам в пути.

- `ls` — Вывод содержимого каталога.
- `file` — Определение типа файла.
- `less` — Просмотр содержимого файла.

Больше интересного с `ls`

Команда `ls`, вероятно, самая используемая в Linux, и на то есть веские причины. С её помощью можно просматривать содержимое каталогов и определять различные важные атрибуты файлов и каталогов. Как мы уже видели, можно просто ввести `ls`, чтобы получить список файлов и подкаталогов, содержащихся в текущем рабочем каталоге.

```
[me@linuxbox ~]$ ls
Desktop Documents Music Pictures Public Templates Videos
```

Помимо текущего рабочего каталога, мы можем указать конкретный каталог для вывода списка, вот так:

```
me@linuxbox ~]$ ls /usr
bin games include lib local sbin share src
```

Мы можем указать даже несколько каталогов. В следующем примере мы выводим список как домашний каталог пользователя (обозначаемый символом «~»), так и каталог `/usr`:

```
[me@linuxbox ~]$ ls ~ /usr
/home/me:
Desktop Documents Music Pictures Public Templates Videos
/usr:
bin games include lib local sbin share src
```

Мы также можем изменить формат вывода, чтобы получить больше подробностей.

```
[me@linuxbox ~]$ ls -l
total 56
drwxrwxr-x 2 me me 4096 2025-10-26 17:20 Desktop
drwxrwxr-x 2 me me 4096 2025-10-26 17:20 Documents
drwxrwxr-x 2 me me 4096 2025-10-26 17:20 Music
drwxrwxr-x 2 me me 4096 2025-10-26 17:20 Pictures
drwxrwxr-x 2 me me 4096 2025-10-26 17:20 Public
drwxrwxr-x 2 me me 4096 2025-10-26 17:20 Templates
drwxrwxr-x 2 me me 4096 2025-10-26 17:20 Videos
```

Добавив «-l» к команде, мы изменили вывод на длинный формат.

Опции и аргументы

Это подводит нас к важному моменту в работе большинства команд. За командами часто следуют одна или несколько опций, изменяющих их поведение, а затем — один или несколько аргументов, то есть объектов, над которыми выполняется команда. Таким образом, большинство команд выглядят так:

```
command -options arguments
```

Большинство команд используют опции, состоящие из одного символа, перед которым стоит дефис, например, -l. Однако многие команды, включая проекты GNU, также поддерживают длинные опции, состоящие из слова, перед которым стоят два дефиса. Кроме того, многие команды позволяют объединять несколько коротких опций в одну строку. В следующем примере команде ls передаются две опции: l для вывода в длинном формате и t для сортировки результата по времени изменения файла.

```
[me@linuxbox ~]$ ls -lt
```

Мы добавим длинный параметр «--reverse», чтобы изменить порядок сортировки на обратный.

```
[me@linuxbox ~]$ ls -lt --reverse
```

Примечание: как и имена файлов в Linux, параметры команд чувствительны к регистру.

ls команда имеет множество возможных параметров. В таблице 3-1 описаны наиболее распространенные.

Таблица 3-1: Общие параметры ls

Параметр	Длинный параметр	Описание
-a	--all	Выводит все файлы, даже те, чьи имена начинаются с точки, которые обычно не отображаются (то есть скрытые).
-A	--almost-all	Как и опция -a, но без вывода. (текущий рабочий каталог) и.. (родительский каталог).

Параметр	Длинный параметр	Описание
-d	--directory	Обычно, если указан каталог, <code>ls</code> выведет содержимое каталога, а не сам каталог. Используйте этот параметр вместе с опцией <code>-l</code> , чтобы увидеть сведения о каталоге, а не о его содержимом.
-F	--classify	Этот параметр добавит символ-индикатор в конце каждого имени, например, косую черту (/), если имя — это каталог.
-h	--human-readable	В длинном формате выводить размер файлов в читаемом виде, а не в байтах.
-l		Выводить результаты в длинном формате.
-r	--reverse	Выводить результаты в обратном порядке. Обычно, <code>ls</code> выводит результаты в порядке возрастания алфавита.
-S		Сортировка результатов по размеру файла.
-t		Сортировка по времени изменения.

Подробный разбор длинного формата

Как мы видели ранее, опция `-l` заставляет `ls` выводить результаты в расширенном формате. Этот формат содержит много полезной информации. Вот Examples каталог из старой системы Ubuntu:

```
-rw-r--r-- 1 root root 3576296 2017-04-03 11:05 Experience ubuntu.ogg
-rw-r--r-- 1 root root 1186219 2017-04-03 11:05 kubuntu-leaflet.png
-rw-r--r-- 1 root root 47584 2017-04-03 11:05 logo-Edubuntu.png
-rw-r--r-- 1 root root 44355 2017-04-03 11:05 logo-Kubuntu.png
-rw-r--r-- 1 root root 34391 2017-04-03 11:05 logo-Ubuntu.png
-rw-r--r-- 1 root root 32059 2017-04-03 11:05 oo-cd-cover.odf
-rw-r--r-- 1 root root 159744 2017-04-03 11:05 oo-derivatives.doc
-rw-r--r-- 1 root root 27837 2017-04-03 11:05 oo-maxwell.odt
-rw-r--r-- 1 root root 98816 2017-04-03 11:05 oo-trig.xls
-rw-r--r-- 1 root root 453764 2017-04-03 11:05 oo-welcome.odt
-rw-r--r-- 1 root root 358374 2017-04-03 11:05 ubuntu Sax.ogg
```

В таблице 3-2 приведены различные поля одного из файлов и их значения.

Таблица 3-2: Поля длинного вывода для `ls`

Поле	Значение
<code>-rw-r--r--</code>	Права доступа к файлу. Первый символ указывает на тип файла. Среди различных типов дефис означает обычный файл, а «d» указывает на каталог. Следующие три символа — это права доступа для владельца файла, следующие три — для участников группы файла, а последние три — для всех остальных. Глава 9 подробнее обсуждает полный смысл этого понятия.
<code>1</code>	Количество жестких ссылок файла. См. разделы «Symbolic Links» и «Hard Links» далее в этой главе.
<code>root</code>	Имя пользователя — владельца файла.
<code>root</code>	Имя группы, владеющей файлом.
<code>32059</code>	Размер файла в байтах.
<code>2017-04-03 11:05</code>	Дата и время последнего изменения файла.
<code>oo-cd-cover.odf</code>	Имя файла.

Определение типа файла с помощью `file`

Как мы уже понимаем систему, полезно знать, какие данные содержат файлы. Чтобы сделать это, мы воспользуемся командой `file`, чтобы определить тип файла. Как мы обсуждали ранее, имена файлов в Linux не обязаны отражать их содержимое. Хотя от файла с именем `picture.jpg` обычно ожидают наличия изображения в формате JPEG, в Linux это не обязательно. Мы вызовем команду `file` следующим образом:

```
file filename
```

При вызове команда `file` выведет краткое описание содержимого файла. Например:

```
[me@linuxbox ~]$ file picture.jpg
picture.jpg: JPEG image data, JFIF standard 1.01
```

Существует множество типов файлов. На самом деле, одна из базовых идей Unix-подобных систем, таких как Linux,

закключается в том, что «всё есть файл». В ходе обучения мы увидим, насколько это утверждение верно.

Хотя многие файлы в нашей системе знакомы (например, MP3 и JPEG), существует множество менее очевидных типов и несколько весьма странных.

Просмотр содержимого файла с помощью `less`

Команда `less` — это программа для просмотра текстовых файлов. Во всей системе Linux многие файлы содержат текст, доступный для чтения человеком. Программа `less` предоставляет удобный способ их изучения.

Что такое «текст»?

Существует множество способов представления информации на компьютере. Все методы подразумевают установление связи между информацией и числами, которые будут её представлять. В конце концов, компьютеры понимают только числа, и все данные преобразуются в числовое представление.

Некоторые системы представления сложны (как сжатые видеофайлы), другие — просты. Одна из самых ранних и простых называется ASCII-текстом. ASCII (произносится как «аски») — это аббревиатура от American Standard Code for Information Interchange. Это простая схема кодирования, которая изначально использовалась в телетайпах для сопоставления символов клавиатуры с числами.

Текст — это простое взаимно однозначное соответствие символов числам. Он компактен: пятьдесят символов текста преобразуются в 50 байт данных. Важно понимать, что текст содержит лишь простое сопоставление символов и чисел. Это не то же самое, что документ текстового процессора, созданный в Microsoft Word или LibreOffice Writer. Такие файлы, в отличие от простого ASCII-текста, содержат множество нетекстовых элементов для описания структуры и форматирования. Файлы простого ASCII-текста содержат только сами символы и несколько элементарных управляющих кодов, таких как табуляция, возврат каретки и перевод строки.

В системе Linux множество файлов хранится в текстовом формате, и существует множество инструментов Linux для работы с ними. Даже Windows признает важность этого

формата. Известная программа NOTEPAD.EXE является редактором для простых ASCII-текстовых файлов.

Зачем нам изучать текстовые файлы? Потому что многие файлы с системными настройками (их называют конфигурационными файлами) хранятся именно в этом формате, и умение читать их дает понимание работы системы. Кроме того, некоторые сами программы, используемые системой (их называют скриптами), также хранятся в текстовом виде. В следующих главах мы научимся редактировать текстовые файлы для изменения настроек системы и написания собственных скриптов, но пока мы просто изучим их содержимое.

Команда `less` используется следующим образом:

```
less filename
```

После запуска `less` program позволяет прокручивать текстовый файл вперед и назад. Например, чтобы изучить файл, определяющий все учетные записи пользователей системы, введите следующую команду:

```
[me@linuxbox ~]$ less /etc/passwd
```

Когда `less` program запустится, мы сможем просмотреть содержимое файла. Если файл длиннее одной страницы, можно прокручивать его вверх и вниз. Чтобы выйти из `less`, нажмите `q`.

В таблице 3-3 перечислены наиболее часто используемые команды клавиатуры для `less`.

Таблица 3-3: `less` команды

Команда	Действие
Page Up или b	Прокрутка на одну страницу назад
Page Down или пробел	Прокрутка на одну страницу вперед
стрелка вверх	Прокрутка на одну строку вверх
стрелка вниз	Прокрутка на одну строку вниз
G	Перейти в конец текстового файла
1G или g	Перейти в начало текстового файла
/ символов	Поиск вперед до следующего вхождения символов

Команда	Действие
n	Поиск следующего вхождения предыдущего запроса
h	Показать справку
q	Выход из less

Меньше — значит больше

Программа `less` была разработана как улучшенная замена более ранней программы Unix под названием `more`. Название `less` — это игра слов на основе фразы «меньше — значит больше» — девиза архитекторов и дизайнеров-модернистов.

`less` относится к классу программ, называемых пейджерами, — программ, позволяющих удобно просматривать длинные текстовые документы по страницам. Если программа `more` позволяла листать только вперед, то `less` поддерживает прокрутку как вперед, так и назад, а также обладает множеством других функций.

Краткий обзор

Структура файловой системы в Linux во многом схожа с другими Unix-подобными системами. Она определена в опубликованном стандарте под названием Linux Filesystem Hierarchy Standard. Не все дистрибутивы Linux строго следуют этому стандарту, но большинство максимально к нему приближены.

Далее мы изучим файловую систему, чтобы понять, как устроена наша система Linux. Это даст возможность попрактиковаться в навигации. Мы обнаружим, что многие интересные файлы представляют собой обычный читаемый текст. В ходе нашего тура попробуйте следующее:

1. `cd` в заданный каталог
2. Выведите содержимое каталога с помощью `ls -l`
3. Если файл кажется интересным, изучите его содержимое с помощью `file`
4. Если файл похож на текстовый, попробуйте просмотреть его через `less`
5. Если вы случайно попытаетесь открыть нетекстовый файл и окно терминала «поплывет», восстановите его, введя команду `reset`.

Не забывайте о трюке с копированием и вставкой! Если вы используете мышь, можно дважды щелкнуть по имени файла, чтобы скопировать его, и нажать на колесико, чтобы вставить его в командную строку.

Пока мы изучаем систему, не бойтесь заглядывать в разные места. Обычным пользователям в основном запрещено что-либо ломать. Это работа системного администратора! Если команда выдает ошибку, просто переходите к следующей. Потратьте время на осмотр. Эта система создана для нашего изучения. Помните: в Linux нет секретов!

В таблице 3-4 приведен лишь небольшой список каталогов, которые можно исследовать. В зависимости от дистрибутива Linux возможны небольшие различия. Не бойтесь изучать систему и пробовать новое!

Таблица 3-4: Каталоги в системах Linux

Каталог	Комментарии
/	Корневой каталог. Точка отсчета системы.
/bin	Содержит бинарные файлы (программы), необходимые для загрузки и работы системы. Примечание: в современных Linux дистрибутивы признали /bin устаревшим в пользу /usr/bin.
/boot	Содержит ядро Linux, образ начального RAM-диска (для драйверов, необходимых при загрузке), и загрузчик. Интересные файлы: ● /boot/grub/grub.cfg или menu.lst, который является используемым для настройки загрузчика. ● /boot/vmlinuz (или нечто подобное), Linux ядро
/dev	Это специальный каталог, содержащий узлы устройств. «Всё есть файл» относится и к устройствам. Здесь ядро хранит список всех поддерживаемых им устройств.
Каталог	Комментарии

Каталог	Комментарии
/etc	Каталог /etc содержит все общесистемные конфигурационные файлы. Все в этом каталоге должно быть читаемым текстом. Хотя в /etc интересно всё, вот некоторые излюбленные темы: ● /etc/crontab определяет, когда будут запускаться автоматические задачи в системах с программой cron ● /etc/fstab — таблица устройств хранения и их соответствующих точек монтирования. ● /etc/passwd — список учетных записей пользователей.
/home	В стандартных конфигурациях каждому пользователю выделяется каталог в /home. Обычные пользователи могут записывать файлы только в свои домашние каталоги. Это ограничение защищает систему от ошибочных действий пользователей.
/lib	Содержит файлы общих библиотек, используемых основной системой программ. Они похожи на динамические библиотеки (DLL) в Windows. Этот каталог устарел в современных дистрибутивах в пользу /usr/lib.
/lost+found	Каждый отформатированный раздел или устройство с традиционной Linux- файловой системой, такой как ext4, будет иметь этот каталог. Он используется в случае частичного восстановления после повреждения файловой системы событие. Если только с нашей системой не случилось чего-то действительно плохого, этот каталог останется пустым.
/media	В современных системах Linux /media каталог будет содержать точки монтирования съемных носителей, таких как USB- накопители, CD-ROM и прочее, которые монтируются автоматически при подключении.
/mnt	В старых системах Linux каталог /mnt содержит точки монтирования для устройств, смонтированных вручную.

Каталог	Комментарии
/opt	Каталог /opt используется для установки «дополнительного» ПО. В основном он служит для хранения коммерческих программных продуктов, которые могут быть установлены в системе.

Каталог	Комментарии
/proc	Каталог /proc особенный. Это не настоящая файловая система в понимании файлов, хранящихся на жестком диске. Скорее, это виртуальная файловая система, поддерживаемая ядром Linux. «файлы», содержащиеся в ней, — это окна в ядро. Эти файлы доступны для чтения и позволят нам понять, как ядро видит компьютер. Просмотр этого каталога может раскрыть множество дета об аппаратном обеспечении компьютера.
/root	Это домашний каталог учетной записи root.
/run	Это современная замена традиционному /tmp каталогу. В отличие от /tmp, каталог /run монтируется с использованием файловой системы типа tmpfs, которая хранит содержи в оперативной памяти, а не на физическом диске.
/sbin	Этот каталог содержит «системные» бинарные файлы. Это программы, выполняющие жизненно важные системные задачи, которые о зарезервировано для суперпользователя. Примечание: современные L дистрибутивы признали /sbin устаревшим в пользу /usr/sbin.
/sys	Каталог /sys содержит информацию об устройствах, обнаруженных ядром. Это похоже на содержимое каталога /dev, но оно более подробное, включая такие данные, как реальные аппаратные адреса.
/tmp	Каталог /tmp предназначен для хранения временных, краткосрочных файлов, создаваемых различными программами. Нек дистрибутивы очищают этот каталог при каждой перезагрузке системы.

Каталог	Комментарии
/usr	Дерево каталогов /usr, скорее всего, является самым большим в системе Linux. Оно содержит все программы и вспомогательные файлы обычными пользователями.
/usr/bin	/usr/bin содержит исполняемые программы, установленные дистрибутивом Linux. Часто в этом каталоге находится тысячи программ.
/usr/lib	Общие библиотеки для программ в /usr/bin.

Каталог	Комментарии
/usr/local	Дерево /usr/local содержит программы, которые не входят в состав дистрибутива, но предназначены для системного- широкого использования. Здесь устанавливаются программы, скомпилированные из исходного кода обычно устанавливаются в /usr/local/bin. В новой установленной системе Linux это дерево существует, но оно будет пустым пока системный администратор не поместит туда что-нибудь.
/usr/sbin	Содержит больше программ для системного администрирования.
/usr/share	/usr/share содержит все общие данные, используемые программами в /usr/bin. Сюда входят такие вещи, как файлы конфигурации по умолчанию, иконки, фоновые рисунки, звуки файлы, и так далее.
/usr/share/doc	Большинство установленных в системе пакетов включают в себя какую-либо документацию. В /usr/share/doc, мы найдем файлы документации, организованные по пакетам.

Каталог	Комментарии
/var	За исключением /tmp и /home, каталоги, которые мы рассмотрели, остаются относительно статичными; то есть их содержимое не меняется. Дерево каталогов /var — это место, где хранятся данные, которые могут меняться. Различные базы данных, файлы spool, почта пользователей и прочее находятся здесь.
/var/log	/var/log содержит лог-файлы, записи различной системной активности. Они важны, и их следует периодически проверять. Самые полезные из них — это /var/log/messages и/или /var/log/syslog хотя они доступны не во всех системах. Примечание: из соображений безопасности некоторые системы позволяют это только суперпользователю просмотр лог-файлов.
~/.config и ~/.local	Эти два каталога находятся в домашнем каталоге каждого пользователя рабочего стола. Они используются для хранения пользовательских настроек и данных о состоянии программ приложения

Символьные ссылки

При осмотре мы, скорее всего, увидим список каталогов (например, в /usr/lib) с такой записью:

```
lrwxrwxrwx 1 root root 11 2025-08-11 07:34 libc.so.6 -> libc-2.6.so
```

Заметили, что первая буква в списке — «l», а у записи как будто два имени файла? Это особый тип файла, называемый символической ссылкой (также известной как мягкая ссылка или sym-link). В большинстве Unix-подобных систем файл может иметь несколько имен. Хотя польза этого не сразу очевидна, это действительно полезная функция.

Представьте сценарий: программе требуется общий ресурс, содержащийся в файле «foo», но версия «foo» часто меняется. Было бы удобно включать номер версии в имя файла, чтобы администратор или другие заинтересованные лица могли видеть, какая версия «foo» установлена. Но здесь возникает проблема. Если мы изменим имя общего ресурса, нам придется

искать каждую программу, которая может его использовать, и каждый раз при установке новой версии менять имя ресурса в настройках программ. Сомнительное удовольствие.

Именно здесь символьные ссылки выручают. Допустим, мы устанавливаем версию 2.6 «foo» с именем файла `foo-2.6`, а затем создаем символьную ссылку `foo`, указывающую на `foo-2.6`. Это значит, что когда программа открывает файл `foo`, она на самом деле открывает файл `foo-2.6`. Теперь все довольны: программы, зависящие от «foo», находят его, а мы по-прежнему видим, какая версия установлена на самом деле. Когда придет время обновиться до `foo-2.7`, мы просто добавим файл в систему, удалим символьную ссылку `foo` и создадим новую, указывающую на новую версию. Это не только решает проблему обновления, но и позволяет хранить обе версии на машине. Представьте, что в `foo-2.7` обнаружился баг (черт бы побрал этих разработчиков!) и нам нужно вернуться к старой версии. Снова просто удаляем символьную ссылку, указывающую на новую версию, и создаем новую, указывающую на старую.

Список содержимого каталога в начале этого раздела (из каталога `/usr/lib` системы Fedora) показывает символическую ссылку `libc.so.6`, которая указывает на файл общей библиотеки `libc-2.6.so`. Это означает, что программы, ищущие `libc.so.6`, на самом деле получают файл `libc-2.6.so`. О том, как создавать символические ссылки, мы узнаем в следующей главе.

Жесткие ссылки

Раз уж мы заговорили о ссылках, стоит упомянуть о втором типе — жестких ссылках. Жесткие ссылки также позволяют файлам иметь несколько имен, но работают иначе. Подробнее о различиях между символическими и жесткими ссылками мы поговорим в следующей главе.

Итоги

Завершив обзор, мы многое узнали о нашей системе. Мы изучили различные файлы и каталоги, а также их содержимое. Главный вывод — насколько открыта эта система. Во многих важных файлах Linux используется обычный читаемый текст. В отличие от проприетарных систем, Linux предоставляет всё для изучения и анализа.

Что почитать

- Полную версию стандарта иерархии файловой системы Linux можно найти здесь: <https://refspecs.linuxfoundation.org/fhs.shtml>
- Статья о структуре каталогов Unix и Unix-подобных систем: [http:// en.wikipedia.org/wiki/Unix_directory_structure](http://en.wikipedia.org/wiki/Unix_directory_structure)
- Подробное описание формата ASCII-текста: <http://en.wikipedia.org/wiki/ASCII>

4 — Манипулирование файлами и каталогами

Теперь мы готовы к настоящей работе! В этой главе будут рассмотрены следующие команды:

- `cp` — Копирование файлов и каталогов.
- `mv` — Перемещение/переименование файлов и каталогов.
- `mkdir` — Создание каталогов.
- `rm` — Удаление файлов и каталогов.
- `ln` — создание жестких и символических ссылок.

Эти пять команд — одни из самых часто используемых в Linux. Они служат для манипуляций как с файлами, так и с каталогами.

Скажем прямо: некоторые задачи, выполняемые этими командами, проще решить с помощью графического файлового менеджера. С ним можно перетаскивать файлы из одного каталога в другой, вырезать и вставлять, удалять и так далее. Так зачем же использовать эти старые программы командной строки?

Ответ — в мощности и гибкости. Если простые операции с файлами легко выполнить в графическом менеджере, то сложные задачи проще решать через командную строку. Например, как скопировать все HTML-файлы из одного каталога в другой, но только те, которых нет в целевом каталоге или которые новее существующих там версий? С файловым менеджером это трудно, а с командной строкой — проще простого.

```
cp -u *.html destination
```

Подстановочные знаки

Прежде чем приступить к командам, стоит обсудить функцию оболочки, которая делает их такими мощными. Поскольку оболочка часто использует имена файлов, она предоставляет специальные символы для быстрого указания групп файлов. Эти символы называются подстановочными знаками. Использование подстановочных знаков (так же известное как *globbing*) позволяет выбирать имена файлов по шаблонам символов. В таблице 4-1 перечислены подстановочные знаки и то, что они выбирают.

Таблица 4-1: Подстановочные знаки

Подстановочный знак	Значение
*	Соответствует любым символам, включая отсутствие символов
?	Соответствует любому одиночному символу
[characters]	Соответствует любому символу из указанного набора
[!characters] or [^characters]	Соответствует любому символу, не входящему в набор <i>символы</i>
[[:class:]]	Соответствует любому символу из указанного класса

В таблице 4-2 приведены наиболее часто используемые классы символов.

Таблица 4-2: Часто используемые классы символов

Класс символов	Значение
[:alnum:]	Любой буквенно-цифровой символ
[:alpha:]	Любая буква
[:digit:]	Любая цифра
[:lower:]	Любая строчная буква
[:upper:]	Соответствует любой заглавной букве

Использование подстановочных знаков позволяет создавать сложные критерии выбора имен файлов. В таблице 4-3 приведены примеры шаблонов и соответствующих им совпадений.

Таблица 4-3: Примеры подстановочных знаков

Шаблон	Соответствия
*	Все файлы
g*	Любой файл, начинающийся на «g»
b*.txt	Любой файл, начинающийся на «b», за которым следует любое количество символов и заканчивающийся на «.txt»

Шаблон	Соответствия
Data???	Любой файл, начинающийся с «Data» и за которым следуют ровно три символа
[abc]*	Любой файл, начинающийся на «a», «b» “b”, или «c»
BACKUP.[0-9][0-9][0-9]	Любой файл, начинающийся с «BACKUP.» за которыми следуют ровно три цифры
[[:upper:]]*	Любой файл, начинающийся с заглавной буквы
[![:digit:]]*	Любой файл, не начинающийся с цифры
*[[:lower:]]123]	Любой файл, заканчивающийся строчной буквой и цифрой «1», «2», или «3»

Подстановочные знаки можно использовать с любой командой, принимающей имена файлов в качестве аргументов, но подробнее об этом мы поговорим в главе 7.

Диапазоны символов

Если вы перешли из другой Unix-подобной среды или читали другие книги по этой теме, вы могли встречать обозначения диапазонов символов [A-Z] и [a-z]. Это традиционные для Unix способы, которые работали и в старых версиях Linux. Они всё еще могут работать, но с ними нужно быть осторожным: без правильной настройки они не дадут ожидаемого результата. Пока что вам следует избегать их и использовать вместо них классы символов.

Точечные файлы

Если мы посмотрим наш домашний каталог с помощью `ls`, используя опцию `-a`, то заметим ряд файлов и каталогов, имена которых начинаются с точки. Как мы уже обсуждали, эти файлы являются скрытыми. Это не специальный атрибут файла; это лишь означает, что файл не появится в выводе `ls`, если не указаны опции `-a` или `-A`. Скрытый статус также относится к подстановочным знакам. Скрытые файлы не будут отображаться,

если не использовать шаблон подстановочного знака, например `.*`. Однако при определенных обстоятельствах (в зависимости от конфигурации нашей оболочки) в результатах могут появиться и. (текущий каталог), и.. (родительский каталог). Чтобы исключить их, можно использовать шаблоны вроде `. [ ! . ] *` или `. ???`.

Подстановочные знаки работают и в GUI

Подстановочные знаки особенно ценны не только из-за их частого использования в командной строке, но и потому, что их поддерживают некоторые графические файловые менеджеры.

- В Nautilus (файловом менеджере GNOME) можно выбирать файлы, нажав Ctrl-s и введя шаблон с подстановочными знаками — будут выбраны файлы в текущем отображаемом каталоге.
- В Dolphin (файловом менеджере KDE) есть опция «Filter...», позволяющая использовать подстановочные знаки. Например, чтобы увидеть все файлы, начинающиеся со строчной «u» в каталоге `/usr/bin`, введите `«/usr/bin/u*»` в поле фильтра, и результат отобразится.

Многие идеи из интерфейса командной строки находят применение и в графическом интерфейсе. Это одна из многих причин, по которой рабочий стол Linux так мощен.

mkdir – Создание каталогов

Команда `mkdir` используется для создания каталогов. Она работает так:

```
mkdir directory...
```

Примечание к обозначениям: если в описании команды после аргумента стоят три точки (как выше), это означает, что аргумент можно повторять. Так, следующая команда создаст один каталог с именем `dir1`,

```
mkdir dir1
```

а следующая команда создаст три каталога с именами `dir1`, `dir2` и `dir3`:

```
mkdir dir1 dir2 dir3
```

cp — Копирование файлов и каталогов

Команда `ср` копирует файлы или каталоги. Её можно использовать двумя способами. Следующая команда копирует одиночный файл или каталог `item1` в файл или каталог `item2`

```
ср item1 item2
```

следующая команда копирует несколько объектов (файлов или каталогов) в каталог:

```
ср item... directory
```

Полезные опции и примеры

В таблице 4-4 перечислены некоторые часто используемые опции для `ср`.

Таблица 4-4: Общие опции `ср` в `те`,

Опция	Длинный параметр	Значение
-a	--archive	Копирует файлы, каталоги и все их атрибуты, включая владельцев и права доступа. Обычно копии получают атрибуты по умолчанию пользователя, выполняющего копирование. Мы рассмотрим права доступа к файлам главе 9.
-i	--interactive	Перед перезаписью существующего файла запрашивай у пользователя подтверждение. Если этот параметр не указан, <code>ср</code> выполнит перезапись файлов безмолвно (то есть без предупреждения).
-r	--recursive	Рекурсивное копирование каталогов и их содержимого. Этот параметр (или параметр -a) обязателен при копировании каталогов.
-u	--update	При копировании файлов из одного каталога в другой копируются только те файлы, которых нет или которые новее существующих соответствующих файлов в целевом каталоге. Это полезно, когда

Опция	Длинный параметр	Значение
		копирование большого количества файлов, так как она пропускает файлы, которые не нужно копировать.
-v	--verbose	вывод информационных сообщений в процессе копирования.

В таблице 4-5 перечислены типичные варианты использования `cp`.

Таблица 4-5: Распространенные приложения `cp`

Команда	Результаты
<code>cp file1 file2</code>	Копирует <code>file1</code> в <code>file2</code> . Если <code>file2</code> существует, он перезаписывается содержимым <code>file1</code> . Если <code>file2</code> не существует, он создается.
<code>cp -i file1 file2</code>	То же, что и предыдущая команда, кроме того, что если <code>file2</code> существует, пользователю будет предложено подтверждение перед перезаписью.
<code>cp file1 file2 dir1</code>	Скопируйте <code>file1</code> и <code>file2</code> в каталог <code>dir1</code> . При этом каталог <code>dir1</code> уже должен существовать.
<code>cp dir1/* dir2</code>	Используя подстановочный знак, скопируйте все файлы из <code>dir1</code> в <code>dir2</code> . Каталог <code>dir2</code> уже должен существовать.
<code>cp -r dir1 dir2</code>	Скопируйте содержимое каталога <code>dir1</code> в каталог <code>dir2</code> . Если каталог <code>dir2</code> не существует, он будет создан и после копирования будет содержать то же самое содержимое, что и каталог <code>dir1</code> . Если каталог <code>dir2</code> существует, то каталог <code>dir1</code> (и его содержимое) будет скопирован в <code>dir2</code> .

`mv` – Перемещение и переименование файлов

Команда `mv` выполняет как перемещение, так и переименование файлов, в зависимости от способа использования. В обоих случаях после операции исходное имя файла исчезает. `mv`

используется почти так же, как `cp`, как показано ниже, чтобы переместить или переименовать файл или каталог `item1` в `item2`

```
mv item1 item2
```

или для перемещения одного или нескольких объектов из одного каталога в другой:

```
mv item... directory
```

Полезные опции и примеры

`mv` имеет много общих опций с `cp`, как описано в таблице 4-6.

Таблица 4-6: `mv` Опции

Опция	Длинная опция	Значение
-i	--interactive	Перед перезаписью существующего файла запрашивать у пользователя подтверждение. Если этот параметр не указан, <code>mv</code> будет перезаписывать файлы без подтверждения.
-u	--update	При перемещении файлов из одного каталога в другой перемещайте только те файлы, которых нет или которые новее существующих соответствующих файлы в целевом каталоге.
-v	--verbose	Выводить информационные сообщения по мере того, выполнено.

В таблице 4-7 приведены примеры использования `mv`.

Таблица 4-7: `mv` Примеры *ся и 1 тает*

Команда	Результаты
<code>mv file1 file2</code>	Переместить <code>file1</code> в <code>file2</code> . Если <code>file2</code> существует, он удаляет <code>file1</code> переименовывается в <code>file2</code> . Если <code>file2</code> не существует, <code>file</code> просто переименовывается в <code>file2</code> . В обоих случаях <code>file1</code> перес существовать.
<code>mv -i file1 file2</code>	То же, что и предыдущая команда, но если <code>file2</code> существует, перед удалением появится приглашение.

Команда	Результаты
<code>mv file1 file2 dir1</code>	Переместите <code>file1</code> и <code>file2</code> в каталог <code>dir1</code> . При этом каталог <code>dir1</code> уже должен существовать.
<code>mv dir1 dir2</code>	Если каталог <code>dir2</code> не существует, переименуйте <code>dir1</code> в <code>dir2</code> . Если каталог <code>dir2</code> существует, переместите каталог <code>dir1</code> (и его содержимое) в каталог <code>dir2</code> .

rm — удаление файлов и каталогов

Команда `rm` используется для удаления файлов и каталогов, как показано в следующем примере:

```
rm item...
```

Здесь `item` — один или несколько файлов или каталогов.

Полезные параметры и примеры

В таблице 4-8 описаны некоторые распространенные параметры `rm`.

Таблица 4-8: параметры `rm`

Опция	Длинная опция	Значение
<code>-i</code>	<code>--interactive</code>	Перед удалением существующего файла запрашивать у пользователя подтверждение. Если эта опция не указана, <code>rm</code> будет удалять файлы без подтверждения.
<code>-r</code>	<code>--recursive</code>	Рекурсивное удаление каталогов. Это значит, что если удаляемый каталог содержит подкаталоги, они тоже будут удалены. Чтобы удалить каталог, необходимо указать этот параметр.
<code>-f</code>	<code>--force</code>	Игнорировать несуществующие файлы и не запрашивать подтверждение. Это переопределяет параметр <code>--interactive</code> .
<code>-v</code>	<code>--verbose</code>	Выводить информационные сообщения в процессе удаления.

В таблице 4-9 приведены примеры использования команды `rm`.

Таблица 4-9: `rm` Примеры

Команда	Результаты
<code>rm file1</code>	Тихо удалить <code>file1</code> .
<code>rm -i file1</code>	Как и предыдущая команда, только пользователь получит приглашение подтвердить удаление.
<code>rm -r file1 dir1</code>	Удалить <code>file1</code> , <code>dir1</code> и его содержимое.
<code>rm -rf file1 dir1</code>	Работает так же, как предыдущая команда, но если <code>file1</code> или <code>dir1</code> не существуют, <code>rm</code> продолжит выполнение без сообщений об ошибках.

Будьте осторожны с `rm`!

В Unix-подобных операционных системах, таких как Linux, нет команды восстановления удаленных файлов. Если вы удалили что-то с помощью `rm`, оно исчезнет навсегда. Linux предполагает, что вы достаточно умны и понимаете, что делаете.

Будьте особенно осторожны с подстановочными знаками. Рассмотрим классический пример. Допустим, вы хотите удалить только HTML-файлы в каталоге. Для этого введите следующее:

```
rm *.html
```

Это правильно, но если вы случайно поставите пробел между `*` и `.html`, вот так:

```
rm * .html
```

команда `rm` удалит все файлы в каталоге, а затем сообщит, что файла с именем `.html` не существует.

Совет: когда используете подстановочные знаки с `rm` (помимо тщательной проверки ввода!), сначала протестируйте подстановочный знак с помощью `ls`. Это позволит увидеть файлы, которые будут удалены. Затем нажмите стрелку вверх, чтобы вызвать команду, и замените `ls` на `rm`.

`ln` — создание ссылок

Команда `ln` используется для создания жестких или символических ссылок. Она применяется одним из двух способов. Следующая команда создает жесткую ссылку:

```
ln file link
```

Следующая команда создает символическую ссылку

```
ln -s item link
```

где `item` — это файл или каталог.

Жесткие ссылки

Жесткие ссылки — это классический способ создания ссылок в Unix, в отличие от более современных символических ссылок. По умолчанию у каждого файла есть одна жесткая ссылка, которая дает ему имя. Создавая жесткую ссылку, мы создаем дополнительную запись в каталоге для файла. У жестких ссылок есть два важных ограничения.

1. Жесткая ссылка не может указывать на файл за пределами своей файловой системы. Это значит, что ссылка не может ссылаться на файл, находящийся не на том же разделе диска, что и сама ссылка.
2. Жесткая ссылка не может указывать на каталог.

Жесткую ссылку невозможно отличить от самого файла. В отличие от символической ссылки, при просмотре содержимого каталога, содержащего жесткую ссылку, никаких специальных признаков не будет. При удалении жесткой ссылки сама ссылка удаляется, но содержимое файла продолжает существовать (то есть его место на диске не освобождается), пока не будут удалены все ссылки на этот файл.

О жестких ссылках важно знать, так как вы можете время от времени с ними сталкиваться, хотя в современной практике предпочтение отдается символическим ссылкам, которые мы рассмотрим далее.

Символические ссылки

Символические ссылки были созданы, чтобы преодолеть ограничения жестких ссылок. Они работают путем создания особого типа файла, содержащего текстовый указатель на целевой файл или каталог. В этом плане они во многом похожи на ярлыки Windows, хотя, разумеется, появились на много лет раньше.

Файл, на который указывает символическая ссылка, и сама ссылка практически неотличимы друг от друга. Например, если мы запишем что-то в символическую ссылку, данные запишутся в целевой файл. Однако при удалении символической ссылки удаляется только сама ссылка, а не файл. Если файл удалить раньше ссылки, ссылка продолжит существовать, но будет указывать в никуда. В таком случае говорят, что ссылка «битая». Во многих реализациях команда `ls` отображает битые ссылки отличительным цветом, например красным, чтобы их можно было заметить.

Концепция ссылок может показаться запутанной, но не сдавайтесь. Мы всё это опробуем на практике, и всё станет ясно.

Создадим игровую площадку

Поскольку мы собираемся заняться реальными манипуляциями с файлами, давайте создадим безопасное место для «игр» с командами управления файлами. Сначала нам понадобится каталог для работы. Мы создадим его в нашем домашнем каталоге и назовем `playground`.

Создание каталогов

Команда `mkdir` используется для создания каталога. Чтобы создать наш каталог для экспериментов, сначала убедимся, что мы находимся в домашнем каталоге, а затем создадим новый каталог.

```
[me@linuxbox ~]$ cd  
[me@linuxbox ~]$ mkdir playground
```

Чтобы сделать нашу песочницу интереснее, создадим в ней два каталога с именами `dir1` и `dir2`. Для этого мы сменим текущий рабочий каталог на `playground` и выполним еще одну `mkdir`.

```
[me@linuxbox ~]$ cd playground  
[me@linuxbox playground]$ mkdir dir1 dir2
```

Обратите внимание, что команда `mkdir` принимает несколько аргументов, что позволяет создать оба каталога одной командой.

Копирование файлов

Теперь добавим в нашу песочницу данные. Мы сделаем это путем копирования файла. С помощью команды `cp` мы скопируем файл `passwd` из каталога `/etc` в текущий рабочий каталог.

```
[me@linuxbox playground]$ cp /etc/passwd .
```

Обратите внимание, что для текущего рабочего каталога мы использовали сокращение — одиночную точку в конце. Теперь, если мы выполним `ls`, мы увидим наш файл.

```
[me@linuxbox playground]$ ls -l
total 12
drwxrwxr-x 2 me me 4096 2025-01-10 16:40 dir1
drwxrwxr-x 2 me me 4096 2025-01-10 16:40 dir2
-rw-r--r-- 1 me me 1650 2025-01-10 16:07 passwd
```

Теперь ради интереса повторим копирование с опцией «-v» (verbose), чтобы увидеть результат.

```
[me@linuxbox playground]$ cp -v /etc/passwd .
'/etc/passwd' -> './passwd'
```

Команда `cp` снова выполнила копирование, но на этот раз вывела краткое сообщение о выполняемой операции. Заметьте, что `cp` перезаписал первую копию без предупреждения. Опять же, это случай, когда `cp` предполагает, что мы знаем, что делаем. Чтобы получить предупреждение, мы добавим опцию «-i» (interactive).

```
[me@linuxbox playground]$ cp -i /etc/passwd .
cp: overwrite './passwd'?
```

Ответ на приглашение вводом `y` приведет к перезаписи файла; любой другой символ (например, `n`) заставит `cp` оставить файл без изменений.

Перемещение и переименование файлов

Сейчас имя `passwd` не кажется очень игровым, а ведь это игровая площадка, так что давайте изменим его на что-нибудь другое.

```
[me@linuxbox playground]$ mv passwd fun
```

Давайте немного развлечемся и переместим наш переименованный файл по каждому из каталогов и обратно. Следующая команда переместит его сначала в каталог `dir1`:

```
[me@linuxbox playground]$ mv fun dir1
```

Затем следующая команда переместит его из `dir1` в `dir2`:

```
[me@linuxbox playground]$ mv dir1/fun dir2
```

Наконец, следующая команда вернет его в текущий рабочий каталог:

```
[me@linuxbox playground]$ mv dir2/fun .
```

Next, давайте посмотрите, как `mv` влияет на каталоги. Сначала мы переместим наш файл данных в `dir1` снова, вот так:

```
[me@linuxbox playground]$ mv fun dir1
```

Затем мы перемещаем `dir1` в `dir2` и подтверждаем это с помощью `ls`.

```
[me@linuxbox playground]$ mv dir1 dir2
[me@linuxbox playground]$ ls -l dir2
total 4
drwxrwxr-x 2 me me 4096 2025-01-11 06:06 dir1
[me@linuxbox playground]$ ls -l dir2/dir1
total 4
-rw-r--r-- 1 me me 1650 2025-01-10 16:33 fun
```

Примечание: так как `dir2` уже существовал, `mv` переместил `dir1` в `dir2`. Если бы `dir2` не существовал, `mv` переименовал бы `dir1` в `dir2`. Наконец, вернем всё на свои места.

```
[me@linuxbox playground]$ mv dir2/dir1 .
[me@linuxbox playground]$ mv dir1/fun .
```

Создание жестких ссылок

Теперь попробуем создать ссылки. Сначала создадим жесткие ссылки на наш файл с данными следующим образом:

```
[me@linuxbox playground]$ ln fun fun-hard
[me@linuxbox playground]$ ln fun dir1/fun-hard
[me@linuxbox playground]$ ln fun dir2/fun-hard
```

Теперь у нас есть четыре экземпляра файла `fun`. Давайте взглянем на наш каталог `playground`.

```
[me@linuxbox playground]$ ls -l
total 16
drwxrwxr-x 2 me me 4096 2025-01-14 16:17 dir1
drwxrwxr-x 2 me me 4096 2025-01-14 16:17 dir2
-rw-r--r-- 4 me me 1650 2025-01-10 16:33 fun
-rw-r--r-- 4 me me 1650 2025-01-10 16:33 fun-hard
```

Мы заметили, что во втором поле вывода `fun` и `fun-hard` стоит 4 — это количество жестких ссылок, существующих для файла. Помните, что у файла всегда будет как минимум одна ссылка, так как имя файла создается именно ссылкой. Как же тогда понять, что `fun` и `fun-hard` — на самом деле один и тот же файл? В данном случае `ls` мало чем помогает. Хотя мы видим, что `fun` и `fun-hard` имеют одинаковый размер (поле 5), вывод не дает возможности убедиться в этом наверняка. Чтобы решить эту проблему, нам придется копнуть глубже.

Размышляя о жестких ссылках, полезно представить, что файлы состоят из двух частей.

1. Часть данных, содержащая содержимое файла
2. Часть имени, содержащая имя файла

Создавая жесткие ссылки, мы на самом деле создаем дополнительные части имени, которые ссылаются на одну и ту же часть данных. Система назначает цепочку дисковых блоков тому, что называется `inode`, а затем связывает его с частью имени. Таким образом, каждая жесткая ссылка ссылается на конкретный `inode`, содержащий содержимое файла.

Команда `ls` позволяет получить эту информацию. Она вызывается с опцией `-li`.

```
[me@linuxbox playground]$ ls -li
total 16
12353539 drwxrwxr-x 2 me me 4096 2025-01-14 16:17 dir1
12353540 drwxrwxr-x 2 me me 4096 2025-01-14 16:17 dir2
12353538 -rw-r--r-- 4 me me 1650 2025-01-10 16:33 fun
12353538 -rw-r--r-- 4 me me 1650 2025-01-10 16:33 fun-hard
```

В этом выводе первым полем является номер `inode`; как видно, `fun` и `fun-hard` имеют одинаковый номер `inode`, что подтверждает: это один и тот же файл.

Создание символических ссылок

Символические ссылки были созданы, чтобы устранить два недостатка жестких ссылок.

1. Жесткие ссылки не могут пересекать границы физических устройств.
2. Жесткие ссылки не могут ссылаться на каталоги, только на файлы.

Символические ссылки — это особый тип файлов, содержащий текстовый указатель на целевой файл или каталог.

Создание символических ссылок похоже на создание жестких ссылок.

```
[me@linuxbox playground]$ ln -s fun fun-sym
[me@linuxbox playground]$ ln -s ../fun dir1/fun-sym
[me@linuxbox playground]$ ln -s ../fun dir2/fun-sym
```

Первый пример предельно прост: мы просто добавляем опцию `-s`, чтобы создать символическую ссылку вместо жесткой. Но как быть со следующими двумя? Помните, что при создании символической ссылки мы создаем текстовое описание того, где находится целевой файл относительно самой ссылки. Это проще понять, если взглянуть на `ls` вывод ниже:

```
[me@linuxbox playground]$ ls -l dir1
total 4
-rw-r--r-- 4 me me 1650 2025-01-10 16:33 fun-hard
lrwxrwxrwx 1 me me 6 2025-01-15 15:17 fun-sym -> ../fun
```

Вывод `fun-sym` в `dir1` показывает, что это символическая ссылка, благодаря ведущему `l` в первом поле, и что она указывает на `../fun fun-sym fun`, что верно. Относительно расположения, находится в каталоге выше. Также заметьте, что размер файла символической ссылки равен 6 — это количество символов в строке `../fun`, а не размер файла, на который она указывает.

При создании символических ссылок можно использовать как абсолютные пути, как показано здесь,

```
[me@linuxbox playground]$ ln -s /home/me/playground/fun dir1/fun-sym
```

так и относительные пути, как в предыдущем примере. В большинстве случаев предпочтительнее использовать относительные пути, так как это позволяет переименовывать или перемещать дерево каталогов с символическими ссылками и их целевыми файлами, не нарушая работу ссылок.

Помимо обычных файлов, символьные ссылки могут указывать и на каталоги.

```
[me@linuxbox playground]$ ln -s dir1 dir1-sym
[me@linuxbox playground]$ ls -l
total 16
drwxrwxr-x 2 me me 4096 2025-01-15 15:17 dir1
lrwxrwxrwx 1 me me 4 2025-01-16 14:45 dir1-sym -> dir1
drwxrwxr-x 2 me me 4096 2025-01-15 15:17 dir2
-rw-r--r-- 4 me me 1650 2025-01-10 16:33 fun
-rw-r--r-- 4 me me 1650 2025-01-10 16:33 fun-hard
lrwxrwxrwx 1 me me 3 2025-01-15 15:15 fun-sym -> fun
```

Удаление файлов и каталогов

Как мы уже разбирали, команда `rm` используется для удаления файлов и каталогов. Мы воспользуемся ею, чтобы немного навести порядок в нашей рабочей среде. Сначала удалим одну из наших жестких ссылок.

```
[me@linuxbox playground]$ rm fun-hard
[me@linuxbox playground]$ ls -l
total 12
drwxrwxr-x 2 me me 4096 2025-01-15 15:17 dir1
lrwxrwxrwx 1 me me 4 2025-01-16 14:45 dir1-sym -> dir1
drwxrwxr-x 2 me me 4096 2025-01-15 15:17 dir2
-rw-r--r-- 3 me me 1650 2025-01-10 16:33 fun
lrwxrwxrwx 1 me me 3 2025-01-15 15:15 fun-sym -> fun
```

Все сработало как ожидалось. Файл `fun-hard` удален, а число ссылок для `fun` уменьшилось с четырех до трех, что видно по

второму полю в списке каталога. Далее мы удалим файл `fun` и, чисто ради интереса, добавим опцию `-i`, чтобы увидеть результат.

```
[me@linuxbox playground]$ rm -i fun
rm: remove regular file `fun'?
```

Введите `y` в приглашении, и файл будет удален. Теперь посмотрим на вывод `ls`. Заметили, что случилось с `fun-sym`? Поскольку это символьная ссылка, указывающая на несуществующий файл, ссылка стала битой.

```
[me@linuxbox playground]$ ls -l
total 8
drwxrwxr-x 2 me me 4096 2025-01-15 15:17 dir1
lrwxrwxrwx 1 me me 4 2025-01-16 14:45 dir1-sym -> dir1
drwxrwxr-x 2 me me 4096 2025-01-15 15:17 dir2
lrwxrwxrwx 1 me me 3 2025-01-15 15:15 fun-sym -> fun
```

Большинство дистрибутивов Linux настроены так, чтобы `ls` отображал битые ссылки. Наличие битой ссылки само по себе не опасно, но создает беспорядок. Если мы попытаемся использовать битую ссылку, то увидим следующее:

```
[me@linuxbox playground]$ less fun-sym
fun-sym: No such file or directory
```

Давайте немного приберемся. Мы удалим здесь символические ссылки:

```
[me@linuxbox playground]$ rm fun-sym dir1-sym
[me@linuxbox playground]$ ls -l
total 8
drwxrwxr-x 2 me me 4096 2025-01-15 15:17 dir1
drwxrwxr-x 2 me me 4096 2025-01-15 15:17 dir2
```

О символических ссылках важно помнить: большинство операций с файлами выполняется над целевым объектом, а не над самой ссылкой. `rm` — исключение. При удалении ссылки удаляется именно ссылка, а не цель.

Наконец, мы удалим нашу песочницу. Для этого вернемся в домашний каталог и используем `rm` с опцией рекурсии (`-r`), чтобы удалить `playground` и всё его содержимое, включая подкаталоги.

```
[me@linuxbox playground]$ cd
[me@linuxbox ~]$ rm -r playground
```

Создание символических ссылок через GUI

Файловые менеджеры в GNOME и KDE предлагают простой способ автоматического создания символических ссылок. В GNOME, если удерживать Ctrl+Shift при перетаскивании файла, создается ссылка, а не копия (или перемещение) файла. В KDE при перетаскивании файла появляется меню с выбором: копировать, переместить или создать ссылку.

Итоги

Мы прошли большой объем материала, и на его освоение потребуется время. Выполняйте упражнения в «песочнице» снова и снова, пока не разберетесь. Важно хорошо понимать базовые команды манипуляции файлами и подстановочные знаки. Вы можете расширить упражнения, добавляя новые файлы и каталоги или используя подстановочные знаки для выбора файлов. Концепция ссылок поначалу может запутать, но уделите время изучению того, как они работают. Они могут стать вашим настоящим спасением.

Что почитать

- Обсуждение символьных ссылок:
[http://en.wikipedia.org/wiki/Symbolic link](http://en.wikipedia.org/wiki/Symbolic_link)

5 — Работа с командами

До этого момента мы видели ряд загадочных команд, каждая со своими таинственными опциями и аргументами. В этой главе мы попытаемся развеять эту тайну и даже научимся создавать собственные команды. В главе рассматриваются следующие команды:

- `type` — Показывает, как интерпретируется имя команды.
- `which` — Показывает, какая исполняемая программа будет запущена.
- `help` — Помощь по встроенным командам оболочки.
- `man` — вывести страницу руководства к команде.
- `apropos` — вывести список подходящих команд.
- `info` — вывести информационную запись команды.
- `whatis` — вывести краткие описания страниц руководства.
- `alias` — создать псевдоним для команды.

Что же такое команды?

Команда может быть одним из четырех вариантов:

- Исполняемая программа, подобная файлам, которые мы видели в `/usr/bin`. К этой категории относятся как скомпилированные бинарные файлы (например, программы на C и C++), так и программы на языках сценариев, таких как `shell`, `Perl`, `Python`, `Ruby` и др.
- Команда, встроенная в саму оболочку. `bash` поддерживает ряд команд, называемых встроенными командами оболочки (`shell builtins`). Например, команда `cd` является встроенной командой оболочки.
- Функция оболочки. Функции оболочки — это миниатюрные скрипты, интегрированные в среду. Мы разберем настройку среды и написание функций оболочки в следующих главах, а пока просто знайте об их существовании.
- Псевдоним. Псевдонимы — это команды, которые мы определяем сами на основе других команд.

Определение команд

Часто полезно точно знать, какой из четырех видов команд используется, и Linux предлагает несколько способов это выяснить.

type — определение типа команды

Команда `type` — это встроенная команда оболочки, которая показывает, какой тип ко манды будет выполнить оболочка при указанном имени. Она работает следующим образом:

```
type command
```

Здесь `command` — это имя команды, которую мы хотим изучить. Рассмотрим несколько примеров:

```
[me@linuxbox ~]$ type type
type is a shell builtin
[me@linuxbox ~]$ type ls
ls is aliased to `ls --color=auto'
[me@linuxbox ~]$ type cp
cp is /usr/bin/cp
```

Здесь показаны результаты для трех разных команд. Обратите внимание на результат для `ls` (взят из системы Fedora) и на то, что команда `ls` на самом деле является псевдонимом для команды `ls` с добавленной опцией `--color=auto`. Теперь мы знаем, почему вывод `ls` отображается в цвете!

which — вывод местоположения исполняемого файла

Иногда в системе установлено несколько версий одной и той же программы. Для настольных систем это редкость, но на крупных серверах такое встречается часто. Чтобы определить точное местоположение конкретного исполняемого файла, используется команда `which`.

```
[me@linuxbox ~]$ which ls
/usr/bin/ls
```

Команда `which` работает только с исполняемыми программами, а не со встроенными командами оболочки или псевдонимами, которые заменяют реальные программы. Если попытаться использовать `which` для встроенной команды оболочки, например `cd`, то в ответ не последует ничего или появится сообщение об ошибке.

```
[me@linuxbox ~]$ which cd
/usr/bin/which: no cd in (/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games)
```

Этот ответ — изысканный способ сказать «команда не найдена».

Получение документации к команде

Теперь, понимая, что такое команда, мы можем искать документацию, доступную для каждого её типа.

`he lp` — Справка по встроенным командам оболочки

`bash` имеет встроенную систему справки для каждой встроенной команды оболочки. Чтобы воспользоваться ею, введите `he lp`, а затем имя встроенной команды. Вот пример:

```
[me@linuxbox ~]$ help cd
cd: cd [-L|[-P [-e]] [-@]] [dir]
    Change the shell working directory.
    Change the current directory to DIR.  The default DIR is the
    value of the HOME shell variable.
    The variable CDPATH defines the search path for the directory
    containing DIR.  Alternative directory names in CDPATH are
    separated by a colon (:). A null directory name is the same as
    the current directory.  If DIR begins with a slash (/), then
    CDPATH is not used.
    If the directory is not found, and the shell option `cdable_vars'
    is set, the word is assumed to be a variable name.  If that
    variable has a value, its value is used for DIR.
    Options:
      -L    force symbolic links to be followed: resolve symbolic
            links in DIR after processing instances of `..'
      -P    use the physical directory structure without following
            symbolic links: resolve symbolic links in DIR before
            processing instances of `..'
      -e    if the -P option is supplied, and the current working
            directory cannot be determined successfully, exit with
            a non-zero status
      -@    on systems that support it, present a file with extended
            attributes as a directory containing the file attributes
    The default is to follow symbolic links, as if `-L' were
    specified. `..' is processed by removing the immediately previous
    pathname component back to a slash or the beginning of DIR.
    Exit Status:
    Returns 0 if the directory is changed, and if $PWD is set
    successfully when -P is used; non-zero otherwise.
```

Примечание о нотации: квадратные скобки в описании синтаксиса команды обозначают необязательные элементы. Вертикальная черта указывает на взаимоисключающие элементы. Рассмотрим предыдущую команду `cd`:

```
cd [-L|[-P[-e]]] [dir]
```

Эта запись означает, что за командой `cd` может следовать либо `-L`, либо `-P` `-e` (необязательно); более того, если опция указана, за ней может следовать необязательный аргумент `dir`.

Хотя вывод `he lp` для команд `cd` лаконичен и точен, он вовсе не является учебным пособием и, как мы видим, упоминает множество вещей, о которых мы еще не говорили! Не волнуйтесь. Мы к этому придем.

Совет: при использовании команды `he lp` с опцией `-m`, `he lp` выведет результат в альтернативном формате, похожем на `man`-

страницу (см. ниже).

- `help` — вывод информации об использовании

Многие исполняемые программы поддерживают опцию «`--help`», которая выводит описание доступного синтаксиса и опций команды. Например:

```
[me@linuxbox ~]$ mkdir --help
Usage: mkdir [OPTION] DIRECTORY...
Create the DIRECTORY(ies), if they do not already exist.
Mandatory arguments to long options are mandatory for short options
too.
  -m, --mode=MODE      set file mode (as in chmod), not a=rwx - umask
  -p, --parents         no error if existing, make parent directories as
                        needed
  -v, --verbose        print a message for each created directory
  --help               display this help and exit
  --version            output version information and exit
  -Z, --context=CONTEXT (SELinux) set security context to CONTEXT
Report bugs to <bug-coreutils@gnu.org>.
```

Некоторые программы не поддерживают опцию «`--help`», но всё же попробуйте её. Часто это вызывает сообщение об ошибке, которое содержит ту же информацию по использованию.

`man` — Просмотр страницы руководства программы

Большинство исполняемых программ, предназначенных для использования в командной строке, предоставляют официальную документацию, называемую руководством или `man`-страницей. Для их просмотра используется специальная программа для постраничного вывода — `man`. Её используют следующим образом, где `program` — имя команды для просмотра:

```
man program
```

Формат `man`-страниц может различаться, но обычно они содержат следующее:

- Заголовок (название страницы)
- Краткий синтаксис команды
- Описание назначения команды
- Список и описание каждой опции команды

Однако страницы `man` обычно не содержат примеров и служат справочником, а не учебным пособием. Для примера попробуем просмотреть `man`-страницу для `ls command`.

```
[me@linuxbox ~]$ man ls
```

В большинстве систем Linux `man` использует `less` для отображения страниц руководства, поэтому при просмотре

страницы работают все привычные команды `less`.

«Руководство», которое выводит `man`, разделено на разделы и охватывает не только пользовательские команды, но также команды системного администрирования, программные интерфейсы, форматы файлов и многое другое. В таблице 5-1 описана структура руководства.

Таблица 5-1: Структура `man`-страницы

Раздел	Содержание
1	Команды пользователя
2	Программные интерфейсы системных вызовов ядра
3	Программные интерфейсы библиотеки C
4	Специальные файлы, такие как узлы устройств и драйверы
5	Форматы файлов
6	Игры и развлечения, такие как хранители экрана
7	Разное
8	Команды системного администрирования

Иногда для поиска нужной информации требуется обратиться к конкретному разделу руководства. Это особенно важно, если мы ищем формат файла, который совпадает с именем команды. Без указания номера раздела всегда будет выводиться первое совпадение, скорее всего, в разделе 1. Чтобы указать номер раздела, используйте `man` следующим образом:

```
man section search_term
```

Вот пример:

```
[me@linuxbox ~]$ man 5 passwd
```

Это отобразит страницу `man` с описанием формата файла `/etc/passwd`.

`apropos` — Отображение подходящих команд

Также можно искать совпадения в списке страниц `man` по ключевому слову. Это грубый, но иногда полезный способ. Вот пример поиска страниц `man` с использованием термина `partition`:

```
[me@linuxbox ~]$ apropos partition
addpart (8)          - simple wrapper around the "add partition"...
all-swaps (7)        - event signalling that all swap partitions...
```

cfdisk (8)	- display or manipulate disk partition table
cgdisk (8)	- Curses-based GUID partition table (GPT)...
delpart (8)	- simple wrapper around the "del partition"...
fdisk (8)	- manipulate disk partition table
fixparts (8)	- MBR partition table repair utility
gdisk (8)	- Interactive GUID partition table (GPT)...
mpartition (1)	- partition an MSDOS hard disk
partprobe (8)	- inform the OS of partition table changes
partx (8)	- tell the Linux kernel about the presence...
resizepart (8)	- simple wrapper around the "resize partition..."
sfdisk (8)	- partition table manipulator for Linux
sgdisk (8)	- Command-line GUID partition table (GPT)...

Первое поле в каждой строке вывода — это имя страницы `man`, а второе — номер раздела. Примечание: команда `man` с опцией «-k» выполняет ту же функцию, что и `apropos`.

whatis — Вывод кратких описаний страниц `man`

Программа `whatis` выводит имя и краткое описание страницы `man`, соответствующей заданному ключевому слову:

```
[me@linuxbox ~]$ whatis ls
ls                (1) - list directory contents
```

Самая жестокая страница `man`

Как мы уже видели, страницы `man`, поставляемые с Linux и другими Unix-подобными системами, предназначены для справки, а не для обучения. Многие страницы `man` трудно читать, но, я считаю, главный приз за сложность должна получить страница `man` для `bash`. Во время подготовки этой книги я внимательно изучил страницу `man` для `bash`, чтобы убедиться, что охватил большинство её тем. В печатном виде она занимает более 80 страниц, крайне плотна, а её структура совершенно лишена смысла для нового пользователя.

С другой стороны, оно очень точное, лаконичное и чрезвычайно полное. Так что изучите его, если осмелитесь, и ждите того дня, когда чтение станет для вас понятным.

`info` – Отображение записи `info` программы

Проект GNU предлагает альтернативу `man`-страницам для своих программ — формат `info`. Руководства `info` просматриваются с помощью программы-ридера, которая весьма метко называется `info`.

Страницы `info` содержат гиперссылки, подобно веб-страницам. Вот пример:

```

File: coreutils.info, Node: ls invocation, Next: dir invocation,
Up: Directory listing
10.1 `ls': List directory contents
=====
The `ls' program lists information about files (of any type,
including directories). Options and file arguments can be intermixed
arbitrarily, as usual.
  For non-option command-line arguments that are directories, by
default `ls' lists the contents of directories, not recursively, and
omitting files with names beginning with `.'. For other non-option
arguments, by default `ls' lists just the filename. If no non-option
argument is specified, `ls' operates on the current directory, acting
as if it had been invoked with a single argument of `.'.
  By default, the output is sorted alphabetically, according to the
--zz-Info: (coreutils.info.gz)ls invocation, 63 lines --Top-----

```

Программа `info` читает `info`-файлы, имеющие древовидную структуру из отдельных узлов, каждый из которых содержит одну тему. `Info`-файлы содержат гиперссылки, позволяющие переходить от узла к узлу. Гиперссылку можно узнать по ведущей звездочке; чтобы активировать её, наведите курсор и нажмите ENTER.

Чтобы вызвать `info`, введите `info`, после чего (необязательно) укажите имя программы. В таблице 5-2 описаны команды для управления чтением при просмотре страницы `info`.

Таблица 5-2: `info` команды

Команда	Действие
?	Вывод справки по команде
PgUp или Backspace	Показать предыдущую страницу
PgDn или Space	Показать следующую страницу
n	Далее: показать следующий узел
p	Назад: показать предыдущий узел
u	Вверх: показать родительский узел текущего узла, обычно меню
ENTER	Перейти по гиперссылке в месте нахождения курсора
q	Quit

Большинство рассмотренных нами программ командной строки входят в пакет `coreutils` проекта GNU, поэтому ввод

```
[me@linuxbox ~]$ info coreutils
```

отобразит страницу меню с гиперссылками на каждую программу из пакета coreutils.

README и другие файлы документации программ

Многие установленные в системе пакеты программ содержат файлы документации в каталоге `/usr/share/doc`. Большинство из них хранятся в формате обычного текста (часто в Markdown — популярном формате текстовых документов) и могут быть просмотрены с помощью `less`. Некоторые файлы представлены в формате HTML и открываются веб-браузером. Встречаются и файлы с расширением `«.gz»`. Это означает, что они сжаты программой `gzip`. Пакет `gzip` включает специальную версию `less`, называемую `zless`, которая позволяет просматривать содержимое текстовых файлов, сжатых с помощью `gzip`.

Создание собственных команд с помощью `alias`

Пришло время нашего первого опыта программирования! Мы создадим собственную команду, используя `alias`. Но прежде чем начать, раскроем небольшой трюк командной строки. Можно разместить несколько команд в одной строке, разделяя их точкой с запятой. Это работает так:

```
command1; command2; command3...
```

Вот пример, который мы будем использовать:

```
[me@linuxbox ~]$ cd /usr; ls; cd -  
bin games include lib local sbin share src  
/home/me  
[me@linuxbox ~]$
```

Как видим, мы объединили три команды в одной строке.

Сначала мы меняем каталог на `/usr` `'cd -'`, затем выводим содержимое каталога и, наконец, возвращаемся в исходный каталог (используя), чтобы оказаться в исходной точке. Теперь превратим эту последовательность в новую команду `us` — `ing alias`. Первым делом нужно придумать имя для нашей новой команды. Попробуем `test`. Но прежде стоит проверить, не занято ли имя `test`. Для этого можно снова использовать команду `type`:

```
[me@linuxbox ~]$ type test  
test is a shell builtin
```

Ой! Имя `test` уже занято. Давайте попробуем `foo`:

```
[me@linuxbox ~]$ type foo
bash: type: foo: not found
```

Отлично! `foo` свободно. Создадим наш `alias`:

```
[me@linuxbox ~]$ alias foo='cd /usr; ls; cd -'
```

Обратите внимание на структуру этой команды:

```
alias name='string'
```

После команды `alias` мы даем `alias` имя, за которым сразу (без пробелов) следует знак равенства, а затем — строка в кавычках, содержащая значение, которое будет присвоено этому имени. После определения `alias` мы можем использовать его везде, где оболочка ожидает команду. Давайте попробуем:

```
[me@linuxbox ~]$ foo
bin games include lib local sbin share src
/home/me
[me@linuxbox ~]$
```

Мы также можем снова использовать команду `type`, чтобы увидеть наш `alias`:

```
[me@linuxbox ~]$ type foo
foo is aliased to `cd /usr; ls; cd -'
```

Для удаления псевдонима используется команда `unalias`, вот так:

```
[me@linuxbox ~]$ unalias foo
[me@linuxbox ~]$ type foo
bash: type: foo: not found
```

Хотя мы намеренно не называли наш `alias` именем существующей команды, такое случается часто. Обычно это делают, чтобы применять часто используемую опцию при каждом вызове обычной команды. Например, ранее мы видели, как для команды `ls` часто создают `alias` для поддержки цвета.

```
[me@linuxbox ~]$ type ls
ls is aliased to `ls --color=auto'
```

Чтобы увидеть все определенные в окружении `alias`, используйте команду `alias` без аргументов. Ниже приведены некоторые `alias`, определенные в системе Fedora по умолчанию. Попробуйте разобраться, что они делают.

```
[me@linuxbox ~]$ alias
alias l.='ls -d .* --color=auto'
alias ll='ls -l --color=auto'
alias ls='ls --color=auto'
```


У определения alias в командной строке есть одна маленькая проблема: они исчезают после завершения сеанса оболочки. В главе 11 мы узнаем, как добавлять собственные alias в файлы, которые настраивают окружение при каждом входе в систему, но пока наслаждайтесь тем, что мы сделали наш первый, пусть и крошечный, шаг в мир программирования оболочек!

Итоги

Теперь, когда мы научились искать документацию к командам, найдите документацию для всех команд, которые мы встретили до сих пор. Изучите доступные дополнительные опции и попробуйте их!

Что почитать

Существует множество онлайн-источников документации по Linux и командной строке. Вот некоторые из лучших:

- Справочное руководство Bash — это справочник по bash оболочке. Это всё ещё справочник, но он содержит примеры и читается легче, чем bash man page.
<http://www.gnu.org/software/bash/manual/bashref.html>
- Bash FAQ содержит ответы на часто задаваемые вопросы о bash. Этот список ориентирован на пользователей среднего и продвинутого уровней, но содержит много полезной информации. <http://mywiki.woolledge.org/BashFAQ>
- Проект GNU предоставляет обширную документацию к своим программам, составляющим основу работы в командной строке Linux. Полный список доступен здесь:
<http://www.gnu.org/manual/manual.html>
- В Wikipedia есть интересная статья о man pages:
http://en.wikipedia.org/wiki/Man_page

6 — Перенаправление

В этой главе мы раскроем возможности, которые могут оказаться самой крутой функцией командной строки. Это перенаправление ввода/вывода. I/O означает input/output; с помощью этого механизма можно перенаправлять ввод и вывод команд в файлы и из них, а также объединять несколько команд в мощные конвейеры. Чтобы продемонстрировать эти возможности, мы представим следующие команды:

- `cat` — Конкатенация файлов
- `sort` — Сортировка строк текста
- `uniq` — Вывод или пропуск повторяющихся строк
- `grep` — Вывод строк, соответствующих шаблону
- `wc` — вывод количества строк, слов и байт для каждого файла
- `head` — вывод первой части файла
- `tail` — вывод последней части файла
- `tee` — чтение из стандартного ввода и запись в стандартный вывод и файлы

Стандартный ввод, вывод и ошибки

Многие программы, которые мы использовали, создают какой-либо вывод. Этот вывод часто состоит из двух типов:

- Результаты работы программы — то есть данные, которые она должна выводить,
- сообщения о статусе и ошибках, которые говорят нам о ходе выполнения программы,

если мы рассмотрим такую команду, как `ls`, то увидим, что она выводит результаты и сообщения об ошибках на экран.

Следуя принципу Unix «всё есть файл», такие программы, как `ls`, на самом деле отправляют результаты в специальный файл под названием стандартный вывод (часто обозначаемый как `stdout`), а сообщения о состоянии — в другой файл, называемый стандартным выводом ошибок (`stderr`). По умолчанию и стандартный вывод, и стандартный вывод ошибок привязаны к экрану и не сохраняются в файл на диске.

Кроме того, многие программы принимают данные через механизм под названием стандартный ввод (`stdin`), который по умолчанию привязан к клавиатуре.

Перенаправление ввода-вывода позволяет менять направление потоков. Обычно вывод идет на экран, а ввод поступает с клавиатуры, но с помощью перенаправления ввода-вывода это можно изменить.

Перенаправление стандартного вывода

Перенаправление ввода-вывода позволяет переопределить место назначения стандартного вывода. Чтобы перенаправить стандартный вывод в файл вместо экрана, мы используем оператор перенаправления `>`, а затем имя файла. Зачем это нужно? Часто полезно сохранять вывод команды в файл. Например, мы можем приказать оболочке отправить вывод `ls` в файл `ls-output.txt` вместо экрана:

```
[me@linuxbox ~]$ ls -l /usr/bin > ls-output.txt
```

Здесь мы создали подробный список содержимого `/usr/bin` и отправили результат в файл `ls-output.txt`. Давайте изучим перенаправленный вывод команды, представленный ниже:

```
[me@linuxbox ~]$ ls -l ls-output.txt
-rw-rw-r-- 1 me me 167878 2025-02-01 15:07 ls-output.txt
```

Отлично — хороший большой текстовый файл. Если мы посмотрим на файл с помощью `less`, то увидим, что файл `ls-output.txt` действительно содержит результаты нашей команды `ls`.

```
[me@linuxbox ~]$ less ls-output.txt
```

Теперь повторим тест перенаправления, но с нюансом. Мы изменим имя каталога на несуществующее:

```
[me@linuxbox ~]$ ls -l /bin/usr > ls-output.txt
ls: cannot access /bin/usr: No such file or directory
```

Мы получили сообщение об ошибке. Это логично, так как мы указали несуществующий каталог `/bin/usr`, но почему сообщение об ошибке отобразилось на экране, а не было перенаправлено в файл `ls-output.txt`? Ответ в том, что программа `ls` не отправляет сообщения об ошибках в стандартный вывод. Вместо этого, как и большинство грамотно написанных программ Unix, она отправляет сообщения об ошибках в стандартный поток ошибок (`stderr`). Поскольку мы перенаправили только стандартный вывод, а не стандартный поток ошибок, сообщение об ошибке все равно попало на экран.

Через минуту мы разберем, как перенаправить стандартный поток ошибок, но сначала посмотрим, что случилось с нашим выходным файлом:

```
[me@linuxbox ~]$ ls -l ls-output.txt
-rw-rw-r-- 1 me me 0 2025-02-01 15:08 ls-output.txt
```

Теперь длина файла равна нулю! Это происходит потому, что при использовании оператора перенаправления «>» целевой файл всегда перезаписывается с самого начала. Поскольку наша команда `ls` не выдала результатов, а только сообщение об ошибке, операция перенаправления начала перезапись файла и прервалась из-за ошибки, что привело к его усечению. На самом деле, если нам нужно намеренно усечь файл (или создать новый пустой файл), можно использовать такой трюк:

```
[me@linuxbox ~]$ > ls-output.txt
```

Простое использование оператора перенаправления без предшествующей команды усечет существующий файл или создаст новый пустой файл.

Так как же добавить перенаправленный вывод в конец файла вместо его перезаписи с начала? Для этого используется оператор перенаправления `>>`, вот так:

```
[me@linuxbox ~]$ ls -l /usr/bin >> ls-output.txt
```

Использование оператора `>>` приведет к тому, что вывод будет добавлен в конец файла. Если файл не существует, он будет создан так же, как при использовании оператора `>`. Давайте проверим это, повторив команду и добавив её вывод в файл:

```
[me@linuxbox ~]$ ls -l /usr/bin >> ls-output.txt
[me@linuxbox ~]$ ls -l /usr/bin >> ls-output.txt
[me@linuxbox ~]$ ls -l /usr/bin >> ls-output.txt
[me@linuxbox ~]$ ls -l ls-output.txt
-rw-rw-r-- 1 me me 503634 2025-02-01 15:45 ls-output.txt
```

Мы повторно выполнили команду `ls` трижды, в результате чего выходной файл стал в три раза больше.

Групповые команды

Представим ситуацию, когда нам нужно выполнить серию команд и отправить результаты в лог-файл. Зная то, что мы уже изучили, мы могли бы сделать так:

```
[me@linuxbox ~]$ command1 > logfile.txt
[me@linuxbox ~]$ command2 >> logfile.txt
[me@linuxbox ~]$ command3 >> logfile.txt
```

Первая команда в этой последовательности создает или очищает файл `logfile.txt`, а каждая последующая добавляет свой вывод в этот файл. Этот метод работает, но требует много лишней работы. Должен быть способ лучше.

Как мы видели в предыдущей главе, можно разместить несколько команд в одной строке вот так:

```
[me@linuxbox ~]$ command1; command2; command3
```

Таким образом, мы могли бы поместить все команды и перенаправления в одну строку:

```
[me@linuxbox ~]$ command1 > logfile.txt; command2 >> logfile.txt;  
command3 >> logfile.txt
```

Но что, если рассматривать всю последовательность как единое целое с одним потоком вывода? Это можно сделать, создав групповую команду. Для этого заключим последовательность в фигурные скобки:

```
[me@linuxbox ~]$ { command1; command2; command3; } > logfile.txt
```

Поскольку наша последовательность заключена в фигурные скобки, оболочка будет рассматривать её как одну команду с точки зрения перенаправления. Примечание: оболочке требуются пробелы вокруг скобок, а последняя команда в последовательности должна завершаться точкой с запятой или символом новой строки.

Перенаправление стандартного потока ошибок

Перенаправление стандартного потока ошибок не так удобно, как использование специального оператора перенаправления. Чтобы перенаправить стандартный поток ошибок, нужно обратиться к его дескриптору файла. Программа может выводить данные в любой из нескольких пронумерованных потоков. Хотя мы называли первые три потока стандартным вводом, выводом и ошибкой, оболочка обращается к ним внутренне как к дескрипторам файлов 0, 1 и 2 соответственно. Оболочка позволяет перенаправлять файлы, используя номер дескриптора. Поскольку стандартный поток ошибок совпадает с дескриптором файла № 2, мы можем перенаправить его следующим образом:

```
[me@linuxbox ~]$ ls -l /bin/usr 2> ls-error.txt
```

Дескриптор файла «2» ставится непосредственно перед оператором перенаправления, чтобы перенаправить

стандартный поток ошибок в файл `ls-error.txt`.

Перенаправление стандартного вывода и стандартного потока ошибок в один файл

Бывают случаи, когда нужно сохранить весь вывод команды в один файл. Для этого необходимо одновременно перенаправить и стандартный вывод, и стандартный поток ошибок. Есть два способа сделать это. Здесь показан традиционный метод, работающий в старых версиях оболочки:

```
[me@linuxbox ~]$ ls -l /bin/usr > ls-output.txt 2>&1
```

Используя этот метод, мы выполняем два перенаправления. Сначала мы перенаправляем стандартный вывод в файл `ls-output.txt`, а затем перенаправляем дескриптор файла 2 (стандартный поток ошибок) на дескриптор файла 1 (стандартный вывод) с помощью записи `2>&1`.

Обратите внимание, что порядок перенаправлений имеет значение. Перенаправление стандартного потока ошибок должно всегда идти после перенаправления стандартного вывода, иначе оно не сработает. Следующий пример перенаправляет стандартный поток ошибок в файл `ls-output.txt`:

```
>ls-output.txt 2>&1
```

Однако, если изменить порядок на следующий, стандартный поток ошибок будет направлен на экран.

```
2>&1 >ls-output.txt
```

В последних версиях `bash` предусмотрен второй, более упрощенный способ выполнения такого комбинированного перенаправления:

```
[me@linuxbox ~]$ ls -l /bin/usr &> ls-output.txt
```

В этом примере мы используем обозначение `&>`, чтобы перенаправить стандартный вывод и стандартную ошибку в файл `ls-output.txt`. Мы также можем добавить потоки стандартного вывода и стандартной ошибки в один файл следующим образом:

```
[me@linuxbox ~]$ ls -l /bin/usr &>> ls-output.txt
```

Утилизация нежелательного вывода

Иногда «молчание — золото», и нам не нужен вывод команды — мы просто хотим его выбросить. Это особенно актуально для сообщений об ошибках и статусе. Система позволяет сделать это, перенаправив вывод в специальный файл «/dev/null». Этот файл является системным устройством, которое часто называют *bit bucket*; он принимает входные данные и ничего с ними не делает. Чтобы подавить сообщения об ошибках команды, выполните следующее:

```
[me@linuxbox ~]$ ls -l /bin/usr 2> /dev/null
```

/dev/null В культуре Unix

Bit bucket — это древняя концепция Unix, и благодаря своей универсальности она встречается во многих аспектах культуры Unix. Теперь вы знаете, что значит фраза «я отправляю ваши комментарии в /dev/null». Для получения дополнительных примеров см. статью в Wikipedia о /dev/null.

/dev/null

.

Перенаправление стандартного потока ввода

До сих пор мы не встречали команд, использующих стандартный ввод (на самом деле встречали, но об этом сюрпризе мы расскажем чуть позже), поэтому нам нужно ввести одну такую команду.

cat – Объединение файлов

Команда `cat` считывает один или несколько файлов и копирует их в стандартный вывод следующим образом:

```
cat [file...]
```

В большинстве случаев `cat` можно считать аналогом команды `TYPE` в DOS.

Его можно использовать для вывода файлов без постраничного просмотра. Например, следующая команда выведет содержимое файла `ls-output.txt`:

```
[me@linuxbox ~]$ cat ls-output.txt
```

cat часто используют для вывода коротких текстовых файлов. Поскольку cat может принимать несколько файлов в качестве аргументов, его также можно использовать для объединения файлов. Допустим, мы скачали большой файл, разделенный на несколько частей (мультимедийные файлы часто так делят в Usenet), и хотим собрать их воедино. Если файлы называются: movie.mpeg.001 movie.mpeg.002... movie.mpeg.099 мы можем объединить их следующей командой:

```
cat movie.mpeg.0* > movie.mpeg
```

Так как подстановочные знаки всегда раскрываются в отсортированном порядке, аргументы будут расположены правильно.

Это всё хорошо, но какое это имеет отношение к стандартному вводу? Пока никакого, но давайте попробуем другое. Что произойдет, если ввести cat без аргументов?

```
[me@linuxbox ~]$ cat
```

Ничего не произойдет, программа просто застынет, как будто зависла. Может показаться, что так оно и есть, но на самом деле она делает именно то, что должна.

Если cat не принимает аргументов, она читает данные из стандартного ввода. Поскольку стандартный ввод по умолчанию привязан к клавиатуре, программа будет ждать ввода! Попробуйте ввести следующий текст и нажать ENTER:

```
[me@linuxbox ~]$ cat
The quick brown fox jumps over the lazy dog.
```

Затем введите Ctrl-d (то есть зажмите Ctrl и нажмите «d»), чтобы сообщить cat, что в стандартном вводе достигнут конец файла (EOF):

```
[me@linuxbox ~]$ cat
The quick brown fox jumps over the lazy dog.
The quick brown fox jumps over the lazy dog.
```

Если аргументы в виде имен файлов отсутствуют, cat копирует стандартный ввод в стандартный вывод, поэтому мы видим повторение нашей строки. Это поведение можно использовать для создания коротких текстовых файлов. Допустим, мы хотим создать файл lazy_dog.txt, содержащий текст из нашего примера. Мы сделаем это так:


```
[me@linuxbox ~]$ cat > lazy_dog.txt
The quick brown fox jumps over the lazy dog.
```

Введите команду, а затем текст, который мы хотим поместить в файл. Не забудьте ввести `Ctrl-d` в конце. С помощью командной строки мы создали самый глупый в мире текстовый процессор! Чтобы увидеть результат, мы можем снова использовать `cat`, чтобы скопировать файл в `stdout`.

```
[me@linuxbox ~]$ cat lazy_dog.txt
The quick brown fox jumps over the lazy dog.
```

Теперь, когда мы знаем, что `cat` принимает стандартный ввод в дополнение к именам файлов, давайте попробуем перенаправление стандартного ввода.

```
[me@linuxbox ~]$ cat < lazy_dog.txt
The quick brown fox jumps over the lazy dog.
```

Используя оператор перенаправления `<`, мы меняем источник стандартного ввода с клавиатуры на файл `lazy_dog.txt`. Мы видим, что результат такой же, как при передаче одного имени файла в качестве аргумента. Это не особенно полезно по сравнению с передачей имени файла, но служит демонстрацией использования файла в качестве источника стандартного ввода. Другие команды используют стандартный ввод эффективнее, что мы скоро увидим.

Прежде чем продолжить, изучите `man`-страницу для `cat`, так как у неё есть несколько интересных опций.

Конвейеры

Возможность команд читать данные из стандартного ввода и отправлять их в стандартный вывод используется в оболочке через механизм, называемый конвейерами. С помощью оператора `|` (вертикальная черта) стандартный вывод одной команды можно передать на стандартный ввод другой.

```
command1 | command2
```

Чтобы продемонстрировать это, нам понадобятся команды. Помните, мы говорили о команде, которая уже принимает стандартный ввод? Это `less`. Мы можем использовать `less`, чтобы постранично отображать вывод любой команды, направляющей результаты в стандартный вывод:

```
[me@linuxbox ~]$ ls -l /usr/bin | less
```

Это чрезвычайно удобно! С помощью этого метода можно легко просматривать вывод любой команды, работающей со стандартным выводом.

Разница между > и |

На первый взгляд может быть трудно понять разницу между перенаправлением через оператор конвейера | и оператор перенаправления >. Проще говоря, оператор перенаправления связывает команду с файлом, а оператор конвейера связывает вывод одной команды со вводом другой.

```
command1 > file1
```

```
command1 | command2
```

Изучая конвейеры, многие пробуют следующее, «просто чтобы посмотреть, что будет»:

```
command1 > command2
```

Ответ: иногда случается что-то действительно плохое.

Вот реальный пример от читателя, который администрировал серверное устройство на базе Linux. Как суперпользователь, он сделал следующее:

```
# cd /usr/bin
```

```
# ls > less
```

Первая команда поместила его в каталог, где хранится большинство программ, а вторая приказала оболочке перезаписать файл less результатом выполнения команды ls. Поскольку в каталоге /usr/bin уже находился файл с именем less (программа less), вторая команда перезаписала файл программы less текстом из ls, тем самым уничтожив программу less в его системе.

Урок здесь в том, что оператор перенаправления > незаметно создает или перезаписывает файлы, поэтому к нему нужно относиться с большой осторожностью.

Фильтры

Конвейеры часто используют для выполнения сложных операций с данными. Можно объединить несколько команд в один конвейер. Часто такие команды называют фильтрами. Фильтры принимают входные данные, как-то их изменяют и выводят результат. Первым мы попробуем sort. Представьте, что мы хотим составить общий список всех исполняемых программ в

/bin и /usr/bin, отсортировать их и просмотреть полученный список:

```
[me@linuxbox ~]$ ls /bin /usr/bin | sort | less
```

Так как мы указали два каталога (/bin и /usr/bin), вывод `ls` состоял бы из двух отсортированных списков — по одному для каждого каталога. Добавив `sort` в наш конвейер, мы изменили данные так, чтобы получить один общий отсортированный список.

`sort` — это мощная команда со множеством функций и опций. Мы подробно разберем их в главе 20.

`uniq` — вывод или пропуск повторяющихся строк

Команду `uniq` часто используют вместе с `sort`. `uniq` принимает отсортированный список данных из стандартного ввода или из одного аргумента — имени файла (подробности см. в `uniq man page`) и по умолчанию удаляет из списка все дубликаты. Таким образом, чтобы убедиться, что в нашем списке нет дубликатов (то есть программ с одинаковыми именами, присутствующих и в каталоге /bin, и в каталоге /usr/bin), мы добавим `uniq` в наш конвейер.

```
[me@linuxbox ~]$ ls /bin /usr/bin | sort | uniq | less
```

В этом примере мы используем `uniq`, чтобы удалить дубликаты из вывода команды `sort`. Если же мы хотим увидеть сам список дубликатов, мы добавляем к `uniq` опцию «-d» следующим образом:

```
[me@linuxbox ~]$ ls /bin /usr/bin | sort | uniq -d | less
```

`wc` — Подсчет строк, слов и байтов

Команда `wc` (подсчет слов) используется для вывода количества строк, слов и байт в файлах. Вот пример:

```
[me@linuxbox ~]$ wc ls-output.txt
7902  64566 503634 ls-output.txt
```

В этом случае команда выводит три числа: количество строк, слов и байтов в `ls-output.txt`. Как и предыдущие команды, если `wc` запустить без аргументов командной строки, она принимает стандартный ввод. Опция «-l» ограничивает вывод только подсчетом строк. Использование этой команды в конвейере — удобный способ подсчета объектов. Чтобы узнать

количество элементов в отсортированном списке, выполните следующее:

```
[me@linuxbox ~]$ ls /bin /usr/bin | sort | uniq | wc -l
2728
```

grep — Вывод строк, соответствующих шаблону

grep — это мощная программа для поиска текстовых шаблонов в файлах. Она используется следующим образом:

```
grep pattern [file...]
```

Когда grep находит в файле «шаблон», он выводит строки, содержащие его. Шаблоны, которые может искать grep, могут быть очень сложными, но пока мы сосредоточимся на простом текстовом поиске. Продвинутое шаблоны, называемые регулярными выражениями, мы разберем в главе 19.

Допустим, мы хотим найти в списке программ все файлы, в названии которых содержится слово zip. Такой поиск поможет нам понять, какие программы в системе связаны со сжатием файлов. Мы сделаем следующее:

```
[me@linuxbox ~]$ ls /bin /usr/bin | sort | uniq | grep zip
bunzip2
bzip2
gunzip
gzip
unzip
zip
zipcloak
zipgrep
zipinfo
zipnote
zipsplit
```

Вот несколько полезных опций для grep:

- -i заставляет grep игнорировать регистр при поиске (обычно поиск чувствителен к регистру)
- -l заставляет grep выводить только имена файлов, содержащих текст, соответствующий шаблону.
- -v заставляет grep выводить только те строки, которые не соответствуют шаблону.
- -w заставляет grep искать только целые слова.

head / tail — вывод начала и конца файлов

Иногда нам не нужен весь вывод команды. Возможно, нам нужны только первые или последние несколько строк. Команда head выводит первые 10 строк файла, а команда tail —

последние 10 строк. Хотя по умолчанию обе команды выводят по 10 строк, это можно изменить с помощью опции -n.

```
[me@linuxbox ~]$ head -n 5 ls-output.txt
total 343496
-rwxr-xr-x 1 root root      31316 2025-12-05 08:58 [
-rwxr-xr-x 1 root root      8240 2025-12-09 13:39 411toppm
-rwxr-xr-x 1 root root     111276 2025-11-26 14:27 a2p
-rwxr-xr-x 1 root root     25368 2024-10-06 20:16 a52dec
[me@linuxbox ~]$ tail -n 5 ls-output.txt
-rwxr-xr-x 1 root root      5234 2025-06-27 10:56 znew
-rwxr-xr-x 1 root root      691 2023-09-10 04:21 zonetab2pot.py
-rw-r--r-- 1 root root      930 2025-11-01 12:23 zonetab2pot.pyc
-rw-r--r-- 1 root root      930 2025-11-01 12:23 zonetab2pot.pyo
lrwxrwxrwx 1 root root        6 2024-01-31 05:22 zsoelim -> soelim
```

Эти команды также можно использовать в конвейерах:

```
[me@linuxbox ~]$ ls /usr/bin | tail -n 5
znew
zonetab2pot.py
zonetab2pot.pyc
zonetab2pot.pyo
zsoelim
```

Использование опции -n вместе с head и tail позволяет вырезать фрагмент из середины файла. Представим, что у нас есть текстовый файл с пятистрочным заголовком и пятистрочным подвалом, которые мы хотим удалить, оставив только «полезную» часть с данными в середине. Мы можем сделать так:

```
[me@linuxbox ~]$ head -n -5 text_header_footer.txt | tail -n +6 >
text.txt
```

Опция -n при использовании с head допускает отрицательное значение, что заставляет выводить все строки, кроме последних n. Аналогично, опция -n с tail допускает знак плюс, что заставляет выводить все строки, начиная с n-й строки.

tail также имеет опцию, позволяющую отслеживать содержимое файла в реальном времени. Это полезно для наблюдения за процессом записи лог-файлов. В следующем примере мы рассмотрим файл messages в /var/log (или файл /var/log/syslog messages, если первый отсутствует). В некоторых дистрибутивах Linux для этого могут потребоваться привилегии суперпользователя, так как лог-файлы могут содержать информацию о безопасности:

```
[me@linuxbox ~]$ tail -f /var/log/messages
Feb  8 13:40:05 twin4 dhclient: DHCPACK from 192.168.1.1
Feb  8 13:40:05 twin4 dhclient: bound to 192.168.1.4 -- renewal in
1652 seconds.
Feb  8 13:55:32 twin4 mountd[3953]: /var/NFSv4/musicbox exported to
both 192.168.1.0/24 and twin7.localdomain in
```

```
192.168.1.0/24,twin7.localdomain
Feb  8 14:07:37 twin4 dhclient: DHCPREQUEST on eth0 to 192.168.1.1
port 67
Feb  8 14:07:37 twin4 dhclient: DHCPACK from 192.168.1.1
Feb  8 14:07:37 twin4 dhclient: bound to 192.168.1.4 -- renewal in
1771 seconds.
Feb  8 14:09:56 twin4 smartd[3468]: Device: /dev/hda, SMART
Prefailure Attribute: 8 Seek_Time_Performance changed from 237 to 236
Feb  8 14:10:37 twin4 mountd[3953]: /var/NFSv4/musicbox exported to
both 192.168.1.0/24 and twin7.localdomain in
192.168.1.0/24,twin7.localdomain
Feb  8 14:25:07 twin4 sshd(pam_unix)[29234]: session opened for user
me by (uid=0)
Feb  8 14:25:36 twin4 su(pam_unix)[29279]: session opened for user
root by me(uid=500)
```

С опцией `-f` `tail` продолжает отслеживать файл, и новые строки сразу же появляются на экране. Это продолжается до тех пор, пока мы не нажмем `Ctrl-c`.

`tee` — чтение из `stdin` и вывод в `stdout` и файлы

Следуя нашей метафоре с водопроводом, Linux предоставляет команду `tee`, которая создает тройник на нашей трубе.

Программа `tee` читает стандартный ввод и копирует его как в стандартный вывод (позволяя данным идти дальше по конвейеру), так и в один или несколько файлов. Это полезно для захвата содержимого конвейера на промежуточном этапе обработки. Здесь мы повторяем один из предыдущих примеров, на этот раз добавляя `tee`, чтобы сохранить весь список каталога в файл `ls.txt`, прежде чем `grep` отфильтрует содержимое конвейера:

```
[me@linuxbox ~]$ ls /usr/bin | tee ls.txt | grep zip
bunzip2
bzip2
gunzip
gzip
unzip
zip
zipcloak
zipgrep
zipinfo
zipnote
zipsplit
```

Итоги

Как и всегда, изучите документацию к каждой команде из этого раздела. Мы рассмотрели лишь основы, но у всех них есть множество интересных опций. По мере освоения Linux вы увидите, насколько полезно перенаправление в командной строке для решения специфических задач. Многие команды используют стандартный ввод и вывод, а почти все программы в

командной строке используют стандартный поток ошибок для вывода информационных сообщений.

Linux — это воображение

Когда меня просят объяснить разницу между Windows и Linux, я часто привожу аналогию с игрушками.

Windows похожа на Game Boy. Вы идете в магазин и покупаете новенькую приставку в коробке. Приносите домой, включаете и играете. Красивая графика, милые звуки. Но через некоторое время игры, идущей в комплекте, вам надоедает, и вы снова идете в магазин за новой. Этот цикл повторяется снова и снова. Наконец, вы приходите в магазин и говорите продавцу: «Я хочу игру, которая делает вот это!», но вам отвечают, что такой игры не существует, так как на нее нет «рыночного спроса». Вы говорите: «Но мне нужно изменить всего одну вещь!». Продавец отвечает, что это невозможно. Все игры запечатаны в картриджах. Вы понимаете, что ваша игрушка ограничена лишь тем, что другие решили вам предложить.

Linux, напротив, подобен крупнейшему в мире конструктору. Вы открываете его, а там — огромная коллекция деталей. Стальные стержни, винты, гайки, шестерни, шкивы, моторы и несколько идей для сборки. Вы начинаете играть. Сначала собираете по инструкции, затем — что-то свое. Со временем у вас появляются собственные идеи. Вам больше не нужно ходить в магазин, ведь у вас уже есть всё необходимое. Конструктор принимает форму вашего воображения. Он делает то, что хотите вы.

Выбор инструментов — дело личное, так какой из них принесет вам больше удовлетворения?

7 — Мир глазами оболочки

В этой главе мы изучим часть «магии», происходящей в командной строке при нажатии клавиши ENTER. Мы рассмотрим несколько интересных и сложных возможностей оболочки, используя всего одну новую команду.

- echo — Вывод строки текста

Раскрытие подстановочных знаков

Каждый раз, когда мы вводим команду и нажимаем клавишу ENTER, bash выполняет несколько замен в тексте перед тем, как выполнить нашу команду. Мы уже видели несколько примеров того, как простая последовательность символов, например *, может иметь большое значение для оболочки. Процесс, который это обеспечивает, называется раскрытием. Благодаря раскрытию мы вводим одно, а перед выполнением оболочка преобразует это в другое. Чтобы продемонстрировать это, рассмотрим команду echo. echo — это встроенная команда оболочки, выполняющая простую задачу: она выводит переданные ей текстовые аргументы в стандартный вывод.

```
[me@linuxbox ~]$ echo this is a test  
this is a test
```

Все довольно просто. Любой аргумент, переданный echo, будет отображен. Давайте попробуем другой пример.

```
[me@linuxbox ~]$ echo *  
Desktop Documents ls-output.txt Music Pictures Public Templates  
Videos
```

Что же произошло? Почему echo не вывел *? Как мы помним из работы с подстановочными знаками, символ * означает соответствие любым символам в имени файла, но в нашем исходном обсуждении не говорилось о том, как именно это делает оболочка. Простой ответ: оболочка разворачивает * во что-то другое (в данном случае — в имена файлов в текущем рабочем каталоге) перед выполнением команды echo. Когда клавиша ENTER нажата, оболочка автоматически разворачивает любые подходящие символы в командной строке перед выполнением команды, поэтому команда echo никогда не видела *, а только её развернутый результат. Зная это, мы видим, что echo сработал как ожидалось.

Раскрытие путей

Механизм работы подстановочных знаков называется раскрытием путей. Если применить техники из предыдущих глав, станет ясно, что они по сути являются раскрытиями. Допустим, домашний каталог выглядит так:

```
[me@linuxbox ~]$ ls
Desktop  ls-output.txt  Pictures  Templates
Documents Music          Public    Videos
```

мы могли бы выполнить следующие раскрытия:

```
[me@linuxbox ~]$ echo D*
Desktop Documents
```

и это:

```
[me@linuxbox ~]$ echo *s
Documents Pictures Templates Videos
```

или даже это:

```
[me@linuxbox ~]$ echo [[:upper:]]*
Desktop Documents Music Pictures Public Templates Videos
```

и, выходя за пределы нашего домашнего каталога, мы могли бы сделать так:

```
[me@linuxbox ~]$ echo /usr/*/share
/usr/kerberos/share /usr/local/share
```

Раскрытие пути для скрытых файлов

Как известно, имена файлов, начинающиеся с точки, являются скрытыми. Раскрытие пути также учитывает это поведение. Такое раскрытие, как в примере ниже, не показывает скрытые файлы.

```
echo *
```

На первый взгляд может показаться, что скрытые файлы можно включить в раскрытие, начав шаблон с точки:

```
echo .*
```

Это работает, если установлена опция оболочки `globskipdots` (в `bash` версии 5.2 и выше она включена по умолчанию). Однако, если она не установлена, в результатах появятся имена. и.. Поскольку эти имена относятся к текущему рабочему каталогу и его родительскому каталогу, использование такого шаблона, скорее всего, приведет к

неверному результату. Это можно увидеть, выполнив следующую команду:

```
ls -d .* | less
```

Чтобы лучше выполнять раскрытие пути в такой ситуации, необходимо использовать более специфичный шаблон.

```
echo .[!..]*
```

Этот шаблон раскрывается в каждое имя файла, которое начинается ровно с одной точки, за которой следуют любые другие символы. Это будет работать правильно для большинства скрытых файлов (хотя файлы с несколькими начальными точками все равно не будут включены).

Команда `ls` с опцией `-A` («почти все») обеспечит правильный список скрытых файлов.

```
ls -A
```

Раскрытие тильды

Как мы уже упоминали в введении к команде `cd`, символ тильды (`~`) имеет особое значение. Если он стоит в начале слова, то разворачивается в имя домашнего каталога указанного пользователя или, если пользователь не указан, в домашний каталог текущего пользователя.

```
[me@linuxbox ~]$ echo ~  
/home/me
```

Если у пользователя «bob» есть учетная запись, то это развернется так:

```
[me@linuxbox ~]$ echo ~bob  
/home/bob
```

Арифметическое раскрытие

Оболочка позволяет выполнять арифметические операции с помощью раскрытия. Это позволяет использовать приглашение оболочки как калькулятор.

```
[me@linuxbox ~]$ echo $((2 + 2))  
4
```

Арифметическое раскрытие имеет следующий вид:

`$((expression))`

где `expression` — это арифметическое выражение, состоящее из значений и арифметических операторов.

Арифметическое раскрытие поддерживает только целые числа (без десятичных дробей), но позволяет выполнять множество различных операций. В таблице 7-1 описаны некоторые из поддерживаемых операторов.

Таблица 7-1: Арифметические операторы степень

Оператор	Описание
+	Сложение
-	Вычитание
*	Умножение
/	Деление (но помните, что так как раскрытие поддерживает только целые вычисления, результаты будут целыми числами)
%	Остаток от деления, что означает «остаток»
**	Возведение в

В арифметических выражениях пробелы не имеют значения, а сами выражения могут быть вложенными. Например, чтобы умножить 5 в квадрате на 3, можно использовать следующее:

```
[me@linuxbox ~]$ echo $((5**2) * 3)
75
```

Одинарные скобки позволяют группировать подвыражения. С их помощью можно переписать предыдущий пример и получить тот же результат с помощью одного раскрытия вместо двух.

```
[me@linuxbox ~]$ echo $((5**2) * 3)
75
```

Вот пример с использованием операторов деления и остатка. Обратите внимание на эффект целочисленного деления.

```
[me@linuxbox ~]$ echo Five divided by two equals $((5/2))
Five divided by two equals 2
[me@linuxbox ~]$ echo with $((5%2)) left over.
with 1 left over.
```

Арифметическое раскрытие более подробно описано в главе 34.

Раскрытие фигурных скобок

Пожалуй, самым необычным видом является раскрытие фигурных скобок. С его помощью можно создавать несколько строк текста из шаблона, содержащего скобки. Вот пример:

```
[me@linuxbox ~]$ echo Front-{A,B,C}-Back
Front-A-Back Front-B-Back Front-C-Back
```

Шаблоны для раскрытия скобок могут содержать начальную часть (преамбулу) и завершающую часть (постскрипtum). Само выражение в скобках может содержать либо список строк через запятую, либо диапазон целых чисел или одиночных символов. Шаблон не должен содержать неэкранированных пробелов. Вот пример с использованием диапазона целых чисел:

```
[me@linuxbox ~]$ echo Number_{1..5}
Number_1 Number_2 Number_3 Number_4 Number_5
```

В bash версии 4.0 и новее целые числа можно дополнять нулями, как показано ниже:

```
[me@linuxbox ~]$ echo {01..15}
01 02 03 04 05 06 07 08 09 10 11 12 13 14 15
[me@linuxbox ~]$ echo {001..15}
001 002 003 004 005 006 007 008 009 010 011 012 013 014 015
```

Вот диапазон букв в обратном порядке:

```
[me@linuxbox ~]$ echo {Z..A}
Z Y X W V U T S R Q P O N M L K J I H G F E D C B A
```

Раскрытие фигурных скобок может быть вложенным.

```
[me@linuxbox ~]$ echo a{A{1,2},B{3,4}}b
aA1b aA2b aB3b aB4b
```

Для чего это нужно? Чаще всего это применяют для создания списков файлов или каталогов. Например, если бы мы были фотографами с большой коллекцией снимков, которые нужно разложить по годам и месяцам, первым делом мы могли бы создать серию каталогов в числовом формате «Год-Месяц». Так названия каталогов будут сортироваться в хронологическом порядке. Можно было бы вводить список каталогов вручную, но это трудоемко и чревато ошибками. Вместо этого можно сделать так:

```
[me@linuxbox ~]$ mkdir Photos
[me@linuxbox ~]$ cd Photos
[me@linuxbox Photos]$ mkdir {2007..2009}-{01..12}
[me@linuxbox Photos]$ ls
2007-01 2007-07 2008-01 2008-07 2009-01 2009-07
2007-02 2007-08 2008-02 2008-08 2009-02 2009-08
2007-03 2007-09 2008-03 2008-09 2009-03 2009-09
2007-04 2007-10 2008-04 2008-10 2009-04 2009-10
2007-05 2007-11 2008-05 2008-11 2009-05 2009-11
2007-06 2007-12 2008-06 2008-12 2009-06 2009-12
```

Довольно изящно!

Раскрытие параметров

В этой главе мы лишь вкратце коснемся раскрытия параметров, но позже разберем эту тему подробно. Эта функция полезнее в сценариях оболочки, чем при прямом использовании в командной строке. Многие возможности связаны со способностью системы хранить небольшие фрагменты данных и присваивать каждому из них имя. Нам доступны для изучения многие такие фрагменты, которые правильнее называть переменными. Например, переменная `USER` содержит наше имя пользователя. Чтобы вызвать раскрытие параметров и показать содержимое `USER`, нужно сделать следующее:

```
[me@linuxbox ~]$ echo $USER  
me
```

Чтобы увидеть список доступных переменных, попробуйте следующее:

```
[me@linuxbox ~]$ printenv | less
```

Вы могли заметить, что при других типах раскрытия ошибка в шаблоне приводит к тому, что раскрытие не происходит, и `echo` просто выводит ошибочный шаблон. При раскрытии параметров ошибка в имени переменной не мешает раскрытию, но результатом становится пустая строка:

```
[me@linuxbox ~]$ echo $$UER  
[me@linuxbox ~]$
```

Подстановка команды

СоПодстановка команды позволяет использовать вывод команды в качестве раскрытия.

```
[me@linuxbox ~]$ echo $(ls)  
Desktop Documents ls-output.txt Music Pictures Public Templates  
Videos
```

Один из моих любимых приемов выглядит примерно так:

```
[me@linuxbox ~]$ ls -l $(which cp)  
-rwxr-xr-x 1 root root 71516 2025-12-05 08:58 /bin/cp
```

Здесь мы передали результаты `which cp` в качестве аргумента команде `ls`, тем самым получая список `cp` без необходимости знать его полный абсолютный путь. Мы не ограничены простыми командами. Можно использовать целые конвейеры (здесь показан лишь частичный вывод):

```
[me@linuxbox ~]$ file $(ls -d /usr/bin/* | grep zip)
/usr/bin/bunzip2:      symbolic link to `bzip2'
/usr/bin/bzip2:        ELF 32-bit LSB executable, Intel 80386,
version 1 (SYSV), dynamically linked (uses shared libs), for
GNU/Linux 2.6.9, stripped
/usr/bin/bzip2recover: ELF 32-bit LSB executable, Intel 80386,
version 1 (SYSV), dynamically linked (uses shared libs), for
GNU/Linux 2.6.9, stripped
/usr/bin/funzip:       ELF 32-bit LSB executable, Intel 80386,
version 1 (SYSV), dynamically linked (uses shared libs), for
GNU/Linux 2.6.9, stripped
/usr/bin/gpg-zip:      Bourne shell script text executable
/usr/bin/gunzip:       symbolic link to `../../bin/gunzip'
/usr/bin/gzip:         symbolic link to `../../bin/gzip'
/usr/bin/mzip:         symbolic link to `mtools'
```

В этом примере результаты конвейера стали списком аргументов для команды `file` com- mand.

Существует альтернативный синтаксис подстановки команд, используемый старыми программами оболочки, который также поддерживается в `bash`. Вместо знака доллара и скобок в нем используются обратные кавычки.

```
[me@linuxbox ~]$ ls -l `which cp`
-rwxr-xr-x 1 root root 71516 2025-12-05 08:58 /bin/cp
```

Кавычки

Теперь, когда мы увидели, сколькими способами оболочка может выполнять раскрытие, пора научиться управлять этим процессом. Рассмотрим, например, следующее:

```
[me@linuxbox ~]$ echo this is a      test
this is a test
```

или ЭТОТ:

```
[me@linuxbox ~]$ echo The total is $100.00
The total is 00.00
```

В первом примере разделение слов оболочкой удалило лишние пробелы из списка аргументов `echo` command. Во втором примере раскрытие параметра заменило значение `$1` пустой строкой, так как переменная не была определена. Оболочка предоставляет механизм под названием «кавычки» для выборочного подавления нежелательного раскрытия.

Двойные кавычки

Первый тип кавычек, который мы рассмотрим, — двойные кавычки. Если поместить текст в двойные кавычки, все специальные символы оболочки теряют свое особое значение и обрабатываются как обычные символы. Исключения составляют

\$, \ (обратный слэш) и ` (обратная кавычка). Это означает, что разделение слов, раскрытие пути, раскрытие тильды и раскрытие фигурных скобок подавляются, но раскрытие параметра, арифметическое раскрытие и подстановка команды по-прежнему выполняются. С помощью двойных кавычек мы можем работать с именами файлов, содержащими пробелы. Допустим, нам попался файл с именем `two words.txt`. Если попытаться использовать его в командной строке, разделение слов приведет к тому, что он будет воспринят как два отдельных аргумента, а не как один нужный нам аргумент.

```
[me@linuxbox ~]$ ls -l two words.txt
ls: cannot access two: No such file or directory
ls: cannot access words.txt: No such file or directory
```

Используя двойные кавычки, мы предотвращаем разделение слов и получаем нужный результат; более того, мы можем даже исправить ошибки.

```
[me@linuxbox ~]$ ls -l "two words.txt"
-rw-rw-r-- 1 me me 18 2016-02-20 13:03 two words.txt
[me@linuxbox ~]$ mv "two words.txt" two_words.txt
```

Готово! Теперь нам не нужно постоянно вводить эти надоедливые двойные кавычки.

Помните, что раскрытие параметров, арифметическое раскрытие и подстановка команд по-прежнему работают внутри двойных кавычек.

```
[me@linuxbox ~]$ echo "$USER $((2+2)) $(df -h)"
me 4 Filesystem      Size  Used Avail Use% Mounted on
tmpfs                1.6G  2.0M  1.6G   1% /run
/dev/sda2            94G   19G   71G  21% /
tmpfs                7.8G   0    7.8G   0% /dev/shm
tmpfs                5.0M  4.0K  5.0M   1% /run/lock
/dev/sda1            975M  6.1M  969M   1% /boot/efi
/dev/sdb1            907G  574G  287G  67% /home
tmpfs                1.6G  1.8M  1.6G   1% /run/user/1000
```

Стоит рассмотреть влияние двойных кавычек на подстановку команд. Сначала разберем подробнее механизм разделения слов. В предыдущем примере мы видели, как разделение слов удаляет лишние пробелы в тексте.

```
[me@linuxbox ~]$ echo this is a      test
this is a test
```

По умолчанию при разделении слов ищутся пробелы, табуляция и символы новой строки (linefeed), которые считаются разделителями. Это значит, что не заключенные в кавычки пробелы, табуляция и переносы строк не считаются частью

текста, а служат лишь разделителями. Поскольку они разделяют слова на разные аргументы, наша командная строка содержит команду и четыре отдельных аргумента. Если добавить двойные кавычки:

```
[me@linuxbox ~]$ echo "this is a      test"
this is a      test
```

разделение слов прекращается, и встроенные пробелы перестают быть разделителями, становясь частью аргумента. После добавления двойных кавычек наша командная строка содержит команду и один аргумент.

Тот факт, что механизм разделения слов считает символы новой строки разделителями, вызывает интересный, хотя и тонкий эффект при подстановке команд. Рассмотрим следующий пример:

```
[me@linuxbox ~]$ echo $(df -h)
Filesystem Size Used Avail Use% Mounted on tmpfs 1.6G 2.0M 1.6G 1%
/run /dev/sda2 94G 19G 71G 21% / tmpfs 7.8G 0 7.8G 0% /dev/shm tmpfs
5.0M 4.0K 5.0M 1% /run/lock /dev/sda1 975M 6.1M 969M 1% /boot/efi
/dev/sdb1 907G 574G 287G 67% /home tmpfs 1.6G 1.8M 1.6G 1%
/run/user/1000
[me@linuxbox ~]$ echo "$(df -h)"
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           1.6G   2.0M   1.6G   1% /run
/dev/sda2       94G    19G    71G   21% /
tmpfs           7.8G     0   7.8G    0% /dev/shm
tmpfs           5.0M   4.0K   5.0M    1% /run/lock
/dev/sda1       975M   6.1M   969M    1% /boot/efi
/dev/sdb1      907G  574G   287G   67% /home
tmpfs           1.6G   1.8M   1.6G    1% /run/user/1000
```

В первом случае подстановка команды без кавычек привела к тому, что командная строка содержала 49 аргументов. Во втором случае получилась командная строка с одним аргументом, включающим встроенные пробелы и символы новой строки.

Одинарные кавычки

Чтобы подавить все раскрытия, мы используем одинарные кавычки. Вот сравнение без кавычек, с двойными и с одинарными кавычками:

```
[me@linuxbox ~]$ echo text ~/.txt {a,b} $(echo foo) $((2+2)) $USER
text /home/me/ls-output.txt a b foo 4 me
[me@linuxbox ~]$ echo "text ~/.txt {a,b} $(echo foo) $((2+2)) $USER"
text ~/.txt {a,b} foo 4 me
[me@linuxbox ~]$ echo 'text ~/.txt {a,b} $(echo foo) $((2+2)) $USER'
text ~/.txt {a,b} $(echo foo) $((2+2)) $USER
```

Как видно, с каждым последующим уровнем кавычек подавляется всё больше раскрытий.

Экранирование символов

Иногда нужно взять в кавычки только один символ. Для этого перед символом можно поставить обратный слэш, который в данном контексте называется экранирующим символом. Часто это делают внутри двойных кавычек, чтобы выборочно предотвратить раскрытие.

```
[me@linuxbox ~]$ echo "The balance for user $USER is: \$5.00"
The balance for user me is: $5.00
```

Также часто используют экранирование, чтобы убрать специальное значение символа в имени файла. Например, в именах файлов можно использовать символы, которые обычно имеют специальное значение для оболочки. К ним относятся \$, !, &, пробелы и другие. Чтобы включить спецсимвол в имя файла, можно сделать так:

```
[me@linuxbox ~]$ mv bad\&filename good_filename
```

Чтобы использовать символ обратной косой черты, экранируйте его, введя \\. Примечание: внутри одинарных кавычек обратная косая черта теряет свое специальное значение и воспринимается как обычный символ.

Еще одно применение обратного слеша — подавление псевдонимов. Например, если для команды `ls` установлен псевдоним `ls='ls --color=auto'` (что часто встречается в дистрибутивах Linux), мы можем поставить перед командой обратный слеш: тогда псевдоним будет проигнорирован, а команда `ls` выполнится без опции цвета.

Последовательности с обратным слешем

Помимо роли символа экранирования, обратный слеш используется в обозначениях определенных специальных символов, называемых управляющими кодами. Первые 32 символа в кодировке ASCII служат для передачи команд телетайпным устройствам, таким как наш терминал. Некоторые из этих кодов знакомы (табуляция, `backspace`, перевод строки и возврат каретки), другие — нет (`null`, конец передачи и подтверждение).

Последовательность экранирования

Значение

\a Звонок (сигнал, заставляющий компьютер издать звуковой сигнал)

`\b` Backspace

`\n` Перевод строки. В Unix-подобных системах это создает перевод строки.

`\r` Возврат каретки

`\t` Tab

В таблице выше приведены некоторые распространенные последовательности с обратным слешем. Идея такого представления с использованием обратного слеша возникла в языке программирования C и была принята многими другими языками, включая оболочку.

Опция `-e` в `echo` включает интерпретацию escape-последовательностей. Их также можно помещать внутри `$'`. Используя команду `sleep` — простую программу, которая ждет указанное количество секунд и завершается, — можно создать примитивный таймер обратного отсчета:

```
sleep 10; echo -e "Time's up\a"
```

Мы также могли бы сделать так:

```
sleep 10; echo "Time's up" $'\a'
```

Итоги

По мере освоения оболочки вы заметите, что раскрытие подстановочных знаков и использование кавычек будут встречаться всё чаще, поэтому важно хорошо понимать принципы их работы. Можно даже сказать, что это важнейшие темы при изучении оболочки. Без понимания механизмов раскрытия оболочка всегда будет казаться чем-то загадочным и запутанным, а её огромный потенциал будет использоваться лишь частично.

Что почитать

- В странице ``bash man`` есть основные разделы, посвященные раскрытию и кавычкам, где эти темы разобраны более формально.
- Справочное руководство Bash также содержит главы о подстановках и кавычках:
<http://www.gnu.org/software/bash/manual/bashref.html>

8 — Продвинутые приемы работы с клавиатурой

Я часто в шутку называю Unix «операционной системой для любителей печатать». Само наличие командной строки — тому подтверждение. Но пользователи командной строки не так уж и любят печатать. Иначе зачем столь многим командам давать такие короткие имена, как `cp`, `ls`, `mv` и `rm`? На самом деле одна из главных целей командной строки — лень: выполнение максимального объема работы минимальным числом нажатий клавиш. Еще одна цель — никогда не отрывать руки от клавиатуры ради мыши. В этой главе мы рассмотрим `bash`-возможности, которые делают работу за клавиатурой быстрее и эффективнее.

Вам встретятся следующие команды:

- `clear` — Очистить экран
- `history` — Вывести список истории

Редактирование командной строки

`bash` использует библиотеку `Readline` (общий набор подпрограмм, доступных разным программам) для редактирования командной строки. Мы уже видели часть её возможностей. Например, мы знаем, что клавиши со стрелками перемещают курсор, но функций гораздо больше. Рассматривайте их как дополнительные инструменты для работы. Необязательно учить все, но многие из них весьма полезны. Выбирайте те, что вам нужны.

Примечание: некоторые из приведенных ниже комбинаций клавиш (особенно использующие `Alt` key) могут перехватываться графическим интерфейсом для других функций. В виртуальной консоли все комбинации должны работать корректно.

Перемещение курсора

В следующей таблице перечислены клавиши для перемещения курсора:

Таблица 8-1: Команды перемещения курсора

Key	Действие
-----	----------

Key	Действие
Ctrl-a	Переместить курсор в начало строки.
Ctrl-e	Переместить курсор в конец строки.
Ctrl-f	Переместить курсор на один символ вперед; то же самое, что и клавиша стрелка вправо.
Ctrl-b	Переместить курсор на один символ назад; то же самое, что и клавиша стрелка влево.
Alt-f	Переместить курсор на одно слово вперед.
Alt-b	Переместить курсор на одно слово назад.
Ctrl-l	Очистить экран и переместить курсор в верхний левый угол. Команда <code>clear</code> делает то же самое.

Изменение текста

Поскольку при вводе команд можно ошибиться, нам нужен эффективный способ их исправления. В таблице 8-2 описаны команды клавиатуры для редактирования символов в командной строке.

Таблица 8-2: Команды редактирования текста

Key	Действие
Ctrl-d	Удалить символ в позиции курсора.
Ctrl-t	Поменять местами (обменять) символ в позиции курсора с предыдущим символом.
Alt-t	Поменять местами слово в позиции курсора с предыдущим.
Alt-l	Перевести символы от позиции курсора до конца слова в нижний регистр.
Alt-u	Преобразует символы от текущего положения курсора до конца слова в верхний регистр.

Вырезание и вставка (убийство и вырывание) текста

В документации Readline для обозначения привычных нам операций вырезания и вставки используются термины `killing` и `yanking`. Вырезанные элементы сохраняются в буфере (временной области памяти), называемом `kill-ring`.

Таблица 8-3: Команды вырезания и вставки

Key	Действие
-----	----------

Key	Действие
Ctrl-k	Удалить текст от курсора до конца строки.
Ctrl-u	Удалить текст от курсора до начала строки
Alt-d	Удалить текст от курсора до конца текущего слова.
Alt-Backspace	Удалить текст от курсора до начала текущего слова. Если курсор в начале слова, удалить предыдущее word.
Ctrl-y	слово. Вставить текст из кольца удаления в позицию курсора.

Клавиша Meta

Если вы обратитесь к документации Readline, которую можно найти в разделе «READLINE» страницы `bash man`, вы встретите термин `meta key`. На современных клавиатурах он соответствует клавише `Alt`, но так было не всегда.

В былые времена (до ПК, но после Unix) компьютеры были не у всех. Вместо них использовали терминалы. Терминал представлял собой устройство связи с текстовым экраном и клавиатурой, содержащее ровно столько электроники, сколько нужно для отображения символов и перемещения курсора. Он подключался (обычно через последовательный кабель) к мощному компьютеру или к сети, объединяющей такие компьютеры. Существовало множество марок терминалов, и все они имели разные клавиатуры и возможности дисплея. Поскольку почти все они поддерживали ASCII, разработчики переносимого ПО ориентировались на этот минимальный стандарт. В Unix-системах предусмотрены сложные механизмы работы с терминалами и их функциями. Поскольку разработчики Readline не могли быть уверены в наличии выделенной управляющей клавиши, они придумали её и назвали `meta`. Хотя на современных клавиатурах роль `meta` выполняет `Alt`, в эмуляторе терминала (что всё ещё возможно в Linux!) вы можете нажать и отпустить `Esc`, чтобы добиться того же эффекта, что и при удержании `Alt`.

Автодополнение

Еще один способ, которым может помочь оболочка, — это автодополнение, срабатывающее при нажатии Tab во время ввода команды. Посмотрим, как это работает. Допустим, наш домашний каталог выглядит так:

```
[me@linuxbox ~]$ ls
Desktop  ls-output.txt  Pictures  Templates  Videos
Documents Music          Public
```

Попробуйте ввести следующее, но не нажимайте ENTER:

```
[me@linuxbox ~]$ ls l
```

Теперь нажмите Tab.

```
[me@linuxbox ~]$ ls ls-output.txt
```

Видите, как оболочка дополнила строку за нас? Попробуем еще раз. Снова не нажимайте EN-TER.

```
[me@linuxbox ~]$ ls D
```

Нажмите Tab.

```
[me@linuxbox ~]$ ls D
```

Никакого дополнения, просто пустота. Это произошло потому, что D соответствует более чем одной записи в каталоге. Чтобы автодополнение сработало, «подсказка», которую мы даем, должна быть однозначной. Если мы пойдем дальше, как в случае с

```
[me@linuxbox ~]$ ls Do
```

а затем нажмите Tab:

```
[me@linuxbox ~]$ ls Documents
```

автодополнение выполнено успешно.

Хотя в этом примере показано дополнение путей — наиболее частое использование, — дополнение также работает для переменных (если слово начинается с \$), имен пользователей (если слово начинается с ~), команд (если слово является первым в строке) и имен хостов (если слово начинается с @). Дополнение имен хостов работает только для имен, перечисленных в /etc/hosts.

Существует ряд последовательностей клавиш управления и meta, связанных с дополнением, как показано в таблице 8-4.

Таблица 8-4: Команды дополнения жно лзовать

Key	Действие
Alt-?	Вывести список возможных вариантов дополнения. В большинстве систем мо также сделать это, нажав Tab второй раз, что гораздо проще.
Alt-*	Вставить все возможные варианты дополнения. Это полезно, если нужно испо более одного совпадения.

Существует еще немало довольно редких вариантов. Список приведен в bash man-странице в разделе «READLINE».

Программируемое дополнение

В последних версиях bash есть функция под названием «программируемое дополнение». Оно позволяет вам (или, скорее, вашему поставщику дистрибутива) добавлять правила дополнения. Обычно это делается для поддержки конкретных приложений. Например, можно добавить дополнение списка опций команды или поиск определенных типов файлов, которые поддерживает приложение. В Ubuntu по умолчанию определен довольно большой набор правил. Программируемое дополнение реализовано с помощью функций оболочки — своего рода мини-скриптов, которые мы разберем в следующих главах. Если вам любопытно, попробуйте

```
set | less
```

и посмотрите, сможете ли вы их найти. Не все дистрибутивы включают их по умолчанию.

Использование истории

Как мы выяснили в главе 1, bash ведет историю введенных команд. Этот список хранится в нашем домашнем каталоге в файле `.bash_history`. Возможность работы с историей — полезный ресурс, позволяющий сократить объем ручного ввода, особенно при редактировании командной строки.

Поиск в истории

В любой момент мы можем просмотреть содержимое списка истории следующим образом:

```
[me@linuxbox ~]$ history | less
```

По умолчанию большинство современных дистрибутивов Linux настраивают `bash` так, чтобы хранить последние 1000 введенных команд. Как изменить это значение, мы увидим в главе 11. Допустим, мы хотим найти команды, которые использовали для вывода списка `/usr/bin`. Вот один из способов:

```
[me@linuxbox ~]$ history | grep /usr/bin
```

И предположим, что среди результатов мы получили строку с интересной командой, подобной этой:

```
88 ls -l /usr/bin > ls-output.txt
```

88 — это номер строки команды в списке истории. Мы можем использовать его немедленно с помощью другого типа подстановки, называемого расширением истории. Чтобы выполнить найденную строку, можно сделать так:

```
[me@linuxbox ~]$ !88
```

`bash` развернет `!88` в содержимое 88-й строки списка истории. Другие формы расширения истории мы рассмотрим в следующем разделе.

`bash` также позволяет осуществлять инкрементальный поиск по списку истории. Это значит, что мы можем приказать `bash` искать по истории по мере ввода символов, причём каждый новый символ будет уточнять поиск. Чтобы начать инкрементальный поиск, нажмите `Ctrl-r`, а затем введите искомый текст. Когда нужная команда найдется, можно нажать `ENTER`, чтобы выполнить её, или `Ctrl-j`, чтобы скопировать строку из истории в текущую командную строку. Чтобы найти следующее совпадение (двигаясь «вверх» по списку истории), нажмите `Ctrl-r` ещё раз. Чтобы выйти из режима поиска, нажмите `Ctrl-g` или `Ctrl-c`. Вот как это работает на практике:

```
[me@linuxbox ~]$
```

Сначала нажмите `Ctrl-r`.

```
(reverse-i-search)`':
```

Приглашение меняется, указывая на выполнение обратного инкрементального поиска. Он называется «обратным», так как поиск идет от «настоящего» к прошлому. Затем мы начинаем вводить текст для поиска. В данном примере это `/usr/bin`:

```
(reverse-i-search)`/usr/bin': ls -l /usr/bin > ls-output.txt
```


Поиск мгновенно выдает результат. Мы можем выполнить команду, нажав ENTER, или скопировать её в текущую командную строку для дальнейшего редактирования, нажав Ctrl-j. Давайте скопируем её. Нажмите Ctrl-j.

```
[me@linuxbox ~]$ ls -l /usr/bin > ls-output.txt
```

Приглашение оболочки возвращается, а командная строка загружается и готова к работе!

В таблице 8-5 перечислены некоторые сочетания клавиш для управления списком истории.

Таблица 8-5: Команды истории

Key	Действие
Ctrl-p	Переход к предыдущей записи истории. Это то же действие, что и стрелка вверх вверх.
Ctrl-n	Переход к следующей записи истории. Это то же действие, что и стрелка вниз вниз.
Alt-<	Переход к началу (верху) списка истории.
Alt->	Переход к концу (низу) списка истории, то есть к текущей командной строке.
Ctrl-r	Обратный инкрементальный поиск. Он выполняет поиск пошагово от текущей командной строки вверх по списку истории.
Alt-p	Обратный поиск, неинкрементальный. С этой клавишей введите строку поиска и нажмите ENTER, чтобы выполнить поиск.
Alt-n	Прямой поиск, неинкрементальный.
Ctrl-o	Выполнить текущий элемент списка истории и перейти к следующему одному. Это удобно, если мы пытаемся повторно выполнить последовательность команд из списка истории.

Расширение истории

Оболочка предлагает особый тип раскрытия элементов списка истории с помощью символа !. Мы уже видели, что после восклицательного знака можно указать число, чтобы вставить запись из истории. Другие возможности раскрытия описаны в таблице 8-6.

Таблица 8-6: Команды расширения истории

Последовательность	Действие
!!	Повторить последнюю команду. Вероятно, проще нажать стрелка вверх и ENTER.
!number	Повторить элемент истории под номером number.
!string	Повторить последний элемент истории, начинающийся на строку string.
!?string	Повторить последний элемент истории, содержащий строку string.

Будьте осторожны с формами !string и !?string, если не уверены в содержимом элементов истории. Эту проблему можно частично смягчить, добавив «:p» к расширению. Это заставит оболочку вывести результат расширения и поместить его в историю команд. Вот пример:

```
[me@linuxbox ~]$ !ls:p
ls -l /usr/bin > ls-output.txt
```

Теперь, когда команда была вызвана и помещена как последний элемент в историю команд, мы можем выполнить её с помощью Up-Arrow и ENTER или !! и ENTER.

Кстати, такие расширения истории, как !!, не записываются в историю команд, но записываются их результаты.

Механизм раскрытия истории обладает множеством других возможностей, но эта тема слишком сложна, и голова может взорваться, если продолжать. Раздел HISTORY EXPANSION в bash man page подробно описывает все детали. Не стесняйтесь изучать!

script

В дополнение к функции истории команд в bash, большинство дистрибутивов Linux включают программу script, которая позволяет записывать всю сессию оболочки и сохранять её в файл. Базовый синтаксис команды выглядит так:

```
script [file]
```

где file — имя файла для хранения записи. Если файл не указан, используется typescript. Полный список опций и возможностей программы см. в странице man script.

Итоги

В этой главе мы рассмотрели приемы работы с клавиатурой, которые предоставляет оболочка, чтобы облегчить труд опытных пользователей. Со временем, когда вы начнете чаще использовать командную строку, вы сможете вернуться к этой главе, чтобы освоить новые приемы. Пока считайте их необязательными, но полезными.

Что почитать

- В Википедии есть хорошая статья о компьютерных терминалах: http://en.wikipedia.org/wiki/Computer_terminal

9 — Права доступа

Операционные системы традиции Unix отличаются от систем традиции MS-DOS тем, что они являются не только многозадачными, но и многопользовательскими.

Что именно это значит? Это значит, что компьютером может одновременно пользоваться более одного человека. Хотя у типичного компьютера, скорее всего, будет только одна клавиатура и монитор, им всё равно могут пользоваться несколько пользователей. Например, если компьютер подключен к сети или Интернету, удаленные пользователи могут войти через `ssh` (`secure shell`) и управлять им. На са мом деле удаленные пользователи могут запускать графические приложения, и их вывод будет отображаться на удаленном дисплее.

Многопользовательская поддержка в Linux — это не недавняя «инновация», а особенность, глубоко заложенная в архитектуру операционной системы. Учитывая среду, в которой создавался Unix, это вполне логично. Много лет назад, до того как компьютеры стали «персональными», они были огромными, дорогими и централизованными. Типичная университетская компьютерная система, например, состояла из большого центрального компьютера в одном здании и терминалов по всему кампусу, каждый из которых был подключен к этому центральному компьютеру. Компьютер поддерживал множество пользователей одновременно.

Для практического применения потребовался метод защиты пользователей друг от друга. В конце концов, нельзя было допускать, чтобы действия одного пользователя приводили к сбою компьютера или позволяли ему вмешиваться в файлы другого пользователя.

Для современных пользователей персональных компьютеров эта многопользовательская возможность важна даже в однопользовательских системах, так как она позволяет работать безопаснее: изменения на системном уровне может вносить только отдельная административная учетная запись.

В этой главе мы рассмотрим этот важнейший аспект безопасности системы и изучим следующие команды:

- `id` — отображение идентификатора пользователя.
- `chmod` — изменение режима доступа к файлу.
- `umask` — установка прав доступа к файлам по умолчанию.

- su — запуск оболочки от имени другого пользователя.
- sudo — выполнение команды от имени другого пользователя.
- chown — Изменение владельца файла.
- chgrp — Изменение группы файла.
- addgroup — Добавление пользователя или группы в систему.
- usermod — Изменение учетной записи пользователя.
- passwd — Изменение пароля пользователя.

Пользователи, члены групп и все остальные

При изучении системы в главе 3 мы могли столкнуться с проблемой при попытке просмотреть такой файл, как `/etc/shadow`:

```
[me@linuxbox ~]$ file /etc/shadow
/etc/shadow: regular file, no read permission
[me@linuxbox ~]$ less /etc/shadow
/etc/shadow: Permission denied
```

Причина этого сообщения об ошибке в том, что у нас, как у обычных пользователей, нет прав на чтение этого файла.

В модели безопасности Unix пользователь может владеть файлами и каталогами. Если пользователь владеет файлом или каталогом, он контролирует доступ к ним. Пользователи, в свою очередь, могут входить в группу, состоящую из одного или нескольких человек, которым владельцы предоставляют доступ к файлам и каталогам. Помимо предоставления доступа группе, владелец может предоставить набор прав доступа всем остальным, которых называют «others» (иногда «world»). Чтобы получить информацию о своей личности, мы используем команду `id`.

```
[me@linuxbox ~]$ id
uid=500(me) gid=500(me) groups=500(me)
```

Посмотрим на вывод. При создании учетных записей пользователям присваивается числовой идентификатор (`uid`), который для удобства людей сопоставляется с именем пользователя. Пользователю также назначается основной идентификатор группы (`gid`), и он может входить в дополнительные группы. Предыдущий пример взят из системы Fedora. В других системах, например Ubuntu, вывод может немного отличаться:

```
[me@linuxbox ~]$ id
uid=1000(me) gid=1000(me)
```

```
groups=4(adm),20(dialout),24(cdrom),25(floppy),29(audio),30(dip),44(v  
ideo),46(plugdev),108(lpadmin),114(admin),1000(me)
```

Как видно, номера `uid` и `gid` различаются. Это связано с тем, что Fedora начинает нумерацию обычных учетных записей с 500, а Ubuntu — с 1000. Мы также видим, что пользователь Ubuntu входит во множество групп. Это обусловлено тем, как Ubuntu управляет привилегиями для системных устройств и служб.

Откуда берется эта информация? Как и многое другое в Linux, она содержится в нескольких текстовых файлах. Учетные записи пользователей определены в файле `/etc/passwd`, а группы — в файле `/etc/group`. При создании пользователей и групп эти файлы изменяются вместе с `/etc/shadow`, где хранится информация о пароле пользователя. Для каждой учетной записи файл `/etc/passwd` определяет имя пользователя (логин), `uid`, `gid`, реальное имя, домашний каталог и оболочку входа. Если изучить содержимое `/etc/passwd` и `/etc/group`, можно заметить, что помимо обычных пользователей там есть учетные записи суперпользователя (всегда `uid 0`) и различных системных пользователей.

В следующей главе, когда мы разберем процессы, мы увидим, что некоторые из этих «пользователей» на самом деле весьма заняты.

Хотя во многих Unix-подобных системах обычные пользователи назначаются в общую группу, например «users», в современном Linux принято создавать уникальную группу из одного участника с тем же именем, что и у пользователя. Это упрощает назначение определенных типов прав доступа.

Чтение, запись и выполнение

Права доступа к файлам и каталогам определяются как доступ на чтение, запись и выполнение. Если взглянуть на вывод команды `ls`, можно понять, как это реализовано:

```
[me@linuxbox ~]$ > foo.txt  
[me@linuxbox ~]$ ls -l foo.txt  
-rw-rw-r-- 1 me me 0 2016-03-06 14:52 foo.txt
```

Первые 10 символов списка — это атрибуты файла. Первый из них указывает тип файла. В таблице 9-1 описаны наиболее вероятные типы файлов (существуют и другие, менее распространенные):

Таблица 9-1: Типы файлов

Атрибут	Тип файла
-	Обычный файл.
d	Каталог.
l	Символическая ссылка. Обратите внимание, что у символических ссылок остальные атрибуты файла всегда «гwxгwxгwx» и являются фиктивными. Реальные атрибуты файла принадлежат файлу, на который указывает символическая ссылка.
c	Символьное специальное устройство. Этот тип файла относится к устройству, которое обрабатывает данные как поток байтов, например, терминал или /dev/null.
b	Блочное специальное устройство. Этот тип файла относится к устройству, которое обрабатывает данные блоками, такое как жесткий диск, SSD, или DVD-привод.

Оставшиеся девять символов атрибутов, называемые режимом файла, определяют права на чтение, запись и выполнение для владельца, группы файла и всех остальных.

User	Группа	Другое
гwx	гwx	гwx

В таблице 9-2 описано влияние атрибутов г, w и x на файлы и каталоги:

Таблица 9-2: Атрибуты разрешений

Атрибут	Файлы	Каталоги
г	Позволяет открывать и читать файл.	Позволяет просматривать выведены в списке, но информация о файлах будет недоступна без прав на выполнение атрибут также установлен.

Атрибут	Файлы	Каталоги
w	Разрешает запись в файл или его усечение, однако этот атрибут не позволяет переименовывать или удалять файлы. Возможность удаление или переименование файло определяется атрибутами атрибуты.	в Позволяет создавать файлы внутри каталога или, deleted, and переименован, если атрибут вполне также установлен.
x	Позволяет обрабатывать файл как программы и выполнения. Программа файлы, написанные на языках скриптов, также должны иметь статус доступных для чтения и выполнения.	Позволяет входить в каталог (т. е. cd directory) и метаданные каталога (т. е. ls -l directory) для доступа к нему. Операции с файлами, такие как cp, rm и mv, требуют этого доступа к каталогу.

В таблице 9-3 приведены примеры настроек атрибутов файлов:

Таблица 9-3: Примеры атрибутов прав доступа

Атрибуты файлов	Значение
-rwx-----	Обычный файл, доступный для чтения, записи и выполнения владельцем файла. У остальных нет прав доступа.
-rw-----	Обычный файл, доступный для чтения и записи владельцу файла. У остальных нет прав доступа.
-rw-r--r--	Обычный файл, доступный для чтения и записи владельцу. Члены группы владельца могут читать файл. Файл доступен для чтения остальным.
-rwxr-xr-x	Обычный файл, доступный для чтения, записи и выполнения владельцу. Файл может быть прочитан и выполнен всеми остальными.
-rw-rw----	Обычный файл, доступный для чтения и записи владельцу и только участникам группы владельца файла.

Атрибуты файлов	Значение
lrwxrwxrwx	Символическая ссылка. Все символические ссылки имеют «фиктивные» права доступа. Реальные права хранятся в самом файле, на который указывает символическая ссылка.
drwxrwx---	Каталог. Владелец и участники группы владельца может входить в каталог, создавать, переименовывать и удалять файлы внутри каталога.
drwxr-x---	Каталог. Владелец может входить в каталог, создавать, переименовывать и удалять файлы внутри каталога. Члены группы владельца могут входить в каталог, но не могут создавать, удалять или переименовывать фай es.

chmod — изменение режима файла

Для изменения режима (прав доступа) файла или каталога используется команда `chmod`. Помните, что только владелец файла или суперпользователь может изменить режим файла или каталога. `chmod` поддерживает два способа задания изменений режима: восьмеричное и символьное представление. Сначала мы рассмотрим восьмеричное представление.

Что же такое восьмеричная система?

Восьмеричная (основание 8) и её «родственница» шестнадцатеричная (основание 16) системы счисления часто используются для представления чисел в компьютерах. Мы, люди, считаем в десятичной системе, так как родились с десятью пальцами (или, по крайней мере, большинство из нас). Компьютеры же «родились» всего с одним пальцем, поэтому ведут все расчеты в двоичной системе (основание 2). В их системе всего две цифры: 0 и 1. Таким образом, двоичный счет выглядит так:

0, 1, 10, 11, 100, 101, 110, 111, 1000, 1001, 1010, 1011...

В восьмеричной системе счет ведется цифрами от нуля до семи, вот так:

0, 1, 2, 3, 4, 5, 6, 7, 10, 11, 12, 13, 14, 15, 16, 17, 20, 21...

Шестнадцатеричный счет использует цифры от нуля до девяти, а также буквы «А»–«F»:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, 10, 11, 12, 13...

Если двоичная система понятна (ведь у компьютера всего один палец), то зачем нужны восьмеричная и шестнадцатеричная? Ответ — для удобства человека. Часто небольшие порции данных в компьютерах представлены в виде битовых последовательностей. Возьмем, к примеру, цвет RGB. На большинстве дисплеев каждый пиксель состоит из трех цветовых компонентов: восьми бит красного, восьми бит зеленого и восьми бит синего. Прекрасный небесно-голубой цвет будет представлен 24-значным числом:

010000110110111111001101

Хотели бы вы читать и записывать такие числа весь день? Вряд ли. Здесь и помогут другие системы счисления. Каждая цифра в шестнадцатеричном числе заменяет четыре двоичных разряда, а в восьмеричной — три. Таким образом, наше 24-значное небесно-голубое число можно сократить до шестизначного шестнадцатеричного:

436FCD

Поскольку цифры шестнадцатеричного числа «совпадают» с битами двоичного, мы видим, что красный компонент нашего цвета — 43, зеленый — 6F, а синий — CD.

Сегодня шестнадцатеричная нотация (часто называемая «hex») встречается чаще восьмеричной, но, как мы скоро увидим, способность восьмеричной системы представлять три бита двоичного кода будет очень полезна...

В восьмеричной нотации мы используем восьмеричные числа для задания шаблона прав доступа. Поскольку каждая цифра восьмеричного числа представляет три двоичных разряда, это отлично соответствует схеме хранения режима файла. В таблице 9-4 показано, что имеется в виду.

Таблица 9-4: Режимы файла в двоичной и восьмеричной системах

Восьмеричная	Бинарный файл	Режим файла
0	000	- - -
1	001	- - x
2	010	- w -
3	011	- wx
4	100	r - -

Восьмеричная	Бинарный файл	Режим файла
5	101	r-x
6	110	rw-
7	111	rwX

Используя три восьмеричные цифры, мы можем задать режим файла для владельца, группы и всех остальных.

```
[me@linuxbox ~]$ > foo.txt
[me@linuxbox ~]$ ls -l foo.txt
-rw-rw-r-- 1 me me 0 2016-03-06 14:52 foo.txt
[me@linuxbox ~]$ chmod 600 foo.txt
[me@linuxbox ~]$ ls -l foo.txt
-rw----- 1 me me 0 2016-03-06 14:52 foo.txt
```

Передав аргумент 600, мы установили владельцу права на чтение и запись, лишив при этом прав группу и остальных. Хотя запоминание соответствия восьмеричных и двоичных чисел может показаться неудобным, обычно достаточно знать лишь несколько основных: 7 (rwX), 6 (rw-), 5 (r-x), 4 (r--) и 0 (---).

chmod также поддерживает символьную запись для указания режимов доступа к файлам. Символьная запись состоит из трех частей.

- На кого повлияет изменение
- Какая операция будет выполнена
- Какие права будут установлены.

Для указания того, на кого распространяются изменения, используется комбинация символов «u», «g», «o» и «a», как показано в таблице 9-5.

Таблица 9-5: chmod Символьное обозначение

Символ	Значение
u	Сокращение от «user» — то есть владельца файла или каталога.
g	Владелец группы.
o	Сокращение для других.
a	Сокращение от «all» (все). Это комбинация «u», «g» и «o».

Если символ не указан, по умолчанию подразумеваются «все». Операция может быть «+» (добавление прав), «-» (удаление прав)

или «=» (применение только указанных прав и удаление всех остальных).

Права доступа задаются символами «r», «w» и «x». В таблице 9-6 приведены примеры символьной записи:

Таблица 9-6: *chmod* Примеры символьных обозначений

Обозначения	Значение
u+x	Добавить право на выполнение для владельца.
u-x	Убрать право на выполнение у владельца.
+x	Добавить право на выполнение для пользователя, группы и остальных. Это эквивалентно a+x.
o-rw	Удалите права на чтение и запись для всех, кроме владельца — пользователя и группы.
go=rw	Установите права на чтение и запись для владельца группы и всех остальных пользователей. Если у владельца группы или остальных были права на выполнение, они будут удалены.
u+x,go=rx	Добавьте право на выполнение для пользователя и установите права на чтение и выполнение для группы и остальных. Несколько спецификаций мож разделять запятыми.

Кто-то предпочитает восьмеричную запись, а кто-то — символьную. Преимущество символьной записи в том, что она позволяет менять од ин атрибут, не затрагивая остальные.

Для получения подробностей и списка опций обратитесь к странице `chmod man`. Предупреждение относительно опции «--recursive»: она применяется и к файлам, и к каталогам, поэтому она не так полезна, как хотелось бы, поскольку нам редко нужно, чтобы файлы и каталоги имели одинаковые права.

Настройка режима файла через GUI

Теперь, когда мы изучили, как устанавливаются права на файлы и каталоги, мы можем лучше понять диалоговые окна прав в GUI. И в Files (GNOME), и в Dolphin (KDE) щелчок правой кнопкой мыши по иконке файла или каталога откроет окно свойств. Вот пример из GNOME:

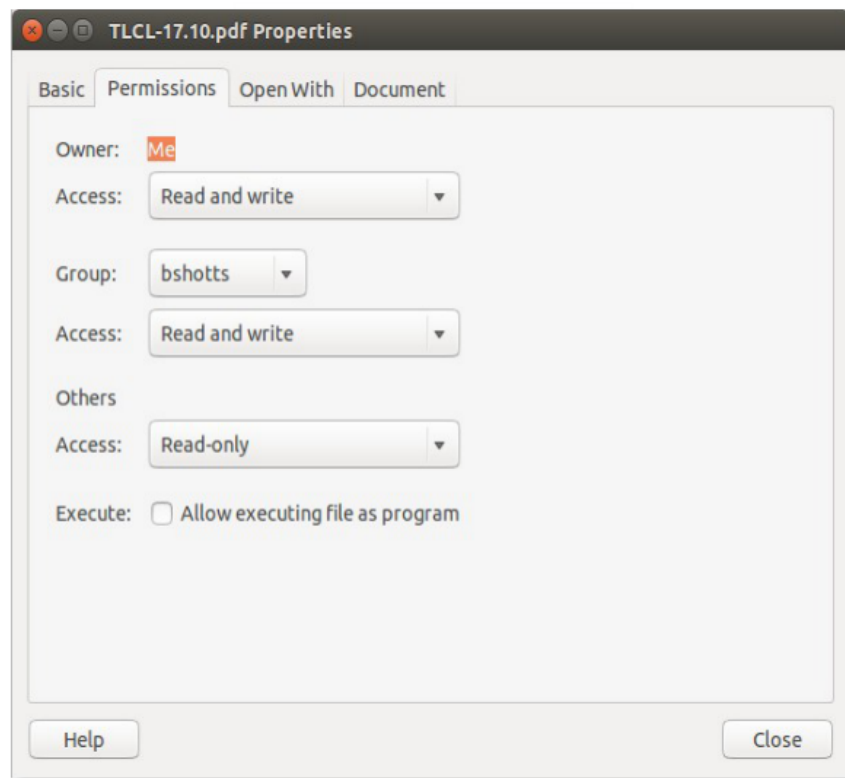


Рисунок 2: Диалог прав доступа к файлам в GNOME

Здесь можно увидеть настройки для владельца, группы и остальных.

umask — Установка прав доступа по умолчанию

Команда `umask` управляет правами по умолчанию, которые назначаются файлу при его создании. Она использует восьмеричную нотацию для задания маски битов, которые будут исключены из атрибутов режима файла. Давайте рассмотрим это.

```
[me@linuxbox ~]$ rm -f foo.txt
[me@linuxbox ~]$ umask
0002
[me@linuxbox ~]$ > foo.txt
[me@linuxbox ~]$ ls -l foo.txt
-rw-rw-r-- 1 me me 0 2025-03-06 14:53 foo.txt
```

Сначала мы удалили все старые копии `foo.txt`, чтобы начать с чистого листа. Затем мы запустили команду `umask` без аргументов, чтобы узнать текущее значение. Она ответила значением `0002` (другим распространенным значением по умолчанию является `0022`), что представляет собой восьмеричное представление нашей маски. Далее мы создаем новый экземпляр файла `foo.txt` и проверяем его права доступа.

Мы видим, что пользователь и группа получают права на чтение и запись, а все остальные — только на чтение. Отсутствие прав на запись у всех остальных объясняется значением маски. Давайте повторим наш пример, на этот раз установив маску самостоятельно.

```
[me@linuxbox ~]$ rm foo.txt
[me@linuxbox ~]$ umask 0000
[me@linuxbox ~]$ > foo.txt
[me@linuxbox ~]$ ls -l foo.txt
-rw-rw-rw- 1 me me 0 2025-03-06 14:58 foo.txt
```

Когда мы устанавливаем маску 0000 (фактически отключая её), мы видим, что файл становится доступен для записи всем. Чтобы понять, как это работает, нужно снова обратиться к восьмеричным числам. Если мы изменим маску на 0002, переведем её в двоичную систему и сравним с атрибутами, мы увидим, что происходит.

--- rw- rw- rw-

Исходный режим файла

000 000 000 010 **Результат**

Mask

--- rw- rw- r--

Пока проигнорируйте ведущие нули (мы разберем их чуть позже) и заметьте, что там, где в нашей маске стоит 1, атрибут был удален — в данном случае право на запись для всех. Именно это и делает маска. Там, где в двоичном значении маски стоит 1, атрибут сбрасывается. Рассмотрим значение маски 0022, чтобы увидеть это в действии.

Исходный режим файла --- rw- rw- rw-

000 000 010 010

Mask

--- rw- r-- r--

Результат

Опять же, там, где в двоичном значении стоит 1, соответствующий атрибут сбрасывается. Поэкспериментируйте с разными значениями (попробуйте семерки), чтобы привыкнуть к этому принципу. Когда закончите, не забудьте всё вернуть в исходное состояние.

```
[me@linuxbox ~]$ rm foo.txt; umask 0002
```

Чаще всего нам не придется менять маску — значений по умолчанию в дистрибутиве будет достаточно. Однако в

ситуациях с повышенными требованиями к безопасности нам может понадобиться контроль над ней.

Некоторые специальные разрешения

Хотя маска разрешений обычно представляется в виде трехзначного восьмеричного числа, технически правильнее использовать четыре цифры. Почему? Потому что, помимо прав на чтение, запись и выполнение, существуют и другие, менее используемые настройки разрешений.

Первая из них — бит `setuid` (восьмеричное 4000). При применении к исполняемому файлу он меняет эффективный ID пользователя с ID реального пользователя (того, кто запускает программу) на ID владельца программы. Чаще всего это свойственно нескольким программам, принадлежащим суперпользователю. Когда обычный пользователь запускает программу «`setuid root`», она выполняется с эффективными привилегиями суперпользователя. Это позволяет программе получать доступ к файлам и каталогам, в которых обычно отказано обычному пользователю. Очевидно, что из-за соображений безопасности количество программ с `setuid` должно быть сведено к абсолютному минимуму.

Второй по частоте использования параметр — это бит `setgid` (восьмеричное 2000), который, как и бит `setuid`, меняет эффективный GID с реального GID пользователя на GID владельца файла. Если бит `setgid` установлен на каталоге, новые файлы в этом каталоге будут принадлежать группе каталога, а не группе создателя файла. Это полезно в общем каталоге, когда участники одной группы должны иметь доступ ко всем файлам в каталоге, независимо от основной группы владельца файла.

Третий называется `sticky bit` (восьмеричное 1000). Это наследие древнего Unix, где можно было пометить исполняемый файл как «не подлежащий подкачке». Linux игнорирует `sticky bit` для файлов, но если он применен к каталогу, то запрещает пользователям удалять или переименовывать файлы, если только пользователь не является владельцем каталога, владельцем файла или суперпользователем. Это часто используется для управления доступом к общему каталогу, например `/tmp`.

Вот несколько примеров использования `chmod` с символьной нотацией для установки этих специальных прав. Вот пример назначения `setuid` программе:

```
chmod u+s program
```

Далее приведен пример назначения setgid каталогу:

```
chmod g+s dir
```

Наконец, вот пример назначения sticky bit каталогу:

```
chmod +t dir
```

При просмотре вывода `ls` можно определить специальные права. Вот несколько примеров. Сначала пример программы с назначенным `setuid`:

```
-rwsr-xr-x
```

Вот пример каталога с установленным атрибутом `setgid`:

```
drwxrwsr-x
```

Вот пример каталога с установленным `sticky bit`:

```
drwxrwxrwt
```

Смена идентификаторов

Иногда может возникнуть необходимость принять личность другого пользователя. Часто нам требуются привилегии суперпользователя для выполнения административных задач, но можно и «стать» обычным пользователем, например, для тестирования учетной записи. Существует три способа смены идентификатора.

1. Выйдите из системы и войдите снова под другим пользователем.
2. Используйте команду `su`.
3. Используйте команду `sudo`. 3. Use the `sudo` command.

Мы пропустим первый метод, так как знаем, как его выполнить, и он менее удобен, чем два других. В рамках текущей сессии оболочки команда `su` позволяет принять личность другого пользователя и либо запустить новую сессию оболочки с его ID, либо выполнить одну команду от его имени. Команда `sudo` позволяет администратору настроить конфигурационный файл `/etc/sudoers` и определить конкретные команды, которые отдельным пользователям разрешено выполнять под чужой личностью. Выбор команды во многом зависит от используемого дистрибутива Linux. Скорее всего, в вашем дистрибутиве есть обе команды, но настройки будут отдавать предпочтение одной из них. Мы начнем с `su`. Однако имейте в виду, что использование

su постепенно выходит из употребления в современных дистрибутивах Linux.

su — Запуск оболочки с заменой ID пользователя и группы

Команда su используется для запуска оболочки от имени другого пользователя. Синтаксис команды выглядит так:

```
su [-[l]] [user]
```

Если указана опция «-l», итоговая сессия станет входной оболочкой для заданного пользователя. Это означает, что загрузится окружение пользователя, а текущий рабочий каталог сменится на его домашний каталог. Обычно нам нужно именно это. Если пользователь не указан, по умолчанию подразумевается суперпользователь. Заметьте, что (как ни странно) -l можно сократить до -, и именно так её чаще всего используют. Если у учетной записи root установлен пароль (что не является стандартом в современных дистрибутивах), мы можем запустить оболочку суперпользователя следующим образом:

```
[me@linuxbox ~]$ su -  
Password:  
[root@linuxbox ~]#
```

После ввода команды появится приглашение ввести пароль суперпользователя. Если он введен верно, появится новое приглашение оболочки, указывающее на наличие привилегий суперпользователя (в конце будет # вместо \$), а текущим рабочим каталогом станет домашний каталог суперпользователя (обычно /root). В новой оболочке можно выполнять команды от имени суперпользователя. По завершении введите exit, чтобы вернуться в предыдущую оболочку.

```
[root@linuxbox ~]# exit  
[me@linuxbox ~]$
```

Используя su этот способ, можно выполнить одну команду вместо запуска новой интерактивной оболочки.

```
su -c 'command'
```

При использовании этой формы одна командная строка передается новой оболочке для выполнения. Важно заключать команду в кавычки, так как мы не хотим, чтобы раскрытие подстановочных знаков происходило в нашей оболочке, а только в новой оболочке.

```
[me@linuxbox ~]$ su -c 'ls -l /root/*'
Password:
-rw----- 1 root root      754 2025-08-11 03:19 /root/anaconda-ks.cfg
/root/Mail:
total 0
[me@linuxbox ~]$
```

sudo — Выполнение команды от имени другого пользователя

Команда `sudo` во многих отношениях похожа на `su`, но обладает важными дополнительными возможностями. Администратор может настроить `sudo`, чтобы обычный пользователь мог контролируемым образом выполнять команды от имени другого пользователя (обычно суперпользователя). В частности, пользователю можно разрешить только одну или несколько конкретных команд. Еще одно важное отличие заключается в том, что использование `sudo` не требует пароля суперпользователя. Аутентификация через `sudo` требует пароля самого пользователя. Предположим, что `sudo` настроена так, чтобы позволить нам запустить фиктивную программу резервного копирования «`backup_script`», требующую привилегий суперпользователя. С `sudo` это делается так:

```
[me@linuxbox ~]$ sudo backup_script
Password:
System Backup Starting...
```

После ввода команды появится приглашение для ввода пароля (не суперпользователя), и после завершения аутентификации будет выполнена указанная команда. Важное отличие между `su` и `sudo` заключается в том, что `sudo` не запускает новую оболочку и не загружает окружение другого пользователя. Это значит, что команды не нужно заключать в кавычки иначе, чем без использования `sudo`. Обратите внимание, что это поведение можно переопределить с помощью различных опций. Также учтите, что `sudo` можно использовать для запуска интерактивной сессии суперпользователя (подобно `su -`), используя опцию `-i`. Подробности см. в [man-странице sudo](#).

Чтобы увидеть привилегии, предоставляемые `sudo`, используйте опцию `-l` для их вывода:

```
[me@linuxbox ~]$ sudo -l
User me may run the following commands on this host:
(ALL) ALL
```

Современные дистрибутивы Linux и sudo

Одна из постоянных проблем обычных пользователей — выполнение задач, требующих привилегий суперпользователя. К таким задачам относятся установка и обновление ПО, редактирование системных конфигурационных файлов и доступ к устройствам. В мире Windows это часто решается предоставлением пользователям прав администратора. Это позволяет выполнять нужные задачи, но также наделяет запущенные пользователем программы теми же возможностями. В большинстве случаев это оправдано, однако это дает вредоносному ПО (например, вирусам) полную свободу действий в компьютере.

В мире Unix из-за многопользовательской природы всегда существовало четкое разделение между обычными пользователями и администраторами. Подход Unix заключается в предоставлении привилегий суперпользователя только тогда, когда это необходимо. Для этого обычно используются команды `su` и `sudo`.

Много лет назад большинство дистрибутивов Linux полагались для этих целей на `su`. `su` не требовал настройки, необходимой для `sudo`, а наличие учетной записи `root` является традиционным для Unix. Однако это создало проблему. У пользователей возникало искушение работать под `root` без необходимости. На самом деле некоторые пользователи работали в системе исключительно как `root`, так как это избавляет от всех этих раздражающих сообщений «permission denied». Так вы снижаете безопасность системы Linux до уровня Windows. Это плохая идея.

При создании Ubuntu её разработчики выбрали иной путь. По умолчанию в Ubuntu отключен вход в учётную запись `root` (пароль для неё не задан), а для предоставления прав суперпользователя используется `sudo`. Первому пользователю предоставляется полный доступ к правам суперпользователя через `sudo`, и он может наделить аналогичными полномочиями последующие учётные записи. Такой метод предоставления прав стал стандартом в большинстве современных дистрибутивов.

`chown` — Изменение владельца и группы файла

Команда `chown` используется для смены владельца и группы владельца файла или каталога. Для использования этой команды

требуется права суперпользователя. Синтаксис `chown` выглядит так:

```
chown [owner][:[group]] file...
```

`chown` может менять владельца файла и/или владельца группы в зависимости от первого аргумента команды. Примеры приведены в таблице 9-7.

Таблица 9-7: Примеры аргументов `chown`

Аргумент	Результаты
bob	Меняет владельца файла с текущего на пользователя bob.
bob:users	Меняет владельца файла с текущего на пользователя bob и меняет группу владельца файла на группу users.
:admins	Меняет группу владельца на группу admins. Владелец файла остается прежним.
bob:	Меняет владельца файла с текущего на пользователя bob и меняет владельца группы на основную группу пользователя bob.

Допустим, у нас есть два пользователя: `janet` с правами суперпользователя и `tony` без них. Пользователь `janet` хочет скопировать файл из своего домашнего каталога в домашний каталог пользователя `tony`. Поскольку `janet` хочет, чтобы `tony` мог редактировать файл, `janet` меняет владельца скопированного файла с `janet` на `tony`.

```
[janet@linuxbox ~]$ sudo cp myfile.txt ~tony
Password:
[janet@linuxbox ~]$ sudo ls -l ~tony/myfile.txt
-rw-r--r-- 1 root  root  root  2025-03-20 14:30 /home/tony/myfile.txt
[janet@linuxbox ~]$ sudo chown tony: ~tony/myfile.txt
[janet@linuxbox ~]$ sudo ls -l ~tony/myfile.txt
-rw-r--r-- 1 tony  tony  tony  2025-03-20 14:30 /home/tony/myfile.txt
```

Здесь мы ее пользователь `janet` копирует файл из своего каталога в домашний каталог пользователя `r tony`. Затем `janet` меняет владельца файла с `root` (результат использования `sudo`) на `tony`. Используя двоеточие в конце первого аргумента, `janet` также сменил группу файла на основную группу пользователя `tony`, которой является `group tony`.

Заметьте, что после первого использования `sudo` у `janet` не запросили пароль. Это происходит потому, что `sudo` в большинстве конфигураций «доверяет» нам в течение нескольких минут, пока не истечет таймер.

`chgrp` — смена владельца группы

В старых версиях Unix команда `chown` меняла только владельца файла, но не группу. Для этой цели использовалась отдельная команда `chgrp`. Она работает почти так же, как `chown`, но имеет меньше возможностей.

Использование наших привилегий

Теперь, когда мы разобрались с правами доступа, пора применить их на практике. Мы продемонстрируем решение типичной задачи — настройку общего каталога. Вспомним наших знакомых `janet` и `tony`. У обоих есть коллекции музыки, и они хотят создать общий каталог, где каждый сможет хранить свои файлы в форматах Ogg Vorbis или MP3. Как и прежде, пользователь `janet` имеет доступ к привилегиям суперпользователя через `sudo`.

Нужно создать группу, участниками которой будут `janet` и `tony`. Это делается в два этапа. Сначала с помощью команды `groupadd` мы создаем группу, а затем с помощью команды `usermod` добавляем в неё пользователей:

```
[janet@linuxbox ~]$ sudo groupadd music
[janet@linuxbox ~]$ sudo usermod -a -G music janet
[janet@linuxbox ~]$ sudo usermod -a -G music tony
```

Опции, используемые с командой `usermod`, являются сокращениями для `--append` и `--group`; они добавляют указанного пользователя в соответствующую группу в файле `/etc/group`.

Затем `janet` создает каталог для музыкальных файлов.

```
[janet@linuxbox ~]$ sudo mkdir /usr/local/share/Music
Password:
```

Поскольку `janet` манипулирует файлами вне своего домашнего каталога, требуются привилегии суперпользователя. После создания каталог будет иметь следующие владельца и права:

```
[janet@linuxbox ~]$ ls -ld /usr/local/share/Music
drwxr-xr-x 2 root root 4096 2025-03-21 18:05 /usr/local/share/Music
```

Как видно, владельцем каталога является `root`, а режим доступа — `755`. Чтобы сделать этот каталог общим, `janet` нужно изменить владельца группы и права группы, разрешив запись.

```
[janet@linuxbox ~]$ sudo chown :music /usr/local/share/Music
[janet@linuxbox ~]$ sudo chmod 2775 /usr/local/share/Music
[janet@linuxbox ~]$ ls -ld /usr/local/share/Music
drwxrwsr-x 2 root music 4096 2025-03-21 18:05 /usr/local/share/Music
```

С помощью команды `chown` `janet` назначает группе каталога `music`, а затем использует `chmod`, чтобы установить права каталога `2775`. Это устанавливает `setgid`, чтобы все файлы в каталоге наследовали ту же группу, что и сам каталог. Мы сделали это, выполнив `chmod 2775`, но могли поступить так же, используя символьный метод с `chmod g+s`.

Что всё это значит? Это значит, что теперь у нас есть каталог `/usr/local/share/Music` `root`, владельцем которого является группа `music` `music` `janet` `tony` `janet` `tony`, и который предоставляет ей права на чтение и запись. Группа имеет участников и, таким образом, может создавать файлы в каталоге `/usr/local/share/Music`. Другие пользователи могут просматривать содержимое каталога, но не могут создавать в нем файлы.

Но проблема остается. Значение `umask` по умолчанию в этой системе — `0022`, что запрещает участникам группы записывать файлы, принадлежащие другим членам группы. Это не было бы проблемой, если бы общий каталог содержал только файлы, но так как здесь будет храниться музыка, а она обычно организована в иерархию исполнителей и альбомов, участникам группы понадобится возможность создавать файлы и каталоги внутри каталогов, созданных другими участниками. Нам нужно изменить `umask`, используемый `janet` и `tony`, на `0002`.

`janet` устанавливает свой `umask` в `0002` и создает новый тестовый файл и каталог:

```
[janet@linuxbox ~]$ umask 0002
[janet@linuxbox ~]$ > /usr/local/share/Music/test_file
[janet@linuxbox ~]$ mkdir /usr/local/share/Music/test_dir
[janet@linuxbox ~]$ ls -l /usr/local/share/Music
drwxrwsr-x 2 janet music 4096 2025-03-24 20:24 test_dir
-rw-rw-r-- 1 janet music 0 2025-03-24 20:22 test_file
[janet@linuxbox ~]$
```

Теперь и файлы, и каталоги создаются с правильными правами, позволяющими всем участникам группы `music` создавать файлы и каталоги внутри каталога `Music`.

Остался один вопрос — `umask`. Необходимая настройка действует только до конца сеанса, после чего её нужно сбросить. В главе 11 мы рассмотрим, как сделать изменения в `umask` постоянными.

Смена пароля

Последняя тема этой главы — установка паролей для себя (и для других пользователей, если у нас есть права суперпользователя). Для установки или смены пароля используется команда `passwd`. Синтаксис команды выглядит так:

```
passwd [user]
```

Чтобы сменить пароль, нужно просто ввести команду `passwd`. Система запросит старый и новый пароли.

```
[me@linuxbox ~]$ passwd
Changing password for me.
Current password:
New password:
Retype new password:
passwd: password updated successfully
```

Команда `passwd` будет пытаться навязать использование «сложных» паролей. Это значит, что она выдаст предупреждение, если пароль слишком короткий, слишком похож на предыдущие, является словарным словом или его слишком легко угадать.

```
[me@linuxbox ~]$ passwd
Changing password for me.
Current password:
New password:
BAD PASSWORD: is too similar to the old one
New password:
BAD PASSWORD: it is WAY too short
New password:
BAD PASSWORD: it is based on a dictionary word
```

Обладая привилегиями суперпользователя, вы можете указать имя пользователя в качестве аргумента команды `passwd`, чтобы установить пароль для другого пользователя. Суперпользователю доступны и другие опции, такие как блокировка учетной записи, истечение срока действия пароля и т. д. Подробности см. в `passwd man page`.

Команды `passwd`, `addgroup` и `usermod` входят в пакет команд `shadow-utils`. В таблице 9-8 перечислены некоторые из команд, содержащихся в этом пакете:

Таблица 9-8: `shadow-utils` Команды

Команда	Описание
---------	----------

Команда	Описание
<code>last log</code>	Выводит данные о последнем входе всех пользователей или конкретного пользователя.
<code>useradd</code>	Создает нового пользователя или обновляет информацию по умолчанию для новых пользователей.
<code>userdel</code>	Удаляет учетную запись пользователя и связанные с ней файлы.
<code>usermod</code>	Изменяет учетную запись пользователя.
<code>groupadd</code>	Создает новую группу.
<code>groupdel</code>	Удаляет группу.
<code>groupmod</code>	.Изменение определения группы в системе.

Мы не будем подробно разбирать эти команды, так как они несколько выходят за рамки данной книги. Для получения дополнительной информации обращайтесь к страницам `man` каждой команды.

Итоги

В этой главе мы рассмотрели, как Unix-подобные системы, такие как Linux, управляют правами пользователей, обеспечивая доступ к чтению, записи и выполнению файлов и каталогов. Основные идеи этой системы прав восходят к ранним дням Unix и отлично выдержали проверку временем. Однако встроенному механизму прав в Unix-подобных системах не хватает гибкости, присущей более современным системам.

Что почитать

- В Wikipedia есть хорошая статья о вредоносном ПО:
<http://en.wikipedia.org/wiki/Malware>

10 — Процессы

Современные операционные системы обычно многозадачны: они создают иллюзию одновременной работы нескольких программ, быстро переключаясь между ними. Ядро Linux управляет этим с помощью процессов. Процессы — это способ, которым Linux организует работу различных программ, ожидающих своей очереди на CPU.

Иногда компьютер начинает тормозить или приложение перестает отвечать. В этой главе мы рассмотрим инструменты командной строки, которые позволяют изучить работу программ и завершить некорректно работающие процессы.

В этой главе будут представлены следующие команды:

- `ps` — вывод снимка текущих процессов.
- `top` — отображение задач.
- `jobs` — список активных заданий.
- `bg` — перевод задания в фоновый режим.
- `fg` — перевод задания в активный режим.
- `kill` — отправка сигнала процессу.
- `killall` — завершение процессов по имени.
- `nice` — запуск программы с измененным приоритетом планировщика.
- `renice` — изменение приоритета запущенных процессов.
- `nohup` — запуск команды, устойчивой к разрыву соединения.
- `halt/poweroff/reboot` — остановка, выключение или перезагрузка системы.
- `shutdown` — завершение работы или перезагрузка системы.

Как работает процесс

При запуске системы ядро иницирует ряд собственных процессов и запускает программу `init`. `init`, в свою очередь, запускает `systemd`, которая запускает все системные службы. В старых дистрибутивах Linux `init` выполняет серию shell-скриптов (расположенных в `/etc`), называемых `init`-скриптами, для выполнения аналогичной функции. Многие системные службы реализованы как программы-демоны — программы, которые работают в фоновом режиме без пользовательского интерфейса. Таким образом, даже если мы не вошли в систему, она занята выполнением рутинных задач.

Тот факт, что одна программа может запускать другие, в схеме процессов выражается как создание дочернего процесса родительским процессом.

Ядро хранит информацию о каждом процессе для поддержания порядка. Например, каждому процессу присваивается номер — идентификатор процесса (PID). PID назначаются в порядке возрастания, при этом `init` всегда получает PID 1. Ядро также отслеживает память, выделенную каждому процессу, и готовность процессов возобновить выполнение. Как и у файлов, у процессов есть владельцы, идентификаторы пользователей, эффективные идентификаторы пользователей и т. д.

Просмотр процессов

Самый распространенный инструмент для просмотра процессов (их несколько) — это команда `ps`. Программа `ps` имеет множество опций, но в простейшем виде она используется так:

```
[me@linuxbox ~]$ ps
  PID TTY          TIME CMD
  5198 pts/1    00:00:00 bash
 10129 pts/1    00:00:00 ps
```

В данном примере результат содержит два процесса: 5198 и 10129, которые являются `bash` и `ps` соответственно. Как видно, по умолчанию `ps` показывает немного — только процессы, связанные с текущей сессией терминала. Чтобы увидеть больше, нужно добавить опции, но прежде рассмотрим другие поля, выводимые `ps`. TTY — это сокращение от «teletype», обозначающее управляющий терминал процесса. Здесь Unix проявляет свой возраст. Поле TIME указывает количество процессорного времени, затраченного процессом. Как видим, ни один из процессов не нагружает компьютер сильно.

Добавление опции позволяет получить более полное представление о работе системы.

```
[me@linuxbox ~]$ ps x
  PID TTY          STAT TIME COMMAND
  2799 ?           Ssl   0:00 /usr/libexec/bonobo-activation-server -ac
  2820 ?           Sl    0:01 /usr/libexec/evolution-data-server-1.10 --
15647 ?           Ss    0:00 /bin/sh /usr/bin/startkde
15751 ?           Ss    0:00 /usr/bin/ssh-agent /usr/bin/dbus-launch --
15754 ?           S     0:00 /usr/bin/dbus-launch --exit-with-session
15755 ?           Ss    0:01 /bin/dbus-daemon --fork --print-pid 4 -pr
15774 ?           Ss    0:02 /usr/bin/gpg-agent -s -daemon
15793 ?           S     0:00 start_kdeinit --new-startup +kcmint_start
15794 ?           Ss    0:00 kdeinit Running...
15797 ?           S     0:00 dcopserver -nosid
and many more...
```

Опция «х» (обратите внимание: без ведущего дефиса) заставляет ps показывать все наши процессы, независимо от того, управляются ли они каким-либо терминалом. Знак «?» в столбце TTY означает отсутствие управляющего терминала. С этой опцией мы видим список всех принадлежащих нам процессов.

Поскольку в системе запущено множество процессов, ps выводит длинный список. Для удобства просмотра часто полезно направить вывод ps в less через конвейер. Некоторые комбинации опций также создают длинные строки вывода, поэтому стоит развернуть окно эмулятора терминала на весь экран.

В вывод добавлен новый столбец STAT. STAT — это сокращение от «state», оно показывает текущий статус процесса, как показано в таблице 10-1.

Таблица 10-1: Состояния процессов

Состояние	Значение
R	Running. Процесс выполняется или готов к выполнению.
S	Спящий режим. Процесс не выполняется, а ожидает события, например, нажатия клавиши или сетевого пакета.
D	Непрерываемое ожидание. Процесс ожидает ввода-вывода, например, диск.
T	Остановлен. Процессу дана команда остановиться. Подробнее об этом в этой главе.
Z	Завершенный или «зомби»-процесс. Это дочерний процесс, который завершил работу, но не был очищен своим родительским процессом.
<	Высокоприоритетный процесс. Можно повысить значимость процесса, выделяя ему больше времени процессора. Это свойство называется niceness. Процесс с высоким приоритетом считается менее «вежливым», так как он занимает больше времени процессора, оставляя меньше другим. всех остальных.
N	Низкоприоритетный процесс. Процесс с низким приоритетом («nice» процесс) будет получать процессорное время только после того, как будут обслужены другие процессы с более высоким приоритетом.

За состоянием процесса могут следовать другие символы. Они указывают на различные специфические характеристики процесса. Подробнее см. в `ps man page`.

Еще один популярный набор опций — «aux» (без ведущего дефиса). Он предоставляет еще больше информации.

```
[me@linuxbox ~]$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.0	2136	644	?	Ss	Mar05	0:31	init
root	2	0.0	0.0	0	0	?	S<	Mar05	0:00	[kt]
root	3	0.0	0.0	0	0	?	S<	Mar05	0:00	[mi]
root	4	0.0	0.0	0	0	?	S<	Mar05	0:00	[ks]
root	5	0.0	0.0	0	0	?	S<	Mar05	0:06	[wa]
root	6	0.0	0.0	0	0	?	S<	Mar05	0:36	[ev]
root	7	0.0	0.0	0	0	?	S<	Mar05	0:00	[kh]

and many more...

Этот набор опций отображает процессы всех пользователей. Использование опций без ведущего дефиса запускает команду в «BSD-стиле». Версия `ps` для Linux может имитировать поведение программы `ps` из различных реализаций Unix. В таблице 10-2 приведены наиболее популярные опции BSD.

Таблица 10-2: Популярные опции `ps` в стиле BSD

Опция	Функция
x	Вывести список запущенных процессов.
ax	Вывести список всех запущенных процессов.
w	Включать полные имена команд.
u	Подробный вывод.

С опциями `aux` мы получаем дополнительные столбцы, показанные в таблице 10-3.

Таблица 10-3: Заголовки столбцов `ps` в стиле BSD

Заголовок	Значение
USER	Идентификатор пользователя. Это владелец процесса.
%CPU	Загрузка CPU в процентах.
%MEM	Использование памяти в процентах.
VSZ	Размер виртуальной памяти.
RSS	Размер резидентного набора. Это объем физической памяти (RAM), используемой процессом, в килобайтах.

Заголовок	Значение
START	Время запуска процесса. Если прошло более 24 часов, указывается дата, used.
TIME	Количество процессорного времени, потребляемого процессом.

Также можно получить подробный снимок одного процесса, передав PID в качестве аргумента команды, как показано в примере ниже.

```
[me@linuxbox ~]$ ps uw 44719
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
me	44719	0.0	0.0	13480	6492	pts/1	S	15:57	0:00	bash

Динамический просмотр процессов с помощью top

Хотя команда `ps` может многое рассказать о работе машины, она дает лишь моментальный снимок состояния системы в момент выполнения команды `ps`. Чтобы увидеть активность машины в динамике, используйте команду `top`:

```
[me@linuxbox ~]$ top
```

Программа `top` непрерывно обновляет (по умолчанию каждые три секунды) список системных процессов, отсортированных по интенсивности их работы. Название `top` происходит от того, что программа `top` используется для просмотра «верхних» процессов в системе. Экран `top` состоит из двух частей: сводки состояния системы в верхней части и таблицы процессов, отсортированных по активности CPU:

```
top - 14:59:20 up 6:30, 2 users, load average: 0.07, 0.02, 0.00
Tasks: 109 total, 1 running, 106 sleeping, 0 stopped, 2 zombie
Cpu(s): 0.7%us, 1.0%sy, 0.0%ni, 98.3%id, 0.0%wa, 0.0%hi, 0.0%si
Mem: 319496k total, 314860k used, 4636k free, 19392k buff
Swap: 875500k total, 149128k used, 726372k free, 114676k cach
  PID USER      PR  NI  VIRT  RES  SHR S %CPU %MEM    TIME+  COMMAND
 6244 me        39   19 31752 3124 2188 S  6.3  1.0   16:24.42 trackerd
11071 me        20    0 2304 1092  840 R  1.3  0.3    0:00.14 top
 6180 me        20    0 2700 1100  772 S  0.7  0.3    0:03.66 dbus-dae
 6321 me        20    0 20944 7248 6560 S  0.7  2.3    2:51.38 multiloa
 4955 root       20    0 104m 9668 5776 S  0.3  3.0    2:19.39 Xorg
    1 root       20    0 2976  528  476 S  0.0  0.2    0:03.14 init
    2 root       15   -5    0    0    0 S  0.0  0.0    0:00.00 kthreadd
    3 root       RT   -5    0    0    0 S  0.0  0.0    0:00.00 migratio
    4 root       15   -5    0    0    0 S  0.0  0.0    0:00.72 ksoftirq
    5 root       RT   -5    0    0    0 S  0.0  0.0    0:00.04 watchdog
    6 root       15   -5    0    0    0 S  0.0  0.0    0:00.42 events/0
    7 root       15   -5    0    0    0 S  0.0  0.0    0:00.06 khelper
   41 root       15   -5    0    0    0 S  0.0  0.0    0:01.08 kblockd/
   67 root       15   -5    0    0    0 S  0.0  0.0    0:00.00 kseriod
  114 root       20    0    0    0    0 S  0.0  0.0    0:01.62 pdflush
  116 root       15   -5    0    0    0 S  0.0  0.0    0:02.44 kswapd0
```

Обзор системы содержит много полезной информации. В таблице 10-4 представлен краткий обзор.

Таблица 10-4: *top* Поля информации

Row	Поле	Значение
1	top	Имя программы.
	14:59:20	Текущее время.
	up 6:30	Это называется uptime. Это время, прошедшее с момента последней загрузки машины. В данном полчасе.
	2 users	В системе вошли два пользователя.
	load average:	Load average — это количество процессов ожидающих выполнения, то есть количество процессов в состоянии готовности к выполнению, использующих CPU. Показаны три значения, каждое
		за разные периоды времени. Первый — это среднее значение за последние 60 секунд, следующий — за предыдущие 5 минут, и, наконец, за предыдущие 15 минут. Значения меньше 1.0 указывают на то, что машина не занята.
2	Tasks:	Здесь приводится сводка по количеству процессов и их различным состояниям.
3	Cpu(s):	Эта строка описывает характер операций, выполняемых CPU.
	0.7%us	0,7 процента CPU используется для пользовательских процессов. Это означает, что процессы вне ядро.
	1.0%sy	1,0% ЦП занято системными (процессами ядра).
	0.0%ni	0,0% ЦП занято «nice» (низкоприоритетными) процессами.
	98.3%id	98,3% ЦП простаивает.
	0.0%wa	0,0% процессора ожидает ввода-вывода.
4	Mem:	Здесь показано использование физической ОЗУ.

Row	Поле	Значение
5	Swap:	Здесь показано использование файла подкачки (виртуальной памяти) используется.

Программа `top` принимает ряд команд с клавиатуры. Две наиболее интересные: `h`, которая выводит справку по программе, и `q`, которая завершает работу `top`.

Обе основные среды рабочего стола предоставляют графические приложения, отображающие схожую информацию, как `top` (подобно диспетчеру задач в Windows), но `top` лучше графических версий, так как работает быстрее и потребляет гораздо меньше системных ресурсов. В конце концов, программа мониторинга системы не должна сама становиться причиной замедления системы, которую мы пытаемся отследить.

Управление процессами

Теперь, когда мы умеем просматривать и отслеживать процессы, давайте научимся ими управлять. В качестве подопытного для наших экспериментов мы возьмем небольшую программу `xlogo`. Программа `xlogo` — это пример приложения из состава X Window System (графического движка, который постепенно уступает место Wayland), она просто отображает окно с логотипом X, размер которого можно изменять. Сначала мы познакомимся с нашим объектом тестирования.

```
[me@linuxbox ~]$ xlogo
```

После ввода команды где-нибудь на экране должно появиться небольшое окно с логотипом (Рисунок 3). В некоторых системах `xlogo` может вывести предупреждение, но его можно смело игнорировать.

Совет: Если в вашей системе нет программы `xlogo`, попробуйте вместо неё использовать `gedit` или `kwrite`.

Мы можем убедиться, что `xlogo` работает, изменив размер его окна. Если логотип перерисовывается под новый размер, значит, программа запущена.

Заметили, что приглашение оболочки не появилось? Это происходит потому, что оболочка ждет завершения программы,

как и всех остальных программ, которые мы использовали до сих пор. Если закрыть окно `xlogo`, приглашение вернется.



Рисунок 3: программа `xlogo`

Прерывание процесса

Давайте посмотрим, что произойдет при повторном запуске `xlogo`. Сначала введите команду `xlogo` и убедитесь, что программа работает. Затем вернитесь в окно эмулятора терминала и нажмите `Ctrl-C`.

```
[me@linuxbox ~]$ xlogo  
[me@linuxbox ~]$
```

В терминале нажатие `Ctrl-C` прерывает программу. Это означает, что мы вежливо просим программу завершиться. После нажатия `Ctrl-C` окно `xlogo` закрылось, и появилось приглашение оболочки.

Многие (но не все) программы командной строки можно прервать с помощью этого метода.

Запуск процесса в фоновом режиме

Допустим, мы хотим вернуть приглашение оболочки, не завершая `xlogo` программу. Это можно сделать, отправив программу в фоновый режим. Представьте, что у терминала есть передний план (где всё на виду, например, приглашение оболочки) и задний план (где всё скрыто). Чтобы запустить программу сразу в фоновом режиме, нужно добавить символ амперсанда (`&`) в конце команды.


```
[me@linuxbox ~]$ xlogo &
[1] 28236
[me@linuxbox ~]$
```

После ввода команды появилось окно `xlogo` и приглашение оболочки, но также были выведены странные числа. Это сообщение — часть функции оболочки под названием управление заданиями. Этим сообщением оболочка сообщает, что мы запустили задание № 1 (`[1]`) с PID 28236. Если мы выполним `ps`, то сможем увидеть наш процесс.

```
[me@linuxbox ~]$ ps
PID TTY          TIME CMD
10603 pts/1        00:00:00 bash
28236 pts/1        00:00:00 xlogo
28239 pts/1        00:00:00 ps
```

Механизм управления задачами в оболочке также позволяет просматривать список задач, запущенных из нашего эмулятора терминала. С помощью команды `jobs` можно увидеть этот список:

```
[me@linuxbox ~]$ jobs
[1]+  Running                  xlogo &
```

Результаты показывают, что у нас есть одна запущенная задача под номером 1, а выполняемая команда — `xlogo &`.

Примечание: можно запустить несколько команд в фоновом режиме, используя следующее сочетание клавиш:

```
me@linuxbox:~$ xlogo & gedit &
[1] 47211
[2] 47212
```

Возврат процесса на передний план

Фоновый процесс не реагирует на ввод с клавиатуры в терминале, включая попытки прервать его с помощью `Ctrl-C`. Чтобы вернуть процесс на передний план, используйте команду `fg` следующим образом:

```
[me@linuxbox ~]$ jobs
[1]+  Running                  xlogo &
[me@linuxbox ~]$ fg %1
xlogo
```

Команда `fg`, за которой следует знак процента и номер задания (так называемый `jobspec`), решит задачу. Если фоновое задание только одно, `jobspec` необязателен. Чтобы завершить `xlogo`, нажмите `Ctrl-C`.

Остановка (приостановка) процесса

Иногда нужно приостановить процесс, не завершая его. Часто это делается для перевода процесса из активного режима в фоновый. Чтобы приостановить процесс и отправить его в фон, нажмите `Ctrl-z`. Давайте попробуем. В командной строке введите `xlogo`, нажмите `ENTER`, а затем нажмите `Ctrl-z`:

```
[me@linuxbox ~]$ xlogo
[1]+  Stopped                  xlogo
[me@linuxbox ~]$
```

Остановив `xlogo`, мы можем убедиться в остановке программы, попытавшись изменить размер окна `xlogo`. Мы увидим, что при изменении размера окна логотип не перерисовывается — это указывает на то, что программа остановлена. Мы можем либо продолжить выполнение программы в фоновом режиме с помощью команды `fg`, либо возобновить её выполнение в фоновом режиме с помощью команды `bg`:

```
[me@linuxbox ~]$ bg %1
[1]+  xlogo &
[me@linuxbox ~]$
```

Как и в случае с командой `fg`, спецификация задания необязательна, если задание всего одно.

Перевод процесса из активного режима в фоновый удобен, если мы запустили графическую программу из командной строки, но забыли отправить её в фон, добавив в конце `&`.

Зачем запускать графическую программу из командной строки? На это есть две причины.

- Программы, которую мы хотим запустить, может не быть в меню оконного менеджера (например, `xlogo`).
- Запуская программу из командной строки, мы можем увидеть сообщения об ошибках, которые были бы невидимы при графическом запуске. Иногда программа не запускается через графическое меню. Запуск из командной строки позволит увидеть сообщение об ошибке, раскрывающее причину проблемы. Кроме того, некоторые графические программы имеют интересные и полезные параметры командной строки.

Изменение приоритета процесса

Как мы видели в выводе команды `ps` (а также `top`), существует атрибут процесса под названием `niceness`, который определяет приоритет планирования процесса. В определенных ситуациях — например, при транскодировании видео или выполнении

трассировки лучей силами CPU — мы можем захотеть повысить приоритет процесса (уменьшить `niceness`) или, наоборот, если мы хотим, чтобы процесс потреблял меньше ресурсов CPU, мы можем увеличить его `niceness`. Значение `niceness` можно изменить с помощью команд `nice` и `renice`. Важно помнить, что только суперпользователь может повысить приоритет процесса, а обычные пользователи могут понизить приоритет только тех процессов, которыми они владеют.

Команда `nice` запускает процесс с заданным уровнем вежливости (`niceness`). Значения варьируются от -20 (наивысший приоритет) до 19 (наинизший), по умолчанию — 0 (без изменений). Рассмотрим это на примере. Представим, что у нас есть программа `cpu-hog`, которую мы хотим запустить с более низким приоритетом, чем её стандартные 20. Мы можем запустить её с `nice` следующим образом:

```
[me@linuxbox ~]$ nice -n 10 cpu-hog
```

Аналогично, если программе `must-run-fast` требуется повысить приоритет CPU, мы (как суперпользователь) можем сделать следующее:

```
[me@linuxbox ~]$ sudo nice -n -10 must-run-fast
```

Запуск команд с повышенным приоритетом требуется редко; это несет риск лишить важные системные процессы необходимого времени CPU, так что будьте осторожны.

Команда `renice` изменяет приоритет запущенного процесса. Например, если мы уже запустили программу `cpu-hog` и хотим изменить её значение `nice`, можно сделать так:

```
[me@linuxbox ~]$ ps
  PID TTY          TIME CMD
 379087 pts/9        00:00:00 bash
 379215 pts/9        00:00:00 cpu-hog
 379223 pts/9        00:00:00 ps
[me@linuxbox ~]$ renice -n 19 379215
```

Сначала запустите `ps`, чтобы узнать ID процесса запущенной программы `cpu-hog`, а затем выполните команду `renice` с нужным уровнем `nice` и ID процесса. Уровень `nice` 19 (максимальное значение) полезен тем, что процесс будет использовать циклы CPU только тогда, когда больше никто не ожидает.

Сигналы

Команда `kill` используется для завершения программ, которые необходимо остановить (то есть для их приостановки или принудительного завершения). Вот пример:

```
[me@linuxbox ~]$ xlogo &  
[1] 28401  
[me@linuxbox ~]$ kill 28401  
[1]+  Terminated                  xlogo
```

Сначала мы запускаем `xlogo` в фоновом режиме. оболочка выводит `jobspec` и `PID` фонового процесса. Затем мы используем команду `kill`, указывая `PID` процесса, который хотим завершить. Можно было также указать процесс через `jobspec` (например — `%1`), вместо `PID`.

Хотя всё это кажется простым, здесь есть нюансы. Команда `kill` не совсем «убивает» процессы, а скорее посылает им сигналы. Сигналы — это один из способов взаимодействия операционной системы с программами. Мы уже видели сигналы в действии при использовании `Ctrl-c` и `Ctrl-z`. Когда эмулятор терминала получает одно из этих сочетаний клавиш, он посылает сигнал программе, работающей на переднем плане. В случае с `Ctrl-c` `INT` `Ctrl-z` `TSTP` посылается сигнал (`interrupt`), а при `<kbd>Ctrl-z</kbd>` — сигнал (`terminal stop`). Программы, в свою очередь, «слушают» сигналы и могут реагировать на них по мере получения. То, что программа может слушать сигналы и реагировать на них, позволяет ей, например, сохранять текущую работу при получении сигнала на завершение.

Отправка сигналов процессам с помощью `kill`

Команда `kill` используется для отправки сигналов программам. Её наиболее распространенный синтаксис выглядит так:

```
kill [-signal] PID...
```

Если в командной строке сигнал не указан, по умолчанию отправляется `TERM` (`terminate`). Команда `kill` чаще всего используется для отправки следующих сигналов:

Таблица 10-5: Распространенные сигналы

Номер	Name	Значение
-------	------	----------

Номер	Name	Значение
1	HUP	Hangup. Это наследие старых добрых времен, когда терминалы подключались к удаленным компьютеры с телефонными линиями и модемами. Этот сигнал используется, чтобы сообщить программам, что управляющий терминал «положил трубку». Эффект от этого сигнала можно продемонстрировать, закрыв сеанс терминала. Фоновой программе, запущенной в терминале, будет отправлен этот сигнал и завершится. Этот сигнал также используют многие демоны для повторной инициализации. Это значит, что при получении этого сигнала демон перезапустится и перечитает свой файл конфигурации. Например, веб-сервер Apache является таким демоном, который использует сигнал HUP таким образом.

Номер	Name	Значение
		Процесс можно сделать невосприимчивым к NIP сигнал путем запуска nohup команда (будет рассмотрена далее в этой главе).
2	INT	Прерывание. Оно выполняет ту же функцию, что и Ctrl-c отправлено из терминала. Обычно оно завершить программу.
9	KILL	Kill. Этот сигнал особенный. Если программы могут выбирать способы обработки полученных сигналов, включая их полное игнорирование, то сигнал KILL на самом деле никогда не отправляется целевой программе. Вместо этого ядро немедленно завершает процесс. Если процесс завершается таким образом, у него не остается возможности «прибраться» за собой или сохранить результаты работы. По этой причине сигнал KILL следует использовать только в крайнем случае, когда другие сигнала не доходят.

Номер	Name	Значение
15	TERM	Завершение. Это сигнал по умолчанию, отправляемый kill командой. Если программа всё ещё «жива» и способна принимать сигналы, она завершится.
18	CONT	Продолжение. Это восстановит процесс после STOP или TSTP сигнала. Этот сигнал отправляется bg и fg командами.
19	STOP	Stop. Этот сигнал заставляет процесс приостановить работу, не завершая её. Как и сигнал KILL, он не отправляется целевому процессу, а значит, игнорируется.
20	TSTP	Остановка терминала. Это сигнал, который посылает терминал при нажатии Ctrl-Z. В отличие от STOP сигнал, TSTP сигнал получает программа, но она может решить его проигнорировать.
Давайте попробуем <pre>[me@linuxbox ~]\$ xlogo &</pre> [1] 13546 команду: <pre>[me@linuxbox ~]\$ kill -1 13546</pre> ~]\$ <pre>[me@linuxbox ~]\$</pre>		
<pre>[me@linuxbox ~]\$ xlogo &</pre> <pre>[1] 13546</pre> <pre>[me@linuxbox ~]\$ kill -1 13546</pre> <pre>[1]+ Hangup xlogo</pre>		

В этом примере мы запускаем `xlogo` программу в фоновом режиме, а затем отправляем ей HUP сигнал с помощью `kill`. `xlogo` программа завершается, и оболочка сообщает, что фоновый процесс получил сигнал `hangup`. Возможно, придется нажать ENTER пару раз, прежде чем появится сообщение.

Примечание: сигналы можно указывать как номером, так и именем, включая имена с префиксом SIG.

```
[me@linuxbox ~]$ xlogo &
[1] 13601
[me@linuxbox ~]$ kill -INT 13601
[1]+  Interrupt                  xlogo
[me@linuxbox ~]$ xlogo &
[1] 13608
[me@linuxbox ~]$ kill -SIGINT 13608
[1]+  Interrupt                  xlogo
```

Повторите предыдущий пример и попробуйте другие сигналы. Помните, что вместо PID можно использовать спецификации заданий.

У процессов, как и у файлов, есть владельцы; чтобы отправлять им сигналы с помощью `kill`, вы должны быть владельцем процесса (или суперпользователем).

В дополнение к списку сигналов выше, наиболее часто используемых с `kill`, в таблице 10-5 приведены другие сигналы, часто используемые системой.

Таблица 10-6: Другие распространенные сигналы

Номер	Name	Значение
3	QUIT	Выход.
11	SEGV	Ошибка сегментации. Этот сигнал отправляется, если программа неправомерно использует память, то есть если она пытается выполнить запись там, где это запрещено write.
28	WINCH	Изменение окна. Это сигнал, отправляемый системой при изменении размера окна. Некоторые программы, такие как <code>top</code> и <code>less</code> , реагируют на этот сигнал, перерисовываясь под новые размеры окна.

Для справки: полный список сигналов можно вывести следующей командой:

```
[me@linuxbox ~]$ kill -l
```


Защита процесса от разрыва соединения

Как мы уже обсуждали, многие программы командной строки завершаются при получении сигнала HUP, когда их управляющий терминал «разрывает соединение» (т. е. закрывается или отключается). Чтобы предотвратить это, можно запустить программу с помощью команды `nohup`. Вот пример.

```
[me@linuxbox ~]$ xlogo
```

Если мы снова запустим `xlogo`, а затем закроем окно терминала, `xlogo` завершится, так как при закрытии управляющего терминала ему будет отправлен сигнал HUP. Чтобы этого избежать, можно запустить `xlogo` с помощью команды `nohup` следующим образом:

```
[me@linuxbox ~]$ nohup xlogo
```

Теперь при закрытии окна терминала `xlogo` продолжит работу.

Отправка сигналов нескольким процессам с помощью `killall`

С помощью команды `killall` можно также отправлять сигналы нескольким процессам, соответствующим указанной программе или имени пользователя. Вот синтаксис:

```
killall [-u user] [-signal] name...
```

Для демонстрации мы запустим несколько экземпляров `xlogo`, а затем завершим их работу.

```
[me@linuxbox ~]$ xlogo &
[1] 18801
[me@linuxbox ~]$ xlogo &
[2] 18802
[me@linuxbox ~]$ killall xlogo
[1]- Terminated          xlogo
[2]+ Terminated          xlogo
```

Помните, как и в случае с `kill`, для отправки сигналов процессам, которые нам не принадлежат, требуются привилегии суперпользователя.

Завершение работы системы

Процесс завершения работы системы включает упорядоченное завершение всех процессов, а также выполнение важных служебных задач (таких как синхронизация всех смонтированных файловых систем) перед выключением

питания. Эту функцию выполняют четыре команды: `halt`, `poweroff`, `reboot` и `shutdown`. Первые три вполне понятны сами по себе и обычно используются без параметров командной строки. Вот пример:

```
[me@linuxbox ~]$ sudo reboot
```

Команда `shutdown` несколько интереснее. С её помощью можно указать тип действия (остановка, выключение или перезагрузка) и задать задержку перед завершением работы. Чаще всего её используют для остановки системы следующим образом:

```
[me@linuxbox ~]$ sudo shutdown -h now
```

Для перезагрузки системы её используют так:

```
[me@linuxbox ~]$ sudo shutdown -r now
```

Задержку можно указать разными способами. Подробности см. в `man shutdown`. После выполнения команды `shutdown` всем вошедшим в систему пользователям «рассылается» сообщение с предупреждением о предстоящем событии.

Другие команды управления процессами

Поскольку мониторинг процессов — важная задача системного администрирования, для этого существует множество команд. В таблице 10-6 приведены некоторые из них:

Таблица 10-7: Другие команды для работы с процессами

Команда	Описание
<code>ps tree</code>	Выводит список процессов в виде дерева, отображая связи «родитель — потомок» между процессами.
<code>vmstat</code>	Выводит состояние системных ресурсов, включая память, подкачку и дисковый ввод-вывод. Для непрерывного отображения запускайте команду с задержкой обновления (в секундах). Вот пример: <code>vmstat 5</code> . Прервите вывод с помощью <code>Ctrl-C</code> .
<code>xload</code>	Графическая программа, рисующая график нагрузки системы <code>time</code> .
<code>tload</code>	Похожа на программу <code>xload</code> , но рисует график в терминале. Завершите вывод с помощью <code>Ctrl-C</code> .

Итоги

Большинство современных систем поддерживают механизм управления множеством процессов. Linux предоставляет для этой цели богатый набор инструментов. Учитывая, что Linux — самая распространенная серверная ОС в мире, это вполне логично. Однако, в отличие от некоторых других систем, Linux полагается на инструменты командной строки. Хотя для Linux существуют графические инструменты, инструменты командной строки предпочтительнее из-за их скорости и малого потребления ресурсов. Графические инструменты могут выглядеть красиво, но сами часто создают большую нагрузку на систему, что частично сводит на нет их смысл.

Часть 2 — Конфигурация и окружение

11 — Окружение

Как мы обсуждали ранее, оболочка хранит во время сеанса набор данных, называемый окружением. Программы используют данные из окружения для определения конфигурации системы. Хотя большинство программ хранят настройки в конфигурационных файлах, некоторые также ищут значения в окружении для изменения своего поведения. Зная это, мы можем использовать окружение для настройки работы в оболочке.

В этой главе мы поработаем со следующими командами:

- `printenv` — вывод части или всего окружения.
- `set` — установка параметров оболочки.
- `export` — экспорт окружения в последующие программы.
- `alias` — создание псевдонима для команды.
- `source` — выполнение команд из файла в текущей оболочке.

Что хранится в окружении?

Оболочка хранит в окружении два основных типа данных; хотя при использовании `bash` они на первый взгляд почти неразличимы. Это переменные окружения и переменные оболочки. Переменные оболочки — это данные, помещенные туда текущим экземпляром `bash`, а переменные окружения — всё остальное. Помимо переменных, оболочка хранит программные данные, а именно псевдонимы и функции оболочки. Мы разбирали псевдонимы в главе 5, а функции оболочки (связанные с написанием скриптов) рассмотрим в части 4.

Изучение окружения

Чтобы просмотреть содержимое окружения, можно использовать встроенную команду `set` в `bash` или программу `printenv`.

Команда `set` покажет и переменные оболочки, и переменные окружения, а `printenv` — только последние. Поскольку список будет длинным, лучше всего направить вывод любой из команд через конвейер в `less`.

```
[me@linuxbox ~]$ printenv | less
Doing so, we should get something
that looks like this:
USER=me
PAGER=less
LSCOLORS=Gxfxcxdxbxegedabagacad
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/usr/share/upstart/xdg:/etc/xdg
PATH=/home/me/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/
sbin:/bin:/usr/games:/usr/local/games
DESKTOP_SESSION=ubuntu
```

```

QT_IM_MODULE=ibus
QT_QPA_PLATFORMTHEME=appmenu-qt5
JOB=dbus
PWD=/home/me
GNOME_KEYRING_PID=1850
LANG=en_US.UTF-8
GDM_LANG=en_US
MANDATORY_PATH=/usr/share/gconf/ubuntu.mandatory.path
MASTER_HOST=linuxbox
IM_CONFIG_PHASE=1
COMPIZ_CONFIG_PROFILE=ubuntu
GDMSESSION=ubuntu
SESSIONTYPE=gnome-session
XDG_SEAT=seat0
HOME=/home/me
SHLVL=2
LANGUAGE=en_US
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
LESS=-R
LOGNAME=me
COMPIZ_BIN_PATH=/usr/bin/
LC_CTYPE=en_US.UTF-8
XDG_DATA_DIRS=/usr/share/ubuntu:/usr/share/gnome:/usr/local/share:/usr/share/
QT4_IM_MODULE=xim
DBUS_SESSION_BUS_ADDRESS=unix:abstract=/tmp/dbus-IwaesmWaT0
LESSOPEN=| /usr/bin/lesspipe %s
INSTANCE=

```

Перед нами список переменных окружения и их значений. Например, мы видим переменную `USER` со значением `me`. Команда `printenv` также может вывести значение конкретной переменной.

```

[me@linuxbox ~]$ printenv USER
me

```

Команда `set` при использовании без опций или аргументов выводит переменные оболочки и окружения, а также все определенные функции оболочки. В отличие от `printenv`, её вывод вежливо отсортирован по алфавиту.

```

[me@linuxbox ~]$ set | less

```

Также можно посмотреть содержимое переменной с помощью команды `echo`, вот так:

```

[me@linuxbox ~]$ echo $HOME
/home/me

```

Элемент окружения, который не отображают ни `set`, ни `printenv`, — это псевдонимы. Чтобы увидеть их, введите команду `alias` без аргументов.

```

[me@linuxbox ~]$ alias
alias l.='ls -d .* --color=auto'
alias ll='ls -l --color=auto'
alias ls='ls --color=auto'

```

```
alias vi='vim'
alias which='alias | /usr/bin/which --tty-only --read-alias --show-dot --show-tilde'
```

Некоторые интересные переменные

Окружение содержит немало переменных, и хотя оно будет отличаться от представленного здесь, переменные из таблицы 11-1, скорее всего, встретятся и у нас.

Таблица 11-1: Переменные окружения

Переменная	Содержание
DISPLAY	Имя дисплея, если запущена графическая среда. Обычно это «:0» — первый дисплей, созданный X сервером. В системах с композитором Wayland существует связанная переменная с именем WAYLAND_DISPLAY.
EDITOR	Имя программы для редактирования текста.
SHELL	Имя оболочки по умолчанию для пользователя.
HOME	Путь к нашему домашнему каталогу.
LANG	Определяет набор символов и порядок сортировки для нашего языка.
OLDPWD	Предыдущий рабочий каталог.
PAGER	Имя программы для постраничного вывода. Часто это установлено как /usr/bin/less.
PATH	Список каталогов, разделенных двоеточием, в которых ищется имя исполняемой программы.
PS1	Означает «строка приглашения 1». Это определяет содержимое приглашения оболочки. Как мы увидим в главе 13, его можно широко настраивать.
PWD	Текущий рабочий каталог.
TERM	Имя вашего типа терминала. Unix-подобные системы поддерживают много протоколов терминала; эта переменная задает протокол, используемый с нашего эмулятора терминала.

Переменная	Содержание
TZ	Указывает наш часовой пояс. Unix-подобные системы ведут внутренний отсчет времени в формате Coordinated Universal Time (UTC), а затем отображают местное время, применяя смещение, заданное этой переменной.
USER	Наше имя пользователя.

Не беспокойтесь, если некоторые значения отсутствуют. Они различаются в зависимости от дистрибутива.

Как устанавливается окружение?

При входе в систему запускается программа `bash` и считывает серию конфигурационных скриптов, называемых файлами запуска, которые определяют стандартное окружение для всех пользователей. Затем считываются файлы запуска в нашем домашнем каталоге, определяющие личное окружение. Точная последовательность зависит от типа запускаемой сессии оболочки. Существует два вида.

- Сессия входа в оболочку Сессия входа в оболочку — это сессия, в которой запрашиваются имя пользователя и пароль. Это происходит, например, при входе в графическую среду, а также при запуске сессии в виртуальной консоли.
- Сессия оболочки без входа в систему Сессия оболочки без входа в систему обычно возникает, когда мы запускаем сессию терминала в GUI с помощью нашего эмулятора терминала.

Оболочки с входом в систему считывают один или несколько файлов запуска, как показано в таблице 11-2.

Таблица 11-2: Файлы запуска для сессий оболочки с входом в систему

File	Содержание
<code>/etc/profile</code>	Глобальный скрипт конфигурации, применяемый ко всем пользователям.
<code>~/.bash_profile</code>	Персональный файл запуска пользователя. Он позволяет расширить или переопределить настройки в глобальном конфигурационном скрипте.
<code>~/.bash_login</code>	Если <code>~/.bash_profile</code> не найден, <code>bash</code> пытается прочитать этот скрипт.

File	Содержание
~/.profile	Если ни ~/.bash_profile, ни ~/.bash_login не найдены, bash пытается прочитать этот файл. Это по умолчанию в дистрибутивах на базе Debian, таких как Ubuntu.

Сессии оболочки без входа в систему считывают файлы запуска, перечисленные в таблице 11-3.

Таблица 11-3: Файлы запуска для сессий оболочки без входа в систему

File	Содержание
/etc/bash.bashrc	Глобальный скрипт конфигурации, применяемый ко всем пользователям.
~/.bashrc	Персональный файл запуска пользователя. Его можно использовать для расширения или переопределения настроек в глобальном конфигурационном скрипте.

Помимо чтения файлов запуска из таблицы 11-3, неавторизованные оболочки наследуют переменные окружения от своего родительского процесса, обычно — от оболочки входа.

Посмотрите, какие из этих файлов запуска установлены. Помните — так как большинство предыдущих имен файлов начинаются с точки (то есть они скрыты), при использовании `ls` нам понадобится опция «-a».

Файл ~/.bashrc, вероятно, является самым важным конфигурационным файлом с точки зрения обычного пользователя, так как он почти всегда считывается. Неинтерактивные оболочки читают его по умолчанию, а большинство файлов запуска интерактивных оболочек написаны так, чтобы считывать и файл ~/.bashrc.

Что внутри файла запуска?

Если заглянуть внутрь типичного .bash_profile, он выглядит примерно так:

```
# .bash_profile
# Get the aliases and functions
if [ -f ~/.bashrc ]; then
    . ~/.bashrc
fi
# User specific environment and startup programs
```

```
PATH=$PATH:$HOME/bin
export PATH
```

Строки, начинающиеся с «#», являются комментариями и не выполняются оболочкой. Они служат для удобства чтения. Первое интересное происходит на четвертой строке, в следующем коде:

```
if [ -f ~/.bashrc ]; then
    . ~/.bashrc
fi
```

Это составная команда if, которую мы подробно разберем в Части 4, когда перейдем к написанию скриптов оболочки, но пока приведем перевод:

```
If the file "~/.bashrc" exists, then
    read the "~/.bashrc" file.
```

Мы видим, что этот фрагмент кода показывает, как оболочка входа получает содержимое .bashrc. Следующий элемент в нашем файле запуска связан с переменной PATH.

Вы когда-нибудь задумывались, как оболочка понимает, где искать команды, когда мы вводим их в командной строке? Например, когда мы вводим ls, оболочка не ищет /bin/ls (полный путь к команде ls) по всему компьютеру; вместо этого она ищет в списке каталогов, указанных в переменной PATH.

Переменная PATH часто (но не всегда, в зависимости от дистрибутива) устанавливается в файле запуска .bash_profile следующим кодом:

```
PATH=$PATH:$HOME/bin
```

PATH изменяется так, чтобы добавить каталог \$HOME/bin в конец списка. Это пример раскрытия параметров, который мы затрагивали в главе 7. Чтобы продемонстрировать, как это работает, попробуйте следующее:

```
[me@linuxbox ~]$ foo="This is some "
[me@linuxbox ~]$ echo $foo
This is some
[me@linuxbox ~]$ foo=$foo"text."
[me@linuxbox ~]$ echo $foo
This is some text.
```

Используя этот метод, мы можем добавлять текст в конец содержимого переменной.

Добавляя строку \$HOME/bin в конец содержимого переменной PATH, мы добавляем каталог \$HOME/bin в список каталогов,

просматриваемых при вводе команды. Это значит, что когда мы захотим создать в домашнем каталоге каталог для хранения своих личных программ, оболочка будет готова к работе. Нам достаточно просто вызвать его `bin`, и всё готово.

Примечание: Многие дистрибутивы предоставляют эту настройку `PATH` по умолчанию. Дистрибутивы на базе Debian, такие как Ubuntu, при входе в систему проверяют наличие каталога `~/bin` и динамически добавляют его в переменную `PATH`, если каталог найден.

Наконец, у нас есть:

```
export PATH
```

Команда `export` указывает оболочке сделать содержимое `PATH` доступным для дочерних процессов этой оболочки. В некотором смысле она преобразует переменную оболочки в переменную окружения.

Как дочерние процессы наследуют окружение

Этот последний момент заслуживает пояснения. Переменные оболочки локальны для текущего экземпляра оболочки и не копируются в дочерние процессы, которые она запускает. Давайте это продемонстрируем.

Сначала мы установим переменную оболочки в нашей текущей оболочке.

```
[me@linuxbox ~]$ foo="bar"
```

Далее мы запустим еще одну копию оболочки.

```
[me@linuxbox ~]$ bash
[me@linuxbox ~]$
```

Кажется, что ничего не произошло, но на самом деле мы запустили еще один экземпляр оболочки. Это можно подтвердить, изучив результат команды `ps`.

```
[me@linuxbox ~]$ ps
  PID TTY          TIME CMD
 1011638 pts/9        00:00:00 bash
 1011650 pts/9        00:00:00 bash
 1011662 pts/9        00:00:00 ps
```

Здесь видно, что запущено две копии `bash`. Поскольку при запуске мы не отправили новый экземпляр в фоновый режим, теперь он является активной задачей, а исходный экземпляр

ожидает завершения этой новой оболочки. Теперь попробуем посмотреть значение переменной `foo`, которую мы установили мгновение назад.

```
[me@linuxbox ~]$ echo $foo  
[me@linuxbox ~]$
```

Отсутствие результата означает, что переменная `foo` пуста.

Причина в том, что в этом экземпляре оболочки мы не присвоили ей значение. Переменные оболочки не копируются и не передаются дочернему процессу при его создании.

Переменные окружения, напротив, копируются и становятся окружением дочернего процесса.

Чтобы продемонстрировать еще одну интересную особенность процессов и их окружения, определим `foo` в этом экземпляре оболочки.

```
[me@linuxbox ~]$ foo="barbar"
```

Затем мы завершим этот `bash` экземпляр и вернемся к родительскому экземпляру, который терпеливо ждал завершения дочернего процесса, прежде чем продолжить работу, как и с любой другой программой, которую мы не запускали в фоновом режиме.

```
[me@linuxbox ~]$ exit  
[me@linuxbox ~]$
```

Мы снова запустим `ps`, чтобы убедиться, что вернулись в первый экземпляр.

```
[me@linuxbox ~]$ ps  
  PID TTY          TIME CMD  
 1011638 pts/9        00:00:00 bash  
 1014900 pts/9        00:00:00 ps
```

Теперь посмотрим на значение `foo` в этом экземпляре.

```
[me@linuxbox ~]$ echo $foo  
bar  
[me@linuxbox ~]$
```

Мы видим, что оно по-прежнему содержит заданное нами значение, а не новое, установленное в дочернем экземпляре. Это демонстрирует важное правило дочерних процессов: дочерний процесс не может изменить окружение своего родителя. Когда дочерний процесс завершается, он уносит свое окружение с собой. Этот факт станет важным, когда мы начнем писать сценарии оболочки.

Запуск программы с временным окружением

Еще один полезный прием, который предоставляет оболочка, — возможность выполнить команду с временной переменной окружения. Иногда нам нужно запустить программу с особым значением окружения. Хороший пример — команда `man`, которая ищет переменную окружения `MANWIDTH`, указывающую `man`, какой ширины форматировать вывод. Например, чтобы `man` форматировала вывод с максимальной шириной 75 символов (удобная настройка для чтения), можно сделать следующее:

```
[me@linuxbox ~]$ MANWIDTH=75 man ls
```

Это выводит `man`-страницу для команды `ls` в удобном формате и с оптимальной шириной. Кстати, это стоит добавить в `alias`.

```
[me@linuxbox ~]$ alias man='MANWIDTH=75 man'
```

Теперь при использовании команды `man` длина строки всегда будет ограничена 75 символами. Для настройки более сложных временных сред см. команду `env`.

Изменение окружения

Зная расположение и содержимое файлов запуска, мы можем изменить их для настройки среды под себя.

Какие файлы следует изменять?

Как правило, чтобы добавить каталоги в ваш `PATH` или определить дополнительные переменные окружения, внесите эти изменения в `.bash_profile` (или эквивалентный файл в вашем дистрибутиве; например, в Ubuntu используется `.profile`). Для всего остального вносите изменения в `.bashrc`.

Примечание: если вы не системный администратор, которому нужно изменить настройки по умолчанию для всех пользователей, ограничивайте изменения файлами в вашем домашнем каталоге. Конечно, можно изменять файлы в `/etc`, такие как `profile`, и во многих случаях это было бы разумно, но пока будем соблюдать осторожность.

Текстовые редакторы

Для редактирования (т. е. изменения) файлов запуска оболочки, а также большинства других конфигурационных файлов системы,

используется программа, называемая текстовым редактором. Текстовый редактор — это программа, которая в чем-то похожа на текстовый процессор, так как позволяет редактировать текст на экране с помощью перемещающегося курсора. Он отличается от текстового процессора тем, что поддерживает только чистый текст и часто содержит функции для написания программ. Текстовые редакторы — основной инструмент разработчиков для написания кода и системных администраторов для управления конфигурационными файлами системы.

Для Linux доступно множество различных текстовых редакторов; в большинстве систем установлено несколько штук. Почему их так много? Потому что программисты любят их писать, а поскольку они сами активно ими пользуются, то создают редакторы, чтобы воплотить свои представления о том, как те должны работать.

Текстовые редакторы делятся на две основные категории: графические и текстовые. GNOME и KDE включают в себя несколько популярных графических редакторов. В GNOME поставляется редактор под названием `gedit`, который в меню GNOME обычно называется «Text Editor». В KDE обычно поставляются два: `kwrite` и более продвинутый `kate`.

Существует множество текстовых редакторов. Мы часто будем встречать такие популярные, как `nano`, `vi` и `emacs`. Редактор `nano` — это простой и удобный инструмент, созданный как замена редактору `rioso`, входящему в пакет PINE. Редактор `vi` (который в большинстве систем Linux заменен программой `vim`, что означает «`vi improved`») является традиционным редактором для Unix-подобных систем. Он станет темой нашей следующей главы. Редактор `emacs` изначально был написан Ричардом Столлманом. Это гигантская универсальная среда программирования, умеющая всё. Несмотря на доступность, она редко предустановлена в большинстве систем Linux по умолчанию.

Использование текстового редактора

Текстовые редакторы запускаются из командной строки путем ввода имени редактора и имени файла, который нужно отредактировать. Если файл еще не существует, редактор решит, что вы хотите создать новый. Вот пример с использованием `gedit`:

```
[me@linuxbox ~]$ gedit some_file
```

Эта команда запустит текстовый редактор `gedit` и загрузит файл «`so me_file`», если он существует.

Графические текстовые редакторы интуитивно понятны, поэтому мы не будем их рассматривать. Вместо этого сосредоточимся на нашем первом текстовом редакторе — nano. Давайте запустим nano и отредактируем файл `.bashrc`. Но прежде применим принцип «безопасных вычислений». Когда вы редактируете важный конфигурационный файл, всегда полезно сначала создать его резервную копию. Это защитит вас, если вы что-то испортите при редактировании. Чтобы создать резервную копию файла `.bashrc`, сделайте следующее:

```
[me@linuxbox ~]$ cp .bashrc .bashrc.bak
```

Неважно, как назвать резервный файл; просто выберите понятное имя. Расширения `«.bak»`, `«.sav»`, `«.old»` и `«.orig»` — популярные способы обозначения резервной копии. И помните, что `cp` будет молча перезаписывать существующие файлы.

Теперь, когда резервная копия готова, мы запустим редактор.

```
[me@linuxbox ~]$ nano .bashrc
```

После запуска nano появится такой экран:

```
GNU nano 8.6          File: .bashrc
# .bashrc
# Source global definitions
if [ -f /etc/bashrc ]; then
    . /etc/bashrc
fi
# User specific aliases and functions
                                [ Read 8 lines ]
^G Get Help^O WriteOut^R Read Fil^Y Prev Pag^K Cut Text^C Cur Pos
^X Exit      ^J Justify ^W Where Is^V Next Pag^U UnCut Te^T To Spell
```

Примечание: если nano не установлен, можно использовать графический редактор.

Экран состоит из заголовка вверху, текста редактируемого файла в центре и меню команд внизу. Поскольку nano был разработан как замена текстовому редактору почтового клиента, функций редактирования в нем немного.

Первая команда, которую стоит выучить в любом текстовом редакторе, — это выход из программы. В случае с nano для выхода нажмите `Ctrl-x`. Это указано в меню в нижней части экрана. Обозначение `^X` означает `Ctrl-x`. Это общепринятое обозначение управляющих символов, используемое во многих программах.

Вторая важная команда — сохранение работы. В nano это делается с помощью Ctrl-o. Теперь мы готовы к редактированию. Используя стрелку вниз и/или PageDown, переместите курсор в конец файла, а затем добавьте следующие строки в файл .bashrc:

```
umask 0002
export HISTCONTROL=ignoredups
export HISTSIZE=1000
alias l='ls -d .* --color=auto'
alias ll='ls -l --color=auto'
```

Примечание: в вашем дистрибутиве некоторые из них могут уже быть установлены, но дубликаты не повредят.

В таблице 11-4 подробно описано значение наших дополнений:

Таблица 11-4: Дополнения к нашему .bashrc

Line	Значение
umask 0002	Устанавливает umask, чтобы решить проблему с общим каталоги, обсуждавшиеся в главе 9.
export HISTCONTROL=ignoredups	Заставляет оболочку игнорировать запись команды, если та же команда была только что записана.
export HISTSIZE=1000	Увеличивает размер истории команд с обычных 500 строк до 1 000 ,строк.
alias l='ls -d .* --color=auto'	Создает новую команду l., которая выводит все каталоги записи, начинающиеся с точки.
alias ll='ls -l --color=auto'	Создает новую команду ll, которая выводит подробный список каталогов.

Как видно, многие наши дополнения неочевидны, поэтому стоит добавить комментарии в `.bashrc` file, чтобы пояснить их человеку. С помощью редактора измените наши дополнения следующим образом:

```
# Change umask to make directory sharing easier
umask 0002
# Ignore duplicates in command history and increase
# history size to 1000 lines
export HISTCONTROL=ignoredups
export HISTSIZE=1000
# Add some helpful aliases
alias l='ls -d .* --color=auto'
alias ll='ls -l --color=auto'
```

Так-то лучше! Когда закончите изменения, нажмите `Ctrl-o`, чтобы сохранить измененный файл `.bashrc`, и `Ctrl-x`, чтобы выйти из nano.

Почему важны комментарии

При изменении конфигурационных файлов полезно добавлять комментарии для документирования правок. Возможно, вы вспомните, что изменили, завтра, но как насчет полугода спустя? Сделайте себе одолжение — добавьте комментарии. Заодно будет не лишним вести журнал внесенных изменений.

В скриптах оболочки и файлах `bash startup` символ «`#`» обозначает начало комментария. В других конфигурационных файлах могут использоваться иные символы. В большинстве таких файлов есть комментарии — используйте их как руководство.

В конфигурационных файлах часто встречаются закомментированные строки, чтобы программа их не использовала. Это делается для подсказок по возможным параметрам или примеров правильного синтаксиса. Например, файл `.bashrc` в Ubuntu 24.04 содержит такие строки:

```
# some more ls aliases #alias ll='ls -aLF' #alias la='ls -A' #alias
l='ls -CF'
```

Последние три строки — это закомментированные определения `alias`. Если убрать в начале этих строк символы «`#`» (метод под названием «раскомментирование»), `alias` активируются. И наоборот, если добавить символ «`#`» к началу строки, можно деактивировать строку конфигурации, сохранив содержащуюся в ней информацию.

Применение изменений

Изменения, внесенные в `.bashrc`, не вступят в силу, пока мы не закроем сессию терминала и не начнем новую, так как файл `.bashrc` считывается только в начале сессии. Однако мы можем заставить `bash` перечитать измененный файл `.bashrc` с помощью следующей команды:

```
[me@linuxbox ~]$ source ~/.bashrc
```

После этого изменения должны вступить в силу. Попробуйте один из новых алиасов.

```
[me@linuxbox ~]$ ll
```

Немного больше о `source`

Команда `source` (которую можно сократить до.) — это встроенная команда оболочки, которая считывает файл напрямую в текущую оболочку, как если бы его содержимое было введено с клавиатуры. Да, все те странные вещи, которые мы видели в файлах запуска оболочки, — это просто команды, которые оболочка понимает и может выполнять. Многие старые текстовые операционные системы (DOS, CP/M и др.) работали в основном как простые запускатели программ. Оболочки в стиле Unix, конечно, могут это делать, как мы уже видели, но они способны на гораздо большее.

Итоги

В этой главе мы освоили важный навык — редактирование конфигурационных файлов в текстовом редакторе. В дальнейшем, читая `man`-страницы команд, обращайтесь внимание на поддерживаемые ими переменные окружения. Там могут встретиться полезные сведения. В следующих главах мы изучим функции оболочки — мощную возможность, которую также можно добавить в `bash` startup файлы, чтобы пополнить свой арсенал пользовательских команд.

Что почитать

- Раздел `INVOCATION` в `bash` `man`-странице подробно описывает `bash` startup файлы.

12 — Мягкое введение в *vi*(*m*)

Есть старая шутка о посетителе Нью-Йорка, который спрашивает прохожего, как пройти к знаменитому залу классической музыки:

Посетитель: Извините, как мне добраться до Carnegie Hall?

Прохожий: Практикуйтесь, практикуйтесь и еще раз практикуйтесь!

Освоение командной строки Linux, как и становление виртуозным пианистом, — это не то, чему можно научиться за один вечер. Это требует многих лет практики. В этой главе мы познакомимся с текстовым редактором *vi* (произносится как «ви ай»), одной из основных программ в традиции Unix. *vi* печально известен своим уникальным пользовательским интерфейсом, но когда мы увидим, как мастер садится за клавиатуру и начинает «играть», мы действительно станем свидетелями настоящего искусства. В этой главе мы не станем мастерами, но по окончании мы будем знать, как «сыграть на палочках» в *vi*.

Почему стоит изучать *vi*

В современную эпоху графических редакторов и простых текстовых редакторов вроде *nano*, зачем нам учить *vi*? На это есть три веские причины.

- *vi* доступен почти всегда. Это может спасти ситуацию, если у нас нет графического интерфейса — например, на удаленном сервере или локальной системе с неисправной конфигурацией GUI. *nano*, хоть и становится популярнее, все еще не универсален. Стандарт POSIX, обеспечивающий совместимость программ в Unix-системах, требует наличия *vi*.
- *vi* легок и быстр. Для многих задач проще запустить *vi*, чем искать графический редактор в меню и ждать загрузки его многомегабайтных файлов. Кроме того, *vi* спроектирован для высокой скорости набора. Как мы увидим, опытным пользователям *vi* при редактировании даже не нужно отрывать пальцы от клавиатуры.
- Мы не хотим, чтобы другие пользователи Linux и Unix считали нас трусами.

Что ж, возможно, две веские причины.

Немного истории

Первая версия `vi` была написана в 1976 году Биллом Джоем, студентом Калифорнийского университета в Беркли, который позже стал сооснователем Sun Microsystems. Название `vi` происходит от слова «visual» («визуальный»), так как редактор предназначался для работы на видеотерминале с перемещаемым курсором. До появления визуальных редакторов существовали строчные редакторы, работавшие с одной строкой текста за раз. Чтобы внести изменения, мы указываем строчному редактору номер строки и описываем действие, например, добавление или удаление текста. С появлением видеотерминалов (в отличие от телетайпов, работавших на печатных устройствах) стала возможна визуальная правка. `vi` фактически включает в себя мощный строчный редактор `ex`, и мы можем использовать команды строчного редактирования при работе в `vi`.

Большинство дистрибутивов Linux не включают настоящий `vi`; вместо этого они поставляются с улучшенной версией под названием `vim` (сокращение от «`vi improved`»), написанной Bram Moolenaar. `vim` — это значительное улучшение традиционного Unix `vi`, и в системах Linux он часто связан символьной ссылкой (или алиасом) с именем `vi`. В последующих обсуждениях мы будем исходить из того, что у нас есть программа `vi`, которая на самом деле является `vim`.

Запуск и завершение `vi`

Чтобы запустить `vi`, просто введите следующее:

```
[me@linuxbox ~]$ vi
```

Должен появиться такой экран:

```
~
~
~              VIM - Vi Improved
~
~              version 8.0.707
~              by Bram Moolenaar et al.
~      Vim is open source and freely distributable
~
~      Sponsor Vim development!
~      type  :help sponsor<Enter>    for information
~
~      type  :q<Enter>                to exit
~      type  :help<Enter> or <F1>    for on-line help
~      type  :help version8<Enter>   for version info
~
~      Running in Vi compatible mode
~      type  :set nocp<Enter>        for Vim defaults
~      type  :help cp-default<Enter> for info on this
```

```
~  
~  
~
```

Как и в случае с nano ранее, первым делом нужно научиться выходить. Чтобы выйти, введите следующую команду (обратите внимание, что символ двоеточия является частью команды):

```
:q
```

Должно появиться приглашение оболочки. Если по какой-то причине vi не завершается (обычно из-за несохраненных изменений в файле), мы можем подтвердить свое намерение, добавив восклицательный знак к команде для vi.

```
:q!
```

Совет: если вы «заблудились» в vi, попробуйте дважды нажать Esc, чтобы вернуться в привычное состояние.

Режим совместимости

На примере экрана запуска выше виден текст «Running in Vi compatible mode». Это означает, что vim будет работать в режиме, более близком к обычному поведению vi, чем к расширенным возможностям vim. В этой главе мы будем использовать vim с расширенными возможностями. Для этого есть несколько способов. Попробуйте запустить vim alias vi='vim' vi.bashrc вместо. Если это работает, добавьте это в свой файл. Либо используйте эту команду, чтобы добавить строку в конфигурационный файл vim:

```
echo "set nosp" >> ~/.vimrc
```

Разные дистрибутивы Linux по-разному упаковывают vim. Некоторые по умолчанию устанавливают минимальную версию vim (vim-tiny), которая поддерживает только ограниченный набор функций vim. При выполнении следующих уроков вы можете столкнуться с отсутствием некоторых функций. В этом случае установите полную версию vim.

Режимы редактирования

Давайте снова запустим vi, на этот раз передав имя несуществующего файла. Так можно создать новый файл с vi:

```
[me@linuxbox ~]$ rm -f foo.txt
[me@linuxbox ~]$ vi foo.txt
```

Если всё пройдет успешно, мы увидим такой экран:

Символы тильды (~) в начале строк указывают на отсутствие текста. Это значит, что файл пуст. Пока ничего не печатайте! Второе по важности (после умения выходить) в изучении vi — это понимание того, что vi является модальным редактором. При запуске vi открывается в обычном режиме. В этом режиме почти каждая клавиша — это команда, поэтому при попытке набора текста vi начнет вести себя непредсказуемо и создаст хаос.

Вход в режим вставки

Чтобы добавить текст в файл, нужно войти в режим вставки. Для этого введите `i`. Если `vim` запущен в обычном расширенном режиме (в режиме совместимости с `vi` этого не будет), внизу экрана появится следующее:

```
-- INSERT --
```

Теперь можно вводить текст. Попробуйте следующее:

The quick brown fox jumps over the lazy dog.

Чтобы выйти из режима вставки и вернуться в командный режим, нажмите Esc.

Сохранение работы

Чтобы сохранить внесенные изменения, нужно перейти в командный режим. Для этого в нормальном режиме нажмите клавишу `:`. После этого в нижней части экрана появится символ двоеточия.

```
:
```

Чтобы записать измененный файл, после двоеточия введите `w`, а затем нажмите ENTER.

```
:w
```

Файл будет записан на жесткий диск, а внизу экрана появится подтверждение, например:

```
"foo.txt" [New] 1L, 45C written
```

Примечание: хотя `vim` выделяет три основных режима редактирования — командный, режим вставки и командный, в реальности `vi` (и его документация) называет их соответственно `command`, `insert` и `ex`. Многие онлайн-ресурсы по `vi` используют именно такие названия, что может сбивать с толку.

Перемещение курсора

В обычном режиме `vi` предлагает множество команд перемещения, некоторые из которых она разделяет с `less`. В таблице 12-1 приведен их подмножество.

Таблица 12-1: Клавиши перемещения курсора

Key	Перемещает курсор
<code>l</code> или стрелка вправо	На один символ вправо.
<code>h</code> или стрелка влево	На один символ влево.
<code>j</code> или стрелка вниз	На одну строку вниз.
<code>k</code> или стрелка вверх	На одну строку вверх.
<code>0</code> (ноль)	В начало текущей строки.

Key	Перемещает курсор
<code>^</code>	К первому символу, не являющемуся пробелом, в текущей line.
<code>\$</code>	В конец текущей строки.
<code>w</code>	К началу следующего слова или знака символ.
<code>W</code>	К началу следующего слова, игнорируя знаки пунктуации.
<code>b</code>	К началу предыдущего слова или знака символ.
<code>B</code>	К началу предыдущего слова, игнорируя знаки пунктуации.
<code>Ctrl-f</code> или <code>Page Down</code>	На одну страницу вниз.
<code>Ctrl-b</code> или <code>Page Up</code>	На одну страницу вверх.
<code>numberG</code>	К номеру строки. Например, <code>1G</code> перемещает на первую строку файла.

Key	Перемещает курсор
<code>G</code>	К последней строке файла.

Почему для перемещения курсора используются `h`, `j`, `k` и `l`? Когда `vi` только создавался, не все видеотерминалы имели клавиши со стрелками, и опытные машинистки могли перемещать курсор обычными клавишами, не отрывая рук от клавиатуры.

Многие команды в `vi` можно дополнять числом, как в случае с командой `G`, перечисленной ранее. Указывая число перед командой, мы задаем количество её повторений. Например, команда `5j` заставляет `vi` переместить курсор на пять строк вниз.

Основы редактирования

Редактирование в основном состоит из нескольких базовых операций: вставки, удаления и перемещения текста методом вырезания и вставки. `vi`, разумеется, поддерживает все эти операции в своей уникальной манере. `vi` также предоставляет ограниченную возможность отмены действий. Если в нормальном режиме ввести `u`, `vi` отменит последнее изменение. Это пригодится при изучении базовых команд редактирования.

Добавление текста

`vi` имеет несколько способов перехода в режим вставки. Мы уже использовали команду `i` для ввода текста.

Давайте ненадолго вернемся к нашему `foo.txt` файлу.

```
The quick brown fox jumps over the lazy dog.
```

Если мы захотим добавить текст в конец этого предложения, то обнаружим, что команда `i` этого не сделает, так как мы не можем переместить курсор за пределы строки. `vi` предоставляет команду для добавления текста — логично названную команду `a`. Если переместить курсор в конец строки и набрать `a`, курсор выйдет за пределы строки, и `vi` перейдет в режим вставки. Это позволит нам добавить еще немного текста.

```
The quick brown fox jumps over the lazy dog. It was cool.
```

Не забывайте нажимать `Esc`, чтобы выйти из режима вставки.

Поскольку нам почти всегда нужно добавлять текст в конец строки, `vi` предлагает сочетание клавиш для перехода в конец текущей строки и начала ввода. Это команда `A`. Давайте попробуем её и добавим в наш файл ещё несколько строк.

Сначала переместим курсор в начало строки с помощью команды «`0`» (ноль). Теперь вводим `A` и добавляем следующие строки текста:

```
The quick brown fox jumps over the lazy dog. It was cool.  
Line 2  
Line 3  
Line 4  
Line 5
```

Снова нажмите `Esc`, чтобы выйти из режима вставки.

Как видно, команда `A` полезнее, так как она перемещает курсор в конец строки перед переходом в режим вставки.

Открытие строки

Еще один способ вставки текста — «открытие» строки. Это вставляет пустую строку между двумя существующими и переходит в режим вставки. Существует два варианта, описанных в таблице 12-2.

Таблица 12-2: Клавиши открытия строк

Команда	Открывает
---------	-----------

Команда	Открывает
o	Строку ниже текущей строки
O	Строку выше текущей строки

WMы можем продемонстрировать это так: установите курсор на строке 3 и введите o.

```
The quick brown fox jumps over the lazy dog. It was cool.
Line 2
Line 3
Line 4
Line 5
```

Ниже третьей строки открылась новая, и мы вошли в режим вставки. Выйдите из режима вставки, нажав Esc. Нажмите u, чтобы отменить изменения.

Введите O, чтобы открыть строку над курсором.

```
The quick brown fox jumps over the lazy dog. It was cool.
Line 2
Line 3
Line 4
Line 5
```

Выйдите из режима вставки, нажав Esc, и отмените изменения, нажав u.

Удаление текста

Как и ожидалось, vi предлагает множество способов удаления текста, каждый из которых использует одно из двух сочетаний клавиш. Сначала команда x удалит символ в позиции курсора. Перед x можно поставить число, указывающее количество удаляемых символов. Команда d более универсальна. Как и x, перед ней можно указать число повторений операции удаления. Кроме того, за d всегда следует команда перемещения, определяющая объем удаления. Примеры приведены в таблице 12-3.

Таблица 12-3: Команды удаления текста

Команда	Удаляет
x	Текущий символ
3x	Текущий символ и два следующих
dd	Текущую строку
5dd	Текущую строку и четыре следующие

Команда	Удаляет
dW	От текущей позиции курсора до начала следующего слова
d\$	От текущего положения курсора до конца текущей строки
d0	От текущего положения курсора до начала строки
d^	От текущего положения курсора до первого непобельный символ в строке
Команда	дактирования Удаляет
dG	От текущей строки до конца файла
d20G	От текущей строки до двадцатой строки файла

Установите курсор на слово `It` в первой строке нашего текста. Нажимайте `x` до тех пор, пока не удалите остаток предложения. Затем нажимайте `u`, пока удаление не будет отменено.

Примечание: Реальный `vi` поддерживает только один уровень отмены. `vim` поддерживает несколько уровней.

Давайте снова попробуем удаление, на этот раз с помощью команды `d`. Снова переместите курсор к слову `It` и введите `dW`, чтобы удалить его.

```
The quick brown fox jumps over the lazy dog. was cool.
Line 2
Line 3
Line 4
Line 5
```

Введите `d$`, чтобы удалить текст от позиции курсора до конца строки.

```
The quick brown fox jumps over the lazy dog.
Line 2
Line 3
Line 4
Line 5
```

Введите `dG`, чтобы удалить текст от текущей строки до конца файла.

```
~
~
~
```

~
~

Введите `u` трижды, чтобы отменить удаление.

Вырезание, копирование и вставка текста

Команда `d` не только удаляет текст, но и «вырезает» его. При каждом использовании `d` удаленный текст копируется в буфер обмена, который можно затем вставить после курсора с помощью `p` или перед курсором с помощью `P`.

Команда `y` используется для «`yank`» (копирования) текста почти так же, как команда `d` используется для вырезания. В таблице 12-4 приведены примеры сочетания команды `y` с различными командами перемещения.

Таблица 12-4: Команды копирования

Команда	Копирование
<code>yy</code>	Текущая строка
<code>5yy</code>	Текущая строка и следующие четыре строки
<code>yW</code>	От текущей позиции курсора до начала следующего слова
<code>y\$</code>	От текущей позиции курсора до конца текущей строки
<code>y0</code>	От текущего положения курсора до начала строки
<code>y^</code>	От текущего положения курсора до первого пробельного символа в строке
<code>yG</code>	От текущей строки до конца файла
<code>y20G</code>	От текущей строки до двадцатой строки файла

Давайте попробуем копирование и вставку. Установите курсор на первую строку текста и введите `yy`, чтобы скопировать текущую строку. Затем переместите курсор на последнюю строку (`G`) и введите `p`, чтобы вставить строку под текущей.

```
The quick brown fox jumps over the lazy dog. It was cool.  
Line 2  
Line 3  
Line 4  
Line 5  
The quick brown fox jumps over the lazy dog. It was cool.
```

Как и прежде, команда `u` отменит изменения. Оставив курсор на последней строке файла, введите `P`, чтобы вставить текст над

текущей строкой.

```
The quick brown fox jumps over the lazy dog. It was cool.  
Line 2  
Line 3  
Line 4  
The quick brown fox jumps over the lazy dog. It was cool.  
Line 5
```

Попробуйте другие команды у из таблицы 12-4, чтобы изучить поведение команд `r` и `R`. По завершении верните файл в исходное состояние.

Объединение строк

`vi` довольно строго относится к понятию строки. Обычно нельзя переместить курсор в конец строки и удалить символ конца строки, чтобы объединить её со следующей. Поэтому `vi` предоставляет специальную команду `J` (не путайте с `j`, которая служит для перемещения курсора), чтобы объединять строки.

Если поместить курсор на строку 3 и ввести команду `J`, произойдет следующее:

```
The quick brown fox jumps over the lazy dog. It was cool.  
Line 2  
Line 3 Line 4  
Line 5
```

Поиск и замена

`vi` умеет перемещать курсор в места, найденные при поиске. Это можно делать как в одной строке, так и во всем файле. Также доступна замена `te` кста с подтверждением пользователя или без него.

Поиск внутри строки

Команда `f` ищет символ в строке и перемещает курсор к следующему его вхождению. Например, команда `fa` переместит курсор к следующему символу `a` в текущей строке. После поиска символа в строке поиск можно повторить, введя точку с запятой.

Поиск по всему файлу

Для перемещения курсора к следующему вхождению слова или фразы используется команда `/`. Она работает так же, как мы изучили ранее в программе `less`. При вводе команды `/` в нижней части экрана появится `/`. Затем введите искомое слово или фразу и нажмите `ENTER`. Курсор переместится к следующему месту,

содержащему строку поиска. Поиск можно повторить с предыдущей строкой с помощью команды `n`. Вот пример:

```
The quick brown fox jumps over the lazy dog. It was cool.  
Line 2  
Line 3  
Line 4  
Line 5
```

Установите курсор на первую строку файла. Введите следующее и ENTER:

```
/Line
```

Курсор переместится на строку 2. Затем введите `n`, и курсор переместится на строку 3. Повторение команды `n` будет перемещать курсор вниз по файлу, пока совпадения не закончатся. До сих пор мы использовали для поиска только слова и фразы, но `vi` позволяет использовать регулярные выражения — мощный метод описания сложных текстовых шаблонов. Мы подробно разберем регулярные выражения в главе 19.

Глобальный поиск и замена

`vi` использует командный режим для выполнения операций поиска и замены (называемых подстановкой в `vi`) в диапазоне строк или во всем файле. Чтобы заменить слово `Line` на `line` во всем файле, нужно ввести следующую команду:

```
:%s/Line/line/g
```

Разберем эту команду по частям и посмотрим, что делает каждая из них (см. таблицу 12-5).

Таблица 12-5: Пример синтаксиса глобального поиска и замены

Item	Значение
:	Символ двоеточия переводит в режим команд.
%	Это задает диапазон строк для операции.% — это сокращение, означающее «от первой до последней строки». Или же диапазон можно было указать как 1,5 (так как в нашем файле пять строк) или 1,\$, что означает «от строки 1 до последней строки в файле». Если диапазон строк не указан, операция выполняется только для текущей строки.

Item	Значение
s	Это указывает на операцию. В данном случае — замена (поиск и замена).
/Line/line/	Здесь указываются шаблон поиска и текст замены.
g	Это означает «глобально» в том смысле, что поиск и замена выполняется для каждого вхождения строки поиска в строке. Если опущено, заменяется только первое вхождение строки поиска в каждой строке, которое будет

После выполнения команды поиска и замены файл выглядит так:

```
The quick brown fox jumps over the lazy dog. It was cool.
line 2
line 3
line 4
line 5
```

Мы также можем указать команду замены с подтверждением пользователем. Для этого добавьте s в конец команды. Вот пример:

```
:%s/line/Line/gc
```

Эта команда вернет файл в прежний вид; однако перед каждой заменой vi останавливается и просит подтвердить действие следующим сообщением:

```
replace with Line (y/n/a/q/l/^E/^Y)?
```

Каждый символ в скобках является возможным вариантом, как описано в таблице 12-6.

Таблица 12-6: Клавиши подтверждения за меня

Key	Действие
y	Выполнить замену.
n	Пропустить этот экземпляр шаблона.
a	Выполнить замену для этого и всех последующих экземпляров шаблона.
q или Esc	Прекратить замену.
l	Выполнить замену и выйти. Это сокращение для «last».

Key	Действие
Ctrl-e,	Прокрутка вниз и вверх соответственно. Это полезно
Ctrl-y	для просмотра контекста предлагаемой замены.

Если вы введете `u`, произойдет подстановка, а `n` заставит `vi` пропустить этот экземпляр и перейти к следующему.

Редактирование нескольких файлов

Часто бывает полезно редактировать сразу несколько файлов. Вам может потребоваться внести изменения в разные файлы или скопировать содержимое одного файла в другой. С помощью `vi` мы можем открыть несколько файлов для редактирования, указав их в командной строке.

```
vi file1 file2 file3...
```

Давайте выйдем из текущей сессии `vi` и создадим новый файл для редактирования. Введите `:wq`, чтобы выйти из `vi`, сохранив измененный текст. Затем мы создадим в домашнем каталоге дополнительный файл для практики. Мы создадим его, перенаправив вывод команды `ls`.

```
[me@linuxbox ~]$ ls -l /usr/bin > ls-output.txt
```

Давайте отредактируем старый и новый файлы с помощью `vi`.

```
[me@linuxbox ~]$ vi foo.txt ls-output.txt
```

`vi` запустится, и мы увидим первый файл на экране.

```
The quick brown fox jumps over the lazy dog. It was cool.
Line 2
Line 3
Line 4
Line 5
```

Переключение между файлами

Чтобы перейти от одного файла к следующему, используйте команду `ex`:

```
:bn
```

Чтобы вернуться к предыдущему файлу, используйте следующее:

```
:bp
```


Хотя мы можем переключаться между файлами, `vi` применяет политику, которая запрещает переключение, если в текущем файле есть несохраненные изменения. Чтобы принудительно заставить `vi` переключить файл и отбросить изменения, добавьте восклицательный знак (!) к команде.

В дополнение к описанному выше способу, `vim` (и некоторые версии `vi`) предоставляет команды командного режима, упрощающие работу с несколькими файлами. Мы можем просмотреть список редактируемых файлов с помощью команды `:buffers`. Это отобразит список файлов в нижней части экрана.

```
:buffers
1 %a "foo.txt" line 1
2 "ls-output.txt" line 0
Press ENTER or type command to continue
```

Чтобы переключиться на другой буфер (файл), введите `:buffer` (для ускорения можно использовать сокращение `:b`), а затем номер буфера, который нужно редактировать. Например, чтобы переключиться с буфера 1 с файлом `foo.txt` на буфер 2 с файлом `ls-output.txt`, введите следующее:

```
:buffer 2
```

Теперь на экране отображается второй файл. Еще один способ смены буферов — использование команд `:bn` (сокращение от «buffer next») и `:bp` (сокращение от «buffer previous»), упомянутых ранее.

Открытие дополнительных файлов для редактирования

В текущую сессию редактирования также можно добавить файлы. Команда командного режима `:e` (сокращение от «edit»), за которой следует имя файла, откроет дополнительный файл. Завершив текущую сессию и вернемся в командную строку.

Снова запустите `vi` только с одним файлом.

```
[me@linuxbox ~]$ vi foo.txt
```

Чтобы добавить второй файл, введите следующее:

```
:e ls-output.txt
```

Файл должен появиться на экране. Первый файл всё еще открыт, что можно проверить следующим образом:

```
:buffers
1 # "foo.txt" line 1
```

```
2 %a "ls-output.txt" line 0
Press ENTER or type command to continue
```

Копирование содержимого из одного файла в другой

Часто при редактировании нескольких файлов возникает необходимость скопировать часть одного файла в другой. Это легко сделать с помощью привычных команд копирования и вставки, которые мы разбирали ранее. Продемонстрируем это следующим образом. Сначала, используя наши два файла, переключитесь на буфер 1 (`foo.txt`)), введя:

```
:buffer 1
```

В результате должно получиться следующее:

```
The quick brown fox jumps over the lazy dog. It was cool.
Line 2
Line 3
Line 4
Line 5
```

Затем переместите курсор на первую строку и введите `yy`, чтобы скопировать (`yank`) строку.

Переключитесь на второй буфер, введя следующее:

```
:buffer 2
```

Теперь на экране отобразится список файлов, подобный этому (здесь показана лишь часть):

```
total 343700
-rwxr-xr-x 1 root root      31316 2024-12-05 08:58 [
-rwxr-xr-x 1 root root      8240 2024-12-09 13:39 411toppm
-rwxr-xr-x 1 root root    111276 2025-01-31 13:36 a2p
-rwxr-xr-x 1 root root    25368 2023-10-06 20:16 a52dec
-rwxr-xr-x 1 root root    11532 2024-05-04 17:43 aafire
-rwxr-xr-x 1 root root      7292 2024-05-04 17:43 aainfo
```

Переместите курсор на первую строку и вставьте строку, скопированную из предыдущего файла, введя команду `p`.

```
total 343700
The quick brown fox jumps over the lazy dog. It was cool.
-rwxr-xr-x 1 root root      31316 2024-12-05 08:58 [
-rwxr-xr-x 1 root root      8240 2024-12-09 13:39 411toppm
-rwxr-xr-x 1 root root    111276 2025-01-31 13:36 a2p
-rwxr-xr-x 1 root root    25368 2023-10-06 20:16 a52dec
-rwxr-xr-x 1 root root    11532 2024-05-04 17:43 aafire
-rwxr-xr-x 1 root root      7292 2024-05-04 17:43 aainfo
```

Вставка целого файла в другой

Также можно вставить содержимое целого файла в тот, который вы редактируете. Чтобы увидеть это в действии, завершим нашу

vi сессию и начнем новую, работая только с одним файлом.

```
[me@linuxbox ~]$ vi ls-output.txt
```

Мы снова увидим список файлов.

```
total 343700
-rwxr-xr-x 1 root root      31316 2024-12-05 08:58 [
-rwxr-xr-x 1 root root      8240 2024-12-09 13:39 411toppm
-rwxr-xr-x 1 root root    111276 2025-01-31 13:36 a2p
-rwxr-xr-x 1 root root    25368 2023-10-06 20:16 a52dec
-rwxr-xr-x 1 root root    11532 2024-05-04 17:43 aafire
-rwxr-xr-x 1 root root      7292 2024-05-04 17:43 aainfo
```

Переместите курсор на третью строку, а затем введите следующую команду в командном режиме:

```
:r foo.txt
```

Команда `:r` (сокращение от «read») вставляет указанный файл под позицией курсора. Теперь экран должен выглядеть так:

```
total 343700
-rwxr-xr-x 1 root root      31316 2024-12-05 08:58 [
-rwxr-xr-x 1 root root      8240 2024-12-09 13:39 411toppm
The quick brown fox jumps over the lazy dog. It was cool.
Line 2
Line 3
Line 4
Line 5
-rwxr-xr-x 1 root root    111276 2025-01-31 13:36 a2p
-rwxr-xr-x 1 root root    25368 2023-10-06 20:16 a52dec
-rwxr-xr-x 1 root root    11532 2024-05-04 17:43 aafire
-rwxr-xr-x 1 root root      7292 2024-05-04 17:43 aainfo
```

Сохранение работы

Как и во всем остальном в `vi`, существует несколько способов сохранения отредактированных файлов. Мы уже рассмотрели команду `:w`, но есть и другие, которые могут оказаться полезными.

В обычном режиме ввод `ZZ` сохранит текущий файл и выйдет из `vi`. Аналогично, команда в режиме команд `:wq` объединит команды `:w` и `:q` в одну, которая и сохранит файл, и выйдет.

Команда `:w` может также принимать необязательное имя файла. Это работает как «Сохранить как...». Например, если бы мы редактировали `foo.txt` и хотели сохранить альтернативную версию под именем `foo1.txt`, мы бы ввели следующее:

```
:w foo1.txt
```

Примечание: хотя эта команда сохраняет файл под новым именем, она не меняет имя редактируемого файла. При дальнейшем редактировании мы всё еще будем работать с `foo.txt`, а не с `foo1.txt`.

Bash поддерживает и vi

В главе 8 мы рассмотрели различные способы редактирования содержимого командной строки. Команды редактирования, которые использует `bash`, выбраны не случайно. Они вдохновлены текстовым редактором `emacs`. Это поведение по умолчанию в `bash`, но `bash` также поддерживает редактирование командной строки в стиле `vi`. Эту функцию легко активировать следующей командой:

```
[me@linuxbox ~]$ set -o vi
```

После этого мы сможем использовать многие изученные команды редактирования в стиле `vi`. Давайте попробуем. Введите следующий пример текста в приглашении командной строки:

```
[me@linuxbox ~]$ the quick brown fox jumps over the lazy dog
```

Как и прежде, мы можем перемещать курсор клавишами со стрелками и вводить символы обычным способом. Это происходит потому, что при запуске новой командной строки редактор находится в режиме вставки и ведет себя так же, как `vim`. Чтобы перейти к полезным функциям, нужно переключиться в командный режим. Выйти из режима вставки можно нажатием `ESC`. Все команды перемещения, копирования, удаления и вставки работают так же, как при редактировании однострочного текстового файла в `vim`. Чтобы вернуться в режим вставки, используйте соответствующую команду командного режима, например `i` или `A`.

Настройка `bash` для редактирования командной строки в стиле `vi` поможет закрепить навыки работы с клавишами `vi`, а также сократит число запоминаемых команд редактирования. Попробуйте это. Чтобы сохранить настройку, добавьте команду `set -o vi` в наш файл `.bashrc`.

Чтобы вернуться к режиму редактирования в стиле `emacs`, введите эту команду:

```
[me@linuxbox ~]$ set -o emacs
```

Примечание: по этой функции доступно множество онлайн-руководств, но имейте в виду, что в большинстве из них используются традиционные названия режимов `vi` (`command`, `insert` и `ex`), а не `vim` (`normal`, `insert` и `command`).

Итоги

Обладая этим базовым набором навыков, мы можем выполнять большинство операций по редактированию текста, необходимых для обслуживания типичной системы Linux. Регулярное использование `vim` окупится в долгосрочной перспективе.

Поскольку редакторы в стиле `vi` глубоко укоренены в культуре Unix, мы встретим множество других программ, созданных под влиянием его дизайна. `less` — хороший пример такого влияния.

Что почитать

Даже после всего изученного в этой главе мы лишь слегка коснулись возможностей `vim`. Вот несколько онлайн-ресурсов, которые помогут вам на пути к мастерству в `vi`:

- Vim, with Vigor — продолжение этого руководства на [LinuxCommand.org](https://linuxcommand.org/lc3_adv_vimvigor.php), которое выводит читателя на средний уровень владения. Оно доступно по адресу: https://linuxcommand.org/lc3_adv_vimvigor.php
- Learning The vi Editor — Wikibook от Wikipedia, предлагающий краткое руководство по `vi` и ряду аналогичных редакторов, включая `vim`. Доступно по адресу: https://en.wikibooks.org/wiki/Learning_the_vi_Editor
- Статья в Wikipedia о Билле Джое, создателе `vi`: https://en.wikipedia.org/wiki/Bill_Joy
- Статья в Wikipedia о Бrame Муленааре, авторе `vim`: https://en.wikipedia.org/wiki/Bram_Moolenaar

13 — Настройка приглашения

В этой главе мы рассмотрим, казалось бы, незначительную деталь — наше приглашение оболочки. Этот разбор раскроет некоторые внутренние механизмы работы оболочки и программы эмулятор терминала.

Как и многое другое в Linux, приглашение оболочки обладает гибкими настройками. Мы привыкли воспринимать его как должное, но, научившись им управлять, вы увидите в нем крайне полезный инструмент.

Анатомия приглашения

Наше приглашение по умолчанию выглядит примерно так:

```
[me@linuxbox ~]$
```

Заметьте, что оно содержит имя пользователя, имя хоста и текущий рабочий каталог. Как же оно так устроено? Все очень просто. Приглашение определяется переменной окружения PS1 (сокращение от «prompt string 1»). Мы можем просмотреть содержимое PS1 с помощью команды `echo`.

```
[me@linuxbox ~]$ echo $PS1
[\u@\h \w]\$
```

Примечание: не беспокойтесь, если ваши результаты отличаются от примера выше. Каждый дистрибутив Linux определяет строку приглашения немного иначе, иногда весьма экзотично.

Из результатов видно, что PS1 содержит некоторые символы из нашего приглашения, такие как скобки, знак «@» и знак доллара, но остальные остаются загадкой. Проницательные заметят, что это специальные символы с экранированием обратным слешем, как в главе 7. В таблице 13-1 приведен частичный список символов, которые `bash` обрабатываются особым образом в строке приглашения.

Таблица 13-1: Коды escape в приглашениях оболочки ов.

Последовательность	Отображаемое значение
<code>\a</code>	ASCII bell. Заставляет компьютер издавать звуковой сигнал при встрече с ним.

Последовательность	Отображаемое значение
\d	Текущая дата в формате день, месяц, число. Например, «Пн мая 26.»
\e	ASCII-символ перевода (033).
\h	Имя хоста локальной машины без конечного доменного имени.
\H	Полное имя хоста.
\j	Количество задач, выполняемых в текущей сессии оболочки.
\l	Имя текущего терминального устройства.
\n	Символ новой строки.
\r	Возврат каретки.
\s	Имя программы оболочки.
\t	Текущее время в формате 24-часового ЧЧ:ММ:СС.
\T	Текущее время в 12-часовом формате.
\@	Текущее время в 12-часовом формате АМ/РМ.
\A	Текущее время в формате 24-часового ЧЧ:ММ.
\u	Имя текущего пользователя.
\v	Номер версии оболочки.
\V	Номер версии и релиза оболочки.
\w	Имя текущего рабочего каталога.
\W	Последняя часть имени текущего рабочего каталога.
\!	Порядковый номер текущей команды в истории.
\#	Количество команд, введенных в течение этой сессии оболочки.
\\$	Это отображает символ «\$», если у нас нет привилегий суперпользователя. В этом случае вместо него отображается «#».

Последовательность	Отображаемое значение
\[	Сигнализирует о начале последовательности из одного или нескольких непечатных символов. Это используется для вставки непечатных управляющих символов, которые каким-либо образом управляют эмулятором терминала, например, перемещают
	курсора или смены цвета текста. Это необходимо для правильного расчета длины приглашения для корректного позиционирования курсора.
\]	Сигнализирует об окончании последовательности непечатных символов.

Попытка использования альтернативных вариантов приглашения

Используя этот список специальных символов, мы можем изменить приглашение и увидеть результат. Сначала мы сделаем резервную копию текущей строки приглашения, чтобы иметь возможность восстановить её позже. Для этого мы скопируем существующую строку в другую оболочку переменную, которую создадим сами.

```
[me@linuxbox ~]$ ps1_old="$PS1"
```

Мы создаем новую переменную `ps1_old` и присваиваем ей значение `PS1`. Мы можем проверить, что строка скопирована, с помощью команды `echo`.

```
[me@linuxbox ~]$ echo $ps1_old
[\u@\h \w]\$
```

Мы можем восстановить исходное приглашение в любой момент сеанса терминала, просто выполнив процесс в обратном порядке.

```
[me@linuxbox ~]$ PS1="$ps1_old"
```

Теперь, когда мы готовы, посмотрим, что произойдет, если строка приглашения будет пустой.

```
[me@linuxbox ~]$ PS1=
```

Если не присваивать строке приглашения никаких значений, мы не получим ничего. Вообще никакой строки приглашения! Само приглашение остается, но ничего не отображает, как мы и просили. Поскольку это выглядит довольно странно, мы заменим его минимальным приглашением.

```
PS1="\$ "
```

Так лучше. По крайней мере, теперь мы видим, что делаем. Обратите внимание на пробел внутри двойных кавычек. Он обеспечивает отступ между знаком доллара и курсором при отображении приглашения.

Добавим звуковой сигнал в наше приглашение.

```
$ PS1="\[\a\]\$ "
```

Теперь при каждом появлении приглашения мы должны слышать звуковой сигнал, хотя в некоторых системах эту «функцию» отключают. Это может раздражать, но полезно, если нужно получить уведомление о завершении особенно долгой команды. Обратите внимание, что мы включили последовательности `\[` и `\]`. Поскольку ASCII-сигнал (`\a`) не «печатается» (то есть не перемещает курсор), нам нужно сообщить об этом `bash`, чтобы оболочка могла правильно определить длину приглашения.

Далее попробуем создать информативное приглашение с именем хоста и временем суток.

```
$ PS1="\A \h \$ "  
17:33 linuxbox $
```

Добавление времени суток полезно, если нужно отслеживать время выполнения определенных задач. В конце мы создадим новое приглашение, похожее на наше исходное.

```
17:37 linuxbox $ PS1="<\u@\h \w>\$ "  
<me@linuxbox ~>$
```

Попробуйте другие последовательности из таблицы выше и посмотрите, сможете ли вы придумать свое блестящее приглашение.

Добавление цвета

Большинство программ — эмуляторов терминала — реагируют на определенные последовательности непечатаемых символов для управления такими вещами, как атрибуты символов (цвет,

жирный шрифт и ужасающий мигающий текст) и положение курсора. Мы разберем положение курсора чуть позже, но сначала рассмотрим цвет.

Путаница с терминалом

В давние времена, когда терминалы подключались к удаленным компьютерам, существовало множество конкурирующих брендов, и все они работали по-разному. У них были разные клавиатуры и разные способы интерпретации управляющей информации. Unix и Unix-подобные системы используют две довольно сложные подсистемы для решения проблемы «вавилонского столпотворения» управления терминалами (называемые `termcap` и `terminfo`). Если заглянуть в самые глубокие настройки вашего эмулятора терминала, вы можете найти параметр типа эмуляции терминала.

Чтобы терминалы понимали общий язык, Американский национальный институт стандартов (ANSI) разработал стандартный набор последовательностей символов для управления видеотерминалами. Пользователи старой DOS вспомнят `ANSI.SYS` файл, который использовался для включения интерпретации этих кодов.

Цвет символов управляется путем отправки эмулятору терминала ANSI-кода, встроенного в поток отображаемых символов. Управляющий код не «печатается» на экране, а интерпретируется терминалом как инструкция. Как мы видели в таблице 13-1, последовательности `\[` и `\]` используются для инкапсуляции непечатных символов. ANSI-код начинается с `\e` (восьмеричное 033, код, генерируемый Esc), за которым следует необязательный атрибут символа и инструкция. Например, код для установки обычного цвета текста (атрибут = 0) и черного цвета текста выглядит так:

```
\e[0;30m
```

В таблице 13-2 перечислены доступные цвета текста. Заметьте, что цвета разделены на две группы, различающиеся применением атрибута жирного шрифта (1), который создает эффект «светлых» цветов.

Таблица 13-2: Последовательности escape-символов для установки цвета текста

Последовательность	Цвет текста	Последовательность	Цвет текста
--------------------	-------------	--------------------	-------------

Последовательность	Цвет текста	Последовательность	Цвет текста
\e[0;30m	Черный	\e[1;30m	Темно-серый
\e[0;31m	Red	\e[1;31m	Светло-красный
\e[0;32m	Зеленый	\e[1;32m	Светло-зеленый
\e[0;33m	Brown	\e[1;33m	Желтый
\e[0;34m	Blue	\e[1;34m	Светло-голубой
\e[0;35m	Фиолетовый	\e[1;35m	Светло-фиолетовый
\e[0;36m	Cyan	\e[1;36m	Светло-голубой
\e[0;37m	Светло-серый	\e[1;37m	Белый

Давайте попробуем сделать красное приглашение. Мы вставим управляющий код в начало.

```
<me@linuxbox ~>$ PS1="\[\e[0;31m\]<\u@h \w>\$ "
<me@linuxbox ~>$
```

Это работает, но заметьте, что весь текст, вводимый после приглашения, тоже будет отображаться красным. Чтобы это исправить, мы добавим в конец приглашения еще один управляющий код, который прикажет эмулятору терминала вернуться к предыдущему цвету.

```
<me@linuxbox ~>$ PS1="\[\e[0;31m\]<\u@h \w>\$ \[\e[0m\] "
<me@linuxbox ~>$
```

Так гораздо лучше!

Также можно установить цвет фона текста, используя коды из таблицы 13-3. Цвета фона не поддерживают атрибут жирного шрифта.

Таблица 13-3: Экранирующие последовательности для установки цвета фона

Последовательность	Цвет фона	Последовательность	Цвет фона
\e[0;40m	Черный	\e[0;44m	Blue

Последовательность	Цвет фона	Последовательность	Цвет фона
\e[0;41m	Red	\e[0;45m	Фиолетовый
\e[0;42m	Зеленый	\e[0;46m	Син
\e[0;43m	Brown	\e[0;47m	Светло-серый

Мы можем создать приглашение с красным фоном, изменив первый escape-код.

```
<me@linuxbox ~->$ PS1="\[\e[0;41m\]<\u@\h \w>\$[\e[0m\] "
<me@linuxbox ~->$
```

Попробуйте цветовые коды и посмотрите, что у вас получится!

Примечание: помимо обычного (0) и жирного (1) начертания, тексту можно задать подчеркивание (4), мигание (5) и инверсию (7). Однако из соображений эстетики многие эмуляторы терминала игнорируют атрибут мигания.

Перемещение курсора

Для позиционирования курсора можно использовать escape-коды. Обычно это применяется для вывода часов или другой информации в определенном месте экрана, например, в верхнем углу при каждом обновлении приглашения. В таблице 13-4 перечислены escape-коды для позиционирования курсора.

Таблица 13-4: Escape-последовательности для перемещения курсора

Escape-код	Действие
\e[l;сh	Переместить курсор на строку l и столбец с
\e[nA	Переместить курсор вверх на n строк
\e[nB	Переместить курсор вниз на n строк
\e[nC	Переместить курсор вперед на n символов
\e[nD	Переместить курсор назад на n символов
\e[2J	Очистить экран и переместить курсор в верхний левый угол (строка 0, столбец 0)

Escape-код	Действие
\e[K	Очистить от позиции курсора до конца текущей строки
\e[s	Сохранить текущую позицию курсора
\e[u	Восстановить сохраненную позицию курсора

Используя коды из таблицы 13-4, мы создадим приглашение, которое при каждом выводе будет рисовать в верхней части экрана красную полосу с часами (выведенными желтым текстом). Код приглашения представляет собой такую внушительную строку:

```
PS1="\[\e[s\e[0;0H\e[0;41m\e[K\e[1;33m\t\e[0m\e[u\]
<\u@\h \w>\$ "
```

В таблице 13-5 описано назначение каждой части строки.

Таблица 13-5: Разбор сложной строки приглашения о

Последовательность	Действие
\[	Начало последовательности непечатаемых символов. Это нужно, чтобы bash мог правильно вычислить размер видимой приглашения. Без точного расчета функции редактирования командной строки не смогут правильно позиционировать курсор.
\e[s	Сохраняйте положение курсора. Это необходимо, чтобы вернуться к месту приглашения после отрисовки панели и часов в верхней части экрана. <i>Имейте в виду, что некоторые эмуляторы терминала не распознают этот код.</i>
\e[0;0H	Переместите курсор в верхний левый угол (строка 0, столбец 0)..
\e[0;41m	Установите красный цвет фона.

Последовательность	Действие
<code>\e[K</code>	Очистите область от текущего положения курсора (верхний левый угол) до конца строки. Поскольку фон теперь красный, строка очищается этим цветом, создавая нашу полосу. Примечание: очистка до конца строки не меняет положение курсора, что оставляет его в верхнем левом углу.
<code>\e[1;33m</code>	Установите желтый цвет текста.
<code>\t</code>	Вывод текущего времени. Хотя это элемент «вывода», мы все равно включаем его в непечатную часть приглашения, так как не хотим, чтобы <code>bash</code> учитывал часы при расчете реальный размер отображаемого приглашения.
<code>\e[0m</code>	Выключить цвет. Это влияет и на текст, и на фон.
<code>\e[u</code>	Восстановить ранее сохраненную позицию курсора.
<code>\]</code>	Завершить последовательность непечатных символов.
<code><\u@\h \w>\\$</code>	Строка приглашения.

Сохранение приглашения

Очевидно, что вводить это чудовище постоянно не хочется, поэтому приглашение стоит где-нибудь сохранить. Мы можем сделать приглашение постоянным, добавив его в файл `.bashrc`. Для этого добавьте в файл следующие две строки:

```
PS1="\[\e[s\e[0;0H\e[0;41m\e[K\e[1;33m\t\e[0m\e[u\]
<\u@\h \w>\$ "
export PS1
```

Итоги

Верите или нет, с помощью приглашения, функций оболочки и скриптов можно сделать гораздо больше, но это лишь хорошее начало. Не всем захочется менять приглашение, так как стандартное обычно вполне устраивает. Но для тех, кто любит

возиться с настройками, оболочка открывает возможности для долгих часов увлекательного досуга.

Что почитать

- Руководство The Bash Prompt HOWTO от Linux Documentation Project содержит довольно полное описание возможностей приглашения оболочки. Оно доступно по адресу:
<https://tldp.org/HOWTO/Bash-Prompt-HOWTO/>
- В Wikipedia есть хорошая статья о кодах ANSI Escape Codes:
https://en.wikipedia.org/wiki/ANSI_escape_code

Часть 3 — Распространенные задачи и основные инструменты

14 — Управление пакетами

В сообществе Linux часто спорят о том, какой дистрибутив «лучший». Порой эти дискуссии доходят до абсурда, когда обсуждают такие мелочи, как красота обоев рабочего стола (некоторые отказываются от Ubuntu из-за стандартной цветовой схемы!) и прочие пустяки.

Главные критерии качества дистрибутива — это система управления пакетами и активность сообщества поддержки. Со временем становится ясно, что программная среда Linux крайне динамична. Всё постоянно меняется. Большинство ведущих дистрибутивов выпускают новые версии каждые полгода, а обновления отдельных программ выходят ежедневно. Чтобы не утонуть в этом потоке ПО, нужны качественные инструменты управления пакетами.

Управление пакетами — это метод установки и обслуживания программного обеспечения в системе. Сегодня большинство пользователей могут удовлетворить все свои потребности, устанавливая пакеты из репозитория своего дистрибутива. Это отличает современный Linux от ранних времен, когда для установки программ приходилось скачивать и компилировать исходный код. В компиляции исходников нет ничего плохого; напротив, доступ к коду — это одно из главных преимуществ Linux. Он дает нам (и всем остальным) возможность изучать и улучшать систему. Просто работать с готовым скомпилированным пакетом быстрее и проще.

В этой главе мы рассмотрим инструменты командной строки для управления пакетами. Хотя все основные дистрибутивы предлагают мощные графические программы для обслуживания системы, важно изучить и консольные инструменты. Они позволяют выполнять многие задачи, которые трудно (или невозможно) реализовать с помощью графических аналогов.

Системы пакетов

Разные дистрибутивы используют разные системы управления пакетами, и, как правило, пакет, предназначенный для одного дистрибутива, несовместим с другим. Большинство дистрибутивов относятся к одному из двух лагерей технологий упаковки: Debian (.deb) или Red Hat (.rpm). Существуют важные исключения, такие как Gentoo, Slackware и Arch, но большинство остальных используют одну из этих двух базовых систем, как показано в таблице 14-1.

Таблица 14-1: Основные семейства систем управления пакетами

Система управления пакетами	Дистрибутивы (частичный список)
Стиль Debian (.deb)	Debian, Ubuntu, Linux Mint, Raspberry Pi OS
Стиль Red Hat (.rpm)	Fedora, CentOS, Red Hat Enterprise Linux, OpenSUSE

Принцип работы пакетной системы

Метод распространения ПО в индустрии проприетарного софта обычно подразумевает покупку установочного носителя, например «установочного диска», либо посещение сайта вендора для загрузки продукта с последующим запуском «мастера установки» для размещения нового приложения в системе.

В Linux всё устроено иначе. Практически всё программное обеспечение для Linux можно найти в Интернете. Большая часть поставляется разработчиками дистрибутивов в виде файлов пакетов, а остальное доступно в виде исходного кода для ручной установки. О том, как устанавливать ПО путем компиляции исходного кода, мы поговорим в главе 23.

Файлы пакетов

Основной единицей ПО в пакетной системе является файл пакета. Файл пакета — это сжатая коллекция файлов, составляющих программный пакет. Пакет может включать множество программ и вспомогательных файлов данных. Помимо устанавливаемых файлов, пакет содержит метаданные, такие как текстовое описание пакета и его содержимого. Кроме того, многие пакеты включают скрипты до и после установки для выполнения задач по настройке системы.

Файлы пакетов создает человек, называемый мейнтейнером пакета, — зачастую (но не всегда) сотрудник поставщика дистрибутива. Мейнтейнер получает программное обеспечение в виде исходного кода от upstream-поставщика (автора программы), компилирует его, создает метаданные пакета и необходимые скрипты установки. Часто мейнтейнер вносит изменения в исходный код, чтобы улучшить интеграцию программы с другими компонентами дистрибутива Linux.

Репозитории

Хотя некоторые проекты предпочитают самостоятельно заниматься упаковкой и распространением, большинство

современных пакетов создается поставщиками дистрибутивов или заинтересованными третьими сторонами. Пакеты доступны пользователям дистрибутива в центральных репозиториях, которые могут содержать многие тысячи пакетов, каждый из которых специально собран и поддерживается для данного дистрибутива.

Дистрибутив может поддерживать несколько различных репозиторий для разных этапов жизненного цикла разработки ПО. Например, обычно существует репозиторий «testing», содержащий только что собранные пакеты для смельчаков, ищущих ошибки перед общим выпуском. Также у дистрибутива часто есть репозиторий «development», где хранятся пакеты в разработке, предназначенные для следующего крупного релиза.

Дистрибутив может иметь связанные сторонние репозитории. Они часто необходимы для поставки ПО, которое по юридическим причинам (например, из-за патентов или законов о защите DRM) не может быть включено в состав дистрибутива. Пожалуй, самый известный пример — поддержка зашифрованных DVD, что незаконно в США. Сторонние репозитории работают в странах, где законы о патентах на ПО и защите от обхода технических средств не действуют. Такие репозитории обычно полностью независимы от поддерживаемого дистрибутива; чтобы использовать их, нужно знать о них и вручную добавить в конфигурационные файлы системы управления пакетами.

Зависимости

Программы редко бывают «автономными»; скорее, они полагаются на наличие других программных компонентов для выполнения своих задач. Общие операции, такие как ввод/вывод, обрабатываются подпрограммами, используемыми многими приложениями. Эти подпрограммы хранятся в так называемых разделяемых библиотеках, которые предоставляют необходимые сервисы более чем одной программе. Если пакет требует общий ресурс, например разделяемую библиотеку, говорят, что у него есть зависимость. Все современные системы управления пакетами предоставляют методы разрешения зависимостей, чтобы при установке пакета устанавливались и все его зависимости.

Инструменты управления пакетами высокого и низкого уровня

Системы управления пакетами обычно состоят из двух типов инструментов.

- Низкоуровневые инструменты, выполняющие такие задачи, как установка и удаление файлов пакетов
- Высокоуровневые инструменты, выполняющие поиск метаданных и разрешение зависимостей

В этой главе мы рассмотрим инструменты систем семейства Debian (таких как Ubuntu и многих других) и продукты Red Hat. Хотя все дистрибутивы семейства Red Hat полагаются на одну и ту же низкоуровневую программу (`rpm`), они используют разные высокоуровневые инструменты. В нашем обсуждении мы разберем высокоуровневую программу `dnf`, используемую в Red Hat Enterprise Linux, CentOS и Fedora. Другие дистрибутивы семейства Red Hat предоставляют высокоуровневые инструменты с сопоставимыми возможностями (см. таблицу 14-2).

Таблица 14-2: Инструменты системы управления пакетами

Дистрибутивы	Низкоуровневые инструменты	Высокоуровневые инструменты
Стиль Debian	<code>dpkg</code>	<code>apt</code> , <code>apt-get</code> , <code>aptitude</code>
Fedora, Red Hat Enterprise Linux, CentOS	<code>rpm</code>	<code>dnf</code> , <code>yum</code>

Общие задачи управления пакетами

С помощью инструментов управления пакетами через командную строку можно выполнить множество операций. Мы рассмотрим наиболее распространенные. Имейте в виду, что низкоуровневые инструменты также поддерживают создание файлов пакетов — эта тема не входит в задачи данной книги.

В нижеприведенном обсуждении термин `package_name` относится к фактическому имени пакета, а не к термину `package_file`, который является именем файла, содержащего пакет. Кроме того, перед выполнением любых операций с пакетами необходимо выполнить запрос к репозиторию пакетов, чтобы синхронизировать локальную копию его базы данных. Программа `dnf` в Red Hat делает это автоматически и обновляет локальную базу данных, если с момента последнего обновления прошло слишком много времени. С другой стороны, программу `apt` в Debian необходимо запускать с командой `update`, чтобы явно обновить локальную базу данных. Это нужно делать время

от времени. В приведенных ниже примерах команда `apt update` выполняется перед любыми операциями, но на практике это требуется делать лишь раз в несколько часов для надежности.

Поскольку операции по установке или удалению программного обеспечения в масштабах всей системы являются административной задачей, требуются привилегии суперпользователя независимо от используемого инструмента управления пакетами.

Поиск пакета в репозитории

С помощью высокоуровневых инструментов поиска по метаданным репозитория пакет можно найти по его имени или описанию (см. таблицу 14-3).

Таблица 14-3: Команды поиска пакетов

Стиль	Команда(ы)
Debian	<code>apt update; apt search search_string</code>
Red Hat	<code>dnf search search_string</code>

Например, чтобы найти текстовый редактор `emacs` в `dnf` репозитории, можно использовать следующую команду:

```
dnf search emacs
```

Установка пакета из репозитория

Инструменты высокого уровня позволяют загружать пакет из репозитория и устанавливать его с полным разрешением зависимостей (см. таблицу 14-4).

Таблица 14-4: Команды установки пакетов

Стиль	Команда(ы)
Debian	<code>apt update; apt install package_name</code>
Red Hat	<code>dnf install package_name</code>

Например, чтобы установить текстовый редактор `emacs` из `apt` репозитория в системе Debian, можно использовать эту команду:

```
apt update; apt install emacs
```

Установка пакета из файла пакета

Если файл пакета был загружен не из репозитория, его можно установить напрямую (но без разрешения зависимостей) с

помощью низкоуровневого инструмента (см. таблицу 14-5).

Таблица 14-5: Команды низкоуровневой установки пакетов

Стиль	Команда(ы)
Debian	<code>dpkg -i package_file</code>
Red Hat	<code>rpm -i package_file</code>

Например, если бы файл `emacs-22.1-7.fc7-i386.rpm` package был загружен не из репозитория, его установили бы так:

```
rpm -i emacs-22.1-7.fc7-i386.rpm
```

Примечание: поскольку этот метод использует низкоуровневую программу `rpm` для установки, разрешение зависимостей не выполняется. Если `rpm` обнаружит отсутствие зависимости, `rpm` завершит работу с ошибкой.

Удаление пакета

Пакеты можно удалить с помощью инструментов высокого или низкого уровня. В таблице 14-6 приведены инструменты высокого уровня.

Таблица 14-6: Команды удаления пакетов

Стиль	Команда(ы)
Debian	<code>apt remove package_name</code>
Red Hat	<code>dnf erase package_name</code>

Например, чтобы удалить пакет `emacs` в системе типа Debian, можно использовать эту команду:

```
apt remove emacs
```

Обновление пакетов из репозитория

Самая частая задача при управлении пакетами — поддержание системы в актуальном состоянии. Высокоуровневые инструменты позволяют выполнить эту важную задачу за один шаг (см. таблицу 14-7).

Таблица 14-7: Команды обновления пакетов

Стиль	Команда(ы)
-------	------------

Стиль	Команда(ы)
Debian	<code>apt update; apt upgrade</code>
Red Hat	<code>dnf update</code>

Например, чтобы применить все доступные обновления для установленных пакетов в системе стиля Debian, можно использовать эту команду:

```
apt update; apt upgrade
```

Обновление пакета из файла пакета

Если обновленная версия пакета была загружена из стороннего источника, ее можно установить, заменив предыдущую версию (см. таблицу 14-8).

Таблица 14-8: Низкоуровневые команды обновления пакетов

Стиль	Команда(ы)
Debian	<code>dpkg -i package_file</code>
Red Hat	<code>rpm -U package_file</code>

Например, чтобы обновить установленный emacs до версии из файла пакета `emacs-22.1-7.fc7-i386.rpm` в системе Red Hat, можно использовать следующую команду:

```
rpm -U emacs-22.1-7.fc7-i386.rpm
```

Примечание: у `dpkg` нет отдельной опции для обновления пакета в отличие от установки, как это реализовано в `rpm`.

Список установленных пакетов

В таблице 14-9 перечислены команды, которые можно использовать для вывода списка всех установленных в системе пакетов.

Таблица 14-9: Команды для вывода списка пакетов

Стиль	Команда(ы)
Debian	<code>dpkg -l</code>
Red Hat	<code>rpm -qa</code>

Определение того, установлен ли пакет

В таблице 14-10 перечислены низкоуровневые инструменты для проверки установки указанного пакета.

Таблица 14-10: Команды проверки статуса пакетов

Стиль	Команда(ы)
Debian	<code>dpkg -s package_name</code>
Red Hat	<code>rpm -q package_name</code>

Например, чтобы узнать, установлен ли пакет `emacs` в системе типа Debian, можно использовать эту команду:

```
dpkg --status emacs
```

Просмотр информации об установленном пакете

Если имя установленного пакета известно, можно использовать команды из таблицы 14-11, чтобы вывести его описание.

Таблица 14-11: Команды для получения информации о пакетах

Стиль	Команда(ы)
Debian	<code>apt show package_name</code>
Red Hat	<code>dnf info package_name</code>

Например, чтобы увидеть описание пакета `emacs` в системе на базе Debian, можно использовать эту команду:

```
apt-cache show emacs
```

Поиск пакета, установившего файл

Чтобы определить, какой пакет отвечает за установку конкретного файла, можно использовать команды из таблицы 14-12.

Таблица 14-12: Команды для идентификации файлов пакетов

Стиль	Команда(ы)
Debian	<code>dpkg -S file_name</code>
Red Hat	<code>rpm -qf file_name</code>

Например, чтобы узнать, какой пакет установил файл `/usr/bin/vim` в системе Red Hat, можно использовать следующее:

```
rpm -qf /usr/bin/vim
```


Универсальные форматы пакетов

За последние несколько лет поставщики дистрибутивов представили универсальные форматы пакетов, не привязанные к конкретному дистрибутиву Linux. К ним относятся Snaps (разработаны и продвигаются Canonical), Flatpaks (внедрены Red Hat, но теперь широко доступны) и AppImages. Хотя они работают немного по-разному, их цель — объединить приложение и все его зависимости в единый пакет для установки. Это не совсем новая идея. На заре Linux (в конце 1990-х) существовал метод статической линковки, который объединял приложение и необходимые библиотеки в один большой бинарный файл.

У такого подхода к упаковке есть свои преимущества. Главное из них — упрощение распространения приложений. Вместо того чтобы адаптировать приложение под библиотеки и файлы поддержки базовой системы дистрибутива, оно собирается один раз и может быть установлено в любой системе. Некоторые из этих форматов также запускают приложение в изолированной контейнерной среде для дополнительной безопасности.

Однако есть и серьезные недостатки. Приложения, упакованные таким образом, имеют большой размер. Иногда — очень большой. Это приводит к двум последствиям. Во-первых, им требуется много места на диске. Во-вторых, из-за большого размера они могут загружаться очень медленно. На современном сверхбыстром оборудовании это не так критично, но на старых машинах это становится реальной проблемой. Следующая техническая проблема связана с интеграцией в дистрибутив. Поскольку такие приложения несут всё необходимое с собой, они не используют возможности базового дистрибутива. Иногда контейнеризированное приложение не может получить доступ к системным ресурсам, необходимым для оптимальной производительности.

Существуют и философские вопросы. Пожалуй, главными бенефициарами таких универсальных пакетов являются разработчики проприетарного ПО. Они могут один раз создать версию для Linux, которую сможет использовать любой дистрибутив, не адаптируя приложение под каждый из них отдельно.

Пользователи не просили о подобных форматах упаковки, и они мало способствуют развитию open-source сообщества.

Поэтому до решения проблем с производительностью использование этих форматов не рекомендуется.

Итоги

В следующих главах мы изучим множество различных программ, охватывающих широкий спектр областей применения. Хотя большинство из них устанавливаются по умолчанию, нам может потребоваться установка дополнительных пакетов, если нужные программы отсутствуют. Благодаря полученным знаниям (и пониманию) принципов управления пакетами, у нас не возникнет проблем с установкой и управлением необходимым ПО.

Миф об установке программного обеспечения в Linux

Пользователи, переходящие с других платформ, иногда попадают в ловушку мифа о том, что в Linux сложно устанавливать ПО, а разнообразие схем упаковки в разных дистрибутивах только мешает. Что ж, это действительно помеха, но только для вендоров проприетарного ПО, которые хотят распространять свои секретные программы исключительно в бинарном виде.

Экосистема программного обеспечения Linux основана на идее открытого исходного кода. Если разработчик выпускает исходный код программы, то кто-то из участников дистрибутива, скорее всего, упакует её и добавит в свой репозиторий. Такой подход гарантирует глубокую интеграцию программы в дистрибутив и удобство «единого магазина» для пользователя, избавляя его от необходимости искать сайт каждой отдельной программы. Недавно крупные вендоры проприетарных платформ начали создавать магазины приложений, имитирующие эту идею.

Драйверы устройств обрабатываются почти так же, но вместо отдельных элементов в репозитории дистрибутива они становятся частью ядра Linux. Как правило, в Linux не существует понятия «диск с драйверами». Либо ядро поддерживает устройство, либо нет, а ядро Linux поддерживает множество устройств — фактически гораздо больше, чем Windows. Разумеется, это не утешение, если нужное вам устройство не поддерживается. В таком случае следует выяснить причину. Отсутствие поддержки драйверов обычно вызвано одной из трех причин:

1. Устройство слишком новое. Поскольку многие производители оборудования не участвуют активно в разработке Linux, написание кода драйвера ядра ложится на плечи сообщества Linux. Это требует времени.
2. Устройство слишком экзотично. Не все дистрибутивы включают все возможные драйверы. Каждый дистрибутив собирает свои ядра, а поскольку ядра очень гибко настраиваются (что позволяет запускать Linux на чем угодно — от наручных часов до мейнфреймов), они могут не учитывать конкретное устройство. Найдя и скачав исходный код драйвера, вы (да, именно вы) можете самостоятельно скомпилировать и установить его. Этот процесс не слишком сложен, но довольно трудоемок. О компиляции программного обеспечения мы поговорим в одной из следующих глав.
3. Производитель оборудования что-то скрывает. Он не выпустил ни исходный код драйвера для Linux, ни техническую документацию, чтобы кто-то другой мог создать его за них. Это означает, что производитель пытается сохранить в секрете программные интерфейсы устройства. Поскольку нам не нужны устройства с секретами в наших компьютерах, лучше избегать таких продуктов.

Что почитать

Потратьте время на изучение системы управления пакетами вашего дистрибутива. Каждый дистрибутив предоставляет документацию к своим инструментам управления пакетами. Кроме того, вот несколько более общих источников:

- Глава Debian GNU/Linux FAQ об управлении пакетами содержит обзор управления пакетами в системах Debian:
<https://www.debian.org/doc/manuals/debian-faq/pkg-basics.en.html>
- Главная страница проекта RPM: <http://www.rpm.org>
- Для справки: в Wikipedia есть статья о метаданных:
<http://en.wikipedia.org/wiki/Metadata>
- Хорошая статья со сравнением форматов Snap, Flatpak и AppImage: <https://www.baeldung.com/linux/snaps-flatpak-appimage>

15 — Носители информации

В предыдущих главах мы рассматривали манипуляции с данными на уровне файлов. В этой главе мы изучим данные на уровне устройств. Linux обладает потрясающими возможностями для работы с устройствами хранения — будь то физические носители (например, жесткие диски), сетевые хранилища или виртуальные устройства, такие как RAID (Redundant Array of Independent Disks) и LVM (Logical Volume Manager).

Поскольку эта книга не посвящена системному администрированию, мы не будем углубляться в эту тему. Мы лишь познакомим вас с основными концепциями и ключевыми командами для управления устройствами хранения.

Для выполнения упражнений этой главы понадобятся USB-флешка, USB-жесткий диск и диск DVD/CD-RW (если в системе есть CD-ROM-привод с возможностью записи).

Мы рассмотрим следующие команды:

- `mount` — Монтирование файловой системы
- `umount` — размонтирование файловой системы
- `parted` — программа для манипуляций с разделами
- `mkfs` — создание файловой системы
- `fsck` — проверка и восстановление файловой системы
- `dd` — преобразование и копирование файла
- `genisoimage` — создание образа ISO 9660
- `wodim` — запись данных на оптические носители
- `sha256sum` — вычисление и проверка контрольных сумм SHA256

Монтирование и размонтирование устройств хранения

Последние достижения в области Linux desktop значительно упростили управление устройствами хранения для пользователей настольных систем. В большинстве случаев мы просто подключаем устройство к системе, и оно «просто работает». Раньше (скажем, в 2004 году) всё приходилось делать вручную. В не-десктопных системах (т. е. серверах) это по-прежнему во многом ручная процедура, так как серверы часто имеют экстремальные потребности в хранении данных и сложные требования к конфигурации.

Первый шаг в управлении устройством хранения — его подключение к дереву файловой системы. Этот процесс называется монтированием; он позволяет устройству взаимодействовать с операционной системой. Как мы помним из главы 2, Unix-подобные операционные системы, такие как Linux, используют единое дерево файловой системы, в которое устройства подключаются в различных точках. Это отличает их от других ОС, таких как MS-DOS и Windows, где для каждого устройства поддерживается отдельное дерево файловой системы (например, C:\, D:\ и т. д.).

Файл с именем `/etc/fstab` (сокращение от «filesystem table») содержит список устройств (обычно разделов жесткого диска), которые должны быть смонтированы при загрузке. Вот пример файла `/etc/fstab` из ранней системы Fedora:

LABEL=/12	/	ext4	defaults	1 1
LABEL=/home	/home	ext4	defaults	1 2
LABEL=/boot	/boot	ext4	defaults	1 2
tmpfs	/dev/shm	tmpfs	defaults	0 0
devpts	/dev/pts	devpts	gid=5,mode=620	0 0
sysfs	/sys	sysfs	defaults	0 0
proc	/proc	proc	defaults	0 0
LABEL=SWAP-sda3	swap	swap	defaults	0 0

Большинство файловых систем, перечисленных в этом примере, являются виртуальными и не относятся к нашему обсуждению. Для наших целей интересны первые три:

LABEL=/12	/	ext4	defaults	1 1
LABEL=/home	/home	ext4	defaults	1 2
LABEL=/boot	/boot	ext4	defaults	1 2

Это разделы жесткого диска. Каждая строка файла состоит из шести полей, описанных в таблице 15-1.

Таблица 15-1: `/etc/fstab` Поля *мер и*,

Поле	Содержание	Описание
1	Устройство	Традиционно в этом поле содержится фактическое имя файла устройства, связанного с физическим устройством, напри <code>/dev/sda1</code> (первый раздел первого обнаруженного жесткого диска). Но современные компьютеры, имеющие множество устройств с поддержкой горячей замены (например, USB-накопитель заставляют многие современные дистрибутивы Linux связывать устройство

Поле	Содержание	Описание
		вместо этого текстовой меткой. Эта метка (которая добавляется к носителям при форматировании) может быть либо простой текстовой меткой, либо случайно сгенерированным UUID (Universally Unique Identifier). Эта метка считывается операционной системой при подключении устройства к системы. Таким образом, какой бы файл устройства ни был назначен реальному физическому устройству, его всё равно можно правильно идентифицировать.
2	Точка монтирования	Каталог, в котором устройство подключается к дереву файловой системы.
3	Тип файловой системы	Linux позволяет монтировать множество типов файловых систем. Большинство систем Linux используют родную файловую систему под названием Fourth Extended Filesystem (ext4), но поддерживается и много других, включая FAT16 (msdos), FAT32 (vfat), NTFS (ntfs), CD-ROM (iso9660), etc.
4	Параметры	Файловые системы можно монтировать с различными параметрами. Можно, например, монтировать их только для чтения или запретить выполнение любых программ на них (полезная функция безопасности для съёмных носителей).
5	Частота	Число, определяющее, нужно ли и когда делать резервную копию файловой системы командой <code>dump</code> .
6	Порядок	Число, определяющее порядок, в котором <code>fsck</code> следует проверять файловые системы команда. Подробнее об этом — далее в этой главе.

Просмотр списка смонтированных файловых систем

Команда `mount` используется для монтирования файловых систем. Ввод команды без аргументов выведет список текущих смонтированных файловых систем:

```
[me@linuxbox ~]$ mount
/dev/sda2 on / type ext4 (rw)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
devpts on /dev/pts type devpts (rw,gid=5,mode=620)
/dev/sda5 on /home type ext4 (rw)
/dev/sda1 on /boot type ext4 (rw)
tmpfs on /dev/shm type tmpfs (rw)
none on /proc/sys/fs/binfmt_misc type binfmt_misc (rw)
sunrpc on /var/lib/nfs/rpc_pipefs type rpc_pipefs (rw)
fusectl on /sys/fs/fuse/connections type fusectl (rw)
/dev/sdd1 on /media/disk type vfat (rw,nosuid,nodev,noatime,
uhelper=hal,uid=500,utf8,shortname=lower)
twin4:/musicbox on /misc/musicbox type nfs4 (rw,addr=192.168.1.4)
```

Совет: пользователям Ubuntu этот список может показаться перегруженным устройствами «loop», созданными снап-пакетами. Чтобы получить более чистый вывод, попробуйте эти команды: `mount | grep -v snap` or `mount | grep ^/dev`

Формат вывода следующий: устройство, точка монтирования, тип файла, система, тип (опции). Например, первая строка показывает, что устройство `/dev/sda2` смонтировано как корневая файловая система, имеет тип `ext4` и доступно для чтения и записи (опция «`rw`»). В конце списка есть еще две интересные записи. Предпоследняя показывает карту памяти на 2 Гб в картридере, смонтированную в `/media/disk`, а последняя — сетевой диск, смонтированный в `/misc/musicbox`.

Для первого эксперимента мы будем использовать флеш-накопитель емкостью 4 Гб. Сначала рассмотрим систему Ubuntu 22.04 до подключения накопителя:

```
[me@linuxbox ~]$ mount | grep /dev/sd
/dev/sda2 on / type ext4 (rw,relatime,errors=remount-ro)
/dev/sda1 on /boot/efi type vfat (rw,relatime,fmask=0077,dmask=0077,
codepage=437,iocharset=iso8859-1,shortname=mixed,errors=remount-ro)
/dev/sdb1 on /home type ext4 (rw,relatime)
```

Для наглядности мы направили вывод через конвейер в `grep`, чтобы в списке отображались только `/dev/sd*`-устройства.

Видно, что в системе два жестких диска, `/dev/sda` и `/dev/sdb`. На `/dev/sda` и `/dev/sdb` по два раздела — `/dev/sda1` и `/dev/sda2`, а на `на`. — один раздел.

Как и многие современные дистрибутивы Linux, эта система попытается автоматически смонтировать флеш-накопитель после его подключения. После подключения накопителя мы увидим следующее:

```
[me@linuxbox ~]$ mount | grep /dev/sd
dev/sda2 on / type ext4 (rw,relatime,errors=remount-ro)
/dev/sda1 on /boot/efi type vfat
(rw,relatime,fmask=0077,dmask=0077,codepage=437,iocharset=iso8859-
1,shortname=mixed,errors=remount-ro)
/dev/sdb1 on /home type ext4 (rw,relatime)
/dev/sdc on /media/me/C911-C314 type vfat (rw,nosuid,nodev,relatime,
uid=1000,gid=1000,fmask=0022,dmask=0022,codepage=437,iocharset=iso885
9-1,shortname=mixed,showexec=utf8,flush,errors=remount-ro)
```

После подключения накопителя мы видим тот же список, но с одной дополнительной записью. В конце списка указано, что флеш-накопитель (который является устройством `/dev/sdc` в этой системе) примонтирован в `/media/me/C911-C314` и имеет тип `vfat` (также известный как FAT32, файловая система, совместимая с Windows). Для целей нашего эксперимента нас интересует имя устройства. При самостоятельном выполнении этого эксперимента имя устройства, скорее всего, будет другим.

Предупреждение: в следующих примерах крайне важно внимательно следить за именами устройств, используемых в вашей системе, и не использовать имена из этого текста!

Теперь, когда мы знаем имя устройства флеш-накопителя, давайте размонтируем его и примонтируем в другом месте дерева файловой системы. Для этого станем суперпользователем (используя команду, подходящую для нашей системы) и размонтируем накопитель командой `umount` (обратите внимание на написание).

```
[me@linuxbox ~]$ sudo -i
[sudo] password for me:
[root@linuxbox ~]# umount /dev/sdc
```

Следующий шаг — создание новой точки монтирования для диска. Точка монтирования — это просто каталог где-либо в дереве файловой системы. В ней нет ничего особенного. Это даже не обязательно должен быть пустой каталог, хотя при монтировании устройства в непустой каталог вы не увидите его прежнее содержимое, пока не размонтируете устройство. Для наших целей мы создадим новый каталог.

```
[root@linuxbox ~]# mkdir /mnt/flash
```


Наконец, мы монтируем флеш-накопитель в новую точку монтирования. Опция `-t` используется для указания типа файловой системы.

```
[root@linuxbox ~]# mount -t vfat /dev/sdc /mnt/flash
```

Afterward, мы можем просмотреть содержимое флеш-накопителя через новую точку монтирования `nt`.

```
[root@linuxbox ~]# cd /mnt/flash  
[root@linuxbox cdrom]# ls
```

Обратите внимание, что происходит при попытке размонтировать диск.

```
[root@linuxbox cdrom]# umount /dev/sdc  
umount: /mnt/flash: device is busy
```

Почему так происходит? Причина в том, что устройство нельзя размонтировать, если оно используется кем-то или каким-либо процессом. В данном случае мы сменили текущий рабочий каталог на точку монтирования флеш-накопителя, из-за чего устройство оказалось занято. Эту проблему легко решить, сменив текущий рабочий каталог на любой другой, отличный от точки монтирования:

```
[root@linuxbox cdrom]# cd  
[root@linuxbox ~]# umount /dev/sdc
```

Теперь устройство успешно размонтировано.

Почему размонтирование важно

Если вы посмотрите на вывод команды `free`, отображающей статистику использования памяти, вы увидите параметр `buffers`. Компьютерные системы проектируются для максимально быстрого выполнения задач. Одним из препятствий для скорости системы являются медленные устройства. Хороший пример — принтеры. Даже самый быстрый принтер крайне медленен по компьютерным меркам. Компьютер работал бы очень медленно, если бы ему приходилось останавливаться и ждать, пока принтер закончит печать страницы. В раннюю эпоху ПК (до появления многозадачности) это было реальной проблемой. Если вы работали с электронной таблицей или текстовым документом, компьютер замирал и становился недоступным при каждой печати. Компьютер отправлял данные на принтер с той скоростью, с которой принтер мог их принимать, но это было очень медленно, так как

принтеры печатают небыстро. Проблема была решена появлением буфера принтера — устройства с некоторой памятью RAM, расположенного между компьютером и принтером. С наличием буфера принтера компьютер отправлял данные на печать в буфер, где они быстро сохранялись в быстрой памяти RAM, позволяя компьютеру вернуться к работе, не дожидаясь завершения. Тем временем буфер принтера медленно передавал данные на принтер из своей памяти со скоростью, которую мог принять принтер.

Идея буферизации широко используется в компьютерах для повышения их скорости. Не позволяйте необходимости периодического чтения или записи данных на медленные устройства замедлять систему. Операционные системы хранят данные, считанные с устройств хранения или предназначенные для записи на них, в памяти как можно дольше, прежде чем фактически взаимодействовать с медленным устройством. В системе Linux, например, вы заметите, что по мере работы память кажется заполненной. Это не значит, что Linux «использует» всю память; это значит, что Linux использует всю доступную память для максимально возможной буферизации.

Такая буферизация позволяет выполнять запись на устройства хранения очень быстро, так как сама запись на физическое устройство откладывается на будущее. Тем временем данные, предназначенные для устройства, накапливаются в памяти. Время от времени операционная система записывает эти данные на физическое устройство.

Размонтирование устройства подразумевает запись всех оставшихся данных на него для безопасного извлечения. Если извлечь устройство, не размонтировав его, часть данных может не успеть передаться. В некоторых случаях это могут быть критически важные обновления каталогов, что приведет к повреждению файловой системы — одному из худших событий для компьютера.

Определение имен устройств

Иногда определить имя устройства бывает непросто. Раньше это было легче: устройство всегда находилось на одном месте и не менялось. Unix-подобные системы это любят. Когда создавался Unix, «замена дисководов» требовала использования погрузчика, чтобы вывезти из серверной устройство размером со стиральную машину. В последние годы конфигурация типичного

настольного ПК стала динамичной, и Linux развивался, становясь более гибким, чем его предшественники.

В приведенных выше примерах мы использовали способность современного рабочего стола Linux «автоматически» монтировать устройство и определять его имя постфактум. Но что, если мы управляем сервером или другой средой, где этого не происходит? Как нам узнать имя устройства?

Сначала разберем, как система именovala устройства. Если вывести содержимое каталога /dev (где находятся все устройства), можно увидеть, что их очень много.

```
[me@linuxbox ~]$ ls /dev
```

Содержимое этого списка выявляет определенные закономерности в именовании устройств. В таблице 15-2 описаны некоторые из них.

Таблица 15-2: Имена устройств хранения в Linux и и

Шаблон	Устройство
/dev/fd*	Дисководы для дискет.
/dev/hd*	IDE (PATA) диски в старых системах. Материнские платы в этих системах имеют два разъема или канала IDE, каждый с кабелем на два подключения дисков. Первый диск на кабеле называется master device, и второй называется slave device. Имена устройств упорядочены так, что /dev/hda относится к ведущему устройству на первом канале, /dev/hdb — ведомое устройство на первом канале; /dev/hdc — ведущее устройство на втором канале и так далее. Замыкающая цифра указывает номер раздела на устройстве. Например, /dev/hda1 относится к первому разделу на первом жестком диске системы, тогда как /dev/hda относится ко всему диску.
/dev/lp*	Принтеры.

Шаблон	Устройство
/dev/sd*	SCSI-диски. В современных системах Linux ядро считает все диско- подобные устройства (включая жесткие диски PATA/SATA, флеш-накопитель USB-устройства массового хранения, такие как портативные плееры и цифровые камеры) SCSI-дисками. Остальная часть системы именования похожа на описанную выше схему именования /dev/hd*.
/dev/sr*	Оптические приводы (CD/DVD-приводы и записывающие устройства).

Кроме того, для удобства часто встречаются символьные ссылки, такие как /dev/cdrom/dev/floppy, /dev/dvd и, указывающие на реальные файлы устройств.

Если система не монтирует съемные устройства автоматически, можно использовать следующий метод, чтобы узнать имя устройства при его подключении. Сначала запустите просмотр файла /var/log/messages или /var/log/syslog в реальном времени (для этого могут потребоваться привилегии суперпользователя).

```
[me@linuxbox ~]$ sudo tail -f /var/log/syslog
```

В современных системах на базе systemd используйте эту команду для отслеживания журнала systemd:

```
[me@linuxbox ~]$ sudo journalctl -f
```

Будут отображены последние несколько строк списка, после чего выполнение приостановится. Затем подключите съемное устройство. В этом примере мы размонтируем и извлечем флеш-накопитель, а затем вставим его снова. Ядро почти мгновенно обнаружит устройство и опросит его.

```
Jul 25 13:15:07 ratel mtp-probe[21318]: checking bus 3, device 8:
"/sys/devices/pci0000:00/0000:00:14.0/usb3/3-6"
Jul 25 13:15:07 ratel mtp-probe[21318]: bus: 3, device: 8 was not an
MTP device
Jul 25 13:15:07 ratel mtp-probe[21321]: checking bus 3, device 8:
"/sys/devices/pci0000:00/0000:00:14.0/usb3/3-6"
Jul 25 13:15:07 ratel mtp-probe[21321]: bus: 3, device: 8 was not an
MTP device
Jul 25 13:15:08 ratel kernel: scsi 6:0:0:0: Direct-Access      General
UDisk              5.00 PQ: 0 ANSI: 2
Jul 25 13:15:08 ratel kernel: sd 6:0:0:0: Attached scsi generic sg3
type 0
Jul 25 13:15:08 ratel kernel: sd 6:0:0:0: [sd] 7864320 512-byte
logical blocks: (4.03 GB/3.75 GiB)
Jul 25 13:15:08 ratel kernel: sd 6:0:0:0: [sd] Write Protect is off
```

```

Jul 25 13:15:08 ratel kernel: sd 6:0:0:0: [sdc] Mode Sense: 0b 00 00
08
Jul 25 13:15:08 ratel kernel: sd 6:0:0:0: [sdc] No Caching mode page
found
Jul 25 13:15:08 ratel kernel: sd 6:0:0:0: [sdc] Assuming drive cache:
write through
Jul 25 13:15:08 ratel kernel: sdc:
Jul 25 13:15:08 ratel kernel: sd 6:0:0:0: [sdc] Attached SCSI
removable disk

```

Когда вывод снова приостановится, нажмите `Ctrl-c`, чтобы вернуть приглашение. Интересная часть вывода — повторяющиеся упоминания `[sdc]`, что соответствует нашему ожиданию имени SCSI-диска. С учетом этого следующие две строки становятся особенно показательными:

```

Jul 25 13:15:08 ratel kernel: sdc:
Jul 25 13:15:08 ratel kernel: sd 6:0:0:0: [sdc] Attached SCSI
removable disk removable disk

```

Совет: использование техники `tail -f /var/log/syslog` **или** `journalctl -f` — **отличный способ наблюдать за работой системы почти в реальном времени.**

Существует и другой способ определить имя устройства (в Linux всегда есть несколько путей решения задач!). Мы можем использовать команду `lsblk`. Эта команда выводит список всех подключенных к системе блочных устройств, независимо от того, смонтированы они или нет.

Предположим, мы размонтировали и извлекли флеш-накопитель. Посмотрим на список подключенных блочных устройств:

```

[me@linuxbox ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda          8:0      0 111.8G  0 disk
├─sda1       8:1      0   976M  0 part /boot/efi
└─          /
└─sda3       8:3      0  14.9G  0 part [SWAP]
sdb          8:16     0  931.5G  0 disk
├─sdb1       8:17     0  922.2G  0 part /home
└─sdb2       8:18     0    9.3G  0 part
sr0         11:0     1   1024M  0 rom

```

Затем мы снова вставим флеш-накопитель и еще раз запустим `lsblk`:

```

[me@linuxbox ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda          8:0      0 111.8G  0 disk
├─sda1       8:1      0   976M  0 part /boot/efi
└─          /
└─sda3       8:3      0  14.9G  0 part [SWAP]
sdb          8:16     0  931.5G  0 disk

```

```
└─sdb1  8:17  0 922.2G  0 part /home
└─sdb2  8:18  0  9.3G  0 part
sdc      8:32  1  3.8G  0 disk
sr0     11:0  1 1024M  0 rom
```

Теперь мы видим, что в списке появилось новое устройство (sdc). Имя устройства будет оставаться прежним, пока оно физически подключено к компьютеру и компьютер не перезагружается.

Зная имя устройства, мы можем примонтировать флеш-накопитель.

```
[me@linuxbox ~]$ sudo mount /dev/sdc /mnt/flash
[me@linuxbox ~]$ df
```

Filesystem	1K-blocks	Used	Available	Use%	Mounted on
tmpfs	1624184	2144	1622040	1%	/run
/dev/sda2	98430196	45133696	48250332	49%	/
tmpfs	8120908	0	8120908	0%	/dev/shm
tmpfs	5120	4	5116	1%	/run/lock
/dev/sda1	997456	6220	991236	1%	/boot/efi
/dev/sdb1	950690656	585574576	316749944	65%	/home
tmpfs	1624180	80	1624100	1%	/run/user/120
tmpfs	1624180	192	1623988	1%	/run/user/1000
/dev/sdc	3924444	4	3924440	1%	/mnt/flash

Создание новых файловых систем

Допустим, у нас есть внешний USB-жесткий диск с одной файловой системой FAT32, и мы хотим разделить его на два раздела: нативный для Linux раздел (ext4) и второй, отформатированный в NTFS для использования в Windows. Это требует двух шагов.

1. Создайте новую схему разделов. 1. Создайте на диске новые пустые файловые системы.

Предупреждение: в следующем упражнении мы будем форматировать внешний жесткий диск. Используйте диск, на котором нет важных данных, так как он будет стерт! Еще раз: убедитесь, что вы указываете правильное имя устройства для вашей системы, а не то, что приведено в тексте. Игнорирование этого предупреждения может привести к форматированию (т. е. стиранию) не того диска!

Манипуляции с разделами с помощью parted

parted — одна из множества программ (как командной строки, так и графических), позволяющих напрямую взаимодействовать с дисковыми устройствами (такими как жесткие диски и флеш-

накопители) на очень низком уровне. Мы подключим диск и воспользуемся `lsblk`, чтобы узнать его имя:

```
[me@linuxbox ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda          8:0    0 111.8G  0 disk
├─sda1       8:1    0   976M  0 part /boot/efi
└─          /
└─sda3       8:3    0   14.9G  0 part [SWAP]
sdb          8:16   0 931.5G  0 disk
├─sdb1       8:17   0 922.2G  0 part /home
└─sdb2       8:18   0    9.3G  0 part
sdd          8:32   0 232.9G  0 disk
└─sdd1       8:33   0 232.9G  0 part /media/me/USB_DISK
sr0         11:0    1  1024M  0 rom
```

Как видим, внешний USB-жесткий диск подключен, имеет имя `/dev/sdd` и содержит один раздел с именем `/dev/sdd1`.

С помощью `parted` мы можем редактировать, удалять и создавать разделы на устройстве. Чтобы работать с жестким диском, сначала нужно размонтировать его (если необходимо), а затем запустить программу `parted` следующим образом:

```
[me@linuxbox ~]$ sudo umount /dev/sdd1
[me@linuxbox ~]$ sudo parted /dev/sdd
```

Обратите внимание, что устройство нужно указывать целиком, а не по номеру раздела. После запуска программы появится следующее приглашение:

```
GNU Parted 3.4
Using /dev/sdd
Welcome to GNU Parted! Type 'help' to view a list of commands.
(parted)
```

Ввод «`help`» выведет список доступных команд.

```
(parted) help
align-check TYPE N                check partition N for
                                  TYPE(min|opt) alignment
help [COMMAND]                    print general help, or
                                  help on COMMAND
mklabel,mktable LABEL-TYPE       create a new disklabel
                                  (partition table)
mkpart PART-TYPE [FS-TYPE] START END make a partition
name NUMBER NAME                  name partition NUMBER as
                                  NAME
print [devices|free|list,all|NUMBER] display the partition
                                  table, available devices,
                                  free space, all found
                                  partitions, or a
                                  particular partition
quit                               exit program
rescue START END                  rescue a lost partition
                                  near START and END
resizepart NUMBER END             resize partition NUMBER
rm NUMBER                         delete partition NUMBER
select DEVICE                     choose the device to edit
disk_set FLAG STATE               change the FLAG on
```

<code>disk_toggle [FLAG]</code>	selected device
<code>set NUMBER FLAG STATE</code>	toggle the state of FLAG on selected device
<code>toggle [NUMBER [FLAG]]</code>	change the FLAG on partition NUMBER
<code>unit UNIT</code>	toggle the state of FLAG on partition NUMBER
<code>version</code>	set the default unit to UNIT
	display the version number and copyright information of GNU Parted

Первым делом нужно изучить текущую разметку разделов. Для этого введите команду «print», чтобы вывести таблицу разделов устройства.

```
(parted) print
Model: WDC WD25 00BEVT-00A0RT0 (scsi)
Disk /dev/sdd: 250GB
Sector size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:
Number  Start   End     Size    Type     File system  Flags
  1      1049kB  250GB   250GB   primary  fat32        lba
```

Мы видим, что наш диск имеет 250 ГБ свободного места в одном разделе FAT32.

Чтобы создать новые разделы, сначала нужно удалить текущий. Это легко сделать с помощью команды `rm`.

```
(parted) rm 1
(parted) print
Model: WDC WD25 00BEVT-00A0RT0 (scsi)
Disk /dev/sdd: 250GB
Sector size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:
Number  Start   End     Size    Type     File system  Flags
```

Мы указываем номер раздела, который хотим удалить, а затем снова просматриваем таблицу разделов, чтобы убедиться, что раздел исчез.

Затем мы создадим новые разделы. Для этого мы используем `mkpart, nd`.

```
(parted) mkpart
Partition type? primary/extended? primary
File system type? [ext2]? ext4
Start? 1
End? 120000
```

При создании раздела на этом диске (использующем таблицу разделов типа MBR, master boot record) мы указываем либо основной, либо расширенный раздел, а затем — начало и конец раздела с шагом в один мегабайт по умолчанию. Использование

значения конца 120 000 МБ займет примерно половину доступного на диске пространства.

Затем мы создадим второй раздел, используя тот же метод.

```
(parted) mkpart
Partition type? primary/extended? primary
File system type? [ext2]? ntfs
Start? 120001
End? 240000
```

Наконец, изучим таблицу разделов, чтобы увидеть результат.

```
(parted) print
Model: WDC WD25 00BEVT-00A0RT0 (scsi)
Disk /dev/sdd: 250GB
Sector size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:
Number Start End Size Type File system Flags
1 1049kB 120GB 120GB primary ext4 lba
2 120GB 240GB 120GB primary ntfs lba
```

Теперь, когда разметка завершена, мы можем выйти из программы parted.

```
(parted) quit
```

Если мы снова запустим `lsblk`, то увидим наш диск и два его раздела.

```
[me@linuxbox ~]$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda 8:0 0 111.8G 0 disk
├─sda1 8:1 0 976M 0 part /boot/efi
└─sda3 8:3 0 14.9G 0 part [SWAP]
sdb 8:16 0 931.5G 0 disk
├─sdb1 8:17 0 922.2G 0 part /home
└─sdb2 8:18 0 9.3G 0 part
sdd 8:48 0 232.9G 0 disk
├─sdd1 8:49 0 111.8G 0 part
└─sdd2 8:50 0 111.8G 0 part
sr0 11:0 1 1024M 0 rom
```

Создание новой файловой системы с помощью `mkfs`

После редактирования разделов пора создать (т. е. отформатировать) новые файловые системы на диске. Для этого мы воспользуемся `mkfs` (сокращение от «make filesystem»), которая поддерживает множество форматов. Чтобы создать файловую систему `ext4`, мы используем опцию `-t` для указания типа `ext4`, затем добавим описательную метку тома и имя устройства с разделом, который нужно отформатировать.

```
[me@linuxbox ~]$ sudo mkfs -t ext4 -L EXT4_Disk /dev/sdd1
mke2fs 1.46.5 (30-Dec-2021)
```

```

Creating filesystem with 29296640 4k blocks and 7331840 inodes
Filesystem UUID: 3365d135-d8d9-4b93-a57a-ec3567c8548a
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632,
    2654208, 4096000, 7962624, 11239424, 20480000, 23887872
Allocating group tables: done
Writing inode tables: done
Creating journal (131072 blocks): done
Writing superblocks and filesystem accounting information: done

```

Теперь создадим раздел NTFS.

```

[me@linuxbox ~]$ sudo mkfs -t ntfs --quick -L NTFS_Disk /dev/sdd2
Cluster size has been automatically set to 4096 bytes.
Creating NTFS volume structures.
mkntfs completed successfully. Have a nice day.

```

Снова указываем тип файловой системы, описательную метку тома и устройство. Мы добавляем опцию `--quick`, чтобы пропустить проверку на битые блоки, так как она занимает много времени.

Примечание: разные типы файловых систем поддерживают разные опции. Подробности см. в `mkfs man page`.

Работа завершена. Отключите и снова подключите накопитель, чтобы система автоматически примонтировала его. Повторный запуск `lsblk` покажет результат.

```

[me@linuxbox ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda          8:0    0 111.8G  0 disk
├─sda1       8:1    0   976M  0 part /boot/efi
└─          /
└─sda3       8:3    0  14.9G  0 part [SWAP]
sdb          8:16    0 931.5G  0 disk
├─sdb1       8:17    0 922.2G  0 part /home
└─sdb2       8:18    0   9.3G  0 part
sdd          8:48    0 232.9G  0 disk
├─sdd1       8:49    0 111.8G  0 part /media/me/EXT4_Disk
└─sdd2       8:50    0 111.8G  0 part /media/me/NTFS_Disk
sr0         11:0    1 1024M  0 rom

```

Как видно, метки томов используются для создания имен точек монтирования в каталоге `/media`, куда автоматически монтируются съемные устройства.

Тестирование и восстановление файловых систем

В предыдущем обсуждении файла `/etc/fstab` мы видели загадочные цифры в конце каждой строки. При каждой загрузке система регулярно проверяет целостность файловых систем перед их монтированием. Это делает программа `fsck` (сокращение от «filesystem check»). Последнее число в каждой

записи `fstab` указывает порядок проверки устройств. В нашем предыдущем примере видно, что сначала проверяется корневая файловая система, а затем файловые системы `home` и `boot`. Устройства, у которых последняя цифра равна нулю, регулярно не проверяются.

Помимо проверки целостности, `fsck` может также восстанавливать поврежденные файловые системы с разной степенью успеха, в зависимости от объема повреждений. В Unix-подобных файловых системах восстановленные части файлов помещаются в каталог `lost+found`, расположенный в корне каждой файловой системы.

Чтобы проверить наш раздел `EXT4_Disk` (который сначала следует размонтировать), мы можем сделать следующее:

```
[me@linuxbox ~]$ sudo umount /dev/sdd1
[me@linuxbox ~]$ sudo fsck /dev/sdd1
fsck from util-linux 2.37.2
e2fsck 1.46.5 (30-Dec-2021)
EXT4_Disk: clean, 11/7331840 files, 606693/29296640 blocks
```

Сегодня повреждение файловой системы случается редко, если только нет проблем с оборудованием, например, неисправности диска. В большинстве систем повреждение, обнаруженное при загрузке, остановит процесс и предложит запустить `fsck`, прежде чем продолжить.

Что такое `fsck`?

В культуре Unix слово `fsck` часто используют вместо популярного с лова, с которым оно совпадает в трех буквах. Это особенно уместно, учитывая, что вы, скорее всего, произнесете это слово, если окажетесь в ситуации, когда вам придется запустить `fsck`.

Прямое перемещение данных между устройствами

Обычно мы представляем данные на компьютере в виде файлов, но их можно рассматривать и в «сыром» виде. Например, дисковый накопитель состоит из множества «блоков» данных, которые операционная система интерпретирует как каталоги и файлы. Однако, если рассматривать накопитель просто как набор блоков данных, можно выполнять полезные задачи, такие как клонирование устройств.

Эту задачу выполняет программа `dd`. Она копирует блоки данных из одного места в другое. Она использует уникальный синтаксис (по историческим причинам) и обычно применяется так:

```
dd if=input_file of=output_file [bs=block_size [count=blocks]]
```

Предупреждение: команда `dd` очень мощная. Хотя её название происходит от «data definition», её иногда называют «destroy disk», так как пользователи часто ошибаются в спецификациях `if` или `of`. Всегда перепроверяйте параметры ввода и вывода перед нажатием `ENTER`!

Допустим, у нас есть две USB-флешки одинакового размера, и мы хотим в точности скопировать содержимое первой на вторую. Если подключить оба накопителя к компьютеру и они будут назначены устройствам `/dev/sdb` и `/dev/sdc` соответственно, мы сможем скопировать всё с первого диска на второй с помощью следующей команды:

```
[me@linuxbox ~]$ sudo dd if=/dev/sdb of=/dev/sdc
```

Иначе говоря, если к компьютеру подключено только первое устройство, мы можем скопировать его содержимое в обычный файл для последующего восстановления или копирования.

```
[me@linuxbox ~]$ sudo dd if=/dev/sdb of=flash_drive.img
```

Создание образов CD-ROM

Запись CD-ROM (CD-R или CD-RW) состоит из двух этапов.

1. Создание ISO-образа, являющегося точной копией файловой системы CD-ROM;
2. Запись файла образа на носитель CD-ROM

Создание образа CD-ROM

Чтобы создать ISO-образ существующего CD-ROM, можно использовать `dd`, чтобы считать все блоки данных с CD-ROM и скопировать их в локальный файл. Допустим, у нас есть диск Ubuntu, и мы хотим создать ISO-файл, который позже можно будет использовать для создания новых копий. Вставив диск и

определив имя его устройства (предположим, что это /dev/cdrom), мы можем создать ISO-файл следующим образом:

```
dd if=/dev/cdrom of=ubuntu.iso
```

Этот метод работает и для DVD с данными, но не подходит для аудио-CD, так как они не используют файловую систему. Для аудио-CD изучите команду `cdrdao`.

Создание образа из набора файлов

Чтобы создать ISO-образ, содержащий содержимое каталога, мы используем `genisoimage` program. Сначала создаем каталог со всеми нужными файлами, а затем выполняем команду `genisoimage` для создания файла образа. Например, если мы создали каталог `~/cd-rom-files` и наполнили его файлами для CD-ROM, то можем создать файл образа с именем `cd-rom.iso` следующей командой:

```
genisoimage -o cd-rom.iso -R -J ~/cd-rom-files
```

Опция `-R` добавляет метаданные для расширений Rock Ridge, что позволяет использовать длинные имена файлов и права доступа в стиле POSIX. Аналогично, опция `-J` включает расширения Joliet, поддерживающие длинные имена файлов в Windows.

Программа под любым именем...

В онлайн-руководствах по созданию и записи оптических носителей, таких как CD-ROM и DVD, вы часто встретите две программы — `mkisofs` и `cdrecord cdrtools 2006`. Эти программы входили в состав популярного пакета, созданного Йоргом Шиллингом. Летом г-н Шиллинг изменил лицензию части пакета `cdrtools`, что, по мнению многих в сообществе Linux, привело к несовместимости с GNU GPL. В результате появился форк проекта `cdrtools`, который теперь включает программы для замены `cdrecord` и `mkisofs`, называемые `wodim` и `genisoimage` соответственно.

Запись образов CD-ROM

Имея ISO-образ, мы можем записать его на оптический носитель. Большинство команд, которые мы рассмотрим ниже, применимы как к CD-ROM, так и к DVD-носителям с возможностью записи.

Прямое монтирование ISO-образа

Существует хитрость, позволяющая смонтировать ISO-образ прямо с жесткого диска, обращаясь с ним так, будто он уже на оптическом носителе. Добавив опцию «-o loop» к `mount` (вместе с необходимым типом файловой системы «-t iso9660»), мы можем смонтировать файл образа как устройство и подключить его к дереву файловой системы.

```
mkdir /mnt/iso_image  
mount -t iso9660 -o loop image.iso /mnt/iso_image
```

В приведенном выше примере мы создали точку монтирования с именем `/mnt/iso_image`, а затем примонтировали файл образа `image.iso` в эту точку. После монтирования образ можно использовать так же, как обычный CD-ROM или DVD. Не забывайте размонтировать образ, когда он станет не нужен.

Очистка перезаписываемого CD-ROM

Перезаписываемые CD-RW носители необходимо стереть или очистить перед повторным использованием. Для этого можно использовать `wodim`, указав имя устройства CD-привода и тип очистки. Программа `wodim` предлагает несколько вариантов. Самый минимальный (и быстрый) — тип «fast».

```
wodim dev=/dev/cdrw blank=fast
```

Запись образа

Для записи образа мы снова используем `wodim`, указывая имя устройства оптической записи и имя файла образа.

```
wodim dev=/dev/cdrw image.iso
```

Помимо имени устройства и файла образа, `wodim` поддерживает множество опций. Две распространенные — «-v» для подробного вывода и «-dao» для записи диска в режиме «disc-at-once». Этот режим следует использовать при подготовке диска для коммерческого тиражирования. Режим по умолчанию для `wodim` — «track-at-once», который удобен для записи музыкальных треков.

Проверка данных

Раз уж мы заговорили об ISO-файлах, рассмотрим проверку целостности загружаемых файлов. Часто при загрузке больших файлов, таких как образ для установки Linux, важно убедиться,

что файл скачан полностью и не поврежден. Одним из инструментов для этого является `sha256sum` — современная замена более старой программы `md5sum`.

Существует два распространенных способа сделать это. Давайте их опробуем. Допустим, мы хотим скачать образ Linux Mint 22. Переходим на сайт Linux Mint и берем файл `linuxmint-22-cinnamon-64bit.iso`. Там же мы скачиваем файл контрольных сумм под названием `sha256sum.txt`. Его содержимое выглядит так:

```
[me@linuxbox ~]$ cat sha256sum.txt
7a04b54830004e945c1eda6ed6ec8c57ff4b249de4b331bd021a849694f29b8f *lin
uxmint-22-cinnamon-64bit.iso
78a2438346cfe69a1779b0ac3fc05499f8dc7202959d597dd724a07475bc6930 *lin
uxmint-22-mate-64bit.iso
55e917b99206187564029476f421b98f5a8a0b6e54c49ff6a4cb39dcfeb4bd80 *lin
uxmint-22-xfce-64bit.iso
```

Три строки данных с длинными последовательностями шестнадцатеричных чисел и именами файлов, доступных для загрузки, включая скачанный нами ISO-файл. Эти шестнадцатеричные строки — контрольные суммы, точное математическое представление ISO-файлов. Их особенность в том, что если в скачанном ISO-файле изменится хотя бы один бит, контрольная сумма станет совершенно иной. Мы проверим ISO-файл, чтобы убедиться, что контрольные суммы совпадают.

```
[me@linuxbox ~]$ sha256sum -b linuxmint-22-cinnamon-64bit.iso
7a04b54830004e945c1eda6ed6ec8c57ff4b249de4b331bd021a849694f29b8f *lin
uxmint-22-cinnamon-64bit.iso
```

Опция `-b` сообщает `sha256sum`, что мы просматриваем бинарный файл, а не текстовый. Как видим, контрольная сумма совпадает с той, что была в `sha256sum.txt`. Однако такой способ проверки довольно утомителен. Другой вариант — заставить `sha256sum` обработать файл `sha256sum.txt`, чтобы сравнить его контрольные суммы с теми, что программа генерирует для файлов из списка. Это можно сделать, запустив программу `sha256sum` следующим образом:

```
[me@linuxbox ~]$ sha256sum -c --ignore-missing sha256sum.txt
linuxmint-22-cinnamon-64bit.iso: OK
```

Опция `-c` (сокращение от «check») включает этот режим, а опция `--ignore-missing` указывает `sha256sum` не выдавать ошибки о том, что нескачанные файлы, такие как `lin-uxmint-22-mate-64bit.iso` `linuxmint-22-xfce-64bit.iso`, отсутствуют.

Итоги

В этой главе мы рассмотрели основные задачи по управлению хранилищем. Конечно, их гораздо больше. Linux поддерживает огромное количество устройств хранения и схем файловых систем. Он также предлагает множество функций для взаимодействия с другими системами.

Что почитать

Изучите man-страницы рассмотренных команд. Некоторые из них поддерживают огромное количество опций и операций. Также поищите онлайн-руководства по добавлению жестких дисков в систему Linux (их много) и работе с оптическими носителями.

- В Arch Wiki есть хорошая статья о разметке разделов:
<https://wiki.archlinux.org/title/Partitioning>
- Полное руководство пользователя GNU Parted доступно по адресу: <https://www.gnu.org/software/parted/manual/parted.html>
- Статья в Wikipedia о контрольных суммах:
<https://en.wikipedia.org/wiki/Checksum>
- Статья в Wikipedia о любимом всеми SCSI:
<https://en.wikipedia.org/wiki/SCSI>

16 — Сети

Что касается сетевых технологий, в Linux, пожалуй, нет ничего невозможного. Linux используют для создания самых разных сетевых систем и устройств, включая межсетевые экраны, маршрутизаторы, серверы имен, сетевые хранилища (NAS) и многое другое.

Тема сетей обширна, как и количество команд для их настройки и управления. Мы сосредоточимся лишь на нескольких наиболее часто используемых. Среди выбранных команд — те, что служат для мониторинга сетей и передачи файлов. Кроме того, мы изучим `ssh` программу для удаленного входа в систему. В этой главе рассматриваются следующие команды:

- `ping` — отправка `_ICMP ECHO REQUEST_` сетевым узлам
- `traceroute` — вывод трассировки маршрута пакетов к сетевому узлу
- `ip` — просмотр и управление маршрутизацией, устройствами, политиками маршрутизации и туннелями
- `netstat` — вывод сетевых соединений, таблиц маршрутизации, статистики интерфейсов, соединений `masquerade` и членства в группах `multicast`
- `ftp` — программа для передачи файлов через интернет
- `curl` - Transfer a URL
- `wget` — неинтерактивный сетевой загрузчик
- `ssh` — SSH-клиент OpenSSH (программа для удаленного входа)

Мы предполагаем наличие базовых знаний о сетях. В нынешнюю интернет-эпоху каждому пользователю компьютера необходимо понимать основы сетевых концепций. Чтобы извлечь максимум пользы из этой главы, следует ознакомиться со следующими терминами:

- IP-адрес (Internet protocol)
- Имя хоста и домена
- Унифицированный идентификатор ресурса (URI)

Полезные статьи по этим терминам см. в разделе «Что почитать» ниже.

Примечание: для некоторых команд (в зависимости от вашего дистрибутива) может потребоваться установка дополнительных пакетов из репозитория, а для

выполнения некоторых из них могут понадобиться привилегии суперпользователя.

Исследование и мониторинг сети

Даже если вы не являетесь системным администратором, анализ производительности и работы сети часто бывает полезен.

ping

Самая базовая сетевая команда — `ping`. Команда `ping` отправляет на указанный хост специальный сетевой пакет, называемый ICMP ECHO_REQUEST. Большинство сетевых устройств, получивших этот пакет, ответят на него, что позволит проверить сетевое соединение.

Примечание: Большинство сетевых устройств (включая хосты Linux) можно настроить так, чтобы они игнорировали эти пакеты. Обычно это делается в целях безопасности, чтобы частично скрыть хост от потенциального злоумышленника. Также часто брандмауэры настраивают на блокировку ICMP-трафика.

Например, чтобы проверить, доступен ли нам `linuxcommand.org` (один из наших любимых сайтов ;-), мы можем использовать `ping` следующим образом:

```
[me@linuxbox ~]$ ping linuxcommand.org
```

После запуска `ping` продолжает отправлять пакеты с заданным интервалом (по умолчанию — одна секунда), пока его не прервут.

```
[me@linuxbox ~]$ ping linuxcommand.org
PING linuxcommand.org (66.35.250.210) 56(84) bytes of data.
64 bytes from vhost.sourceforge.net (66.35.250.210): icmp_seq=1
ttl=43 time=107 ms
64 bytes from vhost.sourceforge.net (66.35.250.210): icmp_seq=2
ttl=43 time=108 ms
64 bytes from vhost.sourceforge.net (66.35.250.210): icmp_seq=3
ttl=43 time=106 ms
64 bytes from vhost.sourceforge.net (66.35.250.210): icmp_seq=4
ttl=43 time=106 ms
64 bytes from vhost.sourceforge.net (66.35.250.210): icmp_seq=5
ttl=43 time=105 ms
64 bytes from vhost.sourceforge.net (66.35.250.210): icmp_seq=6
ttl=43 time=107 ms
--- linuxcommand.org ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 6010ms
rtt min/avg/max/mdev = 105.647/107.052/108.118/0.824 ms
```

После прерывания (в данном случае после шестого пакета) нажатием `Ctrl-c`, `ping` выводится статистика

производительности. У исправно работающей сети потеря пакетов должна составлять 0 процентов. Успешный «`ping`» подтверждает, что элементы сети (сетевые карты, кабельная система, маршрутизация и шлюзы) в целом исправны.

Стоит также отметить, что машина, ответившая на `ping`, может отличаться от той, которую мы запрашивали. Это происходит потому, что один компьютер (в нашем случае `vhost.sourceforge.net`) может размещать множество сайтов с разными именами.

traceroute

Программа `traceroute` (в некоторых системах вместо неё используется аналогичная `tracert`) выводит список всех «прыжков», которые проходит сетевой трафик от локальной системы до указанного узла. Например, чтобы просмотреть маршрут до `slashdot.org`, нужно выполнить следующее:

```
[me@linuxbox ~]$ traceroute slashdot.org
```

Вывод выглядит так:

```
traceroute to slashdot.org (216.34.181.45), 30 hops max, 40 byte packets
 1  ipcop.localdomain (192.168.1.1)  1.066 ms  1.366 ms  1.720 ms
 2  * * *
 3  ge-4-13-ur01.rockville.md.bad.comcast.net (68.87.130.9)  14.622 ms  14.885 ms  15.169 ms
 4  po-30-ur02.rockville.md.bad.comcast.net (68.87.129.154)  17.634 ms  17.626 ms  17.899 ms
 5  po-60-ur03.rockville.md.bad.comcast.net (68.87.129.158)  15.992 ms  15.983 ms  16.256 ms
 6  po-30-ar01.howardcounty.md.bad.comcast.net (68.87.136.5)  22.835 ms  14.233 ms  14.405 ms
 7  po-10-ar02.whitemarsh.md.bad.comcast.net (68.87.129.34)  16.154 ms  13.600 ms  18.867 ms
 8  te-0-3-0-1-cr01.philadelphia.pa.ibone.comcast.net (68.86.90.77)  21.951 ms  21.073 ms  21.557 ms
 9  pos-0-8-0-0-cr01.newyork.ny.ibone.comcast.net (68.86.85.10)  22.917 ms  21.884 ms  22.126 ms
10  204.70.144.1 (204.70.144.1)  43.110 ms  21.248 ms  21.264 ms
11  cr1-pos-0-7-3-1.newyork.savvis.net (204.70.195.93)  21.857 ms  cr2-pos-0-0-3-1.newyork.savvis.net (204.70.204.238)  19.556 ms  cr1-pos-0-7-3-1.newyork.savvis.net (204.70.195.93)  19.634 ms
12  cr2-pos-0-7-3-0.chicago.savvis.net (204.70.192.109)  41.586 ms  42.843 ms  cr2-tengig-0-0-2-0.chicago.savvis.net (204.70.196.242)  43.115 ms
13  hr2-tengigabitethernet-12-1.elkgrovech3.savvis.net (204.70.195.122)  44.215 ms  41.833 ms  45.658 ms
14  csr1-ve241.elkgrovech3.savvis.net (216.64.194.42)  46.840 ms  43.372 ms  47.041 ms
15  64.27.160.194 (64.27.160.194)  56.137 ms  55.887 ms  52.810 ms
16  slashdot.org (216.34.181.45)  42.727 ms  42.016 ms  41.437 ms
```

В выводе видно, что для подключения от нашей тестовой системы к `slashdot.org` требуется пройти через 16 маршрутизаторов. Для маршрутизаторов, предоставивших идентификационные данные, отображаются их имена хостов, IP-адреса и данные о производительности, включая три замера времени круговой задержки от локальной системы до маршрутизатора. Для маршрутизаторов, не предоставивших данные (из-за конфигурации, перегрузки сети, брандмауэров и т. д.), вместо них отображаются звездочки, как в строке для перехода № 2. Если информация о маршрутизации заблокирована, иногда это можно обойти, добавив опцию `-T` или `-I` к команде `traceroute`.

ip

Программа `ip` — это многофункциональный инструмент настройки сети, использующий весь спектр возможностей современных ядер Linux. Она заменяет устаревшую программу `ifconfig`. С помощью `ip` можно просматривать различные сетевые настройки и статистику. Используя множество её опций, мы можем изучить разнообразные параметры нашей сети. С помощью `ip` можно проверить сетевые интерфейсы системы и таблицу маршрутизации. Сначала рассмотрим интерфейсы:

```
[me@linuxbox ~]$ ip address show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
   group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state
UP group default qlen 1000
   link/ether 00:26:6c:26:67:bf brd ff:ff:ff:ff:ff:ff
   inet 192.168.1.223/24 brd 192.168.50.255 scope global dynamic
   noprefixroute enp1s0
       valid_lft 82366sec preferred_lft 82366sec
   inet6 fe80::226:6cff:fe26:67bf/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: wlp2s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq
state DOWN group default qlen 1000
   link/ether 06:8a:59:f4:f6:d3 brd ff:ff:ff:ff:ff:ff permaddr
24:ec:99:46:aa:1e
```

В приведенном выше примере видно, что наша тестовая система имеет три сетевых интерфейса. Первый, называемый `lo`, — это петлевой интерфейс, виртуальный интерфейс, который система использует для «общения с самой собой»; второй, называемый `enp1s0`, — это интерфейс Ethernet (`en` = Ethernet); а третий, называемый `wlp2s0`, — это беспроводной интерфейс (`wl` = wireless).

При обычной диагностике сети важно проверить наличие фразы `state UP` в первой строке интерфейса (что означает, что он включен) и наличие корректного IP-адреса в поле `inet` в третьей строке. Для систем, использующих DHCP, наличие корректного IP-адреса в этом поле подтвердит работоспособность DHCP.

Далее — таблица маршрутизации:

```
[me@linuxbox ~]$ ip route show
default via 192.168.1.1 dev enp1s0 proto dhcp src 192.168.50.223
metric 100
169.254.0.0/16 dev enp1s0 scope link metric 1000
192.168.1.0/24 dev enp1s0 proto kernel scope link src 192.168.1.223
metric 100
```

В этом простом примере показана типичная таблица маршрутизации клиентской машины в локальной сети (LAN) за брандмауэром/маршрутизатором. IP-адреса, заканчивающиеся на ноль, относятся к сетям, а не к отдельным узлам, поэтому данный адрес назначения означает, что любой узел в LAN доступен напрямую. Мы видим две сети: 169.254.0.0 и 192.168.1.0. Сеть 192.168.1.0 — это наша LAN, но что такое 169.254.0.0? Это сетевой трюк под названием Automatic Private IP Addressing (APIPA). Он используется для автоматического назначения IP-адреса, если DHCP-сервер недоступен.

Первая строка содержит адрес назначения `default`. Это означает, что любой трафик, предназначенный для сети, не указанной в таблице, будет отправлен на этот адрес. В нашем примере шлюз по умолчанию определен как роутер с адресом 192.168.1.1 (типичный адрес для домашнего роутера), который, по всей видимости, знает, что делать с целевым трафиком.

Команда `ip` — это сложная программа со множеством опций и команд. Синтаксис команды состоит из:

```
ip [-options] object [command]
```

В предыдущих примерах мы использовали объекты `address` и `route` с командой `show`. Для удобства `ip` позволяет сокращать имена объектов и команд до любого однозначного префикса (часто до одного символа); так как команда по умолчанию — `show`, мы можем сократить команды до `ip a` и `ip r` и получить идентичный результат.

Передача файлов по сети

Какой толк от сети, если мы не можем передавать по ней файлы? Существует множество программ для передачи данных по сетям.

Сейчас мы рассмотрим две из них, а еще несколько — в следующих разделах.

ftp

Одна из настоящих «классических» программ, `ftp`, получила название благодаря используемому протоколу — File Transfer Protocol. Когда-то FTP был самым популярным способом загрузки файлов из Интернета. Некоторые веб-браузеры всё ещё поддерживают его, и мы можем встретить URI, начинающиеся с протокола `ftp://`.

До появления веб-браузеров существовала программа `ftp`. `ftp` используется для связи с FTP-серверами — машинами, содержащими файлы, которые можно загружать и скачивать по сети.

FTP (в исходном виде) небезопасен, так как передает имена учетных записей и пароли в открытом виде. Это означает, что они не зашифрованы, и любой, кто перехватывает сетевой трафик, может их увидеть. Из-за этого почти весь FTP в Интернете осуществляется через анонимные FTP-серверы. Анонимный сервер позволяет любому войти в систему, используя имя пользователя «anonymous» и любой пароль.

В примере ниже показана воображаемая сессия с программой `ftp`, загружающей ISO-образ Ubuntu, расположенный в каталоге `/pub/cd_images/Ubuntu-24.04` анонимного FTP-сервера `filesaver`:

```
[me@linuxbox ~]$ ftp filesaver
Connected to filesaver.localdomain.
220 (vsFTPD 2.0.1)
Name (filesaver:me): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> cd pub/cd_images/Ubuntu-24.04
250 Directory successfully changed.
ftp> ls
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
-rw-rw-r-- 1 500      500      733079552 Apr 25 03:53 ubuntu-
24.04-desktop-amd64.iso
226 Directory send OK.
ftp> lcd Desktop
Local directory now /home/me/Desktop
ftp> get ubuntu-24.04-desktop-amd64.iso
local: ubuntu-24.04-desktop-amd64.iso remote: ubuntu-24.04-desktop-
amd64.iso
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for ubuntu-24.04-desktop-
amd64.iso (733079552 bytes).
226 File send OK.
```

```
733079552 bytes received in 68.56 secs (10441.5 kB/s)
ftp> bye
```

В таблице 16-1 приведены пояснения к командам, введенным в ходе этой сессии.

Таблица 16-1: Примеры интерактивных ftp команд

Команда	Значение
ftp fileserver	Запустить ftp программу и заставить её подключение к FTP-серверу fileserver.
anonymous	Имя пользователя. После приглашения для входа, появится приглашение для ввода пароля. Некоторые серверы примут пустой пароль; другие потребуют пароль в виде адреса электронной почты. В этом случае, попробуйте что-то вроде user@example.com.
cd pub/cd_images/Ubuntu-24.04	Перейдите в каталог на удаленной системе, где находится нужный файл. Примечание: на большинстве анонимных FTP серверов файлы для публичного скачивания находятся где-то в каталоге pub.

Команда	Значение
<code>ls</code>	Выведите список каталогов на удаленной системе.
<code>lcd Desktop</code>	Измените каталог на локальной системе на <code>~/Desktop</code> . В данном примере программа <code>ftp</code> была вызывается, когда рабочий каталог был <code>~</code> . Эта команда меняет рабочий каталог на <code>~/Desktop</code> .
<code>get ubuntu-24.04-desktop-amd64.iso</code>	Дайте удаленной системе команду передать файл <code>ubuntu-24.04-desktop-amd64.iso</code> в локальную систему. Так как текущий рабочий каталог на локальной системе был изменен на <code>~/Desktop</code> , файл будет скачан туда.
<code>bye</code>	Выйдите с удаленного сервера и завершите сеанс программы <code>ftp</code> . Команды <code>quit</code> и <code>exit</code> также могут быть использованы.

Ввод `help` в `ftp>` приглашение выведет список поддерживаемых команд. Используя `ftp` на сервере с достаточными правами, можно выполнять многие обычные задачи по управлению файлами. Это неудобно, но работает.

`lftp` — лучший `ftp`

`ftp` — не единственный FTP-клиент для командной строки. На самом деле их много. Один из лучших (и более популярных) — это `lftp` от Александра Лукьянова. Он работает почти как традиционная программа `ftp`, но обладает множеством дополнительных удобств, включая поддержку нескольких протоколов (в том числе HTTP), автоматический повтор при сбоях загрузки, фоновые процессы, дополнение путей клавишей Tab и многое другое.

`curl` — Передача URL

Еще одна популярная программа для передачи файлов — `curl`. Ее базовое использование выглядит так:

```
[me@linuxbox ~]$ curl https://linuxcommand.org
```

Мы указываем URL, и `curl` загружает первую страницу этого URL, выводя результат в стандартный вывод. Можно указать несколько URL.

`curl` поддерживает большинство сетевых протоколов, включая HTTP, HTTPS, FTP, IMAP, POP3, SFTP, SMB и другие. В таблице 16-2 перечислены некоторые из множества опций, которые поддерживает `curl`.

Таблица 16-2: Распространенные опции `curl`

Параметр	Описание
<code>-o, --output file</code>	Направить вывод в указанный файл вместо стандартного вывода.
<code>-O, --remote-name</code>	Как <code>-o</code> , но с тем же именем для локального файла, что и у удаленного файла.
<code>-s, --silent</code>	Скрыть индикатор прогресса и сообщения об ошибках.
<code>-u, --proxy-user user:password</code>	Укажите пару имя пользователя/пароль комбинацию.
<code>-v, --verbose</code>	Выводить подробные сообщения по мере выполнения.

Страница `curl man` содержит все подробности.

wget — Неинтерактивный сетевой загрузчик

Еще одна программа командной строки для загрузки файлов — `wget`. Она полезна для скачивания контента как с веб-сайтов, так и с FTP-серверов. Можно загружать одиночные файлы, несколько файлов и даже целые сайты. Чтобы скачать первую страницу `linuxcommand.org`, можно выполнить это:

```
[me@linuxbox ~]$ wget http://linuxcommand.org/index.php
--11:02:51--  http://linuxcommand.org/index.php
           => `index.php'
Resolving linuxcommand.org... 66.35.250.210
Connecting to linuxcommand.org[66.35.250.210]:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [text/html]
[ <=> ] 3,120      ---K/s
11:02:51 (161.75 MB/s) - `index.php' saved [3120]
```

Множество опций программы позволяют `wget` рекурсивно скачивать данные, загружать файлы в фоновом режиме (что позволяет выйти из системы, не прерывая загрузку) и докачивать частично загруженные файлы. Эти возможности подробно описаны в отличной `man`-странице программы.

Безопасная связь с удаленными хостами

На протяжении многих лет Unix-подобные операционные системы поддерживают удаленное администрирование по сети. В ранние времена, до широкого распространения Интернета, для входа на удаленные хосты использовались несколько популярных программ, таких как `rlogin` и `telnet`. Однако эти программы страдают от того же фатального недостатка, что и `ftp`: они передают все данные (включая имена пользователей и пароли) в открытом виде. Это делает их совершенно непригодными для использования в эпоху Интернета.

ssh

Для решения этой проблемы был разработан новый протокол под названием Secure Shell (SSH). SSH решает две основные задачи безопасной связи с удаленным хостом.

1. Он подтверждает подлинность удаленного хоста (предотвращая так называемые атаки типа «человек посередине»).
2. Он шифрует все данные, передаваемые между локальным и удаленным хостами.

SSH состоит из двух частей. SSH-сервер работает на удаленном хосте, ожидая входящих соединений (по умолчанию на порту 22),

а SSH-клиент используется в локальной системе для связи с удаленным сервером.

Большинство дистрибутивов Linux поставляют реализацию SSH под названием OpenSSH от проекта OpenBSD. Некоторые дистрибутивы по умолчанию включают пакеты и клиента, и сервера, другие — только клиента. Чтобы система могла принимать удаленные подключения, в ней должен быть установлен, настроен и запущен пакет OpenSSH-server, а также (если система работает напрямую или находится за брандмауэром) должен быть разрешен входящий трафик на TCP-порт 22.

Совет: если у вас нет удаленной системы для подключения, но вы хотите опробовать эти примеры, убедитесь, что в вашей системе установлен пакет OpenSSH-server, и используйте localhost в качестве имени удаленного хоста. Таким образом, ваша машина будет создавать сетевые соединения сама с собой.

Программа SSH-клиента, используемая для подключения к удаленным SSH-серверам, называется весьма подходящим образом — ssh. Чтобы подключиться к удаленному хосту с именем remote-sys, мы воспользуемся программой-клиентом ssh следующим образом:

```
[me@linuxbox ~]$ ssh remote-sys
The authenticity of host 'remote-sys (192.168.1.4)' can't be
established.
RSA key fingerprint is
41:ed:7a:df:23:19:bf:3c:a5:17:bc:61:b3:7f:d9:bb.
Are you sure you want to continue connecting (yes/no)?
```

При первой попытке подключения появится сообщение о том, что подлинность удаленного хоста не может быть установлена. Это происходит потому, что клиентская программа еще никогда не видела этот удаленный хост. Чтобы принять учетные данные удаленного хоста, введите «yes» в ответ на запрос. После установления соединения система запросит пароль:

```
Warning: Permanently added 'remote-sys,192.168.1.4' (RSA) to the list
of known hosts.
me@remote-sys's password:
```

После успешного ввода пароля мы получим приглашение оболочки от удаленной системы.

```
Last login: Sat Aug 30 13:00:48 2025
[me@remote-sys ~]$
```

Удаленная сессия оболочки продолжается до тех пор, пока пользователь не введет команду `exit` в приглашении удаленной оболочки, тем самым закрыв удаленное соединение. В этот момент возобновляется локальная сессия оболочки и снова появляется локальное приглашение оболочки.

Подключиться к удаленным системам можно и с другим именем пользователя. Например, если у локального пользователя «`me`» есть учетная запись «`bob`» на удаленной системе, пользователь `me` может войти в учетную запись `bob` на удаленной системе следующим образом:

```
[me@linuxbox ~]$ ssh bob@remote-sys
bob@remote-sys's password:
Last login: Sat Aug 30 13:03:21 2025
[bob@remote-sys ~]$
```

Как упоминалось ранее, `ssh` проверяет подлинность удаленного хоста. Если аутентификация удаленного хоста не удалась, появится следующее сообщение:

```
[me@linuxbox ~]$ ssh remote-sys
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@    WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!     @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle
attack)!
It is also possible that a host key has just been changed.
The fingerprint for the ECDSA key sent by the remote host is
SHA256:9uh0XHzPz0AjIs0ZDanNYfv7ksU04Mjy5pR4kUTSKkA.
Please contact your system administrator.
Add correct host key in /home/me/.ssh/known_hosts to get rid of this
message.
Offending ECDSA key in /home/me/.ssh/known_hosts:42
  remove with:
    ssh-keygen -f "/home/me/.ssh/known_hosts" -R "remote-sys"
Host key for remote-sys has changed and you have requested strict
checking.
Host key verification failed.
```

Это сообщение вызвано одной из двух возможных причин. Во-первых, злоумышленник может совершать атаку типа «человек посередине». Это случается редко, так как все знают, что `ssh` предупреждает пользователя об этом. Более вероятная причина — изменения на удаленной системе; например, была переустановлена операционная система или SSH-сервер. Однако в целях безопасности нельзя сразу отбрасывать первый вариант. При появлении этого сообщения всегда уточняйте ситуацию у администратора удаленной системы.

Если выяснится, что сообщение вызвано безобидной причиной, проблему можно безопасно исправить на стороне клиента. Для этого воспользуйтесь советом из предупреждения:

```
ssh-keygen -f "/home/me/.ssh/known_hosts" -R "remote-sys"
```

В противном случае можно использовать текстовый редактор (vim, возможно), чтобы удалить устаревший ключ из `~/.ssh/known_hosts`. В приведенном выше примере это выглядит так:

```
Offending key in /home/me/.ssh/known_hosts:42
```

Это означает, что 42-я строка `known_hosts` содержит ошибочный ключ. Удалите эту строку из файла, и `ssh` сможет принять новые учетные данные от удаленной системы.

Помимо открытия сессии оболочки в удаленной системе, `ssh` позволяет выполнить одну команду на удаленной системе. Например, чтобы выполнить команду `free` на удаленном хосте `remote-sys` и вывести результат в локальной системе, используйте это:

```
[me@linuxbox ~]$ ssh remote-sys free
me@twin4's password:
total      used      free      shared    buffers    cached
Mem:       775536  507184    268352          0     110068    154596
-/+ buffers/cache:  242520  533016
Swap:      1572856        0    1572856
[me@linuxbox ~]$
```

Этот метод можно использовать и интереснее, например, в следующем примере, где мы выполняем `ls` в удаленной системе и перенаправляем вывод в файл в локальной системе:

```
[me@linuxbox ~]$ ssh remote-sys 'ls *' > dirlist.txt
me@twin4's password:
[me@linuxbox ~]$
```

Обратите внимание на использование одинарных кавычек в приведенной выше команде. Это сделано для того, чтобы не допускать раскрытия пути на локальной машине; вместо этого мы хотим, чтобы оно выполнялось на удаленной системе. Аналогично, если бы мы хотели перенаправить вывод в файл на удаленной машине, мы могли бы поместить оператор перенаправления и имя файла внутрь одинарных кавычек.

```
[me@linuxbox ~]$ ssh remote-sys 'ls * > dirlist.txt'
```

Туннелирование с помощью SSH

При установке SSH-соединения с удаленным хостом между локальной и удаленной системами создается зашифрованный туннель. Обычно этот туннель

используется для безопасной передачи команд с локальной системы на удаленную и возврата результатов обратно. Помимо этой базовой функции, протокол SSH позволяет пропускать через зашифрованный туннель почти любые типы сетевого трафика, создавая своего рода виртуальную частную сеть (VPN) между локальной и удаленной системами.

Пожалуй, наиболее часто эта функция используется для передачи трафика системы X Window. В системе, где все еще работает X server (в отличие от современных систем на Wayland), можно запустить X-клиент (графическое приложение) на удаленной системе так, чтобы его окно отображалось на локальной системе. Это сделать просто; вот пример. Допустим, мы работаем за системой Linux с именем `linuxbox`, на которой запущен X server, и хотим запустить программу `xload` на удаленной системе `remote-sys`, чтобы увидеть ее графический вывод на нашем экране. Мы можем сделать следующее:

```
[me@linuxbox ~]$ ssh -X remote-sys me@remote-sys's password:Last  
login: Mon Sep 08 13:23:11 2025 [me@remote-sys ~]$ xload
```

После выполнения команды `xload` на удаленной системе её окно появится в локальной системе. На некоторых системах для этого может потребоваться опция «-Y» вместо «-X».

scp и sftp

Пакет OpenSSH также включает две программы, использующие зашифрованный SSH-туннель для копирования файлов по сети. Первая, `scp` (secure copy), работает почти так же, как привычная программа `cp`. Главное отличие в том, что перед путями источника или назначения может стоять имя удаленного хоста, за которым следует двоеточие. Например, если мы хотим скопировать документ с именем `document.txt` из нашего домашнего каталога в удаленной системе `remote-sys` в текущий рабочий каталог нашей локальной системы, это можно сделать так:

```
[me@linuxbox ~]$ scp remote-sys:document.txt .  
me@remote-sys's password:  
document.txt                                100% 5581      5.5KB/s   00:00  
[me@linuxbox ~]$
```

Как и в случае с `ssh`, вы можете добавить имя пользователя перед именем удаленного хоста, если имя учетной записи на удаленном хосте не совпадает с локальным.

```
[me@linuxbox ~]$ scp bob@remote-sys:document.txt .
```

Вторая программа для копирования файлов по SSH — это `sftp`, которая, как следует из названия, является безопасной заменой программы `ftp`. `sftp` работает почти так же, как оригинальная программа `ftp`, которую мы использовали ранее; однако вместо передачи данных в открытом виде она использует зашифрованный туннель SSH. `sftp` имеет важное преимущество перед обычным `ftp`: ей не требуется запущенный FTP-сервер на удаленном хосте. Нужен только SSH-сервер. Это означает, что любая удаленная машина, к которой можно подключиться через SSH-клиент, также может использоваться как FTP-подобный сервер. Вот пример сессии:

```
[me@linuxbox ~]$ sftp remote-sys
Connecting to remote-sys...
me@remote-sys's password:
sftp> ls
ubuntu-24.04-desktop-amd64.iso
sftp> lcd Desktop
sftp> get ubuntu-24.04-desktop-amd64.iso
Fetching /home/me/ubuntu-24.04-desktop-amd64.iso to ubuntu-24.04-
desktop-amd64.iso
/home/me/ubuntu-24.04-desktop-amd64.iso 100% 699MB 7.4MB/s 01:35
sftp> bye
```

Совет: протокол SFTP поддерживается многими графическими файловыми менеджерами в дистрибутивах Linux. В GNOME или KDE можно ввести URI, начинающийся с `sftp://`, в строку адреса и работать с файлами на удаленной системе с запущенным SSH-сервером.

SSH-клиент для Windows?

Предположим, вы сидите за компьютером с Windows, но вам нужно войти на свой Linux-сервер для работы. Что делать? Конечно, установить SSH-клиент для Windows! Существует множество таких программ. Самая популярная — вероятно, PuTTY от Саймона Тэтама и его команды. PuTTY отображает окно терминала и позволяет пользователю Windows открыть SSH (или telnet) сессию на удаленном хосте. Программа также предоставляет аналоги для `scp` и `sftp`.

PuTTY доступен по адресу

<http://www.chiark.greenend.org.uk/~sgtatham/putty/>

Итоги

В этой главе мы рассмотрели несколько сетевых инструментов, доступных в большинстве систем Linux. Поскольку Linux широко используется в серверах и сетевом оборудовании, существует множество других инструментов, которые можно добавить, установив дополнительное ПО. Но даже с базовым набором можно выполнять множество полезных сетевых задач.

Что почитать

- Для общего (хотя и устаревшего) обзора администрирования сетей Linux Documentation Project предлагает Linux Network Administrator's Guide: <https://tldp.org/LDP/nag2/index.html>
- В Wikipedia много хороших статей о сетях. Вот основы:
https://en.wikipedia.org/wiki/Internet_protocol_address
https://en.wikipedia.org/wiki/Host_name
https://en.wikipedia.org/wiki/Uniform_Resource_Identifier
- На curl home page представлено отличное руководство с множеством примеров того, что curl умеет:
<https://curl.se/docs/tutorial.html>
- Хотите узнать, как и почему работает Automatic Private IP Addressing (APIPA)? Читайте здесь:
<https://www.33rdsquare.com/what-does-169-254-0-0-mean/>

17 — Поиск файлов

Пока мы изучали нашу систему Linux, стало предельно ясно одно: в типичной системе Linux огромное количество файлов! Возникает вопрос: «Как же нам искать нужное?» Мы уже знаем, что файловая система Linux хорошо организована согласно традициям, передаваемым от одного поколения Unix-подобных систем к другому, но само число файлов может стать серьезной проблемой.

В этой главе мы рассмотрим два инструмента для поиска файлов в системе.

- `locate` — Поиск файлов по имени
- `find` — Поиск файлов в иерархии каталогов

Мы также изучим команду, которую часто используют вместе с инструментами поиска для обработки полученного списка файлов.

- `xargs` — создание и выполнение командных строк из стандартного ввода

Кроме того, мы представим пару команд, которые помогут нам в исследованиях.

- `touch` — изменение времени файлов
- `stat` — отображение состояния файла или файловой системы

`locate` — простой поиск файлов

Программа `locate` выполняет быстрый поиск по базе данных путей и выводит все имена, соответствующие заданной подстроке. Допустим, мы хотим найти все программы, имена которых начинаются с `zip`. Поскольку мы ищем программы, можно предположить, что имя каталога, содержащего их, заканчивается на `bin/`. Таким образом, мы можем использовать `locate` следующим образом, чтобы найти наши файлы:

```
[me@linuxbox ~]$ locate bin/zip
```

`locate` выполнит поиск в базе данных путей и выведет все, что содержат строку `bin/zip`.

```
/usr/bin/zip  
/usr/bin/zipcloak  
/usr/bin/zipgrep  
/usr/bin/zipinfo  
/usr/bin/zipnote  
/usr/bin/zipsplit
```

Если требования к поиску сложнее, можно комбинировать `locate` с другими инструментами, такими как `grep`, для создания более гибких запросов.

```
[me@linuxbox ~]$ locate zip | grep bin
/bin/bunzip2
/bin/bzip2
/bin/bzip2recover
/bin/gunzip
/bin/gzip
/usr/bin/funczip
/usr/bin/gpg-zip
/usr/bin/preunzip
/usr/bin/prezip
/usr/bin/prezip-bin
/usr/bin/unzip
/usr/bin/unzipsfx
/usr/bin/zip
/usr/bin/zipcloak
/usr/bin/zipgrep
/usr/bin/zipinfo
/usr/bin/zipnote
/usr/bin/zipsplit
```

Программа `locate` существует уже много лет, и сейчас широко используются несколько её вариантов. Два наиболее распространенных в современных дистрибутивах Linux — это `pllocate` и `mlocate`, хотя обычно к ним обращаются через символическую ссылку `locate`. Различные версии `locate` имеют пересекающиеся наборы опций. Некоторые версии поддерживают регулярные выражения (мы разберем их в главе 19) и подстановочные знаки. Чтобы узнать, какая версия `locate` установлена, изучите man-страницу `locate`.

Откуда берется база данных `locate`?

Вы можете заметить, что в некоторых дистрибутивах `locate` не работает сразу после установки системы, но на следующий день всё в порядке. В чем причина? База данных `locate` создается другой программой под названием `updatedb`. Обычно она запускается периодически как задание `cron` — то есть задача, выполняемая через регулярные интервалы демоном `cron`. В большинстве систем с `locate` программа `updatedb` запускается раз в день. Поскольку база данных не обновляется непрерывно, вы заметите, что совсем новые файлы не отображаются при использовании `locate`. Чтобы это исправить, можно запустить программу `updatedb` вручную: станьте суперпользователем и введите `updatedb` в приглашении.

find — Поиск файлов сложным путем

Если программа `locate` может найти файл только по имени, то программа `find` ищет файлы в заданном каталоге (и его подкаталогах) на основе множества различных атрибутов. Мы уделим много времени `find`, так как она обладает множеством интересных возможностей, которые будут постоянно встречаться нам при изучении концепций программирования в следующих главах.

В простейшем случае `find` получает одно или несколько имен каталогов для поиска. Например, чтобы вывести список содержимого нашего домашнего каталога, можно использовать следующее:

```
[me@linuxbox ~]$ find ~
```

В большинстве активных учетных записей это создаст длинный список. Поскольку список направляется в стандартный вывод, мы можем передать его через конвейер в другие программы. Давайте воспользуемся `wc`, чтобы подсчитать количество файлов.

```
[me@linuxbox ~]$ find ~ | wc -l  
47068
```

Ого, мы изрядно потрудились! Прелесть `find` в том, что она позволяет находить файлы по заданным критериям. Это достигается (пусть и несколько необычным) применением опций, тестов и действий. Сначала мы разберем тесты.

Тесты

Допустим, в результатах поиска нам нужен список каталогов. Для этого можно добавить следующий тест:

```
[me@linuxbox ~]$ find ~ -type d | wc -l  
1695
```

Добавление теста `-type d` ограничило поиск каталогами. И наоборот, поиск можно было ограничить обычными файлами с помощью этого теста:

```
[me@linuxbox ~]$ find ~ -type f | wc -l  
38737
```

В таблице 17-1 перечислены распространенные тесты типов файлов, поддерживаемые `find`.

Таблица 17-1: find Типы файлов

Тип файла	Описание
b	Файл блочного специального устройства
c	Файл символьного специального устройства
d	Каталог
f	Обычный файл
l	Символическая ссылка

Мы также можем искать по размеру и имени файла, добавив дополнительные тесты. Давайте найдем все обычные файлы, соответствующие подстановочному знаку *.JPG, размер которых превышает 1 МБ.

```
[me@linuxbox ~]$ find ~ -type f -name "*.JPG" -size +1M | wc -l
840
```

В этом примере мы добавляем -name test, а затем подстановочный знак. Обратите внимание, что мы заключаем его в кавычки, чтобы оболочка не выполняла раскрытие пути. Далее мы добавляем -size test и строку «+1M». Знак плюс в начале означает поиск файлов размером больше указанного числа. Знак минус изменит значение на «меньше указанного числа». Отсутствие знака означает «соответствует значению точно». Буква М в конце указывает, что единицей измерения являются мегабайты. В таблице 17-2 перечислены символы, которые можно использовать для указания единиц измерения.

Таблица 17-2: *find* Единицы размера

Символ	Unit
b	Блоки по 512 байт. Используется по умолчанию, если единица не указана.
c	Байты.
w	2-байтовые слова.
k	Килобайты (единицы по 1 024 байта).
M	Мегабайты (единицы по 1 048 576 байт).
G	Гигабайты (единицы по 1 073 741 824 байт).

find поддерживает множество тестов. В таблице 17-3 перечислены наиболее распространенные из них. Примечание: если требуется числовой аргумент, можно использовать ту же нотацию + и -, что обсуждалась выше.

Таблица 17-3: *find* Тесты и и ие

Test	Описание
-cmin n	Найти файлы или каталоги, содержимое или атрибуты которых был изменено ровно n мин. назад. Чтобы указать менее n мин. назад, используйте -n, а чтобы указать более n мин. назад, используйте +n.
-cnewer file	Найти файлы или каталоги, содержимое или атрибуты которых был изменены позже, чем y file.
-ctime n	Найти файлы или каталоги, содержимое или атрибуты которых (так (как владельца или права) были изменены n*24 часов назад.
-empty	Выбирать пустые файлы и каталоги.
-group name	Выбирать файлы или каталоги, принадлежащие group. group может записываться как имя группы или как числовой идентификатор группы ID.
-iname pattern	Как тест -name, но без учета регистра.
-inum n	Поиск файлов с номером inode n. Это полезно для поиска все жесткие ссылки на конкретный inode.
-mmin n	Найти файлы или каталоги, содержимое которых было изменено n минут назад.
-mtime n	Найти файлы или каталоги, содержимое которых было изменено n*24 часов назад.
-name pattern	Сопоставление файлов и каталогов по указанному подстановочному знаку pattern.
-newer file	Сопоставление файлов и каталогов, содержимое которых было изменено не позднее указанного file. Это полезно при написании сценариев оболочки для резервного копирования файлов. Каждый раз, когда вы создаете резервную копию, обновляете файл (например, лог), а затем используйте find, чтобы определить, какие файлы изменились с последнего обновления.

Test	Описание
-nouser	Находит файлы и каталоги, не принадлежащие существующему пользователю. Это помогает найти файлы удаленных учетных записей или обнаружить действия злоумышленников.
-nogroup	Находит файлы и каталоги, не принадлежащие существующему группе.
-perm mode	Выбирает файлы или каталоги с правами доступа, соответствующими mode. mode можно указать в восьмеричном или символьном виде.
-samefile name	Аналогично тесту -inum. Выбирает файлы с тем же номером inode, что и файл name.
-size n	Поиск файлов размером n.
-type c	Поиск файлов типа c.
-user name	Поиск файлов или каталогов, принадлежащих пользователю name. Пользователь может быть указан как по имени, так и по числовому ID.

Это не полный список. Все подробности — в `find man page`.

Операторы

Даже со всеми тестами, которые предоставляет `find`, нам может понадобиться более эффективный способ описания логических связей между ними. Например, что если нужно проверить, имеют ли все файлы и подкаталоги в каталоге безопасные права доступа? Нам пришлось бы искать все файлы с правами, отличными от 0600, и каталоги с правами, отличными от 0700. К счастью, `find` позволяет комбинировать тесты с помощью логических операторов для создания более сложных связей. Чтобы выразить вышеупомянутую проверку, можно сделать следующее:

```
[me@linuxbox ~]$ find ~ \( -type f -not -perm 0600 \) -or \( -type d -not -perm 0700 \)
```

Ого! Выглядит странно. Что это вообще такое? На самом деле, операторы не так уж сложны, если в них разобраться. В таблице 17-4 описаны логические операторы, используемые с `find`.

Таблица 17-4: *find* Логические операторы. Это, это ы

Оператор	Описание
- and	Соответствие, если условия по обе стороны оператора истинны. Это можно сократить до - a. Примечание: если оператор не указан, по умолчанию подразумевается - and.
- or	Соответствие, если истинно любое из условий по обе стороны от оператора можно сократить до - o.
- not	Соответствие, если условие после оператора ложно. Это можно сократить с помощью восклицательного знака (!).
()	объединяет тесты и операторы в более крупные выражения. Это используется для управления приоритетом логических вычислений. По умолчанию <i>find</i> вычисляется слева на право. Часто требуется изменить порядок вычислений по умолчанию, чтобы получить нужный результат. Даже если это не, не требуется иногда полезно добавлять группирующие символы, чтобы улучшить читаемость команды. Примечание: так как скобки имеют специальное значение для оболочки, их нужно брать в кавычки при использовании в командной строке, чтоб они передавались как аргументы для <i>find</i> . Обычно для их экранирования используется обратный слэш. Также можно важно, чтобы символы (и) были окружены пробелами, чтобы отделить их от других слов в команде. Например, <i>find ~ \(-type f \)</i>

Имея этот список операторов, разберем нашу *find* команду. Если смотреть с самого верхнего уровня, мы увидим, что наши тесты разделены на две группы -or оператором.

(expression 1) -or (expression 2)

Это логично, так как мы ищем файлы с одним набором прав и каталоги с другим. Если мы ищем и файлы, и каталоги, почему мы используем -or вместо -and? Когда *find* сканирует файлы и каталоги, каждый объект проверяется на соответствие заданным условиям. Нам нужно узнать, является ли объект либо файлом с плохими правами, либо каталогом с плохими правами.

Одновременно это быть не может. Если развернуть сгруппированные выражения, это будет выглядеть так:

(file with bad perms) -or (directory with bad perms)

Наша следующая задача — как проверить «плохие права». Как это сделать? На самом деле, никак. Мы будем проверять «неправильные права», так как знаем, какие права являются «правильными». Для файлов правильными мы считаем 0600, а для каталогов — 0700. Выражение для проверки файлов на «неправильные» права выглядит так:

```
-type f -and -not -perm 0600
```

Для каталогов это выглядит так:

```
-type d -and -not -perm 0700
```

Как указано в таблице 17-4 выше, оператор `-and` можно безопасно удалить, так как он подразумевается по умолчанию. Если собрать всё воедино, мы получим итоговую команду.

```
find ~ ( -type f -not -perm 0600) -or ( -type d -not -perm 0700)
```

Однако, поскольку скобки имеют особое значение для оболочки, их нужно экранировать, чтобы оболочка не пыталась их интерпретировать. Для этого достаточно поставить перед каждой из них обратный слэш.

Важно понимать еще одну особенность логических операторов. Допустим, у нас есть два выражения, разделенных логическим оператором.

```
expr1 -operator expr2
```

Во всех случаях `expr1` будет выполнено; однако оператор определяет, будет ли выполнено `expr2`. В таблице 17-5 описан принцип работы.

Таблица 17-5: Логика `find` AND/OR

Результаты <code>expr1</code>	Оператор	<code>expr2</code> — это...
True	-and	Всегда выполняется
Ло жь	-and	Ни разу не выполнялось
True	-or	Ни разу не выполнялось
Ло жь	-or	Всегда выполняется

Почему это происходит? Это сделано для повышения производительности. Возьмем, к примеру, `-and`. Мы знаем, что выражение `expr1 -and expr2` не может быть истинным, если результат `expr1` ложен, поэтому нет смысла выполнять `expr2`.

Точно так же, если у нас есть выражение `expr1 -or expr2` и результат `expr1` истинен, нет смысла выполнять `expr2`, так как мы уже знаем, что выражение `expr1 -or expr2` истинно.

Хорошо, это ускоряет процесс. Почему это важно? Это важно, потому что мы сможем полагаться на такое поведение для управления выполнением действий, как вскоре увидим.

Предопределенные действия

Приступим к делу! Иметь список результатов выполнения команды `find` полезно, но на самом деле нам нужно совершать действия с этими элементами. К счастью, `find` позволяет выполнять действия на основе результатов поиска. Существует набор предопределенных действий и несколько способов применения пользовательских действий. Сначала рассмотрим некоторые предопределенные действия, перечисленные в таблице 17-6.

Таблица 17-6: Предопределенные `find` действия

Действие	Описание
<code>-delete</code>	Удалить текущий подходящий файл.
<code>-ls</code>	Выполнить эквивалент <code>ls -dils</code> над подходящим файлом. Вывод направляется в стандартный вывод.
<code>-print</code>	Вывести полный путь подходящего файла в стандартный вывод. Это действие по умолчанию, если не указано иное действие.
<code>-quit</code>	Завершить работу после нахождения совпадения.

Как и в случае с тестами, доступно множество других действий. Подробности см. в странице `find man`.

В первом примере мы сделали следующее:

```
find ~
```

Это вывело список всех файлов и подкаталогов в нашем домашнем каталоге. Это вывело список, так как действие `-print` подразумевается по умолчанию, если не указано иное. Таким образом, нашу команду можно записать так:

```
find ~ -print
```

Мы можем использовать `find`, чтобы удалять файлы, соответствующие определенным критериям. Например, чтобы

удалить файлы с расширением .bak (которое часто используется для резервных копий), можно выполнить такую команду:

```
find ~ -type f -name '*.bak' -delete
```

В этом примере в домашнем каталоге пользователя (и его подкаталогах) ищутся файлы, имена которых заканчиваются на .bak. Найденные файлы удаляются.

Предупреждение: само собой разумеется, что при использовании действия `-delete` следует проявлять крайнюю осторожность. Всегда сначала тестируйте команду, заменяя действие `-delete` на `-print`, чтобы подтвердить результаты поиска.

Прежде чем продолжить, давайте еще раз посмотрим, как логические операторы влияют на действия. Рассмотрим следующую команду:

```
find ~ -type f -name '*.bak' -print
```

Как мы уже видели, эта команда будет искать каждый обычный файл (`-type f`), имя которого заканчивается на .bak (`-name '*.bak'`), и выводить относительный путь каждого совпавшего файла в стандартный вывод (`-print`). Однако то, как работает команда, определяется логической связью между каждым тестом и действием. Помните, что по умолчанию между тестом и действием подразумевается связь `-and`. Мы также можем записать команду иначе, чтобы логические связи были нагляднее:

```
find ~ -type f -and -name '*.bak' -and -print
```

Теперь, когда команда полностью сформулирована, рассмотрим, как логические операторы влияют на её выполнение:

Тест или действие	Выполняется, если...
<code>-print</code>	<code>-type f</code> и <code>-name '*.bak'</code> истинны
<code>-name '*.bak'</code>	<code>-type f</code> истинно
<code>-type f</code>	Выполняется всегда, так как это первый тест/действие в <code>-and</code> взаимосвязь.

Поскольку логическая связь между тестами и действиями определяет, какие из них будут выполнены, порядок их

следования имеет значение. Например, если изменить порядок так, чтобы `-print action` была первой, поведение команды существенно изменится.

```
find ~ -print -and -type f -and -name '*.bak'
```

Эта версия команды будет выводить каждый файл (так как `-print action` всегда истинно), а затем проверять тип файла и указанное расширение.

Пользовательские действия

Помимо предопределенных действий, мы можем вызывать произвольные команды. Традиционный способ сделать это — использовать `-exec action`. Это действие работает следующим образом:

```
-exec command {};
```

Здесь `command` — это имя команды, `{}` — символьное представление текущего пути, а точка с запятой — обязательный разделитель, обозначающий конец команды. Вот пример использования `-exec` для имитации действия `-delete`, обсуждавшегося ранее:

```
-exec rm '{}' ';' ;'
```

Опять же, поскольку символы скобок и точки с запятой имеют специальное значение для оболочки, их нужно заключать в кавычки или экранировать.

Пользовательское действие также можно выполнять интерактивно. Если использовать `-ok action` вместо `-exec`, перед выполнением каждой указанной команды будет появляться приглашение.

```
find ~ -type f -name 'foo*' -ok ls -l '{}' ';'
< ls ... /home/me/bin/foo > ? y
-rwxr-xr-x 1 me me 224 2016-10-29 18:44 /home/me/bin/foo
< ls ... /home/me/foo.txt > ? y
-rw-r--r-- 1 me me 0 2025-09-19 12:53 /home/me/foo.txt
```

В этом примере мы ищем файлы, имена которых начинаются со строки `foo`, и каждый раз при нахождении выполняем команду `ls -l`. Использование действия `-ok` выводит приглашение пользователю перед выполнением команды `ls`.

Повышение эффективности

При использовании действия `-exec` каждый раз при нахождении подходящего файла запускается новый экземпляр указанной

команды. Иногда может быть удобнее объединить все результаты поиска и запустить команду один раз. Например, вместо выполнения команд следующим образом:

```
ls -l file1
```

```
ls -l file2
```

мы можем выполнить их так:

```
ls -l file1 file2
```

В этом случае команда выполняется один раз вместо многократного запуска. Это можно сделать двумя способами: традиционным, с помощью внешней команды `xargs`, и альтернативным, используя новую функцию в самой `find`. Сначала мы рассмотрим альтернативный способ.

Заменив завершающую точку с запятой на плюс, мы активируем способность `find` объединять результаты поиска в список аргументов для одного выполнения нужной команды. Возвращаясь к нашему примеру, это позволит выполнить `ls` каждый раз, когда будет найден подходящий файл:

```
find ~ -type f -name 'foo*' -exec ls -l '{}' ';'
-rwxr-xr-x 1 me me 224 2016-10-29 18:44 /home/me/bin/foo
-rw-r--r-- 1 me me 0 2025-09-19 12:53 /home/me/foo.txt
```

Изменив команду на следующую:

```
find ~ -type f -name 'foo*' -exec ls -l '{}' +
-rwxr-xr-x 1 me me 224 2016-10-29 18:44 /home/me/bin/foo
-rw-r--r-- 1 me me 0 2025-09-19 12:53 /home/me/foo.txt
```

результат тот же, но системе приходится выполнять команду `ls` лишь один раз.

xargs

Команда `xargs` выполняет интересную функцию. Она принимает данные из стандартного ввода и преобразует их в список аргументов для указанной команды. В нашем примере мы используем её так:

```
find ~ -type f -name 'foo*' -print | xargs ls -l
-rwxr-xr-x 1 me me 224 2016-10-29 18:44 /home/me/bin/foo
-rw-r--r-- 1 me me 0 2025-09-19 12:53 /home/me/foo.txt
```

Здесь мы видим, как вывод команды `find` передается через конвейер в `xargs`, которая, в свою очередь, формирует список аргументов для команды `ls` и выполняет её.

Примечание: хотя количество аргументов, которые можно передать в командную строку, весьма велико, оно не безгранично. Можно создать команды, слишком длинные для оболочки. Когда командная строка превышает максимально допустимую системой длину, `xargs` выполняет указанную команду с максимально возможным числом аргументов, а затем повторяет этот процесс, пока стандартный ввод не исчерпается. Чтобы узнать максимальный размер командной строки, выполните `xargs` с опцией `--show-limits`.

Работа со странными именами файлов

Unix-подобные системы допускают наличие пробелов (и даже переносов строк!) в именах файлов. Это создает проблемы для таких программ, как `xargs`, которые формируют списки аргументов для других программ. Пробел будет воспринят как разделитель, и итоговая команда интерпретирует каждое слово, разделенное пробелом, как отдельный аргумент. Чтобы обойти это, `find` и `xargs` позволяют опционально использовать нулевой символ в качестве разделителя аргументов. В ASCII нулевой символ определяется как символ, представленный числом ноль (в отличие от, например, пробела, который в ASCII представлен числом 32). Команда `find` выполняет действие `-print0`, которое выводит данные, разделенные нулевым символом, а команда `xargs` имеет опцию `--null` (или `-0`), которая принимает входные данные, разделенные нулевым символом. Вот пример:

```
find ~ -iname '*.jpg' -print0 | xargs --null ls -l
```

Используя этот метод, мы гарантируем правильную обработку всех файлов, даже тех, чьи имена содержат пробелы.

Возвращение на игровую площадку

Пришло время применить `find` с (почти) практической пользой. Мы создадим игровую площадку и опробуем то, чему научились. Сначала создадим игровую площадку с множеством подкаталогов и файлов.

```
[me@linuxbox ~]$ mkdir -p playground/dir-{001..100}
[me@linuxbox ~]$ touch playground/dir-{001..100}/file-{A..Z}
```

Восхититесь мощью командной строки! Этими двумя строками мы создали каталог игровой площадки, содержащий 100 подкаталогов, в каждом из которых по 26 пустых файлов. Попробуйте так сделать через GUI!

Для этого волшебства мы использовали знакомую команду (`mkdir`), экзотическое раскрытие оболочки (фигурные скобки) и новую команду `touch`. Сочетая `mkdir` с опцией `-p` (которая заставляет `mkdir` создавать родительские каталоги для указанных путей) и раскрытием скобок, мы смогли создать 100 подкаталогов.

Команда `touch` обычно используется для установки или обновления времени доступа, изменения и модификации файлов. Однако, если в качестве аргумента указано имя несуществующего файла, будет создан пустой файл.

В нашей песочнице мы создали 100 экземпляров файла с именем `file-A`. Давайте их найдем.

```
[me@linuxbox ~]$ find playground -type f -name 'file-A'
```

Примечание: в отличие от `ls`, `find` не выводит результаты в отсортированном порядке. Его порядок определяется структурой устройства хранения. Таким образом мы можем подтвердить, что у нас действительно есть 100 экземпляров файла.

```
[me@linuxbox ~]$ find playground -type f -name 'file-A' | wc -l
100
```

Теперь рассмотрим поиск файлов по времени их изменения. Это полезно при создании резервных копий или организации файлов в хронологическом порядке. Для этого мы сначала создадим эталонный файл, с которым будем сравнивать время изменения.

```
[me@linuxbox ~]$ touch playground/timestamp
```

Это создает пустой файл с именем `timestamp` и устанавливает время его изменения на текущее. Мы можем проверить это с помощью другой полезной команды, `stat`, которая является своего рода улучшенной версией `ls`. Команда `stat` раскрывает все, что системе известно о файле и его атрибутах.

```
[me@linuxbox ~]$ stat playground/timestamp
File: `playground/timestamp'
Size: 0          Blocks: 0          IO Block: 4096 regular empty file
Device: 803h/2051d Inode: 14265061 Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1001/ me)   Gid: ( 1001/ me)
Access: 2025-10-08 15:15:39.000000000 -0400
Modify: 2025-10-08 15:15:39.000000000 -0400
Change: 2025-10-08 15:15:39.000000000 -0400
```

Если мы снова воспользуемся `touch`, а затем изучим файл с помощью `stat`, то увидим, что время файла обновилось.

```
[me@linuxbox ~]$ touch playground/timestamp
[me@linuxbox ~]$ stat playground/timestamp
  File: `playground/timestamp'
  Size: 0          Blocks: 0          IO Block: 4096 regular empty file
Device: 803h/2051d Inode: 14265061 Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1001/ me)   Gid: ( 1001/ me)
Access: 2025-10-08 15:23:33.000000000 -0400
Modify: 2025-10-08 15:23:33.000000000 -0400
Change: 2025-10-08 15:23:33.000000000 -0400
```

Далее давайте воспользуемся `find`, чтобы обновить некоторые файлы в нашей песочнице.

```
[me@linuxbox ~]$ find playground -type f -name 'file-B' -exec touch
'{}' ';' ;
```

Это обновит все файлы в песочнице с именем `file-B`. Затем мы воспользуемся `find`, чтобы найти обновленные файлы, сравнив их все с эталонным файлом `timestamp`.

```
[me@linuxbox ~]$ find playground -type f -newer playground/timestamp
```

Результаты содержат все 100 экземпляров `file-B`. Поскольку мы выполнили `touch` над всеми файлами в `playground` с именем `file-B` после обновления `timestamp`, теперь они «новее», чем `timestamp`, и их можно определить с помощью теста `-newer`.

Наконец, вернемся к тесту на некорректные права доступа, который мы выполняли ранее, и применим его к `playground`.

```
[me@linuxbox ~]$ find playground \( -type f -not -perm 0600 \) -or \(
-type d -not -perm 0700 \)
```

Эта команда выводит список всех 100 каталогов и 2600 файлов в `playground` (а также `timestamp` и сам `playground`, итого 2702), так как ни один из них не соответствует нашему определению «хороших прав доступа». Зная операторы и действия, мы можем добавить в эту команду новые действия, чтобы применить новые права к файлам и каталогам в нашем `playground`.

```
[me@linuxbox ~]$ find playground \( -type f -not -perm 0600 -exec
chmod 0600 '{}' ';' \) -or \( -type d -not -perm 0700 -exec chmod
0700 '{}' ';' \)
```

В повседневной работе нам может быть проще ввести две команды — одну для каталогов, другую для файлов, — чем одну такую длинную составную команду, но полезно знать, что так тоже можно. Главное здесь — понять, как операторы и действия можно использовать вместе для выполнения полезных задач.

Параметры

Наконец, перейдем к опциям. Опции используются для управления областью `find` поиска. Их можно комбинировать с другими тестами и действиями при составлении `find` выражений. В таблице 17-7 перечислены наиболее часто используемые `find` опции.

Таблица 17-7: `find` Параметры

Параметр	Описание
<code>-depth</code>	Направляет <code>find</code> для обработки файлов каталога перед самим каталогом. Этот параметр применяется автоматически, если указано действие <code>-delete</code> .
<code>-maxdepth levels</code>	Устанавливает максимальную глубину, на которую <code>find</code> будет заходить в дерево каталогов при выполнении тестов и действия.
<code>-mindepth levels</code>	Установите минимальное количество уровней, на которые <code>find</code> будет спускаться в дерево каталогов перед применением тестов и действия.
<code>-mount</code>	Укажите <code>find</code> не переходить в каталоги, смонтированные на других файловых системах.
<code>-no leaf</code>	Укажите <code>find</code> не оптимизировать поиск, исходя из предположения, что сканируется Unix-подобная файловая система. Это необходимо при сканировании файловых систем DOS/Windows и CD-ROM.

Итоги

Легко заметить, что `locate` так же прост, как `find` сложен. У обоих есть свои преимущества. Уделите время изучению множества функций `find`. При регулярном использовании это поможет лучше понять работу с файловой системой Linux.

Что почитать

- Программы `locate`, `updatedb`, `find` и `xargs` входят в пакет `findutils` проекта GNU. Проект GNU предоставляет сайт с обширной онлайн-документацией; она весьма качественная и её стоит изучить, если вы используете эти программы в средах с повышенными требованиями к безопасности. Примечание: большинство современных дистрибутивов Linux удаляют `locate` и `updatedb` из пакета `findutils`, заменяя их более новыми и производительными версиями, такими как `plocate`, которые обычно поставляются в отдельном пакете: <https://www.gnu.org/software/findutils/>

18 — Архивация и резервное копирование

Одна из основных задач системного администратора — обеспечение безопасности системных данных. Один из способов сделать это — своевременное резервное копирование системных файлов. Даже если вы не системный администратор, часто бывает полезно создавать копии данных и перемещать большие коллекции файлов с места на место и с устройства на устройство.

В этой главе мы рассмотрим несколько распространенных программ для управления наборами файлов. Это программы для сжатия файлов:

- `gzip` — сжатие или распаковка файлов
- `bzip2` — блочный сортирующий архиватор

А это программы для архивации:

- `tar` — утилита для архивации лент
- `zip` — упаковка и сжатие файлов

Это программа для синхронизации файлов:

- `rsync` — удаленная синхронизация файлов и каталогов

Сжатие файлов

На протяжении всей истории вычислительной техники шла борьба за размещение максимального объема данных в минимально доступном пространстве — будь то память, устройства хранения или пропускная способность сети. Многие современные сервисы, которые мы воспринимаем как должное, такие как мобильная связь, телевидение высокой четкости или широкополосный интернет, существуют благодаря эффективным методам сжатия данных.

Сжатие данных — это процесс удаления избыточности.

Рассмотрим воображаемый пример. Допустим, у нас есть полностью чёрный файл изображения размером 100 на 100 пикселей. С точки зрения хранения данных (при условии 24 бит или 3 байт на пиксель) изображение займет 30 000 байт.

$$100 * 100 * 3 = 30,000$$

Одноцветное изображение содержит полностью избыточные данные. Будь мы умнее, мы могли бы закодировать данные так, чтобы просто описать наличие блока из 10 000 черных пикселей. Таким образом, вместо хранения блока данных из 30 000 нулей

(черный цвет в файлах изображений обычно обозначается нулем), мы могли бы сжать данные до числа 10 000, за которым следует ноль, представляющий наши данные. Такая схема сжатия называется кодированием длин серий и является одной из самых элементарных техник сжатия. Современные методы гораздо сложнее и совершеннее, но основная цель остается прежней — избавиться от избыточных данных.

Алгоритмы сжатия (математические методы, используемые для сжатия) делятся на две общие категории.

- Без потерь: Сжатие без потерь сохраняет все данные оригинала. Это означает, что при восстановлении файла из сжатой версии полученный файл будет идентичен исходному, несжатому варианту.
- С потерями: Сжатие с потерями, напротив, удаляет часть данных в процессе сжатия, чтобы добиться большей степени уменьшения размера. При восстановлении файла он не будет соответствовать оригиналу, а станет лишь его близким приближением. Примеры сжатия с потерями — JPEG (для изображений) и MP3 (для музыки).

В нашем обсуждении мы сосредоточимся исключительно на сжатии без потерь, так как большинство компьютерных данных не допускают потери информации.

gzip

Программа `gzip` используется для сжатия одного или нескольких файлов. При выполнении она заменяет исходный файл его сжатой версией. Соответствующая программа `gunzip` используется для восстановления сжатых файлов в их исходный, несжатый вид. Вот пример:

```
[me@linuxbox ~]$ ls -l /etc > foo.txt
[me@linuxbox ~]$ ls -l foo.*
-rw-r--r-- 1 me me 15738 2025-10-14 07:15 foo.txt
[me@linuxbox ~]$ gzip foo.txt
[me@linuxbox ~]$ ls -l foo.*
-rw-r--r-- 1 me me 3230 2025-10-14 07:15 foo.txt.gz
[me@linuxbox ~]$ gunzip foo.txt
[me@linuxbox ~]$ ls -l foo.*
-rw-r--r-- 1 me me 15738 2025-10-14 07:15 foo.txt
```

В этом примере мы создаем текстовый файл `foo.txt` из списка содержимого каталога. Затем мы запускаем `gzip foo.txt.gz foo.*`, которая заменяет исходный файл сжатой версией с именем. В списке содержимого каталога `foo.txt` мы видим, что исходный файл заменен сжатой версией, размер которой составляет примерно одну пятую от оригинала. Также видно, что

сжатый файл имеет те же права доступа и временную- метку, что и оригинал.

Затем мы запускаем программу `gunzip`, чтобы распаковать файл. После этого можно увидеть, что сжатая версия файла была заменена оригиналом, при этом права доступа и метка времени сохранились.

`gzip` имеет множество опций, описанных в таблице 18-1.

Таблица 18-1: Опции `gzip`

Опция	Длинная опция	Описание
-c	--stdout --to-stdout	Записывать вывод в стандартный вывод и сохранять исходные файлы.
-d	--decompress --uncompress	Распаковать. В этом случае <code>gzip</code> работает как <code>gunzip</code> .
-f	--force	Принудительное сжатие, даже если файл уже сжат версия исходного файла уже существует.
-h	--help	Вывести справку по использованию.
-l	--list	Вывести статистику сжатия для каждого файла сжато.
-r	--recursive	Если один или несколько аргументов в командной строке являются каталогом, рекурсивно сжимать файлы содержащихся в них.
-t	--test	Проверить целостность сжатого файла.
-v	--verbose	Выводить подробные сообщения при сжатии.
-number		Установить степень сжатия. Число — это целое число в диапазоне от 1 (быстрее, меньше сжатие) до 9 (медленнее, сжатия). Значения 1 и 9 также могут быть представлены как -fast и --best, соответственно. Значение по умолчанию — 6.

Вернемся к нашему предыдущему примеру.

```
[me@linuxbox ~]$ gzip foo.txt
[me@linuxbox ~]$ gzip -tv foo.txt.gz
foo.txt.gz: OK
[me@linuxbox ~]$ gzip -d foo.txt.gz
```

Здесь мы заменили файл `foo.txt` сжатой версией с именем `foo.txt.gz`. Затем мы проверили целостность сжатой версии, используя опции `-t` и `-v`. Наконец, мы распаковали файл до его исходного состояния.

`gzip` также можно использовать интересными способами через стандартный ввод и вывод.

```
[me@linuxbox ~]$ ls -l /etc | gzip > foo.txt.gz
```

Эта команда создает сжатую версию списка содержимого каталога.

Программа `gunzip`, которая распаковывает `gzip`-файлы, предполагает, что имена файлов заканчиваются расширением `.gz`, поэтому его не обязательно указывать, если заданное имя не конфликтует с существующим несжатым файлом.

```
[me@linuxbox ~]$ gunzip foo.txt
```

Если нашей целью было только просмотреть содержимое сжатого текстового файла, мы могли бы сделать следующее:

```
[me@linuxbox ~]$ gunzip -c foo.txt | less
```

В качестве альтернативы существует программа `zcat`, поставляемая с `gzip`, которая эквивалентна `gunzip` с опцией `-c`. Её можно использовать подобно команде `cat` для сжатых файлов `gzip`.

```
[me@linuxbox ~]$ zcat foo.txt.gz | less
```

Совет: Существует также программа `zless`. Она выполняет ту же функцию, что и предыдущий конвейер.

bzip2

Программа `bzip2` от Джулиана Сьюарда похожа на `gzip`, но использует другой алгоритм, который обеспечивает более высокую степень сжатия ценой скорости. Почти во всем она работает так же, как `gzip`. Файл, сжатый с помощью `bzip2` обозначается расширением `.bz2`.

```
[me@linuxbox ~]$ ls -l /etc > foo.txt
[me@linuxbox ~]$ ls -l foo.txt
-rw-r--r-- 1 me me 15738 2025-10-17 13:51 foo.txt
[me@linuxbox ~]$ bzip2 foo.txt
[me@linuxbox ~]$ ls -l foo.txt.bz2
-rw-r--r-- 1 me me 2792 2025-10-17 13:51 foo.txt.bz2
[me@linuxbox ~]$ bunzip2 foo.txt.bz2
```

Как видно, bzip2 можно использовать так же, как и gzip. Все опции (кроме `-r` gzip bzip2), которые мы обсуждали для них, поддерживаются и в. Однако учтите, что опция уровня сжатия (`-number`) имеет несколько иное значение, чем у bzip2. bzip2 поставляется вместе с bunzip2 и bzsca для распаковки файлов. bzip2 также включает программу bzip2recover, которая пытается восстановить поврежденные файлы .bz2.

Не будьте одержимы сжатием

Я иногда вижу, как люди пытаются сжать файл, который уже был сжат эффективным алгоритмом, делая нечто подобное:

```
$ gzip picture.jpg
```

Не делайте этого. Скорее всего, вы просто тратите время и место! Если применить сжатие к уже сжатому файлу, в итоге он обычно становится больше. Это происходит потому, что все методы сжатия добавляют к файлу служебную информацию для описания процесса. Если вы попытаетесь сжать файл, в котором уже нет избыточной информации, сжатие чаще всего не даст экономии, способной компенсировать эти дополнительные накладные расходы.

Архивация файлов

Частой задачей управления файлами, часто используемой вместе со сжатием, является архивирование. Архивирование — это процесс сбора множества файлов и объединения их в один большой файл. Архивирование часто выполняется как часть резервного копирования системы. Оно также применяется при переносе старых данных из системы в какое-либо долгосрочное хранилище.

tar

В мире Unix-подобного ПО программа tar — классический инструмент для архивирования файлов. Её название, сокращение от tape archive, указывает на корни как инструмента для создания резервных лент. Хотя её всё ещё используют для

этой традиционной задачи, она не менее эффективна и на других устройствах хранения. Мы часто видим имена файлов, заканчивающиеся на расширение `.tar` или `.tgz`, что указывает на «обычный» tar-архив и gzipped-архив соответственно. tar-архив может состоять из группы отдельных файлов, одной или нескольких иерархий каталогов или их смеси. Синтаксис команды выглядит так:

```
tar mode[options] pathname...
```

Здесь `mode` — это один из режимов работы, перечисленных в таблице 18-2 (здесь приведен лишь неполный список; полный список см. в `tar man page`).

Таблица 18-2: *tar* Режимы

Mode	Описание
c	Создать архив из списка файлов и/или каталогов.
x	Извлечь архив.
r	Добавляет указанные пути в конец архива. Если архив не существует, он будет создан..
t	Вывод содержимого архива.

`tar` использует несколько необычный способ задания опций, поэтому нам понадобятся примеры, чтобы понять, как это работает. Сначала восстановим нашу рабочую среду из предыдущей главы.

```
[me@linuxbox ~]$ mkdir -p playground/dir-{001..100}
[me@linuxbox ~]$ touch playground/dir-{001..100}/file-{A..Z}
```

Затем создадим tar-архив всей рабочей среды.

```
[me@linuxbox ~]$ tar cf playground.tar playground
```

Эта команда создает tar-архив с именем `playground.tar`, содержащий всю иерархию каталогов `playground`. Мы видим, что режим и опцию `f`, используемую для указания имени tar архива, можно объединять — в этом случае ведущий дефис не требуется. Примечание: режим всегда должен указываться первым, перед любыми опциями.

Чтобы вывести содержимое архива, выполните:

```
[me@linuxbox ~]$ tar tf playground.tar
```

Для более подробного вывода можно добавить опцию `v` (`verbose`).

```
[me@linuxbox ~]$ tar tvf playground.tar
```

Теперь извлечем playground в новое место. Для этого создадим новый каталог с именем foo, перейдем в него и распакуем tar-архив.

```
[me@linuxbox ~]$ mkdir foo
[me@linuxbox ~]$ cd foo
[me@linuxbox foo]$ tar xf ../playground.tar
[me@linuxbox foo]$ ls
playground
```

Если изучить содержимое ~/foo/playground, мы увидим, что архив был успешно распакован, создав точную копию исходных файлов. Однако есть один нюанс. Если мы не работаем как суперпользователь, файлы и каталоги, извлеченные из архивов, получают права владельца пользователя, выполняющего восстановление, а не исходного владельца.

Еще одна интересная особенность tar — это способ обработки путей в архивах. По умолчанию пути являются относительными, а не абсолютными. tar делает это, просто удаляя начальный слеш из пути при создании архива. Чтобы продемонстрировать это, мы заново создадим архив, на этот раз указав абсолютный путь.

```
[me@linuxbox foo]$ cd
[me@linuxbox ~]$ tar cf playground2.tar ~/playground
```

Помните, что ~/playground развернется в /home/me/playground, когда мы нажмем ENTER, так что для демонстрации мы получим абсолютный путь. Затем мы распакуем архив, как и прежде, и посмотрим, что получится.

```
[me@linuxbox ~]$ cd foo
[me@linuxbox foo]$ tar xf ../playground2.tar
[me@linuxbox foo]$ ls
home  playground
[me@linuxbox foo]$ ls home
me
[me@linuxbox foo]$ ls home/me
playground
```

Здесь видно, что при извлечении второго архива каталог home/me/playground был воссоздан относительно нашего текущего рабочего каталога ~/foo, а не относительно корневого каталога, как это было бы при использовании абсолютного пути. Это может показаться странным, но на деле такой подход полезнее: он позволяет извлекать архивы в любое место, а не только в их исходные пути. Если повторить упражнение с опцией подробного вывода (v), станет понятнее, что именно происходит.

Рассмотрим гипотетический, но практичный пример работы tar. Представьте, что мы хотим скопировать домашний каталог и его содержимое с одной системы на другую, используя для передачи большой USB-жесткий диск. В современной Linux-системе диск «автоматически» монтируется в каталог /media. Предположим также, что при подключении том диска имеет имя BigDisk. Чтобы создать tar-архив, выполните следующее:

```
[me@linuxbox ~]$ sudo tar cf /media/BigDisk/home.tar /home
```

После записи tar-файла мы размонтируем диск и подключаем его ко второму компьютеру. Он снова смонтирован в /media/BigDisk. Чтобы извлечь архив, выполните следующее:

```
[me@linuxbox2 ~]$ cd /  
[me@linuxbox2 /]$ sudo tar xf /media/BigDisk/home.tar
```

Важно заметить, что сначала нужно сменить каталог на /, чтобы извлечение было относительно корневого каталога, так как все пути внутри архива являются относительными.

При извлечении архива можно ограничить состав извлекаемых данных. Например, если нужно извлечь один файл из архива, это можно сделать так:

```
tar xf archive.tar pathname
```

Добавление в конец команды пути tar позволит восстановить только указанный файл. Можно указать несколько путей.

Примечание: путь должен быть полным и точным относительным путем, как он сохранен в архиве. При указании путей подстановочные знаки обычно не поддерживаются; однако GNU-версия tar (которая чаще всего встречается в дистрибутивах Linux) поддерживает их с помощью опции --wildcards. Вот пример использования нашего предыдущего файла playground.tar:

```
[me@linuxbox ~]$ cd foo  
[me@linuxbox foo]$ tar xf ../playground2.tar --wildcards 'home/me/playground/dir-*/file-A'
```

Эта команда извлечет только те файлы, которые соответствуют указанному пути, включая подстановочный знак dir-*.

tar часто используют вместе с find для создания архивов. В этом примере мы воспользуемся find, чтобы подготовить набор файлов для включения в архив.

```
[me@linuxbox ~]$ find playground -name 'file-A' -exec tar rf playground.tar '{}' '+'
```

Здесь мы используем `find`, чтобы найти все файлы в `playground` с именем `file-A`, а затем, используя действие `-exec`, вызываем `tar` в режиме добавления (`r`), чтобы добавить найденные файлы в архив `playground.tar`.

Использование `tar` с `find` — хороший способ создания инкрементальных резервных копий дерева каталогов или всей системы. Используя `find` для поиска файлов новее, чем файл с меткой времени, можно создать архив, содержащий только те файлы, которые были изменены после создания последнего архива (при условии, что файл метки времени обновляется сразу после создания каждого архива).

`tar` также может использовать стандартный ввод и вывод. Вот подробный пример:

```
[me@linuxbox foo]$ cd
[me@linuxbox ~]$ find playground -name 'file-A' | tar cf - --files-
from=- | gzip > playground.tgz
```

В этом примере мы использовали программу `find`, чтобы получить список подходящих файлов, и передали его через конвейер в `tar`. Если указано имя файла `-`, оно означает стандартный ввод или вывод, в зависимости от ситуации. (Кстати, эта конвенция использования `-` для представления стандартного ввода/вывода применяется и в ряде других программ). Опция `--files-from` (которую также можно указать как `-T`) заставляет `tar` читать список путей из файла, а не из командной строки. Наконец, архив, созданный `tar`, передается через конвейер в `gzip` для создания сжатого архива `playground.tgz`. Расширение `.tgz` — это общепринятое расширение для `tar`-файлов, сжатых с помощью `gzip`. Расширение `.tar.gz` также используется иногда — в некоторых случаях.

Хотя в предыдущем примере мы использовали программу `gzip` извне для создания сжатого архива, современные версии GNU `tar` поддерживают как `gzip`, так и `bzip2` сжатие напрямую с помощью опций `z` и `j` соответственно. Основываясь на нашем предыдущем примере, мы можем упростить его следующим образом:

```
[me@linuxbox ~]$ find playground -name 'file-A' | tar czf
playground.tgz -T -
```

Если бы мы хотели создать архив со сжатием `bzip2`, мы могли бы сделать так:

```
[me@linuxbox ~]$ find playground -name 'file-A' | tar cjf  
playground.tbz -T -
```

Просто изменив опцию сжатия с `z` на `j` (и изменив расширение выходного файла на `.tbz`, чтобы указать на tar-файл со сжатием `bzip2`), мы включили сжатие `bzip2`.

Еще одно интересное применение стандартного ввода и вывода с командой `tar` — передача файлов между системами по сети.

Представьте, что у нас есть две машины под управлением Unix-подобной системы, оснащенные `tar` и `ssh`. В таком сценарии мы могли бы передать каталог с удаленной системы (в данном примере названной `remote-sys`) в нашу локальную систему.

```
[me@linuxbox ~]$ mkdir remote-stuff  
[me@linuxbox ~]$ cd remote-stuff  
[me@linuxbox remote-stuff]$ ssh remote-sys 'tar cf - Documents' | tar  
xf -  
me@remote-sys's password:  
[me@linuxbox remote-stuff]$ ls  
Documents
```

Здесь мы скопировали каталог с именем `Documents` с удаленной системы `remote-sys` в каталог внутри каталога с таким же именем на локальной системе. Как мы это сделали? Сначала мы запустили программу `tar` на удаленной системе с помощью `ssh`. Помните, что `ssh` позволяет удаленно выполнять программу на сетевом компьютере и «видеть» результаты на локальной системе — стандартный вывод, созданный на удаленной системе, отправляется на локальную систему для просмотра. Мы можем воспользоваться этим, если заставим `tar` создать архив (режим `c`) и отправить его в стандартный вывод, а не в файл (опция `f` с аргументом «дефис»), тем самым передавая архив через зашифрованный туннель, предоставляемый `ssh`, на локальную систему. На локальной системе мы запускаем `tar`, чтобы распаковать архив (режим `x`), полученный из стандартного ввода (снова опция `f` с аргументом «дефис»).

zip

Программа `zip` является одновременно инструментом сжатия и архиватором. Формат файлов, используемый программой, знаком пользователям Windows, так как она читает и записывает `.zip` файлы. Однако в Linux основной программой сжатия является `gzip`, а `bzip2` идет следом.

В самом простом варианте `zip` вызывается так:

```
zip options zipfile file...
```

Например, чтобы создать zip-архив нашей песочницы, нужно сделать следующее:

```
[me@linuxbox ~]$ zip -r playground.zip playground
```

Если не указать опцию `-r` для рекурсии, будет сохранен только `playground` каталог (но не его содержимое). Хотя расширение `.zip` добавляется автоматически, для ясности мы укажем его в имени файла.

При создании zip-архива `zip` обычно выводит ряд сообщений, подобных этим:

```
adding: playground/dir-020/file-Z (stored 0%)
adding: playground/dir-020/file-Y (stored 0%)
adding: playground/dir-020/file-X (stored 0%)
adding: playground/dir-087/ (stored 0%)
adding: playground/dir-087/file-S (stored 0%)
```

Эти сообщения отображают статус каждого файла, добавленного в архив. `zip` добавляет файлы в архив одним из двух способов: либо он «сохраняет» файл без сжатия, как показано здесь, либо «сжимает» (deflate) его. Числовое значение после метода хранения указывает на степень сжатия. Поскольку наш каталог содержит только пустые файлы, сжатие их содержимого не производится.

Извлечение содержимого zip-архива с помощью программы `unzip` выполняется очень просто.

```
[me@linuxbox ~]$ cd foo
[me@linuxbox foo]$ unzip ../playground.zip
```

Стоит отметить, что в случае с `zip` (в отличие от `tar`), если указан существующий архив, он обновляется, а не заменяется. Это означает, что существующий архив сохраняется, но в него добавляются новые файлы, а соответствующие им — заменяются.

Из zip-архива можно выборочно просматривать и извлекать файлы, указывая их в `unzip`.

```
[me@linuxbox ~]$ unzip -l playground.zip playground/dir-087/file-Z
Archive:  ../playground.zip
  Length  Date   Time    Name
  -----  ---
           0  10-05-25  09:25  playground/dir-087/file-Z
  -----
           0                          1 file

[me@linuxbox ~]$ cd foo
[me@linuxbox foo]$ unzip ../playground.zip playground/dir-087/file-Z
Archive:  ../playground.zip
replace playground/dir-087/file-Z? [y]es, [n]o, [A]ll, [N]one,
[r]ename: y
extracting: playground/dir-087/file-Z
```

Опция `-l` заставляет `unzip` просто вывести содержимое архива без извлечения файлов. Если файлы не указаны, `unzip` выведет список всех файлов в архиве. Для более подробного вывода можно добавить опцию `-v`. Примечание: если при извлечении возникает конфликт с существующим файлом, перед его заменой появится приглашение.

По умолчанию `unzip` извлекает файлы в текущий рабочий каталог. Чтобы указать другой каталог для извлечения, используйте опцию `-d`, а затем имя нужного каталога.

Как и `tar`, `zip` может использовать стандартный ввод и вывод, хотя его реализация менее полезна. С помощью опции `-@` можно передать список имен файлов в `zip` через конвейер.

```
[me@linuxbox foo]$ cd  
[me@linuxbox ~]$ find playground -name "file-A" | zip -@ file-A.zip
```

Здесь мы используем `find`, чтобы создать список файлов, соответствующих тесту `-name "file-A"`, а затем передаем этот список через конвейер в `zip`, которая создает архив `file-A.zip` с выбранными файлами.

`zip` также поддерживает вывод в стандартный вывод, но это ограничено, так как немногие программы могут его использовать. К сожалению, программа `unzip` не принимает стандартный ввод. Это не позволяет использовать `zip` и `unzip` вместе для сетевого копирования файлов, как в `tar`.

`zip`, однако, может принимать стандартный ввод, поэтому его можно использовать для сжатия вывода других программ.

```
[me@linuxbox ~]$ ls -l /etc/ | zip ls-etc.zip -  
adding: - (deflated 80%)
```

В этом примере мы направляем вывод `ls` через конвейер в `zip`. Как и `tar`, `zip` интерпретирует завершающий дефис как «использовать стандартный ввод в качестве списка файлов». Имейте в виду, что `zip` сохранит сжатый файл (файл внутри `zip`-архива) под именем «-», что является ужасным именем; многие программы командной строки будут работать с ним некорректно.

Чтобы обойти эту проблему, используйте программу `unzip`, так как она позволяет направлять вывод в стандартный вывод при указании опции `-p` (для конвейера).

```
[me@linuxbox ~]$ unzip -p ls-etc.zip | less
```

Мы затронули некоторые базовые задачи, которые могут выполнять `zip/unzip`. Оба инструмента обладают множеством опций, повышающих их гибкость, хотя некоторые из них специфичны для других систем. Руководства `man` для `zip` и `unzip` довольно хороши и содержат полезные примеры. Однако основное назначение этих программ — обмен файлами с системами Windows, а не сжатие и архивация в Linux, где гораздо предпочтительнее использовать `tar` и `gzip`.

Синхронизация файлов и каталогов

Обычная стратегия создания резервных копий системы заключается в синхронизации одного или нескольких каталогов с другим каталогом (или каталогами), расположенным на локальной системе (обычно на съемном носителе) или на удаленной системе. Например, у нас может быть локальная копия разрабатываемого сайта, которую мы время от времени синхронизируем с «живой» копией на удаленном веб-сервере.

В Unix-подобных системах предпочтительным инструментом для этой задачи является `rsync`. Эта программа может синхронизировать локальные и удаленные каталоги с помощью протокола `rsync remote-update`, который позволяет `rsync` быстро находить различия между двумя каталогами и выполнять минимальное копирование, необходимое для их синхронизации. Благодаря этому `rsync` работает очень быстро и эффективно по сравнению с другими программами копирования.

`rsync` вызывается следующим образом:

```
rsync options source destination
```

где источником и назначением является один из следующих вариантов:

- Локальный файл или каталог
- Удаленный файл или каталог в формате `[user@]host:path`
- Удаленный `rsync`-сервер, указанный через URI
`rsync://[user@]host[:port]/path`

Примечание: либо источник, либо назначение должен быть локальным файлом. Копирование с удаленного узла на удаленный не поддерживается.

Давайте опробуем `rsync` на локальных файлах. Сначала очистим наш `foo` каталог.

```
[me@linuxbox ~]$ rm -rf foo/*
```

Затем мы синхронизируем playground каталог с соответствующей копией в foo.

```
[me@linuxbox ~]$ rsync -av playground foo
```

Чтобы создать зеркальную копию playground каталога внутри foo, мы использовали опцию -a (для архивации — включает рекурсию и сохранение атрибутов файлов) и опцию -v (подробный вывод). Пока команда выполняется, мы будем видеть список копируемых файлов и каталогов. В конце появится итоговое сообщение, указывающее объем скопированных данных:

```
sent 135759 bytes  received 57870 bytes  387258.00 bytes/sec
total size is 3230  speedup is 0.02
```

Если запустить команду снова, результат будет другим.

```
[me@linuxbox ~]$ rsync -av playground foo
building file list ... done
sent 22635 bytes  received 20 bytes  45310.00 bytes/sec
total size is 3230  speedup is 0.14
```

Обратите внимание, список файлов отсутствовал. Это произошло потому, что rsync обнаружил отсутствие различий между ~/playground и ~/foo/playground, и поэтому копировать ничего не потребовалось. Если мы изменим файл в playground и снова запустим rsync:

```
[me@linuxbox ~]$ touch playground/dir-099/file-Z
[me@linuxbox ~]$ rsync -av playground foo
building file list ... done
playground/dir-099/file-Z
sent 22685 bytes  received 42 bytes  45454.00 bytes/sec
total size is 3230  speedup is 0.14
```

мы видим, что rsync обнаружил изменение и скопировал только обновленный файл.

При указании rsync источника доступна одна тонкая, но полезная функция. Рассмотрим эти два каталога.

```
[me@linuxbox ~]$ ls
source      destination
```

Каталог source содержит один файл с именем file1, а каталог destination пуст. Если мы выполним копирование source в destination следующим образом:

```
[me@linuxbox ~]$ rsync source destination
```

тогда rsync скопирует каталог source в destination.

```
[me@linuxbox ~]$ ls destination
source
```

Однако, если добавить в конец имени исходного каталога /, `rsync` скопирует только содержимое исходного каталога, а не сам каталог.

```
[me@linuxbox ~]$ rsync source/ destination
[me@linuxbox ~]$ ls destination
file1
```

Это удобно, если мы хотим скопировать только содержимое каталога, не создавая лишний уровень вложенности в месте назначения. Это можно представить как `source/*` in его результате, но этот метод скопирует всё содержимое исходного каталога, включая скрытые файлы.

В качестве практического примера рассмотрим воображаемый внешний жесткий диск, который мы использовали ранее с `tar/media/BigDisk`. Если подключить диск к системе и снова его смонтировать, можно выполнить полезное резервное копирование системы: сначала создать на внешнем диске каталог с именем `/backup`, а затем использовать `rsync`, чтобы скопировать самые важные данные из нашей системы на внешний диск.

```
[me@linuxbox ~]$ mkdir /media/BigDisk/backup
[me@linuxbox ~]$ sudo rsync -av --delete /etc /home /usr/local
/media/BigDisk/backup
```

В этом примере мы скопировали каталоги `/etc`, `/home` и `/usr/local` из нашей системы на воображаемое устройство хранения. Мы использовали опцию `--delete`, чтобы удалить файлы, которые могли быть на устройстве резервного копирования, но отсутствуют в источнике (при первом копировании это не имеет значения, но будет полезно при последующих). Повторение процедуры подключения внешнего диска и запуск этой команды `rsync` — полезный (хотя и не идеальный) способ поддерживать резервную копию небольшой системы. Конечно, здесь пригодился бы псевдоним. Мы могли бы создать псевдоним и добавить его в файл `.bashrc`, чтобы реализовать этот функционал.

```
alias backup='sudo rsync -av --delete /etc /home /usr/local
/media/BigDisk/backup'
```

Теперь нам осталось только подключить внешний диск и запустить команду `backup`, чтобы выполнить работу.

Использование `r sync` через сеть

Одно из главных преимуществ `r sync` заключается в том, что его можно использовать для копирования файлов по сети. В конце концов, `r` в `r sync` означает «удаленный». Удаленное копирование можно выполнить одним из двух способов. Первый способ — использование другой системы с установленным `r sync` и программой удаленной оболочки, такой как `ssh`. Предположим, в нашей локальной сети есть другая система с большим объемом свободного места на жестком диске, и мы хотим выполнить резервное копирование с ее помощью вместо внешнего диска. Если в ней уже есть каталог с именем `/backup`, куда можно передать файлы, мы можем сделать следующее:

```
[me@linuxbox ~]$ sudo r sync -av --delete --rsh=ssh /etc /home  
/usr/local remote-sys:/backup
```

Мы внесли два изменения в команду, чтобы упростить копирование по сети. Во-первых, мы добавили опцию `-rsh=ssh`, которая указывает `r sync` использовать программу `ssh` в качестве удаленной оболочки. Таким образом мы смогли использовать зашифрованный туннель `ssh` для безопасной передачи данных с локальной системы на удаленный хост. В современных системах `ssh` используется по умолчанию, поэтому эта опция требуется редко. Во-вторых, мы указали удаленный хост, добавив его имя (в данном случае имя хоста — `remote-sys`) перед путем назначения.

Второй способ, которым `r sync` можно синхронизировать файлы по сети, — использование `r sync`-сервера. `r sync` можно настроить на работу в режиме демона, чтобы он прослушивал входящие запросы на синхронизацию. Это часто применяется для зеркалирования удаленной системы. Например, Red Hat Software поддерживает обширный репозиторий разрабатываемых пакетов для дистрибутива Fedora. Тестировщикам ПО полезно зеркалировать эту коллекцию на этапе тестирования цикла выпуска дистрибутива. Поскольку файлы в репозитории меняются часто (зачастую чаще раза в день), целесообразнее поддерживать локальное зеркало путем периодической синхронизации, а не массового копирования всего репозитория. Один из таких репозиториях находится в Университете Дьюка; мы могли бы зеркалировать его, используя нашу локальную копию `r sync` и их `r sync` сервер следующим образом:

```
[me@linuxbox ~]$ mkdir fedora-devel  
[me@linuxbox ~]$ r sync -av --delete rsync://archive.linux.duke.edu/  
fedora/linux/development/rawhide/Everything/x86_64/os/ fedora-devel
```

В этом примере мы используем URI удаленного `rsync` сервера, который состоит из протокола (`rsync://`), имени удаленного хоста (`archive.linux.duke.edu`) и пути к репозиторию.

Итоги

Мы рассмотрели распространенные программы сжатия и архивации, используемые в Linux и других Unix-подобных операционных системах. Для архивации файлов в Unix-подобных системах предпочтительным методом является сочетание `tar/gzip`, тогда как `zip/unzip` используется для совместимости с системами Windows. Наконец, мы изучили программу `rsync` (моя любимая), которая очень удобна для эффективной синхронизации файлов и каталогов между системами.

Что почитать

- Руководства `man` для всех рассмотренных здесь команд достаточно понятны и содержат полезные примеры. Кроме того, у проекта GNU есть хорошее онлайн-руководство для его версию `tar`. Её можно найти здесь:

<https://www.gnu.org/software/tar/manual/index.html>

19 — Регулярные выражения

В следующих главах мы рассмотрим инструменты для манипуляции текстом. Как мы уже видели, текстовые данные играют важную роль во всех Unix-подобных системах, таких как Linux. Но прежде чем мы сможем в полной мере оценить возможности этих инструментов, необходимо изучить технологию, которая часто сопровождает их наиболее сложное использование, — регулярные выражения.

Изучая возможности командной строки, мы уже сталкивались с по-настоящему загадочными функциями и командами оболочки, такими как раскрытие подстановочных знаков и кавычки, сочетания клавиш и история команд, не говоря уже о редакторе vi. Регулярные выражения продолжают эту «традицию» и могут быть (спорно) самой сложной из всех функций. Это не значит, что время, затраченное на их изучение, не окупится. Напротив. Глубокое понимание позволит нам совершать удивительные вещи, хотя их истинная ценность может проявиться не сразу.

Что такое регулярные выражения?

Проще говоря, регулярные выражения — это символьные обозначения для поиска шаблонов в тексте. В некотором смысле они напоминают метод подстановочных знаков оболочки для сопоставления имен файлов и путей, но в гораздо большем масштабе. Регулярные выражения поддерживаются многими инструментами командной строки и большинством языков программирования для решения задач по манипуляции текстом. Однако, чтобы еще больше всё усложнить, не все регулярные выражения одинаковы: они немного различаются в зависимости от инструмента или языка программирования. В нашем обсуждении мы ограничимся регулярными выражениями стандарта POSIX (которые охватывают большинство инструментов командной строки), в отличие от многих языков программирования (прежде всего Perl), использующих более обширные и богатые наборы обозначений.

grep

Основная программа для работы с регулярными выражениями — наш старый знакомый grep. Название «grep» происходит от фразы «global regular expression print», так что понятно, что grep связан с регулярными выражениями. По сути, grep ищет в текстовых файлах совпадения с заданным регулярным

выражением и выводит все строки с совпадениями в стандартный вывод.

До сих пор мы использовали `grep` с фиксированными строками, вот так:

```
[me@linuxbox ~]$ ls /usr/bin | grep zip
```

Это выведет список всех файлов в `/usr/bin` каталоге, чьи имена содержат подстроку `zip`.

Программа `grep` принимает опции и аргументы следующим образом, где `regex` — это регулярное выражение:

```
grep [options] regex [file...]
```

В таблице 19-1 описаны часто используемые `grep` options.

Таблица 19-1: `grep` Опции лючена) рокой

Опция	Длинный параметр	Описание
-i	--ignore-case	Игнорировать регистр. Не различать заглавные и строчные символы.
-v	--invert-match	Инвертировать совпадение. Обычно <code>grep</code> выводит строки, содержащие совпадение. Эта опция заставляет <code>grep</code> выводить каждую строку, которая не содержит соответствие.
-c	--count	Выводит количество совпадений (или количество несовпадений, если опция -v вк (также указано) вместо самих строк самих.
-l	--files-with-matches	Выводить имя каждого файла, который содержит совпадение вместо самих строк.
-L	--files-without-match	Как и опция -l, но выводит только имена файлов, которые не содержат совпадения.
-n	--line-number	Добавляйте перед каждой подходящей ст номер строки в файле.
-h	--no-filename	При поиске в нескольких файлах подавляйте вывод имен файлов.

-q

--quiet, --silent Подавить весь вывод. Это полезно в сценариях оболочки, когда нужно проверить наличие совпадения. Мы разберем проверку кода завершения команды статус в главе 27.

Чтобы подробнее изучить грег, создадим несколько текстовых файлов для поиска.

```
[me@linuxbox ~]$ ls /bin > dirlist-bin.txt
[me@linuxbox ~]$ ls /usr/bin > dirlist-usr-bin.txt
[me@linuxbox ~]$ ls /sbin > dirlist-sbin.txt
[me@linuxbox ~]$ ls /usr/sbin > dirlist-usr-sbin.txt
[me@linuxbox ~]$ ls dirlist*.txt
dirlist-bin.txt  dirlist-sbin.txt  dirlist-usr-sbin.txt
dirlist-usr-bin.txt
```

Мы можем выполнить простой поиск по списку файлов следующим образом:

```
[me@linuxbox ~]$ grep bzip dirlist*.txt
dirlist-bin.txt:bzip2
dirlist-bin.txt:bzip2recover
```

В этом примере грег ищет строку bzip во всех перечисленных файлах и находит два совпадения, оба в файле dirlist-bin.txt. Если бы нас интересовал только список файлов с совпадениями, а не сами совпадения, мы могли бы указать опцию -l.

```
[me@linuxbox ~]$ grep -l bzip dirlist*.txt
dirlist-bin.txt
```

И наоборот, если мы хотим увидеть только список файлов, не содержащих совпадений, можно сделать так:

```
[me@linuxbox ~]$ grep -L bzip dirlist*.txt
dirlist-sbin.txt
dirlist-usr-bin.txt
dirlist-usr-sbin.txt
```

Метасимволы и литералы

Хотя это может быть неочевидно, наши грег поиски всегда использовали регулярные выражения, пусть и очень простые. Регулярное выражение bzip означает, что совпадение произойдет только если строка в файле содержит как минимум четыре символа и где-то в строке символы b, z, i и p идут в таком порядке без других символов между ними. Символы в строке bzip являются литералами, так как они соответствуют сами себе. Помимо литералов, регулярные выражения могут включать метасимволы для описания более сложных совпадений. Метасимволы регулярных выражений включают следующее:

`^ $ . [ ] { } - ? * + ( ) | \`

Все остальные символы считаются литералами, хотя обратный слэш используется в нескольких случаях для создания метапоследовательностей, а также для экранирования метасимволов, чтобы они обрабатывались как литералы, а не интерпретировались как метасимволы.

Примечание: Как мы видим, многие метасимволы регулярных выражений также имеют значение для оболочки при выполнении раскрытия. Когда мы передаем регулярные выражения с метасимволами в командной строке, крайне важно заключать их в кавычки, чтобы оболочка не пыталась их раскрыть.

Любой символ

Первый метасимвол, который мы рассмотрим, — это точка, используемая для соответствия любому символу. Если включить её в регулярное выражение, она будет соответствовать любому символу на данной позиции. Вот пример:

```
[me@linuxbox ~]$ grep -h '.zip' dirlist*.txt
bunzip2
bzip2
bzip2recover
gunzip
gzip
funzip
gpg-zip
preunzip
prezip
prezip-bin
unzip
unzipsfx
```

Мы искали в файлах любые строки, соответствующие регулярному выражению «.zip». В результатах есть пара интересных моментов. Заметьте, что программа `zip` не была найдена. Это произошло потому, что использование метасимвола точки в регулярном выражении увеличило длину искомого совпадения до четырех символов, один из которых должен предшествовать «z». Кроме того, если бы в наших списках были файлы с расширением `.zip`, они бы тоже подошли под условие, так как точка в расширении файла также совпала бы с «любым символом».

Якоря

Символы каретки (^) и знака доллара (\$) в регулярных выражениях считаются якорями. Это означает, что совпадение произойдет только в том случае, если регулярное выражение находится в начале строки (^) или в конце строки (\$).

```
[me@linuxbox ~]$ grep -h '^zip' dirlist*.txt
zip
zipcloak
zipgrep
zipinfo
zipnote
zipsplit
[me@linuxbox ~]$ grep -h 'zip$' dirlist*.txt
gunzip
gzip
funzip
pgp-zip
preunzip
prezip
unzip
zip
[me@linuxbox ~]$ grep -h '^zip$' dirlist*.txt
zip
```

Здесь мы искали в списке файлов строку zip, расположенную в начале строки, в конце строки и в строке, где она находится одновременно и в начале, и в конце (то есть сама по себе в строке). Примечание: регулярное выражение ^\$ (начало и конец без содержимого между ними) будет соответствовать пустым строкам.

Помощник для разгадывания кроссвордов

Даже с нашими нынешними ограниченными знаниями регулярных выражений мы можем сделать нечто полезное.

Моя жена обожает кроссворды и иногда просит меня помочь с каким-нибудь вопросом. Что-то вроде: «Какое пятибуквенное слово, где третья буква — „j“, а последняя — „r“, означающее...?» Этот вопрос натолкнул меня на мысль.

Знаете ли вы, что в вашей системе Linux есть словарь? Так и есть. Загляните в /usr/share/dict каталог, и вы наверняка найдете один или несколько. Файлы словарей там представляют собой просто длинные списки слов, по одному на строку, в алфавитном порядке. В моей системе файл words содержит чуть более 98 500 слов. Чтобы найти возможные ответы на вопрос жены, мы можем сделать следующее:

```
[me@linuxbox ~]$ grep -i '^..j.r$' /usr/share/dict/words Major major
```

Используя это регулярное выражение, мы сможем найти в файле словаря все пятибуквенные слова, у которых j стоит на третьей позиции, а r — на последней.

Скобочные выражения и символьные классы

Помимо соответствия любому символу в заданной позиции регулярного выражения, с помощью скобочных выражений можно искать один символ из определенного набора. Скобочные выражения позволяют задать набор символов (включая те, что иначе считались бы метасимволами) для поиска. В этом примере, используя набор из двух символов, мы ищем любую строку, содержащую bzip или gzip:

```
[me@linuxbox ~]$ grep -h '[bg]zip' dirlist*.txt
bzip2
bzip2recover
gzip
```

Набор может содержать любое количество символов, а метасимволы теряют свое специальное значение, если помещены в скобки. Однако есть два случая, когда метасимволы внутри скобочных выражений имеют иное значение. Первый — это символ каретки (^), используемый для отрицания; второй — тире (-), используемое для указания диапазона символов.

Отрицание

Если первым символом в скобочном выражении является символ каретки (^), остальные символы считаются набором, который не должен присутствовать в данной позиции. Мы сделаем это, изменив наш предыдущий пример следующим образом:

```
[me@linuxbox ~]$ grep -h '[^bg]zip' dirlist*.txt
bunzip2
gunzip
funzip
gpg-zip
preunzip
prezip
prezip-bin
unzip
unzipsfx
```

При включенном отрицании мы получаем список файлов, содержащих строку zip, перед которой стоит любой символ, кроме b или g. Заметьте, что файл zip не был найден.

Отрицаемый набор символов все равно требует наличия символа в данной позиции, но этот символ не должен входить в отрицаемый набор.

Символ каретки вызывает отрицание только в том случае, если он является первым символом внутри скобочного выражения; в

противном случае он теряет свое специальное значение и становится обычным символом в наборе.

Традиционные диапазоны символов

Чтобы составить регулярное выражение для поиска всех файлов в наших списках, начинающихся с заглавной буквы, можно сделать следующее:

```
[me@linuxbox ~]$ grep -h '^[ABCDEFGHJKLMNOPQRSTUVWXYZ]' dirlist*.txt
```

Достаточно поместить все 26 заглавных букв в квадратное выражение. Но такая долгая печать вызывает сомнения, поэтому есть другой способ.

```
[me@linuxbox ~]$ grep -h '^[A-Z]' dirlist*.txt
MAKEDEV
ControlPanel
GET
HEAD
POST
X
X11
Xorg
MAKEFLOPPIES
NetworkManager
NetworkManagerDispatcher
```

Используя диапазон из трех символов, можно сократить перечисление 26 букв. Любой диапазон символов можно выразить таким образом, включая несколько диапазонов, как в этом выражении, которое находит все имена файлов, начинающиеся с букв или цифр:

```
[me@linuxbox ~]$ grep -h '^[A-Za-z0-9]' dirlist*.txt
```

В диапазонах символов знак тире обрабатывается особым образом, так как же включить тире в выражение в скобках? Сделав его первым символом в выражении. Рассмотрим эти два примера:

```
[me@linuxbox ~]$ grep -h '[A-Z]' dirlist*.txt
```

Это найдет каждое имя файла, содержащее заглавную букву. Следующее выражение найдет каждое имя файла, содержащее тире, заглавную A или заглавную Z:

```
[me@linuxbox ~]$ grep -h '[-AZ]' dirlist*.txt
```

POSIX-классы символов

Традиционные диапазоны символов — это простой и эффективный способ быстрого указания наборов символов. К

сожалению, они не всегда работают. Хотя при использовании `grep` мы пока не сталкивались с проблемами, в других программах они могут возникнуть.

В главе 4 мы рассматривали, как подстановочные знаки используются для раскрытия путей. Тогда мы упоминали, что диапазоны символов можно использовать почти так же, как в регулярных выражениях, но здесь возникает проблема:

```
[me@linuxbox ~]$ ls /usr/sbin/[ABCDEFGHijklmnopqrstuvwxyz]*  
/usr/sbin/MAKEFLOPPIES  
/usr/sbin/NetworkManagerDispatcher  
/usr/sbin/NetworkManager
```

(В зависимости от дистрибутива Linux список файлов может отличаться или быть пустым; данный пример взят из Ubuntu). Эта команда дает ожидаемый результат — список файлов, имена которых начинаются с заглавной буквы, но с этой командой мы получаем совсем иной результат (показан лишь фрагмент):

```
[me@linuxbox ~]$ ls /usr/sbin/[A-Z]*  
/usr/sbin/biosdecode  
/usr/sbin/chat  
/usr/sbin/chgpasswd  
/usr/sbin/chpasswd  
/usr/sbin/chroot  
/usr/sbin/cleanup-info  
/usr/sbin/complain  
/usr/sbin/console-kit-daemon
```

Почему так происходит? Это долгая история, но вот краткая версия:

Когда Unix только разрабатывался, он знал только символы ASCII, и эта особенность отражает тот факт. В ASCII первые 32 символа (номера 0–31) — это управляющие коды (такие как табуляция, `backspace` и возврат каретки). Следующие 32 (32–63) содержат печатные символы, включая большинство знаков препинания и цифры 0–9. Следующие 32 (номера 64–95) содержат заглавные буквы и еще несколько знаков препинания. Последние 31 (номера 96–126) содержат строчные буквы и еще больше знаков препинания. Исходя из этого расположения, системы, использующие ASCII, применяли следующий порядок сортировки:

ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz

Это отличается от правильного словарного порядка, который выглядит так:

aAbBcCdDeEfFgGhHiIjJkKlLmMnNoOpPqQrRsStTuUvVwWxXyYzZ

По мере распространения Unix за пределами США возникла потребность в поддержке символов, отсутствующих в

американском английском. Таблица ASCII была расширена до восьми бит (символы 128–255), что позволило добавить множество других языков. Для реализации этой возможности стандарты POSIX ввели понятие `locale`, которое можно настраивать для выбора набора символов, необходимого в конкретном регионе. Проверить настройки языка нашей системы можно с помощью этой команды:

```
[me@linuxbox ~]$ echo $LANG
en_US.UTF-8
```

При такой настройке приложения, соответствующие стандарту POSIX, будут использовать словарь для сортировки вместо порядка ASCII. Это объясняет поведение предыдущих команд. Диапазон символов `[A-Z]`, если интерпретировать его в словарном порядке, включает все буквы алфавита, кроме строчных `a`, что и дает наши результаты.

Для частичного решения этой проблемы стандарт POSIX включает ряд классов символов, предоставляющих полезные диапазоны, как описано в таблице 19-2.

Таблица 19-2: POSIX-классы символов

Класс символов	Описание
<code>[:alnum:]</code>	Буквенно-цифровые символы. В ASCII эквивалентны: <code>[A-Za-z0-9]</code>
<code>[:word:]</code>	То же, что и <code>[:alnum:]</code> , но с добавлением нижнего подчеркивания символ (<code>_</code>).
<code>[:alpha:]</code>	Буквенные символы. В ASCII эквивалентно: <code>[A-Za-z]</code>
<code>[:blank:]</code>	Включает пробел и символ табуляции.
<code>[:cntrl:]</code>	Управляющие коды ASCII. Включает символы ASCII от 0 до 31 и 127.
<code>[:digit:]</code>	Цифры от 0 до 9.
<code>[:graph:]</code>	Печатные символы. В ASCII, они включают символы с 33 по 126.
<code>[:lower:]</code>	Строчные буквы.
<code>[:punct:]</code>	Знаки препинания. В ASCII эквивалентны: <code>[- ! " # \$ % & ' () * + , . / : ; < = > ? @ [\] ^ _ ` { } ~]</code>

Класс символов	Описание
<code>[:print:]</code>	Печатные символы. Все символы в <code>[:graph:]</code> а также символ пробела.
<code>[:space:]</code>	Пробельные символы, включая пробел, табуляцию, возврат каретки, <code>newline</code> , вертикальная табуляция, и разрыв страницы. В ASCII, эквивалентно: <code>[\t\r\n\v\f]</code>
<code>[:upper:]</code>	Заглавные буквы.
<code>[:xdigit:]</code>	Символы, используемые для представления шестнадцатеричных чисел. В ASCII, эквивалентно: <code>[0-9A-Fa-f]</code>

Даже с классами символов по-прежнему нет удобного способа выразить частичные диапазоны, такие как `[A-M]`.

Используя классы символов, мы можем повторить вывод списка каталогов и получить улучшенный результат.

```
[me@linuxbox ~]$ ls /usr/sbin/[[:upper:]]*
/usr/sbin/MAKEFLOPPIES
/usr/sbin/NetworkManagerDispatcher
/usr/sbin/NetworkManager
```

Однако помните, что это не пример регулярного выражения, а оболочка выполняет раскрытие пути. Мы приводим это здесь, так как классы символов POSIX применимы в обоих случаях.

Возврат к традиционному порядку сортировки

Вы можете настроить систему на использование традиционного (ASCII) порядка сортировки, изменив значение переменной окружения `LANG`. Как мы видели ранее, `LANG` содержит название языка и кодировки, используемых в вашей локали. Это значение было определено при выборе языка установки во время инсталляции Linux.

Чтобы просмотреть настройки локали, используйте команду `locale`.

```
[me@linuxbox ~]$ locale LANG=en_US.UTF-8 LC_CTYPE="en_US.UTF-8"
LC_NUMERIC="en_US.UTF-8" LC_TIME="en_US.UTF-8" LC_COLLATE="en_US.UTF-8"
LC_MONETARY="en_US.UTF-8" LC_MESSAGES="en_US.UTF-8"
LC_PAPER="en_US.UTF-8" LC_NAME="en_US.UTF-8" LC_ADDRESS="en_US.UTF-8"
LC_TELEPHONE="en_US.UTF-8" LC_MEASUREMENT="en_US.UTF-8"
LC_IDENTIFICATION="en_US.UTF-8" LC_ALL=
```

Чтобы изменить локаль на традиционное поведение Unix, установите переменную `LANG` в значение `POSIX`.

```
[me@linuxbox ~]$ export LANG=POSIX
```

Примечание: это изменение переключит систему на использование американского английского (точнее, ASCII) в качестве набора символов, поэтому убедитесь, что это именно то, что вам нужно.

Вы можете сделать это изменение постоянным, добавив эту строку в ваш файл `.bashrc`.

```
export LANG=POSIX
```

Базовые и расширенные регулярные выражения POSIX

Как раз тогда, когда нам казалось, что запутаться еще сильнее невозможно, выясняется, что POSIX разделяет реализации регулярных выражений на два типа: базовые (BRE) и расширенные (ERE). Функции, которые мы рассмотрели, поддерживаются любым приложением, соответствующим стандарту POSIX и реализующим BRE. Наша `grep` программа — как раз такой пример.

В чем разница между BRE и ERE? Все дело в метасимволах. В BRE распознаются следующие метасимволы:

`^ $ . [ ] *`

Все остальные символы считаются литералами. В ERE добавляются следующие метасимволы (и связанные с ними функции):

`() { } ? + |`

Однако (и это самое интересное), символы `(, , {, и }` считаются метасимволами в BRE, если они экранированы обратным слешем, тогда как в ERE обратный слеш перед любым метасимволом превращает его в литерал. Все нюансы будут разобраны в последующих разделах.

Поскольку следующие функции являются частью ERE, нам потребуется другой `grep`. Традиционно это выполнялось программой `egrep`, но версия `grep` от GNU также поддерживает расширенные регулярные выражения при использовании опции `-E`.

POSIX

В 1980-х Unix стал очень популярной коммерческой операционной системой, но к 1988 году мир Unix погрузился в хаос. Многие производители компьютеров лицензировали исходный код Unix у его создателей, AT&T, и поставляли различные версии ОС вместе со своими системами. Однако в попытках создать уникальный продукт каждый производитель добавлял собственные изменения и расширения. Это начало ограничивать совместимость ПО. Как это часто бывает с проприетарными вендорами, каждый пытался навязать клиентам стратегию «привязки». Этот мрачный период в истории Unix сегодня называют «балканизацией».

Обратимся к Институту инженеров электротехники и электроники (IEEE). В середине 1980-х IEEE начал разработку набора стандартов, определяющих работу Unix-подобных систем. Эти стандарты, официально известные как IEEE 1003, определяют интерфейсы прикладного программирования (API), оболочку и утилиты, которые должны присутствовать в стандартной Unix-подобной системе. Название POSIX, расшифровывающееся как Portable Operating System Interface (буква X добавлена в конце для краткости), предложил Ричард Столлман (да, тот самый Ричард Столлман) и было принято IEEE.

Чередование

Первая из обсуждаемых нами функций расширенных регулярных выражений называется чередованием — это возможность сопоставления с одним из набора выражений. Подобно тому как символьный класс позволяет сопоставить один символ из заданного набора, чередование позволяет искать совпадения среди набора строк или других регулярных выражений.

Для демонстрации мы используем `grep` вместе с `echo`. Сначала попробуем обычный поиск по строке.

```
[me@linuxbox ~]$ echo "AAA" | grep AAA
AAA
[me@linuxbox ~]$ echo "BBB" | grep AAA
[me@linuxbox ~]$
```

Это простой пример: мы передаем вывод `echo` в `grep` через конвейер и смотрим на результат. При совпадении мы видим строку на экране; если совпадений нет — ничего не выводится.

Теперь добавим чередование, обозначаемое метасимволом вертикальной черты.

```
[me@linuxbox ~]$ echo "AAA" | grep -E 'AAA|BBB'
AAA
[me@linuxbox ~]$ echo "BBB" | grep -E 'AAA|BBB'
BBB
[me@linuxbox ~]$ echo "CCC" | grep -E 'AAA|BBB'
[me@linuxbox ~]$
```

Здесь мы видим регулярное выражение 'AAA|BBB', которое означает «совпадение либо со строкой AAA, либо со строкой BBB». Заметьте, что, поскольку это расширенная функция, мы добавили опцию -E к grep (хотя можно было просто использовать программу egrep), а само регулярное выражение заключили в кавычки, чтобы оболочка не интерпретировала метасимвол вертикальной черты как оператор конвейера. Чередование не ограничивается двумя вариантами.

```
[me@linuxbox ~]$ echo "AAA" | grep -E 'AAA|BBB|CCC'
AAA
```

Чтобы сочетать чередование с другими элементами регулярных выражений, можно использовать () для отделения чередования.

```
[me@linuxbox ~]$ grep -Eh '^(bz|gz|zip)' dirlist*.txt
```

Это выражение найдет в наших списках имена файлов, начинающиеся с bz, gz или zip. Если бы мы не использовали скобки, смысл выражения изменился бы: оно искало бы любой файл, который начинается с bz, содержит gz или содержит zip:.

```
[me@linuxbox ~]$ grep -Eh '^bz|gz|zip' dirlist*.txt
```

Квантификаторы

Расширенные регулярные выражения поддерживают несколько способов указания количества совпадений элемента, как описано в следующих разделах.

? — соответствие элемента нулю или одному разу

Этот квантификатор фактически означает: «сделайте предыдущий элемент необязательным». Допустим, мы хотим проверить номер телефона на корректность и считаем его верным, если он соответствует одной из двух форм, где n — цифра:

(nnn) nnn-nnnn

nnn nnn-nnnn

Мы можем составить регулярное выражение следующим образом:

```
^\(?:[0-9][0-9][0-9]\)? [0-9][0-9][0-9]-[0-9][0-9][0-9][0-9]$
```

В этом выражении мы добавляем вопросительные знаки после скобок, чтобы указать, что они должны соответствовать нулю или одному разу. Опять же, поскольку скобки обычно являются метасимволами (в ERE), мы предшествуем им обратными косыми чертами, чтобы они воспринимались как литералы.

Давайте попробуем.

```
[me@linuxbox ~]$ echo "(555) 123-4567" | grep -E '^\(?:[0-9][0-9][0-9]\)? [0-9][0-9][0-9]-[0-9][0-9][0-9][0-9]$'
(555) 123-4567
[me@linuxbox ~]$ echo "555 123-4567" | grep -E '^\(?:[0-9][0-9][0-9]\)? [0-9][0-9][0-9]-[0-9][0-9][0-9][0-9]$'
555 123-4567
[me@linuxbox ~]$ echo "AAA 123-4567" | grep -E '^\(?:[0-9][0-9][0-9]\)? [0-9][0-9][0-9]-[0-9][0-9][0-9][0-9]$'
[me@linuxbox ~]$
```

Здесь мы видим, что выражение соответствует обоим форматам номера телефона, но не подходит для номера с нецифровыми символами. Это выражение не идеально, так как оно все еще допускает непарные скобки вокруг кода города, но оно выполнит первый этап проверки.

* — Соответствие элементу ноль или более раз

Как и метасимвол `?`, `*` используется для обозначения необязательного элемента; однако, в отличие от `?`, элемент может встречаться любое количество раз, а не только один. Допустим, мы хотим проверить, является ли строка предложением: то есть она начинается с заглавной буквы, затем содержит любое количество прописных и строчных букв или пробелов и заканчивается точкой. Чтобы соответствовать этому (грубому) определению предложения, можно использовать такое регулярное выражение:

```
^[[:upper:]]+[[:upper:][:lower:]]*\.
```

Выражение состоит из трех элементов: квадратного выражения с классом символов `[[:upper:]]`, квадратного выражения с классами символов `[[:upper:]]` и `[[:lower:]]` и пробелом, а также точки, экранированной обратным слешем. Второй элемент завершается метасимволом `*`, чтобы после первой заглавной буквы в нашем предложении любое количество заглавных и строчных букв или пробелов все равно соответствовало шаблону.


```
[me@linuxbox ~]$ echo "This works." | grep -E '^[[[:upper:]]*[[[:lower:]]]*\.'
```

```
This works.
```

```
[me@linuxbox ~]$ echo "This Works." | grep -E '^[[[:upper:]]*[[[:upper:]]*[[[:lower:]]]*\.'
```

```
This Works.
```

```
[me@linuxbox ~]$ echo "this does not" | grep -E '^[[[:upper:]]*[[[:upper:]]*[[[:lower:]]]*\.'
```

```
[me@linuxbox ~]$
```

Выражение проходит первые две проверки, но не третью, так как в ней отсутствует требуемая заглавная буква в начале и точка в конце.

+ — Сопоставление элемента один или более раз

Метасимвол **+** работает почти так же, как *****, за исключением того, что для сопоставления требуется наличие хотя бы одного экземпляра предшествующего элемента. Вот регулярное выражение, которое сопоставит только строки, состоящие из групп по одному или более буквенных символов, разделенных одиночными пробелами:

`^([[[:alpha:]]+ ?)+$`

```
[me@linuxbox ~]$ echo "This that" | grep -E '^([[[:alpha:]]+ ?)+$'
```

```
This that
```

```
[me@linuxbox ~]$ echo "a b c" | grep -E '^([[[:alpha:]]+ ?)+$'
```

```
a b c
```

```
[me@linuxbox ~]$ echo "a b 9" | grep -E '^([[[:alpha:]]+ ?)+$'
```

```
[me@linuxbox ~]$ echo "abc d" | grep -E '^([[[:alpha:]]+ ?)+$'
```

```
[me@linuxbox ~]$
```

Мы видим, что это выражение не соответствует строке `a b 9`, так как она содержит небуквенный символ; оно также не соответствует `abc d`, поскольку символы `c` и `d` разделены более чем одним пробелом.

{ } — Сопоставление элемента определенное количество раз

Метасимволы **{** и **}** используются для указания минимального и максимального количества совпадений. Как показано в таблице 19-3, их можно задать четырьмя способами.

Таблица 19-3: Указание количества совпадений

Спецификатор	Значение
<code>{n}</code>	Соответствие предыдущему элементу, если он встречается ровно <i>n</i> раз.
<code>{n,m}</code>	Соответствие предыдущему элементу, если он встречается не менее <i>n</i> , но не более <i>m</i> раз.

Спецификатор	Значение
{n, }	Соответствие предыдущему элементу, если он встречается n или более раз.
{, m}	Соответствие предыдущему элементу, если он встречается не более m раз.

Возвращаясь к нашему примеру с номерами телефонов, мы можем использовать этот метод задания повторений, чтобы упростить исходное регулярное

выражение: `^\([0-9][0-9][0-9]\)? [0-9][0-9][0-9]-[0-9][0-9][0-9]$`

к следующему:

`^\([0-9]{3}\)? [0-9]{3}-[0-9]{4}$`

Давайте попробуем.

```
[me@linuxbox ~]$ echo "(555) 123-4567" | grep -E '^\([0-9]{3}\)? [0-9]{3}-[0-9]{4}$'
(555) 123-4567
[me@linuxbox ~]$ echo "555 123-4567" | grep -E '^\([0-9]{3}\)? [0-9]{3}-[0-9]{4}$'
555 123-4567
[me@linuxbox ~]$ echo "5555 123-4567" | grep -E '^\([0-9]{3}\)? [0-9]{3}-[0-9]{4}$'
[me@linuxbox ~]$
```

Как видим, наше исправленное выражение успешно проверяет числа как со скобками, так и без них, отсеивая при этом некорректно отформатированные значения.

Применение регулярных выражений на практике

Рассмотрим некоторые уже знакомые нам команды и посмотрим, как их можно использовать с регулярными выражениями.

Проверка списка номеров телефонов с помощью grep

В предыдущем примере мы проверяли формат одного номера телефона. В реальности чаще приходится проверять целые списки, так что давайте составим такой список. Мы сделаем это, произнесем магическое заклинание в командной строке. Оно покажется магическим, так как мы еще не разбирали большинство используемых команд, но не волнуйтесь — мы изучим их в следующих главах. Вот это заклинание:

```
[me@linuxbox ~]$ for i in {1..10}; do echo "(${RANDOM:0:3}) ${RANDOM:0:3}-${RANDOM:0:4}" >> phonelist.txt; done
```

Эта команда создаст файл `phonelist.txt`, содержащий 10 номеров телефонов. При каждом повторении команды в список добавляется еще 10 номеров. Мы также можем изменить значение 10 в начале команды, чтобы получить больше или меньше номеров. Однако при изучении содержимого файла мы обнаружим проблему.

```
[me@linuxbox ~]$ cat phonelist.txt
(232) 298-2265
(624) 381-1078
(540) 126-1980
(874) 163-2885
(286) 254-2860
(292) 108-518
(129) 44-1379
(458) 273-1642
(686) 299-8268
(198) 307-2440
```

Некоторые номера имеют неверный формат, что идеально подходит для наших целей, так как мы будем использовать `grep` для их проверки.

Одним из полезных методов проверки было бы просканировать файл на наличие неверных номеров и вывести полученный список.

```
[me@linuxbox ~]$ grep -Ev '^([0-9]{3}\) [0-9]{3}-[0-9]{4}$'
phonelist.txt
(292) 108-518
(129) 44-1379
[me@linuxbox ~]$
```

Здесь мы используем опцию `-v` для инвертированного поиска, чтобы вывести из списка только те строки, которые не соответствуют указанному выражению. Само выражение включает якоря по краям, чтобы гарантировать отсутствие лишних символов в числе. В отличие от нашего предыдущего примера с номером телефона, это выражение также требует обязательного наличия скобок в корректном числе.

Поиск «некрасивых» имен файлов с помощью `find`

Команда `find` поддерживает тестирование на основе регулярного выражения. При использовании регулярных выражений в `find` по сравнению с `grep` важно учитывать один нюанс. Если `grep` выводит строку, когда она содержит подстроку, соответствующую выражению, то `find` требует полного соответствия пути регулярному выражению. В следующем

примере мы используем `find` с регулярным выражением, чтобы найти каждый путь, содержащий любой символ, не входящий в следующий набор:

```
[ -_./0-9a-zA-Z ]
```

Такое сканирование выявит пути, содержащие пробелы и другие потенциально проблемные символы.

```
[me@linuxbox ~]$ find . -regex '.*[^-_./0-9a-zA-Z].*'
```

Поскольку требуется точное совпадение всего пути, мы используем `.` с обеих сторон выражения для соответствия любому количеству любых символов. В середине выражения мы используем инвертированное символьное множество, содержащее набор допустимых символов пути.

Поиск файлов с помощью `locate`

Программа `locate` поддерживает как базовые (опция `--regex`), так и расширенные (опция `-regex`) регулярные выражения. С их помощью можно выполнять многие из тех же операций, что и ранее с нашими `dirlist` файлами.

```
[me@linuxbox ~]$ locate --regex 'bin/(bz|gz|zip)'\n/bin/bzcat\n/bin/bzcmp\n/bin/bzdiff\n/bin/bzegrep\n/bin/bzexe\n/bin/bzfgrep\n/bin/bzgrep\n/bin/bzip2\n/bin/bzip2recover\n/bin/bzless\n/bin/bzmore\n/bin/gzexe\n/bin/gzip\n/usr/bin/zip\n/usr/bin/zipcloak\n/usr/bin/zipgrep\n/usr/bin/zipinfo\n/usr/bin/zipnote\n/usr/bin/zipsplit
```

Используя чередование, мы ищем пути, содержащие `bin/bz`, `bin/gz`, `bin/zip`, или.

Поиск текста с помощью `less` и `vim`

`less` и `vim` используют один и тот же метод поиска текста.

Нажатие клавиши `/` с последующим вводом регулярного выражения запустит поиск. Если мы используем `less`, чтобы просмотреть наш файл `phonenumber.txt`, вот так:

```
[me@linuxbox ~]$ less phonelist.txt
```

и выполним поиск по нашему выражению проверки, вот так:

```
(232) 298-2265
(624) 381-1078
(540) 126-1980
(874) 163-2885
(286) 254-2860
(292) 108-518
(129) 44-1379
(458) 273-1642
(686) 299-8268
(198) 307-2440
~
~
~
/^\[0-9]{3}\) [0-9]{3}-[0-9]{4}$
```

затем less подсветит совпадения, чтобы неверные строки было легко заметить.

```
(232) 298-2265
(624) 381-1078
(540) 126-1980
(874) 163-2885
(286) 254-2860
(292) 108-518
(129) 44-1379
(458) 273-1642
(686) 299-8268
(198) 307-2440
~
~
~
(END)
```

vim, с другой стороны, поддерживает базовые регулярные выражения, поэтому наше выражение поиска будет выглядеть так:

```
/([0-9]\{3\}) [0-9]\{3\}-[0-9]\{4\}
```

Мы видим, что выражение почти не изменилось; однако многие символы, являющиеся метасимволами в расширенных выражениях, в базовых выражениях считаются литералами. Они становятся метасимволами только при экранировании обратным слешем. В зависимости от конфигурации vim в нашей системе, совпадения будут подсвечены. Если нет, попробуйте эту команду в режиме команд, чтобы активировать подсветку:

```
:hlsearch
```

Примечание: В зависимости от вашего дистрибутива, vim может поддерживать или не поддерживать подсветку результатов поиска. В частности, Ubuntu по умолчанию

поставляет упрощенную версию `vim`. На таких системах вы можете использовать менеджер пакетов, чтобы установить более полную версию `vim`.

Итоги

В этой главе мы рассмотрели несколько способов использования регулярных выражений. Мы можем найти еще больше вариантов, если будем использовать регулярные выражения для поиска других приложений, которые их применяют. Это можно сделать с помощью поиска в `man`-страницах.

```
[me@linuxbox ~]$ cd /usr/share/man/man1
[me@linuxbox man1]$ zgrep -El 'regex|regular expression' *.gz
```

Программа `zgrep` служит интерфейсом для `grep`, позволяя ей читать сжатые файлы. В нашем примере мы ищем в обычном месторасположении сжатые файлы страниц `man` раздела 1 — `tion`. Результатом этой команды будет список файлов, содержащих строку `regex` или строку `regular expression`. Как видим, регулярные выражения встречаются во многих программах.

В базовых регулярных выражениях есть одна особенность, которую мы не рассмотрели. Она называется обратными ссылками, и мы обсудим её в следующей главе.

Что почитать

Существует множество онлайн-ресурсов для изучения регулярных выражений, включая различные руководства и шпаргалки.

Кроме того, в Википедии есть хорошие статьи по следующим темам:

- POSIX: <https://en.wikipedia.org/wiki/Posix>
- ASCII: <https://en.wikipedia.org/wiki/Ascii>

20 — Обработка текста

Все Unix-подобные операционные системы полагаются на текстовые файлы для хранения данных. Поэтому логично, что существует множество инструментов для манипуляции текстом. В этой главе мы рассмотрим программы, используемые для «разрезания и дробления» текста. В следующей главе мы изучим более глубокую обработку текста, сосредоточившись на программах для форматирования текста перед печатью и для удобства чтения человеком.

В этой главе мы вернемся к старым знакомым и познакомимся с новыми:

- `cat` — объединяет файлы и выводит результат в стандартный вывод
- `sort` — сортирует строки текстовых файлов
- `uniq` — выводит или пропускает повторяющиеся строки
- `cut` — удаление фрагментов из каждой строки файлов
- `paste` — объединение строк файлов
- `join` — соединение строк двух файлов по общему полю
- `tac` — склейка и вывод файлов в обратном порядке
- `rev` — реверс строк посимвольно
- `comm` — построчное сравнение двух отсортированных файлов
- `diff` — построчное сравнение файлов
- `patch` — применение diff-файла к оригиналу
- `tr` — замена или удаление символов
- `sed` — потоковый редактор для фильтрации и преобразования текста
- `aspell` — интерактивная проверка орфографии

Применение текста

Пока что мы изучили пару текстовых редакторов (`nano` и `vim`), просмотрели множество конфигурационных файлов и увидели вывод десятков команд — и всё это в текстовом виде. Но для чего ещё используется текст? Как выяснилось, для многих вещей.

Документы

Многие пишут документы в формате обычного текста. Если понятно, как малые текстовые файлы полезны для простых

заметок, то большие документы тоже можно создавать в текстовом формате. Популярный подход — писать текст, используя язык разметки для описания форматирования готового документа. Так создается множество научных работ, поскольку системы обработки текста на базе Unix одними из первых поддерживали сложную типографику, необходимую техническим специалистам. В последние годы популярным языком разметки, использующим обычный текст для форматирования, стал Markdown.

Веб-страницы

Самый популярный тип электронных документов в мире — это, вероятно, веб-страницы. Веб-страницы представляют собой текстовые документы, использующие Hypertext Markup Language (HTML) или Extensible Markup Language (XML) в качестве языков разметки для описания визуального формата документа.

Электронная почта

Электронная почта — это по своей сути текстовая среда. Даже нетекстовые вложения преобразуются в текстовое представление для передачи. Мы можем убедиться в этом сами, скачав сообщение и просмотрев его в less. Мы увидим, что сообщение начинается с заголовка, описывающего источник и историю обработки письма, за которым следует тело сообщения с его содержимым.

Вывод на принтер

В Unix-подобных системах данные для принтера отправляются в виде обычного текста или, если страница содержит графику, преобразуются в текстовый язык описания страниц PostScript, который затем передается программе, генерирующей графические точки для печати.

Исходный код программы

Многие программы командной строки в Unix-подобных системах созданы для системного администрирования и разработки ПО, и программы для обработки текста — не исключение. Многие из них предназначены для решения задач программирования. Обработка текста важна для разработчиков, так как любое ПО начинается с текста. Исходный код — часть программы, которую пишет программист, — всегда представлен в текстовом формате.

Возвращение к старым друзьям

В главе 6 мы уже упоминали команды, которые, помимо аргументов командной строки, могут принимать стандартный ввод. Тогда мы лишь вскользь коснулись этой темы, но теперь подробно разберем, как их использовать для обработки текста.

cat

Программа `cat` имеет ряд интересных опций. Многие из них помогают лучше визуализировать текстовое содержимое. Один из примеров — опция `-A`, которая используется для отображения непечатных символов. Иногда нам нужно знать, содержатся ли управляющие символы в обычно видимом тексте. Чаще всего это символы табуляции (в отличие от пробелов) и возвраты каретки, часто встречающиеся в качестве символов конца строки в текстовых файлах в стиле MS-DOS. Другая типичная ситуация — файл, строки которого содержат лишние пробелы в конце.

Давайте создадим тестовый файл, используя `cat` в качестве примитивного текстового процессора. Для этого мы введем команду `cat` (указав файл для перенаправления вывода), наберем текст, нажмем `Enter`, чтобы корректно завершить строку, а затем `Ctrl-d`, чтобы сообщить `cat` о достижении конца файла. В этом примере мы введем символ табуляции в начале строки и добавим несколько пробелов в конце:

```
[me@linuxbox ~]$ cat > foo.txt
    The quick brown fox jumps over the lazy dog.
[me@linuxbox ~]$
```

Затем мы используем `cat` с опцией `-A`, чтобы вывести текст:

```
[me@linuxbox ~]$ cat -A foo.txt
^IThe quick brown fox jumps over the lazy dog.  $
[me@linuxbox ~]$
```

Как видно из результатов, символ табуляции в нашем тексте представлен как `^I`. Это общепринятое обозначение, означающее `Ctrl-i`, что, как выяснилось, идентично символу табуляции. Мы также видим, что в самом конце строки стоит `$`, указывающий на наличие лишних пробелов в конце текста.

Текст MS-DOS против текста Unix

Одна из причин использовать `cat` для поиска непечатных символов — обнаружение скрытых символов возврата каретки. Откуда они берутся? Из DOS и Windows! Unix и DOS по-разному определяют конец строки в текстовых файлах. Unix завершает строку символом перевода строки (ASCII 10),

тогда как MS-DOS и её производные используют последовательность из возврата каретки (ASCII 13) и перевода строки для завершения каждой строки текста.

Существует несколько способов преобразования файлов из формата DOS в Unix. Во многих системах Linux есть программы `dos2unix` и `unix2dos`, которые могут конвертировать текстовые файлы в формат DOS и обратно. Однако, если в вашей системе нет `dos2unix`, не беспокойтесь. Процесс преобразования текста из DOS в Unix прост — он заключается в удалении лишних символов возврата каретки. Это легко сделать с помощью пары программ, которые будут рассмотрены далее в этой главе.

`cat` также имеет опции для модификации текста. Две наиболее заметные — это `-n`, которая нумерует строки, и `-s`, которая подавляет вывод нескольких пустых строк. Продемонстрируем это следующим образом:

```
[me@linuxbox ~]$ cat > foo.txt
The quick brown fox
jumps over the lazy dog.
[me@linuxbox ~]$ cat -ns foo.txt
 1 The quick brown fox
 2
 3 jumps over the lazy dog.
[me@linuxbox ~]$
```

В этом примере мы создаем новую версию нашего `foo.txt` test файла, содержащего две строки текста, разделенные двумя пустыми строками. После обработки `cat` с опциями `-ns` лишняя пустая строка удаляется, а оставшиеся строки нумеруются. Хотя это и несложная операция, это полноценный процесс обработки текста.

sort

`sort` сортирует содержимое стандартного ввода или один или несколько файлов, указанных в командной строке, и отправляет результат в стандартный вывод. Используя тот же метод, что и с `cat`, мы можем продемонстрировать обработку стандартного ввода напрямую с клавиатуры следующим образом:

```
[me@linuxbox ~]$ sort > foo.txt
c
b
a
[me@linuxbox ~]$ cat foo.txt
a
b
c
```

После ввода команды мы печатаем с, b и a, а затем нажимаем Ctrl-d, чтобы указать конец файла. Просмотрев итоговый файл, мы увидим, что строки теперь расположены в отсортированном порядке.

sort может принимать несколько файлов в качестве аргументов командной строки, поэтому можно объединить несколько файлов в один отсортированный массив. Например, если бы у нас было три текстовых файла и мы хотели объединить их в один отсортированный файл, мы могли бы сделать следующее:

```
sort file1.txt file2.txt file3.txt > final_sorted_list.txt
```

sort имеет несколько интересных опций. В таблице 20-1 приведен их частичный список:

Таблица 20-1: Общие опции sort

Опция	Длинная опция	Описание
-b	--ignore-leading-blanks	По умолчанию сортировка выполняется по всей строке, начиная с первого символа. Эта опция заставляет sort игнорировать ведущие пробелы в строках и вычисляет сортировку по первому непробельному символу в line.
-f	--ignore-case	Сортировка без учета регистра.
-n	--numeric-sort	Выполняет сортировку на основе числового значения строки. Эта опция позволяет выполнять сортировку по числовым значениям, а не по алфавиту.
-r	--reverse	Сортировка в обратном порядке. Результат убывающем, а не возрастающем порядок.
-k	--key=field1[, field2]	Сортировка по ключевому полю, расположен от field1 до field2, а не по всей строке. См. далее обсуждение.

Опция	Длинная опция	Описание
-m	--merge	Каждый аргумент считается именем предварительно отсортированного файла. Объедините не файлов в один отсортированный результат без выполнения какой-либо дополнительной сортировки.
-o	--output=file	Направлять отсортированный вывод в фа а не в стандартный вывод.
-t	--field-separator=char	Задать символ разделителя полей. По умолчанию поля разделяются пробелами или табуляцией.

Хотя большинство этих опций интуитивно понятны, некоторые — нет. Сначала рассмотрим опцию -n, используемую для числовой сортировки. С ней можно сортировать данные на основе числовых значений. Продемонстрируем это, отсортировав результаты команды du для определения пользователей, занимающих больше всего места на диске. Обычно команда du выводит сводную информацию в порядке pathname.

```
[me@linuxbox ~]$ du -s /usr/share/* | head
252    /usr/share/aclocal
96     /usr/share/acpi-support
8      /usr/share/adduser
196    /usr/share/alacarte
344    /usr/share/alsa
8      /usr/share/alsa-base
12488  /usr/share/anthy
8      /usr/share/apmd
21440  /usr/share/app-install
48     /usr/share/application-registry
```

В этом примере мы направляем результат в head через конвейер, чтобы ограничить вывод первыми 10 строками. Таким образом можно получить список, отсортированный по числам, чтобы увидеть 10 крупнейших потребителей места.

```
[me@linuxbox ~]$ du -s /usr/share/* | sort -nr | head
509940 /usr/share/locale-langpack
242660 /usr/share/doc
197560 /usr/share/fonts
179144 /usr/share/gnome
146764 /usr/share/myspell
144304 /usr/share/gimp
135880 /usr/share/dict
76508  /usr/share/icons
```

```
68072    /usr/share/apps
62844    /usr/share/foomatic
```

Используя опции `n` и `r`, мы получаем обратную числовую сортировку, где самые большие значения отображаются первыми. Такая сортировка работает, так как числовые значения стоят в начале каждой строки. Но что, если мы хотим отсортировать список на основе значения внутри строки? Например, вот результаты `ls -l`:

```
[me@linuxbox ~]$ ls -l /usr/bin | head
total 152948
-rwxr-xr-x 1 root  root    34824 2016-04-04 02:42 [
-rwxr-xr-x 1 root  root   101556 2007-11-27 06:08 a2p
-rwxr-xr-x 1 root  root    13036 2016-02-27 08:22 aconnect
-rwxr-xr-x 1 root  root    10552 2007-08-15 10:34 acpi
-rwxr-xr-x 1 root  root     3800 2016-04-14 03:51 acpi_fakekey
-rwxr-xr-x 1 root  root     7536 2016-04-19 00:19 acpi_listen
-rwxr-xr-x 1 root  root     3576 2016-04-29 07:57 addpart
-rwxr-xr-x 1 root  root    20808 2016-01-03 18:02 addr2line
-rwxr-xr-x 1 root  root   489704 2016-10-09 17:02 adept_batch
```

Если не принимать во внимание, что `ls` умеет сортировать результаты по размеру, мы могли бы использовать `sort`, чтобы также отсортировать этот список по размеру файла.

```
[me@linuxbox ~]$ ls -l /usr/bin | sort -nrk 5 | head
-rwxr-xr-x 1 root  root   8234216 2016-04-07 17:42 inkscape
-rwxr-xr-x 1 root  root   8222692 2016-04-07 17:42 inkview
-rwxr-xr-x 1 root  root   3746508 2016-03-07 23:45 gimp-2.4
-rwxr-xr-x 1 root  root   3654020 2016-08-26 16:16 quanta
-rwxr-xr-x 1 root  root   2928760 2016-09-10 14:31 gdbtui
-rwxr-xr-x 1 root  root   2928756 2016-09-10 14:31 gdb
-rwxr-xr-x 1 root  root   2602236 2016-10-10 12:56 net
-rwxr-xr-x 1 root  root   2304684 2016-10-10 12:56 rpcclient
-rwxr-xr-x 1 root  root   2241832 2016-04-04 05:56 aptitude
-rwxr-xr-x 1 root  root   2202476 2016-10-10 12:56 smbcacls
```

Многие задачи `sort` связаны с обработкой табличных данных, например, результатов предыдущей команды `ls`. Если применить терминологию баз данных к предыдущей таблице, то каждая строка будет называться записью, а каждая запись будет состоять из нескольких полей, таких как атрибуты файла, количество ссылок, имя файла, размер файла и так далее. `sort` позволяет обрабатывать отдельные поля. Говоря языком баз данных, мы можем указать одно или несколько ключевых полей в качестве ключей сортировки. В предыдущем примере мы использовали опции `n` и `r` для выполнения обратной числовой сортировки и указали `-k 5`, чтобы заставить `sort` использовать пятое поле в качестве ключа сортировки.

Опция `k` интересна и обладает множеством функций, но сначала нужно обсудить, как `sort` определяет поля. Рассмотрим простой

текстовый файл, состоящий из одной строки с именем автора:

```
William Shotts
```

По умолчанию `sort` видит в этой строке два поля. Первое поле содержит следующие символы:

“William”

Второе поле содержит следующие символы:

“Shotts”

Это означает, что пробельные символы (пробелы и табуляция) используются как разделители полей и включаются в поле при выполнении сортировки.

Снова взглянув на строку из нашего `ls output`, представленную ниже, мы видим, что она содержит восемь полей, и пятое поле — это размер файла:

```
-rwxr-xr-x 1 root root 8234216 2016-04-07 17:42 inkscape
```

Для следующей серии экспериментов рассмотрим файл, содержащий историю трех популярных дистрибутивов Linux, выпущенных с 2006 по 2008 год. Каждая строка в файле состоит из трех полей: название дистрибутива, номер версии и дата выпуска в формате ММ/ДД/YYYY.

SUSE	10.2	12/07/2006
Fedora	10	11/25/2008
SUSE	11.0	06/19/2008
Ubuntu	8.04	04/24/2008
Fedora	8	11/08/2007
SUSE	10.3	10/04/2007
Ubuntu	6.10	10/26/2006
Fedora	7	05/31/2007
Ubuntu	7.10	10/18/2007
Ubuntu	7.04	04/19/2007
SUSE	10.1	05/11/2006
Fedora	6	10/24/2006
Fedora	9	05/13/2008
Ubuntu	6.06	06/01/2006
Ubuntu	8.10	10/30/2008
Fedora	5	03/20/2006

С помощью текстового редактора (возможно, `vim`) мы введем эти данные и сохраним файл под именем `dis-tros.txt`.

Теперь попробуем отсортировать файл и посмотрим на результат:

```
[me@linuxbox ~]$ sort distros.txt
Fedora 10      11/25/2008
Fedora 5       03/20/2006
Fedora 6       10/24/2006
Fedora 7       05/31/2007
Fedora 8       11/08/2007
```

Fedora	9	05/13/2008
SUSE	10.1	05/11/2006
SUSE	10.2	12/07/2006
SUSE	10.3	10/04/2007
SUSE	11.0	06/19/2008
Ubuntu	6.06	06/01/2006
Ubuntu	6.10	10/26/2006
Ubuntu	7.04	04/19/2007
Ubuntu	7.10	10/18/2007
Ubuntu	8.04	04/24/2008
Ubuntu	8.10	10/30/2008

В целом всё сработало. Проблема возникла при сортировке номеров версий Fedora. Поскольку в наборе символов 1 идет перед 5, версия 10 оказалась вверху, а версия 9 — внизу.

Чтобы решить эту проблему, нам придется сортировать по нескольким ключам. Мы хотим выполнить алфавитную сортировку по первому полю, а затем числовую — по второму. `sort` позволяет использовать параметр `-k` несколько раз, чтобы указать несколько ключей сортировки. На самом деле ключ может включать диапазон полей. Если диапазон не указан (как в наших предыдущих примерах), `sort` использует ключ, который начинается с указанного поля и продолжается до конца строки. Вот синтаксис для нашей многоключевой сортировки:

```
[me@linuxbox ~]$ sort --key=1,1 --key=2n distros.txt
Fedora 5 03/20/2006
Fedora 6 10/24/2006
Fedora 7 05/31/2007
Fedora 8 11/08/2007
Fedora 9 05/13/2008
Fedora 10 11/25/2008
SUSE 10.1 05/11/2006
SUSE 10.2 12/07/2006
SUSE 10.3 10/04/2007
SUSE 11.0 06/19/2008
Ubuntu 6.06 06/01/2006
Ubuntu 6.10 10/26/2006
Ubuntu 7.04 04/19/2007
Ubuntu 7.10 10/18/2007
Ubuntu 8.04 04/24/2008
Ubuntu 8.10 10/30/2008
```

Хотя для ясности мы использовали длинную форму параметра, `-k 1,1 -k 2n` были бы полностью эквивалентны. В первом случае параметра ключа мы указали диапазон полей для первого ключа. Поскольку мы хотели ограничить сортировку только первым полем, мы указали 1,1, что означает «начать с поля 1 и закончить на поле 1». Во втором случае мы указали 2n, что означает использование поля 2 в качестве ключа и числовую сортировку. В конце спецификатора ключа можно указать букву параметра, чтобы определить тип сортировки. Эти буквы совпадают с глобальными параметрами программы `sort`: `b`

(игнорировать начальные пробелы), n (числовая сортировка), r (обратная сортировка) и так далее.

Третье поле в нашем списке содержит дату в неудобном для сортировки формате. На ком-пьютерах даты обычно записывают в порядке ГГГГ-ММ-ДД для удобства хронологической сортировки, но наши представлены в американском формате ММ/ДД/ГГГГ. Как нам отсортировать этот список в хронологическом порядке?

К счастью, `sort` предоставляет такой способ. Ключевая опция позволяет указывать смещения внутри полей, так что мы можем определять ключи сортировки внутри полей.

```
[me@linuxbox ~]$ sort -k 3.7nbr -k 3.1nbr -k 3.4nbr distros.txt
Fedora 10 11/25/2008
Ubuntu 8.10 10/30/2008
SUSE 11.0 06/19/2008
Fedora 9 05/13/2008
Ubuntu 8.04 04/24/2008
Fedora 8 11/08/2007
Ubuntu 7.10 10/18/2007
SUSE 10.3 10/04/2007
Fedora 7 05/31/2007
Ubuntu 7.04 04/19/2007
SUSE 10.2 12/07/2006
Ubuntu 6.10 10/26/2006
Fedora 6 10/24/2006
Ubuntu 6.06 06/01/2006
SUSE 10.1 05/11/2006
Fedora 5 03/20/2006
```

Указывая `-k 3.7`, мы приказываем `sort` использовать ключ сортировки, начинающийся с седьмого символа третьего поля, что соответствует началу года. Аналогично, мы указываем `-k 3.1` и `-k 3.4`, чтобы выделить месяц и день в дате. Мы также добавляем опции `n` и `r` для выполнения обратной числовой сортировки. Опция `b` добавлена, чтобы убрать ведущие пробелы (количество которых варьируется от строки к строке, влияя на результат сортировки) в поле даты.

Некоторые файлы не используют табуляцию и пробелы в качестве разделителей полей; например, вот файл `/etc/passwd`:

```
[me@linuxbox ~]$ head /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
```


Поля в этом файле разделены двоеточиями (:), так как же отсортировать этот файл по ключевому полю? `sort` предоставляет опцию `-t`, позволяющую задать символ-разделитель полей. Чтобы отсортировать `passwd` по седьмому полю (оболочка по умолчанию для учетной записи), можно выполнить следующее:

```
[me@linuxbox ~]$ sort -t ':' -k 7 /etc/passwd | head
me:x:1001:1001:Myself,,,:/home/me:/bin/bash
root:x:0:0:root:/root:/bin/bash
dhcp:x:101:102:./nonexistent:/bin/false
gdm:x:106:114:Gnome Display Manager:/var/lib/gdm:/bin/false
hplip:x:104:7:HPLIP system user,,,:/var/run/hplip:/bin/false
klog:x:103:104:./home/klog:/bin/false
messagebus:x:108:119:./var/run/dbus:/bin/false
polkituser:x:110:122:PolicyKit,,,:/var/run/PolicyKit:/bin/false
pulse:x:107:116:PulseAudio daemon,,,:/var/run/pulse:/bin/false
```

Указав двоеточие в качестве разделителя полей, мы можем выполнить сортировку по седьмому полю. Примечание: `sort` по умолчанию воспринимает последовательности из нескольких пробельных символов (табуляцию и пробелы) как один разделитель, тогда как другие разделители обрабатываются по отдельности.

uniq

По сравнению с `sort`, программа `uniq` очень легкая. `uniq` выполняет, казалось бы, тривиальную задачу. Получив отсортированный файл (или стандартный ввод), она удаляет все дублирующиеся строки и отправляет результат в стандартный вывод. Ее часто используют вместе с `sort`, чтобы очистить вывод от дубликатов.

Совет: хотя `uniq` — это традиционный инструмент Unix, который часто используют с `sort`, версия `sort` от GNU поддерживает опцию `-u`, которая удаляет дубликаты из отсортированного вывода.

Давайте создадим текстовый файл, чтобы опробовать это, как показано ниже:

```
[me@linuxbox ~]$ cat > foo.txt
a
b
c
a
b
c
```

Не забывайте вводить Ctrl-d, чтобы завершить стандартный ввод. Теперь, если мы запустим `uniq` для нашего текстового файла, то получим следующее:

```
[me@linuxbox ~]$ uniq foo.txt
a
b
c
a
b
c
```

Результаты не отличаются от исходного файла; дубликаты не были удалены. Чтобы `uniq` выполнила свою работу, входные данные сначала нужно отсортировать.

```
[me@linuxbox ~]$ sort foo.txt | uniq
a
b
c
```

Это происходит потому, что `uniq` удаляет только те дублирующиеся строки, которые идут друг за другом.

`uniq` имеет несколько опций. В таблице 20-2 перечислены наиболее распространенные.

Таблица 20-2: Общие опции `uniq`

Опция	Длинная опция	Описание
-c	--count	Выводит список дублирующихся строк, перед которыми стоит количество их повторений.
-d	--repeated	Выводит только повторяющиеся строки, а не уникальные.
-f n	--skip-fields=n	Игнорирует n начальных полей в каждой строке. Поля разделены пробельными символами, как и в <code>sort</code> ; однако, в отличие от <code>sort</code> , <code>uniq</code> имеет
		нет опции для установки альтернативного разделителя.
-i	--ignore-case	Игнорировать регистр при сравнении строк.
-s n	--skip-chars=n	Пропустить (игнорировать) первые n символов каждой строки.

Опция	Длинная опция	Описание
-u	--unique	Выводятся только уникальные строки. Строки с дубликатами игнорируются.

Здесь мы видим, как `uniq` используется для вывода количества дубликатов в текстовом файле с помощью опции `-c`:

```
[me@linuxbox ~]$ sort foo.txt | uniq -c
  2 a
  2 b
  2 c
```

Нарезка и сортировка

Следующие три программы предназначены для извлечения столбцов текста из файлов и их полезной перекombинации.

cut

Программа `cut` используется для извлечения части текста из строки и вывода этой части в стандартный вывод. Она может принимать несколько аргументов-файлов или данные из стандартного ввода.

Указывать нужную часть строки несколько неудобно; для этого используются опции, перечисленные в таблице 20-3.

Таблица 20-3: Параметры выбора `cut` ольких

Опция	Длинная опция	Описание
-c list	--characters=list	Извлечь часть строки, заданной списком. Список может состоять из одного или неск числовые значения через запятую диапазоны.
-f list	--fields=list	Извлечь одно или несколько полей из строки, заданной списком. Список может содержать одно или несколько полей либо диапазоны полей, разделенные запятыми.

Опция	Длинная опция	Описание
-d delim	-- delimiter=delim	При указании -f используется delim в качестве разделителя полей. По умолчанию поля должны разделяться одним символом табуляции.
	--complement	Извлеките всю строку текст, за исключением тех частей, указанных в -с и/или -f.

Как видно, способ, которым cut извлекает текст, довольно негибкий. cut лучше всего использовать для извлечения текста из файлов, созданных другими программами, а не из текста, набранного человеком вручную. Мы изучим наш distros.txt файл, чтобы понять, достаточно ли он «чист» и подходит ли он в качестве образца для наших cut примеров. Если использовать cat с опцией -A, можно проверить, соответствует ли файл нашим требованиям к полям, разделенным табуляцией:

```
[me@linuxbox ~]$ cat -A distros.txt
SUSE^I10.2^I12/07/2006$
Fedora^I10^I11/25/2008$
SUSE^I11.0^I06/19/2008$
Ubuntu^I8.04^I04/24/2008$
Fedora^I8^I11/08/2007$
SUSE^I10.3^I10/04/2007$
Ubuntu^I6.10^I10/26/2006$
Fedora^I7^I05/31/2007$
Ubuntu^I7.10^I10/18/2007$
Ubuntu^I7.04^I04/19/2007$
SUSE^I10.1^I05/11/2006$
Fedora^I6^I10/24/2006$
Fedora^I9^I05/13/2008$
Ubuntu^I6.06^I06/01/2006$
Ubuntu^I8.10^I10/30/2008$
Fedora^I5^I03/20/2006$
```

Все выглядит хорошо. Внутри нет лишних пробелов, только одиночные символы табуляции между полями. Поскольку в файле используются табуляции, а не пробелы, для извлечения поля мы воспользуемся опцией -f.

```
[me@linuxbox ~]$ cut -f 3 distros.txt
12/07/2006
11/25/2008
06/19/2008
04/24/2008
11/08/2007
10/04/2007
10/26/2006
05/31/2007
10/18/2007
04/19/2007
```

```
05/11/2006
10/24/2006
05/13/2008
06/01/2006
10/30/2008
03/20/2006
```

Поскольку наш `distros` file разделен табуляцией, для извлечения полей лучше использовать `cut`, а не символы. В файлах с табуляцией количество символов в строках может различаться, что делает расчет позиций символов в строке сложным или невозможным. Однако в предыдущем примере мы извлекли поле, данные в котором имеют одинаковую длину, поэтому мы можем показать работу извлечения символов, вырезав год из каждой строки.

```
[me@linuxbox ~]$ cut -f 3 distros.txt | cut -c 7-10
2006
2008
2008
2008
2007
2007
2006
2007
2007
2007
2006
2006
2008
2006
2008
2006
```

Запустив `cut` второй раз для нашего списка, мы извлекаем символы с 7-й по 10-ю позицию, что соответствует году в поле даты. Нотация `7-10` является примером диапазона. Страница `cut` man содержит полное описание способов указания диапазонов.

Развертывание табуляции

Наш файл `distros.txt` идеально отформатирован для извлечения полей с помощью `cut`. Но что, если нам нужен файл, которым можно полностью манипулировать через `cut` по символам, а не по полям? Для этого потребуется заменить знаки табуляции в файле соответствующим количеством пробелов. К счастью, пакет GNU Coreutils содержит подходящий инструмент. Программа `expand` принимает один или несколько аргументов-файлов либо стандартный ввод и выводит измененный текст в стандартный вывод.

Если обрабатывать файл `distros.txt` с помощью `expand`, можно использовать `cut -c`, чтобы извлечь любой диапазон

символов. Например, мы можем использовать следующую команду, чтобы извлечь год выпуска из нашего списка, развернув файл и применив `cut` для извлечения всех символов, начиная с 23-й позиции и до конца строки:

```
[me@linuxbox ~]$ expand distros.txt | cut -c 23-
```

Coreutils также предоставляет программу `unexpand` для замены табуляции пробелами.

При работе с полями можно указать другой разделитель вместо знака табуляции. Здесь мы извлечем первое поле из файла `/etc/passwd`:

```
[me@linuxbox ~]$ cut -d ':' -f 1 /etc/passwd | head
root
daemon
bin
sys
sync
games
man
lp
mail
news
```

С помощью опции `-d` можно указать двоеточие в качестве разделителя полей.

paste

Команда `paste` делает обратное тому, что делает `cut`. Вместо извлечения столбца текста из файла она добавляет в файл один или несколько столбцов. Она считывает несколько файлов и объединяет поля из каждого файла в единый поток на стандартном выводе. Как и `cut`, `paste` принимает несколько аргументов-файлов и/или стандартный ввод. Чтобы продемонстрировать работу `paste`, мы проведем «хирургическую операцию» над нашим файлом `distros.txt`, чтобы получить хронологический список релизов.

Опираясь на нашу предыдущую работу с `sort`, сначала мы создадим список дистрибутивов, отсортированный по дате, и сохраним результат в файл `distros-by-date.txt`.

```
[me@linuxbox ~]$ sort -k 3.7nbr -k 3.1nbr -k 3.4nbr distros.txt > distros-by-date.txt
```

Затем мы воспользуемся `cut`, чтобы извлечь из файла первые два поля (название дистрибутива и версию) и сохранить результат в файл `distros-versions.txt`.

```
[me@linuxbox ~]$ cut -f 1,2 distros-by-date.txt > distros-versions.txt
[me@linuxbox ~]$ head distros-versions.txt
Fedora 10
Ubuntu 8.10
SUSE 11.0
Fedora 9
Ubuntu 8.04
Fedora 8
Ubuntu 7.10
SUSE 10.3
Fedora 7
Ubuntu 7.04
```

Заключительный этап подготовки — извлечь даты релизов и сохранить их в файл `distros-dates.txt`.

```
[me@linuxbox ~]$ cut -f 3 distros-by-date.txt > distros-dates.txt
[me@linuxbox ~]$ head distros-dates.txt
11/25/2008
10/30/2008
06/19/2008
05/13/2008
04/24/2008
11/08/2007
10/18/2007
10/04/2007
05/31/2007
04/19/2007
```

Необходимые части у нас есть. Чтобы завершить процесс, мы воспользуемся `paste`, чтобы поставить столбец с датами перед названиями и версиями дистрибутивов, создав тем самым хронологический список. Это легко сделать с помощью `paste`, расположив его аргументы в нужном порядке.

```
[me@linuxbox ~]$ paste distros-dates.txt distros-versions.txt
11/25/2008 Fedora 10
10/30/2008 Ubuntu 8.10
06/19/2008 SUSE 11.0
05/13/2008 Fedora 9
04/24/2008 Ubuntu 8.04
11/08/2007 Fedora 8
10/18/2007 Ubuntu 7.10
10/04/2007 SUSE 10.3
05/31/2007 Fedora 7
04/19/2007 Ubuntu 7.04
12/07/2006 SUSE 10.2
10/26/2006 Ubuntu 6.10
10/24/2006 Fedora 6
06/01/2006 Ubuntu 6.06
05/11/2006 SUSE 10.1
03/20/2006 Fedora 5
```

join

В некотором смысле `join` похож на `paste`, так как он добавляет столбцы в файл, но делает это уникальным способом. `join` — это операция, обычно связанная с реляционными базами данных,

где данные из нескольких таблиц с общим ключевым полем объединяются для получения нужного результата. Программа join выполняет ту же операцию. Она объединяет данные из нескольких файлов на основе общего ключевого поля.

Чтобы понять, как работает операция join в реляционных базах данных, представим небольшую базу из двух таблиц, в каждой из которых по одной записи. Первая таблица, CUSTOMERS, имеет три поля: номер клиента (CUSTNUM), имя (FNAME) и фамилию (LNAME):

```
CUSTNUM
LNAME=====
FNAME  LNAME=====
=====4681934
=====4681934
John
Smith
```

Вторая таблица называется ORDERS и содержит четыре поля: номер заказа (ORDERNUM), номер клиента (CUSTNUM), количество (QUAN) и наименование товара (ITEM).

```
ORDERNUM
QUAN  ITEM=====
CUSTNUM  QUAN  ITEM=====
====  =====3014953305
=====  =====3014953305
4681934
1
Blue widget
```

Обратите внимание, что обе таблицы имеют общее поле CUSTNUM. Это важно, так как позволяет установить связь между таблицами.

Операция join позволяет объединить поля двух таблиц для получения полезного результата, например, для формирования счета. Используя совпадающие значения в полях CUSTNUM обеих таблиц, операция join может выдать следующее:

```
FNAME
QUAN  ITEM=====
```


LNAME QUAN ITEM=====

=====John

=====John

Smith

1

Blue Widget

Чтобы продемонстрировать программу `join`, нам нужно создать пару файлов с общим ключом. Для этого мы воспользуемся нашим файлом `distros-by-date.txt`. На его основе мы создадим два дополнительных файла. Один из них будет содержать даты выхода (которые станут общим ключом для этой демонстрации) и названия релизов, как показано здесь

```
[me@linuxbox ~]$ cut -f 1,1 distros-by-date.txt > distros-names.txt
[me@linuxbox ~]$ paste distros-dates.txt distros-names.txt > distros-
key-names.txt
[me@linuxbox ~]$ head distros-key-names.txt
11/25/2008   Fedora
10/30/2008   Ubuntu
06/19/2008   SUSE
05/13/2008   Fedora
04/24/2008   Ubuntu
11/08/2007   Fedora
10/18/2007   Ubuntu
10/04/2007   SUSE
05/31/2007   Fedora
04/19/2007   Ubuntu
```

Второй файл содержит даты выпуска и номера версий, как показано ниже:

```
[me@linuxbox ~]$ cut -f 2,2 distros-by-date.txt > distros-vernums.txt
[me@linuxbox ~]$ paste distros-dates.txt distros-vernums.txt > distro
s-key-vernums.txt
[me@linuxbox ~]$ head distros-key-vernums.txt
11/25/2008   10
10/30/2008   8.10
06/19/2008   11.0
05/13/2008   9
04/24/2008   8.04
11/08/2007   8
10/18/2007   7.10
10/04/2007   10.3
05/31/2007   7
04/19/2007   7.04
```

Теперь у нас есть два файла с общим ключом (полем «дата выпуска»). Важно отметить, что для корректной работы `join` файлы должны быть отсортированы по этому полю.

```
[me@linuxbox ~]$ join distros-key-names.txt distros-key-vernums.txt |
head
11/25/2008 Fedora 10
10/30/2008 Ubuntu 8.10
```

```
06/19/2008 SUSE 11.0
05/13/2008 Fedora 9
04/24/2008 Ubuntu 8.04
11/08/2007 Fedora 8
10/18/2007 Ubuntu 7.10
10/04/2007 SUSE 10.3
05/31/2007 Fedora 7
04/19/2007 Ubuntu 7.04
```

Примечание: по умолчанию `join` использует пробельные символы как разделитель полей ввода и один пробел как разделитель полей вывода. Это поведение можно изменить с помощью опций. Подробности см. в `join man page`.

`tac`

Команда `tac` работает как `cat`, но буквально в обратном порядке. Она объединяет файлы в обратной последовательности. Один из способов использования `tac` — изменение порядка лог-файлов. Лог-файлы часто отображаются так:

```
[me@linuxbox ~]$ tail /var/log/backup.log
Wed Aug 14 06:00:32 AM EDT 2025: backup (0.6) of linuxbox started.
Wed Aug 14 06:08:01 AM EDT 2025: backup of linuxbox finished.
Thu Aug 15 07:36:30 AM EDT 2025: backup (0.6) of linuxbox started.
Thu Aug 15 07:45:57 AM EDT 2025: backup of linuxbox finished.
Fri Aug 16 07:37:57 AM EDT 2025: backup (0.6) of linuxbox started.
Fri Aug 16 07:44:11 AM EDT 2025: backup of linuxbox finished.
```

Как видно, лог представлен так, что последняя запись находится внизу. Если мы хотим видеть последнее событие в начале списка, можно сделать следующее:

```
[me@linuxbox ~]$ tail /var/log/backup.log | tac
Fri Aug 16 07:44:11 AM EDT 2025: backup of linuxbox finished.
Fri Aug 16 07:37:57 AM EDT 2025: backup (0.6) of linuxbox started.
Thu Aug 15 07:45:57 AM EDT 2025: backup of linuxbox finished.
Thu Aug 15 07:36:30 AM EDT 2025: backup (0.6) of linuxbox started.
Wed Aug 14 06:08:01 AM EDT 2025: backup of linuxbox finished.
Wed Aug 14 06:00:32 AM EDT 2025: backup (0.6) of linuxbox started.
```

`rev`

Аналогично, команда `rev` переворачивает символы в строке:

```
[me@linuxbox ~]$ echo "This is a test." | rev
.tset a si sihT
```

Мило, но в чем польза? Команду `rev` часто используют вместе с `cut`. Давайте снова рассмотрим наш `backup.log` файл.

Представьте, что мы хотим удалить символ точки в конце каждой строки. Поскольку строки имеют разную длину, нет способа указать `cut` команду для удаления чего-либо именно в конце строки. Здесь на помощь приходит `rev`. Развернув

символы в каждой строке, мы переместим точки в начало, где `cut` сможет легко их удалить:

```
[me@linuxbox ~]$ tail /var/log/backup.log | rev | cut -c 2- | rev
Wed Aug 14 06:00:32 AM EDT 2025: backup (0.6) of linuxbox started
Wed Aug 14 06:08:01 AM EDT 2025: backup of linuxbox finished
Thu Aug 15 07:36:30 AM EDT 2025: backup (0.6) of linuxbox started
Thu Aug 15 07:45:57 AM EDT 2025: backup of linuxbox finished
Fri Aug 16 07:37:57 AM EDT 2025: backup (0.6) of linuxbox started
Fri Aug 16 07:44:11 AM EDT 2025: backup of linuxbox finished
```

В этом примере мы используем `rev`, чтобы развернуть символы в каждой строке, вырезать содержимое строки со второй позиции (символ после точки в развернутой строке) до конца строки, а затем снова развернуть символы, чтобы вернуть их в читаемый вид.

Сравнение текста

Сравнение версий текстовых файлов часто бывает полезным. Для системных администраторов и разработчиков ПО это особенно важно. Например, администратору может потребоваться сравнить текущий конфигурационный файл с предыдущей версией для диагностики проблемы в системе. Точно так же программисту часто нужно видеть, какие изменения вносились в программы с течением времени.

`comm`

Программа `comm` сравнивает два текстовых файла и отображает строки, уникальные для каждого из них, а также общие строки. Для демонстрации мы создадим два почти идентичных текстовых файла с помощью `cat`.

```
[me@linuxbox ~]$ cat > file1.txt
a
b
c
d
[me@linuxbox ~]$ cat > file2.txt
b
c
d
e
```

Далее мы сравним два файла с помощью `comm`:

```
[me@linuxbox ~]$ comm file1.txt file2.txt
a
      b
      c
      d
e
```

comm выводит три столбца. Первый содержит строки, уникальные для первого файла, второй — уникальные для второго, а третий — строки, общие для обоих файлов. comm поддерживает опции вида -n, где n — это 1, 2 или 3. Эти опции позволяют скрыть нужные столбцы. Например, если нам нужны только общие строки, мы скроем первый и второй столбцы.

```
[me@linuxbox ~]$ comm -12 file1.txt file2.txt
b
c
d
```

diff

Как и программа comm, diff служит для поиска различий в файлах. Однако diff — гораздо более сложный инструмент: он поддерживает множество форматов вывода и позволяет обрабатывать сразу целые наборы текстовых файлов. Разработчики часто используют diff для анализа изменений между версиями исходного кода, поэтому инструмент умеет рекурсивно обходить каталоги с кодом, называемые деревьями исходников. Распространенный способ применения diff — создание diff-файлов или патчей, которые используются такими программами, как patch (мы разберем её позже), для преобразования одной версии файла (или файлов) в другую.

Если использовать diff для просмотра файлов из предыдущего примера, мы увидим стандартный стиль вывода: краткое описание различий между двумя файлами.

```
[me@linuxbox ~]$ diff file1.txt file2.txt
1d0
< a
4a4
> e
```

В формате по умолчанию каждой группе изменений предшествует команда изменения в виде операции диапазона, описывающей позиции и типы изменений, необходимых для преобразования первого файла во второй, как показано в таблице 20-4.

Таблица 20-4: diff Команды изменения

Изменение	Описание
r1ar2	Добавить строки из позиции r2 во втором файле в позицию r1 в первом файле.

Изменение	Описание
r1cr2	Замените строки на позиции r1 строками, находящимися на позиции r2 во втором файле.
r1dr2	Удалите строки в первом файле на позиции r1, которые должны были появиться в диапазоне r2 во втором файле

В этом формате диапазон представляет собой список начальной и конечной строк, разделенных запятой. Хотя этот формат является стандартным (в основном для соответствия POSIX и обратной совместимости с традиционными версиями `diff` в Unix), он используется не так часто, как другие необязательные форматы. Два наиболее популярных — контекстный и унифицированный форматы.

При просмотре в контекстном формате (опция `-c`) мы увидим следующее:

```
[me@linuxbox ~]$ diff -c file1.txt file2.txt
*** file1.txt2025-12-23 06:40:13.000000000 -0500
--- file2.txt2025-12-23 06:40:34.000000000 -0500
*****
*** 1,4 ****
- a
- b
- c
- d
--- 1,4 ----
- b
- c
- d
+ e
```

Вывод начинается с имен двух файлов и меток времени. Первый файл отмечен звездочками, а второй — тире. В остальной части списка эти маркеры будут указывать на соответствующие файлы. Далее видны группы изменений, включая стандартное количество строк контекста. В первой группе мы видим следующее, что указывает на строки с 1 по 4 в первом файле:

```
*** 1,4 ***
```

Позже мы увидим следующее: это указывает на строки с 1 по 4 во втором файле.:

```
--- 1,4 ---
```

В границах группы изменений строки начинаются с одного из четырех индикаторов, представленных в таблице 20-5.

Таблица 20-5: *diff* Индикаторы изменения формата контекста

Индикатор	Значение
-----------	----------

Индикатор	Значение
blank	Строка для контекста. Она не указывает на различие между двумя файлами.
-	Удаленная строка. Эта строка есть в первом файле, но нет во втором файле.
+	Добавлена строка. Эта строка появится во втором файле, но не в первом файле.
!	Изменена строка. Будут отображены две версии строки, каждая в своей части группы изменений.

Формат unified похож на формат context, но более лаконичен. Он задается опцией -u.

```
[me@linuxbox ~]$ diff -u file1.txt file2.txt
--- file1.txt2008-12-23 06:40:13.000000000 -0500
+++ file2.txt2008-12-23 06:40:34.000000000 -0500
@@ -1,4 +1,4 @@
-a
 b
 c
 d
+e
```

Главное отличие контекстного формата от унифицированного — отсутствие дублирующихся строк контекста, благодаря чему результат в унифицированном формате короче. В предыдущем примере мы видим временные метки файлов, как и в контекстном формате, а затем строку @@ -1,4 +1,4 @@. Это указывает на строки в первом и втором файлах, описанные в группе изменений. Далее следуют сами строки с тремя строками контекста по умолчанию. Каждая строка начинается с одного из трех символов, перечисленных в таблице 20-6.

Таблица 20-6: *diff* Индикаторы изменений в едином формате

Символ	Значение
blank	Эта строка есть в обоих файлах.
-	Эта строка была удалена из первого файла.
+	Эта строка была добавлена в первый файл.

patch

Программа patch используется для внесения изменений в текстовые файлы. Она принимает вывод от diff и обычно применяется для преобразования файлов старых версий в новые. Рассмотрим известный пример. Ядро Linux разрабатывается

огромной, неформально организованной командой участников, которые постоянно вносят небольшие изменения в исходный код. Ядро Linux состоит из нескольких миллионов строк кода, в то время как изменения, вносимые одним участником за раз, весьма малы. Участнику нет смысла отправлять каждому разработчику всё дерево исходного кода ядра при каждом мелком изменении. Вместо этого отправляется diff-файл. Diff-файл содержит различия между предыдущей версией ядра и новой версией с изменениями участника. Затем получатель использует программу patch, чтобы применить изменения к своему дереву исходного кода. Использование diff/patch дает два важных преимущества.

1. Diff-файл мал по сравнению с полным размером дерева исходного кода.
2. Diff-файл лаконично показывает вносимые изменения, позволяя рецензентам патча быстро их оценить.

Разумеется, diff/patch будет работать с любым текстовым файлом, а не только с исходным кодом. Это так же применимо к конфигурационным файлам или любому другому тексту.

Чтобы подготовить diff-файл для использования с patch, документация GNU (см. раздел «Что почитать» ниже) предлагает использовать diff следующим образом:

```
diff -Naur old_file new_file > diff_file
```

где old_file и new_file — это либо отдельные файлы, либо каталоги с файлами. Опция r поддерживает рекурсию дерева каталогов.

После создания diff-файла его можно применить, чтобы преобразовать старый файл в новый.

```
patch < diff_file
```

Мы продемонстрируем это на нашем тестовом файле.

```
[me@linuxbox ~]$ diff -Naur file1.txt file2.txt > patchfile.txt
[me@linuxbox ~]$ patch < patchfile.txt
patching file file1.txt
[me@linuxbox ~]$ cat file1.txt
b
c
d
e
```

В этом примере мы создали diff-файл с именем patchfile.txt, а затем использовали программу patch, чтобы применить патч.

Примечание: нам не пришлось указывать целевой файл для patch, так как diff-файл (в едином формате) уже содержит имена

файлов в заголовке. После применения патча видно, что `file1.txt` теперь совпадает с `file2.txt`.

`patch` имеет множество опций, а для анализа и редактирования патчей существуют дополнительные утилиты.

Редактирование на лету

Наш опыт работы с текстовыми редакторами был преимущественно интерактивным: мы вручную перемещаем курсор и вводим изменения. Однако существуют и неинтерактивные способы редактирования. Например, можно применить набор изменений к нескольким файлам одной командой.

`tr`

Программа `tr` используется для транслитерации символов. Это можно представить как своего рода поиск и замену на основе символов. Транслитерация — это процесс замены символов одного алфавита на другой. Например, преобразование символов из нижнего регистра в верхний — это тоже транслитерация. С помощью `tr` такое преобразование можно выполнить следующим образом:

```
[me@linuxbox ~]$ echo "lowercase letters" | tr a-z A-Z
LOWERCASE LETTERS
```

Как видно, `tr` работает со стандартным вводом и выводит результат в стандартный вывод. `tr` принимает два аргумента: набор символов для замены и соответствующий набор символов, на которые они заменяются. Наборы символов можно задать тремя способами.

1. Перечисление. Например, `ABCDEFGHIJKLMNOPQRSTUVWXYZ`
2. Диапазон символов. Например, `A-Z`. Примечание: этот метод иногда сталкивается с теми же проблемами, что и другие команды, из-за порядка сортировки в локали, поэтому следует использовать его с осторожностью.
3. POSIX-классы символов. Например, `[:upper:]`.

В большинстве случаев оба набора символов должны быть одинаковой длины; однако первый набор может быть больше второго, если мы хотим им заменить несколько символов одним.

```
[me@linuxbox ~]$ echo "lowercase letters" | tr [:lower:] A
AAAAAAAAA AAAAAAA
```


Помимо транслитерации, `tr` позволяет просто удалять символы из входного потока. Ранее в этой главе мы обсуждали проблему преобразования текстовых файлов MS-DOS в формат Unix. Для этого преобразования необходимо удалить символы возврата каретки в конце каждой строки. С помощью `tr` это можно сделать следующим образом:

```
tr -d '\r' < dos_file > unix_file
```

Здесь `dos_file` — файл для преобразования, а `unix_file` — результат. В такой форме команда использует `escape`-последовательность `\r` для представления символа возврата каретки. Чтобы увидеть полный список последовательностей и классов символов, которые поддерживает `tr`, выполните следующее:

```
[me@linuxbox ~]$ tr --help
```

ROT13: не такой уж и секретный шифр

Одно забавное применение `tr` — кодирование текста методом ROT13. ROT13 — это примитивный вид шифрования, основанный на простом шифре подстановки. Называть ROT13 «шифрованием» — значит преувеличивать; точнее будет «обфускация текста». Его иногда используют, чтобы скрыть потенциально оскорбительный контент. Метод просто сдвигает каждый символ на 13 позиций по алфавиту. Поскольку это ровно половина из 26 возможных символов, повторное применение алгоритма возвращает текст в исходный вид. Используйте следующее для кодирования с помощью `tr`:

```
echo "secret text" | tr a-zA-Z n-za-mN-ZA-M frperg grkg
```

Повторное выполнение той же процедуры дает следующий результат:

```
echo "frperg grkg" | tr a-zA-Z n-za-mN-ZA-M secret text
```

Многие почтовые программы и новостные читатели Usenet поддерживают кодирование ROT13. В Wikipedia есть хорошая статья по этой теме:

<https://en.wikipedia.org/wiki/ROT13>

`tr` может и другой трюк. Используя опцию `-s`, `tr` может «сжимать» (удалять) повторяющиеся символы.

```
[me@linuxbox ~]$ echo "aaabbbccc" | tr -s ab
abccc
```

Здесь представлена строка с повторяющимися символами. Указав `tr` набор «ab», мы удалим повторяющиеся буквы из этого набора, оставив неизменным символ, не входящий в него («c»). Примечание: повторяющиеся символы должны идти подряд. Если они разделены другими знаками, сжатие не работает.

```
[me@linuxbox ~]$ echo "abcbabcb" | tr -s ab  
abcbabcb
```

sed

Название `sed` — это сокращение от `stream editor`. Программа выполняет редактирование текста в потоке, будь то набор указанных файлов или стандартный ввод. `sed` — мощная и довольно сложная программа (о ней написаны целые книги), поэтому здесь мы не будем рассматривать её полностью.

В целом, `sed` работает так: ему передается либо одна команда редактирования (в командной строке), либо имя файла со скриптом, содержащим несколько команд, после чего он выполняет их для каждой строки в потоке текста. Вот простой пример работы `sed`:

```
[me@linuxbox ~]$ echo "front" | sed 's/front/back/'  
back
```

В этом примере мы создаем поток текста из одного слова с помощью `echo` и передаем его через конвейер в `sed`. `sed`, в свою очередь, выполняет инструкцию `s/front/back/` над текстом в потоке и в результате выдает «back». Эту команду также можно узнать по сходству с командой «замены» (поиск и замена) в `vi`.

Команды в `sed` начинаются с одной буквы. В предыдущем примере команда замены представлена буквой `s`, за которой следуют строки поиска и замены, разделенные символом косой черты в качестве разделителя. Выбор символа разделителя произволен. По традиции часто используется косая черта, но `sed` примет в качестве разделителя любой символ, следующий сразу за командой. Мы можем выполнить ту же команду следующим образом:

```
[me@linuxbox ~]$ echo "front" | sed 's_ front _back_'  
back
```

Если использовать символ подчеркивания сразу после команды, он станет разделителем. Как мы увидим далее, возможность задавать разделитель помогает сделать команды более читаемыми.

Перед большинством команд в `sed` можно указать адрес, определяющий, какие строки входного потока будут изменены. Если адрес не указан, команда редактирования применяется к каждой строке входного потока. Простейший вид адреса — номер строки. Мы можем добавить его к нашему примеру.

```
[me@linuxbox ~]$ echo "front" | sed '1s/front/back/'
back
```

Добавление адреса 1 к команде заставляет подстановку выполняться в первой строке нашего однострочного потока ввода. Если указать другое число, правка не будет произведена, так как в потоке ввода отсутствует строка 2.

```
[me@linuxbox ~]$ echo "front" | sed '2s/front/back/'
front
```

Адреса можно записывать по-разному. В таблице 20-7 приведены наиболее распространенные варианты.

Таблица 20-7: `sed` Обозначение адреса ие, что ьно

Адрес	Описание
n	Номер строки, где n — положительное целое число.
\$	Последняя строка.
/regexp/	Строки, соответствующие базовому регулярному выражению POSIX. Обратите вниман регулярное выражение ограничено символами косой черты. Дополнител регулярное выражение может быть ограничено другим символом, если указать выражение через \cregexрс, где с — альтернативный символ.
addr1,addr2	Диапазон строк от addr1 до addr2 включительно. Адреса могут быть любыми из перечисленных ранее форматов адресов.
first~step	Сначала выбирается строка по номеру, затем каждая последующая строка с заданным шагом. Например, 1~2 означает каждую нечетную строку, а 5~5 относится к пятой строке и каждой пятой строке далее.
addr1,+n	Найти addr1 и следующие n строк.
addr!	Найти все строки, кроме addr, который может иметь любой из видов упомянутый ранее.

Мы продемонстрируем различные виды адресов на примере файла `distros.txt`, упомянутого ранее в этой главе. Сначала рассмотрим диапазон номеров строк:

```
[me@linuxbox ~]$ sed -n '1,5p' distros.txt
SUSE      10.2      12/07/2006
Fedora    10         11/25/2008
SUSE      11.0       06/19/2008
Ubuntu    8.04       04/24/2008
Fedora     8         11/08/2007
```

В этом примере мы выводим диапазон строк, начиная с 1-й и заканчивая 5-й. Для этого используется команда `p`, которая просто выводит подходящую строку. Чтобы это сработало, необходимо добавить опцию `-n` (опция «no auto-print»), чтобы `sed` не выводил каждую строку, как это происходит по умолчанию.

Далее мы попробуем регулярное выражение.

```
[me@linuxbox ~]$ sed -n '/SUSE/p' distros.txt
SUSE      10.2      12/07/2006
SUSE      11.0       06/19/2008
SUSE      10.3       10/04/2007
SUSE      10.1       05/11/2006
```

Используя разделенное косыми чертами регулярное выражение `/SUSE/`, мы можем выделить строки, содержащие его, почти так же, как `grep`.

Наконец, мы попробуем отрицание, добавив восклицательный знак (!) к адресу `s`.

```
[me@linuxbox ~]$ sed -n '/SUSE/!p' distros.txt
Fedora    10         11/25/2008
Ubuntu    8.04       04/24/2008
Fedora     8         11/08/2007
Ubuntu    6.10       10/26/2006
Fedora     7         05/31/2007
Ubuntu    7.10       10/18/2007
Ubuntu    7.04       04/19/2007
Fedora     6         10/24/2006
Fedora     9         05/13/2008
Ubuntu    6.06       06/01/2006
Ubuntu    8.10       10/30/2008
Fedora     5         03/20/2006
```

Вот ожидаемый результат: все строки файла, кроме тех, что соответствуют регулярному выражению.

До сих пор мы рассматривали две команды `sed` редактирования: `s` и `p`. В таблице 20-8 приведен более полный список основных команд редактирования.

Таблица 20-8: `sed` Основные команды редактирования

Команда	Описание
---------	----------

Команда	Описание
=	Выводит номер текущей строки.
a	Добавляет текст после текущей строки.
d	Удалить текущую строку.
i	Вставить текст перед текущей строкой.
p	Вывести текущую строку. По умолчанию sed выводит каж строку и редактирует только те строки, которые соответств заданному адресу в файле. По умолчанию поведение можно изменить, указав -n опцию.
q	Выход из sed без обработки остальных строк. Если опция -n не указана, выводится текущая строка.
Q	Выход из sed без обработки остальных строк.
s/regexp/replacement/	Заменить содержимое на replacement везде, где regexp найдено. замена может включать специальный символ &, эквивалентный тексту найденному regexp. Кроме того, замена может включать последовательности от \1 до \9, которые являют содержимым соответствующих подвыражений в regexp. Подробнее об этом см. в разделе обратных ссылок ниже. После завершающего слеша замены можно указать необязательный флаг, чтобы изменить поведение команды s.
y/set1/set2	Выполнить транслитерацию путем преобразования симво из set1 в соответствующие символы из set2. Примечание: в отличие от tr, sed требует, чтобы оба набо той же длины.

Команда `s` — самая часто используемая команда редактирования. Мы продемонстрируем лишь часть её возможностей, отредактировав файл `distros.txt`. Ранее мы обсуждали, что поле даты в `distros.txt` представлено не в «компьютерном» формате. Хотя дата записана как ММ/ДД/ГГГГ, для удобства сортировки лучше использовать формат ГГГГ-ММ-ДД. Вносить такие изменения в файл вручную долго и чревато ошибками, но с помощью `sed` это можно сделать в один шаг.

```
[me@linuxbox ~]$ sed 's/\([0-9]\{2\}\)\(/\([0-9]\{2\}\)\(/\([0-9]\{4\}\)
)\$/\3-\1-\2/' distros.txt
SUSE      10.2      2006-12-07
Fedora    10          2008-11-25
SUSE      11.0       2008-06-19
Ubuntu    8.04        2008-04-24
Fedora     8           2007-11-08
SUSE      10.3       2007-10-04
Ubuntu    6.10        2006-10-26
Fedora     7           2007-05-31
Ubuntu    7.10        2007-10-18
Ubuntu    7.04        2007-04-19
SUSE      10.1       2006-05-11
Fedora     6           2006-10-24
Fedora     9           2008-05-13
Ubuntu    6.06        2006-06-01
Ubuntu    8.10        2008-10-30
Fedora     5           2006-03-20
```

Ого! Какая уродливая команда. Но она работает. Всего за один шаг мы изменили формат даты в файле. Это отличный пример того, почему регулярные выражения иногда в шутку называют «языком только для записи». Мы можем их написать, но не всегда можем прочитать. Прежде чем мы в ужасе сбежим от этой команды, давайте разберем, как она устроена. Во-первых, мы знаем, что команда будет иметь следующую базовую структуру.

```
sed 's/regexp/replacement/' distros.txt
```

Следующий шаг — подобрать регулярное выражение, которое выделит дату. Поскольку она представлена в формате ММ/ДД/YYYY и находится в конце строки, можно использовать такое выражение:

```
[0-9]{2}/[0-9]{2}/[0-9]{4}$
```

Это соответствует двум цифрам, косой черте, двум цифрам, косой черте, четырем цифрам и концу строки. С регулярными выражениями разобрались, но как насчет замены? Для этого нужно ввести новую функцию, которая встречается в некоторых приложениях, использующих BRE. Эта функция называется обратными ссылками и работает так: если в строке замены встречается последовательность `\n`, где `n` — число от 1 до 9, она

будет ссылаться на соответствующее подвыражение в предыдущем регулярном выражении. Чтобы создать подвыражения, просто заключите их в скобки следующим образом:

```
([0-9]{2})/([0-9]{2})/([0-9]{4})$
```

Теперь у нас есть три подвыражения. Первое содержит месяц, второе — число, а третье — год. Теперь мы можем сформировать строку замены следующим образом:

```
\3-\1-\2
```

Так мы получаем год, тире, месяц, тире и день.

Теперь наша команда выглядит так:

```
sed 's/([0-9]{2})/([0-9]{2})/([0-9]{4})$/\3-\1-\2/' distros.txt
```

У нас осталось две проблемы. Первая — лишние косые черты в регулярном выражении запутают sed, когда он попытается интерпретировать команду s. Вторая — поскольку sed по умолчанию принимает только базовые регулярные выражения, некоторые символы в нашем выражении будут восприняты как обычный текст, а не как метасимволы. Мы можем решить обе проблемы, используя обратные косые черты для экранирования проблемных символов.

```
sed 's/\([0-9]\{2\}\)\(/\([0-9]\{2\}\)\(\([0-9]\{4\}\)\$/\3-\1-\2/' distros.txt
```

Вот и всё!

Еще одна особенность команды s — использование необязательных флагов после строки замены. Самый важный из них — флаг g, который указывает sed применять поиск и замену ко всей строке, а не только к первому вхождению, как это происходит по умолчанию. Вот пример:

```
[me@linuxbox ~]$ echo "aaabbbccc" | sed 's/b/B/'  
aaaBbbccc
```

Мы видим, что замена выполнена только для первого вхождения буквы b, а остальные остались без изменений. Добавив флаг g, мы сможем заменить все вхождения.

```
[me@linuxbox ~]$ echo "aaabbbccc" | sed 's/b/B/g'  
aaaBBBccc
```

До сих пор мы вводили через командную строку только sed одиночные команды. Однако с помощью -f можно создавать

более сложные команды в файле сценария. Для демонстрации мы воспользуемся `sed` с нашим `distros.txt` файлом, чтобы составить отчет. В отчете будет заголовок, даты изменений и все названия дистрибутивов в верхнем регистре. Для этого нам нужно написать сценарий, поэтому откроем текстовый редактор и введем следующее:

```
# sed script to produce Linux distributions report
1 i\
\
Linux Distributions Report\
s/\([0-9]\{2\}\)\(\([0-9]\{2\}\)\)\(\([0-9]\{4\}\)\)$/\3-\1-\2/
y/abcdefghijklmnopqrstuvwxyz/ABCDEFGHIJKLMNOPQRSTUVWXYZ/
```

Мы сохраним наш `sed` сценарий как `distros.sed` и запустим его следующим образом:

```
[me@linuxbox ~]$ sed -f distros.sed distros.txt
Linux Distributions Report
SUSE      10.2      2006-12-07
FEDORA    10        2008-11-25
SUSE      11.0      2008-06-19
UBUNTU    8.04       2008-04-24
FEDORA    8          2007-11-08
SUSE      10.3      2007-10-04
UBUNTU    6.10       2006-10-26
FEDORA    7          2007-05-31
UBUNTU    7.10       2007-10-18
UBUNTU    7.04       2007-04-19
SUSE      10.1      2006-05-11
FEDORA    6          2006-10-24
FEDORA    9          2008-05-13
UBUNTU    6.06       2006-06-01
UBUNTU    8.10       2008-10-30
FEDORA    5          2006-03-20
```

Как видим, наш скрипт выдает нужный результат, но как он это делает? Давайте еще раз изучим скрипт. Мы воспользуемся `cat`, чтобы пронумеровать строки.

```
[me@linuxbox ~]$ cat -n distros.sed
1  # sed script to produce Linux distributions report
2
3  1 i\
4  \
5  Linux Distributions Report\
6
7  s/\([0-9]\{2\}\)\(\([0-9]\{2\}\)\)\(\([0-9]\{4\}\)\)$/\3-\1-\2/
8  y/abcdefghijklmnopqrstuvwxyz/ABCDEFGHIJKLMNOPQRSTUVWXYZ/
```

Первая строка нашего скрипта — это комментарий. Как и во многих конфигурационных файлах и языках программирования в системах Linux, комментарии начинаются с символа `#`, за которым следует текст, понятный человеку. Комментарии можно размещать где угодно в скрипте (но не внутри самих команд);

они помогают людям идентифицировать скрипт и/или поддерживать его.

Строка 2 — пустая. Как и комментарии, пустые строки можно добавлять для улучшения читаемости.

Многие `sed` команды поддерживают адреса строк. Они используются для указания строк входных данных, к которым будут применены действия. Адреса строк могут быть представлены как одиночными номерами строк, диапазонами номеров строк или специальным номером `$`, обозначающим последнюю строку ввода.

Строки 3, 4, 5 и 6 содержат текст, который будет вставлен по адресу 1 (первая строка ввода). За `i` командой следует последовательность из обратного слеша и символа возврата каретки, образующая экранированный возврат каретки, или так называемый символ продолжения строки. Эта последовательность, которую можно использовать во многих случаях, включая скрипты оболочки, позволяет встроить возврат каретки в поток текста, не сообщая интерпретатору (в данном случае `sed`), что достигнут конец строки. Команды `i`, `a` (которая добавляет текст в конец, а не вставляет его) и `c` (которая заменяет текст) позволяют использовать несколько строк текста, если каждая строка, кроме последней, заканчивается символом продолжения строки. Шестая строка нашего скрипта на самом деле является концом вставленного текста и заканчивается обычным возвратом каретки, а не символом продолжения строки, что сигнализирует о завершении команды `i`.

Примечание: символ продолжения строки образуется обратным слешем, за которым сразу следует возврат каретки. Промежуточные пробелы недопустимы.

Строка 7 — это команда поиска и замены. Поскольку перед ней нет адреса, действие применяется к каждой строке входного потока.

Строка 8 выполняет транслитерацию строчных букв в заглавные. Примечание: в отличие от `tr [a-z] y`, команда `y` в `sed` не поддерживает диапазоны символов (например, `1-5`), а также классы символов POSIX. Опять же, так как перед командой нет адреса, она применяется к каждой строке входного потока.

Те, кому нравится `sed`, также любят...

`sed` — мощная программа, способная выполнять довольно сложные задачи по редактированию текстовых потоков. Чаще всего её используют для простых однострочных задач, а не для длинных скриптов. Для более масштабных задач многие пользователи предпочитают другие инструменты. Самые популярные из них — `awk` и `perl`. Они выходят за рамки простых утилит, описанных в этой книге, и относятся к области полноценных языков программирования. `perl`, в частности, часто используется вместо `shell`-скриптов для многих задач системного управления и администрирования, а также является популярным средством веб-разработки. `awk` более специализированна. Её сильная сторона — умение манипулировать табличными данными. Она напоминает `sed` тем, что программы `awk` обычно обрабатывают текстовые файлы построчно, используя схему, схожую с концепцией `sed` «адрес, за которым следует действие». Хотя `awk` и `perl` не входят в рамки этой книги, их изучение будет полезным навыком для пользователя командной строки Linux.

aspell

Последний инструмент, который мы рассмотрим, — это `aspell`, интерактивное средство проверки орфографии. Программа `aspell` является преемником более ранней программы `ispell` и в большинстве случаев может использоваться как её прямая замена. Хотя `aspell` чаще всего используется другими программами, требующими проверки орфографии, её также можно эффективно применять как самостоятельный инструмент из командной строки. Она способна интеллектуально проверять различные типы текстовых файлов, включая HTML-документы, программы на C/C++, электронные письма и другие виды специализированных текстов. Не все дистрибутивы Linux включают `aspell` по умолчанию, поэтому его может потребоваться установить.

Для проверки орфографии текстового файла с обычным текстом можно использовать такую команду:

```
aspell check textfile
```

где `textfile` — имя проверяемого файла. В качестве примера создадим простой текстовый файл `foo.txt` с намеренными орфографическими ошибками.

```
[me@linuxbox ~]$ cat > foo.txt
The quick brown fox jimps over the laxy dog.
```

Теперь проверим файл с помощью `aspell`:

```
[me@linuxbox ~]$ aspell check foo.txt
```

Поскольку `aspell` в режиме проверки интерактивен, мы увидим следующее:

```
The quick brown fox jumps over the laxy dog.
1) jumps                6) limps
2) imps                 7) pimps
3) gimps                8) wimps
4) jump's               9) comps
5) Jim's                0) gimp's
i) Ignore               I) Ignore all
r) Replace               R) Replace all
a) Add                  l) Add Lower
b) Abort                x) Exit
?
```

В верхней части экрана отображается текст с подсвеченным словом, в котором есть подозрение на ошибку. Посередине — 10 вариантов исправления (от нуля до девяти), а ниже — список других действий. В самом низу находится приглашение, ожидающее нашего выбора.

Если нажать клавишу 1, `aspell` заменит ошибочное слово на «jumps» и перейдет к следующему слову с ошибкой — `laxy`. Если выбрать вариант замены `lazy`, `aspell` заменит его и завершит работу. После того как `aspell` закончит, мы сможем проверить файл и увидеть, что ошибки исправлены:

```
[me@linuxbox ~]$ cat foo.txt
The quick brown fox jumps over the lazy dog.
```

Если не указано иное через опцию командной строки `--dont-backup`, `aspell` создает резервную копию оригинала, добавляя расширение `.bak` к имени файла.

Чтобы продемонстрировать наши навыки `sed` редактирования, мы вернем орфографические ошибки на место, чтобы файл можно было использовать снова.

```
[me@linuxbox ~]$ sed -i 's/lazy/laxy/; s/jumps/jimps/' foo.txt
```

Опция `sed -i` приказывает `sed` редактировать файл «на месте»: вместо вывода результата в стандартный вывод, программа перезапишет файл с внесенными изменениями. Мы также увидим, что в одной строке можно указать несколько команд редактирования, разделяя их точкой с запятой.

Далее мы рассмотрим, как `aspell` справляется с различными типами текстовых файлов. Используя текстовый редактор вроде

vim (смелые могут попробовать sed), мы добавим в наш файл HTML-разметку.

```
<html>
  <head>
    <title>Mispelled HTML file</title>
  </head>
  <body>
    <p>The quick brown fox jimps over the laxy dog.</p>
  </body>
</html>
```

Теперь, если мы попытаемся проверить орфографию в измененном файле, возникнет проблема. Если сделать это так:

```
[me@linuxbox ~]$ aspell check foo.txt
```

мы получим следующее:

```
<html>
  <head>
    <title>Mispelled HTML file</title>
  </head>
  <body>
    <p>The quick brown fox jimps over the laxy dog.</p>
  </body>
</html>
1) HTML
2) ht ml
3) ht-ml
i) Ignore
r) Replace
a) Add
b) Abort
?
4) Hamel
5) Hamil
6) hotel
I) Ignore all
R) Replace all
l) Add Lower
x) Exit
```

aspell увидит содержимое HTML-тегов как опечатки. Эту проблему можно решить, добавив опцию -H (HTML) проверки, вот так:

```
[me@linuxbox ~]$ aspell -H check foo.txt
```

результатом будет следующее:

```
<html>
  <head>
    <title>Mispelled HTML file</title>
  </head>
  <body>
    <p>The quick brown fox jimps over the laxy dog.</p>
  </body>
</html>
1) Mi spelled
2) Mi-spelled
3) Misspelled
4) Dispelled
5) Spelled
i) Ignore
r) Replace
a) Add
6) Misapplied
7) Miscalled
8) Respelled
9) Misspell
0) Misled
I) Ignore all
R) Replace all
l) Add Lower
```

b) Abort
?

x) Exit

HTML игнорируется, и проверяются только текстовые части файла. В этом режиме содержимое HTML-тегов не проверяется на наличие опечаток. Однако содержимое тегов ALT, которые требуют проверки, в этом режиме проверяется.

Примечание: по умолчанию `aspell` будет игнорировать URL и адреса электронной почты в тексте. Это поведение можно изменить с помощью параметров командной строки. Также можно указать, какие теги разметки следует проверять или пропускать. Подробности см. в `aspell man page`.

Итоги

В этой главе мы рассмотрели несколько из множества инструментов командной строки для работы с текстом. В следующей главе мы изучим еще ряд инструментов. Возможно, не сразу станет ясно, как и зачем использовать некоторые из них в повседневной работе, хотя мы и постарались привести практические примеры. В последующих главах мы увидим, что эти инструменты составляют основу набора средств для решения множества практических задач. Это станет особенно очевидным при изучении написания сценариев оболочки, где эти инструменты по-настоящему раскроют свой потенциал.

Что почитать

На сайте проекта GNU представлено множество онлайн-руководств по инструментам, описанным в этой главе.

- Из пакета Coreutils:

<https://www.gnu.org/software/coreutils/manual/coreutils.html#Output-of-entire-files>

<https://www.gnu.org/software/coreutils/manual/coreutils.html#Operating-on-sorted-files>

<https://www.gnu.org/software/coreutils/manual/coreutils.html#Operating-on-fields>

<https://www.gnu.org/software/coreutils/manual/coreutils.html#Operating-on-characters>

- Из пакета Diffutils:

https://www.gnu.org/software/diffutils/manual/html_mono/diff.html

- sed: <https://www.gnu.org/software/sed/manual/sed.html>

- aspell: <https://aspell.net/man-html/index.html>
- Существует множество других онлайн-ресурсов по sed, в частности: <https://www.grymoire.com/Unix/Sed.html>
<https://www.pement.org/sed/sed1line.txt>
- Также попробуйте поискать в Google «sed one liners» или «sed cheat sheets»

Дополнительные задания

Стоит изучить еще несколько интересных команд для манипуляции текстом. Среди них: `split` (разделение файлов на части), `csplit` (разделение файлов на части с учетом контекста) и `sdiff` (построчное сравнение различий в файлах).

21 — Форматирование вывода

В этой главе мы продолжим изучение инструментов для работы с текстом, сосредоточившись на программах, которые форматируют вывод, а не изменяют сам текст. Эти инструменты часто используются для подготовки текста к печати — этой теме мы посвятим следующую главу. В данной главе мы рассмотрим следующие программы:

- `nl` — Нумерация строк
- `fold` — Перенос строк заданной длины
- `fmt` — Простой текстовый форматировщик
- `pr` — Подготовка текста к печати
- `printf` — форматирование и вывод данных
- `groff` — система форматирования документов

Простые инструменты форматирования

Сначала мы рассмотрим простые инструменты форматирования. В основном это узкоспециализированные и не слишком сложные программы, но их можно использовать для небольших задач, а также в составе конвейеров и скриптов.

`nl` — нумерация строк

Программа `nl` — довольно специфический инструмент для выполнения простой задачи. Она нумерует строки. В самом простом виде она напоминает `cat -n`.

```
[me@linuxbox ~]$ nl distros.txt | head
 1  SUSE      10.2      12/07/2006
 2  Fedora    10        11/25/2008
 3  SUSE      11.0       06/19/2008
 4  Ubuntu    8.04       04/24/2008
 5  Fedora    8          11/08/2007
 6  SUSE      10.3       10/04/2007
 7  Ubuntu    6.10       10/26/2006
 8  Fedora    7          05/31/2007
 9  Ubuntu    7.10       10/18/2007
10  Ubuntu    7.04       04/19/2007
```

Как и `cat`, `nl` может принимать либо несколько файлов в качестве аргументов командной строки, либо стандартный ввод. Однако `nl` имеет ряд опций и поддерживает примитивную разметку для более сложных видов нумерации.

`nl` поддерживает концепцию «логических страниц» при нумерации. Это позволяет `nl` сбрасывать (начинать заново)

числовую последовательность. С помощью опций можно задать начальное число и, в определенных пределах, его формат. Логическая страница далее делится на заголовок, тело и подвал. Внутри каждой из этих секций нумерация строк может быть сброшена и/или ей может быть назначен другой стиль. Если `nl` передано несколько файлов, он обрабатывает их как единый поток текста. Секции в потоке текста обозначаются наличием довольно странной разметки, добавленной к тексту, как описано в таблице 21-1.

Таблица 21-1: Разметка `nl`

Разметка	Значение
<code>\:\:\:</code>	Начало заголовка логической страницы
<code>\:\:</code>	Начало основного содержимого страницы
<code>\:</code>	Начало нижнего колонтитула страницы

Каждый элемент разметки, перечисленный в таблице 21-1, должен располагаться на отдельной строке. После обработки элемента разметки `nl` удаляет его из потока текста.

В таблице 21-2 перечислены основные параметры для `nl`.

Таблица 21-2: Общие `nl options`

Option	Значение
<code>-b style</code>	Установите стиль нумерации строк, выбрав один из следующих вариантов: <code>a</code> = Нумеровать все строки <code>t</code> = Нумеровать только непустые строки (по умолчанию). <code>n</code> = Без нумерации <code>pregexp</code> = Нумеровать только строки, соответствующие базовому регулярному выражению <code>regexpr</code> .
<code>-f style</code>	Установите стиль нумерации колонтитула. По умолчанию — <code>n</code> (нет).
<code>-h style</code>	Установить стиль нумерации заголовков. По умолчанию — <code>n</code> (нет).
<code>-i number</code>	Установить шаг нумерации страниц. По умолчанию — один.

Option	Значение
-n format	Устанавливает формат нумерации «format», где format — один из следующих: ln = Выравнивание по левому краю, без ведущих нулей. rn = Выравнивание по правому краю, без ведущих нулей. Это значение по умолчанию. rz = Выравнивание по правому краю, с ведущими нулями.
-p	Не сбрасывать нумерацию страниц в начале каждой логической страницы.
-s string	Добавить строку в конец каждого номера строки для создания разделителя. По умолчанию используется символ табуляции.
-v number	Установить номер первой строки каждой логической страницы. По умолчанию это one.
-w width	Установить ширину поля номеров строк. По умолчанию — 6.

Признаем, нам вряд ли придется часто нумеровать строки, но с помощью `n1` можно изучить комбинацию нескольких инструментов для выполнения сложных задач. Опираясь на наработки предыдущей главы, мы составим отчет по дистрибутивам Linux. Поскольку мы будем использовать `n1`, полезно будет добавить разметку заголовка, тела и подвала. Для этого мы дополним скрипт `sed` из предыдущей главы. С помощью текстового редактора изменим скрипт следующим образом и сохраним его как `distros-n1.sed`:

```
# sed script to produce Linux distributions report
1 i\
\\: \\: \\: \
\
Linux Distributions Report\
\
Name          Ver. Released\
----          -
\\: \\:
s/\([0-9]\{2\}\)\(\([0-9]\{2\}\)\)\(\([0-9]\{4\}\)\)$/\3-\1-\2/
$ a\
\\: \
\
End Of Report
```

Теперь скрипт вставляет `n1` разметку логической страницы и добавляет в конце отчета колонтитул. Примечание: нам пришлось удвоить обратные косые черты в разметке, так как `sed` обычно интерпретирует их как символ экранирования.

Затем мы создадим улучшенный отчет, объединив `sort`, `sed` и `nl`.

```
[me@linuxbox ~]$ sort -k 1,1 -k 2n distros.txt | sed -f distros-nl.sed | nl
```

	Linux	Distributions	Report
	Name	Ver.	Released
	----	-----	-----
1	Fedora	5	2006-03-20
2	Fedora	6	2006-10-24
3	Fedora	7	2007-05-31
4	Fedora	8	2007-11-08
5	Fedora	9	2008-05-13
6	Fedora	10	2008-11-25
7	SUSE	10.1	2006-05-11
8	SUSE	10.2	2006-12-07
9	SUSE	10.3	2007-10-04
10	SUSE	11.0	2008-06-19
11	Ubuntu	6.06	2006-06-01
12	Ubuntu	6.10	2006-10-26
13	Ubuntu	7.04	2007-04-19
14	Ubuntu	7.10	2007-10-18
15	Ubuntu	8.04	2008-04-24
16	Ubuntu	8.10	2008-10-30
	End Of Report		

Наш отчет — результат работы конвейера команд. Сначала мы сортируем список по названию дистрибутива и версии (поля 1 и 2), затем обрабатываем результат с помощью `sed`, добавляя заголовок отчета (включая разметку логической страницы для `nl`) и колонтитул. Наконец, мы обрабатываем результат с помощью `nl`, которая по умолчанию нумерует только те строки текстового потока, что относятся к основной части логической страницы.

Мы можем повторять команду и экспериментировать с разными опциями для `nl`. Некоторые интере- являются

```
nl -n rz
```

и следующие:

```
nl -w 3 -s ' '
```

`fold` — Перенос строк заданной длины

Перенос (folding) — это процесс разбиения строк текста на заданной ширине. Как и другие наши команды, `fold` принимает один или несколько текстовых файлов либо стандартный ввод. Если мы отправим `fold` простой поток текста, можно увидеть, как она работает.

```
[me@linuxbox ~]$ echo "The quick brown fox jumps over the lazy dog." | fold -w 12
The quick br
own fox jump
```

```
s over the l  
azy dog.
```

Здесь мы видим `fold` в действии. Текст, переданный командой `echo`, разбивается на сегменты, указанные опцией `-w`. В этом примере мы задаем ширину строки в 12 символов. Если ширина не указана, по умолчанию используется 80 символов. Обратите внимание, что строки разбиваются без учета границ слов. Использование опции `-s` заставит `fold` разрывать строку на последнем доступном пробеле перед достижением заданной ширины.

```
[me@linuxbox ~]$ echo "The quick brown fox jumps over the lazy dog."  
| fold -w 12 -s  
The quick  
brown fox  
jumps over  
the lazy  
dog.
```

`fmt` — Простой текстовый форматировщик

Программа `fmt` также выполняет перенос строк и многое другое. Она принимает файлы или стандартный ввод, форматирова абзацы в потоке текста. По сути, она заполняет и объединяет строки, сохраняя пустые строки и отступы.

Для демонстрации нам понадобится текст. Давайте возьмем его из страницы `fmt info`.

```
`fmt' reads from the specified FILE arguments (or standard input  
if none are given), and writes to standard output.  
By default, blank lines, spaces between words, and indentation are  
preserved in the output; successive input lines with different  
indentation are not joined; tabs are expanded on input and introduced  
on output.  
`fmt' prefers breaking lines at the end of a sentence, and tries  
to avoid line breaks after the first word of a sentence or before the  
last word of a sentence. A "sentence break" is defined as either the  
end of a paragraph or a word ending in any of `?.!', followed by two  
spaces or end of line, ignoring any intervening parentheses or  
quotes. Like TeX, `fmt' reads entire "paragraphs" before choosing  
line breaks; the algorithm is a variant of that given by Donald E.  
Knuth and Michael F. Plass in "Breaking Paragraphs Into Lines",  
`Software--Practice & Experience' 11, 11 (November 1981), 1119-1184.
```

Мы скопируем этот текст в текстовый редактор и сохраним файл как `fmt-info.txt`. Теперь предположим, что мы хотим переформатировать текст так, чтобы ширина колонки составляла 50 символов. Это можно сделать, обработав файл с помощью `fmt` и опции `-w`.

```
[me@linuxbox ~]$ fmt -w 50 fmt-info.txt | head  
`fmt' reads from the specified FILE arguments  
(or standard input if
```

```
none are given), and writes to standard output.  
By default, blank lines, spaces between words,  
and indentation are  
preserved in the output; successive input lines  
with different indentation are not joined; tabs  
are expanded on input and introduced on output.
```

Что ж, результат довольно странный. Возможно, нам стоит прочитать этот текст, так как он объясняет происходящее.

По умолчанию в выводе сохраняются пустые строки, пробелы между словами и отступы; идущие подряд строки с разными отступами не объединяются; при чтении табуляция разворачивается, а при выводе — вставляется.

Таким образом, `fmt` сохраняет отступ первой строки. К счастью, `fmt` предоставляет опцию для исправления этого.

```
[me@linuxbox ~]$ fmt -cw 50 fmt-info.txt  
'fmt' reads from the specified FILE arguments  
(or standard input if none are given), and writes  
to standard output.  
By default, blank lines, spaces between words,  
and indentation are preserved in the output;  
successive input lines with different indentation  
are not joined; tabs are expanded on input and  
introduced on output.  
'fmt' prefers breaking lines at the end of a  
sentence, and tries to avoid line breaks after  
the first word of a sentence or before the  
last word of a sentence. A "sentence break"  
is defined as either the end of a paragraph  
or a word ending in any of `?.!`, followed  
by two spaces or end of line, ignoring any  
intervening parentheses or quotes. Like TeX,  
'fmt' reads entire "paragraphs" before choosing  
line breaks; the algorithm is a variant of  
that given by Donald E. Knuth and Michael F.  
Plass in "Breaking Paragraphs Into Lines",  
'Software-Practice & Experience' 11, 11  
(November 1981), 1119-1184.
```

Так гораздо лучше. Добавив опцию `-c`, мы получили нужный результат.

`fmt` имеет несколько интересных опций, описанных в таблице 21-3.

Таблица 21-3: Опции `fmt`

Опция	Описание
<code>-c</code>	Работать в режиме полей. Это сохраняет отступы первые две строки абзаца. Последующие строки выравниваются по отступу второй строки.

Опция	Описание
-p string	Форматируются только строки, начинающиеся с указанной строки-префикса. форматирования содержимое string добавляется в качестве префикса к каждой измененной строке. Этот параметр можно использовать для форматирования текста в исходном коде комментариев. Например, любой язык программирования или конфигурационный файл, использующий символ «#» для обозначения коммен можно отформатировать, указав -p '#', чтобы форматировались только комментарии. См. пример ниже.
-s	Режим только разделения. В этом режиме строки будут разделяться только для того, соответствовать заданной ширине столбца. Короткие строки не будут объединены. Этот режим полезен при форматировании текста, например кода, когда объединение строк нежелательно.
-u	Обеспечивает единообразные отступы. Это применит традиционный «печатный стиль» форматирования к тексту. Это означает один пробел между словами и два пробела между предложениями. Этот режим полезен для удаления «выравнивания», то есть текста, дополненного пробелами пробелы для выравнивания по левому и правому краям.
-w width	Форматирует текст так, чтобы он уместился в колонку шириной в символов. Примечание: fmt на самом деле форматирует строки чуть короче указанной ширины для обеспечения баланса строк.

Особенно интересна опция -p. С её помощью можно форматировать выбранные части файла, если строки для форматирования начинаются с одной и той же последовательности символов. Многие языки программирования используют знак решётки (#) для обозначения начала комментария, поэтому их можно форматировать с помощью этой опции. Давайте создадим файл, имитирующий программу с комментариями.

```
[me@linuxbox ~]$ cat > fmt-code.txt
# This file contains code with comments.
```

```
# This line is a comment.  
# Followed by another comment line.  
# And another.  
This, on the other hand, is a line of code.  
And another line of code.  
And another.
```

Наш пример содержит комментарии, начинающиеся со строки «#» (символ # и пробел), и строки «кода», которые начинаются иначе. Теперь, используя `fmt`, мы можем отформатировать комментарии, не затрагивая код.

```
[me@linuxbox ~]$ fmt -w 50 -p '# ' fmt-code.txt  
# This file contains code with comments.  
# This line is a comment. Followed by another  
# comment line. And another.  
This, on the other hand, is a line of code.  
And another line of code.  
And another.
```

Заметьте, что соседние строки комментариев объединяются, тогда как пустые строки и строки, не начинающиеся с указанного префикса, сохраняются.

pr — Форматирование текста для печати

Программа `pr` используется для постраничного вывода текста. При печати часто требуется разделять страницы несколькими пустыми строками, а также задавать верхнее и нижнее поля для каждой страницы. Кроме того, эти пустые строки можно использовать для вставки колонтитулов.

Мы продемонстрируем `pr`, оформив наш `distros.txt` файл в виде серии коротких страниц (показаны только первые две).

```
[me@linuxbox ~]$ pr -l 15 -w 65 distros.txt  
2025-12-11 18:27                                distros.txt                                Page 1  
SUSE      10.2      12/07/2006  
Fedora    10        11/25/2008  
SUSE      11.0      06/19/2008  
Ubuntu    8.04      04/24/2008  
Fedora     8        11/08/2007  
2025-12-11 18:27                                distros.txt                                Page 2  
SUSE      10.3      10/04/2007  
Ubuntu    6.10      10/26/2006  
Fedora     7        05/31/2007  
Ubuntu    7.10      10/18/2007  
Ubuntu    7.04      04/19/2007
```

В этом примере мы используем опцию `-l` (длина страницы) и опцию `-w` (ширина страницы), чтобы задать «страницу» шириной 65 столбцов и длиной 15 строк. `pr` разбивает содержимое `distros.txt` файла на страницы, разделяя каждую несколькими пустыми строками, и создает стандартный заголовок с временем изменения файла, именем файла и номером страницы.

Программа `pr` предоставляет множество опций для управления макетом страницы. Мы рассмотрим их в главе 22.

`printf` — Форматирование и вывод данных

В отличие от других команд этой главы, команда `printf` не используется в конвейерах (она не принимает стандартный ввод) и редко применяется напрямую в командной строке (в основном она используется в скриптах). Почему же она важна? Потому что она чрезвычайно распространена.

`printf` (от фразы «print formatted») изначально был разработан для языка программирования C и реализован во многих языках, включая оболочку. Фактически, в `bash`, `printf` является встроенной командой.

`printf` работает следующим образом:

```
printf [-v var] "format" arguments
```

Команде передается строка с описанием формата, которое затем применяется к списку аргументов. Отформатированный результат отправляется в стандартный вывод, если не указана опция `-v`, в этом случае результат сохраняется в переменную `var`. Вот простой пример работы `printf`:

```
[me@linuxbox ~]$ printf "I formatted the string: %s\n" foo
I formatted the string: foo
```

Вот тот же пример с использованием опции `-v`, когда результат записывается в переменную, а не выводится в стандартный поток вывода:

```
[me@linuxbox ~]$ printf -v result "I formatted the string: %s\n" foo
[me@linuxbox ~]$ echo "$result"
I formatted the string: foo
```

Строка формата может содержать обычный текст (например, «I formatted the string:»), управляющие последовательности (такие как `\n`, символ новой строки) и последовательности, начинающиеся с символа `%`, которые называются спецификациями преобразования. В приведенном выше примере спецификация преобразования `%s` используется для форматирования строки «foo» и её вывода командой. Вот она снова:

```
[me@linuxbox ~]$ printf "I formatted '%s' as a string.\n" foo
I formatted 'foo' as a string.
```

Как видно, в выводе команды спецификация преобразования %s заменяется строкой «foo». Преобразование s используется для форматирования строковых данных. Существуют и другие спецификаторы для иных типов данных. В таблице 21-4 перечислены наиболее часто используемые типы данных.

Таблица 21-4: Распространенные спецификаторы типов printf данных

Спецификатор	Описание
d	Форматирует число как знаковое десятичное целое.
f	Форматирует и выводит число с плавающей точкой.
o	Форматирует целое число как восьмеричное.
s	Форматирует строку.
x	Форматирует целое число как шестнадцатеричное, используя строчные a–f там, где это необходимо.
X	То же, что и x, но заглавными буквами.
%	Вывод символа% как есть (т. е. указание%%)

Мы продемонстрируем действие каждого спецификатора преобразования на строке 380.

```
[me@linuxbox ~]$ printf "%d, %f, %o, %s, %x, %X\n" 380 380 380 380
380 380
380, 380.000000, 574, 380, 17c, 17C
```

Поскольку мы указали шесть спецификаторов преобразования, мы также должны предоставить шесть аргументов для обработки printf. Шесть результатов демонстрируют действие каждого спецификатора.

Для настройки вывода к спецификатору преобразования можно добавить несколько необязательных компонентов. Полная спецификация преобразования может состоять из следующего:

%[flags][width][.precision]conversion_specification

Несколько необязательных компонентов при использовании должны следовать в указанном ранее порядке для корректной интерпретации. В таблице 21-5 приведено описание каждого из них.

Таблица 21-5: Компоненты спецификации преобразования *printf*
ых. ом 0. ько

Компонент	Описание
flags	Существует пять различных флагов: #: использовать «альтернативный формат» вывода. Он зависит от типа данн При преобразовании в o (восьмеричное число) вывод дополняется префикс При преобразовании в x и X (шестнадцатеричное число) вывод дополняется префиксами 0x или 0X соответственно. 0 (ноль): дополнить вывод нулями. Это означает, что поле будет заполнено ведущими нулями, как в 000380. - (дефис): выравнивать вывод по левому краю. По умолчанию <i>printf</i> в ыравнивается output. «» (пробел): добавить ведущий пробел для положительных чисел. + (знак плюс): отображать знак у положительных чисел. По умолчанию <i>printf</i> тол знаков отрицательных чисел.
width	Число, задающее минимальную ширину поля.
.precision	Для чисел с плавающей точкой задает количество знаков точности после десятичной точки. Для преобразования строк точность задает количество выводимых символов.

В таблице 21-6 приведены примеры использования различных форматов.

Таблица 21-6: Примеры спецификаций преобразования *printf*

Аргумент	Форматирование	Результат	Примечания
380	%d	380	Простое форматирование целого числа.
380	%#x	0x17c	Целое число в виде шестнадцатеричного с помощью флага «alternate format».

Аргумент	Форматирование	Результат	Примечания
380	%05d	00380	Целое число, отформатированное с помощью ведущие нули (дополнение) и минимальную ширину поля в пять символов.
380	%05.5f	380.00000	Число, форматированное как число с плавающей точкой с отступы и пять знаков после запятой мест точности. Так как заданная минимальная ширина поля (5) меньше фактической ширины форматированное число, the дополнение не применяется.
380	%010.5f	0380.00000	Увеличив минимальную ширину поля до 10,, отступы теперь заметны.
380	%+d	+380	Флаг + обозначает положительное число.
380	%-d	380	Флаг - выравнивает по левому краю форматирования.
abcdefghijkl	%5s	abcdefghijkl	Строка, отформатированная с минимальной шириной поля,.
abcdefghijkl	%.5s	abcde	при использовании точности для строки, будет усечена.

Опять же, `printf` используется в основном в скриптах для форматирования табличных данных, а не напрямую в командной строке. Тем не менее, мы можем показать, как он помогает решать различные задачи форматирования. Сначала выведем несколько полей, разделенных символами табуляции.

```
[me@linuxbox ~]$ printf "%s\t%s\t%s\n" str1 str2 str3
str1    str2    str3
```

Вставив `\t` (escape-последовательность для табуляции), мы добьемся нужного эффекта. Далее приведены числа с аккуратным форматированием:

```
[me@linuxbox ~]$ printf "Line: %05d %15.3f Result: %+15d\n" 1071
3.14156295 32589
Line: 01071          3.142 Result:          +32589
```

Это демонстрирует влияние минимальной ширины поля на расстояние между ними. А как насчет форматирования крошечной веб-страницы?

```
[me@linuxbox ~]$ printf "<html>\n\t<head>\n\t\t<title>%s</title>\n\t</head>\n\t<body>\n\t\t<p>%s</p>\n\t</body>\n</html>\n" "Page Title" "Page Content"
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <p>Page Content</p>
  </body>
</html>
```

Системы форматирования документов

До сих пор мы рассматривали простые инструменты форматирования текста. Они хороши для небольших задач, но как быть с более масштабными? Одной из причин популярности Unix среди технических специалистов и ученых (помимо мощной многозадачной многопользовательской среды для разработки ПО) стало наличие инструментов для создания различных типов документов, особенно научных и академических публикаций. На самом деле, как указано в документации GNU, подготовка документов сыграла ключевую роль в развитии Unix.

Первая версия UNIX была разработана на PDP-7, стоявшей в Bell Labs. В 1971 году разработчики захотели приобрести PDP-11 для дальнейшей работы над ОС. Чтобы оправдать стоимость системы, они предложили внедрить систему форматирования документов для патентного отдела AT&T. Эта первая программа

форматирования стала переработкой программы McIlroy's `roff`, написанной J. F. Ossanna.

В этой области доминируют две основные группы: потомки оригинальной программы `roff`, включая `nroff` и `troff`, и системы на базе системы верстки TEX (произносится как «тек») Дональда Кнута. И да, пропущенная «Е» в середине — часть её названия.

Название «`roff`» происходит от выражения «`run off`» — в значении «сделать копию». Программа `nroff` используется для форматирования документов для вывода на устройства с моноширинными шрифтами, такие как терминалы или печатные машинки. На момент появления это охватывало почти все принтеры, подключенные к компьютерам. Более поздняя программа `troff` форматирует документы для вывода на наборы, используемые для создания «готовых к печати» макетов в коммерческой полиграфии. Большинство современных принтеров способны имитировать работу таких наборов. Семейство `roff` также включает другие программы для подготовки отдельных частей документов, в том числе `eqn` (для математических уравнений) и `tbl` (для таблиц).

Система TEX (в стабильной версии) появилась в 1989 году и в некоторой степени вытеснила `troff` как основной инструмент верстки. Мы не будем рассматривать TEX, как из-за его сложности (об этом написаны целые книги), так и из-за того, что он не установлен по умолчанию в большинстве современных систем Linux.

Совет: Тем, кто хочет установить TEX, стоит изучить пакет `texlive`, который доступен в репозиториях большинства дистрибутивов, а также графический редактор контента LyX.

`groff`

`groff` — это набор программ, содержащий реализацию `troff` от GNU. Он также включает скрипт для эмуляции `nroff` и остальных программ семейства `roff`.

Хотя `roff` и его потомки используются для создания форматированных документов, они делают это способом, довольно непривычным для современных пользователей. Большинство современных документов создаются в текстовых процессорах, которые позволяют выполнять набор и верстку

документа за один шаг. До появления графических текстовых процессоров создание документов часто проходило в два этапа: сначала с помощью текстового редактора выполнялся набор текста, а затем с помощью процессора, такого как `troff`, применялось форматирование. Инструкции для программы форматирования встраивались в набранный текст с помощью языка разметки. Современным аналогом такого процесса является веб-страница, которая создается в текстовом редакторе, а затем отображается веб-браузером с использованием HTML в качестве языка разметки для описания макета страницы.

Мы не будем рассматривать `groff` целиком, так как многие элементы его языка разметки касаются довольно специфических деталей типографики. Вместо этого мы сосредоточимся на одном из его макропакетов, который до сих пор широко используется. Эти макропакеты объединяют множество низкоуровневых команд в меньший набор высокоуровневых, что значительно упрощает работу с `groff`.

На мгновение рассмотрим скромную страницу `man`. Она хранится в `/usr/share/man` каталоге как `gzip` сжатый текстовый файл. Если изучить её содержимое в несжатом виде, мы увидим следующее (показана страница `man` для `ls` в разделе 1):

```
[me@linuxbox ~]$ zcat /usr/share/man/man1/ls.1.gz | head
.\" DO NOT MODIFY THIS FILE! It was generated by help2man 1.47.3.
.TH LS "1" "January 2025" "GNU coreutils 8.28" "User Commands"
.SH NAME
ls \- list directory contents
.SH SYNOPSIS
.B ls
[fI\,OPTION\|fR]... [fI\,FILE\|fR]...
.SH DESCRIPTION
.\" Add any additional description here
.PP
```

В сравнении с обычным видом страницы `man`, мы можем увидеть связь между языком разметки и результатом его работы.

```
[me@linuxbox ~]$ man ls | head
LS(1)                                User Commands                                LS(1)
NAME
    ls - list directory contents
SYNOPSIS
    ls [OPTION]... [FILE]...
```

Интерес вызывает то, что страницы `man` отрисовываются с помощью `groff`, использующей пакет макросов `mandoc man`. На самом деле, мы можем имитировать эту команду с помощью следующего конвейера:

```
[me@linuxbox ~]$ zcat /usr/share/man/man1/ls.1.gz | groff -mandoc -T
ascii | head
```

LS(1)	User Commands	LS(1)
NAME		
	ls - list directory contents	
SYNOPSIS		
	ls [OPTION]... [FILE]...	

Здесь мы используем программу `groff` с опциями для указания `mandoc` и драйвера вывода для ASCII. `groff` может создавать вывод в нескольких форматах. Если формат не указан, по умолчанию выводится PostScript.

```
[me@linuxbox ~]$ zcat /usr/share/man/man1/ls.1.gz | groff -mandoc |
head
%!PS-Adobe-3.0
%Creator: groff version 1.18.1
%CreationDate: Thu Feb  5 13:44:37 2025
%DocumentNeededResources: font Times-Roman
%+ font Times-Bold
%+ font Times-Italic
%DocumentSuppliedResources: procset grops 1.18 1
%%Pages: 4
%%PageOrder: Ascend
%%Orientation: Portrait
```

В предыдущей главе мы кратко упоминали PostScript и вернемся к нему в следующей. PostScript — это язык описания страниц, используемый для передачи содержимого печатной страницы устройству, подобному наборному станку. Если мы сохраним вывод нашей команды в файл (предполагая, что используем графический рабочий стол с Desktop каталогом), на рабочем столе должен появиться значок выходного файла.

```
[me@linuxbox ~]$ zcat /usr/share/man/man1/ls.1.gz | groff -mandoc >
~/Desktop/ls.ps
```

При двойном щелчке по иконке должен запускаться просмотрщик страниц и отобразить файл в отрендеренном виде, как показано на рисунке 5.

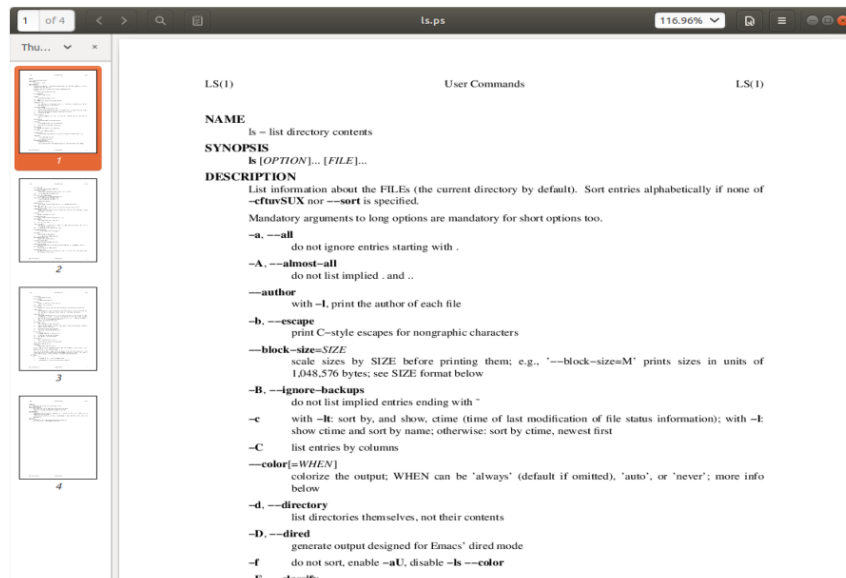


Рисунок 4: Просмотр PostScript-вывода через просмотрщик страниц в GNOME

Перед нами красиво сверстанная страница руководства для `ls`! На самом деле, PostScript-файл можно преобразовать в формат PDF с помощью этой команды:

```
[me@linuxbox ~]$ ps2pdf ~/Desktop/foo.ps ~/Desktop/ls.pdf
```

Программа `ps2pdf` является частью пакета `ghostscript`, который установлен в большинстве Linux-систем с поддержкой печати.

Совет: в Linux часто предустановлено множество программ для конвертации форматов файлов. Их часто называют по шаблону `format2format`. Попробуйте использовать команду `ls /usr/bin/*[[[:alpha:]]2[[[:alpha:]]]*`, чтобы найти их. Также попробуйте искать программы с именами `formattoformat`.

В последнем упражнении с `groff` мы снова обратимся к нашему старому другу `distros.txt`. На этот раз мы воспользуемся программой `tbl`, которая форматирует таблицы для верстки нашего списка дистрибутивов Linux. Для этого мы используем наш предыдущий скрипт `sed`, чтобы добавить разметку в поток текста, который мы передадим в `groff`.

Сначала нужно изменить наш `sed` скрипт, добавив необходимые элементы разметки (называемые запросами в `groff`), которые

требует `tbl`. С помощью текстового редактора мы изменим `distros.sed` на следующее:

```
# sed script to produce Linux distributions report
1 i\
.TS\
center box;\
cb s s\
cb cb cb\
l n c.\
Linux Distributions Report\
=\
Name      Version  Released\
-
s/\([0-9]\{2\}\)\(\([0-9]\{2\}\)\)\(\([0-9]\{4\}\)\)/\3-\1-\2/
$ a\
.TE
```

Примечание: чтобы скрипт работал правильно, важно следить, чтобы слова `Name` `Version` `Released` были разделены табуляцией, а не пробелами. Мы сохраним итоговый файл как `distros-tbl.sed`. `tbl` использует запросы `.TS` и `.TE` для начала и завершения таблицы. Строки после запроса `.TS` определяют глобальные свойства таблицы: в нашем примере они центрированы по горизонтали и обрамлены рамкой. Остальные строки описания задают макет каждой строки таблицы. Теперь, если мы снова запустим наш конвейер генерации отчетов с новым `sed script`, мы получим следующее:

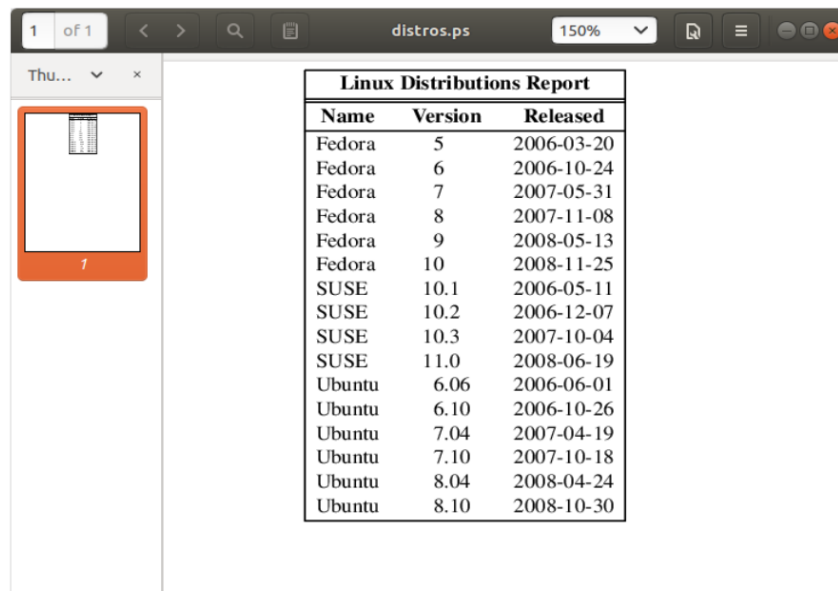
```
[me@linuxbox ~]$ sort -k 1,1 -k 2n distros.txt | sed -f distros-tbl
.sed | groff -t -T ascii
```

+-----+		
Linux Distributions Report		
+-----+		
Name	Version	Released
+-----+		
Fedora	5	2006-03-20
Fedora	6	2006-10-24
Fedora	7	2007-05-31
Fedora	8	2007-11-08
Fedora	9	2008-05-13
Fedora	10	2008-11-25
SUSE	10.1	2006-05-11
SUSE	10.2	2006-12-07
SUSE	10.3	2007-10-04
SUSE	11.0	2008-06-19
Ubuntu	6.06	2006-06-01
Ubuntu	6.10	2006-10-26
Ubuntu	7.04	2007-04-19
Ubuntu	7.10	2007-10-18
Ubuntu	8.04	2008-04-24
Ubuntu	8.10	2008-10-30
+-----+		

Опция `-t` в `groff` указывает программе обрабатывать поток текста с помощью `tbl`. Аналогично, опция `-T` используется для вывода в ASCII вместо стандартного формата PostScript.

Формат вывода ограничен возможностями терминала или печатной машинки. Если же указать вывод в PostScript и просмотреть его графически, результат будет гораздо лучше, как показано на рисунке 6.

```
[me@linuxbox ~]$ sort -k 1,1 -k 2n distros.txt | sed -f distros-tbl.sed | groff -t > ~/Desktop/foo.ps
```

A screenshot of a PDF viewer window titled 'distros.ps' at 150% zoom. The document contains a table titled 'Linux Distributions Report' with three columns: 'Name', 'Version', and 'Released'. The table lists various Linux distributions including Fedora, SUSE, and Ubuntu with their respective versions and release dates.

Name	Version	Released
Fedora	5	2006-03-20
Fedora	6	2006-10-24
Fedora	7	2007-05-31
Fedora	8	2007-11-08
Fedora	9	2008-05-13
Fedora	10	2008-11-25
SUSE	10.1	2006-05-11
SUSE	10.2	2006-12-07
SUSE	10.3	2007-10-04
SUSE	11.0	2008-06-19
Ubuntu	6.06	2006-06-01
Ubuntu	6.10	2006-10-26
Ubuntu	7.04	2007-04-19
Ubuntu	7.10	2007-10-18
Ubuntu	8.04	2008-04-24
Ubuntu	8.10	2008-10-30

Рисунок 5: Просмотр готовой таблицы

Итоги

Поскольку текст играет ключевую роль в архитектуре Unix-подобных систем, логично, что существует множество инструментов для его обработки и форматирования. И они действительно есть! Простые средства форматирования, такие как `fmt` и `pr`, найдут применение в скриптах для создания коротких документов, а `groff` (и его аналоги) можно использовать для написания книг. Возможно, мы никогда не будем писать технические статьи с помощью инструментов командной строки (хотя многие это делают!), но полезно знать, что такая возможность есть.

Что почитать

- `groff` Руководство пользователя
<https://www.gnu.org/software/groff/manual/>
- Написание статей с `groff` с помощью `-me`:
<https://docs.freebsd.org/44doc/usd/19.memacros/paper.pdf>

- *Справочное руководство* - те:
<https://docs.freebsd.org/44doc/usd/20.meref/paper.pdf>
- И, конечно, ознакомьтесь со следующими статьями в Wikipedia: <https://en.wikipedia.org/wiki/TeX>
https://en.wikipedia.org/wiki/Donald_Knuth
<https://en.wikipedia.org/wiki/Typesetting>

22 — Печать

После нескольких глав, посвященных манипуляциям с текстом, пора перенести его на бумагу. В этой главе мы рассмотрим инструменты командной строки для печати файлов и управления принтером. Мы не будем разбирать настройку печати, так как она зависит от дистрибутива и обычно выполняется автоматически при установке. Примечание: для выполнения упражнений этой главы потребуется настроенный принтер.

Мы рассмотрим следующие команды:

- `pr` — подготовка текстовых файлов к печати
- `lp / lpr` — печать файлов
- `a2ps` — форматирование файлов для печати на принтере PostScript
- `lpstat` — показать информацию о состоянии принтера
- `lpq` — показать состояние очереди печати
- `lprm` — отменить задания на печать

Краткая история печати

Чтобы полностью понять возможности печати в Unix-подобных операционных системах, нужно сначала изучить историю. Печать в Unix-подобных системах берет начало еще с момента создания самих ОС. В те времена принтеры и способы их использования сильно отличались от современных.

Печать в темные времена

Как и компьютеры, принтеры в эпоху до появления ПК обычно были громоздкими, дорогими и централизованными. Типичный пользователь в 1980 году работал через терминал, подключенный к удаленному компьютеру. Принтер находился рядом с компьютером под пристальным присмотром его операторов.

Когда принтеры были дорогими и централизованными, как это часто бывало в ранние дни Unix, многие пользователи обычно пользовались одним принтером. Чтобы идентифицировать задания печати, принадлежащих конкретному пользователю, в начале каждого задания часто печаталась титульная страница с его именем. Затем ИТ-специалисты загружали тележку с заданиями за день и разносили их пользователям.

Символьные принтеры

Технологии печати 80-х годов отличались от современных в двух аспектах. Во-первых, принтеры того периода почти всегда были ударными. Ударные принтеры используют механический механизм, который ударяет лентой по бумаге, оставляя оттиски символов на странице. Двумя популярными технологиями того времени были печать на матричном колесе и матричная печать.

Второй, и более важный признак ранних принтеров, заключался в том, что они использовали фиксированный набор символов, встроенный в само устройство. Например, принтер с матричным колесом мог печатать только те символы, которые были отлиты на лепестках колеса. Это делало принтеры похожими на высокоскоростные пишущие машинки. Как и большинство пишущих машинок, они использовали моноширинные шрифты. Это значит, что каждый символ имеет одинаковую ширину. Печать выполнялась в фиксированных позициях на странице, а область печати содержала фиксированное количество символов. Большинство принтеров печатали 10 символов на дюйм (CPI) по горизонтали и шесть строк на дюйм (LPI) по вертикали. При такой схеме лист бумаги формата US-letter имеет ширину 85 символов и высоту 66 строк. С учетом небольших полей с каждой стороны, максимальной шириной строки считалось 80 символов. Это объясняет, почему дисплеи терминалов (и наши эмуляторы терминала) обычно имеют ширину 80 символов. Использование моноширинного шрифта и терминала шириной 80 символов обеспечивает принцип «что видишь, то и получишь» (WYSIWYG) при выводе на печать.

Данные передаются на печатную машинку в виде простого потока байтов, содержащего символы для печати. Например, для печати «а» отправляется ASCII-код 97. Кроме того, низкоуровневые управляющие коды ASCII позволяли перемещать каретку и бумагу, используя коды для возврата каретки, перехода на новую строку, подачи страницы и так далее. С помощью управляющих кодов можно добиться ограниченных эффектов шрифта, например жирного начертания: принтер печатает символ, делает `backspace` и печатает его снова, чтобы оттиск на странице стал темнее. Мы можем увидеть это на практике, если используем `nroff` для отрисовки `man page` и изучим результат с помощью `cat -A`.

```
[me@linuxbox ~]$ zcat /usr/share/man/man1/ls.1.gz | nroff -P -c -man
| cat -A | head
LS(1)                                User Commands                                LS(1)
$
$
$
N^HNA^HAM^HME^HE$
      ls - list directory contents$
```

```
$
S^H$Y^HYN^HNO^HNP^HPS^HSI^HIS^HS$
  l^Hls^Hs [_^HO^HNP^HNT^HNI^HNO^HNN]... [_^HF^HNI^HNL^HNE]...$
```

Символы ^H (Ctrl-h) — это управляющие символы удаления (backspace), используемые для создания жирного шрифта. Аналогично, можно увидеть последовательность удаления и подчеркивания для создания подчеркнутого текста.

Графические принтеры

Развитие графических интерфейсов привело к серьезным изменениям в технологиях печати. По мере перехода компьютеров на дисплеи с упором на изображения, печать сменилась с символьной на графическую. Этому способствовало появление недорогих лазерных принтеров, которые вместо печати фиксированных символов могли ставить крошечные точки в любом месте печатной области страницы. Это сделало возможной печать пропорциональных шрифтов (подобных тем, что используют наборщики), а также фотографий и высококачественных диаграмм.

Однако переход от символьной схемы к графической стал серьезным техническим вызовом. Вот почему: количество байтов, необходимых для заполнения страницы символьным принтером, можно рассчитать следующим образом (предположим, 60 строк на странице по 80 символов в каждой):

$$60 \times 80 = 4,800 \text{ байт}$$

Для сравнения, лазерному принтеру с разрешением 300 точек на дюйм (при области печати 8 на 10 дюймов на страницу) требуется столько байт:

$$(8 \times 300) \times (10 \times 300) / 8 = 900,000 \text{ байт}$$

Многие медленные сети ПК просто не могли справиться с почти мегабайтом данных, необходимым для печати полной страницы на лазерном принтере, поэтому стало ясно, что требуется остроумное изобретение.

Этим изобретением стал язык описания страниц (PDL). Язык описания страниц — это язык программирования, описывающий содержимое страницы. По сути, он говорит: «Перейдите в эту позицию, нарисуйте символ «a» шрифтом Helvetica 10 пунктов, перейдите в эту позицию...» — пока не будет описано всё на странице. Первым крупным PDL стал PostScript от Adobe Systems, который широко используется и сего дня. Язык PostScript — это полноценный язык программирования, адаптированный для типографики и других видов графики и изображений. Он включает встроенную поддержку 35 стандартных

высококачественных шрифтов, а также возможность принимать дополнительные определения шрифтов во время выполнения. Поначалу поддержка PostScript была встроена в сами принтеры. Это решило проблему передачи данных. Хотя типичная программа PostScript была очень многословной по сравнению с простым потоком байтов символьных принтеров, она занимала гораздо меньше места, чем объем байтов, необходимый для представления всей печатной страницы.

Принтер PostScript принимал на вход программу PostScript. Принтер имел собственный процессор и память (зачастую делая его более мощным компьютером, чем компьютер, к которому он был подключен) и выполнял специальную программу под названием интерпретатор PostScript. Интерпретатор считывал входящую программу PostScript и отрисовывал результат во внутреннюю память принтера, формируя тем самым набор битов (точек), которые затем переносились на бумагу. Общее название этого процесса отрисовки чего-либо в большой битовый паттерн (называемый растровым изображением) — растровый процессор (RIP).

С годами и компьютеры, и сети стали намного быстрее. Это позволило перенести RIP с принтера на хост-компьютер, что, в свою очередь, сделало высококачественные принтеры гораздо дешевле.

Многие современные принтеры всё еще принимают потоки символов, но многие бюджетные модели — нет. Они полагаются на RIP хост-компьютера, который передает поток битов для печати в виде точек. Также всё еще встречаются принтеры PostScript.

Печать в Linux

Современные системы Linux используют два программных пакета для выполнения и управления печатью. Первый, Common Unix Printing System (CUPS), предоставляет драйверы и управляет заданиями на печать, а второй, Ghostscript (интерпретатор PostScript), выступает в роли RIP.

CUPS управляет принтерами, создавая и поддерживая очереди печати. Как мы обсуждали в уроке по истории, печать в Unix изначально проектировалась для управления централизованным принтером, используемым несколькими пользователями. Поскольку принтеры по своей природе медленнее компьютеров, которые ими управляют, системам печати необходим способ планирования множества заданий и поддержания порядка. CUPS также способен распознавать различные типы данных (в

разумных пределах) и конвертировать файлы в печатный формат.

Подготовка файлов к печати

Как пользователей командной строки, нас в основном интересует печать текста, хотя печатать можно и другие форматы данных.

pr – Преобразование текстовых файлов для печати

В предыдущей главе мы немного коснулись pr. Теперь рассмотрим некоторые из его многочисленных опций для печати. Изучая историю печати, мы видели, как символьные принтеры используют моноширинные шрифты, что приводит к фиксированному количеству символов в строке и строк на странице. pr используется для подгонки текста под определенный размер страницы с возможным добавлением колонтитулов и полей. В таблице 22-1 приведены наиболее часто используемые опции.

Таблица 22-1: Общие опции pr

Опция	Описание
+first[:last]	Вывод диапазона страниц, начиная с первой и, по желанию, заканчивая последней.
-columns	Разбить содержимое страницы на указанное количество колонок.
-a	По умолчанию многоколоночный вывод идет вертикально. С опцией -a (across) содержимое выводится горизонтально.
-d	Вывод с двойным интервалом.
-D "format"	Форматировать дату в заголовках страниц. См. см. man-страницу команды date для описания строки формата.
-f	Для разделения страниц используйте разрыв страницы вместо возврата каретки.
-h "header"	В центральной части заголовка страницы вместо имени обрабатываемого файла используйте header.
-l length	Установите длину страницы параметром length. По умолчанию — 66 (US letter при шести строк на дюйм)

Опция	Описание
-n	Количество строк.
-o offset	Создать левый отступ заданной ширины в символах.
-w width	Установить ширину страницы равной width. По умолчанию — 72.

`pr` часто используется в конвейерах в качестве фильтра. В этом примере мы сформируем список содержимого каталога `/usr/bin` и отформатируем его в трехколоночный постраничный вывод с помощью `pr`:

```
[me@linuxbox ~]$ ls /usr/bin | pr -3 -w 65 | head
2025-02-18 14:00 Page 1
[
411toppm      apturl          bsd-write
a2p           ar              bsh
a2ps          arecord         btcflash
a2ps          arecordmidi     bug-buddy
a2ps-lpr-wrapper ark              buildhash
```

Отправка задания на печать

Пакет печати CUPS поддерживает два метода печати, исторически использовавшихся в Unix-подобных системах. Один метод, называемый Berkeley или LPD (использовался в версии Unix Berkeley Software Distribution), использует программу `lpr`, а другой, называемый SysV (из версии Unix System V), — программу `lp`. Обе программы выполняют примерно одно и то же. Выбор между ними — вопрос личных предпочтений.

`lpr` — Печать файлов (стиль Berkeley)

Программу `lpr` можно использовать для отправки файлов на принтер. Она также может применяться в конвейерах, так как принимает стандартный ввод. Например, чтобы распечатать результаты нашего предыдущего многоколоночного вывода каталога, можно сделать следующее:

```
[me@linuxbox ~]$ ls /usr/bin | pr -3 | lpr
```

Отчет будет отправлен на принтер по умолчанию. Чтобы отправить файл на другой принтер, можно использовать опцию `-P` следующим образом:

```
lpr -P printer_name
```


Здесь `printer_name` — это имя нужного принтера. Чтобы увидеть список принтеров, известных системе, используйте эту команду:

```
[me@linuxbox ~]$ lpstat -a
```

Совет: Многие дистрибутивы Linux позволяют указать «принтер», который сохраняет файлы в формате PDF вместо печати на физическом устройстве. Это удобно для тестирования команд печати. Проверьте в программе настройки принтера, поддерживает ли она такой режим. В некоторых дистрибутивах для этого может потребоваться установка дополнительных пакетов (например, `cups-pdf`).

В таблице 22-2 описаны основные опции для `lpr`.

Таблица 22-2: Общие параметры `lpr`

Опция	Описание
-# number	Указать количество копий — number.
-p	Печатать каждую страницу с затененным заголовком, содержащим дату, время, имя задания и номер страницы. Эта опция так называемого «красивого вывода» полезно при печати текстовых файлов.
-P printer	Указать имя принтера для вывода. Если принтер не указан, используется принтер по умолчанию.
-r	Удалять файлы после печати. Это полезно для программ, создающих временные файлы вывода на печать.

`lp` — Печать файлов (в стиле System V)

Как и `lpr`, `lp` принимает для вывода либо файлы, либо стандартный ввод. Он отличается от `lpr` тем, что поддерживает другой (и чуть более сложный) набор параметров. В таблице 22-3 описаны общие параметры.

Таблица 22-3: Общие опции `lp`

Опция	Описание
-------	----------

Опция	Описание
-d printer	Установить принтер назначения как printer. Если не указан параметр d, используется принтер по умолчанию в системе. used.
-n number	Установить количество копий как number.
-o landscape	Установить альбомную ориентацию вывода.
-o fitplot	Масштабировать файл под размер страницы. Это полезно, когда печать изображений, например, файлов JPEG.
-o scaling=number	Масштабировать файл до числа. Значение 100 заполняет страницу. Значения меньше 100 уменьшают масштаб, а значения больше 100 приводят к печати файла на нескольких страницах.
-o cpi=number	Установить количество выходных символов на дюйм. по умолчанию 10.
-o lpi=number	Установите количество строк на дюйм. По умолчанию — 6.
-P pages	Укажите список страниц. Страницы можно задать через запятую и/или диапазоном, например, example, 1,3,5,7-10

Мы снова выведем список содержимого каталога, но на этот раз напечатаем 12 CPI и 8 LPI с левым полем в полдюйма.

Примечание: нам придется изменить параметры pr, чтобы учесть новый размер страницы:

```
[me@linuxbox ~]$ ls /usr/bin | pr -4 -w 90 -l 88 | lp -o page-left=36
-o cpi=12 -o lpi=8
```

Этот конвейер создает четырехколоный список, используя шрифт меньшего размера, чем по умолчанию. Увеличенное количество символов на дюйм позволяет уместить на странице больше колонок.

Другой вариант: a2ps

```
[me@linuxbox ~]$ ls /usr/bin | pr -3 -t | a2ps -o ~/Desktop/ls.ps -L
66
[stdin (plain): 11 pages on 6 sheets]
[Total: 11 pages on 6 sheets] saved into the file `~/home/me/Desktop/
ls.ps'
```

[illegible]

355

Как видно, формат вывода по умолчанию — «две страницы на лист». Из-за этого содержимое двух страниц печатается на одном листе бумаги. `a2ps` также добавляет аккуратные заголовки и подвалы страниц.

`a2ps` имеет множество опций. В таблице 22-4 приведено их описание.

Таблица 22-4: Опции `a2ps`

Опция	Описание
<code>--center-title=text</code>	Установить текст заголовка по центру страницы.
<code>--columns=number</code>	Разместить страницы в несколько колонок. По умолчанию — 2.
<code>--footer=text</code>	Установить текст в нижнем колонтитуле.
<code>--guess</code>	Вывести типы файлов, переданных в качестве аргументов.
	Поскольку <code>a2ps</code> пытается преобразовать и форматировать типы данных, этот параметр может быть полезен для прогнозирования того, что <code>a2ps</code> сделает с конкретным файлом.
<code>--left-footer=text</code>	Установить текстовый нижний колонтитул для левой страницы.
<code>--left-title=text</code>	Установить заголовок левой страницы.
<code>--line-numbers=interval</code>	Нумеровать строки вывода через каждые N строк.
<code>--list=defaults</code>	Показать настройки по умолчанию.
<code>--pages=range</code>	Печатать страницы в указанном диапазоне.
<code>--right-footer=text</code>	Установить нижний колонтитул правой страницы.
<code>--right-title=text</code>	Установить заголовок правой страницы.

Опция	Описание
- - rows=number	Разместить страницы по количеству строк. По умолчанию is 1.
-B	Без заголовков страниц.
-b text	Установить текст заголовка страницы.
-f size	Использовать шрифт размером в пункты.
-l number	Установить количество символов в строке. Это и -L опция (см. следующую запись) позволяет файлы, разбитые на страницы другими программами, так <code>pr</code> , правильно помещаются на странице.
-L number	Установить количество строк на страницу.
-M name	Использовать имя носителя. Например, A4.
-n number	Вывести указанное количество копий каждой страницы.
-o file	Направить вывод в файл. Если указан файл -, использовать стандартный вывод.
-P printer	Использовать принтер. Если принтер не указан, использовать принтер по умолчанию.
-R	Книжная ориентация.
-r	Альбомная ориентация.
-T number	Установить позиции табуляции через каждые N символов.
-u text	Наложение текста в качестве подложки (водяного знака).

Это лишь краткий обзор. У `a2ps` есть еще несколько опций.

Примечание: существует другой форматировщик, полезный для преобразования текста в PostScript. Он

называется `enscript` и может выполнять многие те же приемы форматирования и печати, но, в отличие от `a2ps`, принимает только текстовый ввод.

Мониторинг и управление заданиями печати

Поскольку системы печати Unix предназначены для обработки множества заданий от разных пользователей, CUPS работает по тому же принципу. Каждому принтеру назначается очередь печати, где задания ожидают своей очереди на отправку на принтер. CUPS предоставляет несколько программ командной строки для управления состоянием принтеров и очередями печати. Как и программы `lpr` и `lp`, эти инструменты управления основаны на соответствующих программах систем печати Berkeley и System V.

`lpstat` — Отображение состояния системы печати

Программа `lpstat` полезна для определения имен и доступности принтеров в системе. Например, если бы в системе были физический принтер (с именем «printer») и виртуальный PDF-принтер (с именем «PDF»), мы могли бы проверить их статус так:

```
[me@linuxbox ~]$ lpstat -a
PDF accepting requests since Mon 08 Dec 2024 03:05:59 PM EST
printer accepting requests since Tue 24 Feb 2025 08:43:22 AM EST
```

Кроме того, таким способом можно получить более подробное описание конфигурации системы печати:

```
[me@linuxbox ~]$ lpstat -s
system default destination: printer
device for PDF: cups-pdf:/
device for printer: ipp://print-server:631/printers/printer
```

В этом примере видно, что «printer» является принтером по умолчанию, и это сетевой принтер, использующий Internet Printing Protocol (`ipp://`) и подключенный к системе с именем «print-server».

В таблице 22-5 перечислены наиболее полезные опции.

Таблица 22-5: Общие опции `lpstat`

Опция	Описание
-------	----------

Опция	Описание
-a [printer...]	Выводит состояние очереди принтера printer. Примечание: это статус способности очереди принтера принимать заданий, а не состояние физических принтеров. Если принтеры не указаны, отображаются все очереди печати.
-d	Выводит имя принтера по умолчанию в системе.
-p [printer...]	Выводит состояние указанного принтера. Если принтеры не указаны, отображаются все принтеры.
-r	Выводит состояние сервера печати.
-s	Вывод краткой сводки состояния.
-t	Вывод полного отчета о состоянии.

lprq – lprq — Просмотр состояния очереди принтера

Для проверки состояния очереди печати используется программа lprq. Она позволяет просматривать статус очереди и содержащиеся в ней задания. Вот пример пустой очереди для принтера по умолчанию с именем «printer»:

```
[me@linuxbox ~]$ lprq
printer is ready
no entries
```

Если не указать принтер (с помощью опции -P), будет показан принтер по умолчанию. Если отправить задание на печать, а затем проверить очередь, оно будет там отображаться.

```
[me@linuxbox ~]$ ls *.txt | pr -3 | lp
request id is printer-603 (1 file(s))
[me@linuxbox ~]$ lprq
printer is ready and printing
Rank   Owner   Job    File(s)                Total Size
active me      603    (stdin)                1024 bytes
```

lprm / cancel — Отмена заданий печати

CUPS предоставляет две программы для завершения заданий печати и их удаления из очереди. Одна выполнена в стиле Berkeley (lprm), другая — в стиле System V (cancel). Они немного различаются поддерживаемыми опциями, но выполняют по сути одно и то же. На примере нашего предыдущего задания мы можем остановить и удалить его следующим образом:

```
[me@linuxbox ~]$ cancel 603
[me@linuxbox ~]$ lpq
printer is ready
no entries
```

Каждая команда имеет опции для удаления всех заданий конкретного пользователя, конкретного принтера или нескольких заданий по их номерам. Подробности — в соответствующих man-страницах.

Итоги

В этой главе мы рассмотрели, как принтеры прошлого повлияли на архитектуру систем печати в Unix-подобных машинах, и узнали, какие возможности предоставляет командная строка для управления не только расписанием и выполнением заданий, но и различными параметрами вывода.

Что почитать

- Хорошая статья о языке описания страниц PostScript:
<https://en.wikipedia.org/wiki/PostScript>
- Система печати Common Unix Printing System (CUPS):
<https://en.wikipedia.org/wiki/CUPS>
<https://openprinting.github.io/cups/>
- Системы печати Berkeley и System V:
https://en.wikipedia.org/wiki/Berkeley_printing_system
https://en.wikipedia.org/wiki/System_V_printing_system

23 — Компиляция программ

В этой главе мы разберем, как создавать программы путем компиляции исходного кода. Доступность исходного кода — это ключевая свобода, благодаря которой существует Linux. Вся экосистема разработки Linux опирается на свободный обмен знаниями между разработчиками. Для многих пользователей настольных систем компиляция стала забытым искусством. Раньше это было обычным делом, но сегодня поставщики дистрибутивов поддерживают огромные репозитории готовых бинарных файлов, которые можно сразу скачивать и использовать. На момент написания этой книги репозиторий Debian (один из крупнейших среди всех дистрибутивов) содержит более 68 000 пакетов.

Зачем компилировать ПО? На это есть две причины:

1. Доступность. Несмотря на обилие готовых программ в репозиториях дистрибутивов, некоторые из них могут не содержать нужных приложений. В таком случае единственный способ получить программу — скомпилировать её из исходного кода.
2. Актуальность. Хотя некоторые дистрибутивы специализируются на самых свежих версиях программ, многие — нет. Это значит, что для получения последней версии программы компиляция необходима.

Компиляция ПО из исходного кода может быть сложным техническим процессом, недоступным многим пользователям. Однако многие задачи по компиляции просты и включают всего несколько шагов. Всё зависит от пакета. Мы рассмотрим простой случай, чтобы дать обзор процесса и создать базу для тех, кто захочет изучать тему дальше.

Мы представим одну новую команду:

- `make` — утилита для обслуживания программ

Что такое компиляция?

Проще говоря, компиляция — это процесс перевода исходного кода (понятного человеку описания программы, написанного программистом) на машинный язык процессора.

Процессор компьютера (CPU) работает на элементарном уровне, выполняя программы на так называемом машинном языке. Это числовой код, описывающий мельчайшие операции, такие как «прибавьте этот байт», «укажите на это место в памяти» или

«скопируйте этот байт». Каждая такая инструкция выражается в двоичном виде (единицы и нули). Самые первые компьютерные программы писались с помощью этого числового кода, что, возможно, объясняет, почему программистов, писавших их, считали заядлыми курильщиками, любителями литров кофе и обладателями толстых очков.

Эту проблему решило появление языка ассемблера, который заменил числовые коды (чуть более) простыми для использования буквенными мнемониками, такими как CPY (для сору) и MOV (для move). Программы на языке ассемблера преобразуются в машинный язык программой, называемой ассемблером. Язык ассемблера до сих пор используется для определенных специализированных задач программирования, таких как драйверы устройств и встраиваемые системы.

Далее мы перейдем к так называемым языкам программирования высокого уровня. Они называются так потому, что позволяют программисту меньше беспокоиться о деталях работы процессора и больше — о решении текущей задачи. К ранним языкам (разработанным в 1950-х годах) относятся FORTRAN (предназначен для научных и технических задач) и COBOL (предназначен для бизнес-приложений). Оба до сих пор находят ограниченное применение.

Хотя существует множество популярных языков программирования, преобладают два. Большинство программ для современных систем пишутся либо на C, либо на C++. В следующих примерах мы будем компилировать программу на C.

Программы, написанные на языках высокого уровня, преобразуются в машинный язык с помощью другой программы — компилятора. Некоторые компиляторы переводят инструкции высокого уровня в язык ассемблера, а затем используют ассемблер для завершающего этапа преобразования в машинный язык.

Процесс, часто используемый вместе с компиляцией, называется компоновкой. Программы выполняют множество общих задач. Например, открытие файла. Эту задачу выполняют многие программы, но было бы нерационально внедрять в каждую из них собственный алгоритм открытия файлов. Логичнее иметь единый программный модуль, умеющий открывать файлы, и позволить всем нуждающимся программам использовать его. Поддержка общих задач обеспечивается так называемыми библиотеками. Они содержат множество подпрограмм, каждая из которых выполняет определенную общую задачу, доступную многим программам. Если заглянуть в каталоги /lib и /usr/lib,

можно увидеть, где многие из них расположены. Для установления связей между результатом работы компилятора и необходимыми скомпилированной программе библиотеками используется программа под названием компоновщик. Конечным результатом этого процесса является исполняемый файл программы, готовый к использованию.

Все ли программы компилируются?

Нет. Как мы уже видели, существуют программы вроде shell-скриптов, которые не требуют компиляции. Они выполняются напрямую. Такие программы пишутся на так называемых скриптовых или интерпретируемых языках. В последние годы их популярность растет; к ним относятся Perl, Python, PHP, Ruby и многие другие.

Скриптовые языки выполняются специальной программой, называемой интерпретатором. Интерпретатор принимает файл программы, считывает и выполняет каждую содержащуюся в нем инструкцию. Как правило, интерпретируемые программы работают гораздо медленнее компилируемых. Это связано с тем, что в интерпретируемой программе каждая инструкция исходного кода переводится заново при каждом выполнении, тогда как в компилируемой программе инструкция переводится лишь однажды, и этот перевод навсегда записывается в итоговый исполняемый файл.

Почему интерпретируемые языки так популярны? Для многих задач их скорости «достаточно», но главное преимущество в том, что разрабатывать интерпретируемые программы обычно быстрее и проще, чем компилируемые. Разработка обычно идет циклом: написание кода, компиляция, тестирование. По мере роста программы этап компиляции может сильно затянуться. Интерпретируемые языки исключают шаг компиляции, тем самым ускоряя разработку.

Компиляция программы на C

Давайте что-нибудь скомпилируем. Но прежде нам понадобятся такие инструменты, как компилятор, компоновщик и make.

Компилятор C, почти повсеместно используемый в среде Linux, называется gcc (GNU C Compiler), он был изначально написан

Ричардом Столлманом. Большинство дистрибутивов не устанавливают gcc по умолчанию. Проверить наличие компилятора можно так:

```
[me@linuxbox ~]$ which gcc
/usr/bin/gcc
```

Результаты этого примера указывают на то, что компилятор установлен.

Совет: в вашем дистрибутиве может быть метапакет (набор пакетов) для разработки ПО. Если это так, рассмотрите возможность его установки, если планируете компилировать программы в своей системе. Если метапакета нет, попробуйте установить пакеты `gcc` и `make`. Во многих дистрибутивах этого достаточно для выполнения следующего упражнения.

Получение исходного кода

Для упражнения по компиляции мы соберем программу `diction` из проекта GNU. Эта полезная утилита проверяет текстовые файлы на качество и стиль написания. По меркам программ она довольно мала и проста в сборке.

Следуя традиции, сначала мы создадим каталог для нашего исходного кода с именем `src`, а затем скачаем его туда с помощью `ftp`.

```
[me@linuxbox ~]$ mkdir src
[me@linuxbox ~]$ cd src
[me@linuxbox src]$ ftp ftp.gnu.org
Connected to ftp.gnu.org.
220 GNU FTP server ready.
Name (ftp.gnu.org:me): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> cd gnu/diction
250 Directory successfully changed.
ftp> ls
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
-rw-r--r-- 1 1003 65534 68940 Aug 28 1998 diction-0.7.tar.gz
-rw-r--r-- 1 1003 65534 90957 Mar 04 2002 diction-1.02.tar.gz
-rw-r--r-- 1 1003 65534 141062 Sep 17 2007 diction-1.11.tar.gz
226 Directory send OK.
ftp> get diction-1.11.tar.gz
local: diction-1.11.tar.gz remote: diction-1.11.tar.gz
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for diction-1.11.tar.gz
(141062 bytes).
226 File send OK.
141062 bytes received in 0.16 secs (847.4 kB/s)
ftp> bye
221 Goodbye.
[me@linuxbox src]$ ls
diction-1.11.tar.gz
```

Хотя в предыдущем примере мы использовали традиционный `ftp`, существуют и другие способы загрузки исходного кода. Например, проект GNU также поддерживает загрузку через

HTTPS. Мы можем скачать `diction` исходный код с помощью программы `curl`.

```
[me@linuxbox ~]$ curl -O https://ftp.gnu.org/gnu/diction/diction-1.11.tar.gz
```

% Total	% Received	% Xferd	Average	Speed	Time	Time	Time
Current			Dload	Upload	Total	Spent	Left
Speed							
100	137k	100	137k	0	0	652k	0
652k							

Примечание: Поскольку мы являемся «поддержкой» этого исходного кода во время его компиляции, мы будем хранить его в `~/src`. Исходный код, установленный вашим дистрибутивом, будет находиться в `/usr/src`, тогда как исходный код, который мы поддерживаем для использования несколькими пользователями, обычно устанавливается в `/usr/local/src`.

Как видно, исходный код обычно поставляется в виде сжатого tar-архива. Этот файл, называемый tarball, содержит дерево исходников — иерархию каталогов и файлов, составляющих код. Перейдя на FTP-сайт, мы изучаем список доступных tar-файлов и выбираем для загрузки самую новую версию. Используя команду `get` внутри `ftp`, мы копируем файл с FTP-сервера на локальную машину.

После того как tar-файл будет загружен, его необходимо распаковать. Это делается с помощью программы `tar`.

```
[me@linuxbox src]$ tar xzf diction-1.11.tar.gz
[me@linuxbox src]$ ls
diction-1.11      diction-1.11.tar.gz
```

Совет: Программа `diction`, как и все ПО проекта GNU, следует определенным стандартам упаковки исходного кода. Большинство других исходных кодов в экосистеме Linux также следуют этому стандарту. Один из элементов стандарта заключается в том, что при распаковке tar-файла создается каталог, содержащий дерево исходного кода, и этот каталог будет называться `project-x.xx`, включая имя проекта и его версию. Такая схема позволяет легко устанавливать несколько версий одной программы. Однако перед распаковкой полезно изучить структуру дерева. Некоторые проекты не создают каталог, а распаковывают файлы прямо в текущий

рабочий каталог. Это создаст беспорядок в нашем иначе хорошо организованном каталоге `src`. Чтобы избежать этого, используйте следующую команду для просмотра содержимого tar-файла:

```
tar tzvf tarfile | head
```

Изучение дерева исходного кода

Распаковка tar-файла создает новый каталог с именем `diction-1.11`. В этом каталоге находится дерево исходного кода. Давайте заглянем внутрь.

```
[me@linuxbox src]$ cd diction-1.11
[me@linuxbox diction-1.11]$ ls
config.guess  diction.c      getopt.c      nl
config.h.in   diction.pot    getopt.h      nl.po
config.sub    diction.spec   getopt_int.h  README
configure     diction.spec.in INSTALL      sentence.c
configure.in  diction.texi.in install-sh    sentence.h
COPYING       en             Makefile.in  style.1.in
de            en_GB          misc.c        style.c
de.po         en_GB.po       misc.h        test
diction.1.in  getopt1.c      NEWS
```

Здесь мы видим ряд файлов. Программы проекта GNU, а также многие другие, предоставляют файлы документации `README`, `INSTALL`, `NEWS` и `COPYING`. Эти файлы содержат описание программы, информацию о том, как её собрать и установить, а также условия лицензирования. Перед попыткой сборки программы всегда полезно внимательно прочитать файлы `README` и `INSTALL`.

Другие интересные файлы в этом каталоге — те, что заканчиваются на `.c` и `.h`.

```
[me@linuxbox diction-1.11]$ ls *.c
diction.c  getopt1.c  getopt.c  misc.c  sentence.c  style.c
[me@linuxbox diction-1.11]$ ls *.h
getopt.h  getopt_int.h  misc.h  sentence.h
```

`.c`-файлы содержат две программы на C, поставляемые в пакете (`style` и `diction`), разделенные на модули. Для больших программ принято разбивать код на более мелкие и удобные для управления части. Файлы исходного кода — это обычный текст, который можно просмотреть с помощью `less`.

```
[me@linuxbox diction-1.11]$ less diction.c
```

`.h`-файлы называются заголовочными файлами. Это тоже обычный текст. Заголовочные файлы содержат описания процедур, включенных в файл исходного кода или библиотеку.

Чтобы компилятор мог связать модули, он должен получить описание всех модулей, необходимых для сборки всей программы. В начале файла `diction.c` мы видим следующую строку:

```
#include "getopt.h"
```

Это указывает компилятору считывать файл `getopt.h` так же, как он считывает исходный код в `diction.c`, чтобы «знать», что содержится в `getopt.c`. Файл `getopt.c` содержит подпрограммы, общие для программ `style` и `diction`.

Перед оператором `include` для `getopt.h` мы видим другие операторы `include`, например:

```
#include <regex.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
```

Они также ссылаются на заголовочные файлы, но на те, что находятся вне текущего дерева исходного кода. Они поставляются системой для поддержки компиляции любой программы. Если заглянуть в `/usr/include`, их можно там увидеть.

```
[me@linuxbox diction-1.11]$ ls /usr/include
```

Заголовочные файлы в этом каталоге были установлены вместе с компилятором.

Сборка программы

Большинство программ собирается простой последовательностью из двух команд.

```
./configure
make
```

Программа `configure` — это shell-скрипт, входящий в состав исходного кода. Её задача — проанализировать среду сборки. Большинство исходных кодов портативны, то есть предназначены для сборки в различных Unix-подобных системах. Однако для этого при сборке может потребоваться небольшая корректировка кода с учётом различий между системами. `configure` также проверяет наличие необходимых внешних инструментов и компонентов. Давайте запустим `configure`. Поскольку `configure` находится не там, где оболочка обычно ищет программы, мы должны явно указать оболочке её

расположение, добавив перед командой `./`, чтобы обозначить, что программа находится в текущем рабочем каталоге.

```
[me@linuxbox diction-1.11]$ ./configure
```

`configure` выведет множество сообщений в процессе тестирования и настройки сборки. По завершении результат будет выглядеть примерно так:

```
checking libintl.h presence... yes
checking for libintl.h... yes
checking for library containing gettext... none required
configure: creating ./config.status
config.status: creating Makefile
config.status: creating diction.1
config.status: creating diction.texi
config.status: creating diction.spec
config.status: creating style.1
config.status: creating test/rundiction
config.status: creating config.h
[me@linuxbox diction-1.11]$
```

Здесь важно, что сообщений об ошибках нет. Если бы они появились, это означало бы сбой конфигурации, и программа не соберется, пока ошибки не будут исправлены.

Мы видим, что `configure` создал несколько новых файлов в нашем каталоге с исходным кодом. Самый важный из них — `makefile`. `Makefile` — это конфигурационный файл, который точно указывает `make` программе, как именно собирать программу. Без него `make` откажется запускаться. `Makefile` — обычный текстовый файл, поэтому мы можем просмотреть его:

```
[me@linuxbox diction-1.11]$ less Makefile
```

`make` программа принимает на вход `makefile` (который обычно называется `Makefile`), описывающий связи и зависимости между компонентами, из которых состоит готовая программа.

Первая часть `makefile` определяет переменные, которые подставляются в последующих разделах. Например, мы видим следующую строку:

```
CC= gcc
```

Это определяет `gcc` как компилятор `C`. Позже в `makefile` мы увидим один пример его использования.

```
diction: diction.o sentence.o misc.o getopt.o getopt1.o
$(CC) -o $@ $(LD_FLAGS) diction.o sentence.o misc.o \
getopt.o getopt1.o $(LIBS)
```


Здесь выполняется подстановка, и значение `$(CC)` заменяется на `gcc` во время выполнения.

Большая часть `makefile` состоит из строк, определяющих цель — в данном случае исполняемый файл `diction` — и файлы, от которых она зависит. Остальные строки описывают команды, необходимые для создания цели из её компонентов. В этом примере видно, что исполняемый файл `diction` (один из конечных продуктов) зависит от наличия `diction.o`, `sentence.o`, `misc.o`, `getopt.o` и `getopt1.o`. Позже в `makefile` мы увидим определения каждого из этих файлов как целей.

```
diction.o:    diction.c config.h getopt.h misc.h sentence.h
getopt.o:     getopt.c getopt.h getopt_int.h
getopt1.o:    getopt1.c getopt.h getopt_int.h
misc.o:       misc.c config.h misc.h
sentence.o:   sentence.c config.h misc.h sentence.h
style.o:      style.c config.h getopt.h misc.h sentence.h
```

Однако для них не указана ни одна команда. Это обрабатывается общей целью, описанной ранее в файле, которая определяет команду для компиляции любого файла `.c` в файл `.o`.

```
.c.o:
      $(CC) -c $(CPPFLAGS) $(CFLAGS) $<
```

Все это кажется очень сложным. Почему бы просто не перечислить все шаги компиляции и не закончить на этом? Ответ станет ясен через мгновение. А пока давайте запустим `make` и соберем наши программы.

```
[me@linuxbox diction-1.11]$ make
```

Программа `make` будет выполняться, ориентируясь на содержимое `Makefile`. Она выведет много сообщений.

По завершении мы увидим, что все целевые файлы появились в нашем каталоге.

```
[me@linuxbox diction-1.11]$ ls
config.guess  de.po          en              install-sh     sentence.c
config.h      diction        en_GB          Makefile       sentence.h
config.h.in   diction.1     en_GB.mo      Makefile.in    sentence.o
config.log    diction.1.in  en_GB.po      misc.c         style
config.status diction.c     getopt1.c     misc.h         style.1
config.sub    diction.o     getopt1.o     misc.o         style.1.in
configure     diction.pot   getopt.c      NEWS          style.c
configure.in  diction.spec  getopt.h      nl            style.o
COPYING       diction.spec.in getopt_int.h  nl.mo         test
de            diction.texi  getopt.o      nl.po
de.mo         diction.texi.in INSTALL      README
```

Среди файлов мы видим `diction` и `style` — программы, которые мы собирались создать. Поздравляем! Мы только что

скомпилировали наши первые программы из исходного кода!

Но просто из любопытства давайте снова запустим make.

```
[me@linuxbox diction-1.11]$ make
make: Nothing to be done for `all'.
```

Она выдает лишь это странное сообщение. Что происходит? Почему программа не собралась заново? А, в этом и заключается магия make. Вместо того чтобы просто пересобирать всё подряд, make собирает только то, что необходимо. Поскольку все цели на месте, make решил, что делать нечего. Мы можем проверить это, удалив одну из целей и снова запустив make. Давайте избавимся от одной из промежуточных целей.

```
[me@linuxbox diction-1.11]$ rm getopt.o
[me@linuxbox diction-1.11]$ make
```

Мы видим, что make пересобирает её и заново компонует программы `diction` и `style`, так как они зависят от отсутствующего модуля. Такое поведение также указывает на ещё одну важную особенность make: она поддерживает цели в актуальном состоянии. make требует, чтобы цели были новее своих зависимостей. Это логично, ведь программист часто обновляет часть исходного кода, а затем использует make для сборки новой версии продукта. make гарантирует, что всё, что требует пересборки на основе обновленного кода, будет собрано. Если мы воспользуемся программой `touch`, чтобы «обновить» один из файлов исходного кода, мы увидим следующее:

```
[me@linuxbox diction-1.11]$ ls -l diction getopt.c
-rwxr-xr-x 1 me      me      37164 2025-03-05 06:14 diction
-rw-r--r-- 1 me      me      33125 2007-03-30 17:45 getopt.c
[me@linuxbox diction-1.11]$ touch getopt.c
[me@linuxbox diction-1.11]$ ls -l diction getopt.c
-rwxr-xr-x 1 me      me      37164 2025-03-05 06:14 diction
-rw-r--r-- 1 me      me      33125 2025-03-05 06:23 getopt.c
[me@linuxbox diction-1.11]$ make
```

После make запусков мы видим, что целевой файл стал новее зависимости:

```
[me@linuxbox diction-1.11]$ ls -l diction getopt.c
-rwxr-xr-x 1 me      me      37164 2025-03-05 06:24 diction
-rw-r--r-- 1 me      me      33125 2025-03-05 06:23 getopt.c
```

Способность make интеллектуально собирать только то, что необходимо, — огромное преимущество для программистов. Хотя в нашем небольшом проекте экономия времени неочевидна, в крупных проектах она весьма существенна. Помните, что ядро Linux (программа, которая постоянно

модифицируется и совершенствуется) содержит несколько миллионов строк кода.

Установка программы

Хорошо упакованный исходный код часто включает специальную make цель под названием `install`. Эта цель установит готовый продукт в системный каталог для использования. Обычно это каталог `/usr/local/bin` — традиционное место для локально собранного ПО. Однако обычные пользователи обычно не имеют прав на запись в этот каталог, поэтому для выполнения установки мы должны стать суперпользователем.

```
[me@linuxbox diction-1.11]$ sudo make install
```

После завершения установки мы можем проверить, что программа готова к работе.

```
[me@linuxbox diction-1.11]$ which diction
/usr/local/bin/diction
[me@linuxbox diction-1.11]$ man diction
```

Вот и всё!

Итоги

В этой главе мы рассмотрели три

простые команды: `./configure`

`make`

`make install`

можно использовать для сборки множества пакетов исходного кода. Мы также увидели важную роль, которую make играет в обслуживании программ. Программу make можно использовать для любых задач, требующих поддержания связей между целями и зависимостями, а не только для компиляции исходного кода.

Что почитать

- В Wikipedia есть хорошие статьи о компиляторах и программе make: <https://en.wikipedia.org/wiki/Compiler>
[https://en.wikipedia.org/wiki/Make_\(software\)](https://en.wikipedia.org/wiki/Make_(software))
- Руководство GNU Make:
https://www.gnu.org/software/make/manual/html_node/index.html

Часть 4 — Написание сценариев оболочки

24 — Написание вашего первого скрипта

В предыдущих главах мы собрали целый арсенал инструментов командной строки. Хотя эти инструменты решают множество вычислительных задач, мы всё ещё ограничены их ручным использованием по одному в командной строке. Было бы здорово, если бы оболочка могла выполнять больше работы за нас? Это возможно. Объединяя инструменты в программы собственной разработки, мы позволяем оболочке самостоятельно выполнять сложные последовательности задач. Мы можем сделать это с помощью написания скриптов оболочки.

Что такое скрипты оболочки?

Проще говоря, скрипт оболочки — это файл, содержащий серию команд. Оболочка читает этот файл и выполняет команды так, будто они были введены непосредственно в командной строке.

Оболочка уникальна тем, что она является одновременно и мощным интерфейсом командной строки, и интерпретатором языка сценариев. Как мы увидим, почти всё, что можно сделать в командной строке, можно сделать в скриптах, и наоборот.

Мы рассмотрели множество возможностей оболочки, сосредоточившись на тех, что чаще всего используются непосредственно в командной строке. Оболочка также предоставляет набор функций, которые обычно (хотя и не всегда) применяются при написании программ.

Как писать скрипты оболочки

Чтобы успешно создать и запустить скрипт оболочки, нужно сделать три вещи.

1. Написать скрипт. Скрипты оболочки — это обычные текстовые файлы. Поэтому для их написания нужен текстовый редактор. Лучшие редакторы поддерживают подсветку синтаксиса, позволяя видеть элементы скрипта в цвете. Это поможет быстрее находить типичные ошибки. `vim`, `gedit`, `kate` и многие другие редакторы отлично подходят для написания скриптов.
2. Сделайте скрипт исполняемым. Система довольно строго следит за тем, чтобы любой текстовый файл не принимали за программу, и на то есть веские причины! Нам нужно

установить права доступа к файлу скрипта, разрешающие выполнение.

3. Поместите скрипт туда, где его найдет оболочка. Если явный путь не указан, оболочка автоматически ищет исполняемые файлы в определенных каталогах. Для максимального удобства мы разместим наши скрипты именно в этих каталогах.

Формат файла скрипта

Следуя программистской традиции, мы создадим программу «Hello World», чтобы продемонстрировать простейший скрипт. Запустим текстовый редактор и введем следующий скрипт:

```
#!/bin/bash
# This is our first script.
echo 'Hello World!'
```

Последняя строка нашего скрипта вполне знакома — это просто `echo` команда со строковым аргументом. Вторая строка тоже узнаваема. Она выглядит как комментарий, который мы видели во многих изученных и отредактированных нами конфигурационных файлах. Особенность комментариев в скриптах оболочки заключается в том, что они могут располагаться и в конце строки, если перед ними стоит хотя бы один символ пробела, вот так:

```
echo 'Hello World!' # This is a comment too
```

Все, что идет в строке после символа `#`, игнорируется.

Как и многое другое, это работает и в командной строке:

```
[me@linuxbox ~]$ echo 'Hello World!' # This is a comment too
Hello World!
```

Хотя в командной строке комментарии малополезны, они будут работать.

Первая строка нашего скрипта выглядит загадочно. Она напоминает комментарий, так как начинается с `#`, но кажется слишком важной для простого комментария.

Последовательность символов `#!` на самом деле является специальной конструкцией под названием `shebang`. `Shebang` сообщает ядру имя интерпретатора, который должен выполнить последующий скрипт. Каждый скрипт оболочки должен содержать эту строку первой.

Сохраним файл скрипта как `hello_world`.

Права на выполнение

Следующий шаг — сделать скрипт исполняемым. Это легко сделать с помощью `chmod`.

```
[me@linuxbox ~]$ ls -l hello_world
-rw-r--r-- 1 me      me      63 2025-03-07 10:10 hello_world
[me@linuxbox ~]$ chmod 755 hello_world
[me@linuxbox ~]$ ls -l hello_world
-rwxr-xr-x 1 me      me      63 2025-03-07 10:10 hello_world
```

Для скриптов обычно используют два типа прав: 755 — для выполнения всеми пользователями, и 700 — только владельцем. Примечание: для выполнения скрипт должен быть доступен для чтения.

Расположение файла скрипта

Теперь, когда права установлены, мы можем запустить наш скрипт.

```
[me@linuxbox ~]$ ./hello_world
Hello World!
```

Чтобы скрипт запустился, перед его именем нужно указать явный путь. Если этого не сделать, мы получим следующее:

```
[me@linuxbox ~]$ hello_world
bash: hello_world: command not found
```

Почему так? Чем наш скрипт отличается от других программ? Как выяснилось, ничем. С самим скриптом всё в порядке. Проблема в его расположении. В главе 11 мы обсуждали переменную окружения `PATH` и её влияние на поиск исполняемых программ. Напомним: если явный путь не указан, система каждый раз ищет исполняемую программу в списке каталогов. Именно так система понимает, что нужно запустить `/bin/ls` `ls /bin`, когда мы вводим команду в командной строке. Этот каталог — один из тех, что система проверяет автоматически. Список каталогов хранится в переменной окружения `PATH`. Переменная `PATH` содержит список каталогов для поиска, разделенных двоеточием. Мы можем просмотреть содержимое `PATH`.

```
[me@linuxbox ~]$ echo $PATH
/home/me/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games
```

Перед нами список каталогов. Если бы наш скрипт находился в любом из них, проблема была бы решена. Обратите внимание на

первый каталог в списке, `/home/me/bin` `PATH` `bin`. В большинстве дистрибутивов Linux эта переменная настроена на каталог в домашнем каталоге пользователя, чтобы он мог запускать свои программы. Таким образом, если мы создадим каталог `bin` и поместим туда скрипт, он будет работать как и другие программы.

```
[me@linuxbox ~]$ mkdir bin
[me@linuxbox ~]$ mv hello_world bin
[me@linuxbox ~]$ hello_world
Hello World!
```

Так оно и происходит.

Если переменная `PATH` не содержит нужный каталог, мы можем легко добавить его, вставив эту строку в файл `.bashrc`:

```
export PATH=~/bin:$PATH
```

После внесения изменений они вступят в силу в каждой новой сессии терминала. Чтобы применить изменения в текущей сессии, нужно заставить оболочку перечитать файл `.bashrc`. Это можно сделать с помощью команды «sourcing».

```
[me@linuxbox ~]$ . .bashrc
```

Команда `dot` (`.`) является синонимом команды `source` — встроенной команды оболочки, которая считывает указанный файл с командами оболочки и обрабатывает его как ввод с клавиатуры.

Примечание: Ubuntu (и большинство других Debian-подобных дистрибутивов) автоматически добавляет `~/bin` каталог в переменную `PATH`, если `~/bin` каталог существует в момент выполнения `.bashrc` файла пользователя. Таким образом, в системах Ubuntu, если мы создадим `~/bin` каталог, а затем выйдем из системы и войдем снова, всё будет работать.

Удачные места для размещения скриптов

`~/bin` каталог — подходящее место для скриптов личного пользования. Если мы пишем скрипт, который разрешен использовать всем пользователям системы, традиционным местом является `/usr/local/bin`. Скрипты, предназначенные для системного администратора, часто находятся в `/usr/local/sbin`. В большинстве случаев локальное ПО, будь то скрипты или скомпилированные программы, следует размещать

в иерархии `/usr/local`, а не в `/bin` или `/usr/bin`.. Согласно стандарту Linux Filesystem Hierarchy Standard, эти каталоги предназначены только для файлов, поставляемых и поддерживаемых дистрибьютором Linux.

Дополнительные приемы форматирования

Одной из ключевых целей серьезного написания скриптов является простота обслуживания — то есть легкость, с которой автор или другие лица могут изменить скрипт для адаптации к меняющимся потребностям. Сделать скрипт легко читаемым и понятным — один из способов упростить его обслуживание.

Длинные имена опций

Многие изученные нами команды имеют как короткие, так и длинные имена опций. Например, команда `ls` имеет множество опций, которые можно выразить в короткой или длинной форме. Рассмотрим следующий пример:

```
[me@linuxbox ~]$ ls -ad
```

эквивалентно следующему:

```
[me@linuxbox ~]$ ls --all --directory
```

Чтобы меньше печатать, при вводе параметров в командной строке лучше использовать короткие опции, но в скриптах длинные опции повышают читаемость.

Отступы и перенос строк

При использовании длинных команд читаемость можно улучшить, разбив команду на несколько строк. В главе 17 мы рассматривали особенно длинный пример команды `find`.

```
[me@linuxbox ~]$ find playground \( -type f -not -perm 0600 -exec  
chmod 0600 '{}' ';' \) -or \( -type d -not -perm 0700 -exec chmod  
0700 '{}' ';' \)
```

Очевидно, что с первого взгляда эту команду трудно понять. В скрипте ее будет проще прочитать, если записать так:

```
find playground \  
  \( \  
    -type f \  
    -not -perm 0600 \  
    -exec chmod 0600 '{}' ';' \  
  ) \  
  -or \  
  \( \  
    -type d \  
    -not -perm 0700 \  
  )
```

```
-exec chmod 0700 '{}' ';' \  
\\)
```

Использование переносов строк (последовательности обратный слэш-перевод строки) и отступов позволяет более наглядно описать логику этой сложной команды. Этот прием работает и в командной строке, хотя его редко используют из-за неудобства ввода и редактирования. Различие между скриптом и командной строкой заключается в том, что в скрипте для отступов можно использовать символы табуляции, а в командной строке — нет, так как табуляция служит для активации автодополнения.

Настройка vim для написания скриптов

Текстовый редактор `vim` имеет множество настроек конфигурации. Некоторые из них могут облегчить написание скриптов.

Следующая команда включает подсветку синтаксиса:

```
:syntax on
```

С этой настройкой элементы синтаксиса оболочки будут отображаться разными цветами при просмотре скрипта. Это помогает выявлять определенные ошибки программирования. Выглядит тоже круто. Примечание: для работы этой функции должна быть установлена полная версия `vim`, а редактируемый файл должен содержать `shebang`, указывающий, что файл является скриптом оболочки. Если возникли трудности с предыдущей командой, попробуйте вместо неё `:set syntax=sh`.

Следующая команда включает подсветку результатов поиска.

```
:set hlsearch
```

Допустим, мы ищем слово `echo`. С этой опцией каждое вхождение слова будет подсвечено.

Следующая команда задает количество столбцов, занимаемых символом табуляции:

```
:set tabstop=4
```

По умолчанию это восемь столбцов. Установка значения 4 (что является общепринятой практикой) позволяет длинным строкам легче уместиться на экране.

Следующая команда включает функцию «автоотступа»:

```
:set autoindent
```

Это заставляет `vim` делать отступ на новой строке таким же, как у только что набранной строки. Это ускоряет ввод многих конструкций программирования. Чтобы прекратить отступы, нажмите `ctrl-d`.

Эти изменения можно сделать постоянными, добавив данные команды (без начальных двоеточий) в ваш файл `~/.vimrc`.

Итоги

В этой первой главе по написанию скриптов мы рассмотрели, как пишутся скрипты и как обеспечить их легкое выполнение в системе. Мы также узнали, как использовать различные методы форматирования для улучшения читаемости (и, следовательно, удобства сопровождения) наших скриптов. В будущих главах простота сопровождения будет снова и снова упоминаться как центральный принцип написания хороших скриптов.

Что почитать

- Про программы «Hello World» и примеры на различных языках программирования см.:
https://en.wikipedia.org/wiki/Hello_world
- Подробнее о механизме shebang читайте в этой статье Wikipedia: [https://en.wikipedia.org/wiki/Shebang_\(Unix\)](https://en.wikipedia.org/wiki/Shebang_(Unix))

25 — Начало проекта

Начиная с этой главы, мы приступим к написанию программы. Цель проекта — увидеть, как возможности оболочки помогают создавать программы и, что более важно, создавать качественные программы.

Мы напишем программу для генерации отчетов. Она будет представлять различные статистические данные о состоянии нашей системы и формировать отчет в формате HTML, чтобы его можно было просмотреть в веб-браузере, например Firefox или Chrome.

Программы обычно создаются поэтапно, где каждый новый этап добавляет функции и возможности. На первом этапе наша программа будет создавать минимальный HTML-документ без системной информации. Это мы добавим позже.

Первый этап: минимальный документ

Прежде всего нам нужно изучить формат корректного HTML-документа. Он выглядит так:

```
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    Page body.
  </body>
</html>
```

Если ввести это в текстовый редактор и сохранить файл как `foo.html`, то в Firefox можно использовать следующий URL для просмотра файла:

```
file:///home/username/foo.html
```

На первом этапе наша программа сможет выводить этот HTML-файл в стандартный вывод. Написать такую программу довольно просто. Запустите текстовый редактор и создайте новый файл с именем `~/bin/sys_info_page`.

```
[me@linuxbox ~]$ vim ~/bin/sys_info_page
```

Введите следующую программу:

```
#!/bin/bash
# Program to output a system information page
echo "<html>"
echo "  <head>"
echo "    <title>Page Title</title>"
echo "  </head>"
```

```
echo "    <body>"
echo "        Page body."
echo "    </body>"
echo "</html>"
```

Наша первая попытка решения этой задачи содержит shebang, комментарий (всегда полезно) и последовательность echo команд, по одной на каждую строку вывода. После сохранения файла мы сделаем его исполняемым и попробуем запустить.

```
[me@linuxbox ~]$ chmod 755 ~/bin/sys_info_page
[me@linuxbox ~]$ sys_info_page
```

При запуске программы на экране должен появиться текст HTML-документа, так как echo команды в скрипте отправляют вывод в стандартный вывод. Мы запустим программу снова и используем перенаправление, чтобы направить вывод в файл sys_info_page.html, — так мы сможем просмотреть результат в веб-браузере.

```
[me@linuxbox ~]$ sys_info_page > sys_info_page.html
[me@linuxbox ~]$ firefox sys_info_page.html
```

Пока всё идет хорошо.

При написании программ всегда стоит стремиться к простоте и ясности. Поддерживать программу легче, если она понятна и читаема, не говоря уже о том, что это упрощает само написание, сокращая объем ввода. Текущая версия программы работает исправно, но её можно упростить. Мы можем объединить все команды echo в одну, что определенно облегчит добавление новых строк в вывод программы. Итак, изменим нашу программу следующим образом:

```
#!/bin/bash
# Program to output a system information page
echo "<html>
    <head>
        <title>Page Title</title>
    </head>
    <body>
        Page body.
    </body>
</html>"
```

Строка в кавычках может содержать символы новой строки и, следовательно, несколько строк текста. Оболочка будет считывать текст до тех пор, пока не встретит закрывающую кавычку. В командной строке это работает так же:

```
[me@linuxbox ~]$ echo "<html>
>     <head>
>         <title>Page Title</title>
>     </head>
```

```
>         <body>
>         Page body.
>         </body>
> </html>"
```

Первый символ «>» — это приглашение оболочки, хранящееся в переменной PS2 shell. Оно появляется каждый раз, когда мы вводим в оболочку многострочную команду. Сейчас эта особенность малозаметна, но позже, когда мы разберем многострочные конструкции программирования, она окажется весьма полезной.

Второй этап: добавление данных

Теперь, когда наша программа умеет создавать минимальный документ, добавим в отчет данные. Для этого мы внесем следующие изменения:

```
#!/bin/bash
# Program to output a system information page
echo "<html>
    <head>
        <title>System Information Report</title>
    </head>
    <body>
        <h1>System Information Report</h1>
    </body>
</html>"
```

Мы добавили заголовок страницы и заголовок в тело отчета.

Переменные и константы

Однако в нашем скрипте есть проблема. Заметили, что строка «System Information Report» повторяется? В таком маленьком скрипте это не критично, но представьте, что он очень длинный и эта строка встречается многократно. Чтобы изменить заголовок, нам пришлось бы править его в разных местах, что трудоемко. А что, если организовать скрипт так, чтобы строка встречалась лишь один раз? Это значительно упростит поддержку кода в будущем. Вот как это можно сделать:

```
#!/bin/bash
# Program to output a system information page
title="System Information Report"
echo "<html>
    <head>
        <title>$title</title>
    </head>
    <body>
        <h1>$title</h1>
    </body>
</html>"
```

Создав переменную с именем `title System Information Report` и присвоив ей значение, мы сможем использовать подстановку параметров и разместить строку в нескольких местах.

Как же создать переменную? Просто используйте её. Когда оболочка встречает переменную, она создает её автоматически. Это отличает её от многих языков программирования, где переменные нужно явно объявлять или определять перед использованием. Оболочка очень либеральна в этом вопросе, что может привести к проблемам. Рассмотрим, например, такой сценарий в командной строке:

```
[me@linuxbox ~]$ foo="yes"
[me@linuxbox ~]$ echo $foo
yes
[me@linuxbox ~]$ echo $fool
[me@linuxbox ~]$
```

Сначала мы присваиваем значение `yes` переменной `foo`, а затем выводим его с помощью `echo`. Затем мы пытаемся вывести значение переменной с опечаткой — `fool`, и получаем пустой результат. Это происходит потому, что оболочка при встрече с `fool` просто создала новую переменную и присвоила ей значение по умолчанию — пустое. Из этого следует, что нужно внимательно следить за правописанием! Также важно понимать, что именно произошло в этом примере. Зная, как оболочка выполняет раскрытие, мы понимаем, что следующая команда:

```
[me@linuxbox ~]$ echo $foo
```

проходит через раскрытие параметров и дает следующий результат:

```
[me@linuxbox ~]$ echo yes
```

Напротив, следующая команда:

```
[me@linuxbox ~]$ echo $fool
```

раскрывается так:

```
[me@linuxbox ~]$ echo
```

Пустая переменная разворачивается в пустоту! Это может нарушить работу команд, требующих аргументы. Вот пример:

```
[me@linuxbox ~]$ foo=foo.txt
[me@linuxbox ~]$ foo1=foo1.txt
[me@linuxbox ~]$ cp $foo $fool
cp: missing destination file operand after `foo.txt'
Try `cp --help' for more information.
```

Мы присваиваем значения двум переменным, `foo` и `foo1`. Затем мы выполняем `sr`, но ошибаемся в имени второго аргумента (используя строчную `L` вместо цифры `1`). После раскрытия команде `sr` передается только один аргумент, хотя ей требуется два.

Существуют правила именования переменных:

1. Имена переменных могут состоять из букв, цифр и символов подчеркивания.
2. Первый символ имени переменной должен быть буквой или подчеркиванием.
3. Пробелы и знаки препинания не допускаются. 3. Spaces and punctuation symbols are not allowed.

Слово «переменная» подразумевает значение, которое меняется, и во многих приложениях переменные используются именно так. Однако переменная в нашем приложении, `title`, используется как константа. Константа похожа на переменную тем, что у нее есть имя и она содержит значение. Разница лишь в том, что значение константы не меняется. В приложении для геометрических вычислений мы могли бы определить `PI` как константу и присвоить ей значение `3.1415`, вместо того чтобы использовать число буквально по всей программе. Оболочка не делает различий между переменными и константами; они служат в основном для удобства программиста. общепринятая практика — использовать заглавные буквы для обозначения констант и строчные — для обычных переменных. Мы изменим наш скрипт, чтобы соответствовать этому соглашению:

```
#!/bin/bash
# Program to output a system information page
TITLE="System Information Report For $HOSTNAME"
echo "<html>
    <head>
        <title>$TITLE</title>
    </head>
    <body>
        <h1>$TITLE</h1>
    </body>
</html>"
```

Мы также решили украсить заголовок, добавив значение переменной оболочки `HOSTNAME`. Это сетевое имя машины.

Примечание: оболочка действительно позволяет сделать константы неизменяемыми с помощью встроенной команды `declare` с опцией `-r` (read-only). Если бы мы присвоили `TITLE` таким образом:


```
declare -r TITLE="Page Title"
```

оболочка предотвратит любое последующее присваивание для `TITLE`. Эта функция используется редко, но она существует для строго формализованных скриптов.

Присваивание значений переменным и константам

Здесь наши знания о раскрытии начинают приносить реальную пользу. Как мы уже видели, переменным присваиваются значения следующим образом, где `variable` — это имя переменной, а `value` — строка:

```
variable=value
```

В отличие от некоторых других языков программирования, оболочке не важен тип данных, присваиваемых переменной; она рассматривает их все как строки. Вы можете заставить оболочку ограничить присваивание целыми числами, используя команду `declare` с опцией `-i`, но, как и в случае с константами, это делается редко.

Примечание: при присваивании между именем переменной, знаком равенства и значением не должно быть пробелов. Из чего может состоять значение? Из всего, что можно развернуть в строку.

```
a=z          # Assign the string "z" to variable a.
b="a string"  # Embedded spaces must be within quotes.
c="a string and $b" # Other expansions such as variables can be
                # expanded into the assignment.
d="$(ls -l foo.txt)" # Results of a command.
e=$((5 * 7))    # Arithmetic expansion.
f="\t\ta string\n" # Escape sequences such as tabs and newlines.
```

Можно присвоить несколько переменных в одной строке.

```
a=5 b="a string"
```

При раскрытии имени переменных можно заключать в необязательные фигурные скобки, `{}`. Это полезно, если имя переменной становится неоднозначным из-за окружающего контекста. Здесь мы попробуем изменить имя файла с `myfile` на `myfile1`, используя переменную:

```
[me@linuxbox ~]$ filename="myfile"
[me@linuxbox ~]$ touch "$filename"
[me@linuxbox ~]$ mv "$filename" "$filename1"
mv: missing destination file operand after `myfile'
Try `mv --help' for more information.
```

Эта попытка провалится, так как оболочка интерпретирует второй аргумент команды `mv` как новую (пустую) переменную. Проблему можно решить так:

```
[me@linuxbox ~]$ mv "$filename" "${filename}1"
```

Добавив фигурные скобки, мы запрещаем оболочке воспринимать завершающую 1 как часть имени переменной.

Примечание: Хорошей практикой считается заключение переменных и подстановок команд в двойные кавычки, чтобы ограничить влияние разделения слов оболочкой. Кавычки особенно важны, если переменная может содержать имя файла.

Воспользуемся этой возможностью, чтобы добавить в наш отчет данные: дату и время его создания, а также имя пользователя, создавшего отчет.

```
#!/bin/bash
# Program to output a system information page
TITLE="System Information Report For $HOSTNAME"
CURRENT_TIME="$(date +%x %r %Z)"
TIMESTAMP="Generated $CURRENT_TIME, by $USER"
echo "<html>
    <head>
        <title>$TITLE</title>
    </head>
    <body>
        <h1>$TITLE</h1>
        <p>$TIMESTAMP</p>
    </body>
</html>"
```

Here Documents

Мы рассмотрели два способа вывода текста с помощью команды `echo`. Существует и третий вариант — `here document` или `here script`. `Here document` — это разновидность перенаправления ввода-вывода, позволяющая встроить текст прямо в скрипт и передать его на стандартный ввод команды. Это работает так:

```
command << token
```

```
text
```

```
token
```

В этом примере `command` — это имя команды, принимающей стандартный ввод, а `token` — строка, обозначающая конец встроеного текста. Изменим наш скрипт, чтобы использовать `here document`:

```
#!/bin/bash
# Program to output a system information page
TITLE="System Information Report For $HOSTNAME"
CURRENT_TIME="$(date +"%x %r %Z")"
TIMESTAMP="Generated $CURRENT_TIME, by $USER"
cat << _EOF_
<html>
    <head>
        <title>$TITLE</title>
    </head>
    <body>
        <h1>$TITLE</h1>
        <p>$TIMESTAMP</p>
    </body>
</html>
_EOF_
```

Вместо использования `echo` наш скрипт теперь использует `cat` и `here document`. Строка `_EOF_` (что означает конец файла — общепринятое соглашение) была выбрана в качестве маркера и обозначает конец встроенного текста. Примечание: маркер должен стоять отдельно, и в конце строки не должно быть пробелов.

В чем преимущество использования `here document`? В основном оно совпадает с `echo`, за исключением того, что по умолчанию одиночные и двойные кавычки внутри `here document` теряют свое специальное значение для оболочки. Вот пример в командной строке:

```
[me@linuxbox ~]$ foo="some text"
[me@linuxbox ~]$ cat << _EOF_
> $foo
> "$foo"
> '$foo'
> \ $foo
> _EOF_
some text
"some text"
'some text'
$foo
```

Как мы видим, оболочка не обращает внимания на кавычки. Она воспринимает их как обычные символы. Это позволяет свободно вставлять кавычки в `here document`, что может оказаться полезным для нашей программы отчетов.

Текст внутри `here document` проходит через раскрытие параметров, подстановку команд и арифметическое раскрытие, а символы `$` и `\` должны быть экранированы с помощью `\`.

Однако, если заключить начальный токен в кавычки,

```
cat << '_EOF_'
```

кавычки будут удалены из токена, и раскрытие текста не будет выполнено:

```
[me@linuxbox ~]$ foo="some text"
[me@linuxbox ~]$ cat << '_EOF_'
> $foo
> "$foo"
> '$foo'
> \ $foo
> _EOF_
$foo
"$foo"
'$foo'
\ $foo
```

Here documents можно использовать с любой командой, принимающей стандартный ввод. В этом примере мы используем here document, чтобы передать серию команд программе ftp для получения файла с удаленного FTP-сервера:

```
#!/bin/bash
# Script to retrieve a file via FTP
FTP_SERVER="ftp.nl.debian.org"
FTP_PATH="/debian/dists/bookworm/main/installer-amd64/current/images/cdrom"
REMOTE_FILE="debian-cd_info.tar.gz"
ftp -n << _EOF_
open $FTP_SERVER
user anonymous me@linuxbox
cd $FTP_PATH
hash
get $REMOTE_FILE
bye
_EOF_
ls -l "$REMOTE_FILE"
```

Если заменить оператор перенаправления с << на <<-, оболочка будет игнорировать начальные символы табуляции (но не пробелы) в here document. Это позволяет делать отступы в here document, что улучшает читаемость.

```
#!/bin/bash
# Script to retrieve a file via FTP
FTP_SERVER="ftp.nl.debian.org"
FTP_PATH="/debian/dists/bookworm/main/installer-amd64/current/images/cdrom"
REMOTE_FILE="debian-cd_info.tar.gz"
ftp -n <<- _EOF_
    open $FTP_SERVER
    user anonymous me@linuxbox
    cd $FTP_PATH
    hash
    get $REMOTE_FILE
    bye
_EOF_
ls -l "$REMOTE_FILE"
```

Имейте в виду, что эта функция может быть проблематичной, так как многие текстовые редакторы (и сами программисты)

предпочитают использовать пробелы вместо табуляции для создания отступов в скриптах.

Итоги

В этой главе мы начали проект, который проведет нас через процесс создания успешного скрипта. Мы изучили понятия переменных и констант, а также способы их использования. Это лишь первые из многих примеров применения раскрытия параметров. Мы также рассмотрели, как выводить данные из скрипта и различные методы вставки блоков текста.

Что почитать

- Для получения дополнительной информации об HTML см. следующие статьи и руководства:
<https://en.wikipedia.org/wiki/Html>
https://en.wikibooks.org/wiki/HTML_Programming
- В странице `man bash` есть раздел «Here Documents» с полным описанием этой функции.

26 — Сверху вниз

Чем крупнее и сложнее становятся программы, тем труднее их проектировать, писать и поддерживать. Как и в любом крупном проекте, здесь полезно разбивать сложные задачи на ряд мелких и простых. Представим, что мы пытаемся объяснить жителю Марса обычное дело — поход на рынок за продуктами. Весь процесс можно описать следующей последовательностью шагов:

1. Сесть в машину.
2. Поехать на рынок.
3. Припарковать машину.
4. Войти на рынок.
5. Купить еду.
6. Вернуться к машине.
7. Поехать домой.
8. Припарковать машину.
9. Войти в дом.

Однако жителю Марса, скорее всего, потребуется больше подробностей. Мы можем разбить подзадачу «Припарковать машину» на следующие шаги:

1. Найти парковочное место.
2. Заехать на место.
3. Выключить двигатель.
4. Затянуть стояночный тормоз.
5. Выйти из машины.
6. Запереть машину.

Подзадачу «Выключить двигатель» можно разбить на такие шаги, как «Выключить зажигание», «Вынуть ключ зажигания» и так далее, пока каждый этап всего процесса похода на рынок не будет полностью определен.

Процесс определения основных этапов и последующей детализации каждого из них называется нисходящим проектированием. Этот метод позволяет разбивать крупные сложные задачи на множество мелких и простых. Нисходящее проектирование — распространенный способ разработки программ, особенно подходящий для программирования на оболочке.

В этой главе мы воспользуемся методом проектирования сверху вниз, чтобы развить наш скрипт генератора отчетов.

Функции оболочки

Our script сейчас выполняет следующие шаги для генерации HTML-документа t:

1. Открыть страницу.
2. Открыть заголовок страницы.
3. Установить заголовок страницы.
4. Закрыть заголовок страницы.
5. Открыть тело страницы.
6. Вывести заголовок страницы.
7. Вывести метку времени.
8. Закрыть тело страницы.
9. Закрыть страницу.

На следующем этапе разработки мы добавим задачи между шагами 7 и 8. Они будут включать следующее:

- Время работы и нагрузка системы. Это время, прошедшее с момента последнего выключения или перезагрузки, и среднее количество задач, выполняемых процессором за несколько интервалов времени.
- Свободное место на диске. Это общий объем занятого пространства на устройствах хранения системы.
- Место в домашнем каталоге. Это объем дискового пространства, используемый каждым пользователем.

Если бы у нас была команда для каждой из этих задач, мы могли бы просто добавить их в наш скрипт с помощью подстановки команд.

```
#!/bin/bash
# Program to output a system information page
TITLE="System Information Report For $HOSTNAME"
CURRENT_TIME="$(date +%x %r %Z)"
TIMESTAMP="Generated $CURRENT_TIME, by $USER"
cat << _EOF_
<html>
    <head>
        <title>$TITLE</title>
    </head>
    <body>
        <h1>$TITLE</h1>
        <p>$TIMESTAMP</p>
        $(report_uptime)
        $(report_disk_space)
        $(report_home_space)
    </body>
</html>
_EOF_
```

Создать эти дополнительные команды можно двумя способами. Можно написать три отдельных скрипта и поместить их в

каталог, указанный в PATH, или встроить их в программу как функции оболочки. Как мы уже упоминали, функции оболочки — это «мини-скрипты», которые находятся внутри других скриптов и могут работать как автономные программы. Функции оболочки имеют две распространенные синтаксические формы. Сначала рассмотрим более формальную:

```
function name { commands return }
```

Вот более простой (и обычно

предпочтительный) вариант: `name () {commandsreturn }`

В этом примере `name` — это имя функции, а `commands` — последовательность команд внутри функции. Оба варианта эквивалентны и взаимозаменяемы. Ниже приведен скрипт, демонстрирующий использование функции оболочки:

```
1  #!/bin/bash
2
3  # Shell function demo
4
5  function step2 {
6      echo "Step 2"
7      return
8  }
9
10 # Main program starts here
11
12 echo "Step 1"
13 step2
14 echo "Step 3"
```

Когда оболочка читает скрипт, она пропускает строки с 1 по 11, так как они содержат комментарии и определение функции. Выполнение начинается со строки 12 с командой `echo`. Строка 13 вызывает функцию оболочки `step2`, и оболочка выполняет её так же, как любую другую команду. Затем управление передается на строку 6, где выполняется вторая команда `echo`. Следующей выполняется строка 7. Её команда `return` завершает функцию и возвращает управление программе на строку, следующую за вызовом функции (строка 14), после чего выполняется финальная команда `echo`. Примечание: чтобы вызовы распознавались как функции оболочки, а не как имена внешних программ, определения функций оболочки должны находиться в скрипте до их вызова.

Мы добавим в наш скрипт минимальные определения функций оболочки, как показано ниже:

```
#!/bin/bash
# Program to output a system information page
TITLE="System Information Report For $HOSTNAME"
CURRENT_TIME="$(date +%x %r %Z)"
```



```

TIMESTAMP="Generated $CURRENT_TIME, by $USER"
report_uptime () {
    return
}
report_disk_space () {
    return
}
report_home_space () {
    return
}
cat << _EOF_
<html>
  <head>
    <title>$TITLE</title>
  </head>
  <body>
    <h1>$TITLE</h1>
    <p>$TIMESTAMP</p>
    $(report_uptime)
    $(report_disk_space)
    $(report_home_space)
  </body>
</html>
_EOF_

```

Имена функций оболочки подчиняются тем же правилам, что и переменные. Функция должна содержать хотя бы одну команду. Команда `return` (необязательная) удовлетворяет этому требованию.

Локальные переменные

В написанных нами скриптах все переменные (включая константы) были глобальными. Глобальные переменные существуют на протяжении всей работы программы. Это удобно во многих случаях, но иногда может усложнить использование функций оболочки. Внутри функций оболочки часто полезно иметь локальные переменные. Локальные переменные доступны только внутри той функции оболочки, где они определены, а также в любых других функциях, которые она может вызвать. Они перестают существовать после завершения работы функции оболочки.

Наличие локальных переменных позволяет программисту использовать имена, которые уже могут существовать в скрипте глобально или в других функциях оболочки, не беспокоясь о возможных конфликтах имен.

Вот пример скрипта, демонстрирующий определение и использование локальных переменных:

```

#!/bin/bash
# local-vars: script to demonstrate local variables
foo=0    # global variable foo
funct_1 () {
    local foo # variable local to funct_1
    foo=1
}

```

```

    echo "funct_1: foo = $foo"
}
funct_2 () {
    local foo # variable local to funct_2
    foo=2
    echo "funct_2: foo = $foo"
}
echo "global:  foo = $foo"
funct_1
echo "global:  foo = $foo"
funct_2
echo "global:  foo = $foo"

```

Как видно, локальные переменные определяются путем добавления слова `local` перед именем переменной. Это создает переменную, локальную для оболочки функции, в которой она определена. За пределами этой функции переменная перестает существовать. При запуске этого скрипта мы получаем следующие результаты:

```

[me@linuxbox ~]$ local-vars
global:  foo = 0
funct_1: foo = 1
global:  foo = 0
funct_2: foo = 2
global:  foo = 0

```

Мы видим, что присваивание значений локальной переменной `foo` внутри обеих функций оболочки не влияет на значение `foo`, определенное вне функций.

Эта особенность позволяет писать функции оболочки так, чтобы они оставались независимыми друг от друга и от скрипта, в котором они находятся. Это ценно, так как помогает предотвратить вмешательство одной части программы в работу другой. Это также позволяет писать функции оболочки переносимыми. То есть их можно копировать из одного скрипта в другой по мере необходимости.

Функции оболочки и перенаправление

Если внимательнее изучить написание функций оболочки, мы заметим кое-что, о чем упоминалось в главе 6:

```

my_func () {
    command1
    command2
    command3
}

```

Три команды внутри фигурных скобок образуют групповую команду. Как мы помним из главы 6, групповые команды объединяют несколько команд в единое целое, когда дело

касается перенаправления. С помощью групповых команд можно сделать и то, и другое:

```
{ command1; command2; command3; } > some_output.txt
```

and:

```
{ command1; command2; command3; } < some_input.txt
```

То же самое относится и к функциям оболочки. Рассмотрим следующий код:

```
my_func () {  
    echo "My Documents"  
    ls ~/Documents  
    echo "My Music"  
    ls ~/Music  
    echo "My Videos"  
    ls ~/Videos  
    return  
}
```

Легко понять, что делает эта функция, но куда уходит её вывод? Он уходит туда, куда мы его направим. При вызове функции её объединённый вывод отправляется в стандартный вывод, а при желании мы можем направить его в файл:

```
my_func > my_directories.txt
```

Или в конвейер:

```
my_func | sort
```

Мы можем даже сохранить вывод в переменную, используя подстановку команды:

```
my_var="$(my_func)"
```

Перенаправление также применимо к стандартному вводу. Если функция содержит команду, принимающую стандартный ввод, например `cat` без аргументов, это легко сделать так:

```
my_func < input.txt
```

Поддержание работы с криптов

При разработке программы полезно держать её в рабочем состоянии. Частые запуски и тесты позволяют выявлять ошибки на ранних этапах, что значительно упрощает отладку. Например, если после внесения небольшого изменения программа начала работать неверно, скорее всего, причина в этом изменении. Добавляя пустые функции (программисты называют их

заглушками), мы можем проверять логику программы на ранних стадиях. При создании заглушки стоит добавить что-то, что даст программисту обратную связь о прохождении логического пути. Если мы посмотрим на вывод нашего скрипта сейчас, то увидим пустые строки после метки времени, но не можем быть уверены в их причине.

```
[me@linuxbox ~]$ sys_info_page
<html>
  <head>
    <title>System Information Report For twin2</title>
  </head>
  <body>
    <h1>System Information Report For linuxbox</h1>
    <p>Generated 03/19/2009 04:02:10 PM EDT, by me</p>
  </body>
</html>
```

Если изменить функции так, чтобы они выводили обратную связь

```
report_uptime () {
  echo "Function report_uptime executed."
  return
}
report_disk_space () {
  echo "Function report_disk_space executed."
  return
}
report_home_space () {
  echo "Function report_home_space executed."
  return
}
```

и повторный запуск скрипта

```
[me@linuxbox ~]$ sys_info_page
<html>
  <head>
    <title>System Information Report For linuxbox</title>
  </head>
  <body>
    <h1>System Information Report For linuxbox</h1>
    <p>Generated 03/20/2009 05:17:26 AM EDT, by me</p>
    Function report_uptime executed.
    Function report_disk_space executed.
    Function report_home_space executed.
  </body>
</html>
```

теперь мы видим, что на самом деле выполняются все три функции.

Когда каркас функций готов и работает, пора наполнить их кодом. Вот `report_uptime function`»:

```
report_uptime () {
  cat << _EOF_
    <h2>System Uptime</h2>
    <pre>$(uptime)</pre>
  _EOF_
}
```

```
    return  
}
```

Все довольно просто. Мы используем `here document`, чтобы вывести заголовок раздела и результат выполнения `uptime command`, заключая его в `<pre>` tags для сохранения форматирования. Функция `report_disk_space` устроена аналогично.

```
report_disk_space () {  
    cat << _EOF_  
        <h2>Disk Space Utilization</h2>  
        <pre>$(df -h)</pre>  
    _EOF_  
    return  
}
```

Эта функция использует команду `df -h` для определения объема дискового пространства. Наконец, мы создадим функцию `report_home_space`.

```
report_home_space () {  
    cat << _EOF_  
        <h2>Home Space Utilization</h2>  
        <pre>$(du -sh /home/*)</pre>  
    _EOF_  
    return  
}
```

Для выполнения этой задачи мы используем команду `du` с опциями `-sh`. Однако это не является полным решением проблемы. Хотя это сработает в некоторых системах (например, в Ubuntu), в других решение не работает. Причина в том, что во многих системах права доступа к домашним каталогам настроены так, чтобы исключить возможность их чтения всеми пользователями, что является разумной мерой безопасности. В таких системах `report_home_space` в текущем виде будет работать только при запуске скрипта с привилегиями суперпользователя. Лучшим решением было бы заставить скрипт адаптировать свое поведение в зависимости от привилегий пользователя. Мы разберем этот вопрос в следующей главе.

Функции оболочки в файле `.bashrc`

Функции оболочки — отличная замена псевдонимам (`aliases`), и на самом деле это предпочтительный метод создания небольших команд для личного использования. Возможности псевдонимов ограничены типами команд и функций оболочки, тогда как функции оболочки позволяют

использовать всё, что поддается автоматизации. Например, если нам понравится `report_disk_space` функция, которую мы разработали для нашего скрипта, мы могли бы создать аналогичную функцию с именем `ds` для нашего файла `.bashrc`:

```
ds () { echo "Disk Space Utilization For $HOSTNAME" df -h }
```

Итоги

В этой главе мы представили метод проектирования программ под названием «сверху вниз» и рассмотрели, как функции оболочки помогают реализовать требуемое поэтапное уточнение. Мы также узнали, как локальные переменные позволяют с делать функции оболочки независимыми друг от друга и от программы, в которой они находятся. Это позволяет писать функции оболочки переносимыми и повторно использовать их в разных программах, что значительно экономит время.

Что почитать

- В Википедии много статей о философии проектирования ПО. Вот пара хороших: https://en.wikipedia.org/wiki/Top-down_design, [https://en.wikipedia.org/wiki/Function_\(computer_programming\)](https://en.wikipedia.org/wiki/Function_(computer_programming))

27 — Управление потоком: ветвление с помощью *if*

В предыдущей главе перед нами стояла задача: как заставить скрипт генератора отчетов адаптироваться к привилегиям пользователя? Решение потребует способа «смены направления» внутри скрипта на основе результатов проверки. Говоря языком программирования, программе нужна ветвление.

Рассмотрим простой пример логики на псевдокоде — имитации языка программирования, предназначенной для понимания человеком.

X = 5

Если X = 5, то:

Вывести «X равно 5».

В противном случае:

Вывести «X не равно 5».

Это пример ветвления. Мы проверяем значение X: если X = 5, то выводим «X равно 5», иначе — «X не равно 5».

if

В оболочке эту логику можно реализовать так:

```
x=5
if [ "$x" -eq 5 ]; then
    echo "x equals 5."
else
    echo "x does not equal 5."
fi
```

Или мы можем ввести это прямо в командной строке (немного сократив):

```
[me@linuxbox ~]$ x=5
[me@linuxbox ~]$ if [ "$x" -eq 5 ]; then echo "equals 5"; else echo
"does not equal 5"; fi
equals 5
[me@linuxbox ~]$ x=0
[me@linuxbox ~]$ if [ "$x" -eq 5 ]; then echo "equals 5"; else echo
"does not equal 5"; fi
does not equal 5
```

В этом примере мы выполняем команду дважды: первый раз со значением x, равным 5, что выводит строку «equals 5», и второй раз со значением x, равным 0, что выводит строку «does not equal 5».

if Составная команда имеет следующий синтаксис, где `commands` — это список команд:

```
if commands; then commands [elif commands; then
commands...] [else commands] fi
```

На первый взгляд это может запутать. Но прежде чем мы во всем разберемся, нужно понять, как оболочка оценивает успех или неудачу команды.

Код завершения

Команды (включая написанные нами скрипты и функции оболочки) при завершении передают системе значение, называемое кодом завершения. Это значение — целое число в диапазоне от 0 до 255 — указывает на успех или неудачу выполнения команды. По соглашению, значение 0 означает успех, а любое другое — неудачу. Оболочка предоставляет параметр, который можно использовать для проверки кода завершения команды. Вот как это работает на практике:

```
[me@linuxbox ~]$ ls -d /usr/bin
/usr/bin
[me@linuxbox ~]$ echo $?
0
[me@linuxbox ~]$ ls -d /bin/usr
ls: cannot access /bin/usr: No such file or directory
[me@linuxbox ~]$ echo $?
2
```

В этом примере мы дважды выполняем команду `ls`. В первый раз команда выполняется успешно. Если вывести значение параметра `$?`, мы увидим, что оно равно нулю. Мы выполняем команду `ls` во второй раз (указав несуществующий каталог), получаем ошибку и снова проверяем параметр `$?`. На этот раз он содержит 2, что указывает на ошибку при выполнении команды. Некоторые команды используют различные значения кода завершения для диагностики ошибок, в то время как многие просто возвращают 1 при сбое. В руководствах (man pages) часто есть раздел «Exit Status», описывающий используемые коды. Однако ноль всегда означает успех.

Оболочка предоставляет две крайне простые встроенные команды, которые ничего не делают, кроме завершения с кодом выхода 0 или 1. Команда `true` всегда выполняется успешно, а команда `false` — всегда с ошибкой.

```
[me@linuxbox ~]$ true
[me@linuxbox ~]$ echo $?
0
[me@linuxbox ~]$ false
```



```
[me@linuxbox ~]$ echo $?  
1
```

Мы можем использовать эти команды, чтобы понять, как работает оператор `if`. На самом деле оператор `if` проверяет успешность или неудачность выполнения команд.

```
[me@linuxbox ~]$ if true; then echo "It's true."; fi  
It's true.  
[me@linuxbox ~]$ if false; then echo "It's true."; fi  
[me@linuxbox ~]$
```

Команда `echo "It's true."` выполняется, если команда после `if` завершилась успешно, и не выполняется, если команда после `if` завершилась с ошибкой. Если после `if` следует список команд, проверяется последняя команда в списке:

```
[me@linuxbox ~]$ if false; true; then echo "It's true."; fi  
It's true.  
[me@linuxbox ~]$ if true; false; then echo "It's true."; fi  
[me@linuxbox ~]$
```

Использование `test`

Чаще всего с `if` используют `test`. Команда `test` выполняет различные проверки и сравнения. У неё есть две эквивалентные формы. Вот первая:

`test expression`

Вторая, более популярная форма, выглядит так:

`[ expression]`

выражение — это выражение, которое оценивается как истинное или ложное. Команда `test` возвращает код завершения 0, если выражение истинно, и 1, если выражение ложно.

Интересно, что и `test`, и `[` на самом деле являются командами. В `bash` это встроенные команды, но они также существуют как программы в `/usr/bin` для использования в других оболочках. Выражение — это фактически просто его аргументы, при этом команда `[` требует, чтобы в качестве последнего аргумента был передан символ `]`.

Команды `test` и `[` поддерживают широкий спектр полезных выражений и тестов.

Файловые выражения

В таблице 27-1 перечислены выражения для проверки состояния файлов.

Таблица 27-1: *test* Файловые выражения

Выражение	Истинно, если...
<code>file1 -ef file2</code>	<code>file1</code> и <code>file2</code> имеют одинаковые номера inode (оба имени файла указывают на один и тот же файл через жесткую ссылку).
<code>file1 -nt file2</code>	<code>file1</code> новее, чем <code>file2</code> .
<code>file1 -ot file2</code>	<code>file1</code> старше, чем <code>file2</code> .
<code>-b file</code>	файл существует и является блочным (устройством) файлом.
<code>-c file</code>	файл существует и является символьным (устройством) файлом.
<code>-d file</code>	файл существует и является каталогом.
<code>-e file</code>	файл существует.
<code>-f file</code>	файл существует и является обычным файлом.
<code>-g file</code>	файл существует и имеет установленный set-group-ID.
<code>-G file</code>	файл существует и принадлежит эффективному ID группы.
<code>-k file</code>	файл существует и установлен «sticky bit».
<code>-L file</code>	файл существует и является символической ссылкой.
<code>-O file</code>	файл существует и принадлежит эффективному ID пользователя.
<code>-p file</code>	файл существует и является именованным каналом.
<code>-r file</code>	файл существует и доступен для чтения (имеет права на чтение д эффективного пользователя).
<code>-s file</code>	файл существует и его размер больше нуля.
<code>-S file</code>	файл существует и является сетевым сокетом.

Выражение	Истинно, если...
-t fd	fd — это файловый дескриптор, направленный в терминал или из него можно использовать для определения того, выполняется ли перенаправление ввода/вывода/ошибок.
-u file	файл существует и имеет установленный setuid.
-w file	файл существует и доступен для записи (есть права на запись для эффективного пользователя).
-x file	файл существует и доступен для выполнения (есть права на выполнение поиска для эффективного пользователя).

Ниже приведен скрипт, демонстрирующий некоторые из этих выражений:

```
#!/bin/bash
# test-file: Evaluate the status of a file
FILE=~/.bashrc
if [ -e "$FILE" ]; then
    if [ -f "$FILE" ]; then
        echo "$FILE is a regular file."
    fi
    if [ -d "$FILE" ]; then
        echo "$FILE is a directory."
    fi
    if [ -r "$FILE" ]; then
        echo "$FILE is readable."
    fi
    if [ -w "$FILE" ]; then
        echo "$FILE is writable."
    fi
    if [ -x "$FILE" ]; then
        echo "$FILE is executable/searchable."
    fi
else
    echo "$FILE does not exist"
    exit 1
fi
exit
```

Скрипт проверяет файл, назначенный константе `FILE`, и выводит результаты по мере выполнения. В этом скрипте есть два интересных момента. Во-первых, обратите внимание, как параметр `$FILE` заключен в кавычки внутри выражений. Это не требуется для синтаксической полноты выражения, а служит защитой на случай, если параметр окажется пустым или будет содержать только пробелы. Если бы раскрытие параметра `$FILE` привело к пустому значению, это вызвало бы ошибку (операторы

были бы интерпретированы как непустые строки, а не как операторы). Использование кавычек гарантирует, что за оператором всегда будет следовать строка, даже если она пуста. Во-вторых, обратите внимание на команду `exit` в конце скрипта. Команда `exit` принимает один необязательный аргумент, который становится кодом завершения скрипта. Если аргумент не передан, кодом завершения по умолчанию становится код завершения последней выполненной команды. Использование `exit` таким образом позволяет скрипту указать на ошибку, если `$FILE` раскрывается в имя несуществующего файла. Команда `exit` в последней строке скрипта добавлена для формальности. Когда скрипт «доходит до конца» (достигает конца файла), он завершается с кодом завершения последней выполненной команды.

Аналогично, функции оболочки могут возвращать статус выхода, передавая целочисленный аргумент команде `return`. Если мы преобразуем предыдущий скрипт в функцию оболочки для использования в большой программе, мы сможем заменить команды `exit` операторами `return` и получить нужный результат.

```
test_file () {
    # test-file: Evaluate the status of a file
    FILE=~/.bashrc
    if [ -e "$FILE" ]; then
        if [ -f "$FILE" ]; then
            echo "$FILE is a regular file."
        fi
        if [ -d "$FILE" ]; then
            echo "$FILE is a directory."
        fi
        if [ -r "$FILE" ]; then
            echo "$FILE is readable."
        fi
        if [ -w "$FILE" ]; then
            echo "$FILE is writable."
        fi
        if [ -x "$FILE" ]; then
            echo "$FILE is executable/searchable."
        fi
    else
        echo "$FILE does not exist"
        return 1
    fi
}
```

Строковые выражения

В таблице 27-2 перечислены выражения для проверки строк:

Таблица 27-2: *test* Строковые выражения

Выражение	Истинно, если...
-----------	------------------

Выражение	Истинно, если...
<code>string</code>	строка не пуста.
<code>-n string</code>	Длина строки больше нуля.
<code>-z string</code>	Длина строки равна нулю.
<code>string1 = string2</code> <code>string1 == string2</code>	<code>string1</code> и <code>string2</code> равны. Можно использовать как одинарные так и двойные знаки равенства. Использование двойных знаков поддерживается <code>bash</code> и обычно предпочтительнее, но это не соответствует стандарту POSIX.
<code>string1 != string2</code>	<code>string1</code> и <code>string2</code> не равны.
<code>string1 > string2</code>	<i>string1 идет после string2 при сортировке.</i>
<code>string1 < string2</code>	<code>string1</code> идет перед <code>string2</code> при сортировке.

Предупреждение: операторы выражений `>` и `<` необходимо брать в кавычки (или экранировать обратным слэшем) при использовании с `test`. В противном случае оболочка интерпретирует их как операторы перенаправления, что может привести к разрушительным последствиям. Также учтите, что, хотя в документации `bash` указано, что порядок сортировки соответствует порядку сопоставления текущей локали, это может быть не так. В версиях `bash` до 4.0 включительно используется порядок ASCII (POSIX). Эта проблема была исправлена в версии 4.1.

Вот скрипт, использующий строковые выражения:

```
#!/bin/bash
# test-string: evaluate the value of a string
ANSWER=maybe
if [ -z "$ANSWER" ]; then
    echo "There is no answer." >&2
    exit 1
fi
if [ "$ANSWER" == "yes" ]; then
    echo "The answer is YES."
elif [ "$ANSWER" == "no" ]; then
    echo "The answer is NO."
elif [ "$ANSWER" == "maybe" ]; then
    echo "The answer is MAYBE."
else
    echo "The answer is UNKNOWN."
fi
```

В этом скрипте мы проверяем константу ANSWER. Сначала мы определяем, является ли строка пустой. Если да, мы завершаем работу скрипта и устанавливаем код выхода 1. Обратите внимание на перенаправление, применяемое к команде echo. Оно перенаправляет сообщение об ошибке «There is no answer.» в стандартный поток ошибок, что является правильным способом обработки сообщений об ошибках. Если строка не пуста, мы проверяем её значение на соответствие «yes», «no» или «maybe». Мы делаем это с помощью elif (сокращение от «else if»). Используя elif, мы можем выстроить более сложную логическую проверку.

Целочисленные выражения

Чтобы сравнивать значения как целые числа, а не как строки, можно использовать выражения из таблицы 27-3.

Таблица 27-3: *test* Целочисленные выражения

Выражение	Истинно, если...
<code>integer1 -eq integer2</code>	<i>integer1</i> равно <i>integer2</i> .
<code>integer1 -ne integer2</code>	<i>integer1</i> не равно <i>integer2</i> .
<code>integer1 -le integer2</code>	<i>integer1</i> меньше или равно <i>integer2</i> .
<code>integer1 -lt integer2</code>	<i>integer1</i> меньше <i>integer2</i> .
<code>integer1 -ge integer2</code>	<i>integer1</i> больше или равно <i>integer2</i> .
<code>integer1 -gt integer2</code>	<i>integer1</i> больше <i>integer2</i> .

Вот скрипт, демонстрирующий их работу:

```
#!/bin/bash
# test-integer: evaluate the value of an integer.
```

```

INT=-5
if [ -z "$INT" ]; then
    echo "INT is empty." >&2
    exit 1
fi
if [ "$INT" -eq 0 ]; then
    echo "INT is zero."
else
    if [ "$INT" -lt 0 ]; then
        echo "INT is negative."
    else
        echo "INT is positive."
    fi
    if [ $((INT % 2)) -eq 0 ]; then
        echo "INT is even."
    else
        echo "INT is odd."
    fi
fi

```

Интересная часть скрипта — то, как он определяет четность целого числа. Выполняя операцию взятия остатка от деления на 2, он узнает, является ли число четным или нечетным.

Более современная версия test

Современные версии bash включают составную команду, которая служит улучшенной заменой test. Она использует следующий синтаксис, где, как и в случае с test, выражение — это логическое выражение, возвращающее результат «истина» или «ложь»:

```
[[ expression]]
```

Команда `[[ ]]` похожа на `test` (она поддерживает все её выражения), но добавляет важное новое строковое выражение.

```
string1 =~ regex
```

Это возвращает «истина», если `string1` соответствует расширенному регулярному выражению `regex`. Это открывает множество возможностей для таких задач, как проверка данных. В нашем предыдущем примере с целочисленными выражениями скрипт завершился бы ошибкой, если бы константа `INT` содержала что-либо, кроме целого числа. Скрипту нужен способ проверить, что константа является целым числом. Используя `[[ ]]` с оператором строкового выражения `=~`, мы могли бы улучшить скрипт следующим образом:

```

#!/bin/bash
# test-integer2: evaluate the value of an integer.
INT=-5
if [[ "$INT" =~ ^-[0-9]+$ ]]; then
    if [ "$INT" -eq 0 ]; then
        echo "INT is zero."
    else

```

```

        if [ "$INT" -lt 0 ]; then
            echo "INT is negative."
        else
            echo "INT is positive."
        fi
        if [ $((INT % 2)) -eq 0 ]; then
            echo "INT is even."
        else
            echo "INT is odd."
        fi
    fi
else
    echo "INT is not an integer." >&2
    exit 1
fi

```

Применяя регулярное выражение, мы ограничиваем значение INT только строками, которые начинаются с необязательного знака минус, за которым следует одна или несколько цифр. Это выражение также исключает возможность пустых значений.

Еще одна особенность `[]` заключается в том, что оператор `==` поддерживает сопоставление с шаблоном так же, как и раскрытие pathname. Вот пример:

```

[me@linuxbox ~]$ FILE=foo.bar
[me@linuxbox ~]$ if [[ $FILE == foo.* ]]; then
> echo "$FILE matches pattern 'foo.*'"
> fi
foo.bar matches pattern 'foo.*'

```

Это делает `[]` полезным для проверки имен файлов и путей.

(()) — Предназначен для целых чисел

В дополнение к составной команде `[]`, bash также предоставляет составную команду `(( ))`, которая полезна для работы с целыми числами. Она поддерживает полный набор арифметических вычислений — эту тему мы подробно рассмотрим в главе 34.

`(( ))` используется для выполнения арифметических проверок истинности. Арифметическая проверка считается истинной, если результат вычисления не равен нулю.

```

[me@linuxbox ~]$ if ((1)); then echo "It is true."; fi
It is true.
[me@linuxbox ~]$ if ((0)); then echo "It is true."; fi
[me@linuxbox ~]$

```

Используя `(( ))`, мы можем немного упростить скрипт `test-integer2` следующим образом:

```

#!/bin/bash
# test-integer2a: evaluate the value of an integer.
INT=-5
if [[ "$INT" =~ ^-?[0-9]+$ ]]; then

```



```

if ((INT == 0)); then
    echo "INT is zero."
else
    if ((INT < 0)); then
        echo "INT is negative."
    else
        echo "INT is positive."
    fi
    if (( (INT % 2) == 0)); then
        echo "INT is even."
    else
        echo "INT is odd."
    fi
fi
else
    echo "INT is not an integer." >&2
    exit 1
fi

```

Заметьте, что мы используем знаки «меньше» и «больше», а == применяется для проверки эквивалентности. Такая синтаксическая конструкция выглядит естественнее при работе с целыми числами. Также обратите внимание: поскольку составная команда (()) является частью синтаксиса оболочки, а не обычной командой, и работает только с целыми числами, она способна распознавать переменные по имени и не требует предварительного раскрытия. Мы подробнее обсудим (()) и связанное с ним арифметическое раскрытие в главе 34.

Комбинирование выражений

Можно также комбинировать выражения для создания более сложных вычислений. Выражения объединяются с помощью логических операторов. Мы встречали их в главе 17, когда изучали команду find. Для test и [] существуют три логические операции: AND, OR и NOT. test и [] используют разные операторы для представления этих операций:

Таблица 27-4: Логические операторы

Операция	test	[] и ()
AND	-a	&&
OR	-o	
NOT	!	!

Вот пример операции AND. Следующий скрипт определяет, входит ли целое число в диапазон значений:

```

#!/bin/bash
# test-integer3: determine if an integer is within a
# specified range of values.
MIN_VAL=1

```

```

MAX_VAL=100
INT=50
if [[ "$INT" =~ ^-?[0-9]+$ ]]; then
    if [[ "$INT" -ge "$MIN_VAL" && "$INT" -le "$MAX_VAL" ]]; then
        echo "$INT is within $MIN_VAL to $MAX_VAL."
    else
        echo "$INT is out of range."
    fi
else
    echo "INT is not an integer." >&2
    exit 1
fi

```

В этом скрипте мы определяем, лежит ли значение целого числа INT между значениями MIN_VAL и MAX_VAL. Это выполняется с помощью одного использования `[[ ]]`, включающего два выражения, разделенных оператором `&&`. Мы также могли бы реализовать это с помощью `test`:

```

if [ "$INT" -ge "$MIN_VAL" -a "$INT" -le "$MAX_VAL" ]; then
    echo "$INT is within $MIN_VAL to $MAX_VAL."
else
    echo "$INT is out of range."
fi

```

Оператор отрицания `!` меняет результат выражения на противоположный. Он возвращает `true`, если выражение ложно, и `false`, если оно истинно. В следующем скрипте мы изменим логику вычислений, чтобы найти значения INT, выходящие за пределы указанного диапазона:

```

#!/bin/bash
# test-integer4: determine if an integer is outside a
# specified range of values.
MIN_VAL=1
MAX_VAL=100
INT=50
if [[ "$INT" =~ ^-?[0-9]+$ ]]; then
    if [[ ! ("$INT" -ge "$MIN_VAL" && "$INT" -le "$MAX_VAL") ]]; then
        echo "$INT is outside $MIN_VAL to $MAX_VAL."
    else
        echo "$INT is in range."
    fi
else
    echo "INT is not an integer." >&2
    exit 1
fi

```

Мы также заключаем выражение в скобки для группировки. Без них отрицание применилось бы только к первому выражению, а не к их комбинации. С использованием `test` это записывается так:

```

if [ ! \( "$INT" -ge "$MIN_VAL" -a "$INT" -le "$MAX_VAL" \) ];
then
    echo "$INT is outside $MIN_VAL to $MAX_VAL."
else

```

```
echo "$INT is in range."  
fi
```

Поскольку оболочка воспринимает все выражения и операторы, используемые `test`, как аргументы команды (в отличие от `[]` и `(( ))`), символы, имеющие особое значение для `bash`, такие как `<`, `>`, `(`, `)`, необходимо заключать в кавычки или экранировать.

Учитывая, что `test` и `[]` делают примерно одно и то же, что лучше? `test` является традиционным способом (и частью спецификации POSIX для стандартных оболочек, которые часто используются для запуска системных скриптов при загрузке), тогда как `[]` специфичен для `bash` (и некоторых других современных оболочек). Важно уметь использовать `test`, так как это широко применяется, но `[]` явно полезнее и проще в написании, поэтому он предпочтителен для современных скриптов.

Переносимость — удел ограниченных умов

Если вы заговорите с «настоящими» пользователями Unix, то быстро обнаружите, что многие из них не очень любят Linux. Они считают его нечистым и несовершенным. Один из постулатов пользователей Unix заключается в том, что всё должно быть «переносимым». Это значит, что любой написанный вами скрипт должен запускаться без изменений в любой Unix-подобной системе.

У пользователей Unix есть веские причины так считать. Видевшие, что проприетарные расширения команд и оболочек сделали с миром Unix до появления POSIX, они закономерно опасаются влияния Linux на свою любимую ОС.

Но у переносимости есть серьезный недостаток. Она тормозит прогресс. Она требует, чтобы всё делалось с использованием методов «наименьшего общего знаменателя». В случае программирования на оболочке это означает обеспечение совместимости со `sh`, оригинальной оболочкой Bourne.

Этот недостаток служит оправданием для вендоров проприетарного ПО, когда те обосновывают свои закрытые расширения, называя их «инновациями». На деле же это лишь способы привязки клиентов к своим продуктам.

Инструменты GNU, такие как `bash`, не имеют подобных ограничений. Они способствуют переносимости, поддерживая стандарты и будучи общедоступными. Вы можете бесплатно установить `bash` и другие инструменты

GNU практически на любую систему, даже Windows. Так что смело используйте все возможности `bash`. Они действительно переносимы.

Операторы управления: еще один способ ветвления

`bash` предоставляет два оператора управления для ветвления. Операторы `&&` (AND) и `||` (OR) работают как логические операторы в составной команде `[[]]`. Вот синтаксис для `&&`:

```
command1 && command2
```

Вот синтаксис для `||`:

```
command1 || command2
```

Важно понимать их поведение. С оператором `&&` `command1` всегда выполняется, а `command2` — только если `command1` завершился успешно. С оператором `||` `command1` всегда выполняется, а `command2` — только если `command1` завершился с ошибкой.

На практике это означает, что мы можем сделать следующее:

```
[me@linuxbox ~]$ mkdir temp && cd temp
```

Это создаст каталог с именем `temp`, и если операция пройдет успешно, текущий рабочий каталог изменится на `temp`. Вторая команда будет выполнена только в случае успеха команды `mkdir`. Аналогично, команда вида

```
[me@linuxbox ~]$ [[ -d temp ]] || mkdir temp
```

проверит наличие каталога `temp`, и каталог будет создан только в случае неудачи проверки. Такие конструкции удобны для обработки ошибок в скриптах — мы подробнее обсудим это в следующих главах. Например, в скрипте можно сделать так:

```
[ -d temp ] || exit 1
```

Если скрипту требуется каталог `temp`, а он отсутствует, скрипт завершится со статусом выхода 1.

Помните, что если вам нужно выполнить что-то сложное, командой может быть целая группа команд.

```
{ true && echo "true"; } && { false || echo "false"; }
```

Групповые команды возвращают статус выхода последней команды в группе.

Итоги

Мы начали эту главу с вопроса. Как заставить наш `sys_info_page` script определять, есть ли у пользователя права на чтение всех домашних каталогов? Используя знания о `if`, мы можем решить задачу, добавив этот код в функцию `report_home_space`:

```
report_home_space () {
    if [[ "$(id -u)" -eq 0 ]]; then
        cat << _EOF_
        <h2>Home Space Utilization (All Users)</h2>
        <pre>$(du -sh /home/*)</pre>
    _EOF_
    else
        cat << _EOF_
        <h2>Home Space Utilization ($USER)</h2>
        <pre>$(du -sh $HOME)</pre>
    _EOF_
    fi
    return
}
```

Мы оцениваем вывод команды `id`. С опцией `-u` `id` выводит числовой идентификатор (UID) эффективного пользователя. У суперпользователя ID всегда равен нулю, а у всех остальных — больше нуля. Зная это, мы можем составить два разных here document: один с использованием привилегий суперпользователя, а другой — с ограничением только домашним каталогом пользователя.

Мы сделаем перерыв в работе над `sys_info_page` программой, но не волнуйтесь — она вернется. Тем временем мы разберем темы, которые понадобятся нам при возобновлении работы.

Что почитать

В `bash` man-страницах есть несколько разделов, содержащих подробности по темам этой главы:

- Lists (описывает операторы управления `|` и `&&`)
- Compound Commands (описывает `[]`, `(( ))` и `if`)
- УСЛОВНЫЕ ВЫРАЖЕНИЯ
- ВСТРОЕННЫЕ КОМАНДЫ ОБОЛОЧКИ (охватывает `test`)

Кроме того, в Википедии есть хорошая статья о понятии псевдокода:

<https://en.wikipedia.org/wiki/Pseudocode>

28 — Чтение ввода с клавиатуры

Написанным нами скриптам не хватает функции, характерной для большинства компьютерных программ, — интерактивности, то есть способности программы взаимодействовать с пользователем. Хотя многим программам интерактивность не нужна, некоторым полезно уметь принимать ввод напрямую от пользователя. Рассмотрим, к примеру, этот скрипт из предыдущей главы:

```
#!/bin/bash
# test-integer2: evaluate the value of an integer.
INT=-5
if [[ "$INT" =~ ^-?[0-9]+$ ]]; then
    if [ "$INT" -eq 0 ]; then
        echo "INT is zero."
    else
        if [ "$INT" -lt 0 ]; then
            echo "INT is negative."
        else
            echo "INT is positive."
        fi
        if [ $((INT % 2)) -eq 0 ]; then
            echo "INT is even."
        else
            echo "INT is odd."
        fi
    fi
else
    echo "INT is not an integer." >&2
    exit 1
fi
```

Каждый раз, когда нам нужно изменить значение INT, приходится редактировать скрипт. Было бы гораздо удобнее, если бы скрипт мог запрашивать значение у пользователя. В этой главе мы начнем изучать, как добавить интерактивность в наши программы.

read — Чтение значений из стандартного ввода

Встроенная команда `read` используется для чтения одной строки из стандартного ввода. Эту команду можно использовать для чтения ввода с клавиатуры или, при использовании перенаправления, строки данных из файла. Синтаксис команды следующий:

```
read [-options] [variable...]
```

где `options` — это одна или несколько доступных опций, перечисленных далее в таблице 28-1, а `variable` — имя одной или нескольких переменных для хранения входного значения. Если

имя переменной не указано, переменная оболочки `REPLY` будет содержать строку данных.

По сути, `read` присваивает поля из стандартного ввода указанным переменным. Если мы изменим наш скрипт вычисления целых чисел для использования `read`, он может выглядеть так:

```
#!/bin/bash
# read-integer: evaluate the value of an integer.
echo -n "Please enter an integer -> "
read int
if [[ "$int" =~ ^-?[0-9]+$ ]]; then
    if [ "$int" -eq 0 ]; then
        echo "$int is zero."
    else
        if [ "$int" -lt 0 ]; then
            echo "$int is negative."
        else
            echo "$int is positive."
        fi
        if [ $((int % 2)) -eq 0 ]; then
            echo "$int is even."
        else
            echo "$int is odd."
        fi
    fi
else
    echo "Input value is not an integer." >&2
    exit 1
fi
```

Мы используем `echo` с опцией `-n` (которая подавляет символ новой строки в выводе), чтобы отобразить приглашение, а затем используем `read`, чтобы ввести значение для переменной `int`. Запуск этого скрипта дает следующий результат:

```
[me@linuxbox ~]$ read-integer
Please enter an integer -> 5
5 is positive.
5 is odd.
```

`read` может присваивать ввод нескольким переменным, как показано в этом скрипте:

```
#!/bin/bash
# read-multiple: read multiple values from keyboard
echo -n "Enter one or more values > "
read var1 var2 var3 var4 var5
echo "var1 = '$var1'"
echo "var2 = '$var2'"
echo "var3 = '$var3'"
echo "var4 = '$var4'"
echo "var5 = '$var5'"
```

В этом скрипте мы присваиваем и выводим до пяти значений. Обратите внимание, как ведет себя `read` при разном количестве значений, как показано ниже:

```
[me@linuxbox ~]$ read-multiple
Enter one or more values > a b c d e
var1 = 'a'
var2 = 'b'
var3 = 'c'
var4 = 'd'
var5 = 'e'
[me@linuxbox ~]$ read-multiple
Enter one or more values > a
var1 = 'a'
var2 = ''
var3 = ''
var4 = ''
var5 = ''
[me@linuxbox ~]$ read-multiple
Enter one or more values > a b c d e f g
var1 = 'a'
var2 = 'b'
var3 = 'c'
var4 = 'd'
var5 = 'e f g'
```

Если `read` получает меньше ожидаемого количества значений, лишние переменные остаются пустыми, а если ввод превышает норму, последняя переменная содержит весь избыточный ввод. Если после команды `read` не указаны переменные, переменной оболочки `REPLY` будет присвоен весь ввод.

```
#!/bin/bash
# read-single: read multiple values into default variable
echo -n "Enter one or more values > "
read
echo "REPLY = '$REPLY'"
```

Запуск этого скрипта дает следующий результат:

```
[me@linuxbox ~]$ read-single
Enter one or more values > a b c d
REPLY = 'a b c d'
```

Параметры

`read` поддерживает опции, описанные в таблице 28-1.

Таблица 28-1: Параметры `read` а. Мы

Параметр	Описание
-a array	Присваивает входные данные массиву, начиная с нулевого индекса разберем массивы в главе 35.
-d delimiter	Первый символ разделителя строки используется для обозначения конца ввода вместо символа новой строки.

Параметр	Описание
-e	Используйте Readline для обработки ввода. Это позволяет редактировать ввод так же, как в командной строке.
-i string	Используйте string как ответ по умолчанию, если пользователь просто нажмет ENTER. Требуется опция -e.
-n num	Считывает num символов вместо всей строки.
-p prompt	Выводит приглашение для ввода, используя строку prompt.
-r	Сырой режим. Не интерпретировать символы обратной косой черты как экранирующие символы. Для безопасности рекомендуется использовать эту опцию. Например, при вводе пути DOS нам нужно обратные косые черты воспринимаются как обычные символы.
-s	Бесшумный режим. Не выводить символы на экран при их вводе. Это полезно при вводе паролей и другой конфиденциальной информации.
-t seconds	Тайм-аут. Завершить ввод через read секунд. read возвращает ненулевой код выхода, если время ввода истекло.
-u fd	Использовать ввод из дескриптора файла fd вместо стандартного ввода.

Используя различные опции, мы можем делать интересные вещи с read. Например, с помощью опции -p можно задать строку приглашения.

```
#!/bin/bash
# read-single: read multiple values into default variable
read -r -p "Enter one or more values > "
echo "REPLY = '$REPLY'"
```

С помощью опций -t и -s можно написать скрипт, который считывает «секретные» данные и завершается по тайм-ауту, если ввод не завершен в заданное время.

```
#!/bin/bash
# read-secret: input a secret passphrase
if read -r -t 10 -sp "Enter secret passphrase > " secret_pass; then
    echo -e "\nSecret passphrase = '$secret_pass'"
else
    echo -e "\nInput timed out" >&2
```

```
        exit 1
    fi
```

Скрипт запрашивает у пользователя секретную фразу и ждет ввода 10 секунд. Если ввод не завершен в указанное время, скрипт завершается с ошибкой. Благодаря опции `-s` символы фразы не отображаются на экране при наборе.

Используя опции `-e` и `-i` вместе, можно предоставить пользователю ответ по умолчанию.

```
#!/bin/bash
# read-default: supply a default value if user presses Enter key.
read -e -p "What is your user name? " -i $USER
echo "You answered: '$REPLY'"
```

В этом скрипте мы запрашиваем имя пользователя и используем переменную окружения `USER` для значения по умолчанию. При запуске скрипт выводит строку по умолчанию, и если пользователь просто нажмет `ENTER`, `read` присвоит эту строку переменной `REPLY`.

```
[me@linuxbox ~]$ read-default
What is your user name? me
You answered: 'me'
```

IFS

Обычно оболочка выполняет разделение слов во входных данных для `read`. Как мы видели, это означает, что несколько слов, разделенных одним или несколькими пробелами, становятся отдельными элементами в строке ввода и назначаются `read` разным переменным. Это поведение настраивается переменной оболочки `IFS` (сокращение от «Internal Field Separator»). Значение по умолчанию для `IFS` включает пробел, табуляцию и символ новой строки — каждый из них разделяет элементы.

Мы можем изменить значение `IFS`, чтобы управлять разделением полей, поступающих в `read`. Например, файл `/etc/passwd` содержит строки данных, где в качестве разделителя полей используется двоеточие. Изменив значение `IFS` на одиночное двоеточие, мы можем использовать `read` для ввода содержимого `/etc/passwd` и успешно разделить поля по разным переменным. Вот скрипт, который делает именно это:

```
#!/bin/bash
# read-ifs: read fields from a file
FILE=/etc/passwd
read -r -p "Enter a username > " user_name
file_info="$(grep "^$user_name:" $FILE)"
if [ -n "$file_info" ]; then
    IFS=":" read -r user pw uid gid name home shell <<< "$file_info"
```

```

    echo "User =      '$user'"
    echo "UID =       '$uid'"
    echo "GID =       '$gid'"
    echo "Full Name = '$name'"
    echo "Home Dir. = '$home'"
    echo "Shell =     '$shell'"
else
    echo "No such user '$user_name'" >&2
    exit 1
fi

```

Этот скрипт запрашивает у пользователя имя учетной записи в системе, а затем отображает различные поля, найденные в записи пользователя в файле `/etc/passwd`. Скрипт содержит две интересные строки. Первая выглядит так:

```
file_info=$(grep "^$user_name:" $FILE)
```

Эта строка присваивает результат команды `grep` переменной `file_info`. Регулярное выражение, используемое `grep`, гарантирует, что имя пользователя будет соответствовать только одной строке в файле `/etc/passwd`.

Вторая интересная строка — вот эта:

```
IFS=":" read user pw uid gid name home shell <<<
"$file_info"
```

Строка состоит из трёх частей: присваивания переменной, `read` команды со списком имен переменных в качестве аргументов и странного нового оператора перенаправления. Сначала мы разберем присваивание переменной.

Оболочка позволяет выполнять одну или несколько операций присваивания переменных непосредственно перед командой. Эти присваивания изменяют окружение для последующей команды. Эффект присваивания временный и меняет окружение только на время выполнения команды. В нашем случае значение `IFS` меняется на символ двоеточия. Можно было бы написать и так:

```
OLD_IFS="$IFS" IFS=":" read user pw uid gid name home
shell <<< "$file_info" IFS="$OLD_IFS"
```

Здесь мы сохраняем значение `IFS`, присваиваем новое, выполняем команду `read`, а затем возвращаем `IFS` в исходное состояние. Очевидно, что размещение присваивания переменной перед командой — более лаконичный способ сделать то же самое.

Оператор `<<<` обозначает «here string» (строку здесь). «Here string» подобен «here document», но короче и состоит из одной строки. В нашем примере строка данных из файла `/etc/passwd` `read`

подается на стандартный ввод команды. Может возникнуть вопрос, почему был выбран этот довольно косвенный метод вместо этого:

```
echo "$file_info" | IFS=":" read user pw uid gid name  
home shell
```

Что ж, на то есть причина...

Нельзя передать `read` через конвейер

Хотя команда `read` обычно принимает данные из стандартного ввода, так сделать не получится:

```
echo "foo" | read
```

Казалось бы, это должно работать, но нет. Команда завершится успешно, но переменная `REPLY` всегда будет пустой. Почему?

Объяснение кроется в том, как оболочка обрабатывает конвейеры. В `bash` (и других оболочках, таких как `sh`) конвейеры создают дочерние оболочки. Это копии оболочки и её окружения, используемые для выполнения команд в конвейере. В нашем примере `read` выполняется в дочерней оболочке.

В Unix-подобных системах подоболочки создают копии окружения для использования процессами во время выполнения. Когда процесс завершается, копия окружения уничтожается. Это значит, что подоболочка не может изменить окружение родительского процесса. `read` присваивает переменные, которые затем становятся частью окружения. В предыдущем примере `read` присваивает значение `foo` переменной `REPLY` в окружении своей подоболочки, но после выхода из команды подоболочка и её окружение уничтожаются, и эффект присваивания теряется.

Использование `here strings` — один из способов обойти это поведение. Другой метод описан в главе 36.

Валидация ввода

Возможность ввода с клавиатуры ставит перед программистом новую задачу — валидацию ввода. Часто разница между хорошо и плохо написанной программой заключается в умении последней обрабатывать непредвиденные ситуации. Зачастую они проявляются в виде некорректного ввода. Мы немного практиковались в этом в предыдущей главе, создавая программы

для вычислений, где проверяли целые числа и отсеивали пустые значения и нецифровые символы. Важно проводить такие проверки каждый раз, когда программа получает данные, чтобы защититься от некорректных значений. Это особенно важно для программ, которыми пользуются несколько пользователей. Отказ от этих мер защиты ради экономии времени можно простить, если программа предназначена для разового использования самим автором для решения специфической задачи. Но даже в этом случае, если программа выполняет опасные действия, например удаление файлов, стоит включить валидацию данных на всякий случай.

Вот пример программы, которая проверяет различные виды входных данных:

```
#!/bin/bash
# read-validate: validate input
invalid_input () {
    echo "Invalid input '$REPLY'" >&2
    exit 1
}
read -r -p "Enter a single item > "
# input is empty (invalid)
[[ -z "$REPLY" ]] && invalid_input
# input is multiple items (invalid)
(( $(echo "$REPLY" | wc -w) > 1 )) && invalid_input
# is input a valid filename?
if [[ "$REPLY" =~ ^[-[:alnum:]\.]+$ ]]; then
    echo "'$REPLY' is a valid filename."
    if [[ -e "$REPLY" ]]; then
        echo "And file '$REPLY' exists."
    else
        echo "However, file '$REPLY' does not exist."
    fi
# is input a floating point number?
if [[ "$REPLY" =~ ^-?[:digit:]*\.[[:digit:]]+$ ]]; then
    echo "'$REPLY' is a floating point number."
else
    echo "'$REPLY' is not a floating point number."
fi
# is input an integer?
if [[ "$REPLY" =~ ^-?[:digit:]]+$ ]]; then
    echo "'$REPLY' is an integer."
else
    echo "'$REPLY' is not an integer."
fi
else
    echo "The string '$REPLY' is not a valid filename."
fi
```

Этот скрипт выводит приглашение для ввода объекта. Затем объект анализируется для определения его содержимого. Как видно, скрипт использует многие изученные нами концепции, включая функции оболочки, `[[ ]]`, `(( ))`, оператор управления `&&` и `if`, а также активно применяет регулярные выражения.

Меню

Один из распространенных типов взаимодействия — меню-ориентированный. В таких программах пользователю предлагается список вариантов, из которых нужно выбрать один. Например, можно представить программу, выводящую следующее:

```
Please Select:
1. Display System Information
2. Display Disk Space
3. Display Home Space Utilization
0. Quit
Enter selection [0-3] >
```

Используя знания, полученные при написании программы `sys_info_page`, мы можем создать программу с меню для выполнения задач из предыдущего меню.

```
#!/bin/bash
# read-menu: a menu driven system information program
clear
echo "
Please Select:
1. Display System Information
2. Display Disk Space
3. Display Home Space Utilization
0. Quit
"
read -r -p "Enter selection [0-3] > "
if [[ "$REPLY" =~ ^[0-3]$ ]]; then
    if [[ "$REPLY" == 0 ]]; then
        echo "Program terminated."
        exit
    fi
    if [[ "$REPLY" == 1 ]]; then
        echo "Hostname: $HOSTNAME"
        uptime
        exit
    fi
    if [[ "$REPLY" == 2 ]]; then
        df -h
        exit
    fi
    if [[ "$REPLY" == 3 ]]; then
        if [[ "$(id -u)" -eq 0 ]]; then
            echo "Home Space Utilization (All Users)"
            du -sh /home/*
        else
            echo "Home Space Utilization ($USER)"
            du -sh "$HOME"
        fi
        exit
    fi
else
    echo "Invalid entry." >&2
    exit 1
fi
```

Этот скрипт логически разделен на две части. Первая часть отображает меню и считывает ответ пользователя. Вторая часть определяет ответ и выполняет выбранное действие. Обратите

внимание на использование команды `exit` в этом скрипте. Здесь она применяется, чтобы предотвратить выполнение лишнего кода после завершения действия. Наличие множества точек выхода в программе обычно считается плохой практикой (это усложняет понимание логики), но в данном скрипте это работает.

Итоги

В этой главе мы сделали первые шаги к интерактивности, научив пользователей вводить данные в наши программы с помощью клавиатуры. Используя изученные методы, можно писать множество полезных программ, таких как специализированные калькуляторы или удобные интерфейсы для сложных инструментов командной строки. В следующей главе мы усовершенствуем концепцию программ с меню.

Дополнительное задание

Важно внимательно изучить программы из этой главы и полностью понять их логическую структуру, так как последующие программы будут становиться всё сложнее. В качестве упражнения перепишите программы этой главы, используя команду `test` вместо составной команды `[ ]`. Подсказка: используйте `grep` для проверки регулярных выражений и оценки кода завершения. Это будет хорошей практикой.

Что почитать

- Справочное руководство Bash содержит главу о встроенных командах, включая `read` command:
<https://www.gnu.org/software/bash/manual/bashref.html#Bash-Builtins>

29 — Управление потоком: циклы

`while/until`

В предыдущей главе мы написали программу с меню для вывода различной системной информации. Программа работает, но у неё есть серьёзный недостаток в удобстве использования. Она выполняет только один выбранный пункт и завершается. Хуже того, при неверном выборе программа завершается с ошибкой, не давая пользователю возможности попробовать снова. Было бы лучше, если бы мы могли перестроить программу так, чтобы она снова и снова выводила меню и принимала выбор, пока пользователь не решит выйти.

В этой главе мы рассмотрим концепцию программирования под названием «цикл», которая позволяет заставлять части программ повторяться. Оболочка предоставляет три составные команды для циклов. В этой главе мы изучим две из них, а третью рассмотрим позже.

Циклы

Повседневная жизнь полна повторяющихся действий.

Ежедневные походы на работу, прогулки с собакой или нарезка моркови — всё это задачи, включающие повторение серии шагов. Рассмотрим нарезку моркови. Если представить это действие в виде псевдокода, оно может выглядеть примерно так:

1. взять разделочную доску
2. взять нож
3. положить морковь на доску
4. поднять нож
5. сдвинуть морковь
6. нарезать морковь
7. если морковь нарезана полностью, то выйти; иначе перейти к шагу 4

Шаги с 4 по 7 образуют цикл. Действия внутри цикла повторяются до тех пор, пока не будет выполнено условие «вся морковь нарезана».

`while`

`bash` позволяет выразить похожую идею. Допустим, мы хотим вывести пять чисел по порядку от 1 до 5. `bash` скрипт можно составить следующим образом:


```
#!/bin/bash
# while-count: display a series of numbers
count=1
while [[ "$count" -le 5 ]]; do
    echo "$count"
    count=$((count + 1))
done
echo "Finished."
```

При выполнении этот скрипт выводит следующее:

```
[me@linuxbox ~]$ while-count
1
2
3
4
5
Finished.
```

Синтаксис команды `while` таков:

```
while commands; do commands; done
```

Как и `if`, `while` проверяет статус выхода списка команд. Пока статус выхода равен нулю, выполняются команды внутри цикла. В предыдущем скрипте создается переменная `count` и ей присваивается начальное значение 1. Команда `while` проверяет статус выхода составной команды `[[ ]]`. Пока команда `[[ ]]` возвращает статус выхода, равный нулю, команды внутри цикла выполняются. В конце каждого цикла повторяется команда `[[ ]]`. После пяти итераций цикла значение `count` увеличивается до 6, команда `[[ ]]` больше не возвращает статус выхода, равный нулю, и цикл завершается. Программа переходит к следующей инструкции после цикла.

Мы можем использовать цикл `while`, чтобы улучшить `read-menu` из предыдущей главы.

```
#!/bin/bash
# while-menu: a menu driven system information program
DELAY=3 # Number of seconds to display results
while [[ "$REPLY" != 0 ]]; do
    clear
    cat << _EOF_
        Please Select:
        1. Display System Information
        2. Display Disk Space
        3. Display Home Space Utilization
        0. Quit
    _EOF_
    read -r -p "Enter selection [0-3] > "
    if [[ "$REPLY" =~ ^[0-3]$ ]]; then
        if [[ $REPLY == 1 ]]; then
            echo "Hostname: $HOSTNAME"
            uptime
            sleep "$DELAY"
        fi
        if [[ "$REPLY" == 2 ]]; then
```

```

        df -h
        sleep "$DELAY"
    fi
    if [[ "$REPLY" == 3 ]]; then
        if [[ "$(id -u)" -eq 0 ]]; then
            echo "Home Space Utilization (All Users)"
            du -sh /home/*
        else
            echo "Home Space Utilization ($USER)"
            du -sh "$HOME"
        fi
        sleep "$DELAY"
    fi
    else
        echo "Invalid entry."
        sleep "$DELAY"
    fi
done
echo "Program terminated."

```

Обернув меню в цикл `while`, мы заставляем программу повторно выводить меню после каждого выбора. Цикл продолжается, пока `REPLY` не равно 0, и меню отображается снова, позволяя пользователю сделать следующий выбор. В конце каждого действия выполняется команда `sleep`, чтобы программа приостановилась на несколько секунд — это позволяет увидеть результат выбора перед очисткой экрана и повторным выводом меню. Как только `REPLY` станет равно 0 (выбор «quit»), цикл завершится, и выполнение продолжится со строки, следующей за `done`.

break и continue

`bash` предоставляет две встроенные команды для управления потоком выполнения внутри циклов. Команда `break` немедленно завершает цикл, и управление программой переходит к следующей инструкции после цикла. Команда `continue` пропускает оставшуюся часть цикла, и управление переходит к следующей итерации. Ниже представлена версия программы `while-menu`, использующая и `break`, и `continue`:

```

#!/bin/bash
# while-menu2: a menu driven system information program
DELAY=3 # Number of seconds to display results
while true; do
    clear
    cat << _EOF_
    Please Select:
    1. Display System Information
    2. Display Disk Space
    3. Display Home Space Utilization
    0. Quit
_EOF_
    read -p "Enter selection [0-3] > "
    if [[ "$REPLY" =~ ^[0-3]$ ]]; then
        if [[ "$REPLY" == 1 ]]; then
            echo "Hostname: $HOSTNAME"

```

```

        uptime
        sleep "$DELAY"
        continue
    fi
    if [[ "$REPLY" == 2 ]]; then
        df -h
        sleep "$DELAY"
        continue
    fi
    if [[ "$REPLY" == 3 ]]; then
        if [[ "$(id -u)" -eq 0 ]]; then
            echo "Home Space Utilization (All Users)"
            du -sh /home/*
        else
            echo "Home Space Utilization ($USER)"
            du -sh "$HOME"
        fi
        sleep "$DELAY"
        continue
    fi
    if [[ "$REPLY" == 0 ]]; then
        break
    fi
else
    echo "Invalid entry."
    sleep "$DELAY"
fi
done
echo "Program terminated."

```

В этой версии скрипта мы создаем бесконечный цикл (который никогда не завершится сам по себе), используя команду `true` для передачи статус выхода в `while`. Поскольку `true` всегда завершается со статусом выхода 0, цикл никогда не закончится. Это удивительно распространенный прием в скриптах. Так как цикл не завершится сам, программист должен предусмотреть способ выхода из него в нужный момент. В этом скрипте команда `break` используется для выхода из цикла при выборе варианта 0. Команда `continue` добавлена в конце остальных вариантов выбора для повышения эффективности. Благодаря `continue` скрипт будет пропускать ненужный код после того, как вариант выбран. Например, если выбран вариант 1, нет смысла проверять остальные варианты.

select

Сейчас было бы уместно упомянуть встроенную команду `select shell`, которая используется для создания циклических меню. Её синтаксис выглядит следующим образом, где `var` — это переменная, а `string` — текст пункта меню:

```
select var in [string..]; do commands; done
```

При выполнении `select` отображаются строки, а затем содержимое переменной `PS3` (prompt string 3) в качестве приглашения для ввода. После выбора переменная `REPLY`

получает ввод пользователя (как и в случае с `read`), а в переменную `var` записывается строка, соответствующая выбору. Когда значения установлены, выполняются команды, и приглашение снова отображается для следующего выбора. Это может звучать запутанно, но мы можем продемонстрировать это на маленьком скрипте:

```
#!/bin/bash
# select-демо: select builtin demo
PS3="
Your choice: "
select my_choice in First Second Third Fourth Quit; do
    echo "REPLY= $REPLY  my_choice= $my_choice"
    [[ "$my_choice" == "Quit" ]] && break
done
```

Сначала мы записываем в переменную `PS3` нужную строку приглашения. Затем выполняем `select`. В этом примере у нас пять строк; хотя мы использовали одиночные слова, можно использовать любой текст в кавычках. В качестве команд мы просто выводим через `echo` значения, присвоенные `select`. Мы также проверяем содержимое переменной `my_choice`, чтобы узнать, выбрал ли пользователь вариант `Quit`, и если да, выполняем `break`, чтобы выйти из цикла.

```
[me@linuxbox ~]$ select-demo
1) First
2) Second
3) Third
4) Fourth
5) Quit
Your choice:
```

При первом выполнении `select` каждая строка выводится с номером, после которого следует наше приглашение. Затем пользователь вводит номер выбранного пункта. Команда `select` записывает в переменную `REPLY` то, что ввел пользователь, и соответствующую строку, если она есть.

```
Your choice: 1
REPLY= 1  my_choice= First
Your choice: 2
REPLY= 2  my_choice= Second
```

Здесь мы видим, что пользователь ввел 1, и команда `echo` выводит значения переменных `REPLY` `my_choice` `select` и `select`. Цикл будет повторять вывод приглашения, пока пользователь не введет «5». Если пользователь введет недопустимое значение, `select` установит `my_choice` в пустую строку. Если пользователь просто введет `ENTER`, это заставит `select` начать сначала и снова показать список меню.

```
Your choice: 6
REPLY= 6 my_choice=
Your choice: abc
REPLY= abc my_choice=
```

Цикл `select` будет продолжаться бесконечно, пока не встретится команда `break` или пока пользователь не введет `Ctrl-d`, чтобы сигнализировать о конце файла.

```
Your choice: 5
REPLY= 5 my_choice= Quit
[me@linuxbox ~]$
```

Одна интересная особенность `select` заключается в том, что она выводит варианты меню или строку приглашения не в стандартный вывод, а в стандартный поток ошибок. Это удобно, так как позволяет перенаправлять только полезную работу команд внутри цикла. Вот пример:

```
[me@linuxbox ~]$ select-demo > choices.txt
```

При таком перенаправлении меню и приглашение по-прежнему отображаются, но вывод `echo command` перенаправляется.

Создадим альтернативную версию нашего скрипта системной информации, заменив прежний цикл `while` на `select`.

```
#!/bin/bash
# select-menu: a menu driven system information program
DELAY=3 # Number of seconds to display results
PS3="
Enter selection [1-4] > "
select str in \
    "Display System Information" \
    "Display Disk Space" \
    "Display Home Space Utilization" \
    "Quit"; do
    if [[ "$REPLY" == "1" ]]; then
        echo "Hostname: $HOSTNAME"
        uptime
        sleep "$DELAY"
        continue
    fi
    if [[ "$REPLY" == "2" ]]; then
        df -h
        sleep "$DELAY"
        continue
    fi
    if [[ "$REPLY" == "3" ]]; then
        if [[ $(id -u) -eq 0 ]]; then
            echo "Home Space Utilization (All Users)"
            du -sh /home/* 2> /dev/null
        else
            echo "Home Space Utilization ($USER)"
            du -sh "$HOME" 2> /dev/null
        fi
        sleep "$DELAY"
        continue
    fi
fi
```

```

if [[ "$REPLY" == "4" ]]; then
    break
fi
if [[ -z "$str" ]]; then
    echo "Invalid entry."
    sleep "$DELAY"
fi
done
echo "Program terminated."

```

В альтернативном скрипте мы задаем переменную PS3, а затем вызываем `select` с четырьмя строками. Хотя можно проверить переменную `str`, установленную `select`, проще проверить переменную `REPLY` и действовать соответственно. В конце цикла мы проверяем, не является ли переменная `str` пустой, что указывает на недопустимое значение.

Какой метод использовать для создания меню? Команда `select` интересна, но, помимо использования стандартного потока ошибок для вывода меню, она почти не экономит усилия при написании кода и сильно ограничивает визуальный дизайн меню.

until

Команда `until` во многом похожа на `while`, за исключением того, что вместо выхода из цикла при ненулевом коде завершения она делает наоборот. Цикл `until` продолжается до тех пор, пока не получит нулевой код завершения. В нашем скрипте `while-count` мы продолжали цикл, пока значение переменной `count` было меньше или равно 5. Мы могли бы добиться того же результата, используя `until`.

```

#!/bin/bash
# until-count: display a series of numbers
count=1
until [[ "$count" -gt 5 ]]; do
    echo "$count"
    count=$((count + 1))
done
echo "Finished."

```

Если изменить условие теста на `$count -gt 5`, `until` завершит цикл вовремя. Выбор между циклами `while` или `until` обычно является вопросом выбора того варианта, который позволяет написать наиболее понятное условие.

Чтение файлов с помощью циклов

`while` и `until` могут обрабатывать стандартный ввод. Это позволяет обрабатывать файлы с помощью циклов `while` и `until`. В

следующем примере мы отобразим содержимое файла `distros.txt`, использованного в предыдущих главах:

```
#!/bin/bash
# while-read: read lines from a file
while read -r distro version release; do
    printf "Distro: %s\tVersion: %s\tReleased: %s\n" \
        "$distro" \
        "$version" \
        "$release"
done < distros.txt
```

Чтобы перенаправить файл в цикл, мы ставим оператор перенаправления после инструкции `done`. Цикл будет использовать `read` для ввода полей из перенаправленного файла. Команда `read` будет завершаться после прочтения каждой строки с нулевым кодом возврата до достижения конца файла. В этот момент она завершится с ненулевым кодом, тем самым прервав цикл. Также можно передать стандартный ввод в цикл через конвейер.

```
#!/bin/bash
# while-read2: read lines from a file
sort -k 1,1 -k 2n distros.txt | while read -r distro version release;
do
    printf "Distro: %s\tVersion: %s\tReleased: %s\n" \
        "$distro" \
        "$version" \
        "$release"
done
```

Здесь мы берем вывод команды `sort` и отображаем поток текста. Однако важно помнить: так как конвейер выполняет цикл в `subshell`, любые переменные, созданные или назначенные внутри цикла, будут потеряны после его завершения.

Итоги

С введением циклов, а также изучением ветвлений, подпрограмм и последовательностей, мы рассмотрели основные типы управления потоком в программах. У `bash` есть еще несколько хитростей, но они лишь уточняют эти базовые концепции.

Что почитать

- В руководстве «The Bash Guide for Beginners» от Linux Documentation Project есть еще примеры циклов `while`:
https://tldp.org/LDP/Bash-Beginners-Guide/html/sect_09_02.html
- В Wikipedia есть статья о циклах, которая является частью большой статьи об управлении потоком:
https://en.wikipedia.org/wiki/Control_flow#Loops

30 — Поиск и устранение неисправностей

Наши скрипты усложнились, поэтому пора рассмотреть ситуации, когда что-то идет не так. В этой главе мы изучим типичные ошибки в скриптах и разберем полезные методы поиска и устранения проблем.

Синтаксические ошибки

Один из общих классов ошибок — синтаксические. Они возникают при опечатках в элементах синтаксиса оболочки. При обнаружении такой ошибки оболочка прекращает выполнение скрипта.

В следующих разделах мы будем использовать этот скрипт для демонстрации типичных ошибок:

```
#!/bin/bash
# trouble: script to demonstrate common errors
number=1
if [ $number = 1 ]; then
    echo "Number is equal to 1."
else
    echo "Number is not equal to 1."
fi
```

В текущем виде этот скрипт выполняется успешно.

```
[me@linuxbox ~]$ trouble
Number is equal to 1.
```

Отсутствующие кавычки

Давайте отредактируем скрипт и удалим лишнюю кавычку в аргументе после первой команды `echo`.

```
#!/bin/bash
# trouble: script to demonstrate common errors
number=1
if [ $number = 1 ]; then
    echo "Number is equal to 1.
else
    echo "Number is not equal to 1."
fi
```

Происходит следующее:

```
[me@linuxbox ~]$ trouble
/home/me/bin/trouble: line 10: unexpected EOF while looking for
matching `"'
/home/me/bin/trouble: line 13: syntax error: unexpected end of file
```


Возникают две ошибки. Интересно, что номера строк в сообщениях об ошибках указывают не на место пропуска кавычки, а на гораздо более поздний участок программы. Если проследить за выполнением после пропуска, станет ясно почему. `bash` продолжит поиск закрывающей кавычки, пока не найдет её — это произойдет сразу после второй команды `echo`. После этого `bash` окончательно запутается. Синтаксис последующей команды `if` будет нарушен, так как оператор `fi` теперь находится внутри кавычек (но строка всё ещё не закрыта).

В длинных скриптах такие ошибки трудно найти. Поможет редактор с подсветкой синтаксиса, так как в большинстве случаев он выделяет строки иначе, чем другие элементы синтаксиса оболочки. Если установлена полная версия `vim`, подсветку можно включить этой командой:

```
:syntax on
```

Отсутствующие или неожиданные токены

Еще одна частая ошибка — незавершенные составные команды, такие как `if` или `while`. Посмотрим, что произойдет, если убрать точку с запятой после `test` в команде `if`:

```
#!/bin/bash
# trouble: script to demonstrate common errors
number=1
if [ $number = 1 ] then
    echo "Number is equal to 1."
else
    echo "Number is not equal to 1."
fi
```

Результат таков:

```
[me@linuxbox ~]$ trouble
/home/me/bin/trouble: line 9: syntax error near unexpected token
`else'
/home/me/bin/trouble: line 9: `else'
```

Опять же, сообщение об ошибке указывает на проблему, которая возникает позже фактической. Происходящее на самом деле довольно интересно. Напомним, что `if` принимает список команд и вычисляет код завершения последней команды в списке. В нашей программе мы предполагаем, что этот список состоит из одной команды, `[`, — синонима `test`. Команда `[` воспринимает всё последующее как список аргументов; в нашем случае это четыре аргумента: `$number`, `1`, `=` и `]`. Если убрать точку с запятой, в список аргументов добавится слово `then`, что синтаксически допустимо. Следующая команда `echo` тоже

допустима. Она интерпретируется как еще одна команда в списке, которую `if` вычислит для получения кода завершения. Затем встречается `else`, но она оказывается неуместной, так как оболочка распознает её как зарезервированное слово (слово, имеющее особое значение для оболочки), а не как имя команды, что и является причиной сообщения об ошибке.

Непредвиденные расширения

В скриптах могут возникать ошибки, проявляющиеся лишь периодически. Иногда скрипт работает исправно, а иногда дает сбой из-за результатов раскрытия. Если мы вернем пропущенную точку с запятой и изменим значение `number` на пустую переменную, как показано здесь, мы сможем продемонстрировать:

```
#!/bin/bash
# trouble: script to demonstrate common errors
number=
if [ $number = 1 ]; then
    echo "Number is equal to 1."
else
    echo "Number is not equal to 1."
fi
```

Запуск скрипта с этим изменением дает следующий результат:

```
[me@linuxbox ~]$ trouble
/home/me/bin/trouble: line 7: [: =: unary operator expected
Number is not equal to 1.
```

Мы получаем довольно загадочное сообщение об ошибке, а затем вывод второй команды `echo`. Проблема заключается в раскрытии `number` переменной внутри `test` команды. Когда команда:

```
[ $number = 1 ]
```

проходит раскрытие, при этом `number` пуста, результатом будет

```
[ = 1 ]
```

что является недопустимым, и возникает ошибка. Оператор `=` — это бинарный оператор (ему требуется значение с каждой стороны), но первое значение отсутствует, поэтому `test` команда ожидает вместо него унарный оператор (например, `-z`). Кроме того, поскольку `test` завершилась неудачей (из-за ошибки), `if` команда получает ненулевой код завершения и действует соответствующим образом, после чего выполняется вторая `echo` команда.

Эту проблему можно решить, заключив первый аргумент в кавычки в `test` команде.

```
[ "$number" = 1 ]
```

Затем, при выполнении подстановки, результат будет таким:

```
[ "" = 1 ]
```

Это дает правильное количество аргументов. Помимо пустых строк, кавычки следует использовать в случаях, когда значение может развернуться в строку из нескольких слов, например, в именах файлов с пробелами.

Примечание: примите за правило всегда заключать переменные и подстановку команд в двойные кавычки, если только вам не требуется разделение слов.

Логические ошибки

В отличие от синтаксических, логические ошибки не мешают запуску скрипта. Скрипт выполнится, но не даст нужного результата из-за ошибки в логике. Существует бесчисленное множество логических ошибок, но вот несколько наиболее распространенных в скриптах:

1. Неверные условные выражения. Легко ошибиться при написании `if/then/else`, что приведет к неверной логике. Иногда логика оказывается инвертированной или неполной.
2. Ошибки «на единицу». При написании циклов с счетчиками можно забыть, что для корректного завершения отсчет должен начинаться с нуля, а не с единицы. Такие ошибки приводят либо к выходу цикла за пределы диапазона, либо к пропуску последней итерации из-за преждевременного завершения.
3. Непредвиденные ситуации. Большинство логических ошибок возникает, когда программа сталкивается с данными или ситуациями, которые программист не учел. Как мы уже видели, это могут быть и непредвиденные расширения — например, имя файла с пробелами, которое разворачивается в несколько аргументов вместо одного имени файла.

Защитное программирование

При программировании важно проверять свои предположения. Это означает тщательную оценку кода возврата программ и

команд, используемых в скрипте. Вот пример из реальной жизни. Один незадачливый системный администратор написал скрипт для обслуживания важного сервера. Скрипт содержал следующие две строки кода:

```
cd $dir_name
rm *
```

В этих двух строках нет ничего предосудительного, пока существует каталог, указанный в переменной `dir_name`. Но что, если его нет? В таком случае команда `cd` завершится ошибкой, скрипт перейдет к следующей строке и удалит файлы в текущем рабочем каталоге. Совсем не тот результат, на который рассчитывали! Из-за такого проектного решения несчастный администратор уничтожил важную часть сервера.

Давайте рассмотрим способы улучшения этого решения. Во-первых, было бы разумно убедиться, что переменная `dir_name` разворачивается в одно слово (используя кавычки), и сделать выполнение `rm` зависимым от успеха `cd`.

```
cd "$dir_name" && rm *
```

Таким образом, если команда `cd` завершится ошибкой, команда `rm` не будет выполнена. Это лучше, но все еще остается вероятность того, что переменная `dir_name` не задана или пуста, что приведет к удалению файлов в домашнем каталоге пользователя. Этого также можно избежать, проверив, действительно ли `dir_name` содержит имя существующего каталога.

```
[[ -d "$dir_name" ]] && cd "$dir_name" && rm *
```

Часто лучше добавить логику для завершения работы скрипта и вывода ошибки при возникновении ситуации, подобной описанной выше.

```
# Delete files in directory $dir_name
if [[ ! -d "$dir_name" ]]; then
    echo "No such directory: '$dir_name'" >&2
    exit 1
fi
if ! cd "$dir_name"; then
    echo "Cannot cd to '$dir_name'" >&2
    exit 1
fi
if ! rm *; then
    echo "File deletion failed. Check results" >&2
    exit 1
fi
```

Здесь мы проверяем и имя (чтобы убедиться, что это существующий каталог), и успех команды `cd`. Если что-то не так, в поток стандартного вывода ошибок отправляется поясняющее сообщение, а скрипт завершается со статусом выхода 1, указывающим на ошибку.

`set -e`, `set -u` и `set -o pipefail`

Заметим, что в `bash`, если при выполнении скрипта команда завершается ошибкой (не считая синтаксических ошибок в самом скрипте), он спокойно переходит к следующей команде. Часто это нежелательно, поэтому стандарт POSIX, а вслед за ним и `bash`, пытаются решить эту проблему. `bash` предлагает настройку для автоматической обработки ошибок: если она включена, скрипт завершится, если любая команда (за некоторыми исключениями) вернет ненулевой статус выхода. Чтобы активировать эту настройку, мы помещаем команду `set -e` в начало скрипта. Многие стандарты написания кода на `bash` рекомендуют использовать эту функцию вместе с парой связанных настроек: `set -u`, которая завершает скрипт при обращении к неинициализированной переменной, и `set -o pipefail`, которая завершает скрипт, если последний элемент в конвейере завершился ошибкой.

Использовать эти возможности не рекомендуется. Лучше предусмотреть надлежащую обработку ошибок, а не полагаться на `set -e` как на замену правилам написания кода.

В Bash FAQ #105 высказывается следующее мнение:

`set -e` был попыткой добавить в оболочку «автоматическое обнаружение ошибок». Его целью было заставить оболочку прерывать работу при возникновении любой ошибки, чтобы вам не приходилось добавлять `|| exit 1` после каждой важной команды.

Эта цель непростая, так как многие команды намеренно возвращают ненулевой код. Например,

```
`if [ -d /foo ]; then...; else...; fi`
```

Очевидно, мы не хотим прерывать выполнение, если `[ -d /foo ]` command возвращает ненулевое значение (так как каталог не существует) — наш скрипт должен обработать это в else part. Поэтому разработчики решили ввести ряд особых правил, вроде «команды, являющиеся частью условия if, не учитываются» или «команды в конвейере, кроме последней, не учитываются».

Эти правила крайне запутанны, и даже они не справляются с некоторыми очень простыми случаями. Хуже того, правила меняются от одной версии Bash к другой, так как bash пытается следовать крайне расплывчатому определению этой «особенности» в стандарте POSIX. Когда задействована подболочка, становится еще сложнее — поведение меняется в зависимости от того, запущен ли bash в режиме POSIX.

ShellCheck — ваш друг

В репозиториях большинства дистрибутивов доступна программа `shellcheck`, которая анализирует скрипты оболочки и обнаруживает множество ошибок и плохих практик написания кода. Ее использование для проверки качества наших скриптов того стоит. Чтобы проверить скрипт с shebang, просто выполните следующее:

```
shellcheck my_script
```

ShellCheck автоматически определит диалект оболочки на основе shebang. Для тестирования кода, не содержащего shebang (например, библиотек функций), можно использовать ShellCheck следующим образом:

```
shellcheck -s bash my_library
```

Используйте опцию `-s`, чтобы указать нужный диалект оболочки. Дополнительную информацию о ShellCheck можно найти на сайте <https://www.shellcheck.net>.

Осторожнее с именами файлов

У этого скрипта удаления файлов есть еще одна проблема — менее очевидная, но потенциально опасная. Многие считают, что Unix (и Unix-подобные ОС) имеет серьезный изъян в проектировании имен файлов. Unix крайне снисходителен к ним. На самом деле, в имя файла нельзя включить только два символа. Первый — это `/`, так как он используется для разделения элементов пути, а второй — нулевой символ (нулевой байт), который используется внутри системы для обозначения конца строки. Все остальное допустимо, включая пробелы, табуляцию, переводы строк, дефисы в начале имени, возврат каретки и так далее.

Особую проблему представляют ведущие дефисы. Например, вполне допустимо иметь файл с именем «`-rf ~`». Задумайтесь на мгновение, что произойдет, если это имя передать в `rm`.

Чтобы избежать этой проблемы, мы изменим `rm` команду в скрипте удаления файлов с

```
rm *  
  
to  
  
rm ./*
```

Это не позволит интерпретировать имя файла, начинающееся с дефиса, как параметр команды. Как правило, всегда предшествуйте подстановочным знакам (таким как `*` и?) символом `./`, чтобы избежать неверной интерпретации командами. Это касается, например, таких вещей, как `*.pdf` и `???.mp3`.

Переносимые имена файлов

Чтобы имя файла было переносимым между различными платформами (т.е. разными типами компьютеров и операционных систем), следует ограничивать набор используемых символов. Существует стандарт POSIX Portable Filename Character Set, который помогает максимально повысить шансы на корректную работу имени файла в разных системах. Стандарт довольно прост. Разрешены только заглавные буквы A–Z, строчные a–z, цифры 0–9, точка (.), дефис (-) и подчеркивание (_). Стандарт также рекомендует, чтобы имена файлов не начинались с дефиса.

Проверка ввода

Общее правило хорошего программирования гласит: если программа принимает ввод, она должна уметь обрабатывать любые полученные данные. Обычно это означает, что ввод нужно тщательно проверять, чтобы для дальнейшей обработки принимались только корректные значения. Мы видели пример этого в предыдущей главе при изучении команды `read`. В одном скрипте содержалась следующая проверка выбора пункта меню:

```
[[ $REPLY =~ ^[0-3]$ ]]
```

Этот тест очень специфичен. Он вернет нулевой статус выхода только в том случае, если введенная пользователем строка является числом в диапазоне от нуля до трех. Ничего другого не будет принято. Иногда такие тесты бывает сложно написать, но эти усилия необходимы для создания качественного скрипта.

Проектирование зависит от времени

Когда я учился в колледже на промышленного дизайнера, один мудрый профессор сказал, что объем дизайна в проекте определяется временем, выделенным дизайнеру. Если вам давали пять минут на проектирование устройства, «убивающего мух», вы проектировали мухобойку. Если же давали пять месяцев, вы могли бы придумать «лазерную систему борьбы с мухами».

Тот же принцип применим и к программированию. Иногда «быстрый и грязный» скрипт вполне подходит, если он будет использован лишь однажды и только самим программистом. Такие скрипты встречаются часто, и их стоит писать быстро, чтобы не тратить лишних усилий. Им не требуется много комментариев и проверок на ошибки. С другой стороны, если скрипт предназначен для промышленной эксплуатации — то есть будет постоянно использоваться для важных задач или разными пользователями — он требует гораздо более тщательной разработки.

Тестирование

Тестирование — важный этап любого процесса разработки ПО, включая скрипты. В мире открытого исходного кода существует принцип «выпускай раньше, выпускай чаще», который отражает этот факт. Благодаря ранним и частым выпускам программное обеспечение подвергается большему количеству проверок и испытаний. Опыт показывает, что ошибки гораздо проще найти и гораздо дешевле исправить, если они обнаружены на ранних стадиях цикла разработки.

В главе 26 мы рассматривали, как использовать заглушки для проверки потока выполнения программы. С самых ранних этапов разработки с криптом они служат ценным инструментом для контроля хода работы.

Рассмотрим ранее описанную проблему удаления файлов и поймем, как написать код для удобного тестирования.

Тестировать исходный фрагмент опасно, так как он предназначен для удаления файлов, но мы можем изменить код, чтобы сделать тест безопасным.

```
if [[ -d $dir_name ]]; then
    if cd $dir_name; then
        echo rm * # TESTING
    else
        echo "cannot cd to '$dir_name'" >&2
    fi
fi
```



```
        exit 1
    fi
else
    echo "no such directory: '$dir_name'" >&2
    exit 1
fi
exit # TESTING
```

Поскольку условия ошибки уже выводят полезные сообщения, добавлять ничего не нужно. Самое важное изменение — добавление команды `echo` перед командой `rm`, чтобы отобразить саму команду и её развернутый список аргументов вместо фактического выполнения. Это позволяет безопасно проверять код. В конце фрагмента кода мы ставим команду `exit`, чтобы завершить тест и предотвратить выполнение остальной части скрипта. Необходимость в этом зависит от логики скрипта.

Мы также добавляем комментарии, служащие «маркерами» для изменений, связанных с тестами. Они помогут найти и удалить эти изменения после завершения тестирования.

Тестовые случаи

Для эффективного тестирования важно разрабатывать и применять качественные тестовые случаи. Это достигается тщательным подбором входных данных или условий эксплуатации, отражающих граничные и экстремальные случаи. В нашем фрагменте кода (который прост) мы хотим проверить его работу при трех конкретных условиях:

1. `dir_name` содержит имя существующего каталога.
2. `dir_name` содержит имя несуществующего каталога.
3. `dir_name` пуст.

Выполняя тестирование при каждом из этих условий, можно добиться хорошего покрытия.

Как и проектирование, тестирование требует времени. Не каждую функцию скрипта нужно тестировать глубоко. Главное — определить, что наиболее важно. Поскольку сбой может привести к разрушительным последствиям, наш фрагмент кода заслуживает тщательного внимания как при проектировании, так и при тестировании.

Отладка

Если тестирование выявило проблему в скрипте, следующим шагом становится отладка. «Проблема» обычно означает, что скрипт каким-то образом работает не так, как ожидал программист. В этом случае нужно точно определить, что

именно делает скрипт и почему. Поиск багов иногда напоминает настоящую детективную работу.

Хорошо спроектированный скрипт должен помогать пользователю. Его следует писать с учетом защиты от ошибок, чтобы он мог обнаруживать аномальные условия и выдавать полезную обратную связь. Однако иногда возникают странные и неожиданные проблемы, требующие более сложных методов решения.

Поиск проблемной области

В некоторых скриптах, особенно длинных, полезно изолировать участок кода, связанный с проблемой. Это не всегда укажет на саму ошибку, но часто поможет понять её истинную причину. Один из методов изоляции кода — «комментирование» разделов скрипта. Например, наш фрагмент удаления файлов можно изменить, чтобы проверить, связан ли удаленный участок с ошибкой.

```
if [[ -d $dir_name ]]; then
    if cd $dir_name; then
        rm *
    else
        echo "cannot cd to '$dir_name'" >&2
        exit 1
    fi
# else
#   echo "no such directory: '$dir_name'" >&2
#   exit 1
fi
```

Добавляя символы комментария в начало каждой строки в логическом разделе скрипта, мы предотвращаем его выполнение. Затем можно провести повторное тестирование, чтобы проверить, влияет ли удаление кода на поведение ошибки.

Трассировка

Ошибки часто связаны с неожиданным логическим потоком внутри скрипта. То есть части скрипта либо никогда не выполняются, либо выполняются в неверном порядке или в неверное время. Чтобы увидеть реальный поток программы, мы используем метод под названием трассировка.

Один из методов трассировки — добавление в скрипт информационных сообщений, указывающих на место выполнения. Мы можем добавить такие сообщения в наш фрагмент кода.

```
echo "preparing to delete files" >&2
if [[ -d $dir_name ]]; then
    if cd $dir_name; then
```

```

echo "deleting files" >&2
    rm *
else
    echo "cannot cd to '$dir_name'" >&2
    exit 1
fi
else
    echo "no such directory: '$dir_name'" >&2
    exit 1
fi
echo "file deletion complete" >&2

```

Мы отправляем сообщения в standard error, чтобы отделить их от обычного вывода. Мы также не делаем отступов в строках с сообщениями, чтобы их было проще найти и удалить.

Теперь при выполнении скрипта можно увидеть, что удаление файлов прошло успешно.

```

[me@linuxbox ~]$ deletion-script
preparing to delete files
deleting files
file deletion complete
[me@linuxbox ~]$

```

bash также предоставляет метод трассировки, реализованный через опцию -x и команду set с опцией -x. Используя наш предыдущий trouble скрипт, мы можем включить трассировку для всего скрипта, добавив опцию -x в первую строку.

```

#!/bin/bash -x
# trouble: script to demonstrate common errors
number=1
if [ $number = 1 ]; then
    echo "Number is equal to 1."
else
    echo "Number is not equal to 1."
fi

```

При выполнении результат выглядит так:

```

[me@linuxbox ~]$ trouble
+ number=1
+ '[' 1 = 1 ']'
+ echo 'Number is equal to 1.'
Number is equal to 1.

```

При включённом трассировании мы видим выполняемые команды с применёнными подстановками. Ведущие знаки «+» указывают на строки трассировки, отделяя их от обычного вывода. Знак «+» является символом трассировки по умолчанию. Он содержится в переменной оболочки PS4 (prompt string 4). Содержимое этой переменной можно изменить, чтобы сделать приглашение более полезным. Здесь мы модифицируем переменную, чтобы добавить номер текущей строки скрипта, на которой выполняется трассировка. Обратите внимание, что

требуются одинарные кавычки, чтобы предотвратить подстановку до фактического использования приглашения.

```
[me@linuxbox ~]$ export PS4='$LINENO + '
[me@linuxbox ~]$ trouble
5 + number=1
7 + '[' 1 = 1 ']'
8 + echo 'Number is equal to 1.'
Number is equal to 1.
```

Чтобы выполнить трассировку не всего скрипта, а его выбранной части, можно использовать команду `set` с опцией `-x`.

```
#!/bin/bash
# trouble: script to demonstrate common errors
number=1
set -x # Turn on tracing
if [ $number = 1 ]; then
    echo "Number is equal to 1."
else
    echo "Number is not equal to 1."
fi
set +x # Turn off tracing
```

Мы используем команду `set` с опцией `-x` для включения трассировки и опцией `+x` для её выключения. Этот метод позволяет изучать различные участки проблемного скрипта.

Проверка значений во время выполнения

Вместе с трассировкой полезно выводить содержимое переменных, чтобы видеть внутреннюю работу скрипта во время выполнения. Обычно для этого достаточно добавить дополнительные инструкции `echo`.

```
#!/bin/bash
# trouble: script to demonstrate common errors
number=1
echo "number=$number" # DEBUG
set -x # Turn on tracing
if [ $number = 1 ]; then
    echo "Number is equal to 1."
else
    echo "Number is not equal to 1."
fi
set +x # Turn off tracing
```

В этом простом примере мы просто выводим значение переменной `number` и помечаем добавленную строку комментарием, чтобы её было легче найти и удалить позже. Этот метод особенно полезен при отслеживании работы циклов и арифметических операций в скриптах.

Итоги

В этой главе мы рассмотрели лишь некоторые проблемы, которые могут возникнуть при разработке скриптов. Конечно, их гораздо больше. Описанные здесь методы позволят находить большинство распространенных ошибок. Отладка — это искусство, которое развивается с опытом: как в умении избегать ошибок (постоянное тестирование в процессе разработки), так и в умении их находить (эффективное использование трассировки).

Что почитать

- В Википедии есть несколько коротких статей о синтаксических и логических ошибках:
https://en.wikipedia.org/wiki/Syntax_error
https://en.wikipedia.org/wiki/Logic_error
- Существует множество онлайн-ресурсов по техническим аспектам программирования на bash:
<https://mywiki.woledge.org/BashPitfalls>
<https://tldp.org/LDP/abs/html/gotchas.html>
https://www.gnu.org/software/bash/manual/html_node/Reserved-Word-Index.html
- Дэвид Уилер предлагает отличный разбор проблемы имен файлов в Unix и способов написания скриптов оболочки для работы с ними: <https://www.dwheeler.com/essays/ filenames-in-shell.html>
- Для серьезной отладки существует Bash Debugger:
<https://bashdb.sourceforge.net/>

31 — Управление потоком: ветвление с помощью case

В этой главе мы продолжим изучение управления потоком. В главе 28 мы создавали простые меню и логику обработки выбора пользователя. Для этого мы использовали ряд `if` команд, чтобы определить, какой из вариантов был выбран. Подобные логические конструкции часто встречаются в программах — настолько часто, что многие языки программирования (включая оболочку) предоставляют специальные механизмы управления потоком для выбора из нескольких вариантов.

case

В `bash` составная команда с множественным выбором называется `case`. Её синтаксис выглядит так:

```
case word in [pattern [| pattern]...) commands
;;]...esac
```

Если рассмотреть программу `read-menu` из главы 28, можно увидеть логику обработки выбора пользователя.

```
#!/bin/bash
# read-menu: a menu driven system information program
clear
echo "
Please Select:
1. Display System Information
2. Display Disk Space
3. Display Home Space Utilization
0. Quit
"
read -r -p "Enter selection [0-3] > "
if [[ "$REPLY" =~ ^[0-3]$ ]]; then
    if [[ "$REPLY" == 0 ]]; then
        echo "Program terminated."
        exit
    fi
    if [[ "$REPLY" == 1 ]]; then
        echo "Hostname: $HOSTNAME"
        uptime
        exit
    fi
    if [[ "$REPLY" == 2 ]]; then
        df -h
        exit
    fi
    if [[ "$REPLY" == 3 ]]; then
        if [[ "$(id -u)" -eq 0 ]]; then
            echo "Home Space Utilization (All Users)"
            du -sh /home/*
        else
            echo "Home Space Utilization ($USER)"
            du -sh "$HOME"
        fi
    fi
fi
```

```

        exit
    fi
else
    echo "Invalid entry." >&2
    exit 1
fi

```

Используя case, мы можем заменить эту логику на более простую.

```

#!/bin/bash
# case-menu: a menu driven system information program
clear
echo "
Please Select:
1. Display System Information
2. Display Disk Space
3. Display Home Space Utilization
0. Quit
"
read -r -p "Enter selection [0-3] > "
case "$REPLY" in
    0) echo "Program terminated."
        exit
        ;;
    1) echo "Hostname: $HOSTNAME"
        uptime
        ;;
    2) df -h
        ;;
    3) if [[ "$(id -u)" -eq 0 ]]; then
        echo "Home Space Utilization (All Users)"
        du -sh /home/*
        else
        echo "Home Space Utilization ($USER)"
        du -sh "$HOME"
        fi
        ;;
    *) echo "Invalid entry" >&2
        exit 1
        ;;
esac

```

Команда case проверяет значение слова (в нашем примере — значение REPLY переменной) и пытается сопоставить его с одним из указанных шаблонов. При нахождении совпадения выполняются команды, связанные с этим шаблоном. После первого совпадения поиск прекращается.

Шаблоны

Шаблоны, используемые case, такие же, как и при раскрытии пути. Шаблоны завершаются символом). В таблице 31-1 приведены примеры допустимых шаблонов.

Таблица 31-1: Примеры case шаблонов

Подстановочный знак	Описание
------------------------	----------

Подстановочный знак	Описание
a)	Совпадение, если слово равно «a».
[[:alpha:]])	Соответствует, если слово состоит из одного буквенного символа.
???)	Совпадение, если слово состоит ровно из трех символов.
*.txt)	Совпадение, если слово заканчивается на «.txt».
*)	Совпадение с любым значением word. Рекомендуется включать это в качестве последнего шаблона в case command, чтобы перехватить любые значения word, не подошедшие под предыдущие шаблоны, то есть любые возможные недопустимые значения.

Вот пример работы шаблонов:

```
#!/bin/bash
read -r -p "Enter word > "
case "$REPLY" in
    [[:alpha:]] ) echo "is a single alphabetic character." ;;
    [ABC][0-9]) echo "is A, B, or C followed by a digit." ;;
    ???)        echo "is three characters long." ;;
    *.txt)      echo "is a word ending in '.txt'" ;;
    *)          echo "is something else." ;;
esac
```

Можно также объединять несколько шаблонов, используя вертикальную черту в качестве разделителя. Это создает условный шаблон «или». Это полезно, например, для обработки символов в разных регистрах. Вот пример:

```
#!/bin/bash
# case-menu: a menu driven system information program
clear
echo "
Please Select:
A. Display System Information
B. Display Disk Space
C. Display Home Space Utilization
Q. Quit
"
read -r -p "Enter selection [A, B, C or Q] > "
case "$REPLY" in
    q|Q) echo "Program terminated."
        exit
        ;;
    a|A) echo "Hostname: $HOSTNAME"
        uptime
        ;;
    b|B) df -h
```



```

;;
c|C) if [[ "$(id -u)" -eq 0 ]]; then
    echo "Home Space Utilization (All Users)"
    du -sh /home/*
else
    echo "Home Space Utilization ($USER)"
    du -sh "$HOME"
fi
;;
*) echo "Invalid entry" >&2
   exit 1
;;
esac

```

Здесь мы модифицируем case-меню program, чтобы для выбора пунктов меню вместо цифр использовались буквы. Обратите внимание, что новые шаблоны позволяют вводить как заглавные, так и строчные буквы.

Выполнение нескольких действий

В версиях bash до 4.0 case позволял выполнить только одно действие при успешном совпадении. После успешного совпадения команда завершалась. Рассмотрим скрипт, проверяющий символ:

```

#!/bin/bash
# case4-1: test a character
read -r -n 1 -p "Type a character > "
echo
case "$REPLY" in
    [:upper:]) echo "'$REPLY' is upper case." ;;
    [:lower:]) echo "'$REPLY' is lower case." ;;
    [:alpha:]) echo "'$REPLY' is alphabetic." ;;
    [:digit:]) echo "'$REPLY' is a digit." ;;
    [:graph:]) echo "'$REPLY' is a visible character." ;;
    [:punct:]) echo "'$REPLY' is a punctuation symbol." ;;
    [:space:]) echo "'$REPLY' is a whitespace character." ;;
    [:xdigit:]) echo "'$REPLY' is a hexadecimal digit." ;;
esac

```

Запуск этого скрипта дает следующий результат:

```

[me@linuxbox ~]$ case4-1
Type a character > a
'a' is lower case.

```

Скрипт работает в большинстве случаев, но дает сбой, если символ соответствует более чем одному классу символов POSIX. Например, символ «a» является одновременно строчным и буквенным, а также шестнадцатеричной цифрой. В bash версиях bash до 4.0 не было возможности case проверить несколько условий. В современных версиях bash добавлена нотация ;& для завершения каждого действия, так что теперь можно сделать так:

```

#!/bin/bash
# case4-2: test a character

```

```

read -r -n 1 -p "Type a character > "
echo
case "$REPLY" in
    [:upper:])    echo "'$REPLY' is upper case." ;;&
    [:lower:])    echo "'$REPLY' is lower case." ;;&
    [:alpha:])    echo "'$REPLY' is alphabetic." ;;&
    [:digit:])    echo "'$REPLY' is a digit." ;;&
    [:graph:])    echo "'$REPLY' is a visible character." ;;&
    [:punct:])    echo "'$REPLY' is a punctuation symbol." ;;&
    [:space:])    echo "'$REPLY' is a whitespace character." ;;&
    [:xdigit:])   echo "'$REPLY' is a hexadecimal digit." ;;&
esac

```

При запуске этого скрипта мы получаем следующее:

```

[me@linuxbox ~]$ case4-2
Type a character > a
'a' is lower case.
'a' is alphabetic.
'a' is a visible character.
'a' is a hexadecimal digit.

```

Добавление синтаксиса `;;&` позволяет `case` переходить к следующему тесту вместо простого завершения.

Итоги

Команда `case` — полезное дополнение к нашему арсеналу приемов программирования. Как мы увидим в следующей главе, это идеальный инструмент для решения определенных типов задач.

Что почитать

- В разделе Conditional Constructs справочного руководства Bash подробно описана команда `case`:
<https://tiswww.case.edu/php/chet/bash/bashref.html#SEC21>
- В руководстве Advanced Bash-Scripting Guide приведены дополнительные примеры применения `case`:
<https://tldp.org/LDP/abs/html/testbranch.html>

32 — Позиционные параметры

До сих пор в наших программах не хватало возможности принимать и обрабатывать параметры и аргументы командной строки. В этой главе мы изучим возможности оболочки, позволяющие программам получать доступ к содержимому командной строки.

Доступ к командной строке

Оболочка предоставляет набор переменных, называемых позиционными параметрами, которые содержат отдельные слова командной строки. Переменные называются от 0 до 9. Их можно продемонстрировать следующим образом:

```
#!/bin/bash
# posit-param: script to view command line parameters
echo "
\$0 = $0
\$1 = $1
\$2 = $2
\$3 = $3
\$4 = $4
\$5 = $5
\$6 = $6
\$7 = $7
\$8 = $8
\$9 = $9
"
```

Это простой скрипт, который выводит значения переменных от \$0 до \$9. При запуске без аргументов командной строки результат будет таким:

```
[me@linuxbox ~]$ posit-param
$0 = /home/me/bin/posit-param
$1 =
$2 =
$3 =
$4 =
$5 =
$6 =
$7 =
$8 =
$9 =
```

Даже если аргументы не указаны, \$0 всегда содержит первый элемент в командной строке — путь к исполняемой программе. При наличии аргументов результаты будут следующими:

```
[me@linuxbox ~]$ posit-param a b c d
$0 = /home/me/bin/posit-param
$1 = a
$2 = b
$3 = c
```

```
$4 = d
$5 =
$6 =
$7 =
$8 =
$9 =
```

Примечание: С помощью раскрытия параметров можно получить доступ и к большему количеству, чем девять параметров. Чтобы указать число больше девяти, заключите его в фигурные скобки, как в `${10}`, `${55}`, `${211}` и так далее.

Определение количества аргументов

Оболочка также предоставляет переменную `$#`, содержащую количество аргументов в командной строке.

```
#!/bin/bash
# posit-param: script to view command line parameters
echo "
Number of arguments: $#
\$0 = $0
\$1 = $1
\$2 = $2
\$3 = $3
\$4 = $4
\$5 = $5
\$6 = $6
\$7 = $7
\$8 = $8
\$9 = $9
"
```

Вот результат:

```
[me@linuxbox ~]$ posit-param a b c d
Number of arguments: 4
$0 = /home/me/bin/posit-param
$1 = a
$2 = b
$3 = c
$4 = d
$5 =
$6 =
$7 =
$8 =
$9 =
```

shift — Доступ к множеству аргументов

Но что произойдет, если передать программе большое количество аргументов, например, следующих?

```
[me@linuxbox ~]$ posit-param *
Number of arguments: 82
$0 = /home/me/bin/posit-param
```

```
$1 = addresses.ldif
$2 = bin
$3 = bookmarks.html
$4 = debian-500-i386-netinst.iso
$5 = debian-500-i386-netinst.jigdo
$6 = debian-500-i386-netinst.template
$7 = debian-cd_info.tar.gz
$8 = Desktop
$9 = dirlist-bin.txt
```

В данном примере подстановочный знак * разворачивается в 82 аргумента. Как обработать такое количество? Оболочка предлагает метод, пусть и неуклюжий. Команда `shift` при каждом выполнении заставляет все параметры «сдвинуться на один вниз». На самом деле, используя `shift`, можно обойтись всего одним параметром (в дополнение к `$0`, который никогда не меняется).

```
#!/bin/bash
# posit-param2: script to display all arguments
count=1
while [[ $# -gt 0 ]]; do
    echo "Argument $count = $1"
    count=$((count + 1))
    shift
done
```

Каждый раз при выполнении `shift` значение `$2` перемещается в `$1`, значение `$3` — в `$2` и так далее. Значение `$#` также уменьшается на единицу.

В программе `posit-param2` мы создаем цикл, который проверяет количество оставшихся аргументов и продолжает работу, пока их есть хотя бы один. Мы выводим текущий аргумент, на каждой итерации увеличиваем переменную `count`, чтобы вести счет обработанных аргументов, и, наконец, выполняем `shift`, чтобы загрузить `$1` следующим аргументом. Вот как работает программа:

```
[me@linuxbox ~]$ posit-param2 a b c d
Argument 1 = a
Argument 2 = b
Argument 3 = c
Argument 4 = d
```

Простые приложения

Даже без `shift` можно писать полезные приложения, используя позиционные параметры. Вот пример простой программы для получения информации о файле:

```
#!/bin/bash
# file-info: simple file information program
PROGNAME="$(basename "$0")"
```

```

if [[ -e "$1" ]]; then
    echo -e "\nFile Type:"
    file "$1"
    echo -e "\nFile Status:"
    stat "$1"
else
    echo "$PROGNAME: usage: $PROGNAME file" >&2
    exit 1
fi

```

Эта программа отображает тип файла (определяемый командой `file`) и его статус (из команды `stat`) для указанного файла. Одна интересная особенность программы — переменная `PROGNAME`. Ей присваивается значение, полученное в результате выполнения команды `basename "$0"`. Команда `basename` удаляет начальную часть пути, оставляя только базовое имя файла. В нашем примере `basename` удаляет начальную часть пути, содержащегося в параметре `$0` — полный путь к нашей программе. Это значение полезно при составлении сообщений, например, справки по использованию в конце программы. Благодаря такому подходу скрипт можно переименовать, и сообщение автоматически обновится, содержа в себе новое имя программы.

Использование позиционных параметров в функциях оболочки

Так же как иональные параметры используются для передачи аргументов в сценарии оболочки, они также могут использоваться для передачи аргументов в функции оболочки. Для демонстрации мы преобразуем `file_info script` в функцию оболочки.

```

file_info () {
    # file_info: function to display file information
    if [[ -e "$1" ]]; then
        echo -e "\nFile Type:"
        file "$1"
        echo -e "\nFile Status:"
        stat "$1"
    else
        echo "$FUNCNAME: usage: $FUNCNAME file" >&2
        return 1
    fi
}

```

Теперь, если скрипт, использующий функцию `file_info shell`, вызывает эту функцию с аргументом в виде имени файла, аргумент будет передан функции.

Благодаря этой возможности мы можем писать множество полезных функций оболочки, которые можно использовать не только в скриптах, но и в наших `.bashrc` файлах.

Заметьте, что `PROGNAME` изменилась на переменную оболочки `FUNCNAME`. Оболочка автоматически обновляет эту переменную, чтобы отслеживать текущую выполняемую функцию оболочки. Обратите внимание, что `$0` всегда содержит полный путь к первому элементу в командной строке (т. е. имя программы) и, вопреки ожиданиям, не содержит имени функции оболочки.

Массовая обработка позиционных параметров

Иногда полезно управлять всеми позиционными параметрами как единой группой. Например, мы можем захотеть создать «оболочку» вокруг другой программы. Это означает создание скрипта или функции оболочки, которая упрощает вызов другой программы. В данном случае оболочка предоставляет список сложных параметров командной строки, а затем передает список аргументов низкоуровневой программе.

Для этой цели оболочка предоставляет два специальных параметра. Оба они разворачиваются в полный список позиционных параметров, но различаются в некоторых тонких деталях. Они описаны в таблице 32-1.

Таблица 32-1: Специальные параметры `*` и `@` орых

Параметр	Описание
<code>\$*</code>	Разворачивается в список позиционных параметров, начиная с 1. При использовании двойных кавычек разворачивается в двойной — заключенная в кавычки строка, содержащая все позиционные параметры, каждый из которых разделен первым символом переменной оболочки <code>IFS</code> (по умолчанию это символ пробела).
<code>\$@</code>	Разворачивается в список позиционных параметров, начиная с 1. В двойных кавычках каждый позиционный параметр разворачивается как отдельное слово, как если бы он был в двойных кавычках.

Вот скрипт, демонстрирующий работу этих специальных параметров:

```
#!/bin/bash
# posit-params3: script to demonstrate $* and $@
print_params () {
    echo "\$1 = $1"
    echo "\$2 = $2"
    echo "\$3 = $3"
```

```

        echo "\$4 = $4"
    }
    pass_params () {
        echo -e "\n" '$* :';   print_params $*
        echo -e "\n" '"$*" :'; print_params "$*"
        echo -e "\n" '$@ :';   print_params $@
        echo -e "\n" '"$@" :'; print_params "$@"
    }
    pass_params "word" "words with spaces"

```

В этой довольно запутанной программе мы создаем два аргумента, `word words with spaces` `pass_params` и `print_params`, и передаем их функции. Та, в свою очередь, передает их функции `$*`, используя каждый из четырех методов, доступных с помощью специальных параметров `$@` и. При выполнении скрипт демонстрирует различия.

```

[me@linuxbox ~]$ posit-param3
$* :
$1 = word
$2 = words
$3 = with
$4 = spaces
"$*" :
$1 = word words with spaces
$2 =
$3 =
$4 =
$@ :
$1 = word
$2 = words
$3 = with
$4 = spaces
"$@" :
$1 = word
$2 = words with spaces
$3 =
$4 =

```

В обоих случаях, при использовании `$*` и `$@`, результат состоит из четырех слов.

`word words with spaces`

`"$*" дает результат из одного слова. "word words with spaces"`

`"word words with spaces"`

`"$@" дает результат из двух слов. "word" "words with spaces"`

`"word" "words with spaces"`

Это соответствует нашему намерению. Главный урок здесь в том, что, хотя оболочка предлагает четыре способа получения списка позиционных параметров, `"$@"` — самый полезный в большинстве ситуаций, так как он сохраняет целостность каждого параметра. Для обеспечения безопасности его следует использовать всегда, если нет веских причин этого не делать.

Более полное применение

После долгого перерыва мы возобновляем работу над нашей `sys_info_page` программой, которую рассматривали в главе 27. Следующим шагом мы добавим в программу несколько опций командной строки:

- Выходной файл. Мы добавим опцию для указания имени файла, в который будет записываться вывод программы. Она будет задаваться как `-f file`, так и `--file file`.
- Интерактивный режим. Эта опция запросит у пользователя имя выходного файла и проверит, существует ли он. Если файл существует, программа запросит подтверждение перед его перезаписью. Эта опция будет задаваться как `-i`, так и `--interactive`.
- Справка. Можно указать `-h` или `--help`, чтобы программа вывела информационное сообщение о способе использования.

Вот код, необходимый для обработки командной строки:

```
usage () {
    echo "$PROGNAME: usage: $PROGNAME [-f file | -i]"
    return
}
# process command line options
interactive=
filename=
while [[ -n "$1" ]]; do
    case "$1" in
        -f | --file)      shift
                           filename="$1"
                           ;;
        -i | --interactive) interactive=1
                           ;;
        -h | --help)      usage
                           exit
                           ;;
        *)                 usage >&2
                           exit 1
                           ;;
    esac
    shift
done
```

Сначала мы добавим функцию оболочки под названием `usage`, которая выводит сообщение при вызове опции справки или попытке использовать неизвестную опцию.

Затем мы запускаем цикл обработки. Этот цикл выполняется до тех пор, пока позиционный параметр `$1` не станет пустым. В конце цикла используется команда `shift` для сдвига позиционных параметров, чтобы цикл в конечном итоге завершился.

Внутри цикла находится оператор `case`, который проверяет текущий позиционный параметр на соответствие поддерживаемым вариантам. Если поддерживаемый параметр найден, выполняется соответствующее действие. Если найден неизвестный вариант, выводится сообщение о способе использования, и скрипт завершается с ошибкой.

Параметр `-f` обрабатывается интересным способом. При его обнаружении происходит дополнительный `shift`, который сдвигает позиционный параметр `$1` к аргументу-имени файла, переданному опции `-f`.

Опция `getopts`

Ранее показанный код разбора позиционных параметров — хорошее решение, но не единственный возможный подход. Существует встроенная команда оболочки `getopts` (не путать с одноименной внешней командой `getopt`), которая может выполнить часть работы за нас. Каждый раз при вызове `getopts` (обычно в цикле) она возвращает текущий аргумент в указанную переменную и увеличивает счетчик `OPTIND`, чтобы указать на следующий позиционный параметр. Если опции требуется аргумент, он возвращается в переменную `OPTARG`.

Синтаксис `getopts` выглядит так:

```
getopts optstring var [arg ...]
```

Аргумент `optstring` — это строка, состоящая из имен параметров, представленных одиночными символами. `getopts` не поддерживает (полноценно) длинные параметры. Кроме того, если после имени параметра стоит двоеточие, это означает, что параметру требуется аргумент.

Хотя `getopts` поддерживает только односимвольные имена параметров, он позволяет объединять несколько параметров в одну строку без дефисов, как мы видели на примере многих команд, таких как `ls`.

```
ls -la
```

Если `optstring` начинается с двоеточия, `getopts` не будет выводить сообщения об ошибках при наличии неверного параметра или отсутствия обязательного аргумента.

Аргумент `var` — это имя переменной, в которой будет храниться имя текущего параметра.

Обычно `getopts` обрабатывает позиционные параметры, но он также может обрабатывать любые дополнительные аргументы, перечисленные после `var`.

Встроенная команда `getopts` возвращает успешный статус выхода до тех пор, пока у неё не закончатся аргументы для обработки.

Да, это кажется немного сложным, но всё становится понятнее на практике. Используя `getopts`, мы можем написать короткую демонстрацию этого альтернативного метода.

```
#!/bin/bash
# getopts-test: process command line options using getopts
PROGNAME="$(basename "$0")"
interactive=
filename=
usage () {
    echo "$PROGNAME: usage: $PROGNAME [-f file | -i]"
    return
}
while getopts :f:ih opt; do
    case "$opt" in
        f) filename="$OPTARG" ;;
        i) interactive=1 ;;
        h) usage ;;
        \?) echo "option '$OPTARG' invalid" ;;
        :) echo "option '$OPTARG' missing argument" ;;
    esac
done
echo "interactive = '$interactive' filename = '$filename'"
```

Давайте подробно разберем этот код. Мы начинаем с привычного определения переменных и функции для вывода сообщения о правильном использовании. Затем переходим к самому интересному: помещаем команду `getopts` в цикл `while`. Наш `optstring` начинается с двоеточия, которое подавляет сообщения об ошибках. За двоеточием следуют буквы опций. Опция `f` требует аргумент (в данном случае имя файла), поэтому после этой буквы стоит двоеточие. Наконец, мы указываем переменную (`opt`) для хранения результата. Цикл `while` продолжается до тех пор, пока у `getopts` не закончатся параметры для обработки.

В цикле `while` содержится `case`, обрабатывающее результат, возвращаемый в `opt`.

Все выглядит ожидаемо, за исключением двух последних шаблонов в `case`. `?` возвращается `getopts`, когда обнаружен недопустимый параметр (т. е. буква, отсутствующая в списке). Обратите внимание, что в `case` необходимо экранировать вопросительный знак, иначе оболочка интерпретирует его как подстановочный знак для файлов. Далее следует `:`, который

возвращается `getopts`, если параметру не хватает обязательного аргумента. При возникновении любого из этих двух условий ошибки `getopts` помещает ошибочную букву параметра в `OPTARG`.

Посмотрим на эту демонстрацию в действии:

```
[me@linuxbox ~]$ getopts-test
interactive = '' filename = ''
[me@linuxbox ~]$ getopts-test -i
interactive = '1' filename = ''
[me@linuxbox ~]$ getopts-test -f foo.html
interactive = '' filename = 'foo.html'
[me@linuxbox ~]$ getopts-test -if foo.html
interactive = '1' filename = 'foo.html'
[me@linuxbox ~]$ getopts-test -i -f foo.html
interactive = '1' filename = 'foo.html'
[me@linuxbox ~]$ getopts-test -a
option 'a' invalid
interactive = '' filename = ''
[me@linuxbox ~]$ getopts-test -f
option 'f' missing argument
interactive = '' filename = ''
```

Какую же технику выбрать: `while/shift` или `getopts`? Все зависит от наших задач. Метод `while/shift` дает максимальный контроль (включая легкую реализацию длинных имен опций), тогда как `getopts` требует меньше кода и поддерживает синтаксис нескольких опций с одним дефисом.

Интерактивный режим

Теперь, когда код позиционного параметра готов, реализуем интерактивный режим.

```
# interactive mode
if [[ -n "$interactive" ]]; then
    while true; do
        read -r -p "Enter name of output file: " filename
        if [[ -e "$filename" ]]; then
            read -r -p "'$filename' exists. Overwrite? [y/n/q] > "
            case "$REPLY" in
                Y|y) break
                    ;;
                Q|q) echo "Program terminated."
                    exit
                    ;;
                *) continue
                    ;;
            esac
        elif [[ -z "$filename" ]]; then
            continue
        else
            break
        fi
    done
fi
```

Если переменная `interactive` не пуста, запускается бесконечный цикл, содержащий приглашение с именем файла и последующий код обработки существующих файлов. Если нужный выходной файл уже существует, пользователю предлагается перезаписать его, выбрать другое имя или выйти из программы. Если пользователь выбирает перезапись, выполняется `break` для завершения цикла. Обратите внимание, что оператор `case` определяет только выбор между перезаписью или выходом. Любой другой вариант заставляет цикл продолжаться и снова выводит приглашение.

Вывод в файл

Чтобы реализовать функцию указания имени выходного файла, нам сначала нужно преобразовать существующий код записи страниц в функцию оболочки — по причинам, которые станут ясны через мгновение.

```
write_html_page () {
    cat << _EOF_
<html>
  <head>
    <title>$TITLE</title>
  </head>
  <body>
    <h1>$TITLE</h1>
    <p>$TIMESTAMP</p>
    $(report_uptime)
    $(report_disk_space)
    $(report_home_space)
  </body>
</html>
_EOF_
    return
}
# output html page
if [[ -n "$filename" ]]; then
    if touch "$filename" && [[ -f "$filename" ]]; then
        write_html_page > "$filename"
    else
        echo "$PROGNAME: Cannot write file '$filename'" >&2
        exit 1
    fi
else
    write_html_page
fi
```

Код, обрабатывающий логику опции `-f`, приведен в конце предыдущего листинга. В нем мы проверяем наличие имени файла, а если оно найдено — проверяем, доступен ли файл для записи. Для этого выполняется `touch`, после чего проверяется, является ли полученный файл обычным файлом. Эти две проверки обрабатывают ситуации, когда введен неверный путь (`touch` завершится ошибкой), а также гарантируют, что если файл уже существует, то он является обычным файлом.

Как видно, для фактической генерации страницы вызывается функция `write_html_page`. Её вывод либо направляется в стандартный вывод (если переменная `filename` пуста), либо перенаправляется в указанный файл. Поскольку у HTML-кода есть два возможных места назначения, имеет смысл преобразовать процедуру `write_html_page` в функцию оболочки, чтобы избежать дублирования кода.

Итоги

Благодаря позиционным параметрам мы теперь можем писать вполне функциональные скрипты. Для простых повторяющихся задач позиционные параметры позволяют создавать полезные функции оболочки, которые можно поместить в файл `.bashrc` пользователя.

Наша программа `sys_info_page` стала сложнее и совершеннее. Ниже представлен полный список, последние изменения выделены жирным шрифтом:

```
#!/bin/bash
# sys_info_page: program to output a system information page
PROGNAME="$(basename "$0")"
TITLE="System Information Report For $HOSTNAME"
CURRENT_TIME="$(date +%x %r %Z)"
TIMESTAMP="Generated $CURRENT_TIME, by $USER"
report_uptime () {
    cat << _EOF_
    <h2>System Uptime</h2>
    <pre>$(uptime)</pre>
    _EOF_
    return
}
report_disk_space () {
    cat << _EOF_
    <h2>Disk Space Utilization</h2>
    <pre>$(df -h)</pre>
    _EOF_
    return
}
report_home_space () {
    if [[ "$(id -u)" -eq 0 ]]; then
        cat << _EOF_
        <h2>Home Space Utilization (All Users)</h2>
        <pre>$(du -sh /home/*)</pre>
        _EOF_
    else
        cat << _EOF_
        <h2>Home Space Utilization ($USER)</h2>
        <pre>$(du -sh "$HOME")</pre>
        _EOF_
    fi
    return
}
usage () {
    echo "$PROGNAME: usage: $PROGNAME [-f file | -i]"
    return
}
```

```

write_html_page () {
    cat << _EOF_
<html>
    <head>
        <title>$TITLE</title>
    </head>
    <body>
        <h1>$TITLE</h1>
        <p>$TIMESTAMP</p>
        $(report_uptime)
        $(report_disk_space)
        $(report_home_space)
    </body>
</html>
_EOF_
    return
}
# process command line options
interactive=
filename=
while [[ -n "$1" ]]; do
    case "$1" in
        -f | --file)          shift
                               filename="$1"
                               ;;
        -i | --interactive)   interactive=1
                               ;;
        -h | --help)          usage
                               exit
                               ;;
        *)                    usage >&2
                               exit 1
                               ;;
    esac
    shift
done
# interactive mode
if [[ -n "$interactive" ]]; then

```

```

while true; do
    read -r -p "Enter name of output file: " filename
    if [[ -e "$filename" ]]; then
        read -r -p "'$filename' exists. Overwrite? [y/n/q] > "
        case "$REPLY" in
            Y|y) break
                ;;
            Q|q) echo "Program terminated."
                exit
                ;;
            *) continue
                ;;
        esac
    elif [[ -z "$filename" ]]; then
        continue
    else
        break
    fi
done
fi
# output html page
if [[ -n "$filename" ]]; then
    if touch "$filename" && [[ -f "$filename" ]]; then
        write_html_page > "$filename"
    else
        echo "$PROGNAME: Cannot write file '$filename'" >&2
        exit 1
    fi
fi

```

```
    fi
else
    write_html_page
fi
```

Мы еще не закончили. Есть еще несколько вещей, которые можно сделать и улучшить.

Что почитать

- В Bash Reference Manual есть статья о специальных параметрах, включая \$* и \$@:
<https://www.gnu.org/software/bash/manual/bashref.html#Special-Parameters> ●

33 — Управление потоком: циклы *с for*

В этой заключительной главе о управлении потоком мы рассмотрим еще одну конструкцию цикла в оболочке. Цикл `for` отличается от циклов `while` и `until` тем, что позволяет обрабатывать последовательности во время выполнения. Это оказывается полезным при программировании. Соответственно, цикл `for` — популярная конструкция в `bash`.

Цикл `for` реализуется, что вполне логично, с помощью составной команды `for`. В `bash`, `for` доступна в двух формах.

for: традиционная форма оболочки

Синтаксис исходной команды `for` выглядит так:

```
for variable [in words]; do commands done
```

Здесь `variable` — имя переменной, которая будет инкрементироваться во время выполнения цикла, `words` — необязательный список элементов, последовательно присваиваемых `variable`, `commands`, а — команды, выполняемые на каждой итерации цикла.

Команда `for` полезна в командной строке. Мы можем легко продемонстрировать её работу.

```
[me@linuxbox ~]$ for i in A B C D; do echo $i; done
A
B
C
D
```

В этом примере `for` получает список из четырех слов: `A`, `B`, `C` и `D`. Поскольку в списке четыре слова, цикл выполнится четыре раза. При каждом выполнении цикла переменной `i` присваивается новое слово. Внутри цикла находится команда `echo`, которая выводит значение `i`, чтобы продемонстрировать присваивание. Как и в циклах `while` и `until`, ключевое слово `done` завершает цикл.

По-настоящему мощная особенность `for` заключается в множестве способов создания списка слов. Например, это можно сделать с помощью раскрытия фигурных скобок:

```
[me@linuxbox ~]$ for i in {A..D}; do echo $i; done
A
B
```

C
D

или использовать раскрытие пути следующим образом:

```
[me@linuxbox ~]$ for i in distros*.txt; do echo "$i"; done
distros-by-date.txt
distros-dates.txt
distros-key-names.txt
distros-key-vernums.txt
distros-names.txt
distros.txt
distros-vernums.txt
distros-versions.txt
```

Раскрытие пути дает чистый и удобный список путей для обработки в цикле. Единственная мера предосторожности — проверить, что раскрытие действительно нашло совпадения. По умолчанию, если раскрытие не находит файлов, возвращаются сами подстановочные знаки (например, «distros*.txt» в примере выше). Чтобы избежать этого, в скрипте пример выше следует оформить так:

```
for i in distros*.txt; do
    if [[ -e "$i" ]]; then
        echo "$i"
    fi
done
```

Добавив проверку существования файла, мы будем игнорировать неудачное раскрытие.

Еще один распространенный способ формирования слов — подстановка команды.

```
#!/bin/bash
# longest-word: find longest string in a file
while [[ -n "$1" ]]; do
    if [[ -r "$1" ]]; then
        max_word=
        max_len=0
        for i in $(strings "$1"); do
            len=$(echo -n "$i" | wc -c)
            if (( len > max_len )); then
                max_len="$len"
                max_word="$i"
            fi
        done
        echo "$1: '$max_word' ($max_len characters)"
    fi
    shift
done
```

В этом примере мы ищем самую длинную строку в файле. При получении одного или нескольких имен файлов в командной строке программа использует `strings` program (входящую в пакет GNU binutils) для создания списка читаемых текстовых

«слов» в каждом файле. Цикл `for` поочередно обрабатывает каждое слово и определяет, является ли текущее слово самым длинным из найденных. По завершении цикла выводится самое длинное слово.

Стоит отметить, что, вопреки нашей обычной практике, мы не заключаем подстановку команды `$(strings "$1")` в двойные кавычки. Это нужно для того, чтобы произошло разделение по словам и мы получили список. Если бы мы заключили подстановку команды в кавычки, она вернула бы лишь одно слово, содержащее все строки файла. Это не совсем то, что нам нужно.

Если необязательная часть `in words` команды `for` опущена, `for` по умолчанию обрабатывает позиционные параметры. Мы изменим наш скрипт `longest-word`, чтобы использовать этот метод:

```
#!/bin/bash
# longest-word2: find longest string in a file
for i; do
    if [[ -r "$i" ]]; then
        max_word=
        max_len=0
        for j in $(strings "$i"); do
            len=$(echo -n "$j" | wc -c)
            if (( len > max_len )); then
                max_len="$len"
                max_word="$j"
            fi
        done
        echo "$i: '$max_word' ($max_len characters)"
    fi
done
```

Как видно, мы изменили внешний цикл, заменив `while` на `for`. Поскольку в команде `for` список слов опущен, вместо него используются позиционные параметры. Внутри цикла предыдущие вхождения переменной `i` заменены на переменную `j`. Использование `shift` также было исключено.

Почему `i`?

Вы могли заметить, что в каждом из предыдущих `for` примеров с циклами выбиралась переменная `i`. Почему? На самом деле, никакой особой причины, кроме традиции, нет. В `for` можно использовать любую допустимую переменную, но чаще всего выбирают `i`, а затем `j` и `k`.

Основа этой традиции восходит к языку программирования Fortran. В Fortran необъявленные переменные, начинающиеся с букв `I`, `J`, `K`, `L` и `M`, автоматически считаются

целыми числами, тогда как переменные на любую другую букву — вещественными (числами с десятичной дробью). Это привело к тому, что программисты стали использовать I, J и K для переменных цикла, так как это было удобнее, когда требовалась временная переменная (каковой переменные цикла часто и являются).

Это также породило следующую шутку, основанную на Fortran:

«БОГ реален, если не объявлен как integer».

for: формат языка C

В последних версиях bash добавлен второй вариант синтаксиса команды for, напоминающий синтаксис языка программирования C. Многие другие языки также поддерживают эту форму.

```
for (( expression1; expression2; expression3 )); do
commands done
```

Здесь expression1, expression2 и expression3 — арифметические выражения, а commands — команды, выполняемые на каждой итерации цикла.

С точки зрения поведения эта форма эквивалентна следующей конструкции:

```
(( expression1 )) while (( expression2 )); do commands ((
expression3 )) done
```

expression1 используется для инициализации условий цикла, expression2 — для определения момента завершения цикла, а expression3 выполняется в конце каждой итерации.

Вот типичный пример использования:

```
#!/bin/bash
# simple_counter: demo of C style for command
for (( i=0; i<5; i=i+1 )); do
    echo $i
done
```

При выполнении программа выводит следующее:

```
[me@linuxbox ~]$ simple_counter
0
1
2
3
4
```

В этом примере `expression1` инициализирует переменную `i` значением ноль, `expression2` `i` позволяет циклу продолжаться, пока значение остается меньше 5, а `expression3` увеличивает значение `i` на 1 при каждом повторении цикла.

Форма `for` на языке C полезна везде, где требуется числовая последовательность. Мы рассмотрим несколько вариантов её применения в следующих двух главах.

Итоги

Используя знания о команде `for`, мы внесем финальные правки в наш `sys_info_page` script. Сейчас функция `report_home_space` выглядит так:

```
report_home_space () {
    if [[ "$(id -u)" -eq 0 ]]; then
        cat << _EOF_
        <h2>Home Space Utilization (All Users)</h2>
        <pre>$(du -sh /home/*)</pre>
    _EOF_
    else
        cat << _EOF_
        <h2>Home Space Utilization ($USER)</h2>
        <pre>$(du -sh "$HOME")</pre>
    _EOF_
    fi
    return
}
```

Затем мы перепишем команду, чтобы она выводила больше подробностей для каждого домашнего каталога пользователя, включая общее количество файлов и подкаталогов в каждом.

```
report_home_space () {
    local format="%8s%10s%10s\n"
    local i dir_list total_files total_dirs total_size user_name
    if [[ "$(id -u)" -eq 0 ]]; then
        dir_list=/home/*
        user_name="All Users"
    else
        dir_list="$HOME"
        user_name="$USER"
    fi
    echo "<h2>Home Space Utilization ($user_name)</h2>"
    for i in $dir_list; do
        total_files="$(find "$i" -type f | wc -l)"
        total_dirs="$(find "$i" -type d | wc -l)"
        total_size="$(du -sh "$i" | cut -f 1)"
        echo "<H3>$i</H3>"
        echo "<pre>"
        printf "$format" "Dirs" "Files" "Size"
        printf "$format" "-----" "-----" "-----"
        printf "$format" "$total_dirs" "$total_files" "$total_size"
        echo "</pre>"
    done
    return
}
```

Этот переписанный вариант применяет многое из изученного. Мы по-прежнему проверяем права суперпользователя, но вместо выполнения полного набора действий в рамках `if`, мы задаем переменные для последующего цикла `for`. Мы добавили в функцию несколько локальных переменных и использовали `printf` для форматирования вывода.

Что почитать

- В руководстве The Advanced Bash-Scripting Guide есть глава о циклах с примерами использования `for`:
<https://tldp.org/LDP/abs/html/loops1.html>
- Справочное руководство Bash описывает составные команды цикла, включая `for`:
<https://www.gnu.org/software/bash/manual/bashref.html#Looping-Constructs>

34 — Строки и числа

Компьютерные программы работают с данными. В прошлых главах мы фокусировались на обработке данных на уровне файлов. Однако многие задачи программирования требуют использования более мелких единиц данных, таких как строки и числа.

В этой главе мы рассмотрим возможности оболочки для манипуляции строками и числами. Оболочка предоставляет множество вариантов раскрытия параметров для операций со строками. Помимо арифметического раскрытия (которое мы затронули в главе 7), существует известная программа командной строки `bc`, выполняющая более сложные математические вычисления.

Раскрытие параметров

Хотя раскрытие параметров упоминалось в главе 7, мы не разбирали его подробно, так как большинство таких раскрытий используется в скриптах, а не в командной строке. Мы уже работали с некоторыми формами раскрытия, например, с переменными оболочки. Оболочка предоставляет гораздо больше возможностей.

Примечание: Хорошей практикой считается заключение раскрытия параметров в двойные кавычки, чтобы предотвратить нежелательное разделение слов, если для этого нет особых причин. Это особенно важно при работе с именами файлов, так как они часто содержат пробелы и прочие сложности.

Основные параметры

Простейшая форма раскрытия параметров проявляется при обычном использовании переменных. Вот пример:

`$a`

После раскрытия здесь будет подставлено содержимое переменной `a`. Простые параметры также можно заключать в фигурные скобки.

`${a}`

На раскрытие это не влияет, но необходимо, если переменная примыкает к другому тексту, что может запутать оболочку. В

этом примере мы пытаемся создать имя файла с помощью `ар` - добавляя строку `_file` к содержимому переменной `a`.

```
[me@linuxbox ~]$ a="foo"
[me@linuxbox ~]$ echo "$a_file"
```

Если выполнить эту последовательность команд, результатом будет пустота, так как оболочка попытается раскрыть переменную с именем `a_file` вместо `a`. Эту проблему можно решить, заключив «настоящее» имя переменной в фигурные скобки.

```
[me@linuxbox ~]$ echo "${a}_file"
foo_file
```

Мы также видели, что к позиционным параметрам больше девяти можно обратиться, заключив номер в фигурные скобки. Например, чтобы получить доступ к 11-му позиционному параметру, можно сделать так:

```
${11}
```

Раскрытие для работы с пустыми переменными

Некоторые виды раскрытия параметров предназначены для работы с несуществующими и пустыми переменными. Они удобны для обработки отсутствующих позиционных параметров и назначения значений по умолчанию. Первый способ позволяет задать значение по умолчанию для неустановленного параметра.

```
${parameter:-word}
```

Если параметр не установлен (т. е. не существует) или пуст, это раскрытие возвращает значение `word`. Если параметр не пуст, раскрытие возвращает значение `parameter`.

```
[me@linuxbox ~]$ foo=
[me@linuxbox ~]$ echo ${foo:-"substitute value if unset"}
substitute value if unset
[me@linuxbox ~]$ echo $foo
[me@linuxbox ~]$ foo=bar
[me@linuxbox ~]$ echo ${foo:-"substitute value if unset"}
bar
[me@linuxbox ~]$ echo $foo
bar
```

Это раскрытие подставляет значение по умолчанию, если параметр не задан или пуст.

```
${parameter:=word}
```

Если параметр не задан или пуст, это раскрытие возвращает значение `word`. Кроме того, значение `word` присваивается

параметру. Если параметр не пуст, раскрытие возвращает значение параметра.

```
[me@linuxbox ~]$ foo=
[me@linuxbox ~]$ echo ${foo:="default value if unset"}
default value if unset
[me@linuxbox ~]$ echo $foo
default value if unset
[me@linuxbox ~]$ foo=bar
[me@linuxbox ~]$ echo ${foo:="default value if unset"}
bar
[me@linuxbox ~]$ echo $foo
bar
```

Примечание: позиционные и другие специальные параметры нельзя присваивать таким образом.

Это раскрытие завершает работу скрипта, если обязательный параметр не задан или пуст.

`${parameter:?word}`

Если параметр не задан или пуст, это раскрытие приводит к выходу скрипта с ошибкой, а содержимое word отправляется в стандартный поток ошибок. Если параметр не пуст, раскрытие возвращает значение параметра.

```
[me@linuxbox ~]$ foo=
[me@linuxbox ~]$ echo ${foo:? "parameter is empty"}
bash: foo: parameter is empty
[me@linuxbox ~]$ echo $?
1
[me@linuxbox ~]$ foo=bar
[me@linuxbox ~]$ echo ${foo:? "parameter is empty"}
bar
[me@linuxbox ~]$ echo $?
0
```

Это раскрытие подставляет значение вместо непустого параметра.

`${parameter:+word}`

Если параметр не задан или пуст, раскрытие ничего не возвращает. Если параметр не пуст, вместо него подставляется значение word; при этом само значение параметра не меняется.

```
[me@linuxbox ~]$ foo=
[me@linuxbox ~]$ echo ${foo:+ "substitute value if set"}
[me@linuxbox ~]$ foo=bar
[me@linuxbox ~]$ echo ${foo:+ "substitute value if set"}
substitute value if set
```

Раскрытие, возвращающее имена переменных

Оболочка умеет возвращать имена переменных. Это используется в довольно экзотических ситуациях.

```
${!prefix*} ${!prefix@}
```

Это раскрытие возвращает имена существующих переменных, начинающихся с префикса. Согласно bash документации, обе формы этого раскрытия работают идентично, если они не заключены в кавычки. Здесь мы перечислим все переменные окружения, имена которых начинаются с BASH:

```
[me@linuxbox ~]$ echo ${!BASH*}
BASH BASH_ARGC BASH_ARGV BASH_COMMAND BASH_COMPLETION
BASH_COMPLETION_DIR BASH_LINENO BASH_SOURCE BASH_SUBSHELL
BASH_VERSINFO BASH_VERSION
```

Операции со строками

Существует множество видов раскрытия, работающих со строками. Многие из них особенно удобны для операций с путями. Следующее раскрытие возвращает длину строки, содержащейся в параметре:

```
${#parameter}
```

Обычно параметр — это строка; однако, если параметр равен @ или *, то результатом раскрытия будет количество позиционных параметров.

```
[me@linuxbox ~]$ foo="This string is long."
[me@linuxbox ~]$ echo "'$foo' is ${#foo} characters long."
'This string is long.' is 20 characters long.
```

```
${parameter:offset} ${parameter:offset:length}
```

Эти расширения используются для извлечения части строки, содержащейся в параметре. Извлечение начинается с указанного смещения от начала строки и продолжается до конца строки, если не задана длина.

```
[me@linuxbox ~]$ foo="This string is long."
[me@linuxbox ~]$ echo ${foo:5}
string is long.
[me@linuxbox ~]$ echo ${foo:5:6}
string
```

Если значение offset отрицательное, это означает отсчет от конца строки, а не от начала. Примечание: перед отрицательными значениями должен стоять пробел, чтобы избежать путаницы с расширением \${parameter:-word}. Длина length, если она указана, не должна быть меньше нуля.

Если параметр — @, результатом расширения будут positional параметры длиной length, начиная со смещения offset.

```
[me@linuxbox ~]$ foo="This string is long."
[me@linuxbox ~]$ echo ${foo: -5}
long.
[me@linuxbox ~]$ echo ${foo: -5:2}
lo
```

`${parameter#pattern}` `${parameter##pattern}`

Эти расширения удаляют начальную часть строки в параметре, соответствующую шаблону `pattern`. `pattern` — это подстановочный знак, подобный тем, что используются при расширении пути. Различие между двумя формами в том, что форма `#` удаляет кратчайшее совпадение, а форма `##` — самое длинное.

```
[me@linuxbox ~]$ foo=file.txt.zip
[me@linuxbox ~]$ echo ${foo#*.}
txt.zip
[me@linuxbox ~]$ echo ${foo##*.}
zip
```

`${parameter%pattern}` `${parameter%%pattern}`

Эти варианты раскрытия аналогичны предыдущим `#` и `##`, но вместо удаления текста из начала строки они удаляют его из конца значения параметра.

```
[me@linuxbox ~]$ foo=file.txt.zip
[me@linuxbox ~]$ echo ${foo%. *}
file.txt
[me@linuxbox ~]$ echo ${foo%%. *}
file
```

`${parameter/pattern/string}` `${parameter//pattern/string}`

`${parameter/#pattern/string}`

`${parameter/%pattern/string}`

Эти операции выполняют поиск и замену в содержимом параметра. Если текст соответствует подстановочному знаку, он заменяется строкой. В обычном виде заменяется только первое вхождение. В форме `//` заменяются все вхождения. Форма `/#` требует, чтобы совпадение было в начале строки, а `/%` — в конце. В любом случае `/string` можно опустить, тогда текст, соответствующий подстановочному знаку, будет просто удален.

```
[me@linuxbox ~]$ foo=JPG.JPG
[me@linuxbox ~]$ echo ${foo/JPG/jpg}
jpg.JPG
[me@linuxbox ~]$ echo ${foo//JPG/jpg}
jpg.jpg
[me@linuxbox ~]$ echo ${foo/#JPG/jpg}
jpg.JPG
[me@linuxbox ~]$ echo ${foo/%JPG/jpg}
JPG.jpg
```

Полезно знать о раскрытии параметров. Операции со строками могут заменить такие распространенные команды, как `sed` и `cut`. Раскрытие может повысить эффективность скриптов, исключив использование внешних программ. Например, мы изменим программу `longest-word` из предыдущей главы, заменив подстановку команды и создаваемый ею `subshell` на раскрытие параметра `${#j}` `$(echo -n $j | wc -c)`, вот так:

```
#!/bin/bash
# longest-word3: find longest string in a file
for i; do
    if [[ -r "$i" ]]; then
        max_word=
        max_len=0
        for j in $(strings $i); do
            len="${#j}"
            if (( len > max_len )); then
                max_len="$len"
                max_word="$j"
            fi
        done
        echo "$i: '$max_word' ($max_len characters)"
    fi
done
```

Затем мы сравним эффективность двух версий с помощью команды `time`.

```
[me@linuxbox ~]$ time longest-word2 dirlist-usr-bin.txt
dirlist-usr-bin.txt: 'scrollkeeper-get-extended-content-list' (38
characters)
real    0m3.618s
user    0m1.544s
sys     0m1.768s
[me@linuxbox ~]$ time longest-word3 dirlist-usr-bin.txt
dirlist-usr-bin.txt: 'scrollkeeper-get-extended-content-list' (38
characters)
real    0m0.060s
user    0m0.056s
sys     0m0.008s
```

Оригинальная версия скрипта сканирует текстовый файл 3,618 секунды, тогда как новая версия с использованием раскрытия параметров работает всего 0,06 секунды — это значительное улучшение.

Изменение регистра

`bash` имеет четыре способа подстановки параметров и две `declare` опции команды для преобразования строк в верхний или нижний регистр.

Зачем нужно изменение регистра? Помимо очевидной эстетики, это играет важную роль в программировании. Рассмотрим поиск в базе данных. Представьте, что пользователь ввел строку в поле ввода, которую мы хотим найти в базе. Пользователь может

ввести значение целиком заглавными, строчными буквами или в смешанном виде. Мы явно не хотим засорять базу данных всеми возможными комбинациями регистра. Что делать?

Обычный подход к этой проблеме — нормализация пользовательского ввода. То есть преобразование его в стандартизированный вид перед поиском в базе. Мы можем сделать это, переводя все символы ввода либо в нижний, либо в верхний регистр, и следить, чтобы записи в базе данных были нормализованы так же.

Команду `declare` можно использовать для приведения строк к верхнему или нижнему регистру. С помощью `declare` мы можем заставить переменную всегда содержать нужный формат, независимо от того, что ей присваивается.

```
#!/bin/bash
# ul-declare: demonstrate case conversion via declare
declare -u upper
declare -l lower
if [[ $1 ]]; then
    upper="$1"
    lower="$1"
    echo "$upper"
    echo "$lower"
fi
```

В предыдущем скрипте мы использовали `declare` для создания двух переменных, `upper` и `lower`. Мы присвоили каждой переменной значение первого аргумента командной строки (позиционный параметр 1), а затем вывели их на экран.

```
[me@linuxbox ~]$ ul-declare aBc
ABC
abc
```

Как видно, аргумент командной строки (`aBc`) был нормализован.

В дополнение к `declare` существуют четыре раскрытия параметров для преобразования в верхний/нижний регистр, описанные в таблице 34-1.

Таблица 34-1: Раскрытия параметров для преобразования регистра `er`, —

Форматирование	Результат
----------------	-----------

Форматирование	Результат
<code>\${parameter,,pattern}</code>	Преобразовать значение параметра в нижний регистр. <code>pattern</code> — это необязательный шаблон оболочки (наприм <code>[A-F]</code>), который ограничивает набор символов, подлежащих преобразованию. Полное описание шаблонов см. в странице <code>man parameter</code> .
<code>\${parameter,pattern}</code>	Преобразовать значение параметра, изменяя только первый символ в нижний регистр.
<code>\${parameter^^pattern}</code>	Развернуть значение параметра в верхний регистр буквы.
<code>\${parameter^pattern}</code>	Развернуть значение параметра, изменив только первый символ на заглавный (регистр).

Вот скрипт, демонстрирующий эти преобразования:

```
#!/bin/bash
# ul-param: demonstrate case conversion via parameter expansion
if [[ "$1" ]]; then
    echo "${1,,}"
    echo "${1,}"
    echo "${1^^}"
    echo "${1^}"
fi
```

Вот как работает этот скрипт:

```
[me@linuxbox ~]$ ul-param aBc
abc
aBc
ABC
ABc
```

Снова обрабатываем первый аргумент командной строки и выведем четыре варианта, поддерживаемых подстановкой параметров. Хотя в этом скрипте используется первый позиционный параметр, параметром может быть любая строка, переменная или строковое выражение.

Арифметическое вычисление и подстановка

Мы рассматривали арифметическую подстановку в главе 7. Она используется для выполнения различных арифметических операций с целыми числами. Её базовая форма выглядит

следующим образом, где `expression` — это корректное арифметическое выражение:

```
$( (expression) )
```

Это связано с составной командой `( ( ) )`, которую мы встречали в главе 27 для выполнения арифметических вычислений (проверки истинности).

В предыдущих главах мы рассмотрели некоторые распространенные типы выражений и операторов. Здесь мы изучим более полный список.

Системы счисления

В главе 9 мы рассмотрели восьмеричные (основание 8) и шестнадцатеричные (основание 16) числа. В арифметических выражениях оболочка поддерживает целочисленные константы в любой системе счисления. В таблице 34-2 перечислены обозначения, используемые для указания систем счисления.

Таблица 34-2: Указание различных систем счисления

Нотация	Описание
<code>number</code>	По умолчанию числа без обозначений считаются десятичными целые числа (в десятичной системе).
<code>0number</code>	В арифметических выражениях числа с ведущим нулем считаются восьмеричными.
<code>0xnumber</code>	Шестнадцатеричная запись.
<code>base#number</code>	<i>число в системе с основанием</i>

Вот несколько примеров:

```
[me@linuxbox ~]$ echo $((0xff))
255
[me@linuxbox ~]$ echo $((2#11111111))
255
```

В предыдущих примерах мы выводили значение шестнадцатеричного числа `ff` (максимальное двузначное число) и максимальное восьмибитное двоичное (`base 2`) число.

Унарные операторы

Существуют два унарных оператора, `+` и `-`, которые используются для указания того, является ли число положительным или отрицательным. Пример — `-5`.

Простая арифметика

Обычные арифметические операторы перечислены в таблице 34-3.

Таблица 34-3: Арифметические операторы

Оператор	Описание
+	Сложение
-	Вычитание
*	Умножение
/	Целочисленное деление
**	Возведение в степень
%	Остаток от деления

Большинство этих операций понятны сами по себе, но целочисленное деление и остаток от деления требуют дополнительного обсуждения.

Поскольку арифметические операции оболочки работают только с целыми числами, результатом деления всегда будет целое число.

```
[me@linuxbox ~]$ echo $(( 5 / 2 ))  
2
```

This делает определение остатка от деления более важным.

```
[me@linuxbox ~]$ echo $(( 5 % 2 ))  
1
```

Используя операторы деления и взятия остатка, мы можем определить, что при делении 5 на 2 получается 2 с остатком 1.

Вычисление остатка полезно в циклах. Оно позволяет выполнять операцию через определенные интервалы во время выполнения цикла. В следующем примере мы выводим ряд чисел, выделяя каждое число, кратное 5:

```
#!/bin/bash  
# modulo: demonstrate the modulo operator  
for ((i = 0; i <= 20; i = i + 1)); do  
    remainder=$((i % 5))  
    if (( remainder == 0 )); then  
        printf "<td> " "$i"  
    else  
        printf "%d " "$i"  
    fi  
done  
printf "\n"
```


При выполнении результат будет выглядеть так:

```
[me@linuxbox ~]$ modulo
<0> 1 2 3 4 <5> 6 7 8 9 <10> 11 12 13 14 <15> 16 17 18 19 <20>
```

Задание

Хотя их назначение может быть не сразу очевидным, арифметические выражения могут выполнять присваивание. Мы уже многократно выполняли присваивание, но в другом контексте. Каждый раз, когда мы присваиваем переменной значение, мы выполняем присваивание. Это можно делать и внутри арифметических выражений.

```
[me@linuxbox ~]$ foo=
[me@linuxbox ~]$ echo $foo
[me@linuxbox ~]$ if (( foo = 5 )); then echo "It is true."; fi
It is true.
[me@linuxbox ~]$ echo $foo
5
```

В предыдущем примере мы сначала присвоили переменной `foo` пустое значение и убедились, что оно действительно пустое. Затем мы выполнили `if (( foo = 5 ))` с составной командой. Этот процесс делает две интересные вещи: он присваивает значение переменной `foo`, и это выражение возвращает `true`, так как переменной `foo` было присвоено ненулевое значение.

Примечание: важно помнить точное значение `=` в предыдущем выражении. Одиночный `=` выполняет присваивание. `foo = 5` означает «сделать `foo` равным 5», тогда как `==` проверяет эквивалентность. `foo == 5` спрашивает: «равно ли `foo` пяти?». Это распространенная особенность многих языков программирования. В оболочке это может немного сбивать с толку, так как команда `test` принимает одиночный `=` для сравнения строк. Это еще одна причина использовать более современные составные команды `[]` и `()` вместо `test`.

Помимо нотации `=`, оболочка также предоставляет нотации для выполнения полезных присваиваний, как показано в таблице 34-4.

Таблица 34-4: Операторы присваивания `y =`

Обозначения	Описание
<code>parameter = value</code>	Простое присваивание. Присваивает значение параметру.

Обозначения	Описание
<code>parameter += value</code>	Сложение. Эквивалентно <code>= параметру + значение</code> .
<code>parameter -= value</code>	Вычитание. Эквивалентно <code>параметру = параметру - value</code> .
<code>parameter *= value</code>	Умножение. Эквивалентно <code>параметру = параметру * value</code> .
<code>parameter /= value</code>	Целочисленное деление. Эквивалентно <code>параметру = параметру / value</code> .
<code>parameter %= value</code>	Остаток от деления. Эквивалентно <code>параметру = параметр% значение</code> .
<code>parameter++</code>	Постфиксный инкремент переменной. Эквивалентно <code>параметру = параметру + 1</code> (см. далее обсуждение).
<code>parameter--</code>	Постдекремент переменной. Эквивалентно <code>параметру = параметру - 1</code> .
<code>++parameter</code>	Преинкремент переменной. Эквивалентно <code>параметру = параметру + 1</code> .
<code>--parameter</code>	Префиксный декремент переменной. Эквивалентно <code>параметру = параметру - 1</code> .

Эти операторы присваивания служат удобным сокращением для многих распространенных арифметических задач. Особый интерес представляют операторы инкремента (`++`) и декремента (`--`), которые увеличивают или уменьшают значение параметра на единицу. Такой стиль нотации заимствован из языка программирования C и включен во многие другие языки, включая `bash`.

Операторы могут стоять либо перед параметром, либо после него. Хотя оба они увеличивают или уменьшают параметр на единицу, между этими вариантами есть тонкое различие. Если оператор стоит перед параметром, параметр увеличивается (или уменьшается) до того, как будет возвращено его значение. Если после — операция выполняется после возврата значения. Это звучит странно, но такова задумка разработчиков. Вот демонстрация:

```
[me@linuxbox ~]$ foo=1
[me@linuxbox ~]$ echo $((foo++))
1
[me@linuxbox ~]$ echo $foo
2
```

Если мы присвоим переменной `foo` значение 1, а затем увеличим её с помощью оператора `++`, стоящего после имени параметра, `foo` вернет значение 1. Однако, если мы проверим значение переменной во второй раз, мы увидим уже увеличенное значение. Если же поставить оператор `++` перед параметром, поведение будет более ожидаемым.

```
[me@linuxbox ~]$ foo=1
[me@linuxbox ~]$ echo $(++foo)
2
[me@linuxbox ~]$ echo $foo
2
```

Для большинства приложений оболочки полезнее всего будет добавить префикс к оператору.

Операторы `++` и `--` часто используются вместе с циклами. Мы немного улучшим наш скрипт для вычисления остатка от деления, чтобы сделать его более компактным.

```
#!/bin/bash
# modulo2: demonstrate the modulo operator
for ((i = 0; i <= 20; ++i )); do
    if ((i % 5 == 0)); then
        printf "<td> " "$i"
    else
        printf "%d " "$i"
    fi
done
printf "\n"
```

Битовые операции

Один класс операторов манипулирует числами необычным способом. Эти операторы работают на битовом уровне. Они используются для определенных низкоуровневых задач, часто связанных с установкой или чтением битовых флагов. Битовые операторы перечислены в таблице 34-5.

Таблица 34-5: Битовые операторы всеми

Оператор	Описание
<code>~</code>	Побитовое отрицание. Инвертирует все биты в числе.
<code><<</code>	Побитовый сдвиг влево. Сдвигает все биты в числе влево.

Оператор	Описание
>>	Побитовый сдвиг вправо. Сдвигает все биты в числе вправо.
&	Побитовое И. Выполняет операцию И над всеми битами двух чисел.
	Побитовое ИЛИ. Выполняет операцию ИЛИ над всеми битами числа.
^	Побитовое исключающее ИЛИ. Выполняет операцию исключающего ИЛИ над битами двух чисел.

Примечание: для всех операций, кроме побитового отрицания, существуют соответствующие операторы присваивания (например, <=>).

Здесь мы продемонстрируем создание списка степеней двойки с помощью оператора побитового сдвига влево:

```
[me@linuxbox ~]$ for ((i=0;i<8;++i)); do echo $((1<<i)); done
1
2
4
8
16
32
64
128
```

Логика

Как мы выяснили в главе 27, составная команда (()) поддерживает различные операторы сравнения. Есть еще несколько операторов для вычисления логических выражений. Полный список приведен в таблице 34-6.

Таблица 34-6: Операторы сравнения

Оператор	Описание
<=	Меньше или равно.
>=	Больше или равно.
<	Меньше чем.
>	Больше чем.
==	Равно.
!=	Не равно.

Оператор	Описание
&&	Логическое И.
	Логическое ИЛИ.
expr1?expr2:expr3	Оператор сравнения (тернарный). Если выражение expr1 не равно нулю (арифметически истинно), то expr2; иначе expr3.

При использовании в логических операциях выражения следуют правилам арифметической логики: выражения, равные нулю, считаются ложными, а ненулевые — истинными. Составная команда (()) преобразует результаты в обычные коды завершения оболочки.

```
[me@linuxbox ~]$ if ((1)); then echo "true"; else echo "false"; fi
true
[me@linuxbox ~]$ if ((0)); then echo "true"; else echo "false"; fi
false
```

Самым странным из логических операторов является тернарный оператор. Этот оператор (моделирующий аналог из языка программирования C) выполняет самостоятельную логическую проверку. Его можно использовать как своего рода if/then/else инструкцию. Он работает с тремя арифметическими выражениями (строки не подойдут): если первое выражение истинно (или не равно нулю), выполняется второе выражение. В противном случае выполняется третье. Мы можем опробовать это в командной строке:

```
[me@linuxbox ~]$ a=0
[me@linuxbox ~]$ ((a<1?++a:--a))
[me@linuxbox ~]$ echo $a
1
[me@linuxbox ~]$ ((a<1?++a:--a))
[me@linuxbox ~]$ echo $a
0
```

Здесь мы видим тернарный оператор в действии. Этот пример реализует переключатель. Каждый раз при выполнении оператора значение переменной a меняется с нуля на единицу или наоборот.

Примечание: выполнение присваивания внутри выражений не является простым. При попытке сделать это, bash выдаст ошибку.

```
[me@linuxbox ~]$ a=0
[me@linuxbox ~]$ ((a<1?a+=1:a-=1))
bash: ((: a<1?a+=1:a-=1: attempted assignment to non-variable (error token is "-=1")
```

Эту проблему можно решить, заключив выражение присваивания в скобки.

```
[me@linuxbox ~]$ ((a<1?(a+=1):(a-=1)))
```

Далее приводится более полный пример использования арифметических операторов в скрипте, который выводит простую таблицу чисел.

```
#!/bin/bash
# arith-loop: script to demonstrate arithmetic operators
finished=0
a=0
printf "a\t a**2\t a**3\n"
printf "=\t=====\n"
until ((finished)); do
    b=$((a**2))
    c=$((a**3))
    printf "%d\t%d\t%d\n" "$a" "$b" "$c"
    ((a<10?++a:(finished=1)))
done
```

В этом скрипте мы реализуем цикл `until`, основанный на значении переменной `finished`. Изначально переменная равна нулю (арифметическое «ложь»), и цикл продолжается до тех пор, пока она не станет отличной от нуля. Внутри цикла мы вычисляем квадрат и куб переменной-счетчика `a`. В конце цикла проверяется значение счетчика. Если оно меньше 10 (максимальное число итераций), счетчик увеличивается на единицу, иначе переменной `finished` присваивается значение один, что делает `finished` арифметически «истиной» и завершает цикл. Запуск скрипта дает следующий результат:

```
[me@linuxbox ~]$ arith-loop
a    a**2 a**3
=    =====
0    0     0
1    1     1
2    4     8
3    9    27
4   16    64
5   25   125
6   36   216
7   49   343
8   64   512
9   81   729
10  100  1000
```

Оператор «запятая»

Последний оператор, который мы рассмотрим, — это оператор «запятая». Он позволяет помещать несколько выражений в рамках одного вычисления, например: `(( expr, expr, expr ))`. Это полезно, когда нужно объединить несколько операций в одно

вычисление. Для демонстрации мы изменим скрипт `arith-loop`, чтобы упростить его с помощью оператора «запятая».

```
#!/bin/bash
# arith-loop2: script to demonstrate comma arithmetic operator
a=0
printf "a\ta**2\ta**3\n"
printf "=\t====\t====\n"
until (( a++, b=a**2, c=a**3, a>10 )); do
    printf "%d\t%d\t%d\n" $a $b $c
done
```

Здесь мы смогли объединить все вычисления в одну строку.

Примечание: статус выхода вычисления определяется последним выражением в цепочке.

bc — Язык калькулятора произвольной точности

Мы видели, как оболочка справляется с целочисленной арифметикой, но что если нам нужна высшая математика или числа с плавающей точкой? Ответ: напрямую через оболочку это сделать нельзя. Для этого потребуется внешняя программа. Есть несколько подходов. Внедрение программ Perl или AWK — один из вариантов, но это выходит за рамки данной книги.

Другой подход — использование специализированной программы-калькулятора. Одна из таких программ, встречающаяся во многих системах Linux, называется `bc`.

Программа `bc` считывает файл, написанный на её собственном C-подобном языке, и выполняет его. `bc` может быть отдельным файлом или считываться из стандартного ввода. `bc` поддерживает множество возможностей, включая переменные, циклы и функции, определяемые программистом. Мы не будем разбирать `bc` полностью, а лишь дадим краткое представление. `bc` хорошо описана в своей `man`-странице.

Начнем с простого примера. Мы напишем `bc script`, чтобы сложить 2 и 2.

```
/* A very simple bc script */
2 + 2
```

Первая строка скрипта — это комментарий. `bc` использует тот же синтаксис комментариев, что и язык программирования C. Комментарии могут занимать несколько строк; они начинаются с `/*` и заканчиваются `*/`.

Использование `bc`

Если сохранить предыдущий bc скрипт как `foo.bc`, его можно запустить так:

```
[me@linuxbox ~]$ bc foo.bc
bc 1.06.94
Copyright 1991-1994, 1997, 1998, 2000, 2004, 2006 Free Software
Foundation, Inc.
This is free software with ABSOLUTELY NO WARRANTY.
For details type `warranty'.
4
```

Если присмотреться, внизу, после сообщения об авторских правах, виден результат. Это сообщение можно скрыть с помощью опции `-q` (`quiet`).

bc также можно использовать интерактивно.

```
[me@linuxbox ~]$ bc -q
2 + 2
4
quit
```

При интерактивном использовании bc мы просто вводим нужные вычисления, и результаты отображаются мгновенно. Команда `bc quit` завершает интерактивную сессию.

Скрипт также можно передать в bc через стандартный ввод.

```
[me@linuxbox ~]$ bc < foo.bc
4
```

Возможность работы со стандартным вводом позволяет использовать `here documents`, `here strings` и конвейеры для передачи скриптов. Вот пример использования `here string`:

```
[me@linuxbox ~]$ bc <<< "2+2"
4
```

Пример скрипта

В качестве практического примера мы создадим скрипт для выполнения распространенного расчета — ежемесячных платежей по кредиту. В следующем скрипте мы используем `here document`, чтобы передать скрипт в bc:

```
#!/bin/bash
# loan-calc: script to calculate monthly loan payments
PROGNAME="${0##*/}" # Use parameter expansion to get basename
usage () {
    cat << _EOF_
Usage: $PROGNAME PRINCIPAL INTEREST MONTHS
Where:
PRINCIPAL is the amount of the loan.
INTEREST is the APR as a number (7% = 0.07).
MONTHS is the length of the loan's term.
_EOF_
}
```



```

}
if (($# != 3)); then
    usage
    exit 1
fi
principal=$1
interest=$2
months=$3
bc <<- EOF
    scale = 10
    i = $interest / 12
    p = $principal
    n = $months
    a = p * ((i * ((1 + i) ^ n)) / (((1 + i) ^ n) - 1))
    print a, "\n"
EOF

```

При выполнении результат выглядит так:

```

[me@linuxbox ~]$ loan-calc 135000 0.0775 180
1270.7222490000

```

В этом примере вычисляется ежемесячный платеж по кредиту в \$135 000 под 7,75% годовых сроком на 180 месяцев (15 лет). Обратите внимание на точность ответа. Она определяется значением специальной переменной `scale` в `bc` скрипте. Полное описание языка `bc` скриптов приведено в `bc man`-странице. Хотя его математическая нотация немного отличается от нотации оболочки (`bc` она ближе к `C`), большая её часть будет вам знакома на основе того, что мы изучили.

Итоги

В этой главе мы узнали о многих мелочах, которые помогают выполнять «настоящую работу» в скриптах. По мере роста опыта написания скриптов умение эффективно манипулировать строками и числами станет крайне ценным. Наш `loan-calc` скрипт демонстрирует, что даже простые скрипты могут выполнять действительно полезные задачи.

Дополнительное задание

Хотя базовый функционал `loan-calc` script уже готов, скрипт еще далек от завершения. В качестве дополнительного задания попробуйте улучшить `loan-calc` script, добавив следующие возможности:

- Полная проверка аргументов командной строки
- Опцию командной строки для «интерактивного» режима, который будет запрашивать у пользователя сумму основного долга, процентную ставку и срок кредита

- Улучшенный формат вывода

Что почитать

- В Bash Reference Manual подробно описано раскрытие параметров:
<https://www.gnu.org/software/bash/manual/bashref.html#Shell-Parameter-Expansion> - sion
- В Википедии есть хорошая статья о битовых операциях:
https://en.wikipedia.org/wiki/Bit_operation
- и статья о тернарных операциях:
https://en.wikipedia.org/wiki/Ternary_operation
- а также описание формулы расчета платежей по кредиту, используемой в нашем `loan-calc` скрипте:
https://en.wikipedia.org/wiki/Amortization_calculator

35 — Массивы

В предыдущей главе мы рассмотрели, как оболочка может манипулировать строками и числами. Типы данных, которые мы изучили, в компьютерных науках называют скалярными переменными — то есть переменными, содержащими одно значение.

В этой главе мы рассмотрим еще один вид структур данных — массивы, которые хранят несколько значений. Массивы есть практически в каждом языке программирования. Оболочка тоже их поддерживает, хотя и довольно ограниченно. Тем не менее, они могут быть полезны для решения определенных задач программирования.

Что такое массивы?

Массивы — это переменные, которые могут хранить более одного значения одновременно. Они организованы подобно таблице. Возьмем в качестве примера электронную таблицу. Она работает как двумерный массив: у нее есть строки и столбцы, а положение отдельной ячейки определяется адресом строки и столбца. Массив работает так же. Он состоит из ячеек, называемых элементами, и каждый элемент содержит данные. Доступ к отдельному элементу массива осуществляется по адресу, называемому индексом или под индексом.

Большинство языков программирования поддерживают многомерные массивы. Электронная таблица — это пример двумерного массива (ширина и высота). Многие языки поддерживают массивы с произвольным числом измерений, хотя двумерные и трехмерные массивы используются чаще всего.

Массивы в `bash` ограничены одним измерением. Их можно представить как таблицу с одним столбцом. Даже с таким ограничением у них много применений. Поддержка массивов впервые появилась в `bash` версии 2. Оригинальная программа `Unix shell`, `sh`, массивы вообще не поддерживала.

Создание массива

Переменные массива называются так же, как и другие `bash` переменные, и создаются автоматически при обращении к ним. Вот пример:

```
[me@linuxbox ~]$ a[1]=foo
[me@linuxbox ~]$ echo ${a[1]}
foo
```

Здесь показан пример присваивания и обращения к элементу массива. Первая команда присваивает элементу 1 массива `a` значение «foo». Вторая команда выводит сохраненное значение элемента 1. Использование фигурных скобок во второй команде необходимо, чтобы оболочка не пыталась выполнить раскрытие пути для имени элемента массива.

Массив также можно создать с помощью команды `declare`.

```
[me@linuxbox ~]$ declare -a a
```

Используя опцию `-a`, этот пример `declare` создает массив `a`.

Присваивание значений массиву

Значения можно присваивать двумя способами. Одиночные значения присваиваются с помощью следующего синтаксиса, где `name` — имя массива, а `subscript` — целое число (или арифметическое выражение) больше или равное нулю:

```
name[subscript]=value
```

Примечание: первый элемент массива имеет индекс ноль, а не один. `value` — это строка или целое число, присваиваемое элементу массива.

Несколько значений можно присвоить с помощью следующего синтаксиса, где `name` — имя массива, а `value` — заполнители, значения которых последовательно присваиваются элементам массива, начиная с нулевого:

```
name=(value1 value2...)
```

Например, если мы хотим присвоить сокращенные названия дней недели массиву `days`, мы можем сделать следующее:

```
[me@linuxbox ~]$ days=(Sun Mon Tue Wed Thu Fri Sat)
```

Также можно присваивать значения конкретному элементу, указывая индекс для каждого значения.

```
[me@linuxbox ~]$ days=([0]=Sun [1]=Mon [2]=Tue [3]=Wed [4]=Thu
[5]=Fri [6]=Sat)
```

Доступ к элементам массива

Для чего нужны массивы? Как и многие задачи по управлению данными можно решить в табличном процессоре, многие задачи программирования можно решить с помощью массивов.

Рассмотрим простой пример сбора и представления данных. Мы создадим скрипт, который проверяет время изменения файлов в указанном каталоге. На основе этих данных скрипт выведет таблицу, показывающую, в какой час дня файлы изменялись в последний раз. Такой скрипт можно использовать для определения периодов наибольшей активности системы. Этот скрипт под названием `hours` выдает следующий результат:

```
[me@linuxbox ~]$ hours .
Hour  Files  Hour  Files
----  -
00    0      12    11
01    1      13     7
02    0      14     1
03    0      15     7
04    1      16     6
05    1      17     5
06    6      18     4
07    3      19     4
08    1      20     1
09   14      21     0
10    2      22     0
11    5      23     0
Total files = 80
```

Мы запускаем программу `hours`, указывая текущий рабочий каталог в качестве цели. Она выводит таблицу, показывающую количество файлов, измененных в каждый час суток (0–23). Код для этого выглядит так:

```
#!/bin/bash
# hours: script to count files by modification time
usage () {
    echo "usage: ${0##*/} directory" >&2
}
# Check that argument is a directory
if [[ ! -d "$1" ]]; then
    usage
    exit 1
fi
# Initialize array
for i in {0..23}; do hours[i]=0; done
# Collect data
for i in $(stat -c %y "$1"/* | cut -c 12-13); do
    j="${i#0}"
    ((++hours[j]))
    ((++count))
done
# Display data
echo -e "Hour\tFiles\tHour\tFiles"
echo -e "----\t-----\t----\t-----"
for i in {0..11}; do
    j=$((i + 12))
    printf "%02d\t%d\t%02d\t%d\n" \
        "$i" \
        "${hours[i]}" \
        "$j" \
        "${hours[j]}"
```

```
"$j" \
"${hours[j]}"
done
printf "\nTotal files = %d\n" "$count"
```

Скрипт состоит из одной функции (usage) и основного тела с четырьмя разделами. В первом разделе мы проверяем наличие аргумента командной строки и то, является ли он каталогом. Если нет, мы выводим сообщение о правилах использования и завершаем работу.

Во второй секции инициализируется массив hours. Это достигается присвоением каждому элементу значения нуля. Специальных требований к подготовке массивов перед использованием нет, но нашему скрипту нужно гарантировать, что ни один элемент не пуст. Обратите внимание на интересный способ построения цикла. Используя раскрытие фигурных скобок ({0..23}), мы можем легко сгенерировать последовательность слов для команды for.

В следующем разделе данные собираются путем запуска программы stat для каждого файла в каталоге. Мы используем cut, чтобы извлечь из результата двухзначный час. Внутри цикла нам нужно удалить ведущие нули из поля часа, так как оболочка попытается (и в итоге не сможет) интерпретировать значения от 00 до 09 как восьмеричные числа (см. таблицу 34-2). Затем мы увеличиваем значение элемента массива, соответствующего часу суток. Наконец, мы увеличиваем счетчик (count), чтобы отслеживать общее количество файлов в каталоге.

В последней секции скрипта выводятся содержимое массива. Сначала мы выводим пару строк заголовка, затем входим в цикл, который формирует четыре столбца вывода. В завершение выводится итоговое количество файлов.

Операции с массивами

Существует множество распространенных операций с массивами. Такие действия, как удаление массивов, определение их размера, сортировка и прочее, широко применяются в скриптах.

Вывод всего содержимого массива

Подстроки * и @ позволяют получить доступ к каждому элементу массива. Как и в случае с позиционными параметрами, нотация @ является более полезной. Вот пример:

```
[me@linuxbox ~]$ animals=("a dog" "a cat" "a fish")
[me@linuxbox ~]$ for i in ${animals[*]}; do echo $i; done
a
dog
a
cat
a
fish
[me@linuxbox ~]$ for i in ${animals[@]}; do echo $i; done
a
dog
a
cat
a
fish
[me@linuxbox ~]$ for i in "${animals[*]}"; do echo $i; done
a dog a cat a fish
[me@linuxbox ~]$ for i in "${animals[@]}"; do echo $i; done
a dog
a cat
a fish
```

Мы создаем массив `animals` и присваиваем ему три строки, состоящие из двух слов. Затем мы запускаем четыре цикла, чтобы увидеть, как разделение слов влияет на содержимое массива. Поведение записей `${animals[*]}` и `${animals[@]}` идентичны, пока они не заключены в кавычки. Запись `*` дает одно слово, содержащее содержимое массива, тогда как запись `@` дает три строки из двух слов, что соответствует «реальному» содержимому массива.

Определение количества элементов массива

С помощью раскрытия параметров можно определить количество элементов в массиве почти так же, как и длину строки. Вот пример:

```
[me@linuxbox ~]$ a[100]=foo
[me@linuxbox ~]$ echo ${#a[@]} # number of array elements
1
[me@linuxbox ~]$ echo ${#a[100]} # length of element 100
3
```

Мы создаем массив `a` и присваиваем строку `foo` элементу 100. Затем, используя нотацию `@`, мы применяем раскрытие параметров, чтобы узнать длину массива. Наконец, мы проверяем длину элемента 100, содержащего строку `foo`. Интересно, что, хотя мы присвоили строку элементу 100, `bash` сообщает лишь об одном элементе в массиве. Это отличается от поведения некоторых других языков, где неиспользуемые элементы массива (от 0 до 99) инициализировались бы пустыми значениями и учитывались. В `bash` элементы массива существуют только в том случае, если им присвоено значение, независимо от их индекса.

Поиск используемых массивом индексов

Поскольку `bash` позволяет массивам иметь «пропуски» при назначении индексов, иногда полезно определить, какие элементы существуют на самом деле. Это можно сделать с помощью раскрытия параметров в следующих формах, где `array` — имя переменной массива:

```
${!array[*]}
```

```
${!array[@]}
```

Как и другие раскрытия с использованием `*` и `@`, форма `@`, заключенная в кавычки, наиболее полезна, так как она раскрывается в отдельные слова.

```
[me@linuxbox ~]$ foo=( [2]=a [4]=b [6]=c )
[me@linuxbox ~]$ for i in "${foo[@]"}; do echo $i; done
a
b
c
[me@linuxbox ~]$ for i in "${!foo[@]}"; do echo $i; done
2
4
6
```

Присваивание элементов массива с помощью `read -a`

Встроенная команда `read` имеет опцию `(-a)`, позволяющую помещать слова в индексированный массив, а не в набор переменных, как мы делали ранее. Вот пример:

```
[me@linuxbox ~]$ declare -a foo
[me@linuxbox ~]$ read -a foo <<< "0th 1st 2nd 3rd 4th"
[me@linuxbox ~]$ for i in "${foo[@]"}; do echo "$i"; done
0th
1st
2nd
3rd
4th
```

Добавление элементов в конец массива

Знание количества элементов в массиве не помогает, если нужно добавить значения в конец, так как обозначения `*` и `@` не указывают на максимальный используемый индекс массива. К счастью, оболочка предлагает решение. Используя оператор присваивания `+=`, мы можем автоматически добавлять значения в конец массива. Здесь мы присваиваем три значения массиву `foo`, а затем добавляем еще три.

```
[me@linuxbox ~]$ foo=(a b c)
[me@linuxbox ~]$ echo "${foo[@]}"
a b c
[me@linuxbox ~]$ foo+=(d e f)
```



```
[me@linuxbox ~]$ echo ${foo[@]}
a b c d e f
```

Чтение файла в массив

В последних версиях `bash` появилась новая встроенная команда `mapfile`, которая может считывать стандартный ввод напрямую в индексированный массив. Её синтаксис выглядит так:

```
mapfile -options array
```

Команда `mapfile` имеет несколько интересных опций, некоторые из которых приведены в таблице 35-1.

Таблица 35-1: Опции `mapfile`

Опция	Описание
<code>-d delim</code>	Использовать <code>delim</code> вместо символа новой строки для завершения строк.
<code>-n count</code>	Прочитать только <code>count</code> строк.
<code>-O origin</code>	Начать присваивание элементов массива с индекса <code>origin</code> , а не с индекса 0.
<code>-s count</code>	Пропустить первые <code>count</code> строк файла.
<code>-t</code>	Удалить завершающий разделитель в каждой строке.

Чтобы продемонстрировать работу `mapfile`, мы создадим короткий скрипт, генерирующий случайные парольные фразы из четырех слов. Они удобны как альтернатива обычным паролям, так как они длинные, но при этом легко запоминаются (если вы знаете, как их писать ;-).

```
#!/bin/bash
# array-mapfile - demonstrate mapfile builtin
```

```
DICTIONARY=/usr/share/dict/words
WORDLIST=~/.wordlist.txt
declare -a words
# create filtered word list
grep -v \' < "$DICTIONARY" \
  | grep -v "[[:upper:]]" \
  | shuf > "$WORDLIST"
# read WORDLIST into array
mapfile -t -n 32767 words < "$WORDLIST"
# create four word passphrase
while [[ -z $REPLY ]]; do
  echo "${words[$RANDOM]}" \
    "${words[$RANDOM]}" \
    "${words[$RANDOM]}" \
    "${words[$RANDOM]}"
  echo
```

```
read -r -p "Enter to continue, q to quit > "  
echo  
done
```

Этот скрипт использует /usr/share/dict/words file, очищенный от апострофов и заглавных букв. Мы применяем команду shuf, чтобы перемешать список слов и получить случайный порядок.

Затем мы загружаем первые 32 767 слов из файла в массив words. Почему 32 767? Потому что мы будем использовать переменную RANDOM для выбора случайных элементов из массива, а при каждом обращении к переменной RANDOM она возвращает случайное целое число от 0 до 32 767.

```
[me@linuxbox ~]$ ./array-mapfile  
conversions slumbers appendages metastasizing  
Enter to continue, q to quit >  
kettles rhinestones unused demagnetizes  
Enter to continue, q to quit >  
wear conveys characterizing extrusion  
Enter to continue, q to quit > q
```

Вот результат работы. Каждый раз при нажатии ENTER скрипт выводит четыре случайных слова.

Срез массива

Существует вид раскрытия параметров, позволяющий извлечь из массива группу смежных элементов — так называемый срез. В результате такого раскрытия получаются элементы массива из выбранного среза оригинала, как показано ниже.

```
[me@linuxbox ~]$ arr=(0th 1st 2nd 3rd 4th)  
[me@linuxbox ~]$ echo "${arr[@]:2:3}"  
2nd 3rd 4th
```

В этом примере мы создаем массив из пяти элементов. Затем извлекаем три элемента, начиная с индекса 2. Указав отрицательный индекс, мы ведем отсчет с конца массива, а не с начала. В следующем примере мы извлекаем два последних элемента массива. Обратите внимание на обязательный пробел перед знаком минус.

```
[me@linuxbox ~]$ echo "${arr[@]: -2:2}"  
3rd 4th
```

Мы также можем легко создать массив, содержащий элементы среза.

```
[me@linuxbox ~]$ arr2=("${arr[@]:2:3}")  
[me@linuxbox ~]$ echo "${arr2[@]}"  
2nd 3rd 4th
```

Здесь мы создали массив `arr2` и наполнили его тремя элементами из `arr`.

Сортировка массива

Как и в электронных таблицах, часто требуется отсортировать значения в столбце данных. У оболочки нет прямого способа сделать это, но с помощью небольшого кода это не составит труда.

```
#!/bin/bash
# array-sort: Sort an array
a=(f e d c b a)
echo "Original array: " "${a[@]}"
a_sorted=$(for i in "${a[@]}"; do echo "$i"; done | sort)
echo "Sorted array: " "${a_sorted[@]}"
```

При выполнении скрипт выдает следующее:

```
[me@linuxbox ~]$ array-sort
Original array: f e d c b a
Sorted array:  a b c d e f
```

Скрипт работает путем копирования содержимого исходного массива (`a`) во второй массив (`a_sorted`) с помощью хитрого подстановки команды. Этот базовый метод можно использовать для выполнения множества операций с массивом, меняя структуру конвейера.

Удаление массива

Чтобы удалить массив, используйте команду `unset`.

```
[me@linuxbox ~]$ foo=(a b c d e f)
[me@linuxbox ~]$ echo ${foo[@]}
a b c d e f
[me@linuxbox ~]$ unset foo
[me@linuxbox ~]$ echo ${foo[@]}
[me@linuxbox ~]$
```

`unset` также можно использовать для удаления отдельных элементов массива.

```
[me@linuxbox ~]$ foo=(a b c d e f)
[me@linuxbox ~]$ echo ${foo[@]}
a b c d e f
[me@linuxbox ~]$ unset 'foo[2]'
[me@linuxbox ~]$ echo ${foo[@]}
a b d e f
```

В этом примере мы удаляем третий элемент массива с индексом 2. Помните, что массивы начинаются с индекса 0, а не 1! Также обратите внимание, что элемент массива нужно брать в кавычки, чтобы оболочка не выполняла раскрытие пути.

Интересно, что присвоение массиву пустого значения не очищает его содержимое.

```
[me@linuxbox ~]$ foo=(a b c d e f)
[me@linuxbox ~]$ foo=
[me@linuxbox ~]$ echo ${foo[@]}
b c d e f
```

Любая ссылка обращение к переменной массива без индекса означает обращение к его нулевому элементу.

```
[me@linuxbox ~]$ foo=(a b c d e f)
[me@linuxbox ~]$ echo ${foo[@]}
a b c d e f
[me@linuxbox ~]$ foo=A
[me@linuxbox ~]$ echo ${foo[@]}
A b c d e f
```

Ассоциативные массивы

bash Версии 4.0 и выше поддерживают ассоциативные массивы. В них в качестве индексов используются строки, а не целые числа, что позволяет создавать пары «ключ — значение», подобные словарю или хеш-таблице. Это открывает новые интересные подходы к управлению данными. Например, можно создать массив с именем `colors` и использовать названия цветов в качестве индексов.

```
declare -A colors
colors["red"]="ff0000"
colors["green"]="00ff00"
colors["blue"]="0000ff"
```

В отличие от массивов с целочисленными индексами, которые создаются простым обращением к ним, ассоциативные массивы нужно явно объявлять с помощью команды `declare` с использованием опции `-A`. Доступ к элементам ассоциативных массивов осуществляется так же, как и к массивам с целочисленными индексами.

```
echo ${colors["blue"]}
```

Мы можем использовать пары «ключ — значение», характерные для ассоциативных массивов, для выполнения поиска. Ассоциативный массив позволяет обращаться к нужным данным напрямую, а не искать их последовательно, как в индексированном массиве. В следующей программе мы считываем имена всех файлов в `/usr/bin` каталоге вместе с их размерами в массив, а затем выполняем поиск на основе ввода пользователя.

```
#!/bin/bash
# array-lookup - demonstrate lookup using associative array
declare -A cmds
# fill array with commands and file sizes
cd /usr/bin || exit 1
echo "Loading commands..."
for i in ./*; do
    cmds["$i"]=$(stat -c "%s" "$i")
done
echo "${#cmds[@]} commands loaded"
# perform lookup
while true; do
    read -r -p "Enter command (empty to quit) -> "
    [[ -z $REPLY ]] && break
    if [[ -x $REPLY ]]; then
        echo "$REPLY" "${cmds[$REPLY]}" "bytes"
    else
        echo "No such command '$REPLY'."
    fi
done
```

При запуске программы мы получаем следующее:

```
[me@linuxbox ~]$ ./array-lookup
Loading commands...
2329 commands loaded
Enter command (empty to quit) -> ls
ls 138216 bytes
Enter command (empty to quit) -> cp
cp 141832 bytes
Enter command (empty to quit) -> mv
mv 137752 bytes
Enter command (empty to quit) -> rm
rm 59912 bytes
Enter command (empty to quit) ->
[me@linuxbox ~]$
```

В следующей главе мы разберем скрипт, который эффективно использует ассоциативные массивы для создания интересного отчета.

Использование ассоциативных массивов для имитации многомерности

Хотя bash поддерживает только одномерные массивы, можно легко «имитировать» многомерные, используя ассоциативный массив и создавая индексные строки, похожие на адреса многомерных массивов. Вот пример:

```
#!/bin/bash
# array-multi - simulate a multi-dimensional array
declare -A multi_array
# Load array with a sequence of numbers
counter=1
for row in {1..10}; do
    for col in {1..5}; do
        address="$row, $col"
        multi_array["$address"]=$counter
        ((counter++))
    done
done
```

```
# Output array contents
for row in {1..10}; do
    for col in {1..5}; do
        address="$row, $col"
        echo -ne "${multi_array["$address"]}" "\t"
    done
done
echo
done
```

Запуск этой программы дает следующий результат:

```
[me@linuxbox ~]$ ./array-multi
1  2  3  4  5
6  7  8  9  10
11 12 13 14 15
16 17 18 19 20
21 22 23 24 25
26 27 28 29 30
31 32 33 34 35
36 37 38 39 40
41 42 43 44 45
46 47 48 49 50
```

Итоги

Если искать слово `array` в `bash man page`, мы найдем множество примеров использования переменных массива в `bash`.

Большинство из них довольно специфичны, но в особых случаях они могут быть полезны. На самом деле, тема массивов в программировании на оболочке используется недостаточно — во многом из-за того, что традиционные программы Unix shell (такие как `sh`) не поддерживали массивы. Такая непопулярность досадна, ведь массивы широко применяются в других языках программирования и являются мощным инструментом для решения множества задач.

Массивы и циклы тесно связаны и часто используются вместе. Следующая форма цикла особенно удобна для вычисления индексов в индексированных массивах:

```
for ((expr; expr; expr))
```

Что почитать

- Пара статей в Wikipedia о структурах данных из этой главы:

[https://en.wikipedia.org/wiki/Scalar_\(computing\)](https://en.wikipedia.org/wiki/Scalar_(computing))

https://en.wikipedia.org/wiki/Associative_array

https://en.wikipedia.org/wiki/Associative_array

36 — Экзотика

В этой заключительной главе нашего путешествия мы рассмотрим некоторые второстепенные темы. Хотя в предыдущих главах мы прошли большой путь, осталось много `bash` функций, которые мы не затронули. Большинство из них довольно специфичны и полезны в основном тем, кто интегрирует `bash` в дистрибутив Linux. Однако есть несколько функций, которые, хоть и не используются повсеместно, помогают при решении определенных задач программирования. Мы разберем их здесь.

Групповые команды и подболочки

`bash` позволяет группировать команды. Это можно сделать двумя способами: с помощью групповой команды или подболочки. Мы уже рассматривали групповые команды в главе 6, и, как вы помните, они используют следующий синтаксис:

```
{ command1; command2; [command3;...] }
```

Подболочки используют похожий синтаксис.

```
(command1; command2; [command3;...])
```

Групповая команда заключает команды в фигурные скобки, а подболочка — в круглые. Важно отметить, что из-за особенностей реализации групповых команд в `bash`, скобки должны отделяться от команд пробелом, а последняя команда перед закрывающей скобкой должна заканчиваться точкой с запятой или переходом на новую строку.

Зачем нужны групповые команды и подболочки? Хотя между ними есть важное различие (о котором мы поговорим чуть позже), обе они позволяют управлять перенаправлением. Рассмотрим фрагмент скрипта, выполняющий перенаправление для нескольких команд.

```
ls -l > output.txt
echo "Listing of foo.txt" >> output.txt
cat foo.txt >> output.txt
```

Здесь всё просто. Вывод трёх команд перенаправляется в файл `output.txt`. Используя групповую команду, мы можем записать это так:

```
{ ls -l; echo "Listing of foo.txt"; cat foo.txt; } > output.txt
```

Использование подболочки работает аналогично.

```
(ls -l; echo "Listing of foo.txt"; cat foo.txt) > output.txt
```

Этот метод сэкономил нам время на ввод, но групповые команды и подоболочки по-настоящему незаменимы в конвейерах. При построении конвейера команд часто полезно объединить результаты нескольких команд в один поток. Групповые команды и подоболочки позволяют сделать это легко.

```
{ ls -l; echo "Listing of foo.txt"; cat foo.txt; } | lpr
```

Здесь мы объединили вывод трех команд и направили его на вход `lpr`, чтобы получить печатный отчет.

Хотя групповые команды и подоболочки похожи и обе могут использоваться для объединения потоков при перенаправлении, между ними есть важное различие. В то время как групповая команда выполняет все свои команды в текущей оболочке, подболочка (как следует из названия) выполняет команды в дочерней копии текущей оболочки. Это означает, что окружение (включая функции оболочки, псевдонимы и переменные) копируется и передается новому экземпляру оболочки. Эта копия окружения отличается от работы обычного дочернего процесса тем, что подболочка наследует полную копию окружения родителя, тогда как дочерний процесс наследует только экспортированные переменные родительской оболочки. Когда подболочка завершает работу, её копия окружения теряется, а вместе с ней и любые изменения, внесенные в окружение подболочки (включая присваивание переменных).

Что наиболее важно, подболочки, в отличие от групповых команд, подобно дочерним процессам, не могут изменять окружение родительской оболочки. Давайте продемонстрируем это, сначала с помощью групповой команды:

```
[me@linuxbox ~]$ foo="Original Value"
[me@linuxbox ~]$ { foo="Altered Value"; echo $foo; }
Altered Value
[me@linuxbox ~]$ echo $foo
Altered Value
```

Далее мы выполним те же шаги с подболочкой.

```
[me@linuxbox ~]$ foo="Original Value"
[me@linuxbox ~]$ ( foo="Altered Value"; echo $foo )
Altered Value
[me@linuxbox ~]$ echo $foo
Original Value
```

Как видно, групповая команда может изменить значение `foo` в текущей оболочке, а подболочка — нет. Эта особенность подболочек полезна, когда нужно выполнить что-то вроде

смены каталога. При выполнении команды `cd` в подоболочке текущий рабочий каталог меняется лишь на время работы подоболочки, а после возврата в родительскую оболочку текущий рабочий каталог остается прежним.

```
[me@linuxbox ~]$ pwd
/home/me
[me@linuxbox ~]$ ( cd /usr/bin; pwd )
/usr/bin
[me@linuxbox ~]$ pwd
/home/me
```

В целом же, если скрипту не требуется подоболочка, групповые команды предпочтительнее. Они работают быстрее и потребляют меньше памяти.

В следующем скрипте мы используем групповые команды и рассмотрим несколько приемов программирования, которые можно применять вместе с ассоциативными массивами. Этот скрипт под названием `array-2`, при получении имени каталога, выводит список файлов в нем вместе с именами владельца и группы владельца. В конце вывода скрипт подсчитывает количество файлов, принадлежащих каждому владельцу и группе. Вот результаты (сокращенно для краткости) при запуске скрипта для каталога `/usr/bin`:

```
[me@linuxbox ~]$ array-2 /usr/bin
/usr/bin/2to3-2.6          root      root
/usr/bin/2to3              root      root
/usr/bin/a2p               root      root
/usr/bin/abrowser          root      root
/usr/bin/aconnect         root      root
/usr/bin/acpi_fakekey      root      root
/usr/bin/acpi_listen       root      root
/usr/bin/add-apt-repository root      root
.
.
.
/usr/bin/zipgrep           root      root
/usr/bin/zipinfo           root      root
/usr/bin/zipnote           root      root
/usr/bin/zip               root      root
/usr/bin/zipsplit          root      root
/usr/bin/zjsdecode         root      root
/usr/bin/zsoelim           root      root
File owners:
daemon : 1 file(s)
root   : 1394 file(s)
File group owners:
crontab : 1 file(s)
daemon  : 1 file(s)
lpadmin : 1 file(s)
mail    : 4 file(s)
mlocate : 1 file(s)
root    : 1380 file(s)
shadow  : 2 file(s)
ssh     : 1 file(s)
tty     : 2 file(s)
utmp    : 2 file(s)
```

Ниже представлен листинг скрипта (с номерами строк):

```
1  #!/bin/bash
2
3  # array-2: Use arrays to tally file owners
4
5  declare -A files file_group file_owner groups owners
6
7  if [[ ! -d "$1" ]]; then
8      echo "Usage: array-2 dir" >&2
9      exit 1
10 fi
11
12 for i in "$1"/*; do
13     owner="$(stat -c %U "$i")"
14     group="$(stat -c %G "$i")"
15     files["$i"]="$i"
16     file_owner["$i"]="$owner"
17     file_group["$i"]="$group"
18     ((++owners[$owner]))
19     ((++groups[$group]))
20 done
21
22 # List the collected files
23 { for i in "${files[@]}"; do
24     printf "%-40s %-10s %-10s\n" \
25         "$i" "${file_owner["$i"]}" "${file_group["$i"]}"
26 done } | sort
27 echo
28
29 # List owners
30 echo "File owners:"
31 { for i in "${!owners[@]}"; do
32     printf "%-10s: %5d file(s)\n" "$i" "${owners["$i"]}"
33 done } | sort
34 echo
35
36 # List groups
37 echo "File group owners:"
38 { for i in "${!groups[@]}"; do
39     printf "%-10s: %5d file(s)\n" "$i" "${groups["$i"]}"
40 done } | sort
```

Разберем механику этого скрипта:

Строка 5: ассоциативные массивы создаются с помощью команды `declare` с использованием опции `-A`. В этом скрипте мы создаем пять массивов следующим образом:

- `files` содержит имена файлов в каталоге, где ключом является имя файла.
- `file_group` содержит группу-владельца каждого файла, где ключом является имя файла.
- `file_owner` содержит владельца каждого файла, где ключом является имя файла.
- `groups` содержит количество файлов, принадлежащих индексированной группе.

- `owners` содержит количество файлов, принадлежащих индексируемому владельцу.

Строки 7–10: здесь проверяется, было ли передано имя существующего каталога в качестве позиционного параметра. Если нет, выводится сообщение об использовании и скрипт завершается со статусом выхода 1.

Строки 12–20: эти строки перебирают файлы в каталоге. Используя `stat` команду, строки 13 и 14 извлекают имена владельца и группы файла и присваивают эти значения соответствующим массивам (строки 16 и 17), используя имя файла в качестве индекса массива. Аналогично, само имя файла присваивается `files array` (строка 15).

Строки 18–19: общее количество файлов, принадлежащих владельцу и группе владельца, увеличивается на единицу.

Строки 22–27: выводится список файлов. Это достигается с помощью раскрытия параметра `"${array[@]}"`, которое разворачивает весь список элементов массива, где каждый элемент считается отдельным словом. Это позволяет обрабатывать имена файлов с пробелами. Также обратите внимание, что весь цикл заключен в фигурные скобки, образуя групповую команду. Это позволяет передать весь вывод цикла в команду `sort` через конвейер. Это необходимо, так как элементы ассоциативного массива не отсортированы.

Строки 29 `"$ {!array[@]}"` 40: эти два цикла похожи на цикл со списком файлов, за исключением того, что они используют раскрытие, которое разворачивает список индексов массива, а не список его элементов.

Подстановка процессов

В главе 28 мы видели пример проблемы среды подболочки, когда выяснилось, что `read` в конвейере работает не так, как можно было бы ожидать. Напомним: если построить конвейер следующим образом, то содержимое `REPLY` всегда будет пустым, так как `read` выполняется в подболочке, а её копия `REPLY` уничтожается после завершения подболочки.

```
echo "foo" | read  
echo $REPLY
```

Поскольку команды в конвейере всегда выполняются в дочерних оболочках, любая команда, присваивающая значения переменным, столкнется с этой проблемой. К счастью, оболочка предоставляет экзотический вид раскрытия подстановок —

подстановку процессов, которая позволяет обойти это ограничение.

Подстановка процессов записывается двумя способами.

Для процессов, выводящих стандартный вывод, это выглядит так:

```
<(list)
```

или, для процессов, принимающих стандартный ввод, это выглядит так:

```
>(list)
```

где `list` — это список команд.

Чтобы решить нашу проблему с `read`, можно использовать подстановку процессов следующим образом:

```
read < <(echo "foo")
echo $REPLY
```

Подстановка процессов позволяет использовать вывод подпроцесса как обычный файл для перенаправления. На самом деле, поскольку это форма раскрытия, мы можем изучить её реальное значение.

```
[me@linuxbox ~]$ echo <(echo "foo")
/dev/fd/63
```

Используя `echo`, чтобы увидеть результат раскрытия, мы замечаем, что вывод подпроцесса предоставляется файлом с именем `/dev/fd/63`.

Подстановку процессов часто используют в циклах, содержащих `read`. Вот пример цикла `read`, который обрабатывает содержимое списка каталогов, созданного подпроцессом:

```
#!/bin/bash
# pro-sub: demo of process substitution
while read -r attr links owner group size date time filename; do
    cat << _EOF_
    Filename:  $filename
    Size:      $size
    Owner:     $owner
    Group:     $group
    Modified:  $date $time
    Links:     $links
    Attributes: $attr
    _EOF_
done < <(ls -l --time-style="+%F %H:%m"| tail -n +2)
```

Цикл выполняет `read` для каждой строки вывода содержимого каталога. Сам вывод формируется в последней строке скрипта. Эта строка перенаправляет результат подстановки процесса на стандартный ввод цикла. Команда `tail` включена в конвейер

подстановки процесса, чтобы исключить первую строку вывода, которая не нужна.

При выполнении скрипт выводит следующее:

```
[me@linuxbox ~]$ pro-sub | head -n 20
Filename:  addresses.ldif
Size:      14540
Owner:     me
Group:     me
Modified:  2009-04-02 11:12
Links:     1
Attributes: -rw-r--r--
Filename:  bin
Size:      4096
Owner:     me
Group:     me
Modified:  2009-07-10 07:31
Links:     2
Attributes: drwxr-xr-x
Filename:  bookmarks.html
Size:      394213
Owner:     me
Group:     me
```

Конструирование команд с помощью eval

Встроенная команда `eval` — странная и загадочная. Если вкратце, она принимает список аргументов, объединяет их в строку и передает оболочке для выполнения. Естественно возникает вопрос: зачем это нужно? Какой в этом смысл?

В написании сценариев для оболочки бывают случаи, когда команду нужно сформировать динамически во время выполнения, и `eval` позволяет это сделать. Проведем эксперимент. Хотя мы не разбирали это подробно, можно поместить команду в строковую переменную, а затем использовать раскрытие параметров, чтобы развернуть переменную в команду.

```
[me@linuxbox ~]$ cmd="echo foo"
[me@linuxbox ~]$ $cmd
foo
[me@linuxbox ~]$
```

Теперь рассмотрим, что происходит при использовании переменной в команде.

```
[me@linuxbox ~]$ str=abcde
[me@linuxbox ~]$ cmd="echo $str"
[me@linuxbox ~]$ $cmd
abcde
[me@linuxbox ~]$
```

Это работает как ожидалось, но посмотрим, что произойдет, если мы присвоим `str` новое значение.

```
[me@linuxbox ~]$ str=ABCDE
[me@linuxbox ~]$ $cmd
abcde
[me@linuxbox ~]$
```

Результаты не изменились. Это логично, так как мы уже выполнили раскрытие параметров для `$str`, когда присваивали `cmd`. Но что, если мы захотим раскрыть переменную позже? Мы могли бы заключить команду в одинарные кавычки, чтобы подавить раскрытие параметров при первоначальном присваивании переменной.

```
[me@linuxbox ~]$ cmd='echo $str'
[me@linuxbox ~]$ $cmd
$str
[me@linuxbox ~]$
```

Хотя при присваивании `cmd` раскрытие параметров не произошло, оно не произошло и при выполнении `cmd`. Причина в том, что оболочка выполняет раскрытие только один раз, даже если результат раскрытия (`$cmd`) содержит нечто, что можно раскрыть далее (`$str`).

Если мы используем `eval`, мы получим дополнительный уровень раскрытия.

```
[me@linuxbox ~]$ str=abcde
[me@linuxbox ~]$ cmd='echo $str'
[me@linuxbox ~]$ eval "$cmd"
abcde
[me@linuxbox ~]$ str=ABCDE
[me@linuxbox ~]$ eval "$cmd"
ABCDE
[me@linuxbox ~]$
```

Это показывает работу `eval`. Она объединяет аргументы в одну строку, выполняет раскрытие тильды, параметров и путей в содержимом строки, а затем передает строку текущей оболочке (без создания `subshell`) для выполнения.

Будьте осторожны с `eval`

Команда `eval` имеет плохую репутацию. Это связано с опасениями, что с помощью `eval` в оболочку скрипта легко добавить уязвимости. Если строка, переданная в `eval`, поступает из внешнего источника (то есть от пользователя), нужно убедиться, что в строку не встроены несанкционированные команды (это называется атакой типа «инъекция кода»). Всегда проверяйте содержимое строки, передаваемой в `eval`.

Помощник для Wordle

В следующем скрипте мы будем использовать `eval`, чтобы помочь найти возможные ответы в игре Wordle.

Для тех, кто не знаком с этой популярной игрой, в Wikipedia дано следующее описание:

Wordle — это веб-игра в слова, созданная валлийским инженером Джошем Уордлом. У игрока есть шесть попыток, чтобы угадать пятибуквенное слово; при этом каждая попытка сопровождается обратной связью в виде цветных плиток, указывающих, совпадают ли буквы или занимают ли они правильную позицию.

По сути, после каждого ввода игра сообщает, есть ли в нем буквы из секретного слова и совпадают ли они с его позициями. Таким образом, с каждой попыткой мы узнаем о секретном слове всё больше, пока (надемся) не угадаем его. Цель игры — найти слово за минимальное количество попыток.

Вспомогательный скрипт выводит список слов, соответствующих известным данным из предыдущих попыток. Для этого скрипту передается простое регулярное выражение, указывающее, какие буквы известны и на каких позициях они находятся в секретном слове. Кроме того, мы указываем список букв, которые точно есть в слове, и тех, которых в нем точно нет.

Допустим, секретное слово — «about», а мы угадали «bloat». Wordle сообщит, что буквы «o» и «t» стоят на своих местах, а также что в слове есть «a» и «b», но их позиции неизвестны, и что секретное слово не содержит «l». В этом случае мы вызовем скрипт так:

```
[me@linuxbox ~]$ eval-wordle ..o.t +a +b -l
```

Первый аргумент — регулярное выражение из пяти символов, где «o» и «t» стоят на своих местах. Далее следуют известные символы секретного слова и буквы, которые точно в него не входят. Скрипт выдает следующий результат, показывая, что критериям соответствуют лишь два слова из словаря:

```
abort
about
2
```

Скрипт работает путем фильтрации словаря (`/usr/share/dict/words`) согласно критериям выбора и последующего вывода остатка. Для этого мы построим длинный многосоставный конвейер, длина которого меняется в зависимости от количества позиционных параметров, переданных в командной строке. Такой подход избавляет от

необходимости использовать временные файлы для хранения промежуточных результатов.

```
1  #!/bin/bash
2  # eval-wordle - demonstrate eval by solving wordle puzzles
3  PROGRAMNAME=${0##*/}
4  DICTIONARY=/usr/share/dict/words
5  usage() {
6      printf "%s\n" "Usage: ${PROGRAMNAME} [-h|--help]"
7      printf "%s\n" "      ${PROGRAMNAME} regex +-char..."
8  }
9  help_message() {
10     cat << _EOF_
11 $PROGRAMNAME - Wordle Helper
12 $(usage)
13 Options:
14 -h, --help    Display this help message and exit.
15 Arguments:
16 The first argument must be a five character regular
17 expression at minimum of '.....' representing five
18 unknown characters in the answer. This is followed by
19 zero or more character known to be either present or
20 absent from the answer. These are expressed as either
21 +char for letters known to be present or -char for
22 letters known to not be in the answer.
23 Example:
24 $PROGRAMNAME a.... +o -v
25 This means we know that the answer starts with 'a' in
26 the first position and also contains an 'o' but does
27 not contain a 'v'.
28 _EOF_
29 return
30 }
31 add_plus() { # Add a letter
32     local char="$1"
33     echo " | grep $char"
34     return
35 }
36 add_minus() { # Subtract a letter
37     local char="$1"
38     echo " | grep -v $char"
39     return
40 }
41 # Parse command-line
42 if [[ "$1" == "-h" || "$1" == "--help" ]]; then
43     help_message
44     exit 0
45 fi
46 if (( "${#1}" == 5 )); then
47     known_chars="$1"
```

```
50     shift
51 else
52     echo "First argument must be a 5 character regex." >&2
53     exit 1
54 fi
55 cmd="LANG=POSIX grep '^.....$' $DICTIONARY \
56 | grep -v '[:punct:]' \
57 | grep -v '[:upper:]' \
58 | grep '$known_chars'"
59 while [[ -n "$1" ]]; do
60     case "$1" in
61         -[:alpha:])
62             cmd="$cmd${add_minus "$1:1"}"
63     ;;
```



```

61      +[:alpha:]))
62      cmd="${cmd}${add_plus "${1:1}"}"
63      ;;
64      *)
65      echo "Invalid argument '$1'" >&2
66      exit 1
67      ;;
68  esac
69  shift
70  done
71  eval "$cmd | tee >(wc -l)"

```

Разберем интересные особенности скрипта:

Строки 30–39: Определяют две функции для создания отдельных элементов итогового конвейера. Одна функция добавляет фильтр, требующий наличия указанной буквы, а другая — фильтр, исключающий любые слова с этой буквой.

Строки 40–51: Начинают обработку позиционных параметров, начиная с первого. Мы проверяем, является ли первый аргумент командной строки опцией справки или обязательным регулярным выражением из пяти символов.

Строка 52: Создает начало конвейера. Первый шаг — фильтрация словаря для удаления всех слов, длина которых не равна ровно пяти символам. Далее мы отсеиваем слова с пунктуацией (обычно апострофами) и заглавными буквами, так как Wordle не использует имена собственные.

Строка 56: начинает цикл обработки оставшихся аргументов командной строки. Каждый раз, когда находится «+letter» или «-letter», мы удаляем ведущий плюс или минус и вызываем соответствующую функцию, чтобы добавить следующий элемент конвейера в строку команды.

Строка 71: Здесь выполняется основная работа. Чтобы подсчитать общее количество слов, мы направляем отфильтрованный список слов через конвейер в `tee`, который передает список в стандартный вывод (для отображения на экране) и в «файл», являющийся программой `wc`, запущенной через подстановку процесса. Хитро.

Ловушки

В главе 10 мы видели, как программы могут реагировать на сигналы. Эту возможность можно добавить и в наши скрипты. Хотя написанные нами до сих пор скрипты не нуждались в этом (так как они выполняются очень быстро и не создают временных файлов), более крупные и сложные скрипты могут выиграть от наличия подпрограммы обработки сигналов.

При проектировании крупного и сложного скрипта важно предусмотреть, что произойдет, если пользователь выйдет из системы или выключит компьютер во время работы скрипта. При таком событии всем затронутым процессам будет отправлен сигнал. В свою очередь, программы, представляющие эти процессы, могут выполнить действия для обеспечения правильного и упорядоченного завершения работы. Допустим, мы написали скрипт, который создает временный файл во время выполнения. С точки зрения хорошего проектирования, скрипт должен удалять этот файл по завершении работы. Также было бы разумно настроить удаление файла, если получен сигнал, указывающий на преждевременное завершение программы.

bash предоставляет для этой цели механизм, называемый `trap`. Ловушки реализуются с помощью встроенной команды с соответствующим именем — `trap`. `trap` использует следующий синтаксис, где `argument` — это строка, которая будет прочитана и выполнена как команда, а `signal` — это указание сигнала, который инициирует выполнение интерпретируемой команды:

```
trap argument signal [signal...]
```

Вот простой пример:

```
#!/bin/bash
# trap-demo: simple signal handling demo
trap "echo 'I am ignoring you.'" SIGINT SIGTERM
for i in {1..5}; do
    echo "Iteration $i of 5"
    sleep 5
done
```

Этот скрипт определяет `trap`, который будет выполнять `echo command` каждый раз, когда во время работы скрипта поступает сигнал `SIGINT` или `SIGTERM`. Вот как выглядит выполнение программы, когда пользователь пытается остановить скрипт нажатием `Ctrl-C`:

```
[me@linuxbox ~]$ trap-demo
Iteration 1 of 5
Iteration 2 of 5
^CI am ignoring you.
Iteration 3 of 5
^CI am ignoring you.
Iteration 4 of 5
Iteration 5 of 5
```

Как видно, каждый раз, когда пользователь пытается прервать программу, вместо этого выводится сообщение.

Составлять строку для полезной последовательности команд бывает неудобно, поэтому принято указывать в качестве

команды функцию оболочки. В этом примере для каждого обрабатываемого сигнала указана отдельная функция оболочки:

```
#!/bin/bash
# trap-demo2: simple signal handling demo
exit_on_signal_SIGINT () {
    echo "Script interrupted." 2>&1
    exit 0
}
exit_on_signal_SIGTERM () {
    echo "Script terminated." 2>&1
    exit 0
}
trap exit_on_signal_SIGINT SIGINT
trap exit_on_signal_SIGTERM SIGTERM
for i in {1..5}; do
    echo "Iteration $i of 5"
    sleep 5
done
```

Этот скрипт содержит две `trap` команды, по одной для каждого сигнала. Каждый `trap`, в свою очередь, определяет функцию оболочки, которая будет выполнена при получении конкретного сигнала. Обратите внимание на наличие `exit` команды в каждой из функций обработки сигналов. Без `exit` скрипт продолжил бы выполнение сразу после завершения функции.

Если пользователь нажмет `Ctrl-C` во время выполнения этого скрипта, результат будет таким:

```
[me@linuxbox ~]$ trap-demo2
Iteration 1 of 5
Iteration 2 of 5
^CScript interrupted.
```

Помимо сигналов, доступных всем программам, встроенная команда `trap` также поддерживает несколько внутренних и специфичных для `bash` сигналов. Особый интерес представляют `EXIT` и `ERR`. Как и ожидалось, ловушка `EXIT` активируется при завершении скрипта. Ловушка `ERR` срабатывает всякий раз, когда команда (за некоторыми исключениями) завершается с ненулевым кодом выхода. Это работает как более полезная версия опции `set -e`, которую мы рассматривали в главе 30. Далее следует короткий демонстрационный скрипт для ловушек `EXIT` и `ERR`. Обратите внимание на намеренную ошибку в строке 5.

```
1 #!/bin/bash
2 # trap-demo3 - demonstrate ERR and EXIT signal handling
3 trap "echo \"There is an error.\"\" ERR
4 trap "echo \"The program has ended.\"\" EXIT
5 echox "Running..."
6 read -r -p "Say something... " something
7 echo "$something"
```

Этот скрипт определяет ловушки, а затем пытается вывести строку текста, но команда `echo` написана с ошибкой. Скрипт переходит к следующей строке, где ожидает ввода пользователя. В конце он выводит всё, что ввел пользователь.

При запуске этого скрипта мы получаем следующее:

```
[me@linuxbox ~]$ trap-demo3
./trap-demo3: line 8: echox: command not found
There is an error.
Say something...
The program has ended.
```

Сначала мы получаем сообщение об ошибке от оболочки из-за опечатки в команде. Затем следует сообщение от `ERR trap`, указывающее на его активацию. После этого появляется `read prompt`, и если нажать `ENTER`, выведется пустая строка, а сообщение `EXIT trap` объявит о завершении программы.

Временные файлы

Обработчики сигналов включают в скрипты, чтобы удалять временные файлы, создаваемые для промежуточных результатов выполнения. Именование таких файлов — своего рода искусство. Традиционно программы в Unix-подобных системах создают временные файлы в `/tmp` directory, общем каталоге для таких целей. Однако из-за общего доступа это создает риски безопасности, особенно для программ, работающих с привилегиями суперпользователя. Помимо установки правильных прав доступа, важно давать временным файлам непредсказуемые имена. Это предотвращает атаку типа «temp race». Один из способов создать непредсказуемое (но информативное) имя — сделать следующее:

```
tempfile=/tmp/${basename $0}.$$.$RANDOM
```

Это создаст имя файла, состоящее из имени программы, её идентификатора процесса (PID) и случайного целого числа. Однако учтите, что переменная `$RANDOM` 1 32767, shell возвращает значения только в диапазоне -, что невелико для компьютера, поэтому одного экземпляра переменной недостаточно для защиты от целевой атаки.

Лучше использовать `mktemp program` (не путать с функцией `mktemp standard library function`) для создания и именования временного файла. Программа `mktemp` принимает в качестве аргумента шаблон, используемый для построения имени файла. Шаблон должен содержать серию заглавных символов «X», которые заменяются соответствующим

количеством случайных букв и цифр. Чем длиннее серия «X», тем длиннее будет последовательность случайных символов. Вот пример:

```
tempfile=$(mktemp /tmp/foobar.$$XXXXXXXXXX)
```

Это создает временный файл и присваивает его имя переменной `tempfile`. Символы «X» в шаблоне заменяются случайными буквами и цифрами, поэтому итоговое имя файла (которое в данном примере также включает развернутое значение специального параметра `$$` для получения PID) может выглядеть так:

```
/tmp/foobar.6593.U0ZuvM6654
```

Для скриптов, запускаемых обычными пользователями, разумно избегать использования `/tmp` directory и создавать каталог для временных файлов внутри домашнего каталога пользователя, используя подобную строку кода:

```
[[ -d $HOME/tmp ]] || mkdir $HOME/tmp
```

Асинхронное выполнение

Иногда требуется выполнять несколько задач одновременно. Мы уже видели, что все современные операционные системы являются как минимум многозадачными, а зачастую и многопользовательскими. Скрипты также можно настроить на многозадачную работу.

Обычно это подразумевает запуск скрипта, который, в свою очередь, запускает один или несколько дочерних скриптов для выполнения дополнительной задачи, пока родительский скрипт продолжает работу. Однако при таком запуске серии скриптов могут возникнуть проблемы с координацией родительского и дочернего процессов. Что делать, если один из них зависит от другого и должен дожидаться завершения задачи другого скрипта?

`bash` имеет встроенную команду для управления асинхронным выполнением. Команда `wait` заставляет родительский скрипт приостановить работу до завершения указанного процесса (то есть дочернего скрипта).

`wait`

Сначала мы продемонстрируем команду `wait`. Для этого понадобятся два скрипта. Первый — родительский скрипт.

```
#!/bin/bash
# async-parent: Asynchronous execution demo (parent)
echo "Parent: starting..."
echo "Parent: launching child script..."
async-child &
pid=$!
echo "Parent: child (PID= $pid) launched."
echo "Parent: continuing..."
sleep 2
echo "Parent: pausing to wait for child to finish..."
wait "$pid"
echo "Parent: child is finished. Continuing..."
echo "Parent: parent is done. Exiting."
```

Второй — это дочерний скрипт.

```
#!/bin/bash
# async-child: Asynchronous execution demo (child)
echo "Child: child is running..."
sleep 5
echo "Child: child is done. Exiting."
```

В этом примере видно, что дочерний скрипт прост. Основную работу выполняет родительский. В родительском скрипте дочерний скрипт запускается и отправляется в фоновый режим. PID дочернего скрипта записывается путем присвоения переменной `pid` значения параметра оболочки `$!`, который всегда содержит PID последней задачи, отправленной в фоновый режим.

Родительский скрипт продолжает работу, а затем выполняет `wait` команду с PID дочернего процесса. Это заставляет родительский скрипт приостановиться до завершения дочернего скрипта, после чего родительский скрипт завершается.

При выполнении родительский и дочерний скрипты выводят следующее:

```
[me@linuxbox ~]$ async-parent
Parent: starting...
Parent: launching child script...
Parent: child (PID= 6741) launched.
Parent: continuing...
Child: child is running...
Parent: pausing to wait for child to finish...
Child: child is done. Exiting.
Parent: child is finished. Continuing...
Parent: parent is done. Exiting.
```

Именованные каналы

В большинстве Unix-подобных систем можно создать особый тип файла — именованный канал. Именованные каналы используются для связи между двумя процессами и могут применяться так же, как и другие типы файлов. Они не очень популярны, но о них полезно знать.

Существует распространенная архитектура программирования — клиент-серверная, которая может использовать такие методы связи, как именованные каналы, а также другие виды межпроцессного взаимодействия, например, сетевые соединения.

Самый распространенный тип клиент-серверной системы — это, конечно, веб-браузер, взаимодействующий с веб-сервером. Веб-браузер выступает в роли клиента, отправляя запросы серверу, а сервер отвечает браузеру веб-страницами.

Именованные каналы ведут себя как файлы, но на самом деле представляют собой буферы FIFO (первым пришел — первым ушел). Как и в обычных (безымянных) каналах, данные входят с одного конца и выходят с другого. С именованными каналами можно настроить нечто подобное:

```
process1 > named_pipe
```

и это:

```
process2 < named_pipe
```

и работать оно будет так:

```
process1 | process2
```

Настройка именованного канала

Сначала нужно создать именованный канал. Это делается с помощью команды `mkfifo`.

```
[me@linuxbox ~]$ mkfifo pipe1
[me@linuxbox ~]$ ls -l pipe1
prw-r--r-- 1 me me 0 2009-07-17 06:41 pipe1
```

Здесь мы используем `mkfifo`, чтобы создать именованный канал под названием `pipe1`. С помощью `ls` мы изучаем файл и видим, что первый символ в поле атрибутов — «р», что указывает на именованный канал.

Использование именованных каналов

Чтобы продемонстрировать работу именованного канала, нам понадобятся два окна эмулятора терминала (или, как вариант, две виртуальные консоли). В первом терминале мы вводим простую команду и перенаправляем её вывод в именованный канал.

```
[me@linuxbox ~]$ ls -l > pipe1
```

После нажатия ENTER команда, по-видимому, зависнет. Это происходит потому, что на другом конце конвейера пока никому

принимать данные. В таком случае говорят, что конвейер заблокирован. Состояние изменится, как только мы подключим к другому концу процесс, который начнет считывать данные из конвейера. Во втором окне эмулятора терминала введите следующую команду:

```
[me@linuxbox ~]$ cat < pipe1
```

Список содержимого каталога, сформированный в первом окне эмулятора терминала, появится во втором как результат выполнения команды `cat`. Команда `ls` в первом окне успешно завершится, как только блокировка будет снята.

Итоги

Что ж, наше путешествие завершено. Теперь остается только практика, практика и еще раз практика. Хотя мы прошли большой путь, впереди еще столько открытий. Тысячи программ командной строки ждут, когда их изучат и оценят. Начните исследовать `/usr/bin` и убедитесь в этом сами!

Тем, кто хочет узнать больше, рекомендую мою следующую книгу «Adventures with the Linux Command Line», которую можно бесплатно скачать на LinuxCommand.org.

Что почитать

- В разделе «Compound Commands» страницы `man bash` приведено полное описание обозначений групп команд и подболочек.
- В разделе `EXPANSION` страницы `man bash` есть подраздел, посвященный подстановке процессов.
- В руководстве `Advanced Bash-Scripting Guide` также рассматривается подстановка процессов:
<https://tldp.org/LDP/abs/html/process-sub.html>
- В `Linux Journal` есть две хорошие статьи об именованных каналах. Первая, от сентября 1997 года:
<https://www.linuxjournal.com/article/2156>
- и вторая, от марта 2009:
<https://www.linuxjournal.com/content/using-named-pipes-fifos-bash>

Колофон

Эта книга была изначально написана в OpenOffice.org Writer с использованием шрифтов Liberation Serif и Sans на Dell Inspiron 530N с предустановленной Ubuntu 8.04. PDF-версия текста была создана непосредственно в OpenOffice.org Writer. Второе интернет-издание было подготовлено на том же компьютере с помощью LibreOffice Writer на Ubuntu 12.04. Третье и последующие интернет-издания создавались в LibreOffice Writer на компьютере System76 Ratel Pro под управлением Ubuntu LTS (версии 14.04–24.04). Напечатано из 100% переработанных электронов.